Process Mining Demonstrator

Web Edition — User Manual

The complete guide to the application, the administration interface and the data-source integration console.

Offline edition

Generated 20 October 2026

For demonstration and educational use

About this manual

This manual documents the **Process Mining Demonstrator — Web Edition** in full: the analysis application, the administration interface and the integration console for loading data. It is written to be read offline and used as a reference, so every screen, control, option and error message is described where the behaviour is defined by the product.

> **Scope** This application is intended for demonstration and educational purposes. It is not a production-ready process mining solution, and it should not be used for business-critical decisions without independent verification.

How this manual is organised

The manual is divided into seven parts. *Part I* introduces process mining and gets you to your first process map. *Part II* covers the analysis application in depth. *Part III* is for administrators. *Part IV* covers loading your own data through the integration console. *Part V* is reference material: the data model, the extractor API, troubleshooting and a glossary. *Part VI* covers Actions, the business-readable queries you attach to process-map nodes. *Part VII* covers the API Server - Event Receiver, the push API for logging events from an external program.

Conventions

On-screen labels
: Names of buttons, fields and menu items appear exactly as they do in the interface, for example **Run extraction** or **Delta upload**.

Procedures
: Numbered lists are step-by-step instructions to be followed in order.

Commands and code
: Shell commands, configuration values and API bodies appear in a `monospaced` typeface.

> **Note** Additional context or a detail that is easy to miss.

> **Tip** A suggestion that makes a task easier or faster.

> **Warning** Something that can lose data, expose information, or is hard to undo. Read these before acting.

> Requires: **Administrator** — a capability callout naming the role needed for the task described.

Screenshots

Screenshots were taken from a live instance loaded with the bundled demonstration datasets (“Online Bookstore” and “Online Credit Application”). Your own figures will differ in data but not in layout.

Contents

PART I · GETTING STARTED

1 Introduction 7

2 Architecture and the Four Surfaces 20

3 Installation and Deployment 30

4 Getting Started: Your First Session 47

PART II · USING THE APPLICATION

5 Signing In, Sessions and Security 67

6 The Application: Orientation 87

7 Connections and Projects 101

8 Filters, Metrics, Sampling and Configuration 116

9 The Process Map 133

10 Analysis: Statistics, Conformance, Happy Path and Simulation 151

11 AI Documentation and Notes 166

PART III · ADMINISTRATION

12 Administration: Overview, App Control and TLS 180

13 Administration: Users and Permissions 195

14 Administration: Database Connections and Directory 208

15 Administration: Logging, Backup and Customisation 225

PART IV · THE INTEGRATION CONSOLE

16 The Integration Console 239

17 Source Types: Teaching the System to Read a Log 253

18 Compound Steps 270

19 Sources and Running an Import 281

20 The Watchdog: Automatic Imports 294

PART V · REFERENCE

21 Reference: The Exasol Data Model 307

22 Reference: Writing an Extractor 322

23 Troubleshooting and FAQ 341

24 Glossary 365

PART VI · ACTIONS

25 Actions 381

PART VII · THE API SERVER - EVENT RECEIVER

26 The API Server - Event Receiver 386

PART VIII · THE MCP SERVER

27 The MCP Server 396

PART I

Getting Started

1 Introduction

> This chapter explains what process mining is, what the Process Mining Demonstrator — Web Edition does and, just as importantly, what it deliberately does not do. It introduces the vocabulary used throughout the manual, describes the four audiences the product serves — plain user, power user, developer and administrator — and sets out reading paths so that you can go straight to the chapters that concern you. Read it before any other chapter; every later chapter assumes the terms defined here.

1.1 What process mining is

Business processes leave traces. Every order, claim, application or ticket that passes through a system writes rows into a database: who or what the case was, which activity happened, and when. Process mining takes that raw record — the **event log** — and reconstructs the process as it actually ran, rather than as it was drawn on a whiteboard.

The starting point is always the same three pieces of information per row: a **case identifier** that says which end-to-end instance the row belongs to, an **activity name** that says what happened, and a **timestamp** that says when. In this application those three columns are `EVENT_ID`, `STEP` and `EVENT_TIME` in the `JOURNEYS` table. Everything else — colours, scores, case attributes, groupings — is enrichment layered on top.

From that log, four kinds of insight can be derived, and this product provides all four:

Discovery

Sorting every case's rows by time and joining each consecutive pair produces a **transition**. Aggregating all transitions across all cases produces a **directly-follows graph**: a map in which each node is an activity and each arrow is an observed hand-over between two activities. This is the A-Chart. Nobody draws it; it is derived entirely from the data, so it shows the rework loops, shortcuts and dead ends that a designed model omits.

Variants

Two cases that visit exactly the same activities in exactly the same order follow the same **variant** (also called a route or path pattern). Real processes typically have a small number of very common variants and a long tail of rare ones. The Statistics view lists the distinct variants with their frequencies, which is how you separate "the way it usually goes" from "the exceptions".

Bottlenecks

Because every transition carries the elapsed time between its two steps, the map can be weighted by duration instead of frequency. Switching the transition metric to **Avg Time**, **Max Time** or **Std Dev** makes slow and unpredictable hand-overs visible as thick, strongly coloured arrows, and the Statistics view adds a duration histogram, step traffic and a transition heat map.

Conformance

Discovery tells you what happens; conformance tells you whether that is what should happen. You define the intent — a **Happy Path** (the ideal step sequence, including legitimate alternative branches) or per-edge **norms** (target values for a metric) — and the application scores the real data against it, edge by edge, and reports the gaps.

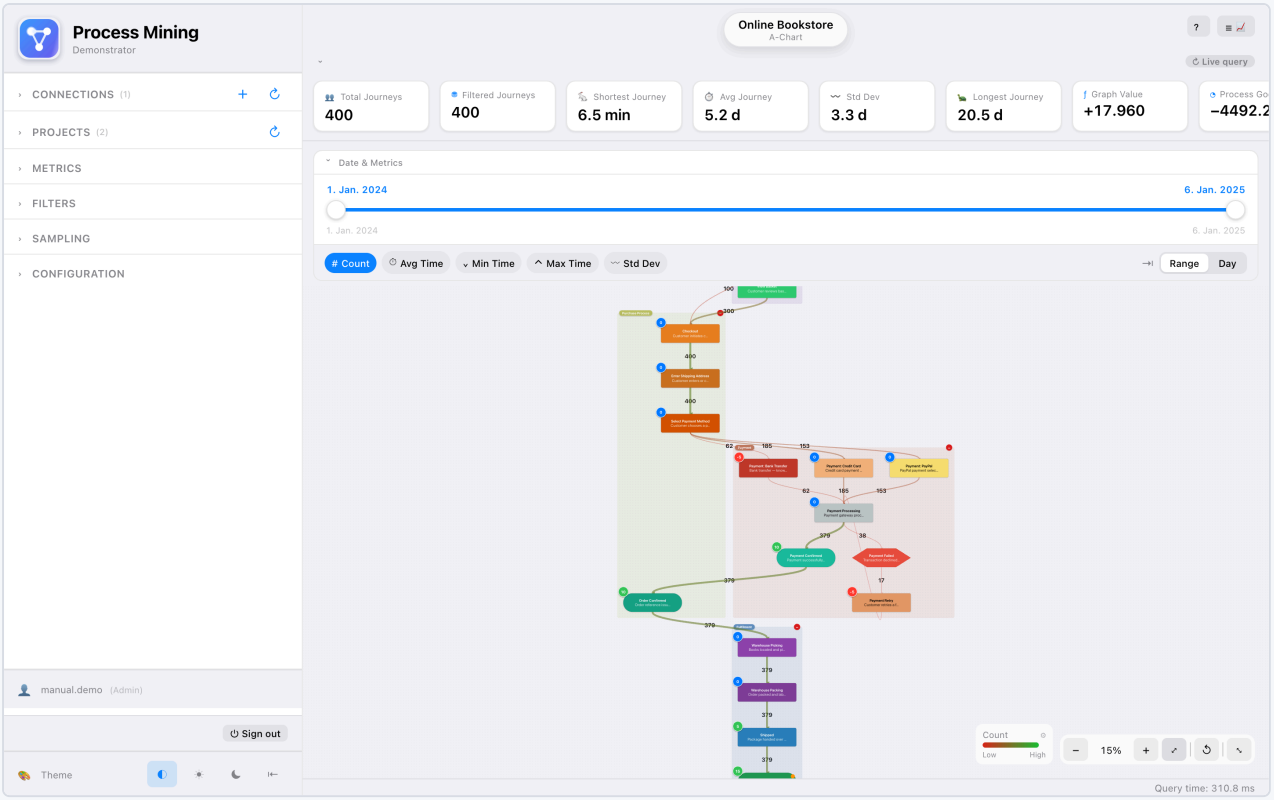


Figure 1.1 — The A-Chart: a directly-follows graph discovered from the `JOURNEYS` table. Arrow thickness and colour encode the selected transition metric.

Process mining matters because it replaces opinion with evidence. Where an interview produces "approvals usually take a day or two", an event log produces a distribution, a worst case and the exact share of cases that looped back through a rework step. That is the value this product is built to demonstrate.

1.2 What this product is

The **Process Mining Demonstrator — Web Edition** is a multi-user web application that connects to an **Exasol** database, reads an event log from it, and visualises how real cases flow through the recorded process. It is the web port of a macOS application of the same name; backup files remain interchangeable between the two.

Concretely, it offers:

- Ten chart views, selected from the ≡ menu in the top-right of the main area: **A-Chart**, **B-Chart**, **A/B Comparison**, **Individual Journey**, **AI supported Documentation**, **Statistics**, **Conformance Check**, **Happy Path**, **Notes** and **Simulation**.
- Journey-level filtering — date window, included and excluded steps, step count, journey time, journey score and three case-level meta attributes — with named, reusable filter presets stored per project.
- Two independent process maps side by side, with a **Process Similarity** score between them, so two segments or two time periods can be compared directly.
- A composite **Process Goodness** quality score, per-journey inspection with animated replay, annotation of nodes and edges with threaded notes, journey sampling, Markov-model simulation, and a Markdown report generated by an OpenAI-compatible language model.

- A dedicated administration interface for users, roles, TLS, directory sign-in, logging, backups and licensing, plus a separate integration console for defining data sources and importing event data from files.

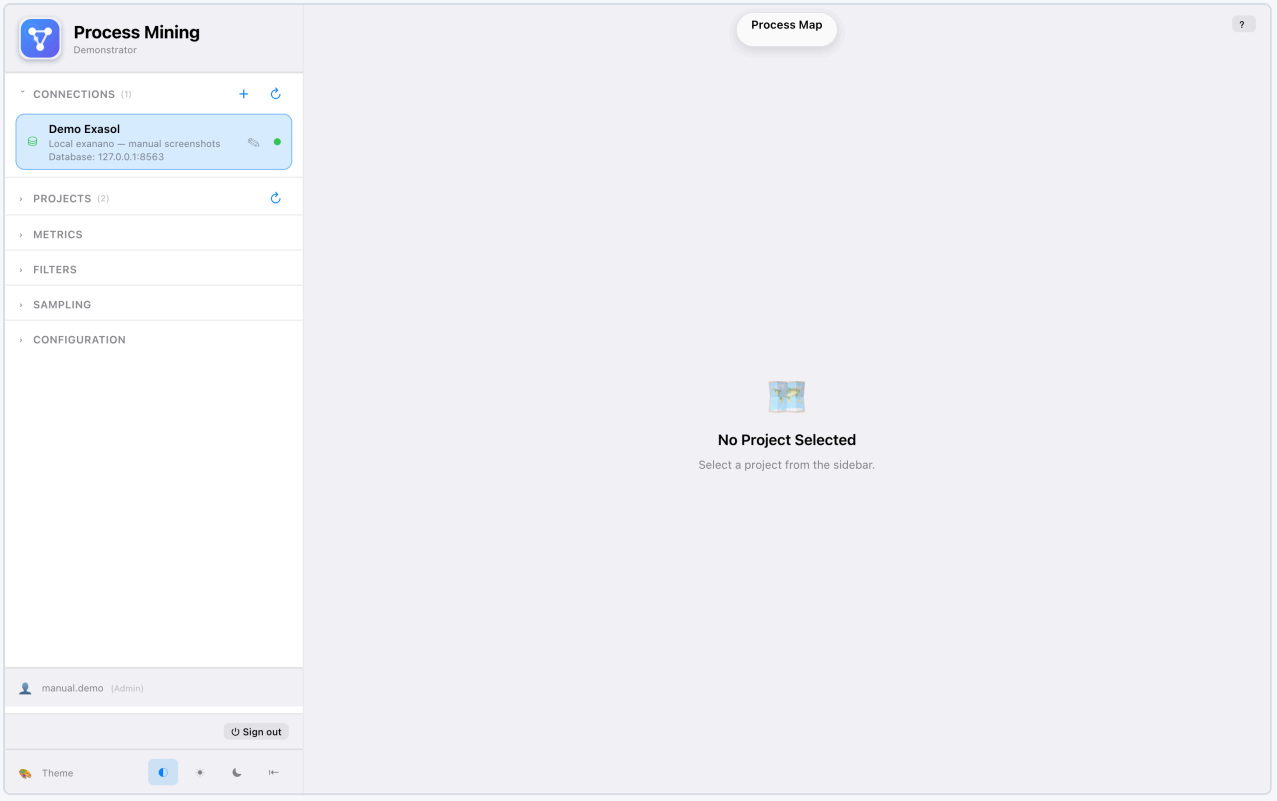


Figure 1.2 — The main application window: sidebar with Connections, Projects, Filters and Configuration on the left; chart area with the view menu and KPI strip on the right.

1.2.1 Scope: a demonstrator, not a production tool

> **Warning** This application is intended for **demonstration and educational purposes only**. It is **not a production-ready process mining solution**. Do not use its output as the sole basis for operational or commercial decisions.

That statement is not a formality; it follows from deliberate design choices which you will meet throughout this manual, and which you should know about before you rely on a number:

Area	Deliberate limitation
Database support	Exasol only. There is no adapter for any other database.
Heavy queries	Statistics and Process Goodness run under a 30-second wall-clock timeout; when it expires, the affected tile shows **—** rather than a partial result.
Process Similarity	Alignment cost is an edge-coverage approximation, not a formal Petri-net alignment, and the variant set is capped at 500 per side (1 000 combined). Past the cap the badge is hidden rather than showing a biased score.
Process Goodness	A relative indicator, not an absolute benchmark. Its constants (alpha 0.5, time penalty 0.01, gamma 0.5) are fixed and not adjustable.
Simulation	A first-order Markov model. History-dependent routing, case attributes, resource contention and queueing are not modelled.
Data import	Events are appended and **never de-duplicated**. Importing the same file twice stores its events twice; to genuinely reload a project you delete the project and import again.
AI documentation	Language models can produce incorrect, incomplete or misleading results. Every finding must be verified independently.

Used as intended — to show what process mining is, to teach the concepts, and to explore a dataset on an Exasol instance — the product is complete and self-contained, including four generated demo datasets so you can produce a meaningful process map without having any event log of your own.

1.2.2 Four processes, three surfaces

The product is not a single server. One launcher starts four Python processes, three of which present a user interface. Knowing which is which will save you time in every later chapter.

Process	Default HTTP	Default HTTPS	Purpose	Who signs in
GUI server (main application)	8080	8443	Serves the single-page application, proxies API calls to the backend, enforces sign-in	All users
Admin interface	8090	8453	Users, roles, TLS certificates, database connections, directory, logging, backups, licence, customisation, restart	Administrators only
Integration console	8100	8463	Data-source definition and event-data import	Restricted (see 1.4)
Compute backend	—	8000	Exasol access, analytics, simulation, sampling, language-model proxy, settings store, licence watchdog, import layer	No direct sign-in; not published

> **Note** The browser only ever talks to the GUI server. The compute backend is reached through the GUI proxy over an always-on TLS connection using an internal self-signed certificate, even on loopback, and it refuses API calls that do not carry the proxy's authentication secret. The integration console's ports are derived by convention as the admin port plus ten.

> **Note** All port numbers in this manual are the defaults for a direct installation. Under the shipped Docker Compose file, host ports are offset by **+2000**: the main application is on **10080**, the admin interface on **10090** and the integration console on **10100**. Every environment variable that changes a port is listed in the installation chapter.

1.2.3 Two different licences

The manual refers to a "licence" in two unrelated senses; do not confuse them.

Software licence

The source code is distributed under the MIT licence. This governs your rights to the code itself.

Runtime licence

The compute backend requires a signed `license.json`, uploaded by an administrator under **App Control → License**. Without one, the application runs in **Demo Mode** for a one-time grace period (30 minutes by default), after which the backend stops itself until a licence is applied. The admin interface keeps running even then, so a licence can always be uploaded. The grace period is granted once per installation and restarting does not renew it.

1.3 Vocabulary

These terms are used with precise meanings throughout the manual and in the application's own screens.

Journey (case)

One end-to-end instance of a process, identified by `EVENT_ID`. All rows sharing an `EVENT_ID` form one journey. "Case" and "journey" are interchangeable.

Step (event)

A single activity within a journey — one row in the `JOURNEYS` table. On the process map, one node per distinct step name.

Transition

A chronologically consecutive pair of steps within the same journey. On the map, one arrow.

Project

A named body of event data inside one schema, identified by `PROJECT_ID`. You select a project after connecting; all filters, layouts, presets, norms and happy paths are stored per project.

Connection

A stored bundle of one Exasol server (host, port, credentials, schema, TLS options) and one optional OpenAI-compatible language-model server. Connections are created by administrators or power users and assigned to individual users. You never type a database password into the main application.

Meta attributes

Up to three free-text, case-level columns (`META_1` – `META_3`) carrying attributes such as department, region or customer segment. Their business names come from the `METAS` table.

Variant (route)

A distinct sequence of steps. All journeys following the same sequence belong to the same variant.

Metric

The value that drives an arrow's thickness, opacity, colour and label: **Count** (the default), **Avg Time**, **Median Time**, **Min Time**, **Max Time** or **Std Dev**. Each chart view remembers its own metric selection.

Sample set

A named representative subset of a project's journeys, written to a `SAMPLE_SET` column on `JOURNEYS` . Sample sets live in the database and are therefore **shared by everyone** using that connection.

Norm

A target value attached to one transition for one metric, used by the Conformance Check view. Stored per project and per metric.

Happy Path

A user-defined ideal step sequence, optionally containing named splits with alternative branches, against which real journeys are scored.

> **Tip** The same explanations are available inside the application at any time. Press ⌘/ (or **Ctrl+/**) to open or close the Help panel; it floats above the application, can be dragged by its title bar and resized from its bottom-right corner.

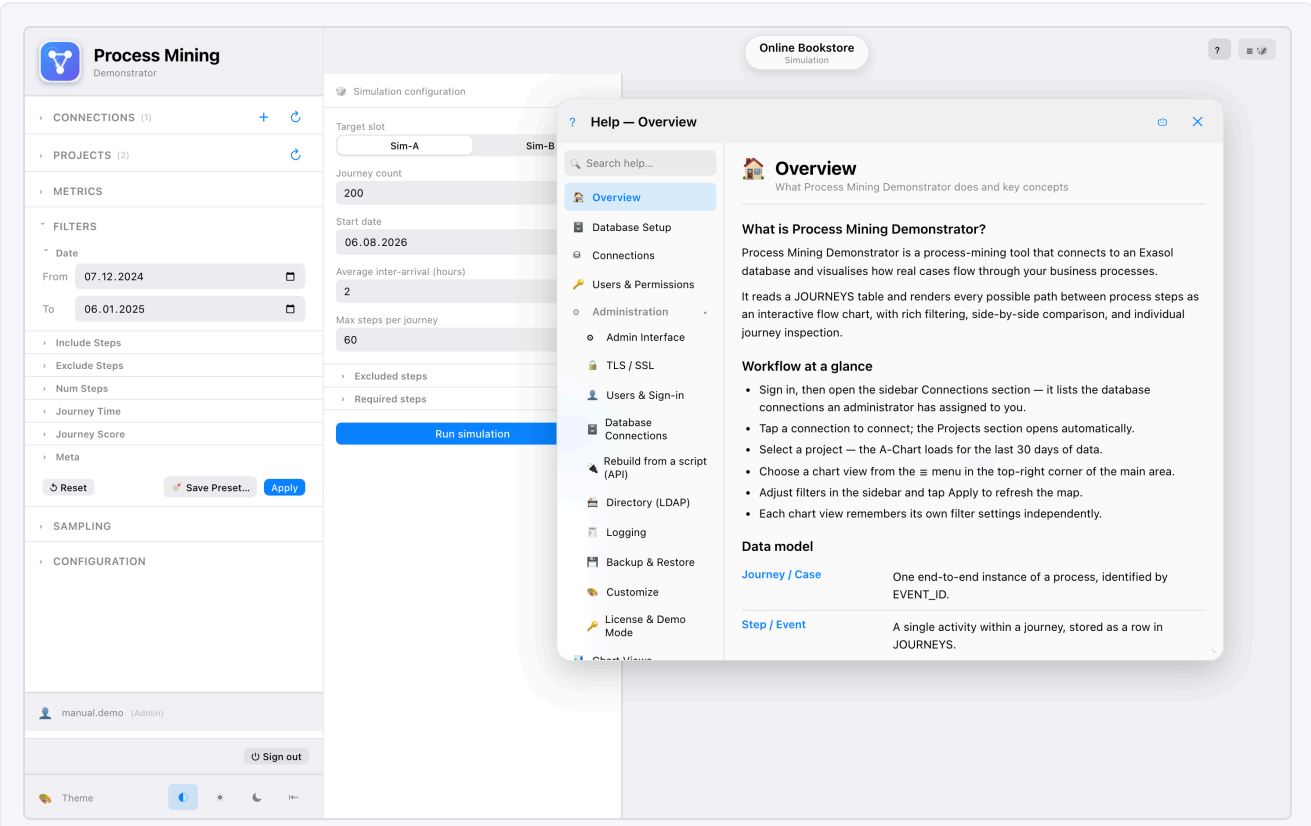


Figure 1.3 — The floating Help panel, opened with ⌘/ or Ctrl+/. Its table of contents hides administration and integration topics from users who cannot reach those surfaces.

1.4 Who this manual is for

Four audiences use this product, and every chapter is written with one or more of them in mind. Where a task requires a particular permission, the chapter states it in a callout like this one:

> Requires: **Administrator**

1.4.1 The four audiences

Plain user (Regular user)

The default level for every new account. A plain user explores the process: opens an assigned connection, selects a project, reads the maps and charts, applies filters and saved presets, reads and writes notes, inspects individual journeys, generates AI documentation, and adjusts personal settings such as KPI tiles, node layout, font sizes and theme. A plain user cannot create database connections and does not see the advanced-analysis views.

Power user

A plain user with additional in-application privileges, granted with **Make power** in the admin Users tab. Power users create, edit, delete and assign **their own** database connections from inside the main application, provision schemas, generate demo datasets, create and delete journey samples, and use the three advanced-analysis views — **Conformance Check**, **Happy Path** and **Simulation**. They manage only the connections they created.

Developer

An additional grant (the **Developer** checkbox in the admin Users tab), independent of the level above it. It admits the account to the **Integration console**, where data sources and their parsing rules are defined and event data is imported into a connection's schema. A Regular user who holds only the Developer grant may enter the console but gains no other elevated powers in the main application.

Administrator

Full control. Everything a power user can do, over **every** connection rather than only their own, plus exclusive access to the admin interface: all users and roles, TLS certificates and mode, directory (LDAP) sign-in, licensing, logging, backups, login-page customisation, and restarting the servers.

The three permission levels build on each other: Power includes Regular, and Administrator includes Power. The Developer grant is orthogonal and can be combined with any level.

1.4.2 Capability overview

This table is the quick answer to "may I do this?". Later chapters repeat the relevant line as a role callout at the point of use.

Capability	Regular	Power	Developer grant	Administrator
View process maps, charts and KPIs	✓	✓	✓	✓
Filters, saved presets and the date window	✓	✓	✓	✓
Read and write notes and comments	✓	✓	✓	✓
Personal settings, layout, manual backup of preferences	✓	✓	✓	✓
Individual Journey, Statistics, AI supported Documentation	✓	✓	✓	✓
Advanced views: Conformance Check, Happy Path, Simulation	—	✓	—	✓
Journey sampling (create and delete sample sets)	—	✓	—	✓
Create, manage and assign own database connections	—	✓	✓ (in the console)	✓ (all connections)
Provision a schema and generate demo data	—	✓	—	✓
Integration console: source types, sources, imports	—	—	✓	✓
Admin interface: users, TLS, LDAP, licensing, logging, backups	—	—	—	✓

Note The in-application help text and the deployment notes differ on whether a power user without the Developer grant may sign in to the integration console: the help states that only developers and administrators may enter, while the launcher documentation lists power users as well. Anyone not permitted is refused at the console's sign-in page with a clear message, so the behaviour of your build is unambiguous the first time you try. The integration chapter treats the Developer grant as the requirement.

Warning Roles require a signed-in identity. If an administrator turns off **"Require sign-in for the main application"** in the admin Users tab, the application runs with no user identity at all — every Power-only and Admin-only capability becomes unavailable, the Sampling section and the three advanced views disappear from the interface, and their APIs return **403**. Per-user settings and filter presets also collapse into one shared profile. There is no way to have open, sign-in-free access and the power features at the same time.

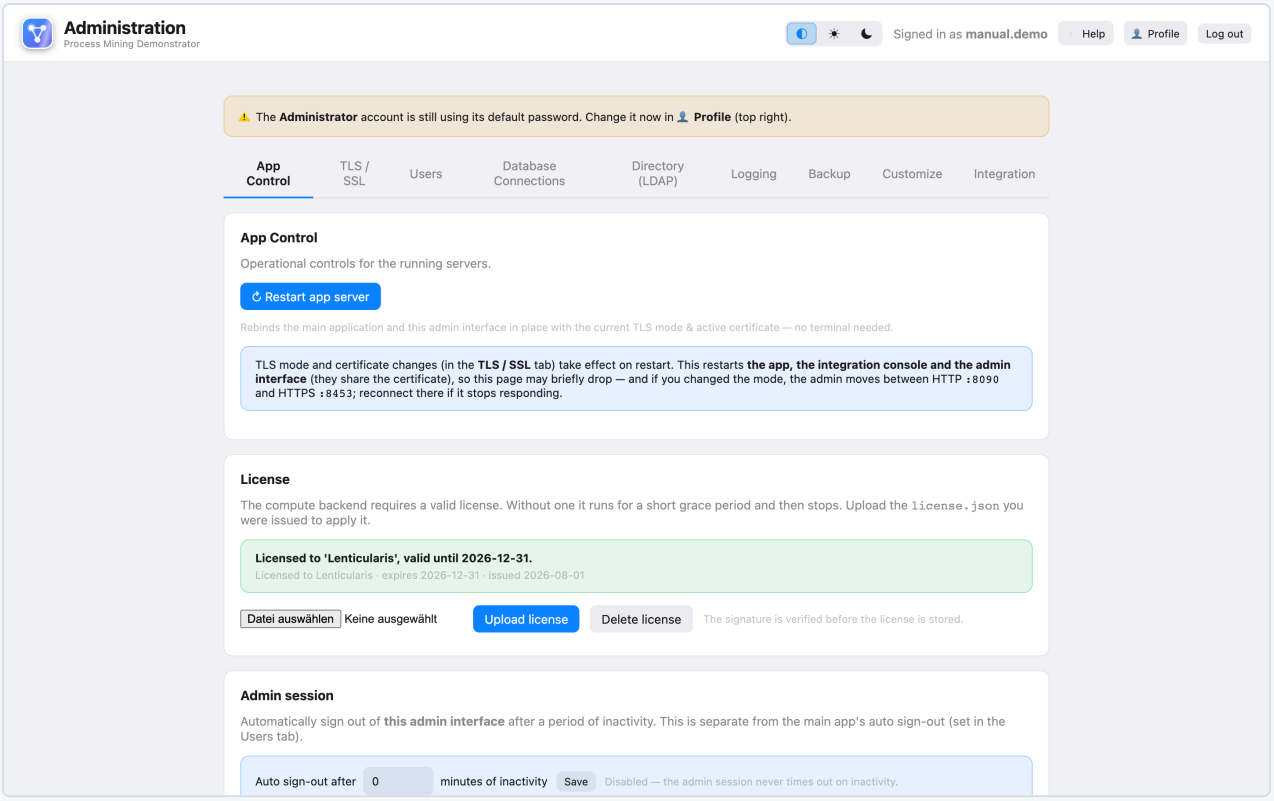


Figure 1.4 — The admin interface, a separate surface on its own port with its own sign-in, admitting administrators only.

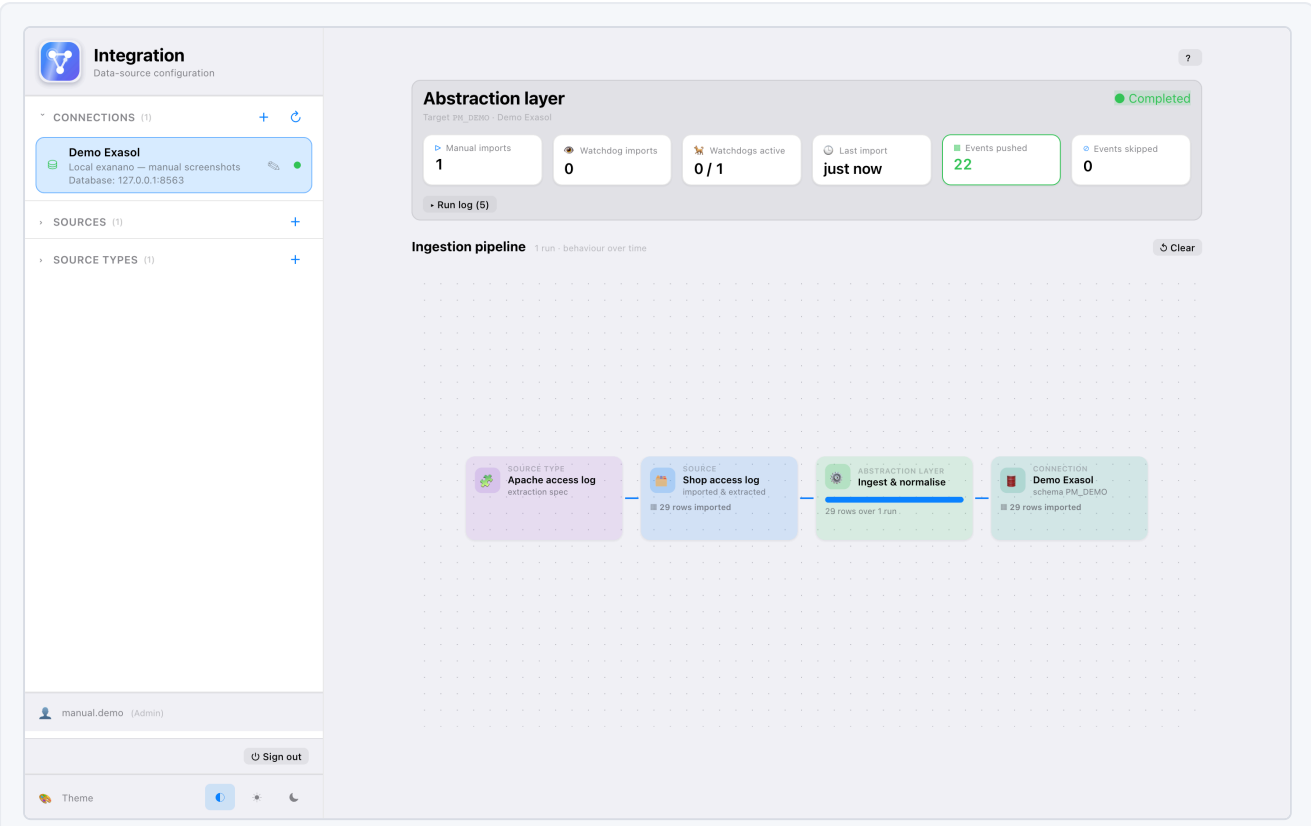


Figure 1.5 — The integration console, where source types and sources are defined and event data is imported into a connection's schema.

1.5 How this manual is organised

The chapters that follow move from installing the product, through daily analysis work, to administration and data integration. They fall into five themed groups:

Group	What it covers	Primary audience
Installation and operations	System requirements, local and Docker installation, the four processes and their ports, the complete `PMW_*` environment-variable reference, the `data/` directory, TLS, licensing, upgrade notes	Administrator
Getting started	Signing in (password, passkey, two-factor), choosing a connection, selecting a project, the shape of the main window, the database schema the application expects	Everyone
Analysis	The chart views, filters and presets, the process map and its context actions, the KPI panel, Process Goodness, Process Similarity, statistics, notes, AI documentation, configuration	Plain user, power user
Advanced analysis	Conformance Check, Happy Path, Journey Sampling, Simulation	Power user, administrator
Administration and integration	The admin interface tab by tab — users and roles, database connections, directory sign-in, logging, backups, customisation, licence — and the integration console: source types, file sources, compound steps, imports, the watchdog and pipeline monitoring	Administrator, developer

A troubleshooting chapter collects the symptoms users report most often, together with the permission or configuration that usually causes them.

1.6 Reading paths

You do not need to read this manual from cover to cover. Pick the path that matches your role.

1.6.1 Business analyst or plain user

1. Read this introduction, in particular the vocabulary in section 1.3 — the rest of the manual assumes the terms *journey*, *step*, *transition*, *variant* and *metric*.
2. Read the getting-started chapter: how to sign in, how to open a connection assigned to you, and how to select a project.
3. Read the chart-views chapter, then the filters and filter-presets chapters. These three cover the everyday loop: load a project, narrow it, press **Apply**, read the map.
4. Read the process-map chapter for navigation, node context actions and step groups, and the KPI-panel chapter for the tiles above the map.
5. Read Process Goodness and Process Similarity when you start comparing segments or periods; both chapters include worked numeric examples.
6. Read the notes chapter before you begin recording findings, and the configuration chapter when you want to change the default date window, KPI tiles, fonts or step colours.
7. Skip the installation, admin-interface and integration chapters entirely — nothing in them is required for analysis work.

> **Tip** If you have no data of your own and no power user to help, ask an administrator to generate one of the four demo datasets — Online Bookstore, Online Credit Application, Flight Booking & Management, or Airport Passenger Flow. Every example in the analysis chapters works against them.

1.6.2 Power user

1. Follow the business-analyst path first; everything there applies to you unchanged.
2. Read the connections chapter's power-user sections: creating your own connections with the **+** button, editing them with ✎, assigning them to other users, provisioning a schema, and generating demo content.
3. Read the database-setup chapter so you know which database privileges each feature needs — SELECT for reading, CREATE TABLE for notes, ALTER TABLE and INSERT on `JOURNEYS` for sampling, UPDATE on `STEPS` for the step editor.
4. Read Journey Sampling before creating a sample: samples are written into the database and therefore change what every other user of that connection sees.
5. Read Conformance Check and Happy Path together — both express intent and are scored against the same filtered population.
6. Read Simulation last, including its limitations section.

1.6.3 Administrator or operator

1. Read this introduction's section 1.2.2 (the four processes) and 1.2.3 (the two licences).
2. Read the installation and operations chapter end to end, then perform the first-run checklist: start the stack, sign in to the admin interface, change the seeded administrator password.
3. Read the database-connections chapter and define at least one connection, test it, and assign it to users.
4. Read the users chapter: roles, the login gate, failed-sign-in lockout, passkeys and two-factor authentication. Decide these policies before inviting users, because some of them (mandatory two-factor in particular) change what happens at the next sign-in.
5. Read the TLS chapter and move off plain HTTP. Passkeys require a secure context and a real hostname, so this decision constrains later ones.
6. Read the backup chapter and enable scheduled encrypted backups; then read the operations chapter's note on backing up the whole `data/` directory, which is the only way to preserve the encryption key.
7. Read the licensing sections and upload the runtime licence before the demo grace period expires.
8. Keep the logging chapter to hand — it is the first place to look when an import, a rebuild or a connection misbehaves.

1.6.4 Developer or data engineer

1. Read the vocabulary in section 1.3 and the database-setup chapter, so you know the exact shape of the rows you are producing.
2. Read the integration-console chapter: the abstraction layer, source types and sources, and how to reach the console.
3. Read the source-types chapter carefully, including compound steps, helper fields and the ordering rule that the first matching rule wins.
4. Read the import sections on delta upload, the checkpoint, the transaction bracket and the watchdog before you run an import against real data — in particular the rule that events are appended and never de-duplicated.
5. Read the pipeline-monitoring chapter for the live flowchart and the ingestion KPIs, especially **Events skipped**.
6. If you also automate rebuilds of pre-materialized transitions, read the per-connection API-token section in the administration group.

1.7 Conventions used in this manual

On-screen labels

Controls are named exactly as they appear, in bold — for example **Apply**, **Save Preset…**, **Use pre-materialized transitions**. Icon-only controls are given by their glyph and function, for example the **+** button in the Connections header.

Code and identifiers

Table names, column names, file paths, environment variables, commands and API endpoints are set in a monospaced face, for example `JOURNEYS`, `PMW_DATA_DIR`, `data/backups/`.

Role callouts

A "Requires" box before a procedure names the minimum permission needed. If you do not hold it, the control described is either hidden or the request is refused.

Note, Tip and Warning

Note adds context you need for the surrounding text to make sense. **Tip** is optional but saves time. **Warning** marks behaviour that can lose data, lock someone out, or produce misleading results.

Ports and URLs

Given as the direct-installation defaults (8080, 8090, 8100). Under Docker, add 2000 to each host port. Where TLS is enabled, substitute the HTTPS port (8443, 8453, 8463).

Spelling

The manual uses British spelling; the application's own labels are reproduced verbatim and therefore sometimes use American spelling, as in **Colorise edges by weight** and **Customize**.

> **Tip** Everything described in the analysis chapters is also summarised in the in-application help, which hides administration and integration topics from accounts that cannot reach those surfaces. If a help topic you have read about is missing from your table of contents, that is the permission system telling you the feature is not available to your account.

2 Architecture and the Four Surfaces

> Process Mining Demonstrator — Web Edition is not one server but four independent processes: one compute backend and three browser surfaces. This chapter explains what each process does, which ports it listens on, how the browser surfaces pass your identity to the compute backend, where every piece of persistent state is written, and what happens between a click in the browser and a row coming back from Exasol. Read it before you install, before you put the product behind a proxy, and before you plan backups. It is written for administrators and for anyone who needs to reason about the product beyond a single machine.

The four processes at a glance

All four processes are started by the single launcher `run.sh` (or, in Docker, by the same script used as the image's `CMD`). They are separate operating-system processes, they can fail independently, and only one of them ever speaks to the database.

Process	Source directory	HTTP	HTTPS	What it does
Compute backend	`backend/`	—	**8000** (always TLS)	Exasol access, analytics, simulation, sampling, LLM proxy, settings store, licence watchdog, the integration abstraction layer and the file-source watchdog
Main application (GUI server)	`frontend/`	**8080**	**8443**	Serves the React single-page application, proxies `/api/*` to the compute backend, enforces sign-in
Admin interface	`admin/`	**8090**	**8453**	TLS and certificates, users, database connections, LDAP, logging, backups, licence upload, customisation, in-place restart
Integration console	`integration/`	**8100**	**8463**	Data-source configuration and imports; sign-in restricted to developers and administrators

The compute backend (port 8000)

The compute backend is the only process that opens a connection to Exasol, the only one that calls a configured LLM server, and the only one that runs analytics. It listens on **HTTPS only**, on loopback by default (`PMW_BACKEND_HOST` is `127.0.0.1`), and it is never published to the browser. On start it mints an internal self-signed certificate at `data/certs/internal.crt` and `data/certs/internal.key`; that certificate exists independently of the user-facing TLS mode, so the hop from the GUI server to the backend is encrypted even when the product is otherwise served over plain HTTP.

Because the browser never talks to the backend directly, the backend can run on a different host — ideally one close to the database. See Splitting the deployment.

> **Note** The backend also hosts the integration API under `/api/integration`. The integration console has no analytics engine of its own; it proxies to the same backend that the main application uses.

The main application (ports 8080 / 8443)

The GUI server delivers the compiled SPA and acts as the authenticating reverse proxy for `/api/*`. It holds the session cookie, validates it on every request, and only then forwards the call to the compute backend. Any enabled user may sign in here. Sign-in can be switched off entirely for single-user demonstrations with the **Require sign-in for the main application** checkbox in the admin **Users** tab (default: on).

The admin interface (ports 8090 / 8453)

> Requires: **Administrator**

The admin interface is a server-rendered application, not part of the SPA. It manages everything the other surfaces merely consume: users and roles, database connections and their assignments, TLS certificates and mode, the directory (LDAP) configuration, logging, backups, the runtime licence, sign-in page customisation, and the on/off switch for the integration console. It also owns the **↻ Restart app server** button that rebinds all listeners after a TLS change.

Two properties make the admin interface the recovery surface of the whole product: it keeps running when the compute backend has stopped (so a licence can always be uploaded), and it falls back to HTTP if the configured TLS mode has no usable certificate (so a bad certificate can never lock you out).

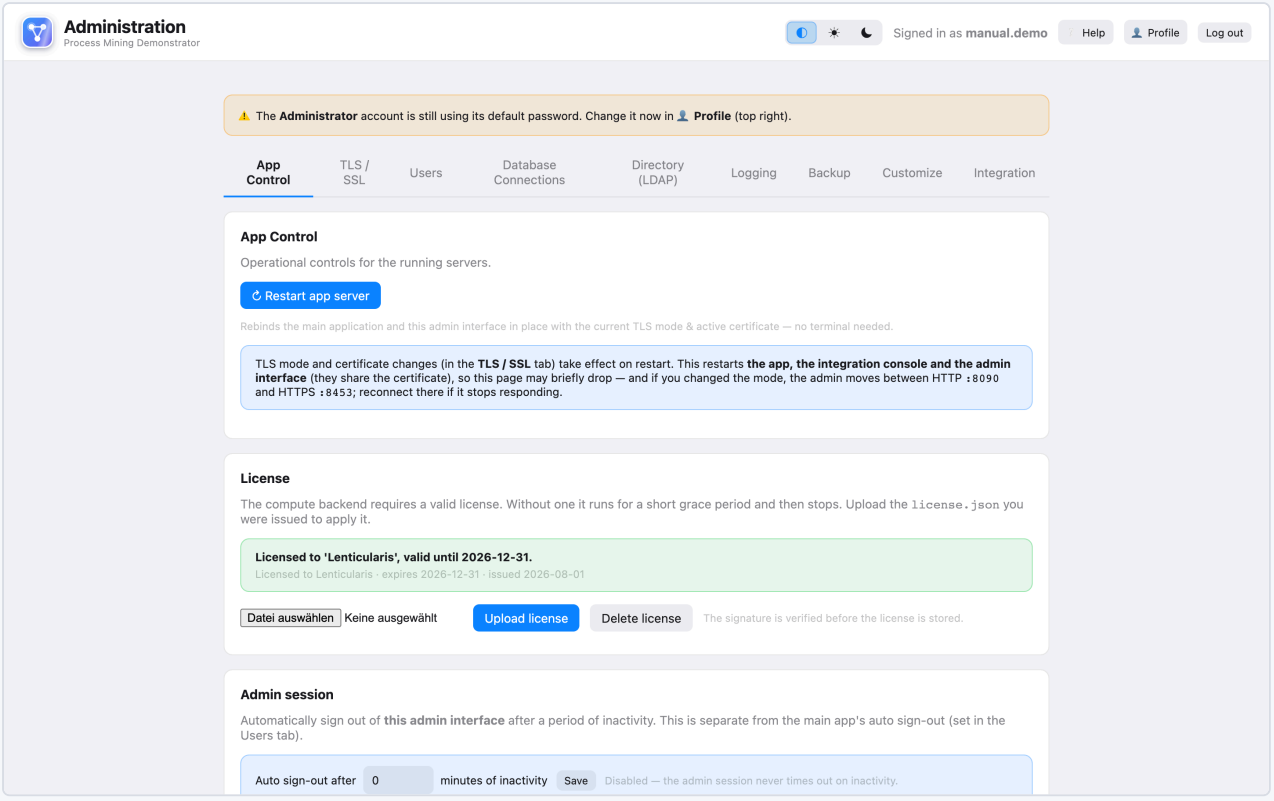


Figure 2.1 — The admin interface on its own port — a separate process, a separate session, and the only surface that can change the deployment's configuration.

The integration console (ports 8100 / 8463)

> Requires: **Developer** or **Administrator**

The integration console configures data sources and source types and runs imports into a destination connection. Its ports are derived by convention as *admin port* + *10* (8090 + 10 and 8453 + 10) unless `PMW_INTEGRATION_PORT` / `PMW_INTEGRATION_HTTPS_PORT` override them, and its bind address defaults to whatever `PMW_ADMIN_HOST` is.

Power users are refused here. A valid password from an account without the Developer or Administrator role is answered with 403 and the message "*You need the Developer role to use the integration console. Ask an administrator to grant it.*" The same gate is repeated inside the compute backend on every `/api/integration` endpoint, so the restriction cannot be side-stepped by calling through the main application's proxy.

An administrator can switch the console off completely (admin **Integration** tab, checkbox **Enable the integration console**). While it is off, browser navigations receive a 503 page titled **Console unavailable** with the text "*The integration console has been turned off by the administrator.*", and API calls receive the same text as JSON.

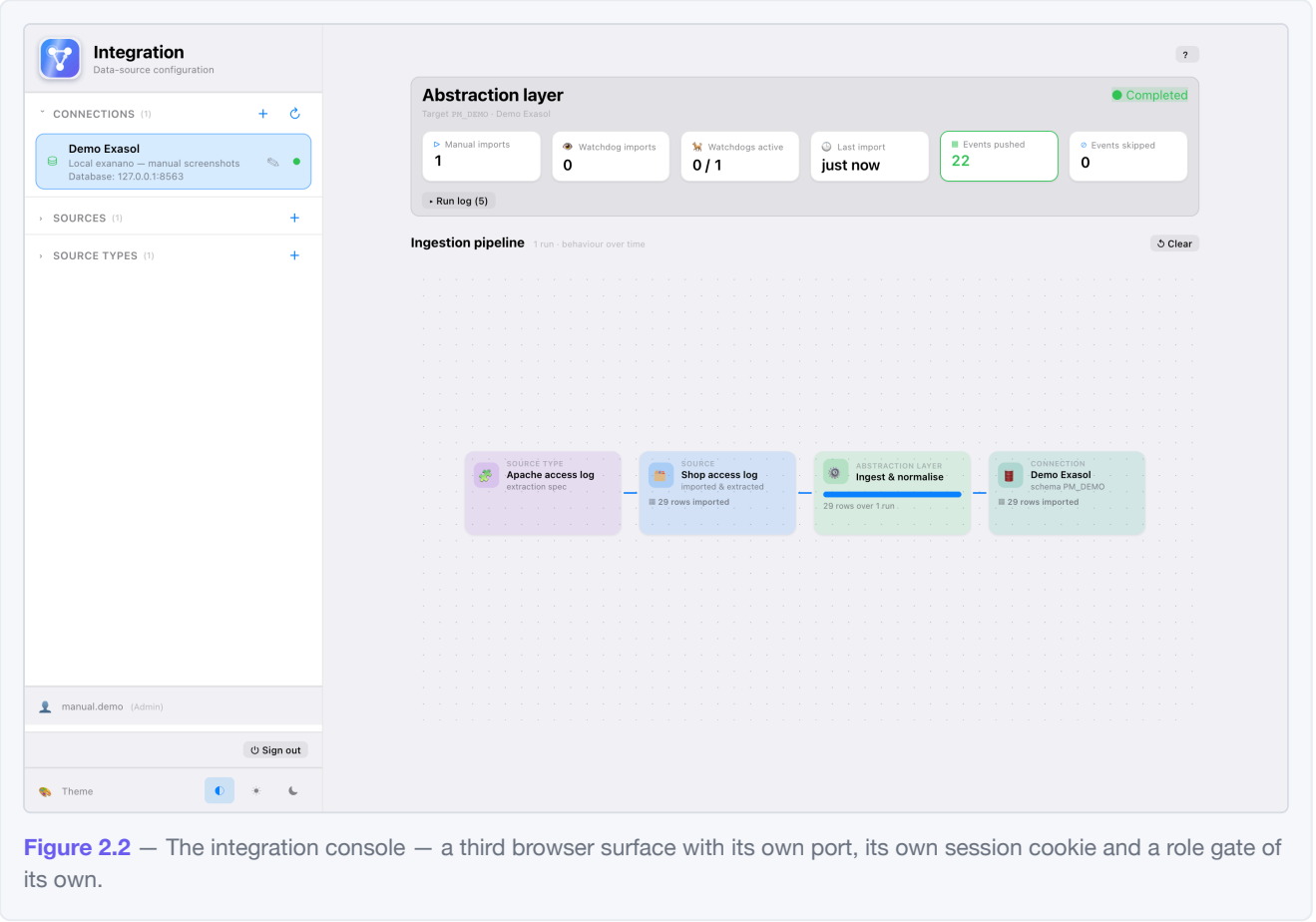


Figure 2.2 — The integration console — a third browser surface with its own port, its own session cookie and a role gate of its own.

Ports, binding and the Docker +2000 offset

Every port and bind address is an environment variable read by all four processes from one shared configuration module.

Variable	Default	Effect
`PMW_BACKEND_HOST`	`127.0.0.1`	Bind address of the compute backend. Keep it on loopback unless the backend runs on its own host.
`PMW_BACKEND_PORT`	`8000`	Compute-backend port (HTTPS, internal certificate).
`PMW_FRONTEND_HOST`	`127.0.0.1` (Docker image: `0.0.0.0`)	Bind address of the GUI server.
`PMW_FRONTEND_PORT` / `PMW_FRONTEND_HTTPS_PORT`	`8080` / `8443`	Main-application HTTP and HTTPS ports.
`PMW_ADMIN_HOST`	`127.0.0.1` (Docker image: `0.0.0.0`)	Bind address of the admin interface.
`PMW_ADMIN_PORT` / `PMW_ADMIN_HTTPS_PORT`	`8090` / `8453`	Admin HTTP and HTTPS ports.
`PMW_INTEGRATION_HOST`	= `PMW_ADMIN_HOST`	Bind address of the integration console.
`PMW_INTEGRATION_PORT` / `PMW_INTEGRATION_HTTPS_PORT`	admin port + 10 → `8100` / `8463`	Integration console HTTP and HTTPS ports.

In Docker all four processes run inside **one image** (`process-mining-web:latest`, container `process-mining-web`), and the published host ports are offset by **+2000** so they do not collide with other local services:

Host port	Container port	Surface
10080 / **10443**	8080 / 8443	Main application (HTTP / HTTPS)
10090 / **10453**	8090 / 8453	Admin interface (HTTP / HTTPS)
10100 / **10463**	8100 / 8463	Integration console (HTTP / HTTPS)

Port 8000 is deliberately **not published**: inside the image `PMW_BACKEND_HOST` is pinned to `127.0.0.1`, and only the six surface ports are exposed. The HTTPS host ports do nothing until TLS is switched to *Optional* or *Required* in the admin interface.

> **Warning** The README's "Running with Docker" section still quotes `http://localhost:8080` and `http://localhost:8090`. That text predates the +2000 offset. With the shipped `docker-compose.yml` the correct addresses are **http://localhost:10080** and **http://localhost:10090**; the compose file is authoritative.

What changes in development mode

`./run.sh --dev` starts only the compute backend, the admin interface and the Vite dev server on `http://localhost:5173`. There is **no GUI server and no integration console** in this mode, and `PMW_REQUIRE_PROXY_AUTH` is set to `0` automatically because Vite proxies straight to the backend with no authenticating proxy in between. Everything is loopback in development; do not carry that setting into a real deployment.

How a request travels

The following walkthrough covers an ordinary data call in the main application — for example, loading a process map. The integration console follows exactly the same path with its own cookie and its own `/api` proxy.

1. The browser requests a page from the GUI server on port 8080 (or 8443 when TLS is on). The SPA is served as static files; nothing about the database is involved yet.
2. The SPA issues an API call, for example `GET /api/graph…`, to the same origin. It never addresses the compute backend directly and does not know its address.
3. The GUI server validates the `pmw_session` cookie: signature, audience, session epoch, absolute lifetime, and the live user record (the account must still exist, still be enabled, and still hold any role the surface requires). If validation fails, the call is answered with **401 "Authentication required."** and never reaches the backend.
4. Having established who the caller is, the proxy injects the trusted header `X-PMW-User` with the signed-in username, adds the proxy-authentication secret, and forwards the request to `PMW_BACKEND_URL` over HTTPS, verifying the backend against the pinned internal certificate.
5. The compute backend checks the proxy-auth secret first. Only then does it read `X-PMW-User`, look the user up, apply the endpoint's own role and ownership rules (for example, "You are not allowed to manage connections." for a plain user), and select that user's private database session.
6. The backend runs the query against Exasol on the user's own `pyexasol` session, serialised through a per-user lock, and returns JSON. The GUI server passes the response back to the browser and remints the session cookie so the sliding idle window stays alive.

Two proxy-level failures are worth recognising on sight: **502 "The compute backend is not reachable. Start it with ./run.sh or check PMW_BACKEND_URL."** means the GUI server is up but the backend is not, and **504 "The compute backend timed out."** means the backend accepted the call but did not answer in time.

The trusted header and `PMW_REQUIRE_PROXY_AUTH`

Identity is carried between the surfaces and the compute backend in one plain header, `X-PMW-User`. That is only safe because the backend refuses every `/api/*` call that does not carry the proxy-authentication secret, which only the surface processes hold. Without that check, any local process able to reach port 8000 could claim to be any user simply by setting the header.

Variable	Default	Meaning
`PMW_REQUIRE_PROXY_AUTH`	`1` (on)	The backend requires the proxy-auth secret on `/api/*`. `0`, `false`, `no` and `off` disable it; `run.sh --dev` sets `0` automatically.
`PMW_BACKEND_URL`	`https://127.0.0.1:8000`	Where the surface proxies find the compute backend.
`PMW_BACKEND_CA`	`{data}/certs/internal.crt`	CA bundle the proxy verifies the backend against. Point it at a real CA if the backend is fronted by one, or set it empty to fall back to the system trust store.

> **Warning** Never disable `PMW_REQUIRE_PROXY_AUTH` in a deployment where anything other than the surface processes can reach the backend port. With the check off, the backend trusts `X-PMW-User` unconditionally and any caller can impersonate an administrator.

When there is no identity at all

If sign-in is switched off for the main application, no `X-PMW-User` header is sent. The backend treats this as a single-user deployment: per-user settings and filter presets share one profile, notes have no author, and every power-user endpoint refuses the call because there is no identity to authorise. The integration console ignores the toggle — it always requires sign-in.

Sessions are per surface

Each surface issues its own cookie, and each cookie is bound to that surface's audience. A cookie minted by the main application is not accepted by the admin interface or the integration console, and vice versa. Signing in to one surface does not sign you in to the others.

Surface	Cookie	Audience	Default lifetime when no idle timeout is set
Main application	`pmw_session`	app	12 h (`PMW_SESSION_TTL` , 43200 s)
Integration console	`pmw_integration`	integration	12 h (same variable)
Admin interface	`pmw_admin`	admin	8 h (`PMW_ADMIN_SESSION_TTL` , 28800 s)

All three cookies are HttpOnly, `SameSite=Lax` , path `/` , and marked Secure when served over HTTPS. They are stateless and Fernet-signed: the token carries the username, the user's session epoch, the audience, and the original sign-in time, and it is re-minted on every authenticated request. Regardless of activity, a sign-in dies **7 days** after it began (`PMW_SESSION_MAX_LIFETIME` , 604800 s).

> **Note** Separate cookies do not mean separate sign-outs. Signing out anywhere bumps the user's session epoch server-side, which invalidates **every** outstanding token of that user on all three surfaces. Any password change or admin password reset does the same; the single exception is an administrator changing their own password in the Profile overlay, whose current admin session is re-issued in the same response.

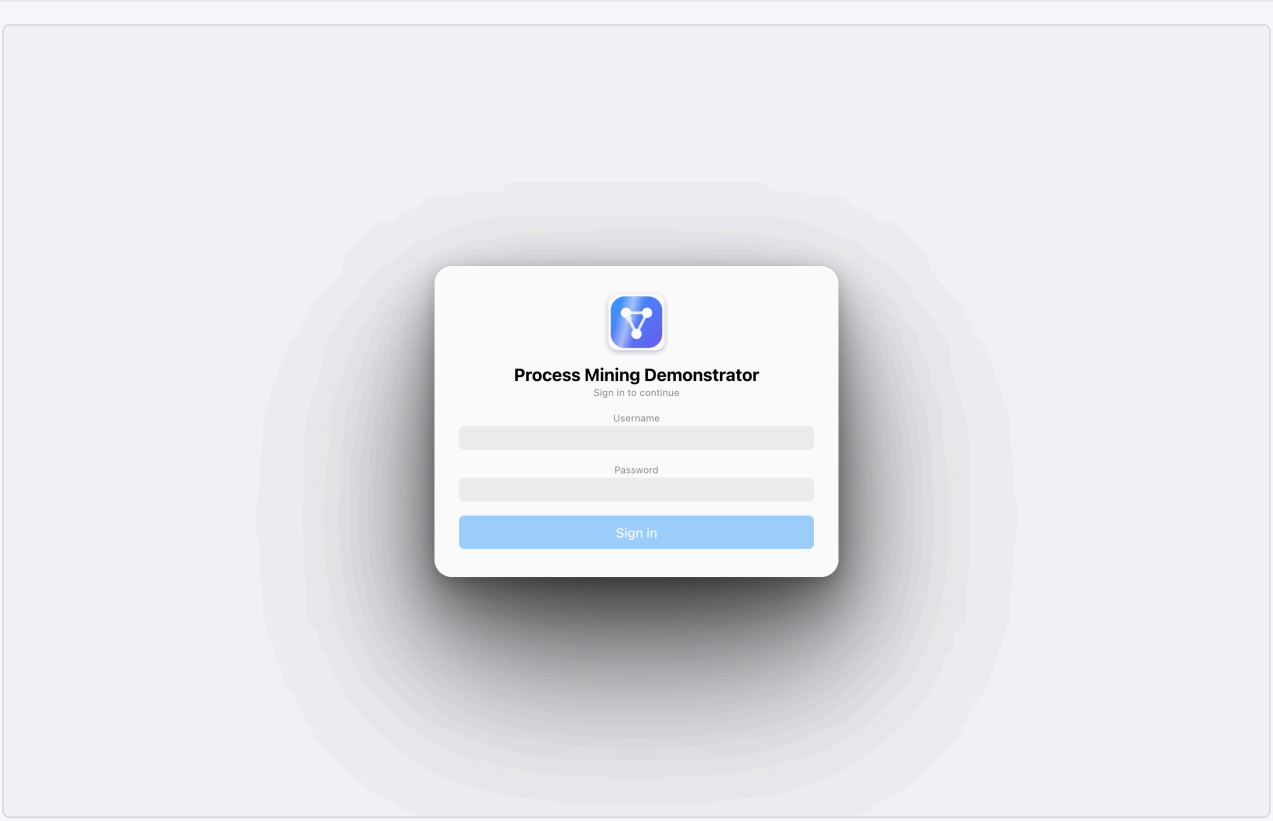


Figure 2.3 — The main application's sign-in panel. The integration console and the admin interface use the same design with a second title line — but a different cookie and a different set of permitted accounts.

Where state lives

Everything the product must remember lives in one directory, `PMW_DATA_DIR` — `<repo>/data` by default, `/app/data` in the container, bind-mounted from `./data` by the shipped compose file. The directory is created on start, is entirely git-ignored, and is **the backup unit**.

Path	Contents
`settings.sqlite3`	Application preferences per user: filter presets, happy paths, target norms, node layouts, legal-disclaimer acceptance
`security.sqlite3`	Users (scrypt-hashed passwords), roles, database connections with ownership and assignments, TLS certificates and mode, LDAP configuration, passkey credentials, encrypted TOTP secrets and recovery-code hashes, lockouts, backup schedule, integration source checkpoints
`logs.sqlite3`	The live structured log, shared by all four processes
`logs/`	Rotated log segments exported as `.log` files (`DATE -- TIME -- SEVERITY -- CLIENT-IP -- USER -- TAG -- text`)
`secret.key`	The Fernet key that encrypts every stored secret. Created atomically on first run with mode `0600` and never overwritten
`certs/`	`active.crt` / `active.key` for the user-facing TLS listeners, `internal.crt` / `internal.key` for the backend hop, plus stored certificates
`backups/`	Timestamped AES-256-GCM encrypted `.json` envelopes, pruned beyond the configured retention count
`license.json`	The uploaded Ed25519-signed runtime licence
`demo_grace.json`	The one-time demo-grace marker
`gui.pid` , `admin.pid` , `integration.pid`	PID files that let the admin interface restart the listeners in place with SIGHUP
`integration_files/`	The sandbox that File sources may read from; the bundled `examples/*.log` are seeded here on first run

> **Warning** `secret.key` encrypts database passwords, LLM API keys, the LDAP service password, certificate private keys and the backup password. Lose it and every stored secret is unrecoverable — restoring the SQLite files alone is not enough. Back up the whole `data/` directory.

What lives in Exasol

No process-mining data is stored in the `data/` directory. Projects, event logs and their annotations live in the Exasol schema named by each connection, in six tables:

PROJECTS

One row per project: `PROJECT_ID` , `TITLE` , `DESCRIPTION` .

JOURNEYS

The event log and the only large table: one row per event step, carrying `PROJECT_ID` , `EVENT_ID` (the case identifier), `STEP` , `STEP_ID` , `EVENT_TIME` , three free-form `META_*` columns and `SAMPLE_SET` .

STEPS

Per-step presentation and scoring — description, colours, shape, score, end-of-process flag and step group — edited through the app's Step editor.

METAS

The business titles shown for the three META columns.

NOTES

Annotations attached to a node or an edge, including a filter snapshot; created lazily the first time notes are used.

TRANSITIONS_RAW

Optional. A pre-materialised directly-follows pairing that turns the map query into a plain aggregate. It is a snapshot: it goes stale as soon as JOURNEYS changes, and any read failure falls back silently to the live query.

Because analysis data is in the database and configuration is in `data/`, the two are backed up and moved independently. Restoring `data/` onto a fresh host restores users, roles, connections, certificates and settings, but not a single event.

Integration checkpoints

Each integration data source keeps one read checkpoint — byte offset, file size, a signature of the file's first 64 bytes, the cumulative record count and the last error — in `security.sqlite3`. That checkpoint is **shared between manual imports and the file-source watchdog**: both triggers advance the same offset, which is exactly what stops the two from importing the same line twice. It advances only after a successful run, so a failed import re-reads the same lines on the next attempt.

One TLS mode for three surfaces

TLS is configured once, in the admin interface's **TLS / SSL** tab, and applies to the GUI server, the admin interface and the integration console together. Exactly one stored certificate is marked active and is materialised to `data/certs/active.crt` and `active.key` so the listeners can read it; certificate private keys are encrypted at rest and written with mode `0600`.

Mode	Main application	Admin interface	Integration console
Off	HTTP only (:8080)	HTTP only (:8090)	HTTP only (:8100)
Optional	:8080 + :8443	:8090 + :8453	:8100 + :8463
Required	HTTPS only (:8443)	HTTPS only (:8453)	HTTPS only (:8463)

Changes take effect on restart. The **↻ Restart app server** button in **App Control** does this without a terminal: each TLS-aware launcher writes a PID file into `data/`, and the admin service sends SIGHUP so the listeners rebind with the current plan. If a mode needs a certificate but none is active, every server falls back to HTTP rather than becoming unreachable.

All surfaces send `X-Frame-Options: DENY`, a `frame-ancestors 'none'` content-security policy, `X-Content-Type-Options: nosniff` and `Referrer-Policy: no-referrer`, plus HSTS when TLS is on.

> **Warning** The per-source-IP sign-in throttle keys on the socket peer address and deliberately ignores `X-Forwarded-For`. Behind a reverse proxy every client therefore shares one throttle bucket. Terminate TLS at the application itself, as `run.sh` does, unless you add trusted-proxy handling first.

Splitting the deployment

Because only the GUI server talks to the compute backend, and only the compute backend talks to Exasol, the backend can be moved next to the database while the surfaces stay near the users.

1. On the database-side host, start the compute backend with `PMW_BACKEND_HOST` set to an address the surface host can reach, and keep `PMW_BACKEND_PORT` (default 8000).
2. Copy the backend's `data/certs/internal.crt` to the surface host, or place a real certificate in front of the backend.
3. On the surface host, set `PMW_BACKEND_URL` to `https://<backend-host>:8000` and `PMW_BACKEND_CA` to the copied certificate (or to the real CA bundle; an empty value falls back to the system trust store).
4. Leave `PMW_REQUIRE_PROXY_AUTH` at its default of `1`, and firewall port 8000 so that only the surface host can reach it.
5. Restart both sides and confirm the backend answers on `/api/health`.

Checking that all four processes are running

1. Start the stack with `./run.sh` (or `docker compose up -d --build`).
2. Read the launcher's output. A healthy start prints one line per process:

   ```
   → compute backend on https://127.0.0.1:8000 (TLS, internal cert)
   → admin interface (HTTP 8090 / HTTPS 8453, per TLS mode)
   → GUI server (HTTP 8080 / HTTPS 8443, per TLS mode)
   → integration console (HTTP 8100 / HTTPS 8463, per TLS mode)
   ```

3. Open `http://127.0.0.1:8080` (Docker: `http://localhost:10080`). If the page loads but every data call returns 502, the surfaces are up and the compute backend is not.
4. Probe the backend directly on the host that runs it:

   ```
   curl -k https://127.0.0.1:8000/api/health
   ```

   The `-k` is required because the backend serves its internal self-signed certificate. This is the same probe the container's `HEALTHCHECK` runs every 30 seconds.
5. Open `http://127.0.0.1:8090` for the admin interface and `http://127.0.0.1:8100` for the integration console (Docker: ports 10090 and 10100). A 503 page titled **Console unavailable** on the latter means the console is switched off in the admin **Integration** tab, not that the process failed.
6. Press `Ctrl+C` to stop everything: `run.sh` traps the signal and kills all child processes.

> **Tip** In Docker the four processes share one container and one `tini` init process, so `docker compose logs` shows all four interleaved. The container is marked healthy only when the compute backend answers its health probe — the surfaces can be serving pages while the container is still reported unhealthy.

3 Installation and Deployment

> This chapter tells you how to get Process Mining Demonstrator — Web Edition running, either directly on a machine with Python and Node.js installed or as a single Docker container. It covers the system requirements, the `run.sh` launcher and its flags, the shipped Docker Compose file with its port offsets and data volume, the complete reference of `PMW_*` environment variables, a first-run checklist, where the runtime licence file lives, where backups are written, and what to check after an upgrade. It is written for the person who installs and operates the system — normally the same person who holds the Administrator account. End users need none of it.

Note Process Mining Demonstrator — Web Edition is intended for demonstration and educational purposes. It is not a production-ready process mining solution. Size and secure your deployment accordingly.

3.1 System requirements

Only the compute backend has hard runtime requirements. Node.js is needed solely to build the single-page application (SPA) and is not required at runtime — the Docker image builds the SPA in a separate stage and ships only the compiled result.

Item	Requirement	Notes
Python	3.13 or 3.14	Falls back gracefully to 3.11+. The Docker image pins `python:3.13-slim`.
Node.js	20 or newer	Needed only to build the React SPA locally. The Docker image builds it in a `node:22-slim` stage, so the runtime container never needs Node.
Exasol	Any reachable instance	The only supported analytics database. Accessed through `pyexasol`.
LLM endpoint	Optional; any OpenAI-compatible endpoint	Used by the AI Documentation view. Configured per database connection, not globally.
Docker	Optional; Docker Compose	Compose 2.22 or newer if you want `docker compose watch` auto-rebuild.

Python dependencies

All backend dependencies are pinned in `requirements.txt`: fastapi 0.139.2, uvicorn[standard] 0.51.0, pyexasol 2.3.0, openai 2.46.0, pydantic 2.13.4, pydantic-settings 2.14.2, cryptography 49.0.0, ldap3 2.9.1, httpx 0.28.1, orjson 3.11.9, python-multipart 0.0.20, regex 2026.7.19 (the ReDoS-safe engine behind admin log search), tzdata 2026.3, webauthn 3.0.0 (passkeys), pyotp 2.10.0 and qrcode 8.2 (TOTP enrolment; the SVG factory is used so no imaging library is required).

Note `tzdata` is not optional. The slim container image carries no system IANA zone database, and the admin display-timezone setting — which also governs when scheduled backups fire — depends on it.

Test and development dependencies live in `requirements-dev.txt` and are installed *on top of* `requirements.txt`: pytest 9.1.1, pytest-asyncio 1.4.0 and httpx2 2.9.1. The httpx2 package is test-only; production code continues to use httpx.

Two different licences

Do not confuse the two:

Software licence
: The source code is distributed under the MIT licence (file `LICENSE` in the repository).

Runtime application licence
: A separate Ed25519-signed `license.json` that the compute backend requires in order to keep running. See section 3.9.

3.2 Local installation

The launcher `run.sh` hard-codes the interpreter path `./.venv/bin/python`. The virtual environment must therefore live at `.venv` in the repository root — not elsewhere, and not under a different name.

1. Open a terminal in the repository root.
2. Create the virtual environment and install the backend dependencies:

   ```
   python3.13 -m venv .venv
   .venv/bin/pip install -r requirements.txt
   ```

   Python 3.14 works equally well; substitute `python3.14`.
3. Optionally add the test dependencies:

   ```
   .venv/bin/pip install -r requirements-dev.txt
   ```

4. Build the single-page application:

   ```
   ./run.sh --build
   ```

   On the first run this prints `→ installing frontend dependencies` and runs `npm install` inside `frontend/web`; afterwards it prints `→ building the SPA` and runs `npm run build`. The script then exits without starting any server.
5. Start the stack:

   ```
   ./run.sh
   ```

> **Warning** If the virtual environment is missing, `run.sh` refuses to start and prints:
> `error: virtualenv missing — create it with:`
> `  python3.13 -m venv .venv && .venv/bin/pip install -r requirements.txt`

3.3 The `run.sh` launcher

`run.sh` is the single entry point for all four Python processes. It changes to its own directory before doing anything, runs under `set -euo pipefail`, and traps `EXIT`, `INT` and `TERM` so that pressing Ctrl-C kills every child process cleanly. There is no separate stop script.

Invocation	What it starts
`./run.sh`	Compute backend, admin interface, GUI server and integration console. If `frontend/web/dist/index.html` is missing, the SPA is built first automatically.
`./run.sh --dev`	Compute backend, admin interface and the Vite dev server on `http://localhost:5173` with hot reload (HTTP only). No GUI server and no integration console. Runs `npm install` first if `node_modules` is absent.
`./run.sh --build`	Rebuilds the SPA and exits. No servers are started.

A normal start prints these lines; use them to confirm that all four processes came up:

```
→ compute backend on https://127.0.0.1:8000 (TLS, internal cert)
→ admin interface (HTTP 8090 / HTTPS 8453, per TLS mode)
→ GUI server (HTTP 8080 / HTTPS 8443, per TLS mode)
→ integration console (HTTP 8100 / HTTPS 8463, per TLS mode)
```

In development mode the third and fourth lines are replaced by `→ Vite dev server on http://localhost:5173`.

> **Warning** `--dev` sets `PMW_REQUIRE_PROXY_AUTH=0` automatically. This is safe only because Vite proxies straight to the backend over loopback with no GUI proxy in between. Never use `--dev` for anything but local development: with proxy authentication off, any local process can forge the `X-PMW-User` header and reach `/api/*` directly.

The four processes and their ports

Process	Directory	HTTP	HTTPS	Purpose
Compute backend	`backend/`	—	8000 (always TLS)	Exasol access, analytics, simulation, sampling, LLM proxy, settings store, licensing watchdog, integration layer and file watchdog
GUI server	`frontend/`	8080	8443	Serves the SPA, proxies `/api/*` to the backend, enforces sign-in
Admin interface	`admin/`	8090	8453	TLS, users, database connections, LDAP, logging, backups, licence, customise, restart
Integration console	`integration/`	8100	8463	Data-source configuration; sign-in restricted to power users, developers and administrators

After a local start, open:

Main application

`http://127.0.0.1:8080`

Admin interface

`http://127.0.0.1:8090`

Integration console

`http://127.0.0.1:8100` (power users, developers and administrators only; every other account is refused with a clear message)

Four operational facts follow from this layout and are worth understanding before you plan a deployment:

- The browser only ever talks to the GUI server. The compute backend can therefore run on a separate host close to the database; point the GUI proxy at it with `PMW_BACKEND_URL`.
- The GUI-to-backend hop is always TLS-encrypted, even on loopback. The backend mints an internal self-signed certificate at start (`data/certs/internal.crt` and `internal.key`), independently of the user-facing TLS mode, and the GUI proxy pins it through `PMW_BACKEND_CA`.
- The integration console ports are derived by convention as the admin port plus ten (8090 → 8100, 8453 → 8463) unless you override them explicitly.
- All three user-facing surfaces share one sign-in stack (password, TOTP two-factor, WebAuthn passkey) and one TLS mode with one active certificate.

> **Warning** The per-source-IP sign-in throttle keys on the socket peer address and never trusts `X-Forwarded-For`. Behind a reverse proxy, every client shares a single throttle bucket. Terminate TLS at the application itself, as `run.sh` does, or add trusted-proxy handling before fronting it.

3.4 Docker deployment

One image, `process-mining-web:latest`, runs all four services through `run.sh` as its `CMD`. The container is named `process-mining-web`.

1. Edit `docker-compose.yml` and set `PMW_DEFAULT_ADMIN_PASSWORD` (and optionally `PMW_DEFAULT_ADMIN_USER`) in the `environment:` block. The lines are present but commented out, with `Administrator` and `"change-me"` placeholders.
2. Build and start:

   ```
   docker compose up -d --build
   ```

3. Open the admin interface at `http://localhost:18090` and sign in.
4. Open the main application at `http://localhost:18080` — or the Launcher at `http://localhost:18080/launcher.html`.

Host port mapping — the +10000 offset

Host ports are offset by 10000 from the container ports so that they do not clash with other local services (the compose file cites phpLDAPadmin on host port 8080). This is only the shipped default — the left-hand

side of each `ports:` mapping in `docker-compose.yml` is the host port and may be remapped to any free port.

Host port	Container port	Surface
18080	8080	Main application (HTTP)
18443	8443	Main application (HTTPS)
18090	8090	Admin interface (HTTP)
18453	8453	Admin interface (HTTPS)
18100	8100	Integration console (HTTP)
18463	8463	Integration console (HTTPS)
18110	8110	Actions surface (HTTP)
18473	8473	Actions surface (HTTPS)

Note The HTTPS host ports do nothing until TLS is switched to *Optional* or *Required* in the admin interface. Until then only the HTTP ports answer.

Warning Do not map a surface to host port `10080`: Chrome, Firefox and Safari block it as a restricted port, so a page served there never loads. The shipped compose file therefore uses the +10000 offset (host `18080`, `18090`, …); the compose file is authoritative for the URLs.

Container port 8000 — the compute backend — is deliberately not published. It stays private inside the container, where `PMW_BACKEND_HOST` is baked to `127.0.0.1`. The image's `EXPOSE` lists 8080, 8443, 8090, 8453, 8100, 8463, 8110 and 8473.

What the image contains

The build is a two-stage Dockerfile:

- Stage 1 (`node:22-slim`) runs `npm ci` against the lockfile — cached until the lockfile changes — and then `npm run build`.
- Stage 2 (`python:3.13-slim`) creates `/app/.venv` and pip-installs `requirements.txt` into it, so `run.sh`'s `./.venv/bin/*` paths work unchanged. It copies `backend/`, `admin/`, `integration/`, `frontend/`, `examples/` (the bundled demo logs) and `run.sh`, and drops the built SPA into `frontend/web/dist`.

Environment baked into the image: `PYTHONUNBUFFERED=1`, `PIP_NO_CACHE_DIR=1`, `PMW_DATA_DIR=/app/data`, `PMW_BACKEND_HOST=127.0.0.1`, and `PMW_FRONTEND_HOST`, `PMW_ADMIN_HOST` and `PMW_INTEGRATION_HOST` all set to `0.0.0.0` so that the published ports are reachable from the host.

A `HEALTHCHECK` fetches `https://127.0.0.1:8000/api/health` every 30 seconds, with a 5-second timeout, a 25-second start period and 3 retries. Certificate verification is disabled for that probe because the backend serves its internal self-signed certificate and loopback liveness does not need verification.

Compose sets `init: true`, so tini runs as PID 1 for clean signal handling and no zombie processes, and `restart: unless-stopped`.

`.dockerignore` keeps local state and secrets out of the build context: `.git`, `.venv`, `node_modules`, `frontend/web/dist`, the entire `data` directory, `**/*.sqlite3`, `**/secret.key`, `**/certs` and the usual caches. A locally populated data directory or encryption key can therefore never be baked into an image by accident.

Persistence: the data volume

The compose file declares a single bind mount: `./data`, relative to the compose file, onto `/app/data` inside the container — which is `PMW_DATA_DIR`. Everything the application must keep lives there.

> **Note** There is no separate `integration_files` mount in the shipped compose file. The File-source sandbox lives inside the same volume at `data/integration_files`, and the bundled `examples/*.log` demo logs are seeded into it on first run. Add a second mount only if you move the sandbox elsewhere with `PMW_INTEGRATION_FILES_DIR`.

Auto-rebuild during development

Run `docker compose watch` (or `docker compose up --watch`) to rebuild the image whenever source changes. The watch rules cover `./backend`, `./admin`, `./integration`, `./frontend` (ignoring `node_modules/` and `dist/`), `./requirements.txt`, `./run.sh` and `./Dockerfile`; each change triggers `action: rebuild`. Docker's layer cache keeps this cheap because untouched layers are reused. This requires Docker Compose 2.22 or newer.

3.5 Environment variable reference

All four processes read the same configuration module, so every variable below applies wherever it is relevant. Boolean-style variables accept `1`, `true`, `yes` or `on` (case-insensitive) as true; `PMW_REQUIRE_PROXY_AUTH` additionally treats `0`, `false`, `no` and `off` as false.

Hosts and ports

Variable	Default	Effect
`PMW_BACKEND_HOST`	`127.0.0.1`	Bind address of the compute backend. Keep it on loopback unless the backend runs on a separate host.
`PMW_BACKEND_PORT`	`8000`	Compute backend port. Always HTTPS, using the internal certificate.
`PMW_FRONTEND_HOST`	`127.0.0.1` (Docker image: `0.0.0.0`)	Bind address of the GUI server.
`PMW_FRONTEND_PORT`	`8080`	GUI HTTP port.
`PMW_FRONTEND_HTTPS_PORT`	`8443`	GUI HTTPS port, used when the TLS mode is Optional or Required.
`PMW_ADMIN_HOST`	`127.0.0.1` (Docker image: `0.0.0.0`)	Bind address of the admin interface.
`PMW_ADMIN_PORT`	`8090`	Admin HTTP port.
`PMW_ADMIN_HTTPS_PORT`	`8453`	Admin HTTPS port.
`PMW_INTEGRATION_HOST`	same as `PMW_ADMIN_HOST` (Docker image: `0.0.0.0`)	Bind address of the integration console.
`PMW_INTEGRATION_PORT`	admin port + 10 → `8100`	Integration console HTTP port.
`PMW_INTEGRATION_HTTPS_PORT`	admin HTTPS port + 10 → `8463`	Integration console HTTPS port.

Backend reachability and internal TLS

Variable	Default	Effect
`PMW_BACKEND_URL`	`https://{PMW_BACKEND_HOST}:{PMW_BACKEND_PORT}`, i.e. `https://127.0.0.1:8000`	The URL the GUI proxy uses to reach the compute backend. Set this when the backend runs on another host.
`PMW_BACKEND_CA`	`{data}/certs/internal.crt`	CA bundle the GUI proxy verifies the backend against; by default the pinned internal self-signed certificate. Point it at a real CA when the backend sits behind one, or set it empty to fall back to the system trust store.
`PMW_REQUIRE_PROXY_AUTH`	`1` (on)	Makes the backend require a proxy-auth secret on `/api/*`, so only the GUI proxy — which has already validated the session — can reach it. `run.sh --dev` sets it to `0`. Do not disable it in production.

Data and storage

Variable	Default	Effect
`PMW_DATA_DIR`	`<repo>/data` (Docker: `/app/data`)	Root of all persisted state. Created at start together with the `logs/`, `certs/`, `backups/` and `integration_files/` subdirectories. Git-ignored; this is the directory you back up.

Sessions and sign-in

Variable	Default	Effect
`PMW_SESSION_TTL`	`43200` (12 hours)	Main-application session lifetime. A sliding idle window: the cookie is re-minted on every authenticated request.
`PMW_SESSION_MAX_LIFETIME`	`604800` (7 days)	Hard cap on one sign-in regardless of activity, measured from the original sign-in. Reaching it forces a fresh sign-in, which defeats a captured cookie kept alive by periodic requests.
`PMW_ADMIN_SESSION_TTL`	`28800` (8 hours)	Admin-interface session lifetime.
`PMW_DEFAULT_ADMIN_USER`	`Administrator`	Username of the bootstrap administrator, created on first run only if no users exist.
`PMW_DEFAULT_ADMIN_PASSWORD`	`Administrator`	Password of the bootstrap administrator, subject to the same first-run-only rule. Override it before the first start, especially under Docker.
`PMW_RESET_LOCKOUTS`	unset	Break-glass valve. Set it to `1` and restart to clear all failed-sign-in lockouts; the log records `PMW_RESET_LOCKOUTS set — cleared all failed-sign-in lockouts.`

> **Note** Accounts are disabled automatically after a configurable number of wrong passwords. The threshold lives in the admin interface, tab **Users**; the default is 3 and `0` switches lockout off.

Passkeys and TOTP

Variable	Default	Effect
`PMW_PASSKEY_RP_ID`	unset (derived from the request `Host` header)	Relying-Party ID: the registrable domain without a port, so the application on :8443 and the admin interface on :8453 of one host share a single passkey. Set it only when application and admin live on separate sub-domains, and then use the shared parent domain.
`PMW_PASSKEY_RP_NAME`	`Process Mining Demonstrator`	Human-readable Relying-Party name shown by the authenticator.
`PMW_PASSKEY_ORIGINS`	empty (accepts the request's own origin)	Comma-separated allowlist of expected origins in the form `scheme://host[:port]`. Single-host deployments need neither this nor the RP ID.
`PMW_MFA_ISSUER`	`Process Mining Demonstrator`	Issuer label shown in authenticator apps for TOTP enrolments.

Warning WebAuthn requires a secure context — HTTPS, or `localhost` — and a real hostname. Reaching the system by bare IP address makes the browser fail with "*the effective domain … is not a valid domain.*". The interface hides the passkey controls and shows a hint when it detects an IP address. Password sign-in always remains available.

Licensing and demo mode

Variable	Default	Effect
`PMW_LICENSE_GRACE_SECS`	`1800` (30 minutes)	How long the backend keeps running without a valid licence, giving you time to upload one through the admin interface.
`PMW_LICENSE_POLL_SECS`	`15`	How often the backend re-reads the licence during the grace period. Short enough that an upload cancels a pending shutdown within seconds.
`PMW_RESET_DEMO`	unset	Set to `1` to clear the one-time demo marker `data/demo_grace.json` at backend start, granting a fresh demo window. The log records `PMW_RESET_DEMO set — cleared the one-time demo marker.` Deleting the file has the same effect.

Queries and LLM access

Variable	Default	Effect
`PMW_QUERY_TIMEOUT`	`30` seconds	Wall-clock limit for the heavy statistics queries.
`PMW_BLOCK_PRIVATE_LLM_HOSTS`	unset (off)	SSRF hardening for the user-configurable LLM endpoint. By default loopback and private LLM hosts are allowed, because Ollama, LM Studio and internal vLLM servers are primary use cases; link-local and cloud-metadata addresses (169.254.169.254), multicast, reserved and unspecified addresses and non-HTTP(S) schemes are always refused. Set it to `1` in hardened or cloud deployments to refuse loopback and private targets as well.

Note Whatever the setting, outbound LLM connections are pinned to the IP address that was vetted, which defeats DNS-rebinding attacks.

Integration console

Variable	Default	Effect
`PMW_INTEGRATION_FILES_DIR`	`{data}/integration_files`	The sandbox directory File sources may read from. Nothing outside it can be opened; path traversal and symlink escapes are rejected. The bundled `examples/*.log` demo logs are seeded here on first run.
`PMW_INTEGRATION_ALLOW_ANY_PATH`	unset (off)	Set to `1` to let File sources read any absolute path the server process can read. Developer-trusted deployments only — it enables reading arbitrary server files.
`PMW_INTEGRATION_WATCHDOG`	`1` (on)	Globally enables the file-source watchdog, the background loop that imports newly appended lines when a watched file grows. `0` disables it.
`PMW_INTEGRATION_WATCHDOG_TICK`	`10` seconds (floor of 5 enforced)	Base wake interval of the watchdog loop. Each source additionally has its own configured poll interval.

3.6 Enabling TLS

> Requires: **Administrator**

TLS is configured entirely in the admin interface, tab **TLS / SSL**. One mode and one active certificate are shared by the GUI server, the admin interface and the integration console.

Mode	GUI binds	Admin binds	Integration binds
Off	HTTP only (:8080)	HTTP only (:8090)	HTTP only (:8100)
Optional	:8080 and :8443	:8090 and :8453	:8100 and :8463
Required	HTTPS only (:8443)	HTTPS only (:8453)	HTTPS only (:8463)

1. Sign in to the admin interface and open the **TLS / SSL** tab.
2. Either generate a self-signed certificate — supplying common name, subject alternative names, validity and key size — or upload a PEM certificate together with its key.
3. Mark exactly one certificate as active.
4. Choose the mode: Off, Optional or Required.
5. Open **App Control** and press **↻ Restart app server**. The listeners rebind in place; no terminal is needed.

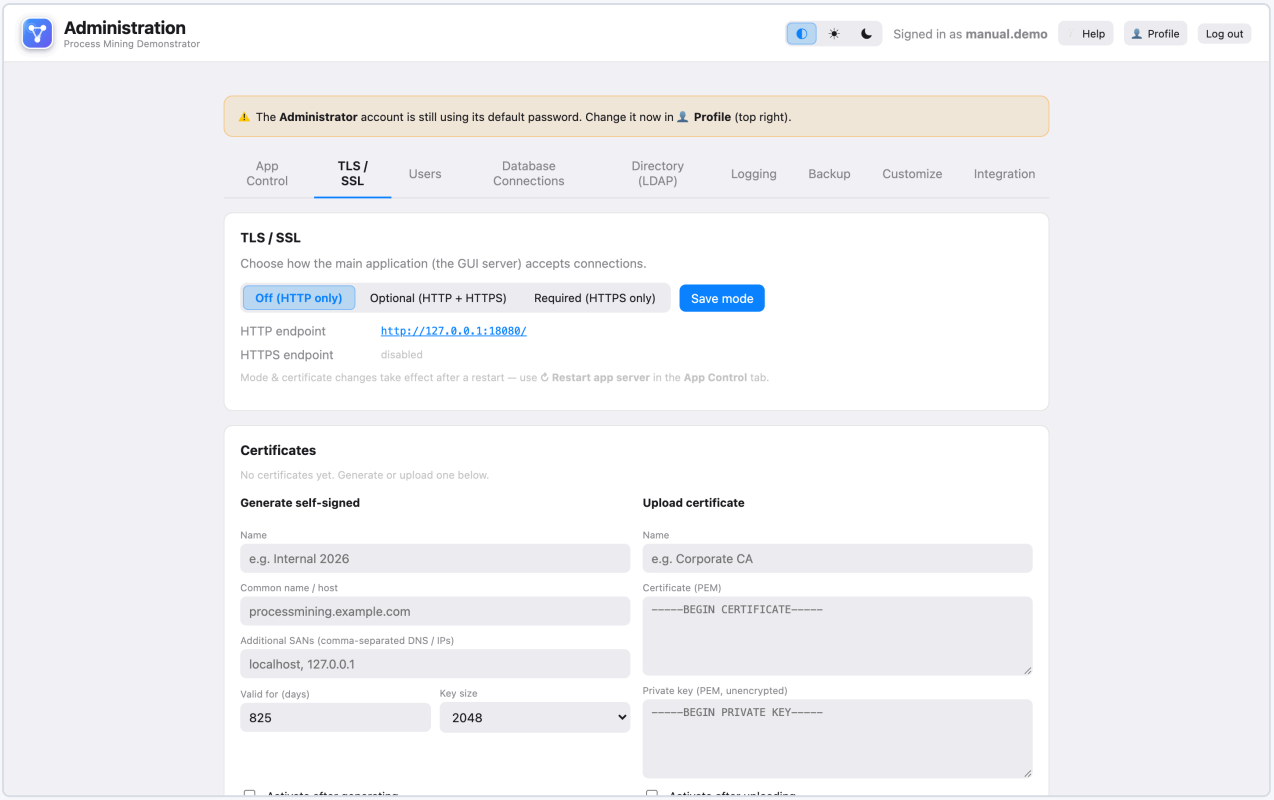


Figure 3.1 — The TLS / SSL tab: certificate list, generation form and the mode selector shared by all three user-facing surfaces.

The restart works because each TLS-aware launcher writes a PID file — `data/gui.pid`, `data/admin.pid` and `data/integration.pid` — and the admin service sends `SIGHUP`, on receipt of which the launcher re-binds according to the current TLS plan.

> **Note** The design is fail-safe. If a mode needs a certificate but none is active, every server falls back to HTTP so that nothing becomes unreachable — including the admin interface itself. A bad certificate can never lock you out.

The active certificate is materialised to disk as `data/certs/active.crt` and `active.key` so that the TLS listener can read it. Certificate private keys are encrypted at rest with Fernet in the security store, and key files are written with mode `0600` from the moment they are created. Both the application and admin servers send `X-Frame-Options: DENY`, a Content-Security-Policy of `frame-ancestors 'none'`, `X-Content-Type-Options: nosniff` and `Referrer-Policy: no-referrer`, plus HSTS when TLS is on.

3.7 The `data/` directory

The data directory defaults to `<repo>/data`, is overridable with `PMW_DATA_DIR`, and is bind-mounted as `./data` under Docker. It is entirely git-ignored, and it is the unit you back up.

Path	Contents
`settings.sqlite3`	Application preferences: filter presets, happy paths, target norms and per-user node layouts
`security.sqlite3`	Users with scrypt-hashed passwords, roles, database connections with ownership and assignments, TLS certificates and mode, LDAP configuration, passkey credentials, encrypted TOTP secrets and recovery-code hashes, lockouts and the backup schedule
`logs.sqlite3`	The live structured log shared by all four servers
`logs/`	Rotated log segments exported as `.log` files, one line per entry in the form `DATE -- TIME -- SEVERITY -- CLIENT-IP -- USER -- TAG -- text`
`secret.key`	The Fernet key that encrypts every secret at rest: database passwords, LLM API keys, the LDAP service password, certificate private keys and the backup password. Created atomically on first run with mode `0600` and never overwritten.
`certs/`	`active.crt` and `active.key` for user-facing TLS, `internal.crt` and `internal.key` for the backend loopback hop, plus the stored certificates
`backups/`	Scheduled and on-demand encrypted backups as timestamped AES-256-GCM `.json` envelopes
`license.json`	The uploaded Ed25519-signed runtime licence
`demo_grace.json`	The one-time demo-grace marker
`gui.pid`, `admin.pid`, `integration.pid`	PID files that enable the admin interface's in-place restart
`integration_files/`	The sandbox for integration File sources; `examples/*.log` is seeded here on first run

Warning Losing `secret.key` orphans every stored secret. Database passwords, API keys, certificate private keys and the backup password become unrecoverable, and no amount of database restore brings them back. Treat it with the same care as a private key, and never copy it into a build context or an image.

3.8 Backups

Requires: **Administrator**

The admin interface, tab **Backup**, runs encrypted backups on an in-process scheduler. No external cron job is involved.

1. Open the **Backup** tab in the admin interface.
2. Enable scheduled backups.

3. Build the schedule either with the dropdowns — hourly, daily, weekly or monthly, plus a time — or by typing a raw five-field cron expression. The panel shows the resulting cron string and a plain-English preview as you edit.
4. Set the retention count: files beyond this number are pruned automatically.
5. Set the AES-256-GCM password.
6. Save, then press **Run backup now** to test the configuration. The outcome of the last run is displayed.

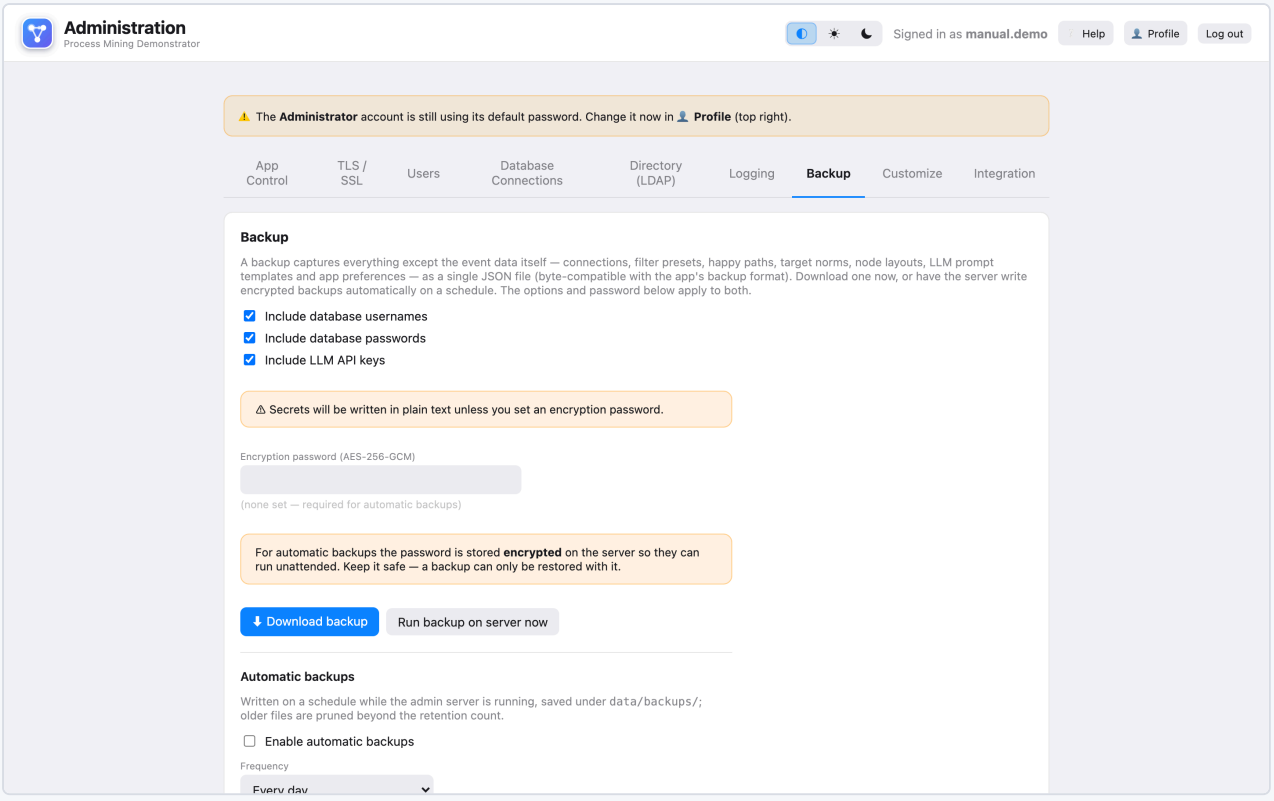


Figure 3.2 — The Backup tab, with the schedule builder, retention count, encryption password and the Run backup now button.

Backups are written to `data/backups/` as timestamped, encrypted `.json` envelopes. The backup password is stored Fernet-encrypted at rest so that unattended runs work, and it is never returned to the client: the API exposes only a `hasPassword` flag. Schedule times follow the display-timezone setting in **App Control**, so "daily at 02:00" fires at 02:00 in the chosen zone and is DST-aware; the default is *Server local*.

> **Warning** In-app backups are not a substitute for disaster recovery. For that, back up the whole `data/` directory, including `secret.key` — without it the encrypted secrets inside the backup cannot be decrypted.

3.9 Where to put the licence file

> Requires: **Administrator**

The compute backend requires a signed licence. Upload the issued `license.json` in the admin interface under **App Control** → **License**; the panel shows the licensee and the expiry date, and offers a remove op-

tion. Verification is Ed25519 against a public key embedded in the application. The uploaded file is stored as `data/license.json`.

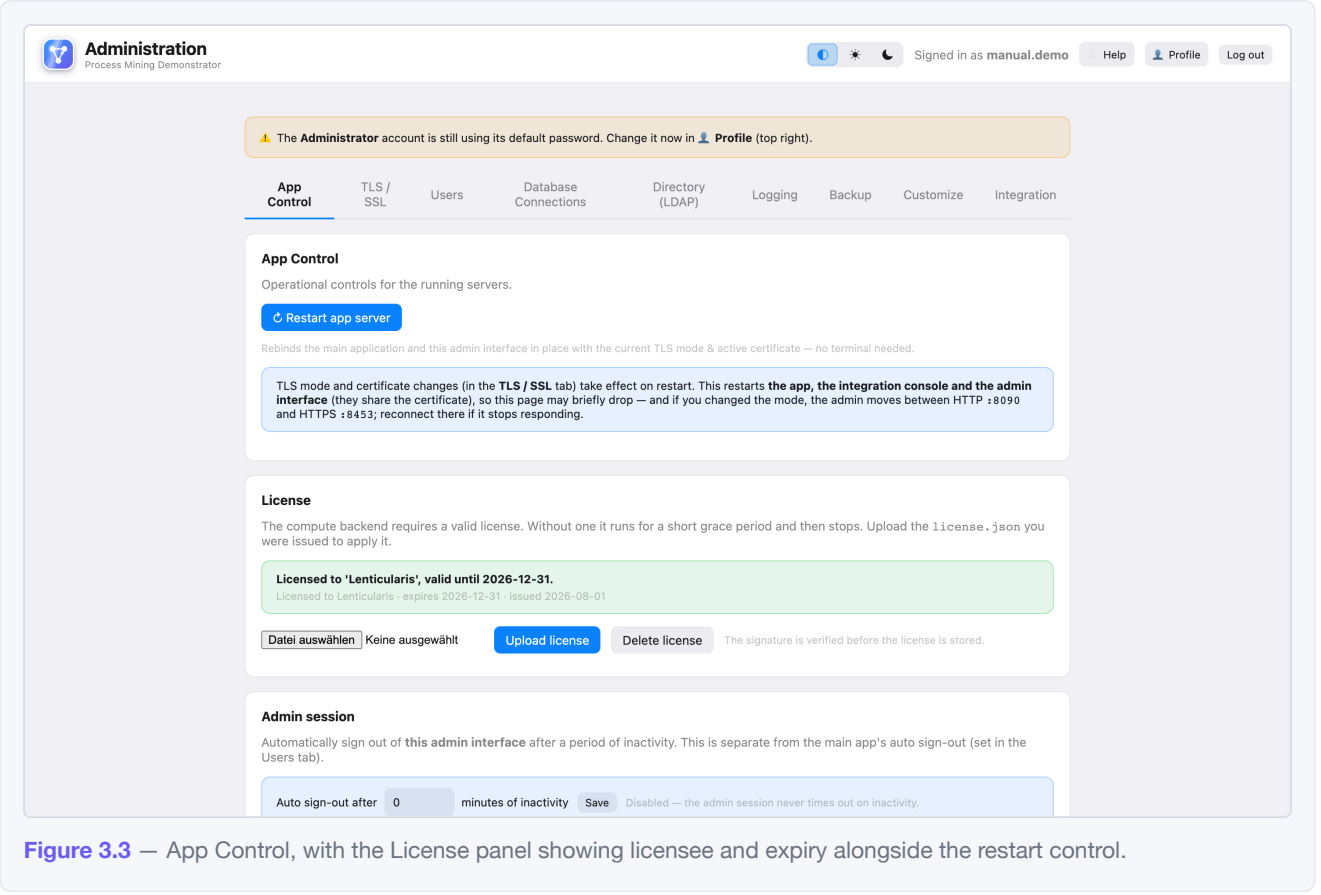


Figure 3.3 — App Control, with the License panel showing licensee and expiry alongside the restart control.

Without a valid licence the system enters **Demo Mode**. The sign-in panel shows a *Demo Mode — remaining time* banner during a one-time grace period, 30 minutes by default (`PMW_LICENSE_GRACE_SECS`). Once the grace is spent the banner reads *No License installed* and the backend stops itself until a licence is applied.

1. Open the admin interface — it keeps running even after the backend has stopped, so you can always get in.
2. Go to **App Control** and find the **License** panel.
3. Upload the issued `license.json`.
4. Wait a few seconds. The backend re-reads the licence every 15 seconds by default (`PMW_LICENSE_POLL_SECS`), so a pending shutdown is cancelled almost immediately.

> **Note** The grace period is anchored in `data/demo_grace.json` and is never renewed by restarting the application — that is what makes it one-time. Delete the file, or set `PMW_RESET_DEMO=1`, if you genuinely need a fresh demo window.

3.10 First-run checklist

Work through this list once, in order, after the very first start.

1. Start the stack with `./run.sh` or `docker compose up -d --build`.

2. Sign in to the admin interface. The default credentials are **Administrator / Administrator**, and a password change is forced on first use. If you set `PMW_DEFAULT_ADMIN_PASSWORD` before the first start, use that instead — it applied only because the security database was still empty, and it is ignored from then on.
3. Open **App Control** → **License** and upload `license.json` if the sign-in panel showed the Demo Mode banner.
4. Open **Database Connections** and define a connection: Exasol host, port, user, password and schema, optionally an OpenAI-compatible LLM server, and the TLS options. Use **Test connection**, then assign the connection to the users who should see it.
5. Optionally press **Create schema & tables** to provision `PROJECTS`, `JOURNEYS`, `STEPS`, `METAS` and `NOTES`. The statements are idempotent (`IF NOT EXISTS`), but the database account needs `CREATE SCHEMA` and `CREATE TABLE` privileges, which only a DBA can grant.
6. Open **Users**, create the accounts you need, and grant roles: Admin, the *Make power* action and the *Developer* checkbox. Enable the Passkey and 2FA columns per user if you want them.
7. Configure TLS as described in section 3.6, and restart the app server.
8. Configure scheduled backups as described in section 3.8.
9. Have a user open the main application, click a connection card under **Connections**, open **Projects**, and check that the views load from the ≡ menu.

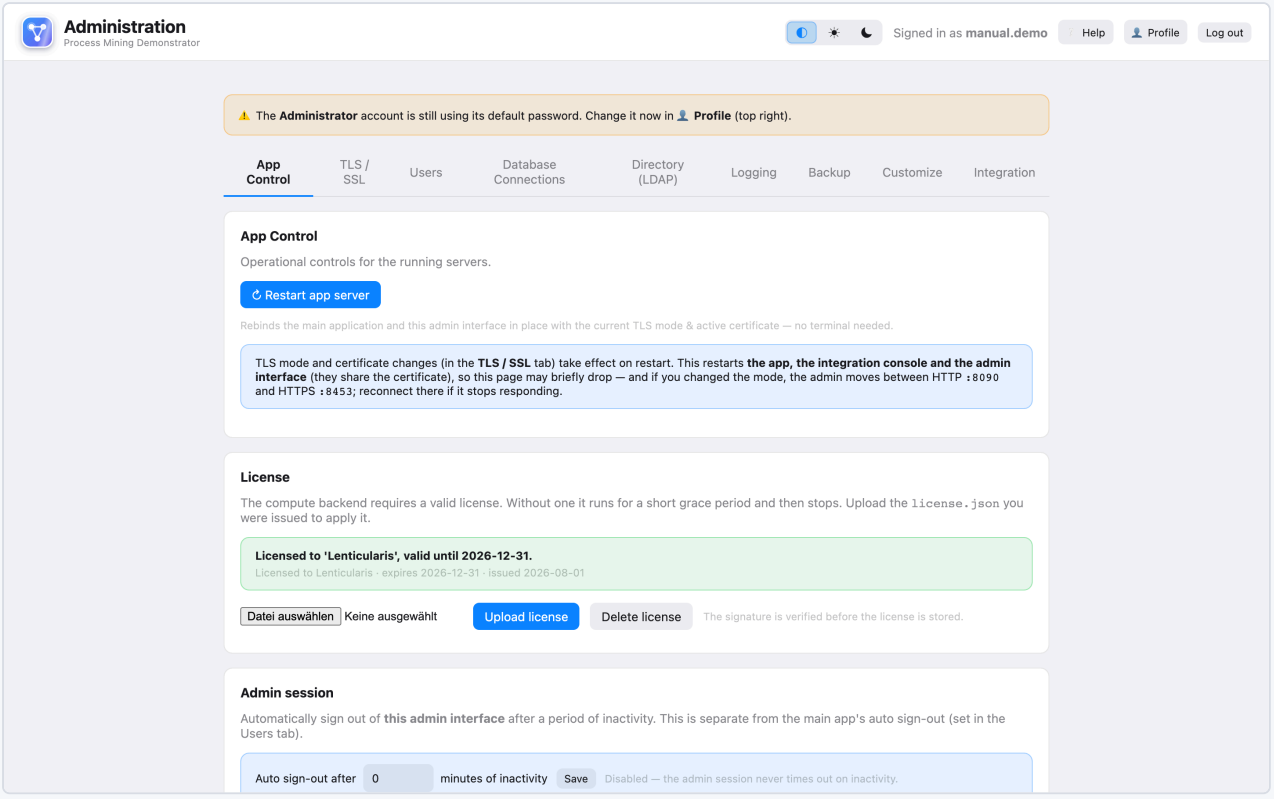


Figure 3.4 — The admin interface after the first sign-in, with the tab strip that structures the whole checklist.

> **Tip** Leaving a password or API-key field blank when you edit an existing database connection keeps the stored value. It does not clear it. Type a new value only when you actually want to change the secret.

3.11 Upgrading

Everything under `data/` is state, and the image contains none of it, so rebuilding the image or redeploying the container is safe — the state survives in the volume. Locally, upgrading means pulling the new source, reinstalling dependencies and rebuilding the SPA.

1. Stop the stack (Ctrl-C locally, or `docker compose down`).
2. Back up the `data/` directory.
3. Fetch the new version of the source.
4. Locally: run `.venv/bin/pip install -r requirements.txt` again, then `./run.sh --build`. Under Docker: run `docker compose up -d --build`, which rebuilds both stages.
5. Start the stack and work through the post-upgrade actions below.

Rebuild `TRANSITIONS_RAW`

> Requires: **Administrator**

Any pre-materialised transitions table that was built before the partition fix can show wrong per-edge values. Older builds partitioned the `LEAD()` window by `EVENT_ID` alone, which produces incorrect edges wherever an `EVENT_ID` recurs across projects or sample sets. Press **Rebuild now** once on each affected connection after upgrading. The equivalent API call on the admin server is:

```
curl -k -X POST \
  -H "Authorization: Bearer <token>" \
  https://localhost:8453/api/connections/<id>/rebuild-transitions
```

> **Note** The `-k` flag is present because the admin certificate is typically self-signed. Drop it once you have installed a certificate the client trusts.

Distribution and partition clauses on `JOURNEYS`

New tables are created with `DISTRIBUTE BY EVENT_ID` and `PARTITION BY EVENT_TIME`. Tables that predate those clauses are not modified by `CREATE TABLE IF NOT EXISTS`, because the table already exists and the statement does nothing. Apply the clauses once by hand:

```
ALTER TABLE JOURNEYS DISTRIBUTE BY EVENT_ID;
ALTER TABLE JOURNEYS PARTITION BY EVENT_TIME;
```

Migrating existing users to LDAP

A directory login never adopts a local account of the same name. If you switch an existing installation to LDAP authentication, delete or rename the old local rows first; otherwise those users keep signing in with their local passwords and the directory is never consulted for them.

> **Tip** Older README text and the header comment in `run.sh` speak of "all three services". The current system runs four — the integration console was added later. If a document mentions three, it predates that change.

4 Getting Started: Your First Session

> This chapter walks you through a complete first run of Process Mining Demonstrator — Web Edition, from opening the address in your browser to reading your first process map. It is written for a new user sitting in front of the application for the first time, and it assumes nothing has been set up for you yet. Where a step needs a permission you may not have, the chapter says so and tells you who to ask. Later chapters return to each subject in depth; here the aim is simply to get a map on screen and to make sure you can name everything you see.

4.1 Before you begin

You need four things to complete a first session. If any of them is missing, the relevant section below explains how to obtain it.

What you need	Detail
The application address	The main application runs on port **8080** (HTTP) or **8443** (HTTPS). Your administrator gives you the host name — for example `https://pm.example.com:8443`.
A browser	Any current desktop browser. Reach the server by **host name**, not by a bare IP address, if you intend to use passkeys.
An application account	A username and password, either a local account or a directory (LDAP / Active Directory) account. Only accounts that are **enabled** can sign in.
A database connection	At least one Exasol connection assigned to your account. Without one, the Connections list is empty and no data can be loaded.

> **Note** Your application sign-in is completely separate from the database credentials. You never type a database password into the application; those live only in the connection definition, encrypted at rest, and are never sent to the browser.

4.2 Opening the application

1. Type the address your administrator gave you into the browser and press Return.
2. The application shows a spinner and the word **Starting…** while it checks for an existing session and loads your settings.
3. The sign-in panel appears, headed **Process Mining Demonstrator**.

Two banners can appear above the sign-in fields before you have typed anything:

Demo Mode — remaining time: N min
: No license has been applied yet, and the installation is inside its one-time grace period. The application works normally; when the period expires the compute backend stops until a license is uploaded.

No License installed

The grace period is spent. Ask your administrator to apply a license in the admin interface under **App Control → License**.

If a user directory is configured, a small **status light** sits below the buttons: green when the directory server answered a connection test, red when it did not. The light is hidden entirely when no directory is configured.

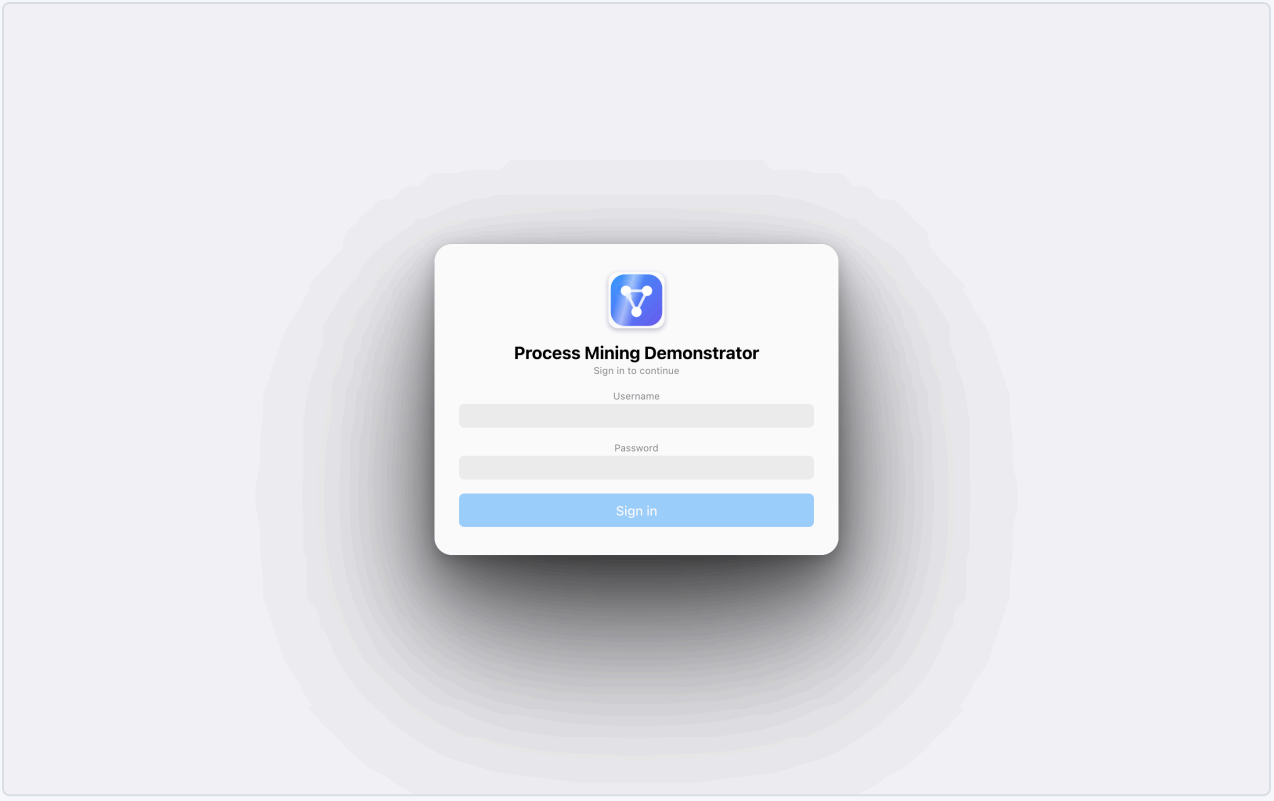


Figure 4.1 — The sign-in panel. The background is set by an administrator under Customize; the passkey button appears only when passkeys are permitted for your account and the page is served over a secure context.

4.3 Signing in

1. Type your account name into **Username**. The field takes focus automatically.
2. Type your password into **Password**.
3. Click **Sign in**, or press Return. The button is disabled while the username is blank.

If two-factor authentication is required for your account, sign-in continues with a code step. If you have not yet enrolled, the application *forces* enrolment there and then — you cannot skip it by declining to set it up.

1. Scan the displayed QR code with an authenticator app (Google Authenticator, 1Password, Authy or similar).
2. Type the current 6-digit code into the confirmation field and confirm.
3. The panel shows **Two-factor is on — save your recovery codes** together with a grid of one-time codes. Copy them somewhere safe now.

> **Warning** The recovery codes are shown once and never again. Each one signs you in a single time if you lose your authenticator. If you lose both, an administrator cannot read your secret — they can only untick two-factor for your account so you can enrol afresh.

On later sign-ins you are simply asked for the current 6-digit code (or one recovery code) after your password. Once inside the application you can review the setup at any time from 🔒 **Two-factor** in the sidebar footer.

Signing in with a passkey

If your administrator has permitted passkeys for your account, a 🔑 **Sign with Passkey** button sits beside **Sign in**. A passkey is an alternative to your password, never a replacement — the password always keeps working, so you cannot be locked out.

1. Sign in with your password first.
2. Open 🔑 **Passkeys** in the sidebar footer, next to **Sign out**, and add a passkey for this device (Touch ID, Windows Hello, or a hardware security key).
3. From then on, type your username on the sign-in panel and click 🔑 **Sign with Passkey**.

> **Note** Passkeys are tied to a domain name and require a secure context. They work only when you reach the application by host name (or `localhost` on the same machine) over HTTPS — never by a bare IP address. If the application detects an IP address it hides the passkey controls entirely and enrolment is blocked; password sign-in works either way. A passkey sign-in also skips the two-factor code step, because it is already strong authentication.

When sign-in does not work

Symptom	Cause and remedy
A message says the account is locked	Too many wrong passwords. The lockout threshold defaults to **3** attempts (0 turns it off). The account carries a **Locked** badge in the admin Users tab; an administrator unlocks it there.
Repeated failures are refused outright	The sign-in page throttles repeated failures from the same IP address with a short, self-clearing cooldown (HTTP 429). Wait, then try again.
Correct credentials, still refused	Your account may be disabled. Only enabled users can sign in; ask an administrator to check the Users tab. Directory accounts must also exist in the directory.
You were signed out due to inactivity.	Your administrator has configured an idle timeout. Simply sign in again.

> **Note** Your administrator can turn the login gate off altogether. The application then opens without any sign-in — but it also runs with **no user identity**, so every Power and Administrator capability disappears (Sampling and the advanced views are hidden and their APIs return 403), and all personal settings and filter presets fall back to one shared profile. There is no way to have both open access and the power features.

4.4 Accepting the legal disclaimer

The first time you enter the application, a full-screen **Legal Disclaimer** dialog appears. Read it — it sets out what the product is for and how far you may rely on its output.

Purpose and scope

The application is provided for educational and exploratory purposes only. Any productive or business-critical use is the sole responsibility of the user.

AI-generated results

The AI-powered analysis features use large language models to interpret process data, and such models can produce results that are incorrect, incomplete or misleading.

Data accuracy

Process maps, KPIs, journey statistics and every other computed result depend entirely on the quality, completeness and correctness of the data in your database.

🖥 **Device transfer**

Acceptance is recorded for this installation. If the device or account passes to another person, the obligation to comply with the terms — and the acknowledgement of their content — passes automatically to the new user.

1. Scroll through all four blocks of text.
2. Tick **I have read and accept all terms stated above**. Until you do, the confirm button stays disabled.
3. Click **Accept & Continue**.

Clicking **Decline** signs you out and returns you to the sign-in panel. Acceptance is stored with your personal settings, so the dialog does not reappear on every visit.

4.5 Connecting to a database

After the disclaimer you see the application shell: the sidebar on the left with the **Connections** section already open, and an empty canvas on the right. There is no KPI strip and no date slider yet — both appear only once a chart has loaded data.

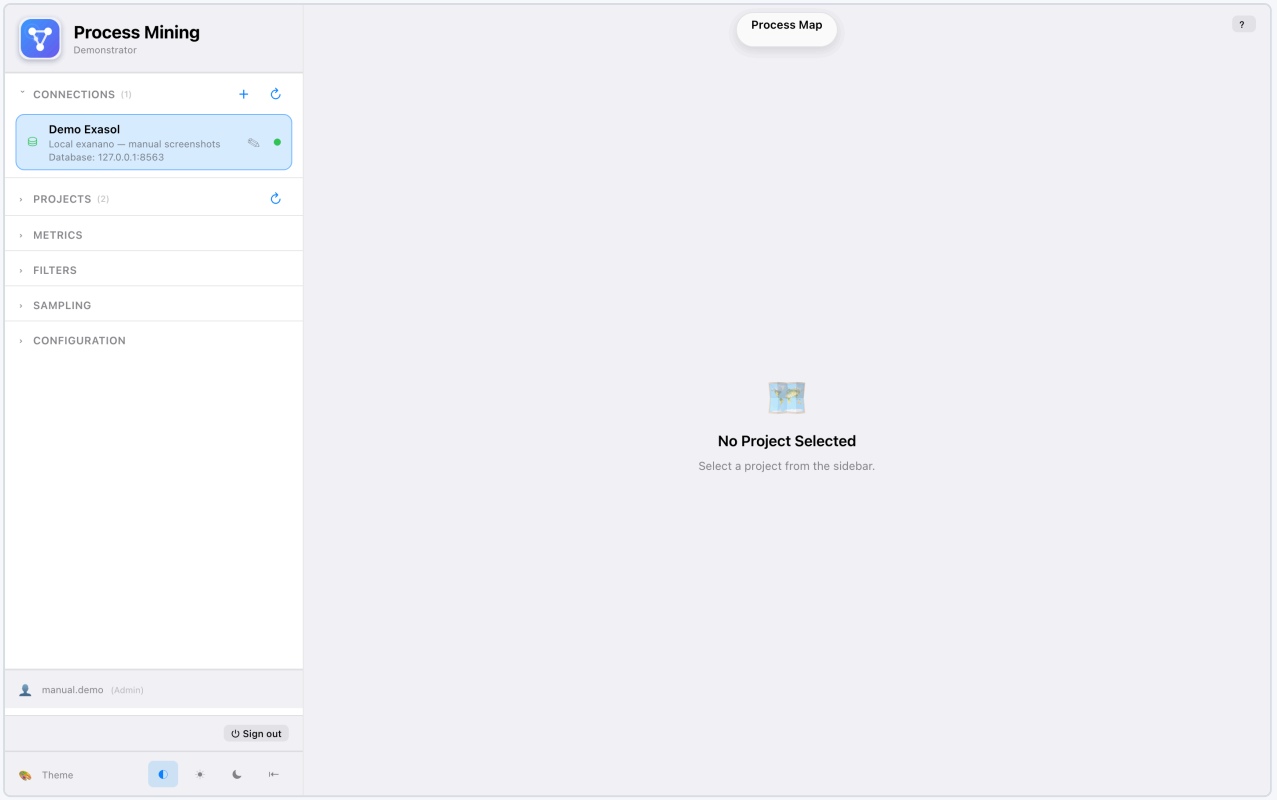


Figure 4.2 — The application shell immediately after sign-in: sidebar with the Connections section open, and an empty main area awaiting a project.

The Connections section lists only the connections assigned to you, as read-only cards sorted alphabetically by name. You cannot change a host, credentials or TLS settings here — that is the administrator's responsibility. Each card shows a database glyph (⛁), the connection **Name**, an optional comment line, **Database:** *host:port*, and — when a model server is attached — **LLM:** followed by its URL.

1. Click a connection card. The application opens your own Exasol session against it.
2. Watch the **status dots** on the right of the active card. The first dot is the database: green means "Connected", orange means "Not connected". The second dot appears only when the connection carries an LLM server: blue means "LLM reachable", orange means "LLM not reachable".
3. On success the sidebar closes Connections and opens the **Projects** section for you, with the project list already loading.

Clicking the connected card again disconnects. Clicking a *different* card while connected disconnects first, then connects to the new one. Only one connection is active at a time, and disconnecting clears the whole session state — project, map, KPIs and filters all reset. It does *not* sign you out of the application.

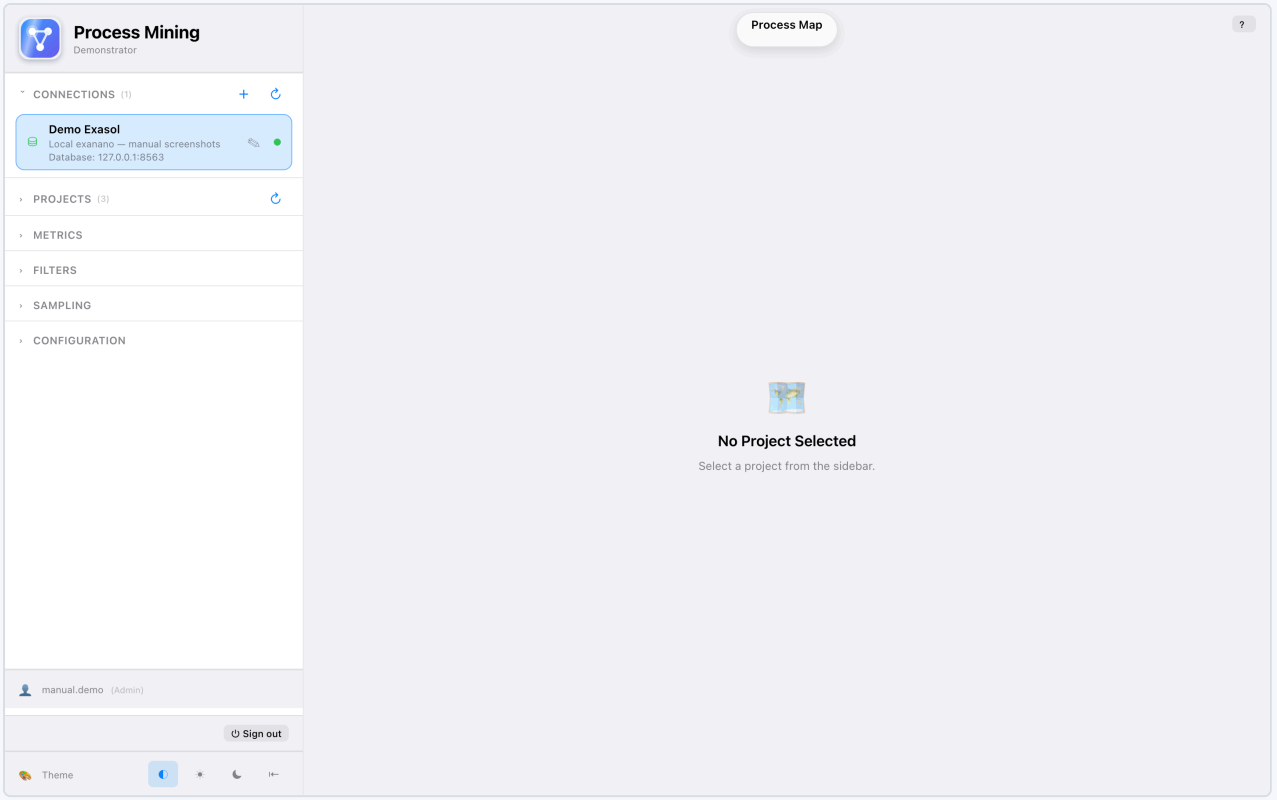


Figure 4.3 — The Connections section. The ↻ button refreshes the list; the + button appears only for users who may manage connections.

> **Tip** Use the ↻ button in the Connections header (tooltip "Refresh connections") after an administrator has just granted you a new connection — the list does not poll on its own.

If the list is empty you see one of two messages. Plain users see "*No connections assigned to you. Ask an administrator to grant access.*"; users who may manage connections see "*No connections yet. Use + above to create one and assign users.*"

A failed connection raises an alert titled **Connection Failed** with the message and two buttons, **Retry** and **Cancel**. The message is deliberately generic — the detailed driver error goes to the server log, not to your screen.

Message	What it usually means
Authentication failed — check your username and password.	The stored database credentials are wrong or have expired.
Connection timed out. Verify the host address, port, and network connectivity.	No answer within the 15-second connect timeout.
Connection refused. No server is accepting connections at this host and port.	Wrong port, or the database is not running.
Host not found. Check the hostname spelling and your DNS / network connectivity.	The host name does not resolve.
TLS certificate error. Try 'Skip verification' or configure a fingerprint in the server's security settings.	The certificate is not trusted. For Exasol 7.x and the Docker image the connection's minimum RSA key size must be set to 1024 bits (legacy).
This connection is not available to you.	The connection is no longer assigned to your account (HTTP 403).
This connection changed during sign-in. Please reconnect.	An administrator edited or deleted the connection while your session was opening.

4.6 If you have no data yet: provision a schema and generate demo data

> Requires: **Power user**, **Developer** or **Administrator**

Regular users skip this section — ask an administrator to assign you a connection instead. With the power role (or the developer grant, or administrator rights) a **+** button appears in the Connections header, and you can build a working environment from scratch without leaving the application: create a connection, create the schema and tables, and load a ready-made dataset.

Creating a connection

1. Click **+** in the Connections header (tooltip "New connection"). The **New Connection** sheet opens on the **Database / LLM Details** tab.
2. Fill in the fields below. Only **Name** is mandatory to save; the database fields are needed before anything can be tested or provisioned.
3. Click **Test** in the footer. The result appears beside the button: green when everything is fine, orange when one of two components failed, red when all failed. The text reads "Database OK" or "Database: *error*", joined by "·" with "LLM OK (N models)" or "LLM: *error*" when an LLM URL is set.
4. Tick the users who should see this connection under **Assigned users**, then click **Save**.

Field	Default	Notes
Name	—	Required. The display name in everyone's Connections list. Saving without it shows "Name is required."
Comment	—	Optional description line shown on the card.
Host / Port	— / **8563**	The Exasol server. 8563 is the Exasol default.
Username	—	The database login.
Schema	—	The schema holding the process-mining tables. Optional for connecting, required for provisioning, demo data and project listings.
Password	—	Write-only. When a password is already stored the placeholder reads "•••••• (unchanged)". Leave it untouched to keep the stored value; type to replace it; clearing it deliberately to empty **erases** it.
Use TLS	off	When ticked, three more fields appear.
Certificate mode	Verify (system trust store)	Also "Pin fingerprint" and "Accept any (insecure)". The last disables both certificate and host-name verification — lab and self-signed setups only.
Fingerprint (SHA-256)	—	Used with "Pin fingerprint". Must be pure hexadecimal; a blank or non-hex value in that mode refuses the connection rather than silently skipping verification.
Minimum RSA key size	2048	Stored and carried through every operation for compatibility with the macOS edition's connection records, but not currently enforced by the web backend.
LLM Server URL / Model / API key	—	Optional OpenAI-compatible endpoint; enables the AI supported Documentation view. The API key is write-only like the password.

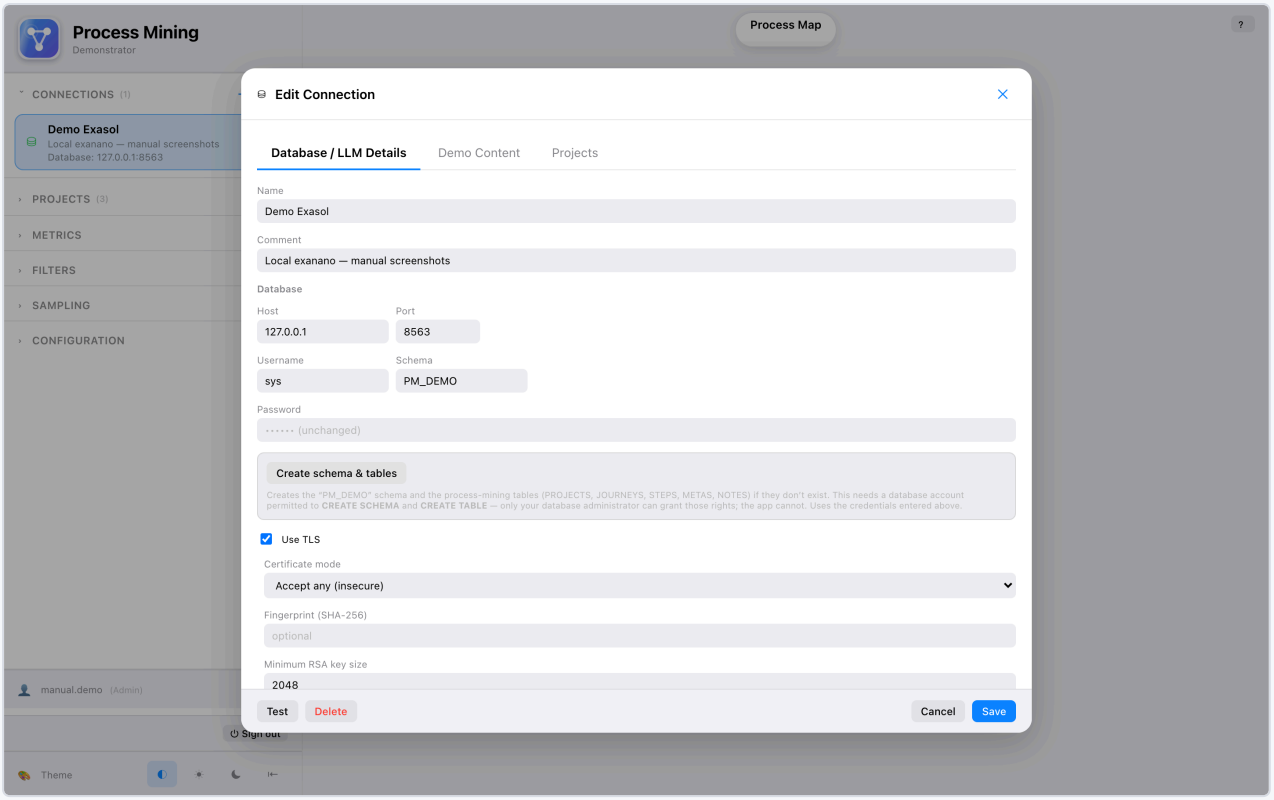


Figure 4.4 — The connection editor. The footer Test button checks the values currently typed into the sheet, not the stored ones.

Warning **Test**, **Create schema & tables** and demo generation all use the password *typed into the sheet*, not the stored secret. When you reopen a saved connection the password field is blank, so those operations run with an empty password and typically report an authentication failure even though the saved connection itself works. Retype the password before using them.

Warning Saving an edit disconnects every user currently connected through that connection — they must re-connect against the new definition. Un-assigning a user drops their live session immediately, and deleting a connection drops all sessions on it first.

Creating the schema and tables

1. On the **Database / LLM Details** tab, enter the target **Schema** name and make sure the database credentials are complete.
2. Click **Create schema & tables**. The button is disabled while the Schema field is blank.
3. On success a green line reads **Created:** followed by what was made — the schema plus PROJECTS, JOURNEYS, STEPS, METAS and NOTES. On failure the error appears in red, or the fallback "Could not create the schema."

Provisioning is idempotent: every statement uses `IF NOT EXISTS`, so re-running never disturbs existing data. If it fails part-way, the result still lists everything that was created before the failure.

Requires on the database: **CREATE SCHEMA** and **CREATE TABLE**

> **Warning** Only your database administrator can grant those rights. The application cannot authorise you, and no amount of application-level permission substitutes for them.

Generating a demo dataset

The **Demo Content** tab loads a complete, realistic dataset into the connection's schema, creating the schema and tables first if they are missing. Four datasets are offered.

Group	Dataset	Project ID	Meta attributes 1 / 2 / 3
Retail	📚 Online Bookstore	`BOOKSTORE`	Payment Method / Customer Segment / Order Value
Finance/Insurance	💶 Online Credit Application	`CREDIT`	Applied Credit Sum / Income Class / Channel
Transportation	✈️ Flight Booking & Management	`FLIGHTS`	Journey Type / Airline / Payment Method
Transportation	🛬 Airport Passenger Flow	`APF`	Terminal / — / —

Online Bookstore generates an order lifecycle — login, browse, basket, checkout, payment, fulfilment, delivery — with roughly 5 % of orders taking a returns flow and a deliberately unreliable bank-transfer path that fails about 35 % of the time per attempt and retries. Its 24 steps are grouped into Customer Interaction, Purchase Process, Payment, Fulfilment and Returns, with scores from −10 (Payment Failed, Return Initiated) to +15 (Delivered) — which means the Process Goodness KPI works out of the box.

Online Credit Application models bank and affiliate intake, an application check with a rework loop, credit assessment, and a score- and sum-driven decision with agent-review loops, ending in payment or rejection. **Flight Booking & Management** models a Star Alliance-style booking: 80 % new bookings (with a modify loop in search, and 50 % interline itineraries that query a partner airline's system) and 20 % manage-booking journeys.

Airport Passenger Flow models a departing passenger's walk through the terminal — departure hall, baggage drop / check-in, security, then the airside amenities (duty free, lounge, dining) in any order before boarding — where Passport Control switches the airside and boarding steps from their domestic (Dom) to their international (Int) variant, 2 % leave the hall without travelling, and 5 % of boardings are denied. It is a *streamed* dataset, so it scales far beyond the others. Unlike the other three, its passenger volume is **seasonal across the year**: the summer holidays (July and August) and Christmas (December) are peak months, September is high (travel resumes after the summer), May is a low month, and the remaining months are normal — so the **journeys-over-time** chart shows a realistic seasonal pattern rather than a flat line. Within each month the day and time of day are spread uniformly across the departure hall's active window (08:00–21:59).

Event times are spread across a **rolling two-year window ending on the day you generate the dataset**, so the newest events always reach “today” and recent views — the Dashboard’s ingest sparklines and the journeys-over-time chart — are never empty. For Bookstore, Credit and Flights the journey start is uniform across that window; for Airport Passenger Flow its month is drawn in proportion to the seasonal weights above (day and time of day still uniform). A journey’s own duration is added on top of its start, so a few of the most recent journeys finish just after today.

1. Open the **Demo Content** tab of the connection editor.

2. Check the **Schema** field on the dataset panel you want. It is the same value as on the Details tab — editing it in any demo panel changes the connection's schema.
3. Set **Journeys**. The default is **500**; the minimum is 1 and the field accepts up to 20,000.
4. Click **Generate**. The button reads "Generating…" with a spinner while it runs.
5. A green line confirms the result — *Created N journeys for "<title>" in <SCHEMA>.JOURNEYS* — or a red one reports the error.

> **Note** Generation replaces only that dataset's own project: its PROJECTS row, its METAS row and **all** of its JOURNEYS rows, including any sample sets. Other projects in the schema are untouched. STEPS rows are only *added* where missing, so colours, shapes and scores you have customised in the Step Editor survive a regeneration.

> **Note** The in-memory datasets are hard-capped at 20,000 journeys on the server and a larger request is silently clamped to exactly that (the streamed Airport Passenger Flow allows far more). Generation runs with autocommit *off*: rows are inserted in batches and committed in transactions of about 25,000 rows each, so a large load is durable in bounded chunks rather than one giant transaction — and a load that fails part-way is rolled back, leaving the project untouched.

Demo journeys do not use plain order numbers as case keys. Each `EVENT_ID` is the MD5 hash of a synthetic reference built from the dataset prefix and a 1-based, zero-padded six-digit sequence number:

```
Online Bookstore (BOOKSTORE):          EVENT_ID = md5("ORD-000001"), md5("ORD-000002"), …
Online Credit Application (CREDIT):    EVENT_ID = md5("CRA-000001"), …
Flight Booking & Management (FLIGHTS): EVENT_ID = md5("FLT-000001"), …
Airport Passenger Flow (APF):          EVENT_ID = md5("APF-0000001"), …   (7-digit sequence)

# reproduce a specific ID from a shell:
printf 'ORD-%06d' 1  | md5      # macOS  → the stored 32-char hex EVENT_ID
printf 'FLT-%06d' 42 | md5sum   # Linux
```

> **Tip** You rarely need to hash anything by hand. In the Individual Journey view you can type the friendly reference — `ORD-000001` — straight into the Event ID field and the application hashes it for you.

4.7 Selecting a project

With a connection live, the **Projects** section lists what the schema contains. Each project card shows a 📈 icon, the title and an optional description; the selected project carries a ✓.

1. Open the **Projects** section if it is not already open, or press ↻ (tooltip "Reload projects") to re-read the list. The button is disabled while no database is connected.
2. Click a project card.

Selecting a project collapses every sidebar section and does a good deal of work at once: it switches the main area to **A-Chart**, resets all filters, A/B state, statistics and AI results; loads the project's bootstrap data

(steps, meta titles and values, date range, filter bounds and sample counts); applies your **Default date window**; and restores your per-user, per-project state — filter presets, target norms, happy paths and the LLM prompt template.

> **Note** The default date window is **30 days**. The map opens on the window ending at the project's latest event date and reaching back that many days, clamped to the project's earliest date. Set it to **0** in **Configuration → Default date window** to load the full range instead. The change applies the next time a project is opened — it does not move the slider on the current map.

If the list stays empty you see "*No projects found.*"; before connecting it reads "*Connect to a database to load projects.*" A failure raises the alert **Failed to Load Projects** (or **Failed to Load Project** when the selection itself fails) with **Retry** and **Cancel**.

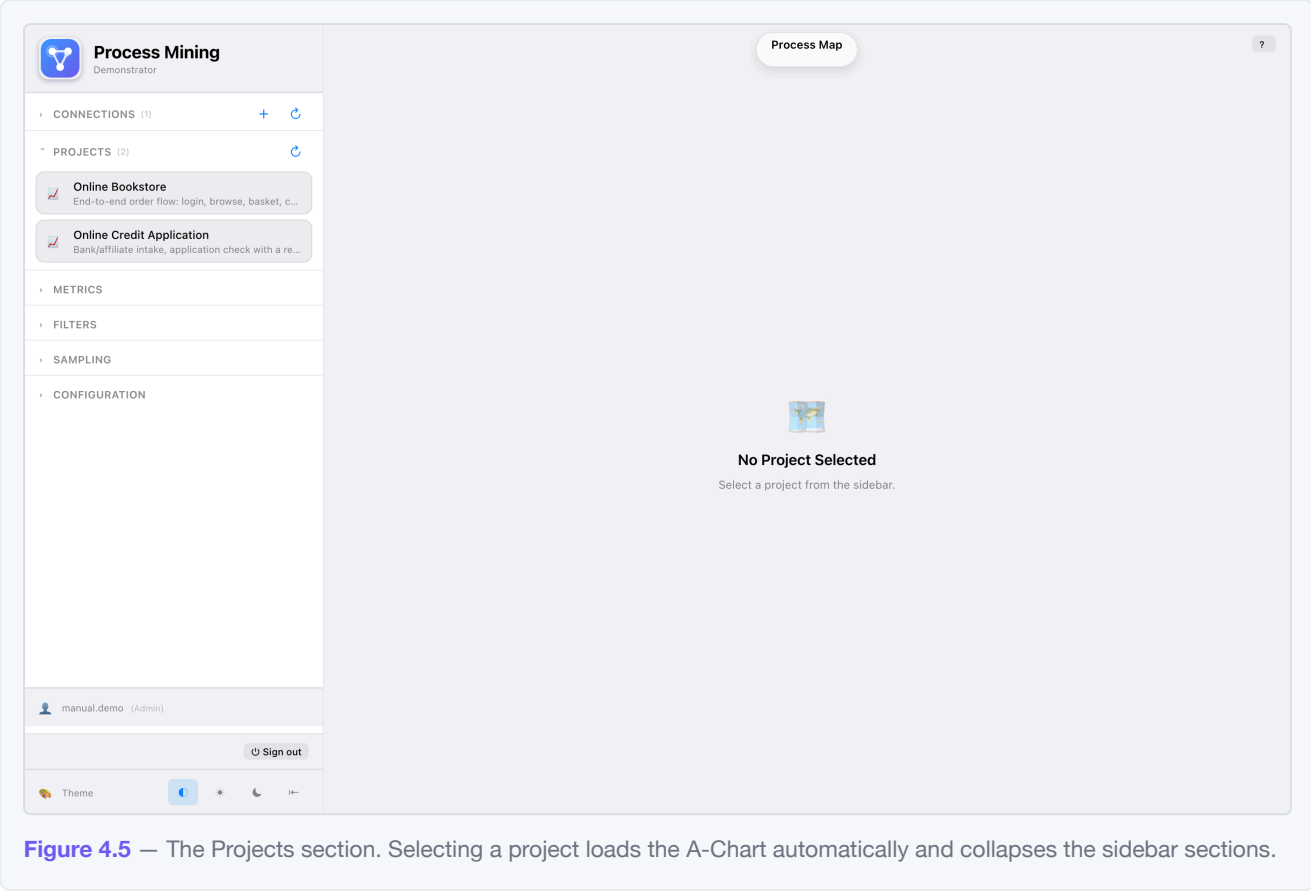


Figure 4.5 — The Projects section. Selecting a project loads the A-Chart automatically and collapses the sidebar sections.

4.8 Reading your first process map

The A-Chart is an aggregated **directly-follows graph**: every node is a distinct process step, and every arrow is a transition observed between two chronologically consecutive steps within the same journey. The map shows all journeys that match the current filters.

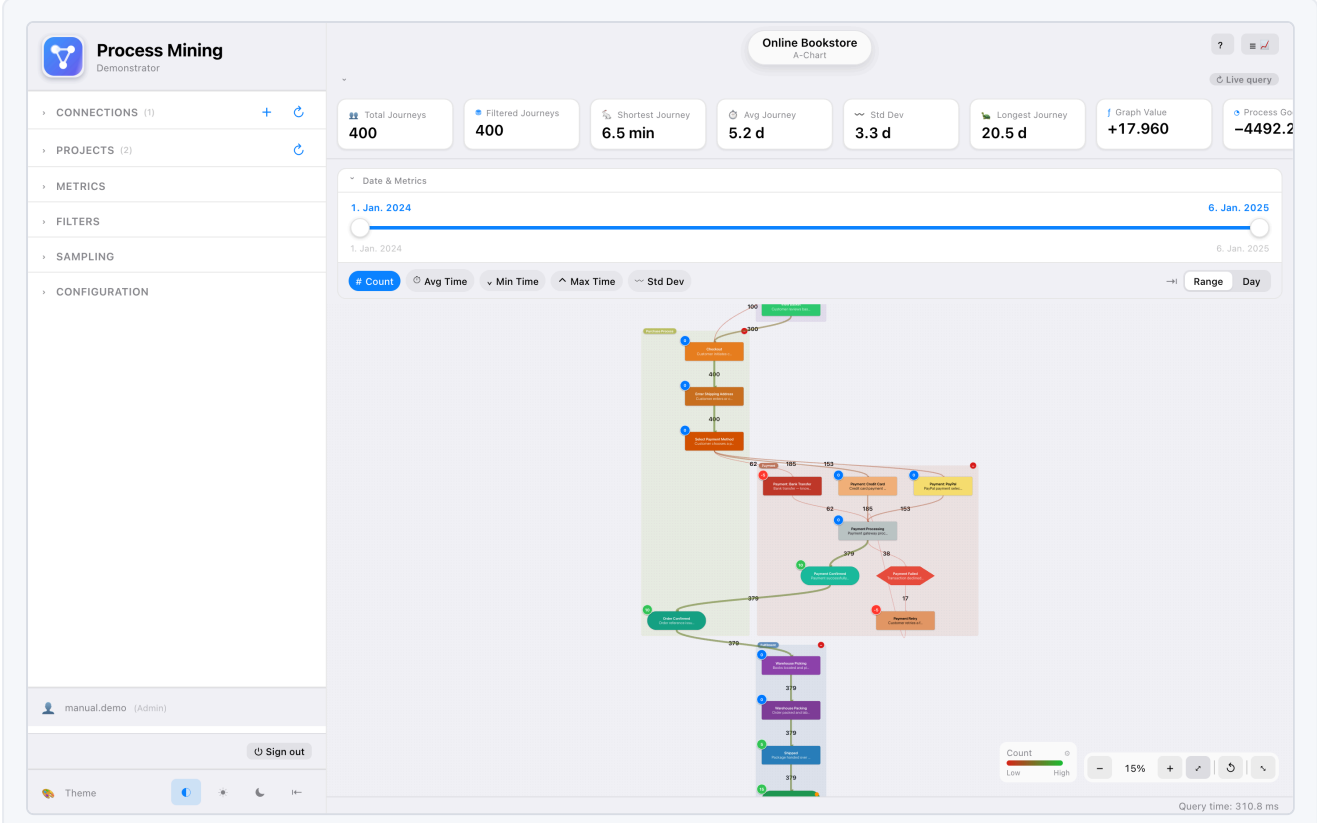


Figure 4.6 — The A-Chart. Arrow thickness encodes the selected transition metric relative to the highest value on the map.

Arrows

Thickness and opacity reflect the selected transition metric *relative to the largest value on the map*. The label carries the value — a plain number for Count, a readable duration (4m, 1.2h, 3.5d) for the time-based metrics.

Green arrow above a node

An entry point: a step with no incoming transitions.

Orange arrow below a node

An exit point: a step with no outgoing transitions.

Score badge (top-left of a node)

The step's SCORE from the STEPS table — green for positive, red for negative, blue for zero.

Orange dot

Marks an end-of-process step.

Yellow ✎ badge

The node or edge carries at least one note.

Dashed box with a name pill

A step group: all steps sharing a `BELONGS_TO` value. The **+/−** badge collapses or expands it; a collapsed group sums connection counts and weight-averages the times.

A small pill in the chart header tells you how the transitions were computed: **↻ Live query** (computed from the event log on each load), ⚡ **Pre-materialized** (read from a prebuilt table), or ⚠ **Live (not built)** (the fast

path is enabled but the table is not ready, so the live query is running meanwhile). The chart footer shows the server-side query time, for example "123 ms". Chapter 12 covers the pre-materialized path.

Moving around

1. Scroll or pinch to zoom; drag the background to pan.
2. **Double-click** the background to fit the whole graph to the window. The ⤢ button does the same, and the ± buttons zoom in precise steps.
3. Drag a node to reposition it. Positions snap to a 20 pt grid and are saved **per project and per chart view**. Dragging a group box moves all its members together.
4. Press ↺ to discard your custom positions and return to the automatic layout.

Click any node to open its small action card: **Require in journeys** (adds the step to Include Steps and re-loads), **Exclude from journeys** (the mirror), **Show Notes (n)** — where *n* is how many notes it already holds — and **Show description**, which displays the step's full DESCRIPTION when it differs from the name.

4.9 A guided tour of the screen

The sidebar

The sidebar runs from a brand header at the top, through six accordion sections, to your identity footer and the theme bar. Only one section is open at a time — opening one collapses the others. On start-up, Connections is open.

Section	What it does	Visibility
Connections	Pick the database you work against. Header shows a count when there is at least one.	Everyone
Projects	Pick the project inside that database.	Everyone
Metrics	Choose the transition metric: **# Count** (default), ⏱ **Avg Time**, ½ **Median Time**, ⌄ **Min Time**, ⌃ **Max Time**, 〰 **Std Dev**.	Hidden in Individual Journey
Filters	Date, Include Steps, Exclude Steps, Num Steps, Journey Time, Journey Score and Meta 1–3, plus Reset, Save Preset… and Apply.	Everyone
Sampling	Create up to three shared sample sets and choose the data source for the A and B charts.	Power and Administrator only
Configuration	Step groups, font sizes, KPI order, the Step Editor, display toggles, default date window and the LLM prompt.	Everyone

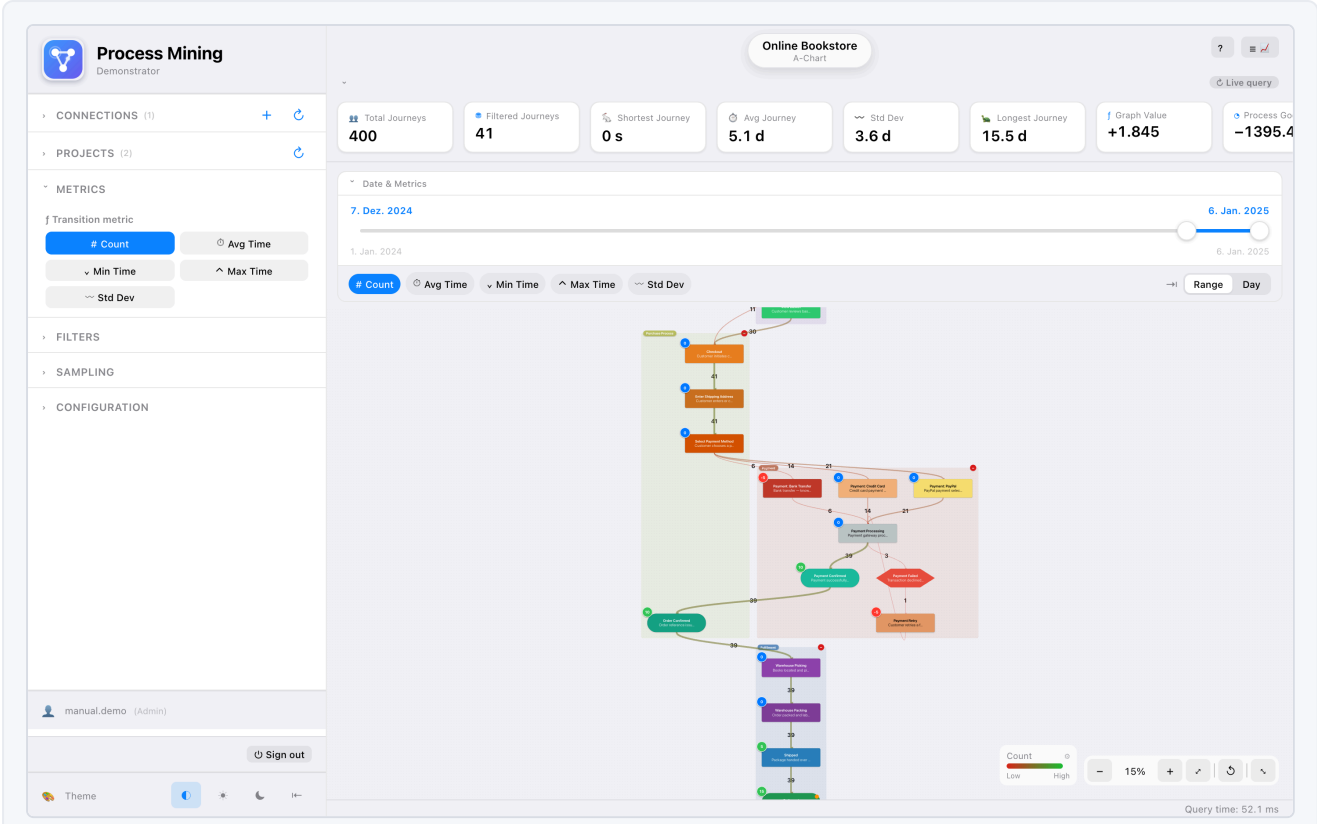


Figure 4.7 — The Metrics section. Each chart view remembers its own metric selection independently.

Below the sections, the **identity footer** shows 👤 followed by your display name and your role in parentheses — **Admin**, **Power user**, **Developer** or **User** — plus the buttons 🔒 **Two-factor**, 🔑 **Passkeys** (each shown only when available to you) and ⏻ **Sign out**. The footer is hidden entirely when sign-in is not required.

At the very bottom sits the theme bar: 🎨 **Theme** with three buttons — **System** (◐), **Light** (☀) and **Dark** (☾) — and a **Hide sidebar** button (⇤). When the sidebar is hidden, a **Show sidebar** button (▤) appears in the chart header.

The KPI strip

The KPI strip sits above the canvas and appears only once a chart has loaded data. It is hidden on launch, after disconnecting, and on any view that has not yet loaded. Tap the chevron handle to collapse it — the journey counts then appear inline in the handle bar.

Tile	Meaning
Total Journeys	Distinct journeys in the project with **no** filters applied. It does not change when you adjust filters.
Filtered Journeys	Distinct journeys matching all current filters — the population the map draws.
Shortest / Avg / Longest Journey	Fastest, mean and slowest journey duration in the filtered set, first event to last.
Std Dev	Standard deviation of journey durations. Low means most journeys take a similar time.
Graph Value	Sum of (step score × visit count) over every scored node on the map.
Process Goodness	Composite quality score for the whole process. Hidden when no step has a SCORE.
Process Similarity	A/B Comparison only — floats between the two panels.
Active Sample	Which data source the chart is reading.

All tiles are on by default. Reorder them by dragging, and switch individual tiles off, in **Configuration → KPIs**.

The ≡ view menu

The ≡ menu in the top-right corner of the main area switches between the ten views. A checkmark marks the active one, and the title capsule shows it on a second line so your context is always visible.

View	Purpose	Chapter
A-Chart	The primary process map.	5
B-Chart	An independent second map with its own filter set. It starts empty — apply filters and tap **Apply**, or use **Load** on its empty state.	5
A/B Comparison	Both maps side by side, with a shared-viewport valve and a Process Similarity badge.	7
Individual Journey	The full step sequence of one case, replayed by a moving dot.	8
AI supported Documentation	An LLM-written report over the current A-Chart transition table.	13
Statistics	Journey variants, duration histogram, step traffic and a transition heat map.	9
Conformance Check	Target norms per edge and a gap analysis. *Power and Administrator only.*	10
Happy Path	An ideal step sequence scored against reality. *Power and Administrator only.*	10
Notes	Every note on the project, with its own filter bar.	11
Simulation	Synthetic event logs from a calibrated Markov model. *Power and Administrator only.*	14

> **Note** Conformance Check, Happy Path and Simulation do not appear in the ≡ menu at all for Regular users. If you expect them and cannot see them, check the role shown in your sidebar footer.

The help panel

Press ⌘/ (or **Ctrl+/**) at any time to open or close the in-application help. The panel floats above the workspace: drag it by its title bar to move it, and resize it from the bottom-right corner. Its table of contents mirrors this manual, with an Administration sub-menu for administrators and an Integration console sub-menu for developers.

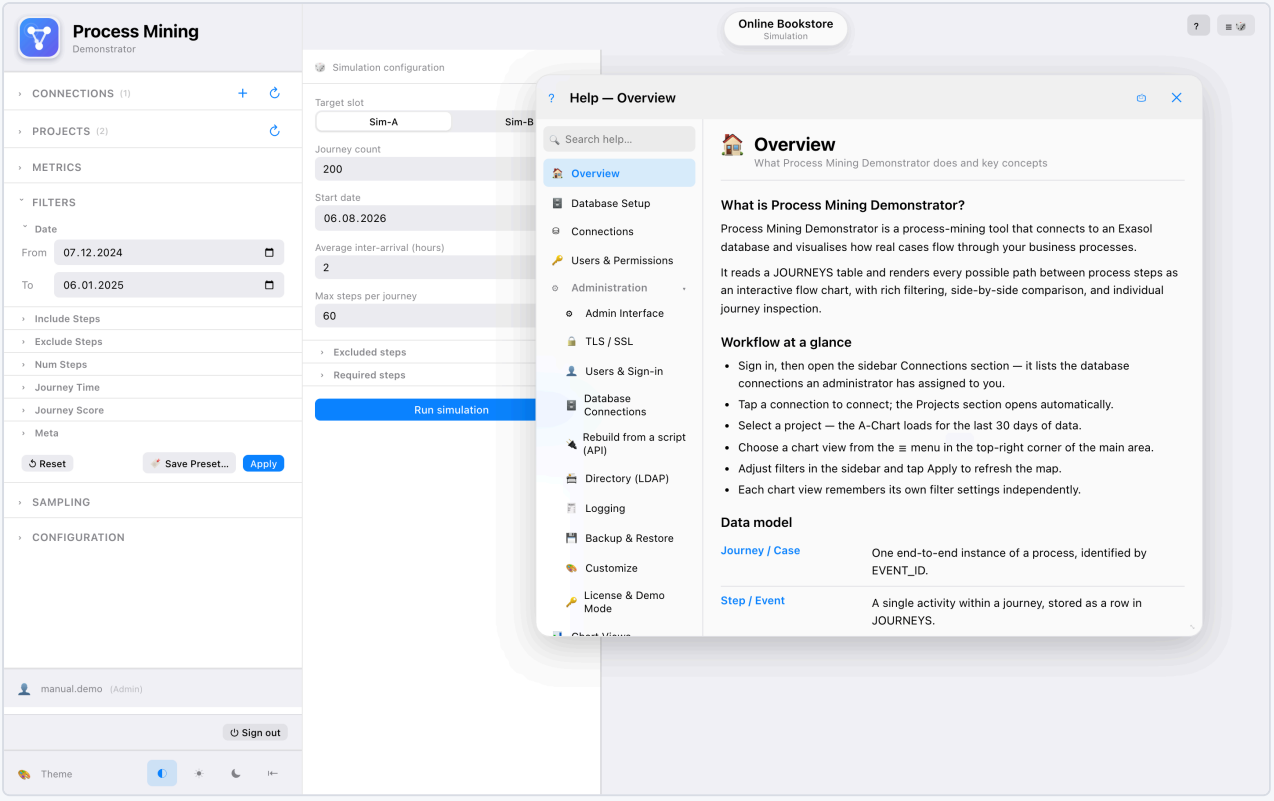


Figure 4.8 — The floating help panel, opened with ⌘/ or Ctrl+/.

4.10 Your first filter

Filters work at **journey level**: a journey either qualifies in full or not at all. Nothing is applied automatically — you set the filters, then press **Apply**.

1. Open the **Filters** section. The **Date** sub-section is expanded by default; all others start collapsed.
2. Adjust **From** and **To**, or drag the two thumbs of the date slider above the map. Both thumbs respond to the arrow keys once focused.
3. Expand **Include Steps** and select one or two steps. Any step already chosen for **Exclude Steps** is disabled here, and vice versa — a step can never be both.
4. Click **Apply**. The map reloads, and **Filtered Journeys** in the KPI strip updates while **Total Journeys** stays where it was.

5. Happy with the result? Click **Save Preset…**, name it, and click **Save**. Leave the name blank and it is auto-named "Preset N".
6. To start over, click **Reset** — it restores the project defaults and reapplies immediately.

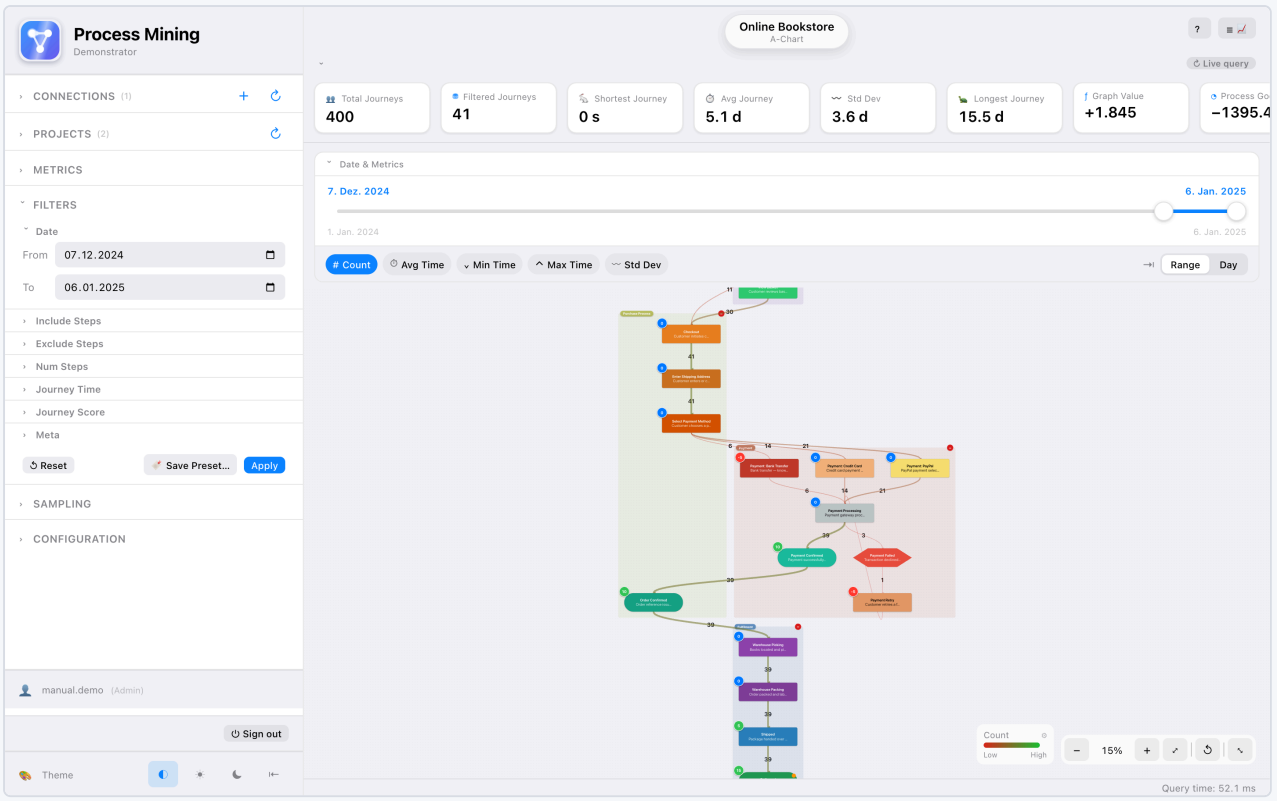


Figure 4.9 — The Filters section. Active sub-sections carry a badge with the current value and a ⊗ button that clears just that filter.

> **Note** A preset is a snapshot taken at the moment you save it — later filter changes do not update it. Presets are stored per user and per project, survive a change of browser, and are included in backups. Deleting one from the sidebar list happens immediately, with no confirmation dialog.

> **Tip** Each chart view keeps its own filter settings and its own metric selection, so exploring in the B-Chart never disturbs your A-Chart.

4.11 Ending the session

Click **Sign out** in the sidebar footer to end the session; this also releases your database connection. If your administrator has configured an idle timeout, the application signs you out on its own after that many idle minutes and the sign-in panel then reports "*You were signed out due to inactivity.*" Nothing is lost: your settings, presets, layouts and notes are all stored server-side and return with you, and when you sign back in the application **reconnects and reopens the project and view you were last in** (see §5.6.6).

4.12 Where to go next

Chapter 5 — The Process Map

Metrics and colour scales, groups, layout, node context actions and the colour-legend wizard.

Chapter 6 — Filters and Presets

Every filter dimension in detail, including meta attributes and the score range.

Chapter 9 — Statistics

Variants, duration histograms and the journeys-over-time chart.

Chapter 11 — Notes

Annotating nodes and edges, importance levels, threads and sharing.

Chapter 12 — Administration

Users and roles, TLS, directory sign-in, licensing, logging, backups — and pre-materialized transitions.

PART II

Using the Application

5 Signing In, Sessions and Security

> This chapter explains how you get into the Process Mining Demonstrator and how you stay in. It covers password sign-in on all three browser surfaces — the main application, the integration console and the admin interface — together with two-factor authentication using an authenticator app, passkeys (Touch ID, Windows Hello, security keys), how long a session lasts, what happens when it ends, and what the system does after repeated failed attempts. It closes with the full role capability matrix and a catalogue of every message you may see on a sign-in screen. Read the first three sections if you only need to sign in; read the rest if you administer accounts or are troubleshooting an access problem.

5.1 The three surfaces and who may sign in

The product presents three separate web surfaces. Each has its own session cookie and its own access rule. Signing in on one surface does **not** sign you in on the others: the cookies are audience-bound, so an application cookie is never accepted by the integration console or the admin panel, and vice versa.

Surface	Default ports (HTTP / HTTPS)	Session cookie	Who may sign in	Is sign-in ever optional?
Main application (GUI)	8080 / 8443	`pmw_session`	Any enabled user, in any role	Yes — controlled by **Require sign-in for the main application** (default on)
Integration console	8100 / 8463 (`PMW_INTEGRATION_PORT`, `PMW_INTEGRATION_HTTPS_PORT`)	`pmw_integration`	Developers and administrators only; power users are refused	No — always requires sign-in
Admin interface	8090 / 8453 (`PMW_ADMIN_PORT`, `PMW_ADMIN_HTTPS_PORT`)	`pmw_admin`	Administrators only	No

All session cookies are HttpOnly, use SameSite=Lax, are marked Secure when the surface is served over HTTPS, and are scoped to path `/`.

The main application and the integration console share the same sign-in panel; the only visual difference is that the integration console adds a second title line, **Integration**, beneath **Process Mining Demonstrator**. The admin sign-in is rendered by the admin server itself but deliberately mirrors the same panel, with the second title line **Administration**; its page title is "Administration — Sign in".

All three sign-in panels show a small **release label** beneath the caption (for example *2027.1.0 — Tongariro*). It is read from the top-level `VERSION` file — edit that one line to change the label everywhere, or override it with the `PMW_APP_VERSION` environment variable. It is display-only and never affects behaviour.

Note The first start of a new installation seeds a built-in administrator with the username **Administrator** and the password **Administrator** (both overridable with the environment variables `PMW_DEFAULT_ADMIN_USER` and `PMW_DEFAULT_ADMIN_PASSWORD`). Until that password is changed, the admin dashboard displays the banner "⚠ The **Administrator** account is still using its default password. Change it now in 👤 **Profile** (top right)." Changing the password clears the banner permanently.

Warning An administrator can switch the integration console off entirely (Admin → **Integration** tab → **Enable the integration console**). While it is off, browser navigations to that surface receive a 503 page headed **Console unavailable** with the text "The integration console has been turned off by the administrator."; programmatic calls receive the same text as JSON.

5.2 Signing in with a password (application and integration console)

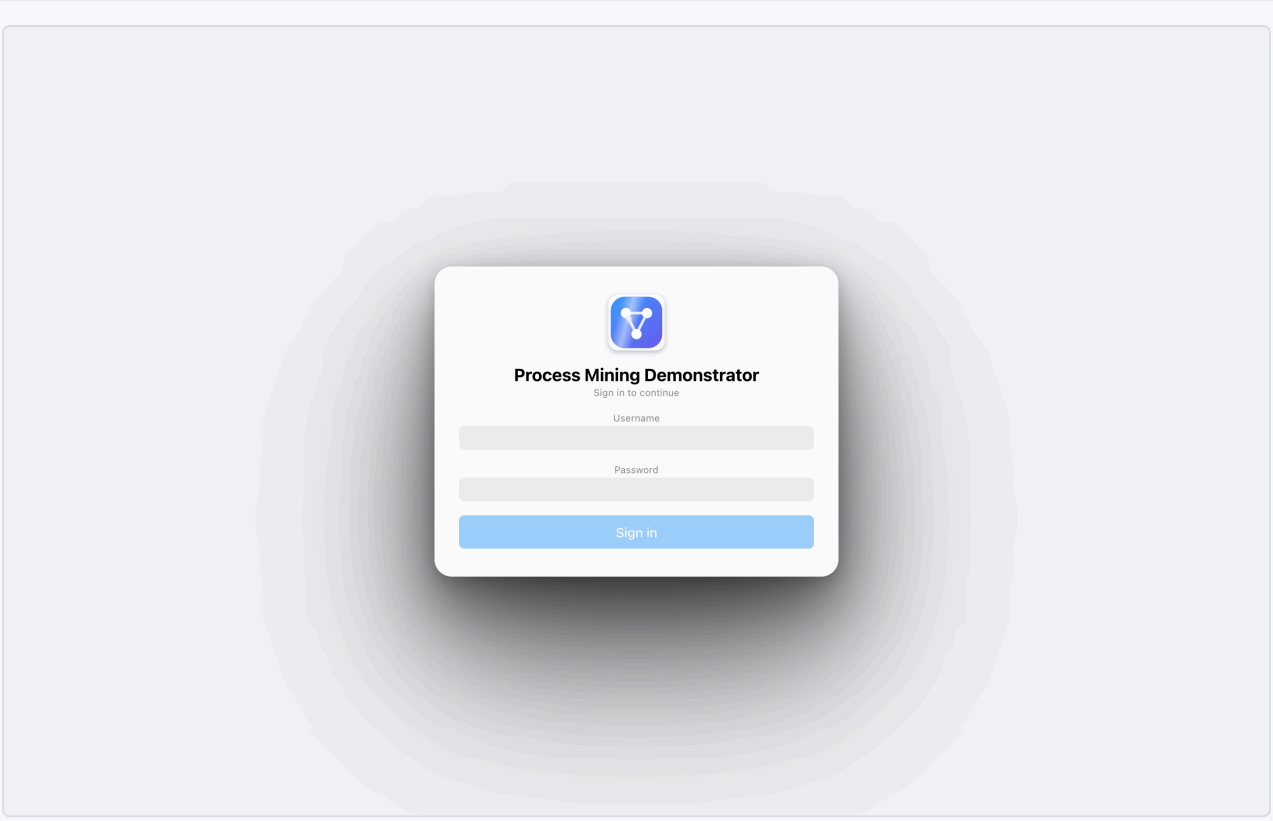


Figure 5.1 — The sign-in panel. The caption under the title changes with the step: "Sign in to continue", "Enter your authentication code" or "Set up two-factor to continue".

What is on the panel

Username

Autofocused when the panel opens. Required — the **Sign in** button stays disabled until you type something here.

Password

An empty password is accepted by the form and simply fails authentication.

Sign in

Primary button. Shows a spinner while the request is in flight.

🔑 **Sign with Passkey**

Shown only when passkeys are usable in this browser (see section 5.5). Tooltip: "Sign in with a passkey (Touch ID, Windows Hello, security key…)". Also disabled until a username is entered.

Three notices can appear above the form:

- An orange **inactivity notice** — "You were signed out due to inactivity." — after an automatic idle sign-out. It is suppressed while an error is showing.
- An orange **license banner** when no valid license is installed: "**Demo Mode** — remaining time: N min" or "**No License installed**". It refreshes every 30 seconds.
- A red **error box** carrying any sign-in error (section 5.10).

If a directory (LDAP) server is configured, a small LED and a label appear below the form: green **Directory server available** ("The user directory server responded to a connection test.") or red **Directory server unavailable** ("The user directory server did not respond to a connection test."). The probe result is cached for about 20 seconds. Nothing is shown when no directory is configured. An administrator can also hide this indicator entirely — even when a directory is set up — with the **Show the Authentication Server availability indicator on the login panels** switch on the Directory tab; when it is off, neither the app nor the admin sign-in panel shows the LED.

> **Tip** The panel background is customisable by an administrator (Admin → **Customize**): the theme default, a solid colour, or an uploaded image — over an image the panel becomes 50 % translucent. The same background applies to all three sign-in pages.

Signing in

1. Open the application in your browser. The sign-in panel appears.
2. Type your **Username**.
3. Type your **Password**.
4. Press **Sign in** (or Enter).
5. If your account uses two-factor authentication, the panel switches to the code step — continue with section 5.4. If two-factor is required but not yet configured, the panel switches to enrolment instead.
6. On the first successful entry in the main application you are shown the **Legal Disclaimer** gate. Read it, tick **I have read and accept all terms stated above** — **Accept & Continue** stays disabled until you do — and press **Accept & Continue**.

> **Warning** Pressing **Decline** on the Legal Disclaimer signs you out and returns you to the sign-in panel. Acceptance is stored in your personal settings, so the gate appears once per user, not once per session.

When a sign-in fails, the message appears in the red box, the password field is cleared and the username is kept, so you can retype the password without retyping your name.

The integration console role gate

> Requires: **Developer** or **Administrator**

On the integration console the role check runs *before* any second factor. A correct password from an account that is neither a developer nor an administrator is refused with "You need the Developer role to use the integration console. Ask an administrator to grant it." — no code step is offered. Power users are refused in exactly the same way; the power role grants nothing here.

If the Developer role is revoked while you are working, the console does not wait for your session to expire: your next request drops you to an **Access denied** screen reading "The integration console is available to developers and administrators only. Ask an administrator to grant you the Developer role.", with a ⏻ **Sign out** button.

Directory (LDAP) accounts

When a directory server is configured and enabled, the application and integration sign-in also accept directory accounts. Local accounts are always checked **first**, so a local break-glass account can never be taken over by a directory identity that happens to share its name.

Directory users are provisioned on their first successful sign-in as ordinary enabled local rows marked `ldap`. Administrator rights are **never** granted from the directory — a local administrator must promote the account. Directory users have no local password, so the failed-sign-in lockout counter does not apply to them, and the admin Users tab hides **Reset password** on their rows.

5.3 Signing in to the admin interface

> Requires: **Administrator**

1. Open the admin interface (by default port 8090, or 8453 over HTTPS). Visiting `/` without a session redirects you to `/login`; visiting `/login` with a valid session redirects you to the dashboard.
2. Enter your **Username** and **Password** and press **Sign in**.
3. Complete the authentication code step or the forced enrolment step if your account requires them (sections 5.4.3 and 5.4.4).
4. You arrive at the admin dashboard.

To leave, press **Log out** in the top-right corner, next to 👤 **Profile**. You are returned to `/login`.

> **Note** Bad credentials and *valid credentials belonging to a non-administrator* produce the same message here: "Invalid credentials, or the account is not an administrator." This is deliberate — the page never reveals whether a username exists or which of the two conditions failed.

Directory accounts can sign in to the admin panel only when **Also allow directory sign-in to this admin interface** is enabled *and* the account has been locally promoted to administrator. The 🔑 **Sign with Passkey** button appears on this page only when the browser supports WebAuthn and you reached the panel by a usable hostname; it is hidden entirely when you reach the panel by IP address.

Adding `?inactivity=1` to the login URL — which is what the idle watcher does — shows the same orange "You were signed out due to inactivity." notice.

5.4 Two-factor authentication (TOTP)

5.4.1 How it works

Two-factor authentication adds a six-digit time-based one-time code on top of your password. It never replaces the password. Two independent facts govern it for each account:

Allowed
: An administrator has ticked the per-user **2FA** checkbox in the Users tab (or the all-users master checkbox). This means two-factor *applies* to the account.

Enrolled
: You have scanned the QR code and confirmed a code, so a secret is stored for your account (encrypted at rest).

The login gate is *allowed **and** enrolled*. The consequences are worth understanding:

Allowed	Enrolled	What happens at your next sign-in
No	No	Password only.
Yes	No	Enrolment is **forced**. The password step succeeds but no session is issued until you finish enrolling.
Yes	Yes	Password, then a six-digit code or a recovery code. A passkey sign-in skips the code step entirely.
No	Yes	Password only. The secret is kept, not deleted — re-ticking **2FA** re-arms the same authenticator without re-enrolment.

> **Warning** Allowing 2FA for a user is the same as making it **mandatory** for that user. They cannot decline: their next sign-in on any surface stops after the password and forces enrolment.

One secret protects all three surfaces. Codes are six digits on 30-second steps with one step of clock-skew tolerance either side; spaces in a typed code are ignored. Your authenticator app labels the entry with the issuer **Process Mining Demonstrator** (overridable with `PMW_MFA_ISSUER`). Any standard authenticator works — Google Authenticator, 1Password, Authy and equivalents.

5.4.2 Setting up two-factor yourself

> Requires: two-factor **allowed** for your account by an administrator

In the main application and the integration console, the sidebar footer shows a 🔒 **Two-factor** button (tooltip "Manage two-factor authentication"). It is visible only when an administrator has allowed 2FA for your account.

1. Press 🔒 **Two-factor** in the sidebar footer. The **Two-factor authentication** sheet opens.

2. Press **Set up authenticator**. A QR code appears with the instruction "Scan this with your authenticator app, then enter the 6-digit code it shows."
3. Scan the QR code with your authenticator app. If your device cannot scan, use the line "Can't scan? Enter this key manually:" and type the key shown.
4. Type the six-digit code your app displays into the field marked with the placeholder **6-digit code** and press **Confirm**. A wrong code gives "That code didn't match. Try again." — the QR code and secret stay valid, so you do not need to rescan.
5. The recovery block appears, headed **Save your recovery codes**. Press **Copy codes** or write them down, then press **Done**.

On the admin interface the same functions live in the **Profile** overlay, section **Two-factor authentication**, with the banner states "Two-factor is on. N recovery codes left.", "Two-factor is off for your account." and "Two-factor is not enabled for your account — enable it in the Users tab first." Its buttons are **Set up authenticator**, **Regenerate recovery codes** and **Turn off**, and it confirms with the toasts "Two-factor enabled", "Recovery codes regenerated" and "Two-factor turned off".

5.4.3 Signing in with a code

1. Enter your username and password as usual and press **Sign in**.
2. The panel switches to the caption "Enter your authentication code" with the field **Authentication code** (placeholder "6-digit code or a recovery code").
3. Type the current code from your authenticator app, or one of your recovery codes, and press **Verify**.

Back returns you to the username and password step and clears both the code and the password. On the admin interface the code step is a plain form with a **Verify** button and no Back button — reload `/login` to start again.

> **Note** You have **5 minutes** after the password step to enter the code. The state is carried by a short-lived signed cookie, not a session — no session exists until the code is accepted. After 5 minutes you see "Your sign-in session expired. Start again." (the admin wording is "Your sign-in session expired. Please start again.") and must re-enter your password.

> **Tip** Wrong codes are throttled per source IP but **never** count towards the account lockout. A mistyped or expired code can never disable your account.

5.4.4 Mandatory enrolment at sign-in

If an administrator has allowed 2FA for your account and you have not yet enrolled, the sign-in panel switches after the password step to the caption **Set up two-factor to continue**.

1. Read the text "Two-factor is required for your account. Scan this with an authenticator app (Google Authenticator, 1Password…), then enter the 6-digit code." While the QR code is being fetched, a spinner shows **Preparing…**
2. Scan the QR code, or use the "Can't scan? Key:" line to enter the secret manually.

3. Type the code into the **6-digit code** field (placeholder "123456") and press **Confirm & sign in**. On the admin interface the button is the same and success leads straight to the recovery-codes page.
4. Your session is issued, but the panel stays up for one more screen headed **Two-factor is on — save your recovery codes**. Press **Copy codes**, store them, then press **Continue** to enter the application. On the admin interface, follow the **Continue to the admin panel** link.

Back abandons enrolment and no session is issued. Nothing is stored on the server until a code confirms the authenticator, so an abandoned enrolment leaves no half-configured account behind. The QR code and secret are valid for **10 minutes**; after that you see "Setup expired. Start again."

5.4.5 Recovery codes

Each set contains **10** codes in the form `a1b2c3d4-e5f6g7h8` — lower case, with look-alike characters avoided. Each works exactly once. They are stored only as salted hashes, so nobody, including an administrator, can read them back.

You may type a recovery code anywhere a TOTP code is accepted: at the sign-in code step and in the turn-off confirmation. Dashes, spaces and letter case are ignored.

> **Warning** Recovery codes are displayed exactly once, at setup or regeneration. Generating a new set with **Regenerate recovery codes** replaces the previous set entirely — every old code stops working immediately.

5.4.6 Turning two-factor off

1. Open the **Two-factor authentication** sheet from the sidebar footer. The status row reads "✅ Two-factor is on. N recovery code(s) left."
2. Press **Turn off**. You are asked: "Enter your current code (or a recovery code) to turn two-factor off."
3. Type a current code into the field (placeholder "Current code") and press **Confirm off**.

The re-authentication is deliberate: it stops someone who has hijacked an open session from silently stripping the second factor. A wrong code gives "Enter your current authentication code to turn off two-factor." Turning 2FA off deletes both the secret and all remaining recovery codes. On the admin interface the equivalent control prompts "Enter your current authentication code (or a recovery code) to turn off two-factor:".

5.4.7 If you lose your authenticator

There are two routes back in:

1. Sign in with your password and enter one of your **recovery codes** at the code step.
2. If you have no recovery codes left, ask an administrator to untick the **2FA** checkbox on your row in the Users tab. Your password alone then works again and you can re-enrol from scratch.

> **Note** There is no administrator function that resets your two-factor while preserving enrolment. Unticking the permission relaxes the requirement but leaves the old secret in place; if the administrator re-ticks it before you re-enrol, your lost authenticator is re-armed and you are back where you started. Turn 2FA off from the sheet first (which deletes the secret), then enrol again.

5.5 Passkeys (WebAuthn)

5.5.1 What a passkey is here

A passkey lets you sign in with Touch ID, Windows Hello or a hardware security key instead of typing a password. It is an *alternative* to the password, never a replacement: your password always remains a fallback.

> Requires: passkeys **allowed** for your account by an administrator

An administrator controls this with a per-user **Passkey** checkbox in the Users tab and an all-users master checkbox. Both local and directory accounts may be allowed. Turning the permission off blocks passkey sign-in immediately — the check runs at every step — and hides the management controls.

A passkey registered on a host works on **all surfaces of that host**: the application, the integration console and the admin panel share the same relying-party identifier, which is the hostname without the port.

> **Warning** A passkey sign-in satisfies two-factor by itself. Even for an account enrolled in TOTP, no code step is shown after a successful passkey ceremony — the passkey counts as strong authentication on its own. If your organisation requires a TOTP code on every sign-in, do not allow passkeys.

5.5.2 Prerequisites

Requirement	Detail
Browser support	A WebAuthn-capable browser. Otherwise: "This browser does not support passkeys."
Secure context	HTTPS, or `localhost`.
Real hostname	WebAuthn rejects bare IP addresses and single-label hosts. Reached by IP, the buttons are hidden or disabled and you see: "Passkeys need a hostname, not an IP address. Reach the server by its name over HTTPS — e.g. its ".local" name — instead of its IP, then try again."
Username	Sign-in is username-first: you must type your username before pressing 🔑 **Sign with Passkey**. Usernameless, discoverable-credential sign-in is not used.

For deployments where the application and admin panel live on different subdomains, three environment variables tune the relying party:

Variable	Purpose
`PMW_PASSKEY_RP_ID`	Set to the shared parent domain when the surfaces are split across subdomains.
`PMW_PASSKEY_ORIGINS`	Comma-separated allowlist of expected origins.
`PMW_PASSKEY_RP_NAME`	Display name shown by the browser. Default: "Process Mining Demonstrator".

5.5.3 Enrolling a passkey

1. Sign in with your password.

2. Press 🔑 **Passkeys** in the sidebar footer (tooltip "Manage passkeys for this account"). The **Passkeys** sheet opens; with none registered it reads "No passkeys registered yet."
3. Type a name in the field marked **Name (e.g. MacBook Touch ID)**. The name is capped at 60 characters and defaults to "Passkey" if you leave it empty.
4. Press **Add a passkey** and complete the ceremony your operating system or browser presents.
5. The passkey appears in the list with its name and "Added" date and time.

On the admin interface, use 👤 **Profile** → **Passkeys**, the field **Passkey name** (placeholder "e.g. MacBook Touch ID") and the button **Add a passkey**. Toasts confirm with "Passkey added" and "Passkey removed".

To remove a passkey, press **Remove** on its row (tooltip "Remove this passkey"). There is no confirmation prompt.

> **Note** If your device already holds a passkey for this account you see: "A passkey for this account already exists on this device. If it isn't shown in the list here, remove it from your device's passkey settings (iOS: Settings → Passwords) and try again."

5.5.4 Signing in with a passkey

1. Type your **Username** on the sign-in panel. The passkey button stays disabled until you do.
2. Press 🔑 **Sign with Passkey**. Leave the password field empty.
3. Complete the biometric or security-key prompt. You are signed in — with no code step, even if your account uses TOTP.

A challenge is valid for **5 minutes**; after that you see "Passkey challenge expired. Try again." Replay is prevented by signature counters. Cancelling the browser or operating-system prompt is treated as a silent cancellation — no error is shown.

> **Note** Asking for a passkey challenge for an unknown or ineligible username produces an indistinguishable decoy challenge, so the sign-in page cannot be used to discover which usernames exist. Your device then finds no matching passkey and the browser cancels, which looks exactly like a user cancellation.

5.6 Sessions, idle sign-out and session lifetime

5.6.1 How long a session lasts

Session tokens are stateless signed cookies, re-minted on every authenticated request (including the background session polls). Each token carries your username, your personal session epoch, the surface it belongs to, and the time of your original sign-in — the last of which is carried through every refresh.

Limit	Value	Setting
Idle window (application / integration), when no idle timeout is configured	12 hours	`PMW_SESSION_TTL` (seconds)
Idle window (admin), when no idle timeout is configured	8 hours	`PMW_ADMIN_SESSION_TTL` (seconds)
Idle window when an idle timeout *is* configured	Equals the configured timeout, sliding on activity	Users tab / App Control tab
Absolute session lifetime, all three surfaces	7 days from the original sign-in	`PMW_SESSION_MAX_LIFETIME` (seconds)

> **Warning** The absolute lifetime is not a sliding window. Seven days after you first signed in, the session dies even if you have been working continuously, and you must sign in again.

Every request also re-validates the live user record. If your account is disabled or deleted, the session is dead at once. If a role that gates the surface is revoked, the session is dead on that surface immediately.

5.6.2 Idle sign-out

> Requires: **Administrator** to configure

Two separate settings govern idle sign-out.

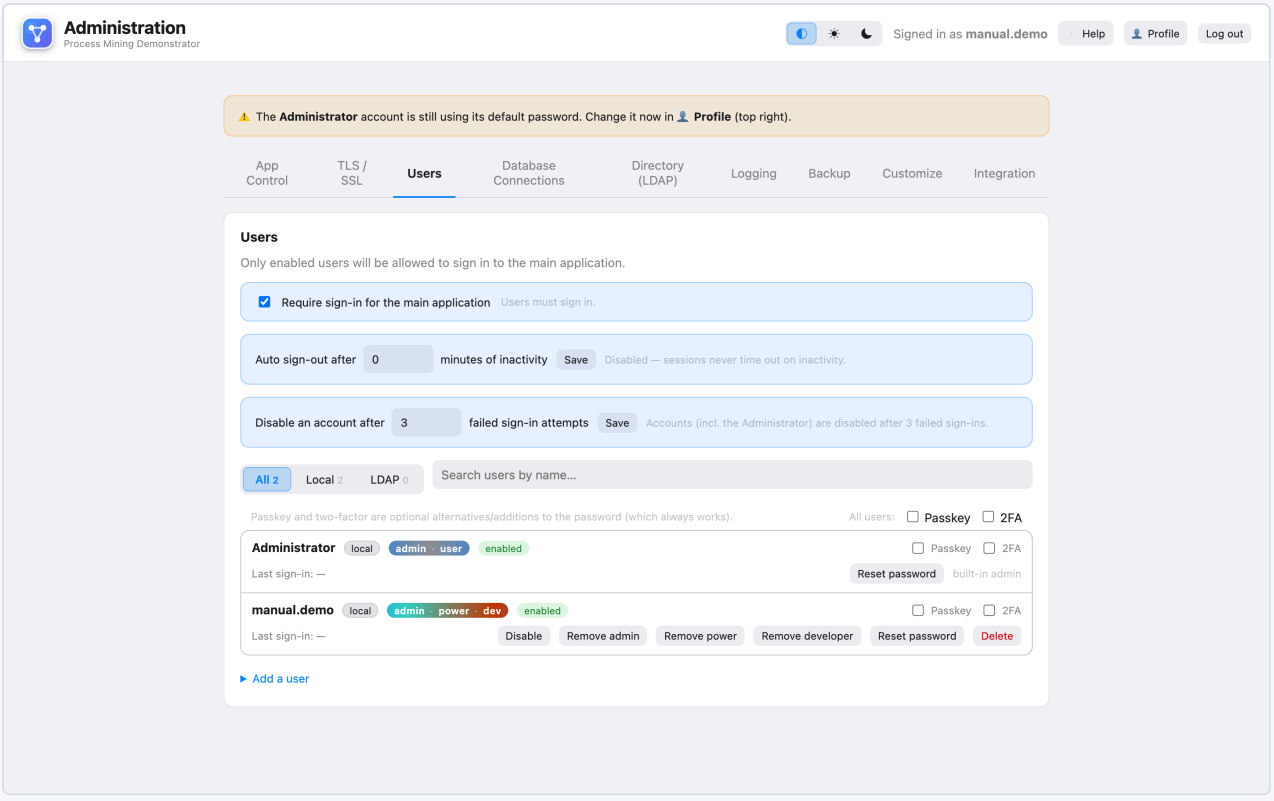


Figure 5.2 — Admin → Users. The **Auto sign-out after [N] minutes of inactivity** control governs the main application and the integration console.

1. Open the admin interface and go to the **Users** tab.
2. Set **Auto sign-out after [N] minutes of inactivity** — accepted range 0 to 1440, where **0 disables** the feature.
3. Save. The hint below the field confirms the effect: "Idle sessions are signed out after N min." or "Disabled — sessions never time out on inactivity."

In the main application, an invisible watcher observes mouse movement, mouse clicks, key presses, touches, scrolling and wheel events. After N idle minutes it signs you out and the sign-in panel shows "You were signed out due to inactivity." While you are active it pings the session endpoint — at most once every 15 to 60 seconds — so the server-side sliding window stays alive even during long stretches without API calls. The watcher runs only when sign-in is required, a user is signed in, and N is greater than zero.

> **Warning** The integration console honours the *same* idle value on the server side, so its session expires after N idle minutes — but it has **no client-side watcher**. You are not bounced to a sign-in screen; instead your next action simply fails authentication. Expect this if you leave the console open in a background tab.

The admin interface has its own, independent setting.

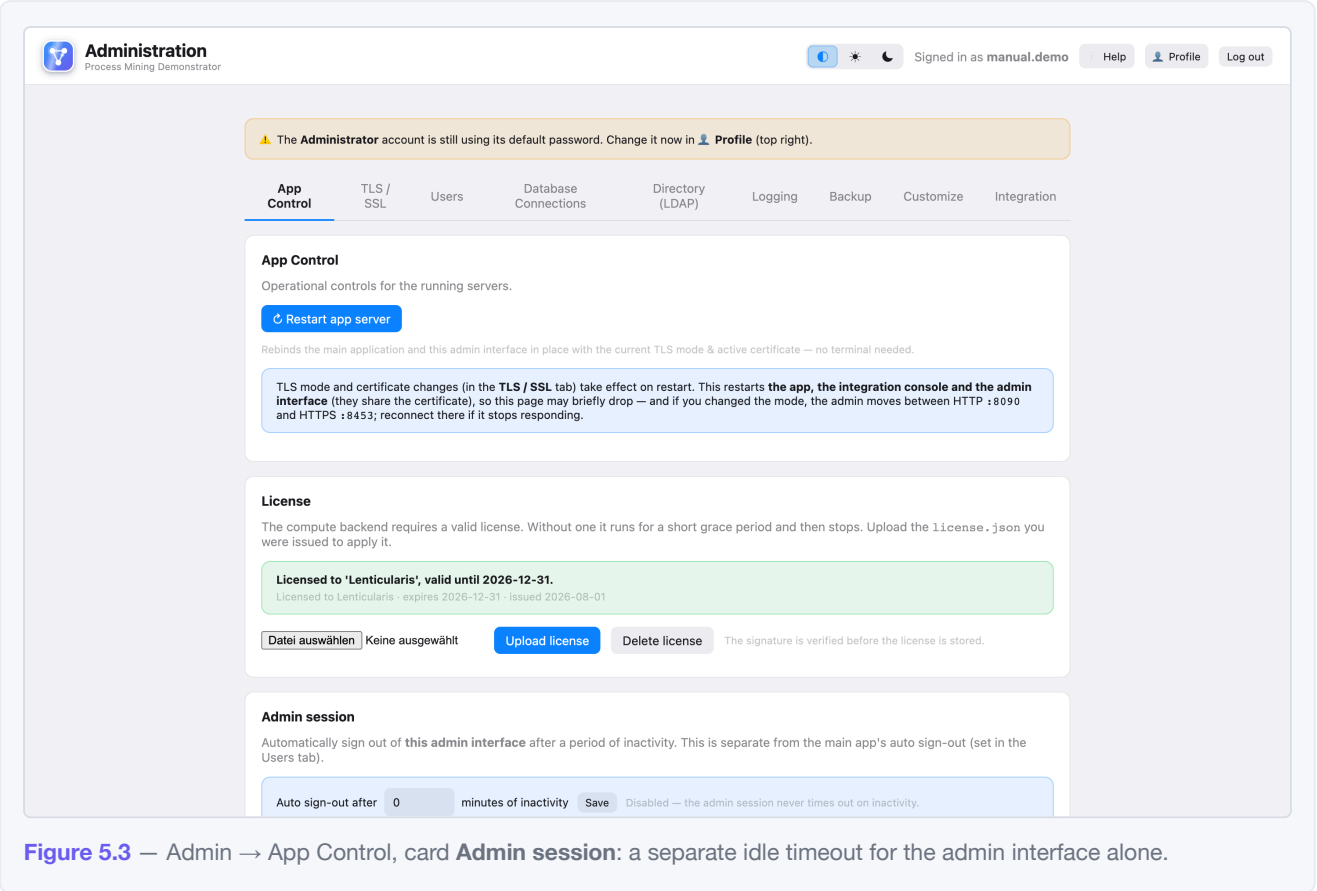


Figure 5.3 — Admin → App Control, card **Admin session**: a separate idle timeout for the admin interface alone.

1. Go to the **App Control** tab and find the card **Admin session** — "Automatically sign out of **this admin interface** after a period of inactivity. This is separate from the main app's auto sign-out (set in the Users tab)."
2. Set **Auto sign-out after [N] minutes of inactivity**.

3. Press **Save**. The toast reads "Admin auto sign-out updated" and the hint becomes "You are signed out of this admin after N min idle." or "Disabled — the admin session never times out on inactivity."

The admin watcher mirrors the application's and redirects you to `/login?inactivity=1`.

5.6.3 Signing out — and what else it ends

⏻ **Sign out** in the application and integration console sidebar footer, and **Log out** in the admin interface, do more than delete the local cookie. They bump your session epoch on the server, which invalidates **every outstanding session token of yours on all three surfaces**.

> **Warning** Signing out of the main application also kills your admin and integration sessions — they fail on their next request. This is intentional, but it surprises people who keep the admin panel open in another tab. Signing out also releases your per-user database connection on the backend.

5.6.4 Password changes invalidate sessions

Any password set or reset bumps the session epoch, so all of that user's sessions die.

You change your own password (admin Profile overlay)

Your *current* admin session survives — a fresh cookie is issued in the same response. Every *other* session of yours is invalidated.

An administrator resets someone else's password

That user is signed out silently at their next request and must sign in again with the new password.

5.6.5 When a session dies while you are working

API calls begin returning 401 with "Authentication required." The application surfaces this as an error, and reloading the page lands you on the sign-in panel. Two other server messages you may meet are unrelated to authentication but appear in the same place: "The compute backend is not reachable. Start it with ./run.sh or check PMW_BACKEND_URL." (502) and "The compute backend timed out." (504).

5.6.6 Resuming where you left off

The main application remembers your last working position and restores it automatically the next time it loads. When you sign in again — including after an idle sign-out — it **reconnects to the database connection** you were using, **reopens the project** you had open and **returns to the view** you were in (A-Chart, Statistics, A/B Comparison, and so on). The same restore happens on a plain page reload while your session is still alive.

The position is stored per user in your server-side settings, so it follows you between browsers and machines, exactly like your presets and layouts. It records only *where* you were — the connection, the project and the view — not any transient in-progress edits. A few natural limits apply:

- If the saved connection has since been un-assigned from you, or the saved project no longer exists, that step is skipped silently and you land one level up (connected but no project, or signed in but not connected).
- If the automatic reconnection fails — for example the database is down — the usual "Connection Failed" retry dialog appears, just as if you had clicked the connection yourself.

- Explicitly pressing **Disconnect** clears the saved connection, so a deliberate disconnect is never silently undone on your next sign-in. (An idle sign-out does *not* count as a disconnect — that is exactly the case this feature is for.)

5.7 Account lockout and attempt throttling

5.7.1 Failed-sign-in lockout

> Requires: **Administrator** to configure

1. In the admin interface, open the **Users** tab.
2. Set **Disable an account after [N] failed sign-in attempts** — accepted range 0 to 100, where 0 turns the feature off. Fresh installations default to **3**.
3. Press **Save**. The toast reads "Failed-sign-in lockout updated" and the hint becomes "Accounts (incl. the Administrator) are disabled after N failed sign-ins." or "Disabled — accounts are never locked on failed sign-ins."

The counter records **consecutive wrong-password attempts** against a local account, pooled across all three surfaces — they share one store. A successful sign-in resets it. The following do *not* count: wrong TOTP codes, failed passkey ceremonies, directory (LDAP) accounts, and accounts that are already disabled.

When the threshold is reached the account is disabled and flagged. Its row in the Users tab shows a **locked** pill (tooltip "Disabled after too many failed sign-ins — Unlock to restore.") and the action button reads **Unlock**.

> **Note** Despite the admin hint's wording, the **built-in Administrator is never auto-locked**. Its failures are counted, but it is the break-glass account and must stay reachable; its protection is the per-IP throttle plus deliberately slow password hashing. Other administrator accounts do lock.

There are three ways back in:

Route	Detail
Wait	A lockout lifts itself after **15 minutes**. The expiry is applied lazily, at the next sign-in attempt, so simply trying again after the window works.
Administrator	Press **Unlock** (or **Enable**) on the user's row in the Users tab. This also clears the counter. Disabling and re-enabling an account clears it too.
Break-glass restart	Restart the server with `PMW_RESET_LOCKOUTS=1` to clear every lockout — for the case where your only non-built-in administrator is locked out.

> **Warning** The 15-minute auto-expiry is a fixed value with no control in the admin interface. Only the threshold is configurable there.

5.7.2 Why the message is sometimes vague

The sign-in panel avoids telling an attacker whether a username exists. The specific reason for a refusal is shown **only when the supplied password is correct**:

- Correct password, account auto-locked → "This account has been locked after too many failed sign-in attempts. Contact an administrator."
- Correct password, account disabled by an administrator → "This account has been disabled. Contact an administrator."
- Anything else — wrong password, unknown username, disabled account with a wrong password → the generic "Invalid username or password." (application and integration console) or "Invalid credentials, or the account is not an administrator." (admin interface).

The same principle governs the passkey path: a request for an unknown or ineligible account receives a decoy challenge rather than a refusal.

5.7.3 Per-IP throttle

Independently of the account lockout, all three surfaces throttle by source address. After **3 failures within 15 minutes from one IP**, that IP must wait a **5-minute cooldown**, measured from its last failure. The counter lives in memory in the serving process and is cleared by a restart, or by one fully successful sign-in from that address.

The throttle covers password sign-ins, TOTP verification, forced-enrolment confirmation, the 2FA turn-off reauthentication and, on the admin panel, passkey sign-in completion. The socket peer address is used; the `X-Forwarded-For` header is deliberately ignored, so a client cannot forge its way around the limit.

Messages differ slightly by surface: "Too many attempts. Try again shortly." on the application and integration console; "Too many failed attempts. Try again in about N seconds." on the admin login page (accompanied by a Retry-After header); "Too many failed attempts. Try again shortly." on the admin code and setup steps.

5.8 Single-user (open) mode

> Requires: **Administrator**

For a single-user installation or a demonstration machine, sign-in on the *main application* can be switched off entirely.

1. Open the admin interface and go to the **Users** tab.
2. Untick **Require sign-in for the main application**. The toast reads "Access updated" and the hint changes from "Users must sign in." to "The app is open — no sign-in required."

Setting	Behaviour
On (default)	Every visitor sees the sign-in panel. All API calls require a session; without one they return 401 "Authentication required." Only the health probe is open.
Off (open mode)	The application loads with no sign-in panel and **no user identity at all**.

In open mode:

- Personal settings and filter presets are shared by everyone who uses the machine — there is one profile.
- The sidebar authentication footer disappears: no name, no 🔒 **Two-factor**, no 🔑 **Passkeys**, no ⏻ **Sign out**.
- The idle sign-out is inactive.
- Power-user and administrator features are hidden in the interface and refused by the server — with no identity there is nothing to authorise, so those endpoints return 403.
- Notes carry no author.

> **Note** If the session probe itself fails at start-up, the application assumes open access so that it still loads rather than trapping you on a broken sign-in panel.

The integration console **ignores** this toggle and always requires sign-in, because it is role-gated. So does the admin interface.

5.9 Roles and what each one may do

Roles are independent, additive flags on an account: **Administrator**, **Power** and **Developer**, on top of the implicit plain-user baseline. An account may hold any combination. The sidebar footer shows your identity as "👤 name (Role)" using the highest role you hold, in the order Admin > Power user > Developer > User. The admin Users tab instead shows a combined badge such as `admin·power` or `power·dev`, with tooltips "administrator", "power user (manages own connections)", "developer (integration console)" and "Regular user".

Capability	Plain user	Power	Developer	Admin
Sign in to the main application (enabled account)	✔	✔	✔	✔
Sign in to the integration console	✖ (403)	✖ (403)	✔	✔
Sign in to the admin interface	✖	✖	✖	✔
Use database connections assigned to them	✔	✔	✔	✔
Create and manage *their own* database connections	✖	✔	✔	✔ (manages all)
Conformance Check, Happy Path, Simulation views	✖ (hidden from the view menu)	✔	✖ (unless also Power)	✔
Journey sampling — create and delete representative subsets, A/B sources	✖	✔	✖	✔
Notes — create and comment	✔	✔	✔	✔
Enrol a passkey; set up two-factor	Governed by the per-user Passkey and 2FA permissions, which are independent of role			
Manage users, TLS, LDAP, backups, license, customisation, integration on/off	✖	✖	✖	✔

Two points are easy to get wrong:

- The **Developer** role is purely an additional access grant to the integration console. It does not confer power features — a developer who tries to use journey sampling is refused with "Journey sampling is available to power users and administrators only." — although developers can manage their own database connections.
- **Power** does not open the integration console. Power users are refused there at the password step.

Role revocation is enforced live: the surface predicate is re-checked on every request, so a demoted user loses access on their next action rather than at their next sign-in.

> **Note** The built-in Administrator cannot be disabled, demoted or deleted — attempts produce "The built-in Administrator account cannot be disabled.", "…must remain an administrator." and "…cannot be deleted." In addition, any change that would leave the installation with no enabled administrator is refused: "This would leave no enabled administrator. Promote or enable another admin first."

5.10 Administering accounts and permissions

> Requires: **Administrator**

The **Users** tab carries the note "Only enabled users will be allowed to sign in to the main application." and holds the three global settings described above — require sign-in, application idle timeout and the failed-sign-in threshold — plus the per-user controls.

Filter chips **All** / **Local** / **LDAP** with counts and a search field ("Search users by name…") narrow the list; an empty result reads "No matching users."

The all-users master row

The master row is introduced by the sentence "Passkey and two-factor are optional alternatives/additions to the password (which always works)." It offers, under the label "All users:", a **Passkey** checkbox ("Allow every user to enrol and sign in with a passkey") and a **2FA** checkbox ("Allow every user to set up two-factor (TOTP)"). Ticking either writes the flag for every existing user at once, confirmed by "Passkeys allowed/disabled for all users" or "Two-factor allowed/disabled for all users". Each checkbox reflects whether every user is currently allowed.

> **Warning** The master checkboxes act on the users that exist at the moment you tick them. They do not set a default for users created later — you must tick the flag on new accounts yourself.

Per-user controls

Each user card shows the username and display name, a **local** or **LDAP** pill, the role badge, an **enabled** / **disabled** / **locked** pill, and the per-user **Passkey** and **2FA** checkboxes ("Allow this user to enrol and sign in with a passkey" / "Allow this user to set up two-factor (TOTP)"). The second line shows "Last sign-in: …" followed by the action buttons.

Button	Effect
Disable / **Enable**	Blocks or restores sign-in. Re-enabling also clears the lockout counter.
Unlock	Appears on a locked row. Restores the account and clears the counter.
Make admin / **Remove admin**	Grants or revokes the administrator role.
Make power / **Remove power**	Grants or revokes the power role.
Make developer / **Remove developer**	Grants or revokes access to the integration console.
Reset password	Local users only. Prompts "New password for *user*:". Signs that user out everywhere.
Delete	Confirms with 'Delete user "*user*"?'. Also deletes the user's passkeys and recovery codes.

The built-in administrator's row carries only the note **built-in admin** (tooltip "The built-in administrator cannot be disabled, demoted or deleted.") — and an **Unlock** button if it has somehow been flagged.

To create an account, use **Add a user**: fill in **Username** and **Password**, optionally tick **Administrator**, and press **Create user**. Errors are "Username is required.", "Password is required." and "A user named '…' already exists."

5.11 Message catalogue

Every message you may see on a sign-in or authentication screen, with what it means.

Password step

Message	Meaning
Invalid username or password.	Wrong credentials, or an unknown or disabled account together with a wrong password. Application and integration console.
Invalid credentials, or the account is not an administrator.	The admin interface equivalent. Deliberately does not distinguish the two cases.
This account has been locked after too many failed sign-in attempts. Contact an administrator.	Your password was correct but the account auto-locked. Wait 15 minutes or ask an administrator to unlock it.
This account has been disabled. Contact an administrator.	Your password was correct but an administrator disabled the account.
Too many attempts. Try again shortly.	Per-IP cooldown active (429). Application and integration console.
Too many failed attempts. Try again in about N seconds.	Per-IP cooldown on the admin login page, with a Retry-After header.
You need the Developer role to use the integration console. Ask an administrator to grant it.	Integration console only: valid password, but no Developer or Administrator role (403).
The server is not reachable.	The browser could not reach the GUI server at all — a network or service problem, not a credential problem.

Two-factor

Message	Meaning
Incorrect code. Try again.	Wrong or expired code, or a recovery code that has already been used. The pending state is kept so you can retry without retyping your password.
Your sign-in session expired. Start again. / Your sign-in session expired. Please start again.	The 5-minute window after the password step lapsed. Begin again from username and password. The second wording is the admin interface.
That code didn't match. Try again.	Wrong confirmation code during enrolment. The same QR code and secret remain valid — do not rescan.
Setup expired. Start again.	The 10-minute enrolment window lapsed. Request a new QR code.
Two-factor authentication is not enabled for your account.	An administrator has not allowed 2FA for you (403).
Two-factor authentication isn't set up.	You tried to regenerate recovery codes or turn 2FA off without an enrolment.
Enter your current authentication code to turn off two-factor.	The code supplied on the turn-off confirmation was wrong.

Passkeys

Message	Meaning
Passkey sign-in failed.	Verification failed: unknown or removed credential, permission revoked, disabled account or — on the integration console — a missing role. The admin page variant adds "Check the passkey is registered and still enabled."
Passkey challenge expired. Try again.	The 5-minute challenge window lapsed. Press the button again.
Passkey sign-in is not available. / Could not start passkey setup.	The server could not begin the ceremony.
Passkeys are not supported in this browser. / This browser does not support passkeys.	No WebAuthn support. The second wording appears in the management sheets.
Passkeys are not enabled for your account.	An administrator has not allowed passkeys for you (403). The admin banner variant adds "— enable them in the Users tab first."
Passkeys need a hostname, not an IP address…	You reached the page by IP address. Use the server's name over HTTPS — for example its ".local" name.
A passkey for this account already exists on this device…	Remove the stale passkey in your device's own passkey settings (on iOS: Settings → Passwords), then try again.
Could not register passkey: *reason*	Verification failed while enrolling.
Could not load passkeys. / Could not remove the passkey.	The management sheet could not reach the server.

Tip Cancelling an operating-system or browser passkey prompt is always silent — no error appears. If you press the passkey button and nothing seems to happen, the ceremony was cancelled, either by you or because your device holds no matching passkey.

Session and state

Message	Meaning
Not signed in.	An authentication-management endpoint was called without a session (401).
Authentication required.	A gated API was called without a session (401). Reload to reach the sign-in panel.
You were signed out due to inactivity.	The idle watcher signed you out. Sign in again.
The compute backend is not reachable. Start it with ./run.sh or check PMW_BACKEND_URL.	502 — the web surface is up but the analysis back-end is not.
The compute backend timed out.	504 — the backend accepted the request but did not answer in time.
The integration console has been turned off by the administrator.	503 — the console is disabled in Admin → Integration.

5.12 Points to remember

1. Enabling 2FA for a user enables **mandatory** 2FA. There is no opt-out and no grace period.
2. A passkey sign-in bypasses the TOTP step by design. Do not allow passkeys if you require a code on every sign-in.
3. Signing out anywhere signs you out everywhere, as does any password change or reset — except that an administrator changing their own password keeps their current admin session.
4. Revoking the 2FA permission does not delete the enrolment; re-granting it re-arms the same authenticator. Only **Turn off** from the sheet deletes the secret and the recovery codes.
5. Recovery codes are shown once. Regeneration invalidates every earlier code, and no administrator can retrieve them.
6. Remember the three timers: 5 minutes to enter a code after the password, 10 minutes for an enrolment QR code, 5 minutes for a passkey challenge.
7. Wrong codes never lock an account; wrong passwords do — three attempts by default, auto-unlocking after 15 minutes. The built-in Administrator never auto-locks, and `PMW_RESET_LOCKOUTS=1` at start-up clears every lockout.
8. The integration console has no idle watcher, always requires sign-in even in open mode, and refuses power users before any second factor.
9. Open mode has no identity: shared settings and presets, unattributed notes, and no power features at all.
10. The absolute 7-day session cap ends even a continuously active session.

6 The Application: Orientation

> This chapter is your map of the application window. It explains what happens between opening the address and seeing your first process map, describes every permanent element of the shell — sidebar, detail pane, title capsule, toolbar, alerts and theme — and then covers the two controls you will use most often: the **view-mode menu**, which switches the detail pane between the eleven modes (the Dashboard overview plus ten analysis modes), and the **KPI strip**, the band of figures above the chart. It is written for every user of the application, whatever their role; where a feature is restricted to power users or administrators this is stated explicitly. Later chapters describe each view mode and each sidebar section in detail.

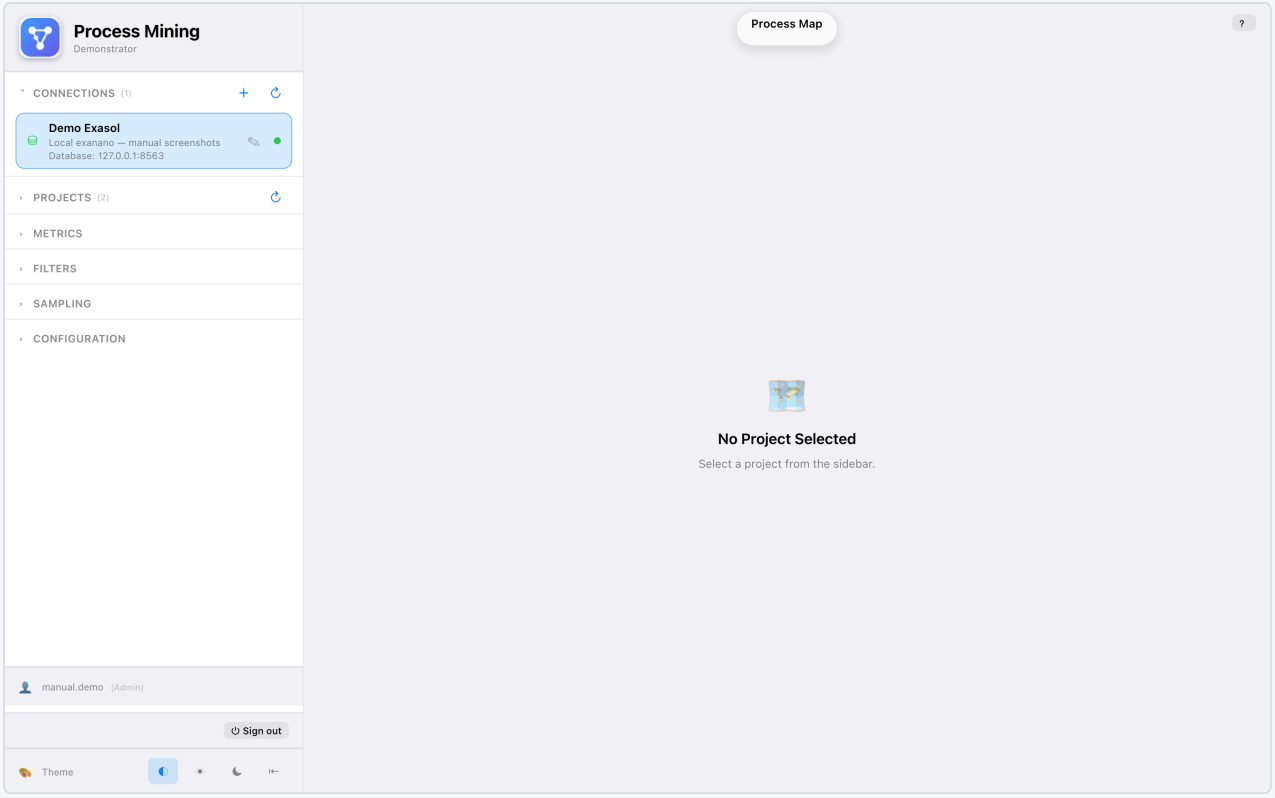


Figure 6.1 — The application shell: sidebar on the left, detail pane on the right with the title capsule, toolbar, KPI strip and process map.

The Launcher — one entry point

Rather than remembering a port per surface, open the **Launcher** at `/launcher.html` on the application address (for example `http://127.0.0.1:8080/launcher.html`, or `:18080` under Docker). It is a single page, served by the application itself, that links to all four surfaces — **Administration**, **Integration Console**, **Action Builder** and the **Work-Bench** — with floating cards and arrows showing how they relate, and a **Training & Documentation** row that opens the built-in guides and the full PDF manual over HTTP(S). Its links adapt to wherever it is opened (native or Docker, HTTP or HTTPS), and it picks up the admin-configured sign-in background and the current version. The Launcher needs no sign-in of its own; each card leads to that surface's own login.

Starting the application

The application always passes through the same sequence of states before the working surface appears. Understanding the sequence makes it obvious where you are if something stalls.

1. While the application boots it shows a large spinner with the text **Starting…**. Nothing is interactive at this point.
2. If the administrator has made login mandatory and you have no valid session, the **Login** view is shown instead of the application. The Login view also stays up during a forced two-factor enrolment, until your recovery codes have been displayed and acknowledged.
3. The **legal gate** appears next. You must accept it before any part of the application UI is shown.
4. The shell appears: sidebar on the left with the **Connections** section already open, and the **Dashboard** in the detail pane on the right — the overview of every process available to you. (Opening a specific process from the `/home` launcher skips the Dashboard and drops you straight onto that process map.)

> **Warning** Declining the legal gate signs you out and returns you to the Login view. There is no way past the gate other than accepting it; if you cannot accept the terms, contact your administrator.

The Dashboard reads across all your connections and needs none selected. The other view modes do: until you connect to a database and select a project, each shows the placeholder glyph ⛁, the title **Not Connected** and the text “Add a connection in the sidebar and tap it to connect to your Exasol database.” This is a global state, not a per-mode error — switching among those modes will not make it go away (the Dashboard remains available throughout).

The application shell

The window is divided into two areas that never change: the **sidebar** on the left and the **detail pane** on the right.

The sidebar

The sidebar is the control surface. From top to bottom it contains the brand header (“Process Mining” over the caption “Demonstrator”, with the logo), a divider, six accordion sections, the signed-in identity footer, a divider and the theme bar.

The accordion sections are, in order: **Connections**, **Projects**, **Metrics**, **Filters**, **Sampling** and **Configuration**. Only one section is open at a time — opening a section collapses all the others. **Connections** is open when the application starts. The **Connections** and **Projects** headers show the number of items in parentheses, for example “Connections (3)”, whenever that number is greater than zero.

Two sections react to the active view mode:

Individual Journey

Switching to this mode opens the **Filters** section automatically so that the Event ID field is immediately at hand. This happens once per mode change; you may collapse the section again afterwards. The **Metrics** and **Sampling** sections are hidden entirely in this mode, because an individual journey is always drawn with a fixed *Avg Time* metric.

Sampling

Visible to power users and administrators only.

To reclaim horizontal space for the chart, hide the sidebar:

1. Click ⇤ **Hide sidebar** in the theme bar at the very bottom of the sidebar.
2. The sidebar disappears and a floating ▤ button (tooltip and screen-reader label “Show sidebar”) appears at the top left of the detail pane.
3. Click ▤ to bring the sidebar back.

The detail pane

The detail pane renders the active view mode. Whatever the mode, its top edge carries the same two elements.

Title capsule (top left)

Shows the title of the selected project, falling back to “Process Map” when no project is selected. Underneath it, the name of the active view mode is shown; this second line is invisible until a project is selected.

Toolbar (top right)

Contains the **?** button (tooltip “Help”) and, when applicable, the ≡ view-switch button.

The Help panel

The **?** button opens a floating Help panel over the detail pane. The panel can also be toggled from the keyboard.

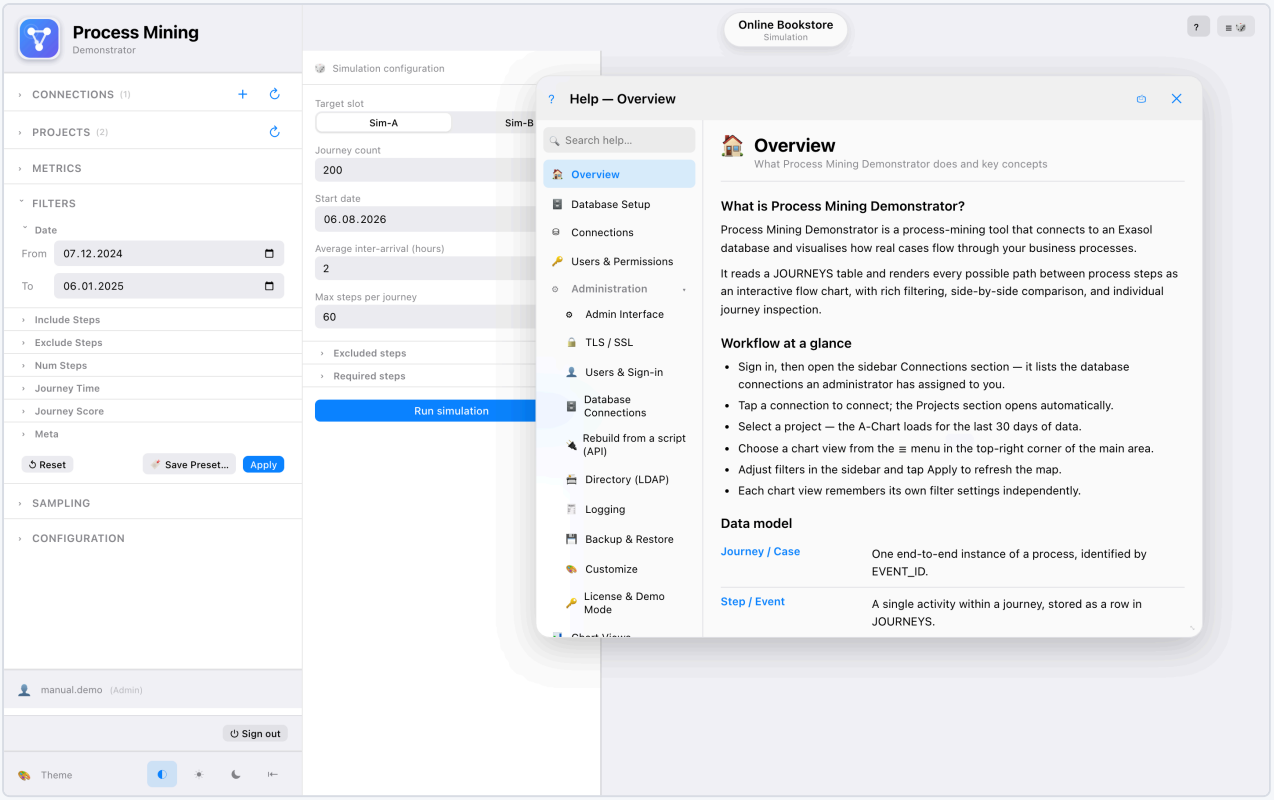


Figure 6.2 — The floating Help panel, opened with the ? toolbar button or ⌘/Ctrl + /.

Alerts

Errors and confirmations that require your attention appear as modal alert boxes centred over a dimming scrim. An alert has a title, a message and one or two buttons — typically a primary action and a secondary **Cancel**. Clicking the scrim outside the box dismisses the alert.

Alerts are used consistently throughout the application. Examples you will meet early are **Connection Failed** (buttons *Retry* / *Cancel*, where Retry re-attempts the same connection), **Failed to Load Projects** and **Failed to Load Project** (both *Retry* / *Cancel*), **No Journeys on <date>** when the day slider lands on an empty day, and **Note Not Synced** when a note cannot be written to the database.

Theme

The theme bar at the bottom of the sidebar is labelled 🎨 **Theme** and offers three icon buttons: **System (◐)**, **Light (☀)** and **Dark (☾)**.

1. Click the theme bar button you want. The change applies immediately.
2. Leave the setting on **System** (the default) to follow your operating system's light/dark preference automatically.

> **Note** The theme is stored per user in the backend settings store as `app.theme` (default `system`). It follows you to another browser or another machine, and it is included in backups.

Keyboard shortcuts

Shortcut	Where	Effect
⌘/Ctrl + / or **⌘/Ctrl + ?**	Anywhere in the application	Toggles the Help panel
Esc	Popovers, menus, sheets	Closes the preset menu, the view-mode menu, the value list of a meta filter, and modal sheets (without saving)
Enter	Event ID field, norm editor, name prompts	Confirms the field (loads the journey, saves the norm, confirms the name)
← / **→**	Journey time slider	Moves the focused thumb by one day and commits
Home / **End**	Journey time slider	Jumps to the start / end of the data range and commits
Double-click	Chart background	Fits the graph to the view

Switching view modes

The detail pane can display the Dashboard overview plus ten analysis modes. You change between them with the toolbar's view-switch button.

1. The ≡ **<mode icon>** button (tooltip "Switch view") is available on the Dashboard (your landing screen) and whenever you are connected with a project selected — so you can always return to the Dashboard and switch away from it.
2. Click ≡. A popover menu lists the available view modes; the current mode is marked with a ✓.
3. Click a mode to switch to it. Clicking anywhere outside the menu closes it without changing the mode.

The view modes

Mode	Icon	Purpose	Who can see it
Dashboard	🧭	Cross-connection overview of every process you can see; the landing screen on a direct sign-in	All users
A-Chart	📈	The primary process map with its own filter set	All users
B-Chart	📉	A second, fully independent process map	All users
A/B Comparison	⇄	A and B side by side with a similarity score	All users
Individual Journey	🧍	One journey, drawn in chronological order	All users
AI supported Documentation	🧠	LLM-generated process documentation	All users
Statistics	📊	Journey routes and analytics	All users
Conformance Check	🛡️	Actual transition values against per-edge norms	Power users and administrators
Happy Path	🚩	Ideal step sequence and its coverage	Power users and administrators
Notes	🗒️	All notes on the project, with filters	All users
Simulation	🎲	Monte-Carlo simulation from the observed graph	Power users and administrators

> Requires: **Power user** or **Administrator** — Conformance Check, Happy Path and Simulation. For plain users these three entries are not merely disabled, they are absent from the menu.

The Dashboard

The **Dashboard** is the first entry in the ≡ menu and the screen you land on after a direct sign-in to the application. (If you instead launched a specific process from the `/home` launcher, the application skips the Dashboard and opens that process directly.) Unlike every other mode, the Dashboard does not need a connection or a selected project — it reads across *all* the connections assigned to you at once, so it is a true overview rather than a view of one process.

The Dashboard lists only **base process flows**. Derived aggregate projects — a high-level **Σ** map or a **#** detail / sub-flow produced by the Aggregate designer — are not shown, so their copied data never double-counts against the base process.

It is laid out as cards:

- An **Overview** card totals your **processes**, **connections** and **events**, with a sparkline of all ingests combined.
- One **tile** per **process**, in a single flat, rearrangeable set (not grouped under connection headers). Each tile carries its **connection** as a coloured pill in its upper-left corner — tiles of the same connection share a colour and a matching coloured left edge, so a connection still reads at a glance — and shows the

number of **events**, when the **last log** entry was recorded, and a weekly **ingest sparkline**. **Clicking a tile connects to that process and opens its A-Chart.**

- **Drag a tile** (by anywhere on it; the ⠿ grip in its corner marks it as draggable) onto another to set the order. Your arrangement is **saved per user**; a newly-appearing process is added at the end. A connection whose schema can't be read is listed in a small note beneath the tiles.

The **ingest sparkline** plots how many events were ingested per week over the recent weeks, ending at the current week. When the most recent weeks have no new events, the line ends in a flat, dashed segment running up to today — so a stalled or finished feed is obvious at a glance.

Events, Last log and the sparkline are always shown. The **Metrics** toolbar lets you add up to **three** more metrics — *Journeys*, *Days since last ingest*, *Active span (days)*, *Avg events / journey* and *Open notes* — which then appear on *every* process tile. Use the ◀ ▶ controls to reorder a chosen metric and ✕ to remove it. The selection is saved per user. A connection whose schema cannot be read is listed in a note beneath the tiles instead of failing the whole Dashboard.

Open notes shows the project's unresolved notes broken down by severity, as small coloured counts (red = Urgent, orange = Important, blue = Info, grey = Normal), or a muted `0` when there are none. As in the Notes view, it counts only the notes you may see — your own plus shared notes — never another user's private notes.

Auto-refresh. The Dashboard header carries an **Auto-refresh** control: pick an interval — **10s**, **30s**, **1m**, **2m**, **5m** or **Off** — and the *whole* Dashboard reloads on that timer: the process tiles (and, when the feature is enabled, the Notifications section below). A small **countdown ring** shows the time to the next refresh, and the **↻ Refresh** button beside it reloads everything at once on demand. The interval is remembered per user. This simply re-reads the current data, so a live feed (for example from the demo-event generator) shows up without reopening the view; it does *not* re-run the notification checks — that is the background engine's job, or the Notifications section's own *Refresh all*.

Notifications

When an administrator has enabled the feature (Admin → **Notifications**), a second **Notifications** section appears below the process tiles. A **notification rule** has a **check type**. A **transition** rule watches the transition from one step to a *following* step: if a chosen metric — average, median, minimum or maximum time, standard deviation, or the plain count — crosses a threshold you set, the rule **triggers**. A **journey duration** rule instead watches how long a *whole case* takes end to end: a chosen time aggregate of the case duration — average, median, minimum or maximum time, or standard deviation (there is no count) — against a threshold, with no step selection. A background **engine** evaluates each rule on its own interval (1 minute, 5 minutes, 15 minutes, 1 hour, 2 hours, 5 hours or 1 day); the check runs on the server, so the page need not be open.

A **blue summary strip** heads the section with the headline counts — rules, how many are triggered, how many are enabled, how many connections are being watched, and how many urgent alerts are live, plus a **Refresh all** button that re-checks every rule against the current data at once. Below it, **each rule is a card**, sorted by **urgency then time** so a firing alert floats to the top. A card shows the rule's criterion, a **status** pill (Triggered / OK / Off / Error) and a **severity** pill (Urgent / Important / Info), a **bar** of the measured value against its limit (with the limit marked), and four figures: the *last value*, how far *over* (or *under*) the limit it is, when it *triggered*, and the *check interval*. A ↻ button re-runs the check against the current data on demand

(any viewer, without waiting for the next interval), and a per-card switch enables or disables that one rule (managers only).

A **filter row** below the strip narrows which cards are shown — a **client-side view** only, so it changes nothing for anyone else and sends no request. Filter by **severity** (Urgent / Important / Info), by whether a rule is currently **triggered** (Yes / No), by whether it is **enabled** (Yes / No), and by how recently it fired — **Triggered at** Last hour / Last 24 hours / Last week / Last month / All (a time window shows only rules that are triggered now and crossed within it). The count beside the row shows how many of the total are visible, and **Clear filters** resets them.

Rules are **shared per connection**: everyone assigned to the connection sees the same alerts. They are created, edited, deleted and toggled by **power users, developers and administrators** through the **+ Create / Edit** button in the section header, which opens a wizard — pick a connection and process, the **check type** (transition between two steps, or journey duration), then — for a transition rule — the from-step and the following to-step, the metric, the comparison and threshold (with a time unit for duration metrics), a **base period** (how far back to look for events — a number of minutes / days / months / years, or *all history*), the check interval, a severity and an optional name. The base period narrows the measurement to recent events: "Avg Time over the last 7 days > 2h" averages only the past week's transitions, not the whole log. A **Test now** button evaluates the rule against the current data straight away — without saving — and reports the measured value and whether it would trigger, so you can check a threshold before committing to it. The same rules can be listed and toggled through the MCP server (see the MCP chapter).

Remote notifications. Besides the Dashboard card, a rule can deliver a **Web browser push** when it triggers. Turn on **Web browser push** in the wizard's *Remote notifications* section. Each person enables push **once per device** (the wizard offers it; a secure `https://` context is required and the browser asks permission); thereafter a rule with push on pops a browser notification to **everyone assigned to its connection who has enabled push** — even when the app is not open. The push fires **once**, when the rule crosses into triggered, not on every check — so a **Refresh** on an already-triggered rule will *not* re-send; use **Send a test notification** in the Remote notifications section to confirm push works on your device. An administrator sets the push **contact** (the VAPID `sub`) under **Admin → Notifications**; some push services, **notably Apple / Safari**, reject a push whose contact isn't a valid `mailto:` or `https://` URL. A rule can also deliver by **Email**: turn on **Email** and optionally list recipients (comma-separated); leave it blank to email everyone assigned to the connection who has an address on file (an administrator sets a user's email in **Admin → Users**). Email needs an **SMTP server** configured by an administrator under **Admin → Notifications → Email (SMTP)** (host, port, security, credentials and a From address, with a "send test email" button). The delivery layer is a channel registry, so **further channels** slot in without changing the rule model.

Per-mode filter memory

Each view mode keeps **its own filter state** — date window, included and excluded steps, meta filters and the selected transition metric. When you switch modes, the application saves the state of the mode you are leaving and restores the state of the mode you are entering. This is why the A-Chart still shows last week's window after you have been editing filters in the B-Chart.

Two modes deviate from this rule:

Statistics

Always inherits the **A-Chart's** filter state on entry. It never keeps a private filter set. The Statistics view itself displays the reminder "ⓘ Statistics use the same filters as Chart A."

A/B Comparison

Maps onto the saved A-Chart and B-Chart states rather than holding a third state. The active (editable) side starts as **A**.

> **Note** Switching to A-Chart, B-Chart, A/B Comparison, AI supported Documentation or Notes also reloads the project's notes list, so note badges on nodes and edges are up to date in those modes.

When a project is loaded for the first time, the A-Chart loads immediately using the default date window (the last **30** days of data by default, controlled by the *Default date window* setting in Configuration; `0` means the full range). Every other mode is seeded with the same window plus the project's real filter bounds, so no mode starts with an impossible range.

> **Tip** Use this deliberately. Set the A-Chart to your reference period and the B-Chart to the period you want to question; both survive any number of trips through the other modes, and A/B Comparison then shows exactly those two states side by side.

The KPI strip

The KPI strip is the horizontal band of figures between the title capsule and the chart. It summarises the currently filtered data set in a fixed set of tiles.

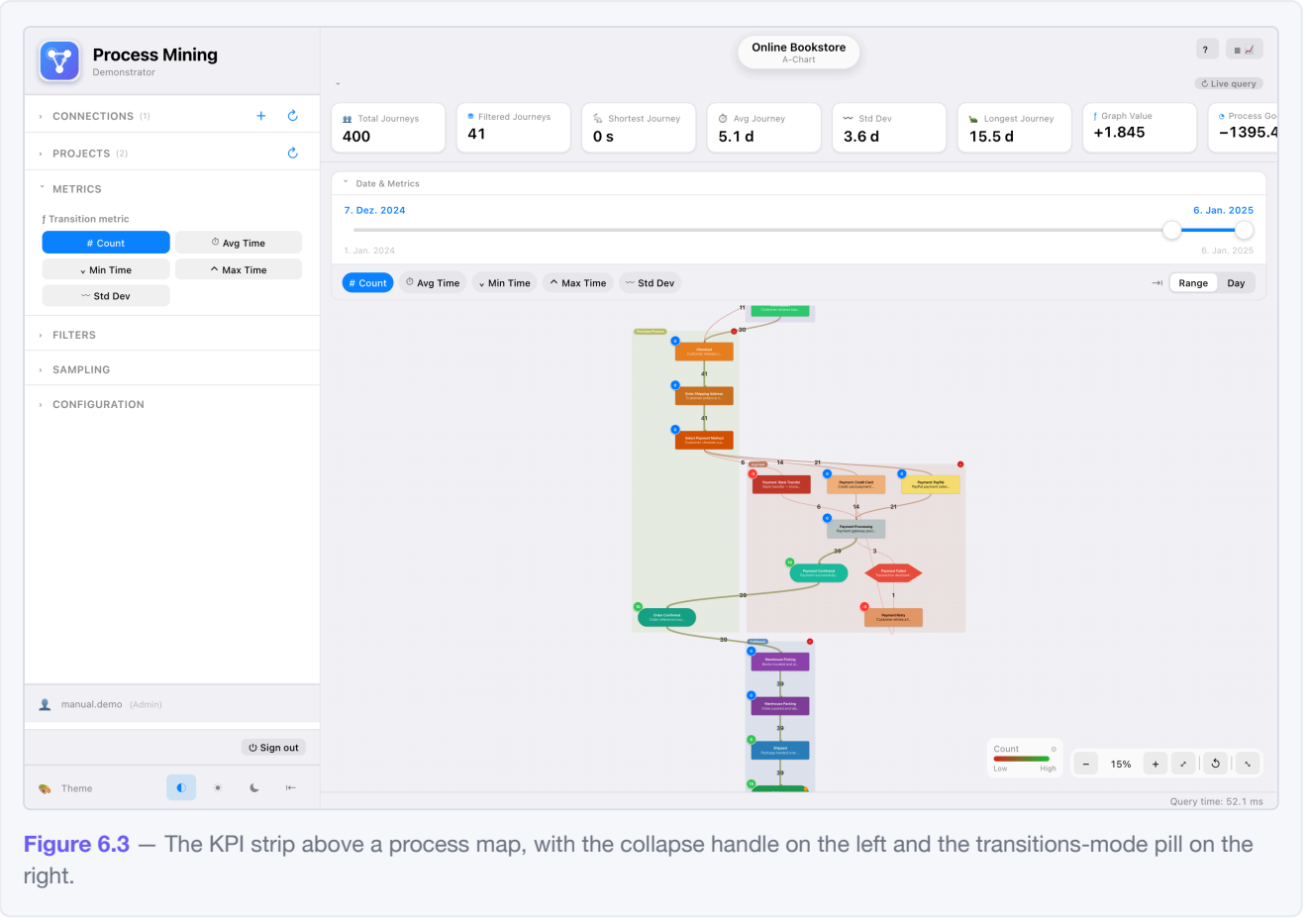


Figure 6.3 — The KPI strip above a process map, with the collapse handle on the left and the transitions-mode pill on the right.

Where the strip appears

The strip is shown when you are connected, a project is selected and data is present. In **Individual Journey** mode it appears once a journey has been loaded, and it shows a different set of tiles (see below).

The following modes render their own header instead of the shared strip: A/B Comparison (one strip per panel), Statistics, Notes, AI supported Documentation, Happy Path, Conformance Check and Simulation. Statistics and Conformance Check show the standard tiles under their own collapse handle; Notes and Simulation show tile sets of their own.

The handle: collapsing and expanding

1. Click the chevron on the KPI handle — the narrow bar along the top edge of the strip.
2. Collapsed, the handle shows a one-line summary in the form "*filtered* / *total* Journeys", for example "1,234 / 10,000 Journeys".
3. Click the chevron again to expand the tiles.

The expanded/collapsed state is remembered per user and per surface:

Surface	Setting	Default
A-Chart, B-Chart, Individual Journey	`processmap.kpiExpanded`	Expanded
A/B Comparison, panel A / panel B	`abpanel.a.kpiExpanded` / `abpanel.b.kpiExpanded`	Expanded
Statistics	`statistics.kpiExpanded`	Expanded
Conformance Check	`compliance.kpiExpanded`	Expanded

> **Note** In Conformance Check the collapsed handle summarises compliance instead of journeys: "N violations · M compliant".

The transitions-mode pill

At the right-hand end of the KPI handle a small pill tells you *how* the transitions behind the current chart were obtained. This matters for both performance and freshness. The pill is not shown in Individual Journey mode.

Pill	Tooltip	What it means for you
⚡ **Pre-materialized**	“Transitions are read from the pre-materialized TRANSITIONS_RAW table.”	Fastest path. Queries read a prepared table rather than deriving transitions from the raw event log.
↻ **Live query**	“Transitions are computed live from the event log on each load.”	Pre-materialisation is switched off for this connection. Every load recomputes the transitions; results are always current but slower on large logs.
⚠ **Live (not built)**	“Pre-materialized transitions are enabled for this connection, but TRANSITIONS_RAW is not built yet — running the live query meanwhile. Rebuild it in the admin interface (Connections → Rebuild now).”	An intended optimisation is missing. The chart is correct but slower than it should be.

> Requires: **Administrator** — building or rebuilding TRANSITIONS_RAW. If you see ⚠ **Live (not built)**, ask your administrator to run *Connections → Rebuild now* in the admin interface.

The standard KPI tiles

Every tile shows an icon, a label and a value. A value that cannot be determined displays as **—**; while a figure is still being fetched the tile shows a small spinner in place of the value.

Tile	Icon	Meaning and formatting
Total Journeys	👥	The unfiltered journey count of the project.
Filtered Journeys	⛃	The number of journeys matching the current filters.
Shortest Journey	🐇	Minimum journey duration. Durations are formatted by magnitude: under 60 seconds as “N s”, then “N.n min”, “N.n h”, “N.n d”.
Avg Journey	⏱	Mean journey duration, same duration formatting.
Median Journey	◑	Median journey duration (the database `MEDIAN`), same formatting — the typical duration, not pulled by outliers.
Std Dev	〰	Standard deviation of the journey durations, same duration formatting.
Longest Journey	🦕	Maximum journey duration, same duration formatting.
Graph Value	*ƒ*	The sum of (step score × visits) over all scored nodes, where *visits* = max(incoming, outgoing) occurrences of that step. Shown with an explicit **+** or **–** sign. The tile is hidden entirely when no step of the project carries a score.
Process Goodness	◔	A server-computed goodness score, shown signed with two decimals. Hidden when the server returns no value. In A/B Comparison the tile is tinted: green when this side is better than the other, red when worse, blue when the two are equal within 0.005.
Process Similarity	⇄	The A/B similarity, a value from 0 to 1 with two decimals. Green at 0.7 and above, red at 0.3 and below, blue in between. The tile is only present once an A/B similarity has actually been computed.
Active Sample	▤	The label is the side’s data source — “Original”, “Sample 1” to “Sample 3”, or “Sim-A” / “Sim-B” when a simulation is the source. The value is the journey count of that source.

> **Note** *Graph Value* and *Process Goodness* disappear rather than showing “—” when they cannot be computed. If you expected a Graph Value and see none, no step in the project has a score; assign scores in *Configuration → Steps*.

Choosing and ordering the tiles

Which tiles appear, and in which order, is a per-user preference.

1. Open the sidebar section **Configuration**.
2. Expand 📊 **KPIs**. The list shows every tile with its icon, label and an on/off switch. All tiles are on by default.
3. Switch a tile off to remove it from the strip.
4. To reorder, drag a row by its ≡ handle onto another row.

The default order is: Total Journeys, Filtered Journeys, Shortest Journey, Avg Journey, Median Journey, Std Dev, Longest Journey, Graph Value, Process Goodness, Process Similarity, Active Sample. Visibility is stored as `kpi.show.<id>` and the order as `kpi.order`.

> **Tip** The KPI settings, like all Configuration settings, are stored per user in the backend rather than in the browser. They follow you to a different browser or machine and are included in administrator backups.

The Individual-Journey KPI strip

In Individual Journey mode the standard tiles are replaced entirely — averages and totals are meaningless for a single journey. The transitions-mode pill is not shown either.

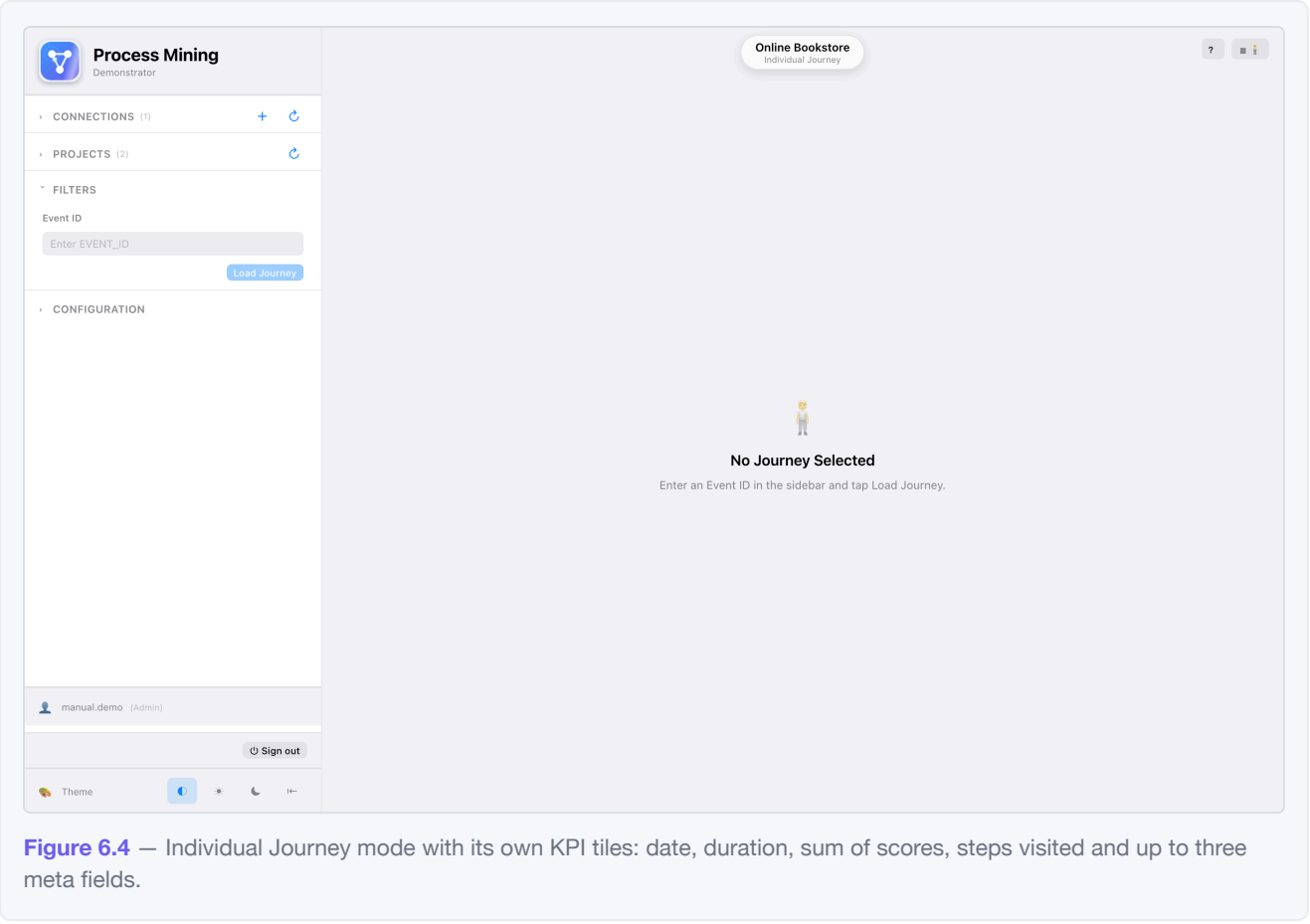


Figure 6.4 — Individual Journey mode with its own KPI tiles: date, duration, sum of scores, steps visited and up to three meta fields.

Tile	Icon	Meaning
Date	📅	The start date of the journey, in short date format.
Duration	⏱	End minus start of the journey.
Sum of Scores	*ƒ*	The sum of the step scores across the visited steps, shown signed. Displays “—” when no scores are defined.
Steps visited / distinct	⑂	Two numbers, “*total* / *distinct*”. The total is the number of transitions traversed plus one; the distinct count ignores repeats, so a journey that loops shows a smaller second number.
Meta fields	🏷	Up to three additional tiles, one per meta attribute the project defines. The label is the project’s meta title, the value is this journey’s value. A tile only appears when both the title and a value exist.

Note Individual Journey always draws its edges with the **Avg Time** metric and hides the sidebar's *Metrics* section, because a transition count is meaningless when each transition is traversed once.

7 Connections and Projects

> A *connection* is the app's definition of one Exasol database, one schema inside it, and — optionally — one LLM server. Everything you analyse comes out of a connection, so the Connections section of the sidebar is where every session begins. This chapter explains how to connect and disconnect, how to read the status indicators, and, for users who are allowed to manage connections, how to create and edit a connection definition field by field, test it, provision its schema, load ready-made demo data into it, and delete projects from it. If you only ever pick a connection from a list that someone else prepared for you, sections 7.1 to 7.3 are all you need; the remainder requires the Power user, Developer or Administrator role.

The Connections section

Connections is the first accordion section of the sidebar and the one that is open when the app starts. Its header shows the number of connections available to you in parentheses — for example **Connections (3)** — and the count is omitted when it is zero. Opening any other sidebar section collapses this one; only one section is open at a time.

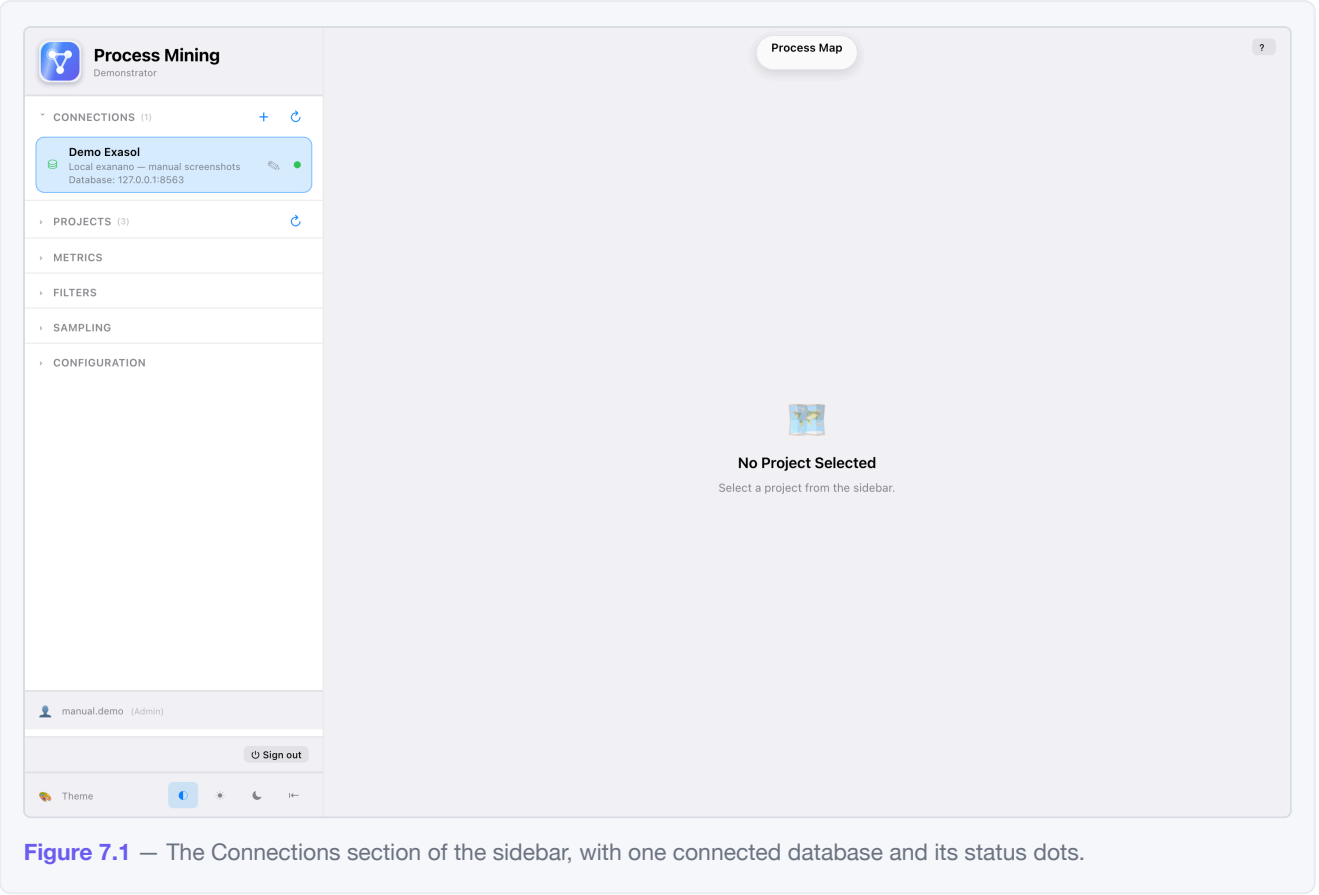


Figure 7.1 — The Connections section of the sidebar, with one connected database and its status dots.

Header controls

Control	Tooltip	Who sees it	What it does
+	New connection	Power user, Developer, Administrator	Refreshes the list of connections you may manage, then opens the Connection Editor in **New Connection** mode.
↻	Refresh connections	Everyone	Re-fetches the list of connections assigned to you and the current connection status.

Reading a connection card

Cards are sorted alphabetically by name. The list scrolls when it grows beyond roughly 300 pixels. Each card carries:

Database glyph (⛁)
: Green when this card is the connection you are currently connected to, accent-coloured otherwise.

Name
: The connection's display name; shown as **(unnamed)** if it has none.

Comment
: The optional description line, when one was entered.

Database: host:port
: The database endpoint. The host reads **(no host)** when it is empty.

LLM: <URL>
: Shown only when the connection has an LLM server configured; falls back to **(configured)** when the URL cannot be shown.

Status dots appear on the right of the *active* card only — that is, the card you are connected to or last attempted:

Dot	Colour	Tooltip	Meaning
Database	Green	Connected	A live Exasol session is open for you.
	Orange	Not connected	No session is open — connect, or the last attempt failed.
LLM	Blue	LLM reachable	The LLM server answered the reachability probe.
	Orange	LLM not reachable	The probe failed, timed out, or the server returned a 5xx status.

> **Note** The LLM dot is shown only for connections that have an LLM server URL. Connections without one simply have no LLM indicator, and the AI supported Documentation view has nothing to talk to.

A ✎ **Edit** button (tooltip “Edit connection”) appears on a card only if you may manage that particular connection: a power user or developer sees the pencil on the connections they own, an administrator sees it on all of them. Clicking it opens the editor without connecting.

Empty states and errors

When the list is empty the message depends on your role:

- Connection managers see *“No connections yet. Use + above to create one and assign users.”*
- Everyone else sees *“No connections assigned to you. Ask an administrator to grant access.”*

The most recent connection error is printed in red beneath the list, or inside the empty state. A failed connect additionally raises an alert titled **Connection Failed** carrying the error text and the buttons **Retry** and **Cancel**; Retry re-attempts the same connection. Unexpected API failures raise the same title with a single **OK** button.

Connecting and disconnecting

1. Open the **Connections** section in the sidebar.
2. Click the card of the database you want to work with. There is no separate “Connect” button — the card *is* the button.
3. Wait for the database dot to turn green. The sidebar then opens the **Projects** section automatically and loads the project list.
4. To disconnect, click the card you are connected to. To switch databases, click a different card: the app disconnects from the current one first, then connects to the new one.

> **Warning** Disconnecting clears the whole session state — the selected project, all filters, and the loaded graphs. It does *not* sign you out of the application; your identity, saved presets and personal settings are untouched.

Each signed-in user gets their own database session. Two users never share a live connection, so one person's queries and results can never leak into another's. Because the Exasol driver is synchronous, all of *your* queries are serialised: two long-running queries you start yourself queue behind one another, while queries from different users run in parallel.

Connection error messages

When a connect fails, the app deliberately shows guidance rather than driver internals — database codes, internal host names and SQL-state fragments go to the server log instead, under the operation `db-connect`.

Message	Usual cause
Authentication failed — check your username and password.	Wrong database credentials.
The database rejected the request.	An Exasol query error during login.
Connection timed out. Verify the host address, port, and network connectivity.	The 15-second connect timeout elapsed.
Connection refused. No server is accepting connections at this host and port.	Wrong port, or the database is down.
TLS certificate error. Try 'Skip verification' or configure a fingerprint in the server's security settings.	The server certificate was not accepted under the connection's certificate mode.
Host not found. Check the hostname spelling and your DNS / network connectivity.	Name resolution failed.
Connection failed. Check the host, port, and credentials.	Fallback for anything else.
This connection is not available to you.	The connection is no longer assigned to you (HTTP 403).
Connection not found.	The connection was deleted (HTTP 404).
This connection changed during sign-in. Please reconnect.	An administrator edited or deleted the connection while your session was being opened; the half-open session was dropped for safety.
Not connected to a database.	A data request was made with no live session (HTTP 409).

Note Users who may manage connections see richer text when they use **Test** in the Connection Editor: the same categories with the raw driver detail appended, and Exasol query errors rendered as “[code] message”. That extra detail is deliberately withheld from the ordinary connect path.

Who may manage connections

Requires: **Power user**, **Developer** or **Administrator** to create or edit connections. Every role may use the connections assigned to them.

Capability	User	Power user	Developer	Administrator
See and connect to assigned connections	✔	✔	✔	✔
＋ New connection / ✎ Edit	—	✔ own only	✔ own only	✔ all
Test, provision, generate demo content	—	✔ own only	✔ own only	✔ all
List and delete projects in a connection	—	✔ own only	✔ own only	✔ all
Sampling section in the sidebar	—	✔	—	✔

Ownership versus assignment

The two concepts are independent and it is worth keeping them apart:

Ownership

The user who created the connection becomes its **owner**. Ownership decides who may *manage* the definition — edit it, delete it, test it, provision it, generate demo data into it, and delete projects from it. A power user or developer owns only what they created; an administrator manages every connection regardless of owner.

Assignment

The list of users who may *use* the connection, ticked in the editor's **Assigned users** list. Assignment decides whose sidebar the connection appears in. It grants no management rights whatsoever.

When you create a connection you are made its owner and are auto-assigned to it, even if you leave your own name unticked in the Assigned users list — otherwise you could build a connection you are unable to open.

The backend enforces both gates independently and will answer with these messages verbatim: *“You are not allowed to manage connections.”*, *“You cannot edit this connection.”*, *“You cannot delete this connection.”*, *“You cannot manage this connection.”*, *“You cannot re-assign this connection.”* and *“Connection not found.”*

The Connection Editor

> Requires: **Power user**, **Developer** or **Administrator** (and ownership of the connection, unless you are an administrator)

The editor is a wide modal sheet marked with the ⛁ icon and titled either **New Connection** or **Edit Connection**. Pressing `Esc`, clicking ✕, or clicking outside the sheet closes it *without saving*. It has two or three tabs: **Database / LLM Details**, **Demo Content**, and — only when editing an existing, already-saved connection — **Projects**.

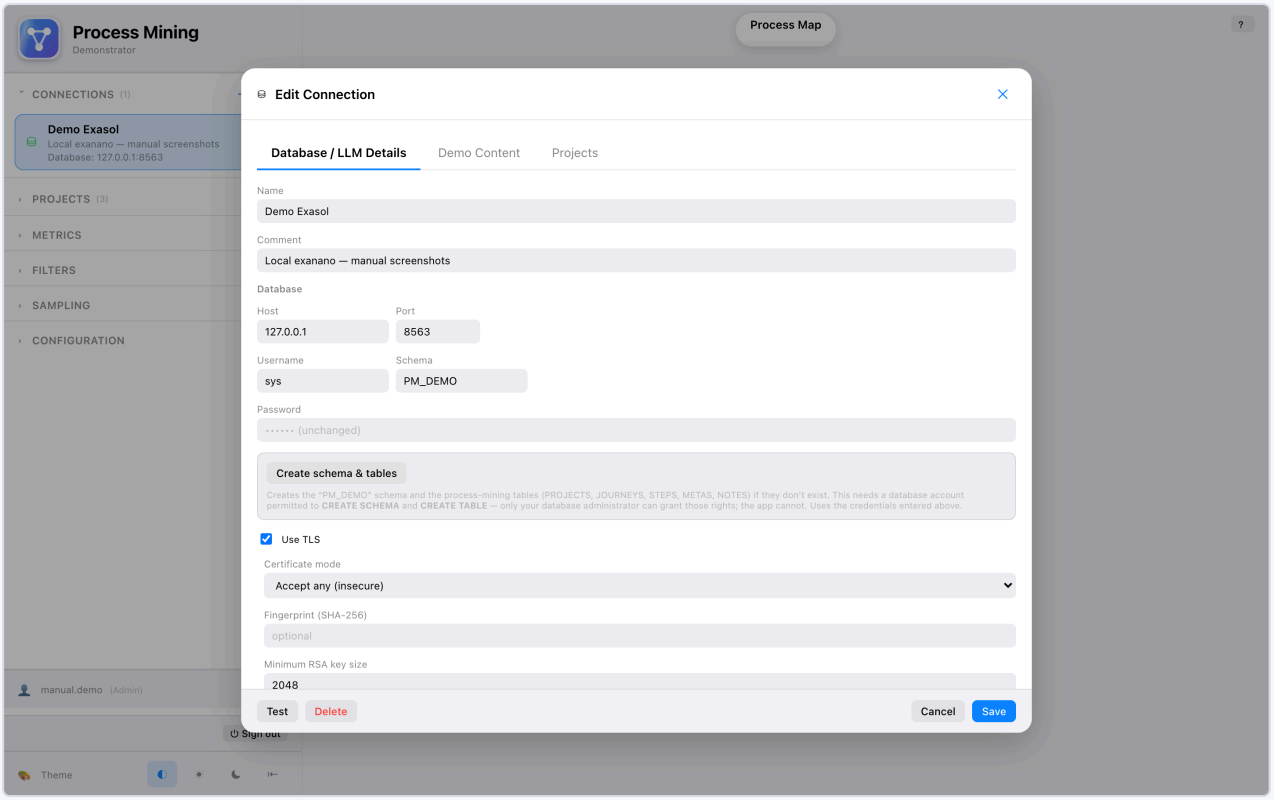


Figure 7.2 — The Connection Editor, Database / LLM Details tab.

The footer is visible from every tab and holds **Test**, the test result text, **Delete** (edit mode only, red), **Cancel** and **Save**. Save validates that a name has been entered; otherwise *"Name is required."* appears in red at the top of the sheet. Any error the backend returns is shown in the same place.

> **Warning** Saving any change to a connection immediately disconnects *every* user currently connected through it; they must reconnect against the new definition. The same applies when you change its assignments or delete it. This is intentional: a revoked assignment has to take effect at once, not at the end of somebody's session.

Database / LLM Details tab

Field	Default	Purpose and behaviour
Name	empty	Required. The display name every assigned user sees in their Connections list.
Comment	empty	Optional description line printed under the name on the card.
Host	empty	Exasol host name or address.
Port	8563	Numeric. 8563 is the Exasol default.
Username	empty	Database login name.
Schema	empty	The schema holding the process-mining tables. Optional for merely connecting, but *required* for provisioning, demo generation and the Projects tab.
Password	empty	Write-only secret — see below.
Use TLS	off	Enables encryption and reveals the three certificate fields.
Certificate mode	Verify (system trust store)	Also “Pin fingerprint” and “Accept any (insecure)”.
Fingerprint (SHA-256)	empty, placeholder “optional”	Used with the pin mode.
Minimum RSA key size	2048	Numeric, in bits. Stored and carried through every operation, but see the note below.
Server URL (LLM)	empty, placeholder “https://…”	Leave empty for no LLM; the app then hides all LLM indicators for this connection.
Model (LLM)	empty	The model name to request.
API key (LLM)	empty	Write-only secret — see below.
Assigned users	none ticked	Scrollable checkbox list of every enabled user account. Empty state: “No users available.”

Write-only secrets

The **Password** and **API key** fields never reveal what is stored. When a secret exists the placeholder reads **“•••••• (unchanged)”**. The rules are the same for both fields:

- Leave the field untouched and the stored secret is kept.
- Type a value and it replaces the stored secret.
- Deliberately clear the field to empty and the stored secret is *erased*.

> **Warning** **Test**, **Create schema & tables** and demo generation all authenticate with the password *typed into the sheet*, never with the stored one. On a saved connection with a blank password field, the database test runs with an empty password and normally reports an authentication failure even though the saved connection itself works perfectly. Re-enter the password before using any of those three buttons.

TLS and certificate modes

With **Use TLS** unticked, the database link is not encrypted at all. Ticking it reveals the certificate settings:

Certificate mode	Behaviour
Verify (system trust store)	Default. The server certificate must validate against the system trust store, and the host name must match.
Pin fingerprint	The driver pins the server certificate to the SHA-256 fingerprint you enter. Colons are stripped, and the remainder must be pure hexadecimal, 16 to 128 characters long.
Accept any (insecure)	Disables certificate verification *and* host name checking. For laboratory and self-signed setups only.

Mode handling fails closed. Any stored value the app does not recognise — a typo, a legacy record, restored data — is treated as **Verify**, never as "no verification". In the same spirit, selecting **Pin fingerprint** without a valid fingerprint refuses to connect rather than silently falling back:

```
This connection is set to pin a certificate fingerprint, but the
fingerprint is missing or not hexadecimal. Enter the server's SHA-256
fingerprint, or change the certificate mode.
```

> **Note** **Minimum RSA key size** is stored with the connection and carried through connect, test, provisioning, demo generation and rebuilds for compatibility with connection records created by the macOS edition, but the web backend does not currently enforce it. Do not rely on it as an active security control.

The LLM section

The LLM section is optional. When a **Server URL** is set, the app probes it at connect time and during every test by requesting `GET <serverURL>/models`, sending `Authorization: Bearer <API key>` when a key is stored, with a 5-second timeout. The server counts as reachable unless the request errors or answers with HTTP 500 or above — a 4xx still means a server is there. The user-visible failure text is always *"LLM server not reachable."*; the underlying cause is logged under the operation `llm-test`.

Because the URL is user-supplied it passes an SSRF guard before any request is made. Only `http://` and `https://` are accepted, the host must resolve, and every resolved address is checked; link-local and cloud-metadata addresses (such as 169.254.169.254), multicast, reserved and unspecified addresses are always refused. Loopback and private RFC 1918 addresses are permitted by default, because local model servers are a first-class use case. The outbound connection is pinned to the vetted address and redirects are disabled. A refusal is not shown as an error — the LLM simply reports as unreachable, and the reason is logged.

Testing a connection

1. Open the connection in the editor and go to the **Database / LLM Details** tab.
2. Re-enter the database **Password** (and the LLM **API key**, if you want the LLM half of the test to be authenticated).

3. Click **Test** in the footer.
4. Read the result text beside the button.

The result is colour-coded: **green** when everything tested succeeded, **orange** when one of two components failed, **red** when everything failed. The database half reads *“Database OK”* or *“Database: <error>”*. When an LLM URL is configured, the LLM half is appended after a “ · ” separator and reads *“LLM OK (N models)”* — the model count is omitted when it is zero — or *“LLM: <error>”*.

> **Tip** The test opens a throwaway database session and closes it again immediately. It never touches your own live session or anybody else's, so it is safe to run at any time — including while other users are working through the same connection.

Creating the schema and tables

> Requires: a database account permitted to **CREATE SCHEMA** and **CREATE TABLE**

The boxed panel on the Database / LLM Details tab holds the **Create schema & tables** button and this explanation:

> Creates the “<schema>” schema and the process-mining tables (PROJECTS, JOURNEYS, STEPS, METAS, NOTES) if they don't exist. This needs a database account permitted to CREATE SCHEMA and CREATE TABLE — only your database administrator can grant those rights; the app cannot. Uses the credentials entered above.

1. Enter **Host**, **Port**, **Username**, **Password** and **Schema**. The button stays disabled while the Schema field is blank or an operation is already running.
2. Click **Create schema & tables**.
3. On success a green line appears to the right of the button: *“Created: <list>”*, naming the schema and each table that was created.
4. On failure a red error line appears, with the fallback text *“Could not create the schema.”*

Provisioning connects *without* opening a schema (the target may not exist yet), then issues `CREATE SCHEMA IF NOT EXISTS`, `OPEN SCHEMA`, and the table definitions in the order PROJECTS, JOURNEYS, STEPS, METAS, NOTES. Everything uses `IF NOT EXISTS`, so the operation is idempotent and re-running it never disturbs existing data. If a step fails part-way, the reported list still names everything that was created *before* the failure, so you can see how far it got. A blank schema name produces *“A schema name is required.”*

> **Note** The schema identifier is quoted safely, so mixed-case and special-character names work — but remember that a quoted identifier is case-sensitive in Exasol. Type the schema name exactly as it must exist.

> **Warning** Because provisioning uses `IF NOT EXISTS`, it never alters a table that is already there. If JOURNEYS already existed before the app saw it, the performance layout the app would have declared (`DISTRIBUTE BY EVENT_ID`, `PARTITION BY EVENT_TIME`) is not applied. A database administrator must apply it manually with `ALTER TABLE JOURNEYS DISTRIBUTE BY EVENT_ID;` and `ALTER TABLE JOURNEYS PARTITION BY EVENT_TIME;`

Generating demo content

> Requires: a database account permitted to **CREATE SCHEMA**, **CREATE TABLE** and **INSERT**

The **Demo Content** tab loads a complete, realistic dataset into the connection's schema so that you can explore every feature of the app without waiting for a customer extract. It creates the schema and tables if they are missing, then writes the journeys. It uses the credentials on the Database / LLM Details tab — including the typed password.

Three datasets are offered, grouped under the headings **Retail**, **Finance/Insurance** and **Transportation**:

Dataset	Project	What it contains	Meta attributes
📚 Online Bookstore	BOOKSTORE	Order lifecycle: login → browse → basket → checkout → payment → fulfilment → delivery, with a returns flow (about 5%) and a deliberately flaky bank-transfer path (35% failure per processing attempt, with a retry loop). 24 steps in the groups Customer Interaction, Purchase Process, Payment, Fulfilment and Returns; scores from –10 for Payment Failed and Return Initiated up to +15 for Delivered.	Payment Method / Customer Segment / Order Value
💶🎱 Online Credit Application	CREDIT	Bank or affiliate intake → application check with a rework loop of at most two iterations → credit assessment → score-band decision (below 75% auto-reject, 75–90% agent review with a 50% rejection rate, above 90% auto-accept). Sums over 10,000 EUR need Senior Agent Approval, which declines 5%. Accepted applications end in payment, which takes up to about seven days — larger sums are slower, the affiliate channel faster.	Applied Credit Sum / Income Class / Channel
✈️ Flight Booking & Management	FLIGHTS	Star Alliance-style booking: login → search (with a modify loop of at most two iterations) → select → book → payment → confirm. 80% are new bookings, of which half are interline itineraries that query a partner airline's system; the remaining 20% only manage an existing booking (seat reservation or ancillary services), and 60% of those pay. Payment methods: Credit Card, SEPA, Apple Pay, Google Pay, Advance Payment.	Journey Type / Airline / Payment Method

1. Open the connection in the editor, re-enter the database **Password** on the Database / LLM Details tab, then switch to **Demo Content**.
2. Check the **Schema** field in the dataset panel. It is the *same value* as the Schema field on the details tab — editing it in any demo panel changes the connection's schema draft everywhere.
3. Set **Journeys**. The default is 500; the field accepts 1 to 20,000 and is typed rather than stepped.
4. Click **Generate**. The button reads “Generating…” with a spinner while it runs and is disabled while busy, while the schema is blank, or while the journey count is below 1.
5. Read the result line: green for success, with the fallback text *“Created N journeys.”*, or red for failure, with the fallback *“Could not generate demo content.”* The full success message is `Created N journeys for "<title>" in <SCHEMA>.JOURNEYS`.

6 Connect to the database and open the new project from the **Projects** section.

Event times are spread across a **rolling two-year window ending on the generation day** (today back to two years ago), so freshly generated data always includes recent activity — the Dashboard sparklines and the journeys-over-time chart are never empty. Each EVENT_ID is the MD5 hash of a synthetic key — `ORD-000001`, `ORD-000002`, … for the bookstore, `CRA-000001` … for credit applications, and `FLT-000001` … for flights — which is why event identifiers look like long hexadecimal strings in the Individual Journey view and in its autocomplete suggestions.

What generation touches	Effect
Schema and the five tables	Created if missing; existing objects untouched.
PROJECTS and METAS rows of this dataset	Deleted and re-inserted.
JOURNEYS rows of this dataset's project	All deleted and re-inserted — *including any sample sets* built from them.
STEPS rows	Only added where missing, so colours, shapes, scores, groups and step notes you customised in the Step editor survive a regeneration.
Other projects in the schema	Not touched at all.

Warning The in-memory datasets are hard-capped at 20,000 journeys on the server and a larger request is clamped *silently* — ask for 50,000 and you get exactly 20,000 with a success message. (The streamed Airport Passenger Flow dataset allows far more.) Rows are inserted in batches per statement and committed in transactions of about 25,000 rows, with autocommit off; a load that fails part-way is rolled back so the project is never left half-written.

Other error messages you may see: *“Unknown demo dataset '<x>'.”*, *“A schema name is required.”* and *“Enter how many journeys to generate.”* (when the count is below 1), plus the ordinary connection errors listed in section 7.2.

Note If the connection uses pre-materialised transitions, generated data does *not* rebuild the TRANSITIONS_RAW table. Until an administrator rebuilds it, the chart header shows the pill ⚠ **Live (not built)** and the map is computed live. That is a performance matter only — the results are identical.

Keeping a process live — the demo-event generator

The Demo Content tab loads a fixed batch of history. For a *live* demo — journeys trickling in while you watch — there is a separate standalone tool, `generator/launch.py`, run from a shell (or in Docker) rather than from the app. It **learns an existing project's structure** — the same Markov directly-follows model the Simulation view fits (transition probabilities, per-edge timing, entry and end steps) plus the project's META value mix — and then, every few seconds, writes **one full synthetic journey** into a target project's `JOURNEYS`. Point an A-Chart with **Auto-refresh** on (chapter 9), or the Dashboard, at that project and you watch data arrive in real time.

```
# every 5–60 s, append a journey shaped like project 1 of schema PM_PROD, into project 1
python generator/launch.py --host 127.0.0.1 --port 8563 --user sys --password exasol \
    --schema PM_PROD --project 1 --insecure --min 5 --max 60

# write into a different project, faster, and stop after 100 journeys
python generator/launch.py … --project 1 --target-project 2 --min 2 --max 8 --count 100
```

Option	Meaning
`--schema` / `--project`	The source project to **learn** the structure from (and, by default, write into).
`--target-project`	Write the journeys into a *different* project than the one learned from. Default: the same project.
`--min` / `--max`	Seconds between journeys, drawn uniformly in that range (default 5–60).
`--max-steps`	Safety cap on a single journey's length — important when the learned graph has loops (default 60).
`--count`	Stop after N journeys; `0` (the default) runs forever until Ctrl-C / SIGTERM.
`--tls` / `--no-tls` / `--insecure` / `--fingerprint`	TLS to Exasol, mirroring a connection's certificate mode.
`--seed`	Deterministic RNG, for a reproducible demo.

Every journey is anchored so its **last event is "now"** (its events run up to the present, never into the future), so an A-Chart whose auto-refresh window slides to today shows the whole just-finished journey immediately. The tool needs the same database rights as any writer (`INSERT`), draws its connection details from the command line or the `PMG_*` environment variables, logs each journey it emits, retries a transient database error on the next tick, and stops cleanly on Ctrl-C. Run `python generator/launch.py --help` for the full list of options.

> **Note** It is a **demo / load-shaping** tool: the generated journeys are synthetic, tagged `ORIGINAL` like real data, and append to the project — they are not de-duplicated and there is no undo other than deleting the project's journeys. Learn from, and write into, a demo or sandbox project, not a project holding real customer data.

Managing the projects in a connection

> Requires: **Power user**, **Developer** or **Administrator**, plus ownership of this connection (administrators may manage all)

The **Projects** tab exists only when you are editing a connection that has already been saved; a brand-new, unsaved connection has no Projects tab. It reads the schema with the connection's *stored* credentials — this is the one operation in the editor that does not need you to retype the password.

The tab's description reads: *"Projects stored in the "<schema>" schema, with their journey and event counts. Deleting a project clears its rows from every table (PROJECTS, JOURNEYS, STEPS, METAS, NOTES, TRANSITIONS_RAW) — this cannot be undone."*

The list loads automatically the first time you open the tab and can be reloaded with ↻ **Refresh**. While loading it shows “Loading projects…” with a spinner; when the schema holds nothing it shows *“No projects found in this schema.”*; on failure it shows the backend message in red, or the fallback *“Could not read the projects for this connection.”*

Each row shows the project **title** — with the raw project ID printed underneath when the two differ — then “**N journeys**”, “**N events**”, and a red **Delete** button.

> **Note** The counts consider ORIGINAL data only; sample sets are excluded. “Journeys” is the number of distinct EVENT_IDs and “events” is the number of rows. A project is listed if it appears in *either* PROJECTS or JOURNEYS, so orphaned rows on one side are still visible. A project with no title displays its PROJECT_ID instead. Missing tables are tolerated: before provisioning you get an empty list rather than an error.

Deleting a project

1. Open the connection in the editor and switch to the **Projects** tab.
2. Confirm the counts on the row — they are your last sanity check on which project you are about to erase.
3. Click **Delete** on that row.
4. Read the confirmation dialog **Delete project?**: *“<title>” (N journeys, M events) will be cleared from PROJECTS, JOURNEYS, STEPS, METAS, NOTES and TRANSITIONS_RAW in this schema. This cannot be undone.*
5. Click **Delete project** to proceed, or **Cancel** to abandon. A failure shows the backend message or *“Could not delete the project.”*

The deletion runs one `DELETE … WHERE PROJECT_ID = '<id>'` per table, children before parents, in this order:

Order	Table	What is removed
1	JOURNEYS	Every event row of the project — *including every sample set*, because no SAMPLE_SET filter is applied.
2	STEPS	All step presentation and scoring rows: colours, scores, shapes, groups, step notes.
3	METAS	The project's meta-attribute titles.
4	NOTES	Every note attached to a node or edge of the project, from every author.
5	TRANSITIONS_RAW	The pre-materialised transition pairs, when that optional table exists.
6	PROJECTS	The project row itself.

Tables that do not exist are skipped rather than treated as errors — NOTES may never have been created, and TRANSITIONS_RAW is optional — and the response reports exactly which tables were actually cleared, together with the project's pre-deletion counts. Blank inputs are refused with *“A schema name is required.”* or *“A project id is required.”*, and a permission failure with *“You cannot manage this connection.”* or *“Connection not found.”*

Warning Project deletion is permanent. There is no undo, no recycle bin and no automatic backup, and it takes every note written about that process with it. If the notes matter, export or copy them first. Every deletion is written to the administration audit log, recording who deleted which project from which connection.

Assigning users to a connection

Requires: **Power user**, **Developer** or **Administrator** (owner of the connection, or any administrator)

1. Open the connection with ✎ **Edit connection**, or create a new one with **+**.
2. Scroll to the **Assigned users** section at the foot of the Database / LLM Details tab.
3. Tick every user who should see and use this connection. The list contains every enabled account; if there are none it reads “No users available.”
4. Click **Save**.

Assignment changes take effect immediately, in both directions. A newly ticked user finds the connection in their sidebar on their next refresh. A user you untick is disconnected on the spot: their live session is dropped and the connection disappears from their list. Attempting to reconnect to it produces *“This connection is not available to you.”*

Tip Because saving an edit kicks everyone off the connection, batch your changes. Adjust the endpoint, the LLM settings and the assignment list in one editing session and save once, rather than saving after each individual change.

Deleting a connection

1. Open the connection with ✎ **Edit connection**.
2. Click the red **Delete** button in the sheet footer.
3. Read the confirmation **Delete connection?**: *“<name>” will be removed for all assigned users. This cannot be undone.*
4. Click **Delete** to confirm, or **Cancel**. A backend failure shows “Delete failed.”

Deleting a connection first drops every live session on it, then removes the definition — including its stored secrets and, where one exists, its scripted-rebuild token and materialisation status.

Note Deleting a connection removes the app's *definition* of the database. It does not touch the database itself: the schema, its tables and all their data remain exactly as they were. To erase data you must delete the projects first, from the Projects tab described above.

Summary of destructive operations

Operation	Confirmation?	Reversible?	Scope
Disconnect	No	Yes — reconnect	Clears your session state only.
Save an edited connection	No	n/a	Disconnects all its users.
Un-assign a user	No	Yes — re-assign	Drops that user's live session at once.
Generate demo content	No	No	Replaces that dataset's project, its journeys and its samples; keeps customised STEPS rows.
Delete a project	Yes	No	Clears six tables, including all notes and all samples.
Delete a connection	Yes	No	Removes the definition for all users; database contents untouched.

8 Filters, Metrics, Sampling and Configuration

> This chapter is the reference for the four sidebar sections that shape what a chart shows and how it looks: **Metrics** (which transition figure the edges carry), **Filters** (which journeys are counted at all), **Sampling** (which data source each chart reads from) and **Configuration** (display settings, KPI layout, step properties and the AI prompt). Every user needs the Filters and Metrics sections; Sampling is restricted to power users and administrators. Each control is listed with its exact on-screen label, its default and — where the behaviour is easy to misread — the reason it works the way it does.

8.1 Where these sections live

All four sections are accordion panels in the sidebar, in this fixed order: **Connections**, **Projects**, **Metrics**, **Filters**, **Sampling**, **Configuration**. Only one section is open at a time — opening one collapses the others. On start-up the **Connections** section is open; selecting a project collapses every section so the chart gets the full height.

Two sections change with the active view mode:

- Switching to **Individual Journey** hides **Metrics** and **Sampling** entirely and automatically opens **Filters**, so that the Event ID field is immediately at hand. The automatic opening happens once per mode change; you can collapse the section again afterwards.
- Switching to **AI supported Documentation** replaces the filter controls with a notice — that view always analyses the A-Chart's filters.

> **Note** Each view mode keeps its own filter state. Switching modes saves the state you were editing and restores the target mode's state. There are two exceptions: **Statistics** always inherits the A-Chart's filters, and **A/B Comparison** maps onto the saved A-Chart and B-Chart states, with side A active first.

8.2 The Metrics section

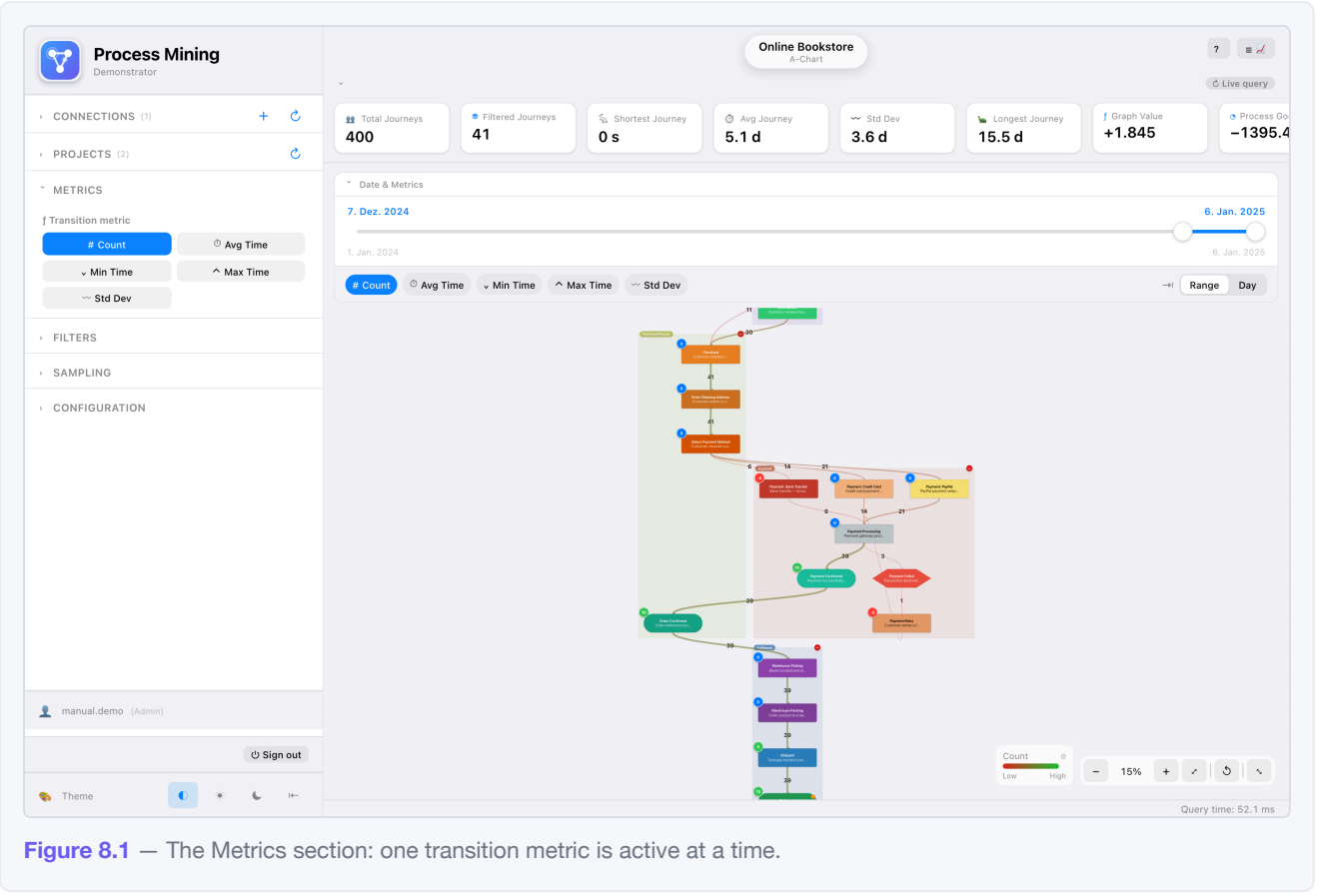


Figure 8.1 — The Metrics section: one transition metric is active at a time.

The section header reads ***ƒ* Transition metric** and contains a two-column grid of chips. The chips are disabled until a project is selected. The same five chips also appear in the **Date & Metrics** card above each chart, so you can change the metric without opening the sidebar.

Chip	Meaning on an edge	Label format
# Count	Number of times the transition was traversed across the filtered journeys.	Compact count, e.g. "1.2k", "3.4M"
% Percentage	The share of journeys *leaving the source node* that take this edge — the branching probability at that step. Every node's outgoing edges sum to **100 %** (a node with a single outgoing edge shows 100 %).	Percent, e.g. "70%" / "3.5%"
% Journey % (default)	The share of *all filtered journeys* that traverse this edge (edge count ÷ total filtered journeys). The denominator is the whole map rather than the source node, so it shows how common an edge is overall. A value can exceed 100 % for an edge a single journey repeats.	Percent, e.g. "50%" / "0.4%"
⏱ **Avg Time**	Mean elapsed time between the two steps.	"Ns", "Nm", "N.nh", "N.nd"
½ **Median Time**	Median elapsed time between the two steps — the middle value, computed with the database's `MEDIAN` function. Unlike the mean it is not pulled by a few very slow or very fast outliers, so it is often a better "typical" duration.	as above
⌄ **Min Time**	Shortest observed elapsed time for that transition.	as above
⌃ **Max Time**	Longest observed elapsed time.	as above
〰 **Std Dev**	Standard deviation of the elapsed times — a measure of how erratic the transition is.	as above

Where a time value is unavailable the label shows an em dash ("—"). Hovering an edge label shows "`<from> → <to> · <occurrences>`", so the traversal count is always available even when a time metric is selected.

8.2.1 How the metric is drawn

The chosen metric drives three visual channels at once:

Thickness

Edge width runs from 3 px to 22 px, scaled linearly against the largest value among the visible edges (for **Percentage** and **Journey %** the scale is a fixed 0–100 %). The arrowhead scales with the width.

Opacity

Alpha runs from 0.3 to 0.8 on the same linear scale, so weak transitions recede.

Colour

When **Colorise edges by weight** is on (see §8.5.4) the edge is tinted along the gradient configured for that metric; otherwise it is grey.

Each metric has its own colour schema, configured in the **Edge Colour Schema** sheet (open it by clicking the colour-scale legend on the canvas) and stored separately per metric. The built-in defaults are:

Metric	Default schema	Gradient
Count	Green (high is good)	red → green
Percentage	Green (high is good)	red → green
Journey %	Green (high is good)	red → green
Avg Time	Orange scale	light → dark orange
Median Time	Orange scale	light → dark orange
Min Time	Blue scale	light → dark blue
Max Time	Red (low is good)	green → red
Std Dev	Purple scale	light → dark purple

8.2.2 Metrics and nodes

Nodes do not carry the transition metric themselves. What a node shows is its own step definition — name, colours, shape, optional description, an orange end-of-process dot, a note badge, and a **score** badge when the step has a score (green above zero, red below, blue at zero). Node *visits*, used by the Graph Value KPI, by Happy Path coverage and by the Statistics step-traffic chart, are computed as `max(incoming, outgoing)` occurrences — not as the sum of both, which would double-count pass-through steps.

> **Warning** When a step group is collapsed, its edges are merged: connection counts are **summed** and times are **weighted-averaged**, and transitions inside the group disappear. Median Time, Min Time, Max Time and Std Dev cannot be reconstructed for a merged edge, so those four metrics show "—" on it. A grey hint at the bottom of the canvas repeats this while any group is collapsed.

> **Note** In **A/B Comparison**, changing the metric applies it to the side you are currently editing (A or B), not to both. In **Individual Journey** the metric selector is hidden and the edge metric is forced to **Avg Time**, because a count is meaningless for a single traversal. In **Conformance Check** the metric chips select which metric's norms you are viewing or editing.

8.3 The Filters section

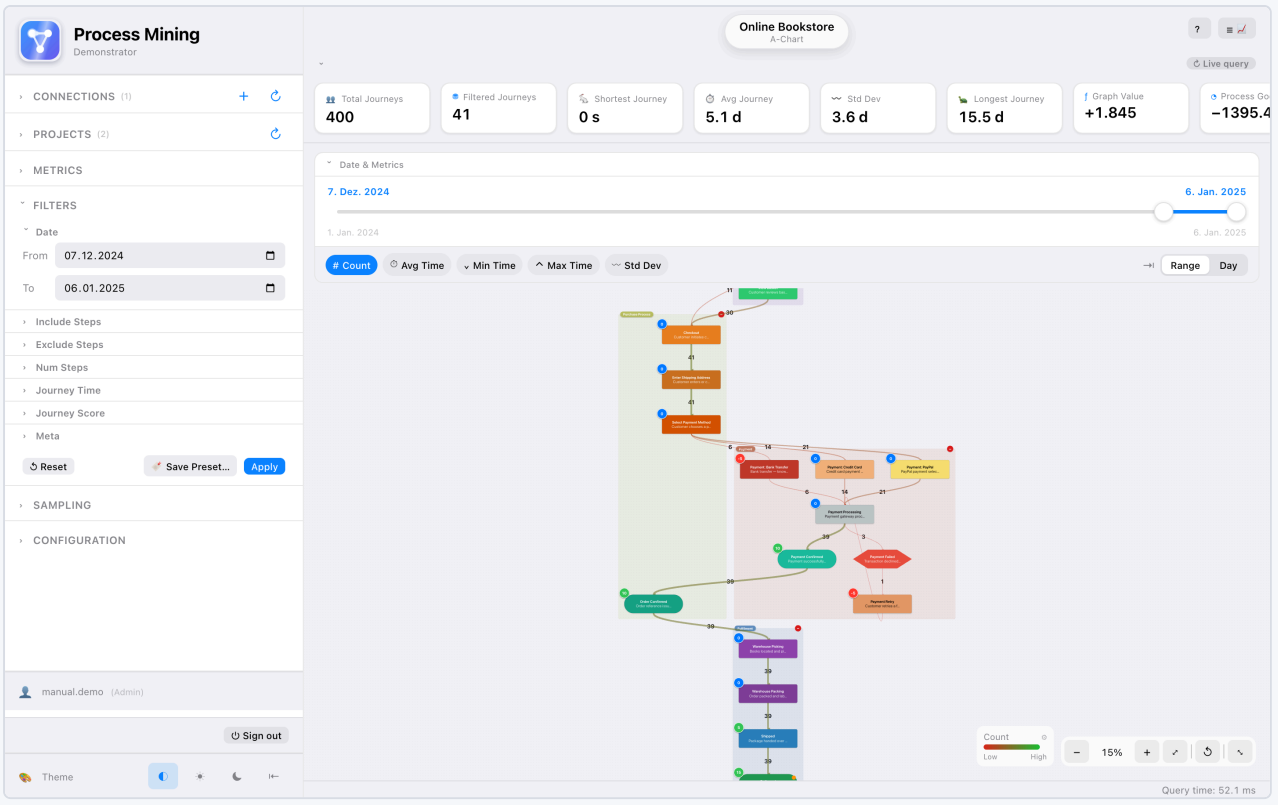


Figure 8.2 — The Filters section with its collapsible sub-sections, action row and preset list.

Every filter sub-section has its own collapsible header. The expansion state is remembered per user. Out of the box only **Date** is expanded. A sub-section that is currently narrowing the data shows a **badge** with its present value and a ⊗ button (tooltip "Clear <Title>") that resets just that sub-section.

8.3.1 The filter controls

Sub-section	Control	Badge	Disabled when
Date	"From" and "To" date pickers	—	no project
Include Steps	Searchable step list, radio-style rows (◉ selected / ○ not)	Number of included steps	no project
Exclude Steps	Identical, mirrored list	Number of excluded steps	no project
Num Steps	Dual-handle range slider over the project's min/max step count	e.g. "2–14"	no project, or min equals max
Journey Time	Range slider with human-readable durations (e.g. "3m 20s")	Formatted range	no project, or degenerate bounds
Journey Score	Range slider over the project's score bounds	Formatted range	no project, or degenerate bounds
Meta	Up to three autocomplete combo fields, labelled with the project's meta titles	Number of meta filters set	no project

The step lists carry a search field with the placeholder **Search** and a ⊗ button (tooltip "Clear search"). A step already present in **Exclude Steps** appears disabled in **Include Steps** and vice versa — a step can never be required and forbidden at the same time. The lists show "*No project selected*" when no steps are loaded and "*No matches*" when the search finds nothing.

The **Meta** sub-section appears only if the project defines at least one meta attribute title. Each field uses the placeholder **All values**, which is also its cleared state. Focusing a field opens the full list of distinct values (at most 50 entries are shown; the list scrolls); typing narrows it. The ▾ button (tooltip "Show values") opens and closes the list, ⊗ (tooltip "Clear") empties the field, **Enter** confirms the typed value and **Escape** closes the list. The ⊗ on the sub-section header clears all three fields at once.

8.3.2 How filters combine and when they take effect

The sub-sections narrow the journey set together: a journey must fall inside the date window, contain every step listed under **Include Steps**, contain none of the steps listed under **Exclude Steps**, and fall inside the Num Steps, Journey Time and Journey Score ranges as well as matching any meta values you have set. Clearing a sub-section removes that condition without touching the others.

Edits you make in the sidebar are **staged**, not live. They reach the chart when one of the following happens:

- You press **Apply**.
- You press ↺ **Reset** — the reset is applied immediately.
- You click a preset, in the sidebar list or in the chart header's preset menu.
- You use a node's context actions ✓ **Require in journeys** or ⊖ **Exclude from journeys** on the process map — these write straight into the include/exclude lists (removing the step from the opposite list) and reload the graph at once.
- You release a handle of the journey time slider above the chart. Dragging only previews; the query fires on release.

> **Tip** In **Statistics** mode the filter controls look and behave normally, but **Apply** recomputes the statistics view instead of the graph. Statistics always starts from the A-Chart's filters, and shows an orange "⚠ Filters changed since last load" banner if you change them afterwards.

8.3.3 The action row

↺ Reset

Restores every filter to the project's initial values — the initial date window, empty include and exclude lists, empty meta fields and full range bounds — and reapplies immediately.

Save Preset…

Opens the **Save Filter Preset** sheet.

Apply

Reloads the graph (or the statistics view) with the staged filters.

All three buttons stay disabled until a project is selected.

8.3.4 Filter presets

A preset captures the entire current filter state — dates, include and exclude step lists, meta values and all three range bounds — under a name.

1. Set the filters you want to keep.
2. Press **Save Preset…**. The **Save Filter Preset** sheet opens with the message "Saves the current filter settings as a named preset."
3. Type a name in the **Name** field. Leaving it empty is allowed — the preset is then auto-named "Preset N".
4. Press **Save**, or simply press Enter. **Cancel** discards.

Saved presets appear under the action row beneath the heading **Presets**, sorted alphabetically (A→Z). Four rows are visible at a time; more scroll. The same list, in the same order, is available from the preset menu button in each chart's **Date & Metrics** header.

1. Click a preset name to apply it — the filters are replaced and the graph reloads immediately. The applied row is highlighted.
2. Press ✎ (tooltip "Rename preset") to open the **Rename Filter Preset** sheet, which is prefilled with the current name; confirm with **Rename**.
3. Press 🗑 (tooltip "Delete preset") to remove it.

> **Warning** Preset deletion is immediate — there is **no confirmation dialogue**, either in the sidebar list or in the chart header's preset menu. Re-creating a deleted preset means rebuilding its filter state by hand.

> **Note** Presets are stored in the backend settings store **per user and per project**. They follow you to another browser or machine and are included in backups, but they are private to you — unlike sample sets, which are shared with everyone on the same project database.

8.3.5 The Event ID filter (Individual Journey)

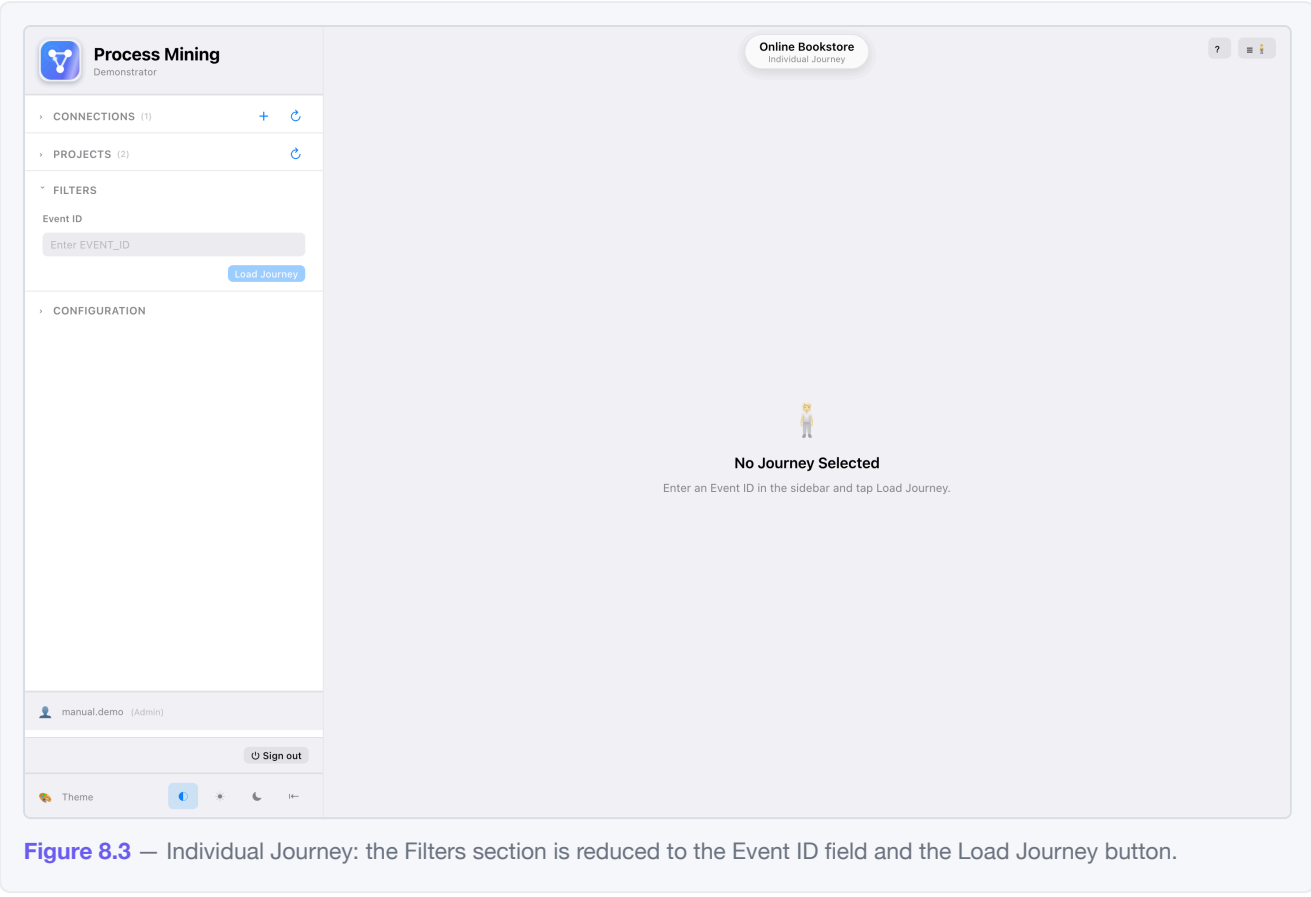


Figure 8.3 — Individual Journey: the Filters section is reduced to the Event ID field and the Load Journey button.

In **Individual Journey** mode the Filters section shows a single control labelled **Event ID**, a text field with the placeholder **Enter EVENT_ID**. Autocorrect, autocapitalisation and spellcheck are switched off so that identifiers are not mangled.

1. Type at least two characters. The app fetches matching event IDs from the current project and the active A-side sample set and shows them in a dropdown while the field has focus. Typing fewer than two characters clears the suggestions.
2. Click a suggestion — the field is filled and the journey loads at once — or finish typing the identifier yourself.
3. Press **Enter** in the field, or press the **Load Journey** button on the right. The button is disabled while the field is blank or no project is selected.
4. Press ⊗ (tooltip "Clear") to empty the field and the suggestion list.

Errors from loading appear in the app's standard error area, not inside the sidebar. If nothing matches, the canvas shows "🔍 **Journey Not Found** — No steps found for "<entered id>". Queried hash: <hash>", which gives support staff the exact value that was looked up.

> **Tip** In the bundled demo datasets an EVENT_ID is the MD5 hash of a key such as "ORD-000001", "CRA-000001" or "FLT-000001", so the suggestions are hexadecimal strings. Type a few hex characters and pick from the list rather than trying to guess an identifier.

Note The **Notes** view is not governed by the sidebar filters at all; it has its own filter bar (type, importance, status, user, time, sort, grouping and page size). Notes do, however, record the filter context that was active when they were created, shown in the note editor as "Filter context: …".

8.4 The Sampling section

Requires: **Power user** or **Administrator**

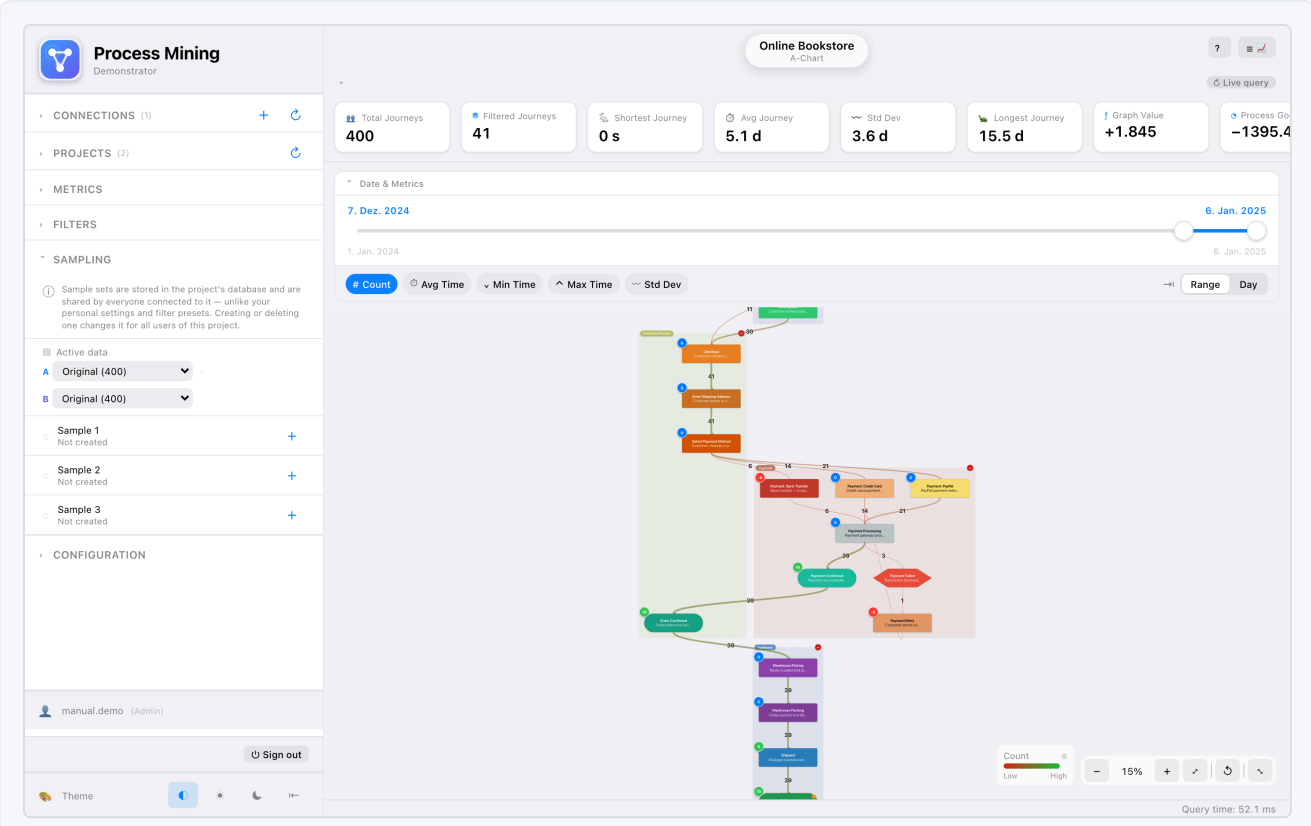


Figure 8.4 — The Sampling section: active data pickers for A and B above the three sample slots.

Sampling lets you work on a representative subset of a large project so that charts, statistics and simulations respond quickly, and it selects which data source each chart side reads from. The section is hidden from plain users and from developers, and it is hidden in Individual Journey mode.

Warning The section carries this notice at the top: "*Sample sets are stored in the project's database and are shared by everyone connected to it — unlike your personal settings and filter presets. Creating or deleting one changes it for all users of this project.*" Creating or deleting a sample is therefore not a private action — agree the use of the three slots with your colleagues before overwriting one.

8.4.1 Active data (A and B)

Under the heading ▤ **Active data** sit two dropdowns, labelled **A** (accent blue) and **B** (purple). Each chooses the data source that side's chart reads from. Both are disabled until a project is selected.

Option	Shown as	Availability
Original	"Original (N)" — N = total journey count	always selectable
Sample 1 / 2 / 3	"Sample 1 (1,000 · Random)"	disabled and labelled "Sample 1 — not created" until the set exists
Sim-A / Sim-B	"Sim-A (N sim)"	disabled and labelled "Sim-A — not loaded" until a simulation has been run into that slot

Your choice is stored per user (settings `sampling.activeSampleSetA` and `sampling.activeSampleSetB`, both defaulting to `ORIGINAL`) and the affected chart reloads at once. If a persisted choice points at a sample set that no longer exists when the project loads, that side silently falls back to **Original**.

> **Warning** Choosing a simulation slot makes that side **read-only**. The canvas shows "<slot>: simulation data is read-only — interactive filtering, drill-down and step editing are not available.", and the node actions ✓ **Require in journeys** and ⊖ **Exclude from journeys** are hidden.

8.4.2 The three sample slots

Below the pickers are three rows, **Sample 1**, **Sample 2** and **Sample 3**. Each row shows ◉ in the accent colour when the set exists and ○ when it does not; the title is bold and accent-coloured while that set is the active source on A or B. A created set shows "N journeys" plus a method tag (icon and label, tooltip "Sampling strategy: <Method>"); an empty slot shows "Not created".

The row buttons are **+** (tooltip "Create Sample N") on an empty slot — disabled while no project is selected or another sampling operation is running — and 🗑 (tooltip "Delete Sample N") on a filled one. While an operation runs, a spinner row shows the progress text ("Preparing…", "Writing N journeys…", or "Working…"); errors appear in red beneath the pickers.

8.4.3 Creating a sample set

1. Press **+** on an empty slot. The sheet **Create Sample Set N** (icon ▤) opens and reports "N journeys available" for the original data.
2. Type the size in the **Journeys** field. The default is **1000**. The value is typed freely; **Create Sample** stays disabled unless it is a positive whole number.
3. Choose a **Sampling Method** card (see the table below). The method you used last is remembered per user and becomes your default; before that the default is **Random**.
4. Press **Create Sample**. A spinner runs in the button; **Cancel** and closing the sheet are disabled while the sample is being written.
5. On success the sheet closes and the slot fills. On failure the sheet stays open and shows "⚠ <error>" so you can adjust and retry.

Method	Description	Use it when
🔀 **Random** (initial default)	Uniform random selection	You want an unbiased miniature of the whole project.
📅 **Temporal Stratified**	Proportional across time periods	Volume varies strongly over time and each period must stay represented.
🌿 **Path Diversity**	Coverage across journey variants	You care about rare routes, which uniform sampling would drop.

8.4.4 Deleting a sample set

1. Press 🗑 on the slot. The confirmation **Delete Sample Set N?** appears with the message "This removes <count> sample journey rows from the database. Original data is not affected."
2. Press **Delete** (red) to confirm, or **Cancel** to keep the set.

> **Note** If the deleted set was the active source on side A or B, that side automatically switches back to **Original** and the chart reloads. The original journeys are never touched by sampling — only the generated sample rows are removed.

8.5 The Configuration section

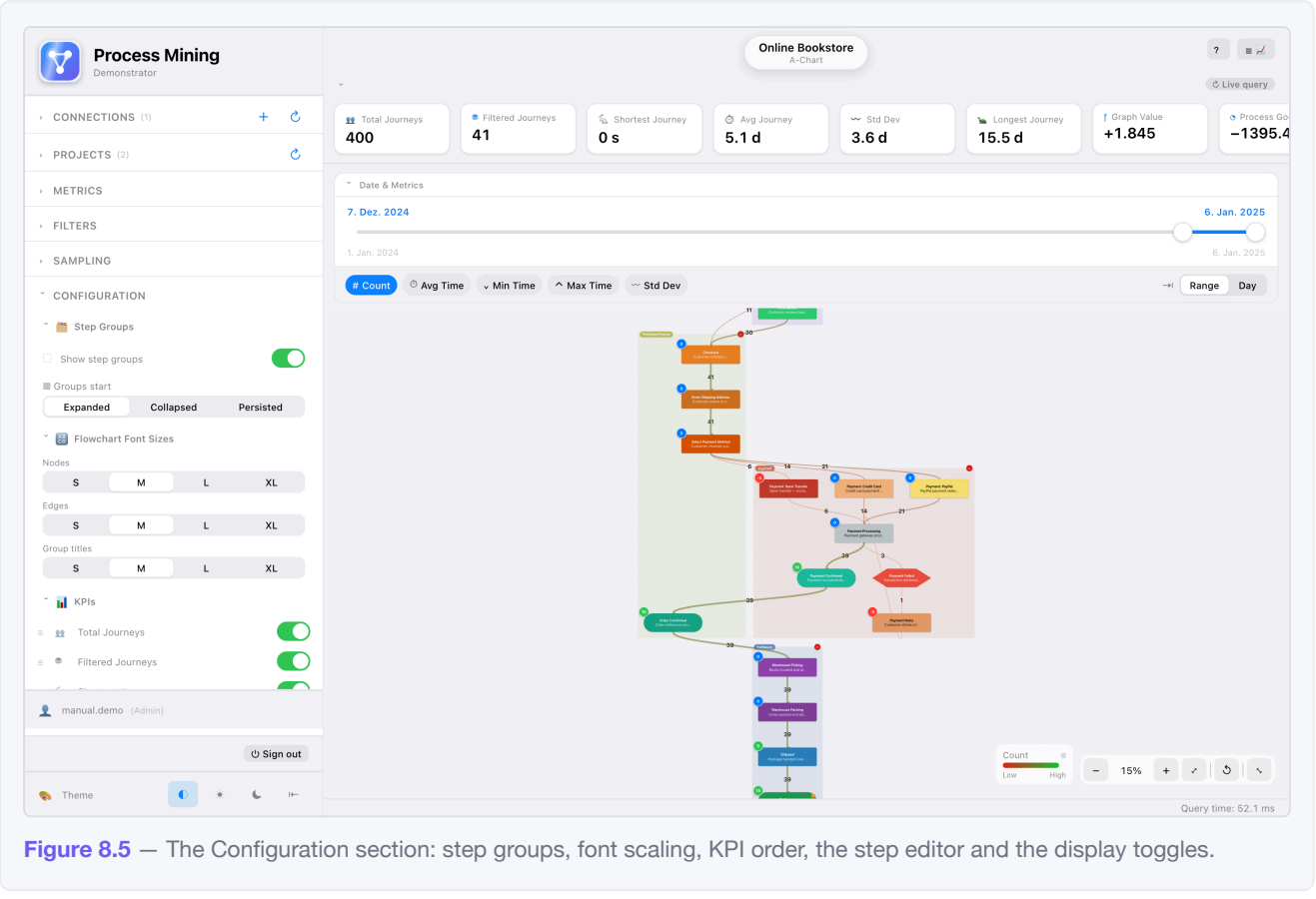


Figure 8.5 — The Configuration section: step groups, font scaling, KPI order, the step editor and the display toggles.

Configuration holds display settings, the KPI strip layout, the step editor and the AI prompt. Its six collapsible sub-headers — **Step Groups**, **KPIs**, **Steps**, **Layout**, **Dates and Times** and **AI related** — are all expand-

ed by default, and their expansion state is remembered per user. Every setting on this page is stored per user in the backend settings store, so it follows you between browsers and machines; the exception is the **Steps** editor, which writes shared step properties to the project database.

8.5.1 🗂️ Step Groups

Show step groups

Toggle, default **on**. Draws a dashed tinted box with a coloured name pill around every set of steps sharing a BELONGS_TO value.

▦ Groups start

Segmented control, visible only while the toggle is on: **Expanded** (default) | **Collapsed** | **Persisted**. It decides whether group containers on the process map start open, start closed, or restore the state you last left them in for that project and chart mode.

8.5.2 📊 KPIs

This is a drag-to-reorder list of the ten KPI tiles shown in the strip above the charts. Each row has a ≡ handle, an icon, a label and an on/off switch. **All ten default to on**, in this order:

#	Tile	What it reports
1	👥 **Total Journeys**	Unfiltered journey count of the project
2	⛃ **Filtered Journeys**	Journeys matching the current filters
3	🐇 **Shortest Journey**	Minimum journey duration
4	⏱ **Avg Journey**	Mean journey duration
5	◑ **Median Journey**	Median journey duration (the database `MEDIAN`) — the typical duration, unaffected by outliers
6	〰 **Std Dev**	Standard deviation of journey durations
7	🦕 **Longest Journey**	Maximum journey duration
8	ƒ **Graph Value**	Σ (step score × visits) over scored nodes, shown with an explicit sign. Hidden when no step carries a score.
9	◔ **Process Goodness**	Server-computed goodness score, signed, two decimals. Hidden when null.
10	⇄ **Process Similarity**	A/B similarity between 0 and 1. Shown only in A/B Comparison, and only once a similarity has been computed — never on a single-process chart.
10	▤ **Active Sample**	The side's data source ("Original", "Sample 1"–"Sample 3", "Sim-A", "Sim-B") and its journey count.

1. Grab a row by its ≡ handle and drag it onto another row to change the order. The new order persists per user.
2. Use each row's switch to hide or show that tile.

The same two actions are also available **directly on the KPI strip** above the chart, so you rarely need to open this list. Drag any tile onto another to reorder it, and hover a tile to reveal a **×** in its top-right corner that hides it. The strip and this list share one order and one set of on/off flags: change either place and the other updates at once, and both persist per user. A tile you hide with its × is switched back on from this list.

> **Note** Order and visibility apply to the KPI strip above A-Chart, B-Chart and the A/B panels — reorder or remove from either the strip or this list. Individual Journey renders its own fixed tiles (Date, Duration, Sum of Scores, Steps visited/distinct plus up to three meta tiles), which are neither reorderable nor removable and are not affected by these settings.

8.5.3 ⬭ Steps (the Step Editor)

> **Warning** Step properties live in the project's STEPS table and are therefore **shared with every user of that database**. Recolouring or rescoring a step changes it for everyone, and scores feed the Graph Value KPI and the Journey Score filter.

1. Choose a step in the **Step** dropdown ("— Select a step —" plus every step of the project). The editor appears only once a step is chosen; the dropdown is disabled when no project or no steps are loaded.
2. Set **Colors** using the two pickers **BG** and **FG**. A live pill preview shows the step in those colours.
3. Set the **Score**: tick the "set" checkbox and enter a number between −999 and 999. The number input stays disabled while the checkbox is clear; un-ticking it saves the score as unset.
4. Pick a **Shape**: ▭ **Stadium** (default), ▢ **Rounded**, ⬡ **Hexagon** or ◯ **Circle**.
5. Type a **Group** name (placeholder "BELONGS_TO"); existing group names are offered as autocomplete. This assigns the step to a step group.
6. Type a **Note** — free multi-line text, placeholder "Shown under the step name on the map".
7. Press **Save Step**. The button shows a spinner while writing; errors appear in red beside it.

Saving refreshes the score filter bounds and reloads the graph, so a changed score is reflected in the Journey Score slider and the Graph Value KPI straight away.

At the top of this **Steps** sub-section sits the 🗒 **Show node notes** toggle (default **on**). It controls whether each node shows its note / description under the step name on the map (truncated to 20 characters plus an ellipsis; the full text is in the hover tooltip and behind the node action **≡ Show description**), along with the yellow note badges. It lives here because the note it shows is the one set in the Step editor above.

Below it, ✨ **Highlight Trigger** sets how long you must rest the pointer on a node before the process map spotlights that node and its incoming and outgoing transitions (dimming everything else — see §9). Choose **1 s** (default), **2 s** or **3 s**, or **Off** to disable the spotlight. It applies to every process map except the Individual Journey and is stored per user.

8.5.4 ✨ Layout

A collapsible sub-section (default **expanded**) that groups the flow-chart rendering options: three display toggles and the flowchart font sizes.

✨ Optimise layout

Default **on**. Uses the optimised automatic layout for the flow chart.

🎨 Colorise edges by weight

Default **on**. Tints each transition along its metric's gradient; when off, all edges are grey and only thickness and opacity encode the metric. The same switch also appears at the top of the Edge Colour Schema sheet.

▦ Transition-table button

Default **off**. When on, a ▦ **Transitions** button appears in the lower-right of the A-Chart, B-Chart and both A/B-Comparison panels; it opens the underlying transition table (see §9). The button is hidden everywhere until you switch it on here.

🔡 Flowchart font sizes

Three segmented controls, each offering **S | M | L | XL**. Enlarging a node font also enlarges the node box, so text never spills outside its shape.

Control	Default	Scale factors (S / M / L / XL)	Affects
Nodes	M (2.25)	1.95 / 2.25 / 2.7 / 3.15	Node box and node text
Edges	M (3.375)	2.925 / 3.375 / 4.05 / 4.725	Edge labels, badges and note bubbles
Group titles	M (2.25)	1.95 / 2.25 / 2.7 / 3.15	Group name pill and its collapse toggle

> **Note** The edge presets are deliberately 50 % larger than the node presets at every step, because transition labels are small floating text and would otherwise be unreadable next to a node caption of nominally the same size.

8.5.5 🗓️ Dates and Times

This category groups the time-related display settings. It currently holds one control, **Default date window**.

🗓️ Default date window

A number input (whole numbers, minimum 0) followed by the word "days", default **30**. The hint below it reads: "*On load, show the last N days. 0 = full range. Applies next project load.*"

When a project is opened, the from-date is set to the latest event date minus N days, clamped to the project's earliest date; with N = 0 the whole range is shown. The A-Chart loads immediately with that window and every other view mode is seeded with the same window plus the project's real filter bounds.

> **Warning** Changing this value does **not** reload the project you are currently working on. It takes effect the next time a project is opened. To apply it now, reselect the project — which also resets filters, A/B state, statistics and AI results.

8.5.6 ✨ AI related

This category groups the settings for the AI supported Documentation. It currently holds one control, **LLM Prompt**.

💬 LLM Prompt

This row pairs the label **LLM Prompt** with an **Edit** button, which is disabled until a project is selected.

1. Press **Edit**. The **LLM Prompt Template** sheet opens with the explanation "This prompt is sent to the LLM together with the current transition table. Use it to guide the analysis style and output format."
2. Edit the template in the large textarea.
3. Press **Save** to store it, or **Cancel** to discard your edits.
4. Press **Reset to Default** to load the built-in template back into the editor. This only fills the editor — you still have to press **Save** for it to take effect.

The built-in default is:

```
Analyze the transitions table and identify outliers, min, max, avg
values for transitions. Use project name as a title, make a decent
layout.
```

The template is stored **per user and per project**. The AI supported Documentation report prints the template it used, block-quoted, in its "Analysis Parameters" chapter, and the exact prompt that was sent is available there behind **Show prompt**.

8.6 Defaults at a glance

Setting	Section	Default
Transition metric	Metrics	# Count
Filter sub-section expansion	Filters	Date expanded, all others collapsed
Active data A / B	Sampling	Original
Sample size	Sampling	1000 journeys
Sampling method	Sampling	Random (then your last used method)
Show step groups	Configuration	On
Groups start	Configuration	Expanded
Font size — Nodes / Edges / Group titles	Configuration → Layout	M / M / M
KPI tiles	Configuration	All ten on, in the order listed in §8.5.2
Show node notes	Configuration → Steps	On
Highlight Trigger	Configuration → Steps	1 s
Optimise layout	Configuration → Layout	On
Colorise edges by weight	Configuration → Layout	On
Transition-table button	Configuration → Layout	Off
Default date window	Configuration → Dates and Times	30 days
Date slider mode	Chart header	Range
Theme	Theme bar	System

8.7 What is private and what is shared

Because some of the controls in this chapter change only your own view and others change the database for everybody, it is worth knowing which is which.

Scope	Covers
Per user, cross-device (backend settings store; included in backups; private to you)	Theme; all Configuration toggles and font scales; KPI order and visibility; filter sub-section expansion; active sample-set choices; default sampling method; filter presets; LLM prompt templates; happy paths; target norms; node layouts.
Shared in the project database (visible to and changed for every user of that connection)	Sample sets (SAMPLE_1–3); step properties edited in the Step Editor — colours, scores, shapes, groups and step notes; notes; projects; demo data.
Session only (lost on reload or disconnect)	The current connection, the selected project, unsaved filter edits and the Notes view's own filter settings.

Note Disconnecting from a database clears the whole session state — project, filters and graphs — but does not sign you out of the app, and nothing stored per user or in the project database is lost.

9 The Process Map

> The process map is the flowchart canvas at the heart of the application. It appears in almost every view mode — A-Chart, B-Chart, both panels of the A/B Comparison, Individual Journey, the left pane of Happy Path, Conformance Check and the Simulation "Flow" tab — and it always behaves the same way. This chapter explains the canvas in detail: how nodes are drawn and what their badges mean, how start and end markers are placed, how edges encode the selected metric, how grouping boxes collapse and expand, and how you zoom, pan, drag and reset the layout. It then covers the four view modes built directly on the canvas: the A-Chart and the B-Chart, the A/B Comparison with its Process Goodness and Process Similarity tinting, and the Individual Journey. This chapter is for every user; the two role-restricted views built on the same canvas (Conformance Check and Happy Path) have chapters of their own.

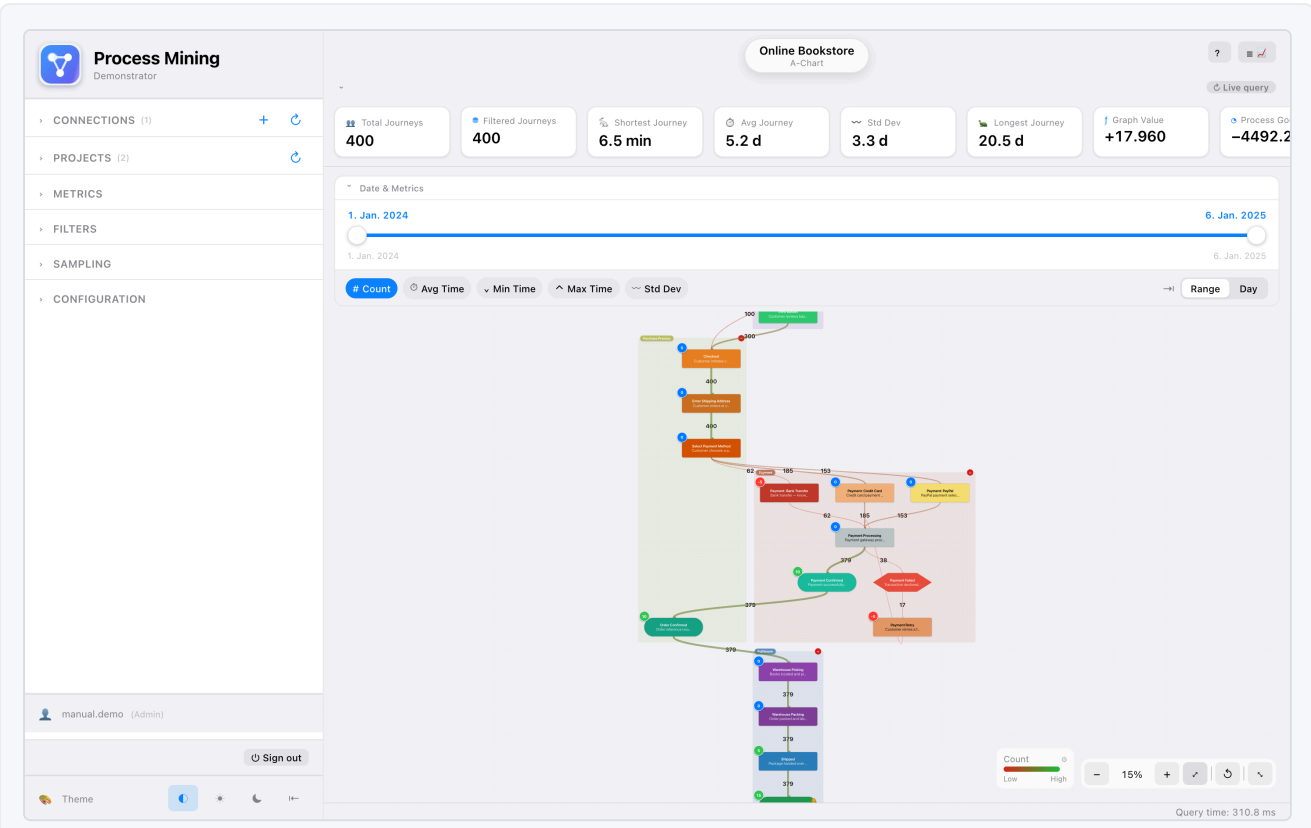


Figure 9.1 — The A-Chart: KPI strip, the "Date & Metrics" controls card and the flowchart canvas with its zoom overlay and edge-colour legend.

Nodes

Every process step in the filtered result set is drawn as one coloured node. Shape and colour come from the step definition held in the database, not from anything you set on the canvas, so the same step always looks the same in every chart and every view mode.

Shape value	Appearance
`stadium`	Pill with fully rounded ends. This is the default when a step defines no shape.
`round`	Rectangle with a 12 px corner radius.
`circle`	Circle.
`hex`	Flat-top hexagon.

Background and foreground colours are also per step. They may be given either as a named SwiftUI colour (a legacy of the desktop edition the web edition was ported from) or as a six-digit hex value. Any value the application cannot resolve falls back to the accent blue, so an unexpectedly blue node usually means a mal-formed colour in the step definition rather than a rendering fault.

What a node shows

The node always carries the step name. A second line underneath it is filled according to the following order of precedence:

1. If the step carries an event time — which happens only in Individual Journey mode — the time is shown as `HH:MM:SS` in 24-hour notation.
2. Otherwise, if **Show node descriptions** is enabled and the step has a description that differs from its name, a truncated description is shown: the first 20 characters followed by an ellipsis.
3. Otherwise the node shows only its name.

Hovering a node shows the full description as a tooltip, or the step name if there is no description. The description line is controlled by the setting `processmap.showNodeDescriptions`, which is on by default and is reached from the sidebar under Config.

Badges

Up to three badges can be attached to a node. They are independent of each other and can appear together.

End-of-process dot

An orange dot marking a step that is flagged *endOfProcess* in its definition. This is a property of the step itself — it says "this is a legitimate place for a journey to finish" — and is not derived from the current data.

Score badge

Shows the step's score. The badge is green when the score is greater than zero, red when it is less than zero, and blue when it is exactly zero. Steps without a score carry no badge.

Note badge

A yellow ✎ badge appears when the node has one or more notes attached, with the tooltip "This node has notes". Click the node and choose **Show Notes** to read them.

Note Node scores drive the **Graph Value** KPI tile: it is the sum of step score × visits over all scored nodes, where the visit count of a node is the larger of its incoming and outgoing occurrence totals. If no step in the project carries a score, the tile is hidden entirely rather than showing zero.

Start and end markers

The map annotates where journeys enter and leave the process, derived purely from the currently visible transitions:

- A **green downward arrow** — a dot, a stem and an arrowhead — is drawn *above* every node that has no incoming transitions. That node is a process start under the current filters.
- An **orange arrow** is drawn *below* every node with no outgoing transitions. That node is a process end under the current filters.

When grouping boxes are visible the markers are offset so that they clear the box outline rather than overlapping it.

Tip Start and end markers are recomputed from the filtered graph, so they change as you narrow the date window or exclude steps. A node that suddenly grows a green start marker usually means the step that used to precede it has been filtered out.

Selecting a node and its context actions

Clicking a node opens a context popover titled with the step name. The popover is the primary way of drilling into the process from the canvas.

Action	Effect
✓ **Require in journeys**	Adds the step to the *included steps* filter and removes it from the excluded list if it was there, then reloads the graph. Only journeys that visit this step remain.
⊖ **Exclude from journeys** (red)	Adds the step to the *excluded steps* filter and removes it from the included list, then reloads. Journeys visiting this step are dropped.
≡ **Show description**	Shown only when node descriptions are enabled and the step really has a description. Opens a centred popover with the step name, the full description text and a **Done** button.
✎ **Show Notes (N)** (yellow)	Opens the note flow for this node; *N* is the number of notes already attached. With no existing notes the editor opens directly on a new note; with one or more, a list sheet opens first.
⚡ **Actions**	Shown only when the administrator has enabled the Actions feature and you may run actions. Lists the saved actions whose availability matches this node; picking one runs its query — guided by the chart's current filters — and shows the result over the map. See the Actions chapter.

Require and Exclude are mutually exclusive per step: applying one always clears the other, so you cannot accidentally require and exclude the same step at the same time. Both immediately trigger a reload of the chart you are working in.

> **Warning** When the chart's data source is a **simulation** slot (Sim-A or Sim-B), the source is read-only: **Require in journeys** and **Exclude from journeys** are hidden from the popover and only the description and notes items remain. A red notice at the top right of the canvas states this — "Sim-A: simulation data is read-only — interactive filtering, drill-down and step editing are not available."

9.9 Aggregates (developers)

Developers can abstract several connected sub-processes into **Σ** super-steps on *one* high-level map. Click **Σ Select steps to aggregate** (lower-left of the map) to enter pick mode, then click a set of interconnected steps. Choose **+ Add group** to bank that group and start another — banked steps are ringed and locked, so a step can belong to only one aggregate. Bank as many groups as you need.

Choose **Create map** to open a dialog with one row per group. Each group is checked to be a single interconnected set (a step with no connection to the others is rejected) and takes a **Σ step name**, a **detail-project name**, and its **own target** (same schema, a new schema, or a different connection). It produces (the original project is never modified):

One high-level project
: A full copy of the map where each journey's run through each group collapses into that group's single **Σ** event. The normal engine then computes every Σ node's black-box metrics — its frequency, the time spent inside the group, and the incoming/outgoing edges to the rest of the map — while internal transitions disappear. A Σ step inherits the group's swimlane when its members share one.

One detail project per group
: Only that group's events, so each sub-process can be opened on its own and drilled into.

Each Σ node's menu offers **two ways to drill down**, and both show the *same* figures — the real numbers the un-aggregated map would show for the current date range, derived live from the source project:

↘ **Drill down · new panel**
: Opens the aggregate's real steps in a pop-up panel, wired to the steps just before and after it (drawn dashed, for context). The panel shows the date range and journey count, carries the same metric pills as the main map, and is **resizable** — drag its corner and the chart re-fits. Close it with ✕ or `Esc`.

↘ **Drill down · in place**
: Bursts the Σ step open *within the current map*: its real steps replace the Σ node and connect to the same neighbours, while everything else stays where it is. Several Σ steps can be expanded independently. A detail node's menu (and a button above the map) offers ↗ **Drill up** to collapse it back.

Drill-down is **read-only** and always follows your current **date window**, refreshing when you move the slider or apply a filter, so a Σ step's detail and the high-level map always tell the same story. To add more aggregates later, open the high-level map, enter pick mode again, select more original steps and choose **Add to map** — the map is rebuilt from its source with the new Σ step(s) added, and only the new detail project is written.

Dragging nodes

Nodes can be dragged freely. They snap to a 20 px grid, and the new position is written to storage only when you release the pointer, not continuously during the drag.

Layouts are stored server-side per user, and they are keyed by project *and* by chart mode. A-Chart, B-Chart, Happy Path, Conformance, each simulation slot (`sim_<slot>`) and each individual journey (`ij_<eventId>`) therefore keep completely independent arrangements of the same steps. Because the storage is server-side, your layouts follow you to another browser or machine and are included in backups.

Spotlighting a step's connections

Dense maps with many overlapping edges can be hard to read. To isolate one step, **hover the pointer over its node and hold still** for the dwell time — **one second by default**. The node and its immediate neighbours brighten and every transition that enters or leaves it stays at full strength, while all other nodes and edges fade to the background. This makes a single step's inputs and outputs easy to trace even in a crowded chart.

The dwell time is the **Highlight Trigger** setting in **Configuration** → **Steps**: choose **1 s** (default), **2 s** or **3 s**, or **Off** to disable the spotlight entirely. The choice is stored per user, so it follows you between browsers and machines.

The spotlight clears as soon as you move the pointer off the node, or when you click, pan or drag. Nothing is saved — it is a momentary, read-only aid. It is available on every process map **except the Individual Journey**, whose single trace is already strictly sequential and needs no untangling.

Edges and metric labels

Each edge is one observed transition from one step to another. Five metrics can be displayed, selected with the metric chips in the "Date & Metrics" card above the chart:

Chip	Metric
# Count	Number of times the transition occurred.
⏱ Avg Time	Mean time between the two steps.
½ Median Time	Median time between the two steps (the database `MEDIAN`).
⌄ Min Time	Shortest observed time.
⌃ Max Time	Longest observed time.
〰 Std Dev	Standard deviation of the transition times.

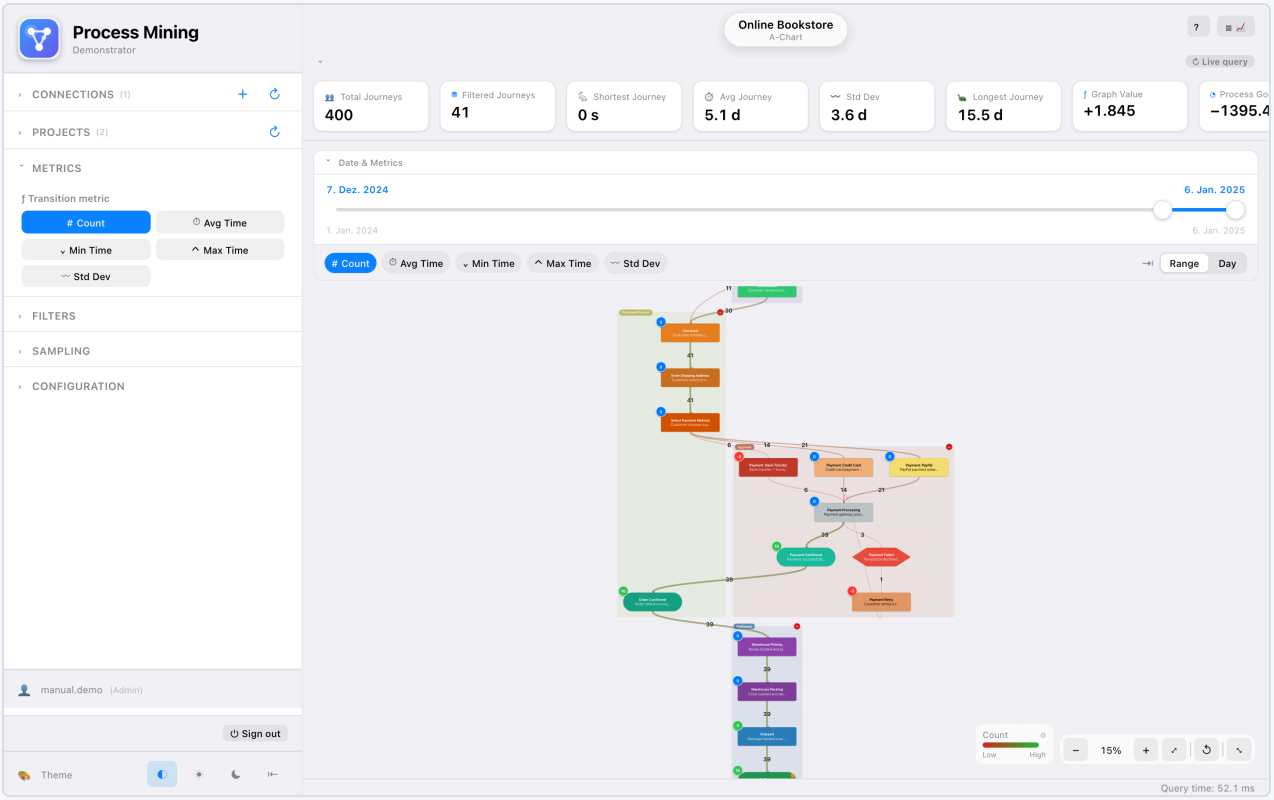


Figure 9.2 — The five metric chips. Exactly one is active at a time; the chips are disabled while a chart holds no data.

Thickness, opacity and colour

Thickness and opacity always encode the selected metric. Line width runs from 3 px to 22 px and alpha from 0.3 to 0.8, scaled linearly against the largest value among the currently visible edges — so the encoding is relative to what is on screen, and changing the filter can make a formerly thin edge thick. The arrowhead scales with the line width.

Colour is a separate channel. When **Colorise edges by weight** is on, each edge is tinted along the gradient configured for the active metric; when it is off, all edges are grey and only thickness carries the value.

Edge labels

Every edge carries a label with the metric value:

- **Count** is shown compactly — for example `1.2k` or `3.4M`.
- **Time metrics** are shown as `Ns`, `Nm`, `N.nh` or `N.nd` depending on magnitude.
- A missing time value is shown as `–`.

The label tooltip reads "`<from> → <to> · <occurrences>`", which gives you the raw occurrence count even while a time metric is displayed. A yellow ✎ bubble sits next to the label when the transition has notes, with the tooltip "This transition has notes".

A transition from a step to itself is drawn as a **self-loop**: a circle above the node, without an arrowhead.

Clicking an edge

The edge line has a wide invisible hit area, and the label itself is clickable too, so you do not have to hit the stroke exactly. In the normal charts a click opens a popover titled "`<from> → <to>`" containing ✎ **Show Notes (N)**. (In Conformance Check's edit mode the same click opens the norm editor instead — see the Conformance Check chapter.)

The colour-scale legend and the Edge Colour Wizard

When `graph.edge.colorizeByWeight` is on — it is on by default — a small legend card floats over the canvas showing the current metric name, a ⚙ icon and a Low→High gradient bar. If the neutral schema is selected the bar is replaced by the text "No color scale".

1. Click the legend card (tooltip "Configure edge colour schema per metric") to open the **Edge Colour Schema** sheet.
2. Use the **Colorise edges by weight** switch at the top to turn colouring on or off globally.
3. For each metric — Count, Avg Time, Median Time, Min Time, Max Time, Std Dev — pick a schema from its dropdown. A live gradient preview updates beside it.
4. Choose **Reset to defaults** to restore the per-metric defaults, or **Done** to close.

Schema	Gradient	Default for
None (grey)	Flat grey, no scale	—
Green (high is good)	Red → green	Count
Red (low is good)	Green → red	Max Time
Blue scale	Light → dark blue	Min Time
Orange scale	Light → dark orange	Avg Time
Purple scale	Light → dark purple	Std Dev

Choices are saved per metric under `graph.edge.colorSchema.<metric>` and are included in backups. The sheet's own explanatory text summarises the rule: edge thickness always encodes the selected metric; colouring is an additional layer tinting each transition from the lowest to the highest value.

The transition table behind the chart

Every edge on the map is one row of the underlying directly-follows graph. To read the exact figures rather than compare edge widths, open the **transition table**: the raw data the chart is drawn from, shown in full with **all metrics at once**.

The table is reached from a ▦ **Transitions** button in the lower-right of the canvas. That button is **hidden by default**; switch it on under **Configuration → Layout → Transition-table button** (see §8.5.4). It is offered only on the **A-Chart, B-Chart and the two A/B-Comparison panels** — not on the Individual Journey, Conformance or Happy Path maps.

The panel lists one row per transition with these columns: **From**, **To** (self-loops are marked ↺), **# Count**, **% Outgoing** (the transition's share of its source step's departures), **Journey %** (its share of all filtered

journeys), and **Avg**, **Min**, **Max** time and **Std Dev** — a dash where a time statistic is unavailable. A search box filters by step name, and clicking any column header sorts by it (click again to reverse); the table opens sorted by Count, largest first. A count above the table reports how many transitions match and the total occurrences. The view is read-only and reflects exactly the graph currently drawn, filters and all.

A ⧉ **Copy** button copies the rows currently shown (in their current filter and sort order) to the clipboard as tab-separated values, ready to paste straight into a spreadsheet. The copied data uses raw numbers rather than the display strings — times in **seconds** (blank when unavailable) and shares as plain percentages — so it is immediately usable for further analysis.

Grouping boxes

Steps that share a `BELONGS_TO` value are treated as a group. With **Show grouping** on (setting `processmap.showGrouping`, on by default) each group is surrounded by a dashed, tinted box with a coloured name pill. The pill's hue is derived deterministically from the group name, so a given group keeps its colour across charts and sessions.

At the top right of each box sits a round collapse toggle: a red **–** collapses the group (tooltip "Collapse <group>") and a green **+** expands it again (tooltip "Expand <group>").

Collapsed groups

A collapsed group is replaced by a single accent-coloured proxy node showing the member count, for example "7 nodes". Edges to and from the members are merged into edges to and from the proxy, and this merging changes the numbers:

- **Connection counts are summed.**
- **Times are weighted-averaged.**
- **Intra-group transitions are dropped** — a transition between two members of the same collapsed group disappears from the map entirely.

A grey hint at the bottom centre of the canvas restates this while any group is collapsed: "Collapsed groups: connection counts summed · times weighted-averaged".

> **Warning** Min Time, Max Time and Std Dev cannot be reconstructed from merged transitions. On an edge that touches a collapsed group these metrics are unavailable and the label shows `–`. If you need those figures, expand the group first.

Dragging the group **box** moves all of its member nodes together — or the proxy node when the group is collapsed — which is the quickest way to rearrange a large map.

Initial collapse state

What happens when you open a project is governed by the setting `graph.startMode`:

Value	Behaviour
`expanded` (default)	All groups open.
`collapsed`	All groups collapsed.
`persisted`	Restore the last collapse state, remembered per project and per chart mode.

When grouping is on and the project actually contains groups, the zoom overlay gains an extra button: ⤡ **Collapse all groups** or ⤢ **Expand all groups**, whichever is appropriate for the current state.

Layout, zoom and pan

The canvas has a dotted-grid background and an overlay of viewport controls along the bottom edge.

> **Note** To keep the map uncluttered, the bottom-corner controls stay **hidden until you move the pointer into that corner**, each marked at rest by a dim **corner handle** (a small ⋯ at the lower-right, a **Σ** at the lower-left) so the reveal is easy to find. The viewport/control cluster (zoom, ⤢ fit, ↺ reset, and — when enabled — ▦ **Transitions**) reveals near the **lower-right**, and the developer **Σ Select steps to aggregate** button reveals near the **lower-left**; move away and they fade back to the handle. The **auto-refresh countdown ring** is the exception — it stays visible on its own so you can watch a live feed at a glance.

Control	Effect
–	Zoom out (divides the current zoom by 1.3).
Percentage readout	Current zoom level, for example "100%".
+	Zoom in (multiplies the current zoom by 1.3).
⤢	Fit to view — frames the whole graph.
↺	Reset node layout — clears all manual drag positions for this chart and re-fits.

Zoom is clamped between 15 % and 500 %. Double-clicking anywhere on the canvas background also fits the view, which is usually faster than reaching for the ⤢ button.

> **Note** ↺ **Reset node layout** affects only the chart you are looking at. Because layouts are keyed per project *and* per chart mode, resetting the A-Chart leaves your B-Chart, Happy Path, Conformance and simulation arrangements untouched.

While a reload is in flight, a small "Loading…" pill with a spinner appears over the canvas. The chart stays interactive during the reload — you can keep panning and zooming.

Display scaling

Four settings in the sidebar under Config → Fonts control how large the map is drawn. They are read directly by the canvas, so changes take effect immediately.

Setting	Default	Effect
`graph.node.scale`	2.25	Grows the node box and its font together, so text never spills out of the shape.
`graph.edge.scale`	3.375	Sizes edge labels, badges and the note bubbles on edges.
`graph.group.scale`	2.25	Sizes the group name pill and its collapse toggle.
`graph.optimisedLayout`	on	Uses the optimised automatic layout algorithm.

The A-Chart

The A-Chart is the primary analysis chart and the default view mode. It consists of the KPI strip, the collapsible "Date & Metrics" controls card and the canvas. It is the chart other parts of the application defer to: Statistics always mirrors the A-Chart's filters, AI supported Documentation always analyses the A-Chart dataset, and the Simulation builds its Markov model from the observed A-Chart graph.

When you open a project for the first time, the A-Chart loads immediately using the default window — the last 30 days of data, configurable with `graph.defaultWindowDays` (0 means the full range). Every other mode is seeded with the same window and the project's real filter bounds but is not loaded until you ask for it.

A-Chart states

State	Glyph, title and message
Not connected	⛁ **Not Connected** — "Add a connection in the sidebar and tap it to connect to your Exasol database."
No project	🗺️ **No Project Selected** — "Select a project from the sidebar."
Loading	Spinner + "Loading process map".
No data	📊 **No Process Data** — "No event transitions found for this project.", with a prominent **Load** button.
Load error	⚠️ **Failed to Load** — the backend error message.

With data present, a right-aligned footer reads "Query time: N ms" (or "N.NN s" for longer queries). Its tooltip — "Server-side execution time of the queries behind this view" — is a reminder that this figure measures the database, not the browser.

Automatic refresh

The A-Chart can reload itself on a timer — useful when the underlying data is growing while you watch (a live feed, or the background demo-event generator). You set the interval in **Configuration → Auto-refresh** in the sidebar (that section appears **only while you are in the A-Chart**; no other view auto-refreshes), and the live controls then appear **on the A-Chart itself**.

- **Refresh every** (Configuration → Auto-refresh) — a dropdown: **Off** (the default), **10s**, **30s**, **1m**, **2m**, **5m**.
- Once an interval is chosen, a round **countdown ring** and a ↻ **Refresh** button appear in the A-Chart's control cluster at the **bottom-right of the canvas**, between the ▦ **Transitions** button and the zoom

controls. The ring shows the seconds remaining; the button reloads immediately and restarts the countdown.

The window follows "now". On each refresh (timed or via the button), if the date window currently ends in the *past*, it slides forward so its end is **today** — keeping the same span — so journeys that arrived since the last load actually show. A window that already reaches today is left untouched. (This is why a plain reload previously missed new rows: it kept the window captured when the project was first loaded.) The unfiltered KPI tiles — **Total Journeys** and the active-sample count — are re-read on each auto-refresh as well, so they keep pace with the incoming data instead of sitting at their load-time values.

The timer keeps running while you work in other sidebar sections or collapse the sidebar; it is paused automatically the moment you switch to any other view mode, and resumes when you return to the A-Chart. The chosen interval is remembered per user.

Flowchart orientation, or Sankey

A small switch floats at the top of the A-Chart and B-Chart canvas with three options — **Flowchart** / ➡ **Horizontal** / 🌊 **Sankey**. It flips the same data between renderings; your choice is remembered per user and applies to both charts.

The first two are the **same directed-follows flowchart** in two orientations. **Flowchart** lays the process out **top-down** (the default): the start markers are above the first steps and the flow runs downward. ➡ **Horizontal** lays exactly the same map out **left-to-right** — layers become columns, the start markers sit to the left of the first steps and the end markers to the right, and the edges curve sideways. Everything else is identical: the node badges, the grouping boxes, the metric on the edges, the node context menu and the transition table all behave the same. Horizontal is often the more natural reading order for a long, mostly-sequential process, or when you want a wide process to fill a wide screen. Node positions you drag are remembered **separately per orientation**, so switching back and forth never disturbs a layout you have arranged.

The **Sankey** view draws the process as a left-to-right flow: each band is a transition, and its width follows the **currently selected transition metric** (the same chips as the flowchart — # Count, % Percentage, Journey %, Avg / Min / Max Time, Std Dev), so switching the metric re-weights the flows just as it re-weights the flowchart's edges. A footer states which metric the widths encode. Nodes are laid out in columns by their distance from the start. Hover any band for its value in that metric plus its share of the source step's outflow and journey count; hover a node for its throughput. Whatever you hover stays lit while everything else dims, which makes a single path easy to trace.

A Sankey has to flow strictly forward, but a process is full of loops. The view resolves this automatically: it finds each **cluster of steps that loop among one another** — formally, each strongly-connected component of the graph — and **collapses it into a single node**, marked ↺. The loops that used to tangle the diagram become an "internal loops" count, and the amenities that passengers circulate between (Duty Free ⇄ Dining ⇄ Lounge, for example) become one tidy "airside" node. A collapsed node is named after the members' shared step group when they have one, otherwise after its busiest member. A footer states how many clusters were collapsed. This needs no configuration and works on any process; a purely sequential process simply produces no clusters.

Hovering a collapsed node highlights it and **lists every step inside the group**, together with the number of internal loops between them — so nothing is hidden by the grouping: you can always see exactly which steps were folded together and why.

The B-Chart

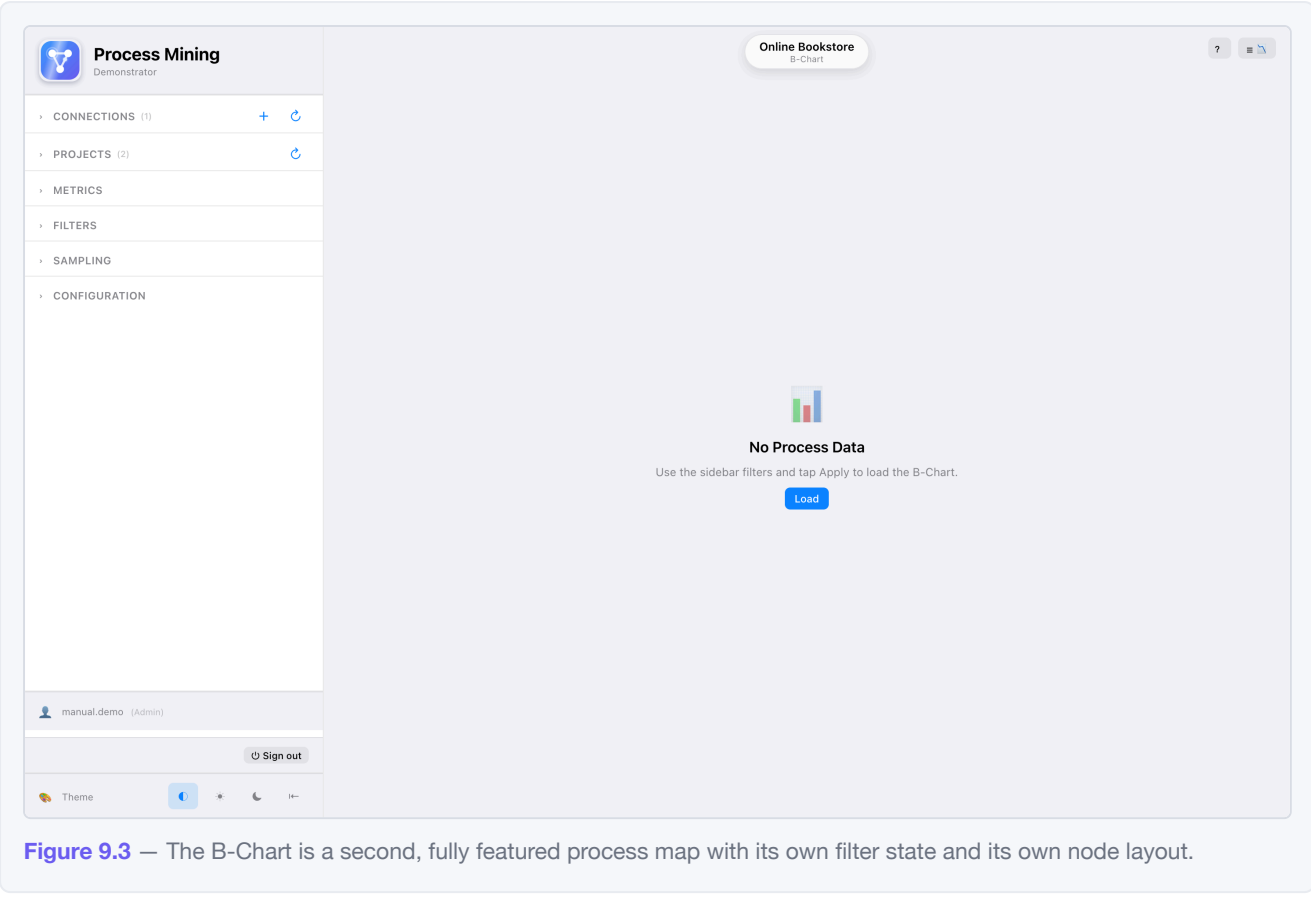


Figure 9.3 — The B-Chart is a second, fully featured process map with its own filter state and its own node layout.

The B-Chart is a second, entirely independent process map. It has the same KPI strip, the same controls card and the same canvas, and it remembers its own filter state (dates, included and excluded steps, meta filters and metric) and its own node layout. Nothing you do on the A-Chart changes the B-Chart or vice versa.

Two things differ from the A-Chart:

- The B-Chart does not load automatically. Its empty state reads **No Process Data** — "Use the side-bar filters and tap Apply to load the B-Chart.", with a **Load** button.
- Its loading message is "Loading B-Chart".

Use the B-Chart when you want to keep one configuration of the process on screen while you build a second one — for example one product line against another — and then switch to A/B Comparison to see both at once.

> **Note** Each view mode remembers its own filter state. When you switch modes the application saves the state you are leaving and restores the state of the mode you are entering. Statistics is the exception: it always inherits the A-Chart's filters.

A/B Comparison

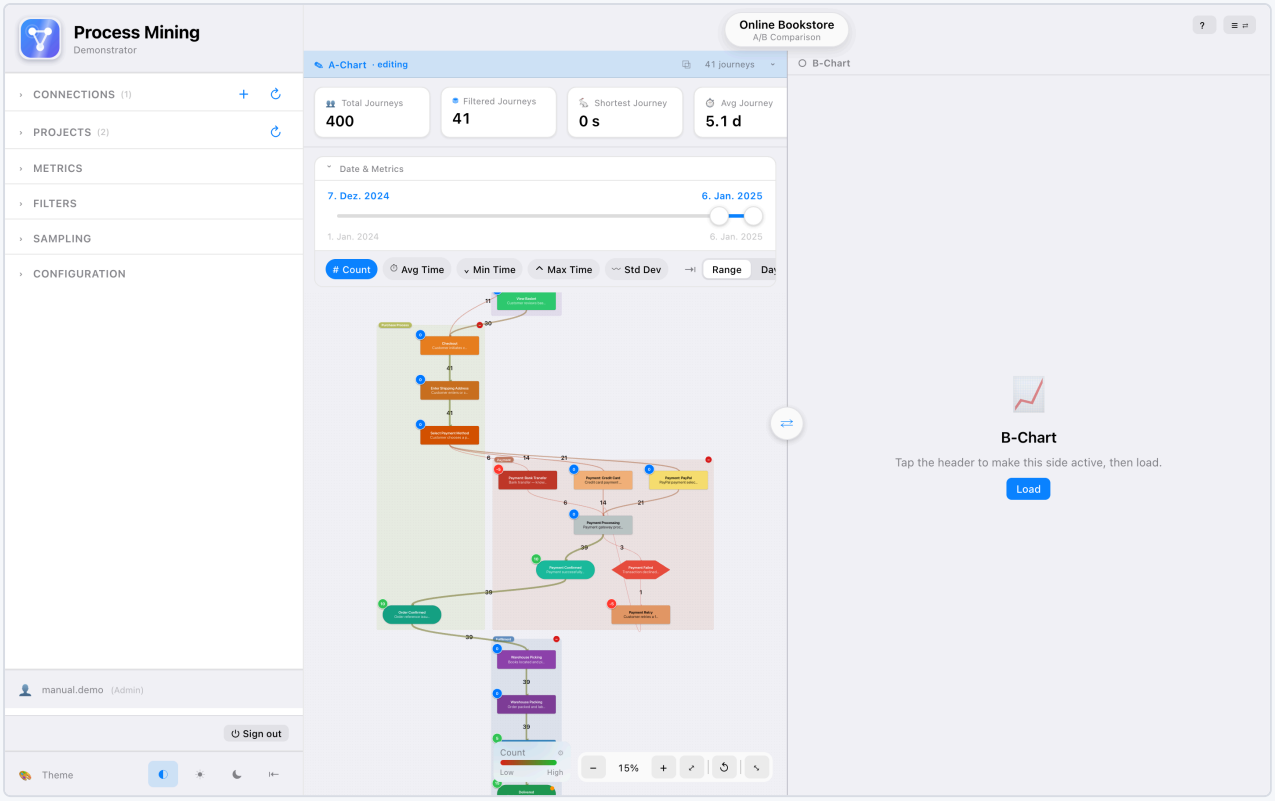


Figure 9.4 — A/B Comparison: panel A on the left, panel B on the right, with the valve button between them and the similarity badge along the bottom.

A/B Comparison places the two charts side by side — A on the left, B on the right. Each panel has its own header, an optional KPI strip, its own "Date & Metrics" card and its own flowchart. The panels map onto the saved A-Chart and B-Chart states, so anything you already configured in those modes is what you see here. The active side starts as A.

The strip between the two panels carries two buttons. The ⇄ **valve** synchronises panning, zooming and reset across both sides (see below). Below it, an **orientation toggle** (➡ when the maps are top-down, ⬇ when they are left-to-right) flips **both** panels between the top-down and horizontal flowchart layouts at once — the same setting as the A-Chart/B-Chart switch above, so the two stay in step.

Choosing the active side

The sidebar filters can only edit one side at a time. The side currently being edited is the **active** side, and the panel header tells you which is which: the active panel shows "✎ A-Chart · editing", the inactive one shows "○ B-Chart".

1. Click the header button of the panel you want to work on. That side becomes active.
2. Adjust filters in the sidebar, or use that panel's own "Date & Metrics" card, and apply them. Only the active side reloads.
3. Click the other panel's header to switch, then repeat.

Each panel header also shows that side's journey count ("N journeys") and a chevron that collapses or expands that panel's KPI strip (`abpanel.a/b.kpiExpanded`, expanded by default). Panel A carries one extra

button: ⧉ **Copy A's node layout to B**, which copies both the viewport and the node positions across so that the two maps line up for visual comparison.

Changing a metric inside one panel's card changes that panel only — and also the shared metric if that panel happens to be the active side.

The valve

Between the two panels sits a ⇄ **valve** button that controls whether the panels move together.

State	Tooltip	Behaviour
Open (default)	"Valve open — pan, zoom and reset are synchronised"	Panning, zooming and node dragging in one panel are mirrored in the other.
Closed	"Valve closed — charts move independently"	Each panel pans, zooms and is dragged on its own.

Closing the valve first snapshots A's current viewport and node positions into B, so the two maps start from the same arrangement before diverging. The state persists as `abComparison.valveOpen` and defaults to open.

Data sources per side

Each side's data source is chosen in the sidebar Sampling section and can be the original data, one of the sample sets (Sample 1–3), or a simulation slot (Sim-A or Sim-B). The **Active Sample** KPI tile on each side names the source it is actually reading — "Original", "Sample 1" … "Sample 3", or "Sim-A"/"Sim-B" — and its value is that source's journey count.

> **Warning** A side fed from a simulation slot is read-only: it shows the red canvas notice and its nodes offer no Require or Exclude actions. You can still read the map, open descriptions and work with notes.

Reading the comparison: Goodness and Similarity

Two figures make the comparison quantitative.

Process Goodness ◔

A server-computed goodness score for one side, shown signed with two decimals. In A/B mode the tile is tinted by comparison with the other side: **green** when this side is better, **red** when it is worse, and **blue** when the two are equal within 0.005. The tile is hidden when the score is null.

Process Similarity ⇄

How alike the two sides are, on a scale of 0 to 1 with two decimals, computed from both sides' journey-path variants together with their graphs. It is tinted **green** at 0.70 and above, **red** at 0.30 and below, and **blue** in between. The tile only exists once a similarity has been computed.

The same similarity figure is repeated as a badge along the bottom row of the comparison — "⇄ Similarity N.NN" — once both sides have been computed, using the identical green/red/blue thresholds.

Empty and loading panels

An unloaded panel shows 📈 with the title **A-Chart** or **B-Chart**. The message depends on whether that side is active: the active side reads "Apply filters in the sidebar to load."; the inactive side reads "Tap the header to make this side active, then load." Both offer a **Load** button, which makes that side active and loads it in one step. While loading, the panel shows a spinner with "Loading A-Chart" or "Loading B-Chart".

Individual Journey

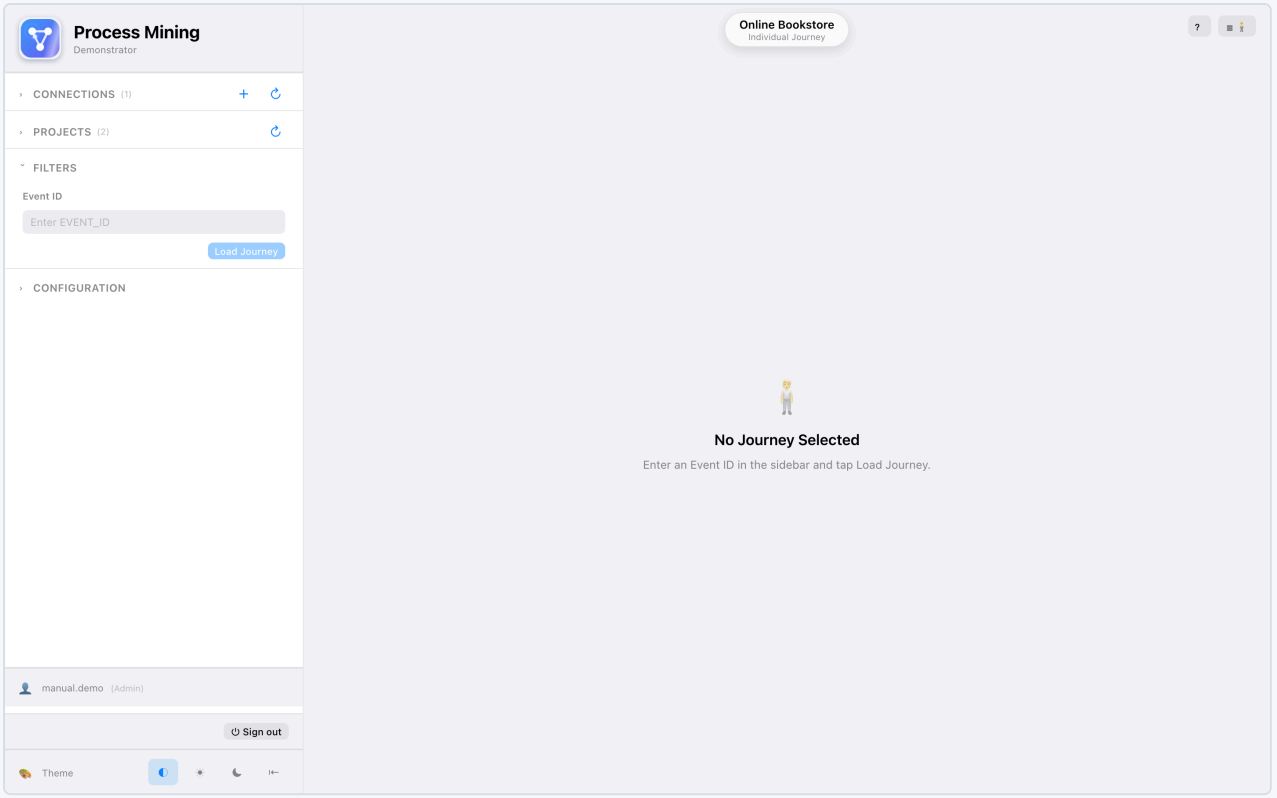


Figure 9.5 — Individual Journey: a single case drawn step by step, with event times on the nodes and the journey KPI tiles above.

Individual Journey draws exactly one case. Instead of an aggregated map you see that journey's steps in the order they occurred, with the clock time of each event on the node.

Finding a journey

> Requires: **any user**

1. Switch to **Individual Journey** from the ≡ "Switch view" menu in the toolbar.
2. In the sidebar, type into the **Event ID** field. After two or more characters the field offers autocomplete suggestions.
3. Pick a suggestion or finish typing the identifier.
4. Tap **Load Journey**.

The sidebar's Metrics selector is hidden in this mode, because a single traversal makes counts meaningless: the edge metric is forced to **Avg Time**.

Reading the journey

On the canvas:

- Each node shows the event **time** as `HH:MM:SS` beneath the step name, in place of the usual description line.
- Node positions are stored per event ID (chart mode key `ij_<eventId>`), so rearranging one journey does not disturb another.

The KPI strip is replaced by a journey-specific set of tiles:

Tile	Icon	Meaning
Date	📅	The journey's start date, in short format.
Duration	⏱	End minus start of the journey.
Sum of Scores	*ƒ*	Sum of the step scores across the visited steps, shown signed; "—" when no step carries a score.
Steps visited / distinct	⑂	"*total* / *distinct*", where the total is the number of transitions traversed plus one.
Meta tiles	🏷	Up to three further tiles for the project's meta fields. The label is the project's meta1/meta2/meta3 title and the value is this journey's value. A tile appears only when both the title and the value exist.

The meta tiles are what let you recognise the case in business terms — customer, order number, region, or whatever the project's meta fields have been configured to hold.

Flowchart or swimlane

A small switch floats over the canvas, top-centre, toggling between two layouts of the *same* journey. It is the only place this choice lives — it is on the canvas, not in the left panel — and it applies only to Individual Journey (the aggregated views, where loops carry meaning, always use the flowchart).

- 🕸 **Flowchart** (default) — the directed-follows graph described above. Because it aggregates the case into unique nodes, a revisited step is drawn as a *loop* back to the same node.
- 🚤 **Swimlane** — the journey re-formatted as a **strictly sequential** layout: one column per event, left to right in time order. A revisited step simply appears again in a later column, so the view has **no loops at all** — the loop is "unrolled" into repeated, sequential nodes.

In the swimlane, each horizontal **lane is a node group** (a step's *belongs-to* group), tinted with that group's colour, and a step sits in its group's lane — so a diagonal edge is the visual signal that the process moved from one group to another. A **header row above the top lane** carries each event's date and time, and every **edge is labelled with the time taken from that node to the next** (the elapsed time between the two consecutive events). The swimlane pans and zooms like the flowchart and fits itself to the window when opened; a matching zoom cluster sits in the lower-right corner (zoom out / in with the live percentage, ⤢ fit-to-view, and ↺ reset to 100 %).

Below the lanes the swimlane draws a **cumulative-value line chart**: for each event it plots the **running sum of the step scores from the process start** (a YTD-style total), so you can read how the journey's value develops left→right. Each step contributes its own score (positive or negative — the marker is green when it added value and red when it subtracted), the line joins the running totals, and the value at each point is labelled. A dashed zero line appears whenever the running total crosses zero. The chart shares the lanes' horizontal scale — each point sits directly under its step's column — and pans and zooms together with them.

> **Note** The swimlane deliberately has no per-node menu — it is a read-only, at-a-glance timeline of one case. Its columns are evenly spaced by *occurrence order*, not by real elapsed time; the actual gaps are the edge labels, and the absolute clock times are the header row. The cumulative chart uses each step's **score** (set in the Step Editor) as its per-step value — the same score that drives journey totals — so steps left at score 0 leave the line flat.

Individual Journey states

State	Glyph, title and message
No project	🗺️ **No Project Selected** — "Select a project from the sidebar to view a journey."
No Event ID entered	🕴️ **No Journey Selected** — "Enter an Event ID in the sidebar and tap Load Journey."
Loading	Spinner + "Loading journey".
Not found	🔍 **Journey Not Found** — "No steps found for "<entered id>". Queried hash: <hash>"
Error	⚠️ **Failed to Load** + the error message.

> **Tip** The "Journey Not Found" message deliberately echoes the **queried hash** as well as the identifier you typed. Event IDs are hashed before being sent to the database; quoting the hash when you raise a support request lets an administrator check the stored data directly.

What persists, and where

Because the process map is highly configurable, it is worth knowing what survives a reload and what does not.

Item	Scope
Node positions	Server-side per user, keyed per project *and* chart mode (A-Chart, B-Chart, HappyPath, Conformance, `sim_<slot>`, `ij_<eventId>`).
Collapsed-group state	Per project and chart mode; restored only when `graph.startMode` is `persisted`.
Edge colour schemas	Per metric, under `graph.edge.colorSchema.<metric>`; included in backups.
Selected metric	Per chart, as part of that chart's saved state.
KPI strip expansion	`processmap.kpiExpanded`, and per panel in A/B (`abpanel.a/b.kpiExpanded`).
Valve state	`abComparison.valveOpen`, default open.
Controls-card expansion	Per surface: `achart.controlsExpanded`, `bchart.controlsExpanded`, `abpanel.a/b.controlsExpanded`.

Server-side storage means these settings travel with your account rather than with the browser, and they round-trip through the administrator's backups.

10 Analysis: Statistics, Conformance, Happy Path and Simulation

> The charts in the previous chapters show you what the process looks like. The four views in this chapter tell you what it means: **Statistics** enumerates and ranks every route your journeys actually take, **Happy Path** lets you record the route they were *supposed* to take and scores reality against it, **Conformance Check** compares individual transitions against numeric target values ("norms"), and **Simulation** generates synthetic journeys from the observed process so you can test a hypothesis without touching the database. Statistics is available to every user; Happy Path, Conformance Check and Simulation are reserved for power users and administrators and are hidden from the view menu of plain users.

Statistics

Statistics answers three questions: which routes do journeys take, how many journeys take each route, and how are duration and traffic distributed across the process. It is reached from the ≡ (Switch view) menu as **Statistics**.

> **Note** Statistics does not have its own filter state. It always uses the **same filters as Chart A**, and the view says so in an information line: "ⓘ Statistics use the same filters as Chart A." To change what Statistics reports on, change the A-Chart filters in the sidebar. This is the one exception to the rule that each view mode remembers its own filter state.

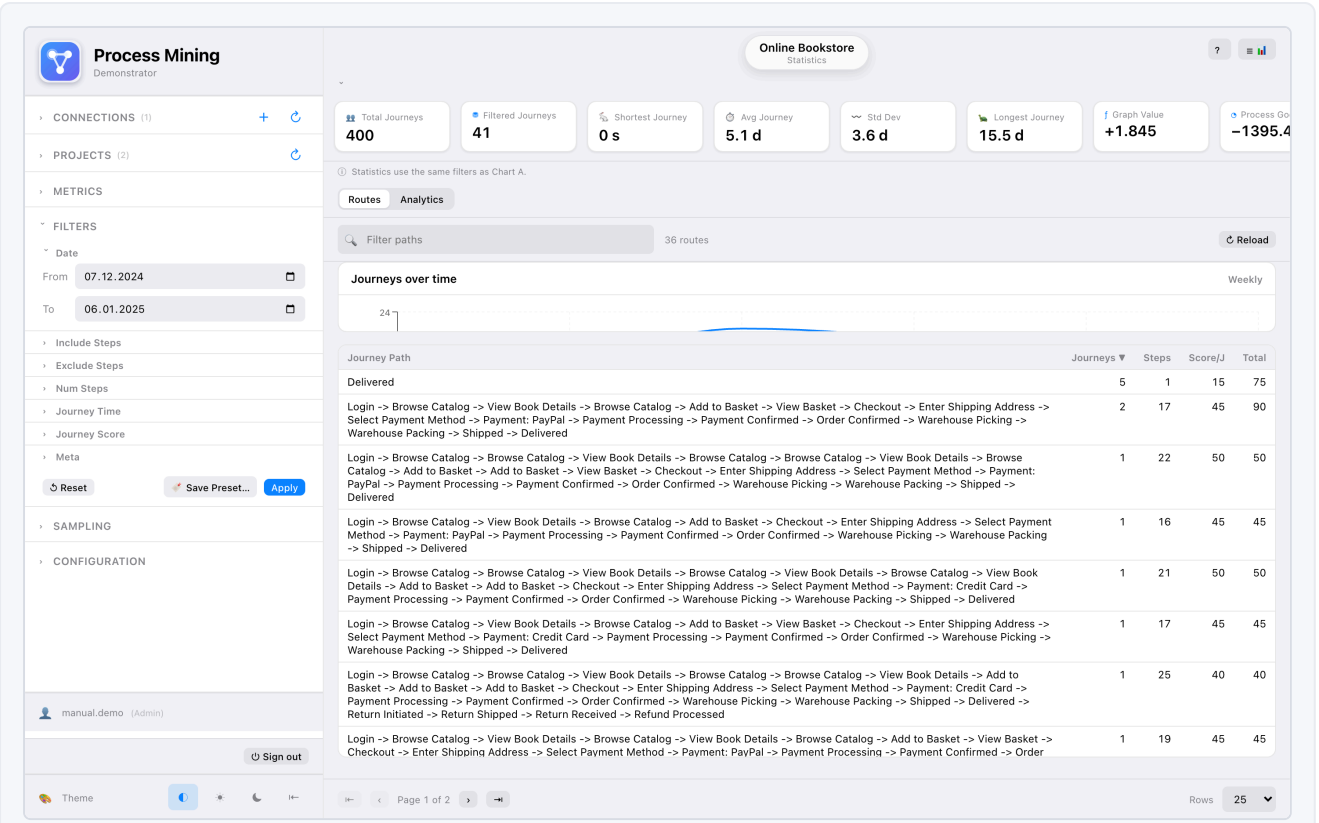


Figure 10.1 — The Statistics view, Routes tab: journeys-over-time chart above the sortable route table.

Loading routes

Route computation runs once per project automatically the first time you open the view. Afterwards you reload it explicitly.

1. Select a project in the sidebar and set the filters you want on the A-Chart.
2. Switch to **Statistics**. If routes have never been computed for this project, the empty state ⑂ **No Routes Yet** appears with the text "Compute all journey routes for the current filters." — press ↻ **Compute Routes**.
3. While the server works, a spinner reads "Computing journey routes".
4. When the computation fails, ⚠ **Query Failed** appears with the backend message and a ↻ **Retry** button; a modal alert titled "Statistics Error" with **Retry** and **Cancel** is shown at the same time.

If you change any sidebar filter after the last computation, an orange banner appears above the tabs: "⚠ Filters changed since last load", with a ↻ **Reload** button. The figures on screen still describe the *old* filter set until you reload — the banner exists precisely because the numbers would otherwise be silently stale.

The standard KPI strip sits at the top of the view with its own collapse handle; the expanded/collapsed state is remembered in the setting `statistics.kpiExpanded` (default expanded). Without a project selected the view shows 🗺 **No Project Selected**.

The Routes tab

A *route* (also called a path or variant) is the ordered sequence of steps one journey passed through, written "Step A → Step B → Step C". All journeys with an identical sequence are collapsed into one row.

Control	Purpose
🔍 **Filter paths**	Free-text search across the route strings; the row count updates as you type.
N route(s)	Number of routes currently listed. "(truncated)" is appended when the server capped the result set — the rarest routes are then missing from the table.
↻ Reload	Recomputes the routes with the current A-Chart filters.
Journeys over time	Line chart of journey volume across the filtered window. The granularity label reads **Daily**, **Weekly** or **Monthly** and is chosen by the server according to the window length; you cannot set it manually.
Rows	Page size: 25 (default), 50, 100 or 250.
⇤ ‹ **Page X of Y** › ⇥	First, previous, next and last page.

The table itself has four sortable columns. Click a header to sort by it; an arrow (▲ ascending / ▼ descending) marks the active column. The default sort is **Journeys** descending, so the dominant routes are at the top.

Column	Meaning
Journey Path	The step sequence of this route.
Journeys	Number of journeys that followed exactly this sequence.
Steps	Length of the route in steps.
Score/J	Sum of the step scores along the route — the score one journey earns by taking it.
Total	Score/J × Journeys: the route's total contribution to the process value.

> **Tip** Sort by **Total** descending to find the routes that carry the most value, and by **Score/J** ascending to find the most damaging route pattern regardless of how rare it is. A route with a poor Score/J but a large Journeys count is usually the most rewarding thing to fix first.

The Analytics tab

The Analytics tab summarises the same filtered journey set from four angles.

Journey duration distribution

Histogram of end-to-end journey durations. Reads "No duration data." when the filtered set carries no usable timestamps. A long right-hand tail points at a small number of stuck journeys; two separate humps usually mean two different journey types are mixed in your filter.

Step traffic

Horizontal bar chart, **Top 25 by visits**. A step's visit count is `max(incoming occurrences, outgoing occurrences)`, so a step is counted once per journey pass whether it is measured from its inbound or outbound side. The busiest step is highlighted.

Transition heat map

Matrix with "rows = from · columns = to". Blue intensity is proportional to the number of occurrences; the tooltip of each cell reads "<from> → <to>: N". Shows "No transitions." when empty. Cells away from the leading diagonal band are the process's shortcuts and rework loops.

Duration KPIs

Shortest, Average, Std deviation and Longest journey duration for the filtered set.

Happy Path

> Requires: **Power user** or **Administrator**

A happy path is your *intended* step sequence — the route a journey should take when nothing goes wrong. You record it as an ordered list of steps (optionally with alternative branches), and the application scores how closely real journeys follow it. Happy paths are stored on the server per project and per user, so they survive a browser change and are included in backups.

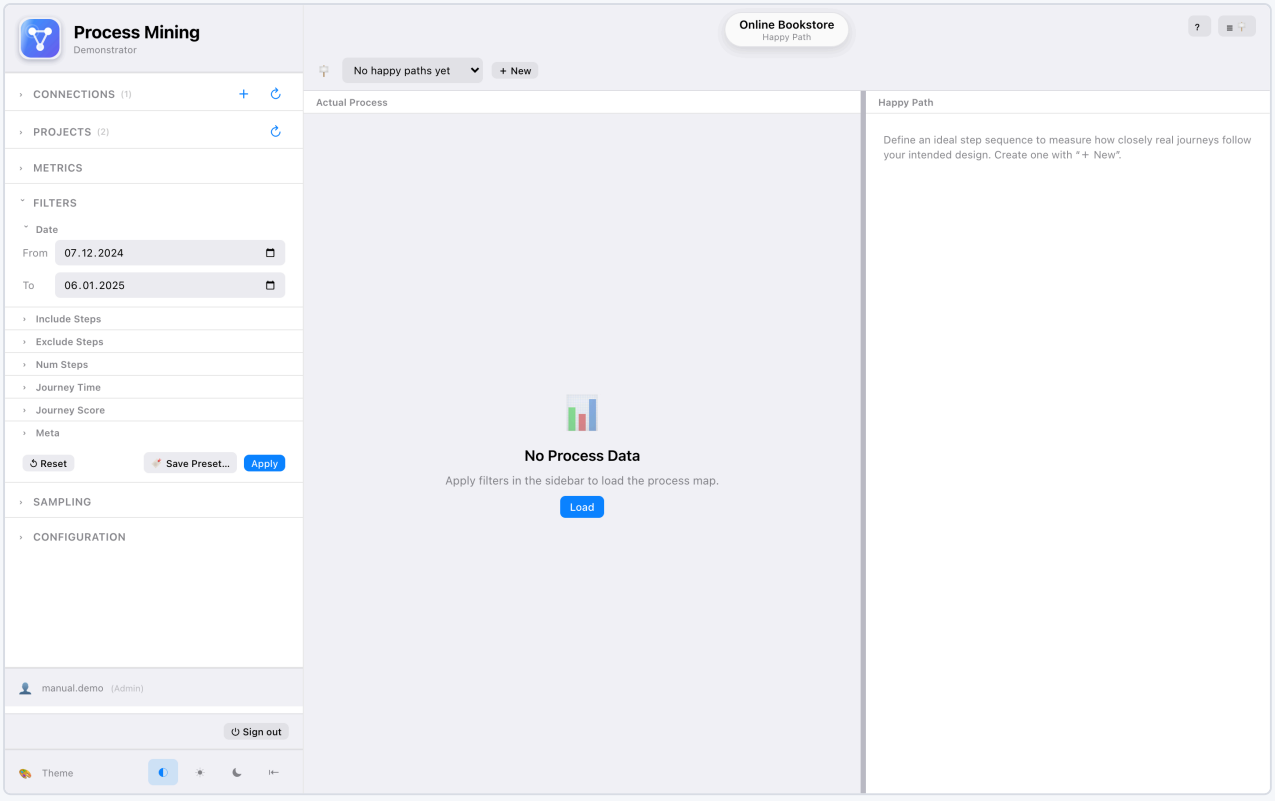


Figure 10.2 — Happy Path: the actual process map on the left, the ideal step sequence on the right.

Layout

The view is split in two. The left pane, headed "Actual Process" with the filtered journey count ("N journeys"), is the full process flowchart with the same node, note and drag behaviour as the A-Chart (its node layout is stored under the chart mode "HappyPath", independently of the A-Chart). If no data is loaded it shows **No Process Data** — "Apply filters in the sidebar to load the process map." with a **Load** button.

The right pane holds the ideal path. Drag its left edge to resize it between **300** and **1000** px; double-click the drag handle to reset the width to **460** px. The width persists as `happyPath.editorWidth`.

Creating and editing a path

1. Press **＋ New** in the top bar. The sheet "New Happy Path" ("Give the ideal step sequence a name.") asks for a name. Creating a path also switches the pane into edit mode.
2. Use **＋ Add step…** to append steps in the intended order. The dropdown lists only steps that are not yet used anywhere in the path — a step may appear **only once** across the whole path, branches included.
3. Reorder with **▲ Move up** / **▼ Move down** on each row, and remove a step with the red ⊖ button.
4. For genuine alternatives, press **⑂ Split** ("Add a split — alternatives that rejoin and continue"). The split card offers **✎ Rename split** ("Rename Split"), **▲/▼**, 🗑 **Delete split**, one embedded editor per branch, **＋ Branch** to add another, and 🗑 **Delete branch** on branches beyond the second.
5. When steps follow the split, name the convergence point with **✎ Rejoin: <name>** (prompt "Name Rejoin", confirm **Save**). Steps after the split are the shared continuation of every branch.
6. Press **✓ Done** to leave edit mode. Use **↻ Recompute conformance** at any time to re-run the scoring.

Splits may be nested: a split can be placed inside a branch of another split. The helper text in the editor states the rule the scoring follows: "Each journey is scored against the route it matches best."

Top-bar control	Behaviour
Path selector	Dropdown of the project's happy paths; reads "No happy paths yet" when none exist.
＋ New	Creates a path and enters edit mode.
✎ Rename	Prompt "Rename Happy Path", confirm button **Rename**. Only shown when a path is selected.
🗑 **Delete**	Deletes the selected path **immediately, without a confirmation**.
Conformance N.NN	Score badge, shown once computed. Green ≥ 0.70, orange in between, red ≤ 0.30. Tooltip: "Journey-count-weighted edge coverage".
✎ Edit Path / ✓ Done	Toggles edit mode.

> **Warning** Happy-path deletion takes effect at once — there is no confirmation dialogue and no undo. Rename rather than delete when you are unsure.

Reading the visualisation

Outside edit mode the path is drawn as a vertical trunk of step cards joined by ⌄ chevrons.

- A card is **green-tinted** when that step exists in the currently filtered process (tooltip = the step name) and **grey** when it does not (tooltip "<name> — not present in the filtered process"). A grey card means the step you designed into the process never occurs under the current filters at all.
- Green cards carry a **coverage percentage**: `min(1, max(incoming, outgoing) ÷ filtered journeys)` — the share of filtered journeys that touch this step. A step at 100 % is on every journey; a step

at 12 % is an exception in practice, however central it is on paper.

- A split renders as a "⑂ <label or 'Split'>" fork line followed by side-by-side branch columns headed **Branch 1**, **Branch 2**, … (an empty branch shows "No steps"), and, when a shared continuation exists, a "⑃ <rejoin label or 'rejoin'>" line before it.

Two helper states replace the trunk: with no path selected the pane explains "Define an ideal step sequence to measure how closely real journeys follow your intended design. Create one with "+ New"."; with an empty path it shows 🪧 **No Steps Defined** — "Use + to add the first step." in edit mode, or "Enable Edit Path to define the happy path." otherwise.

How routes are enumerated and scored

The conformance score is computed on the server and recomputes automatically whenever the project or the selected path changes.

1. **Route enumeration.** The path is expanded into every possible start-to-end step sequence: plain step nodes append themselves, and each split multiplies the routes collected so far by all routes through each of its branches. Nested splits are expanded recursively. The expansion is capped at **2 000** routes so that heavily nested paths cannot explode; routes shorter than two steps are discarded because they contain no transition to compare.
2. **Edge sets.** Each enumerated route and each observed journey variant is reduced to its set of consecutive transitions ("A → B", "B → C", …).
3. **Best-match coverage.** For each observed variant the application computes, against every enumerated route, the fraction of that route's transitions the variant contains, and keeps the **highest** value. This is why a split costs nothing: a journey is measured against the branch it actually took.
4. **Weighting.** Coverage values are averaged weighted by the number of journeys behind each variant, so a route walked by 5 000 journeys moves the score far more than one walked by three. The result is the 0–1 figure in the badge.
5. **Variant limit.** The scoring is run against the top **500** journey variants of the current filter. Extremely long-tailed processes are therefore scored on their meaningful mass, not on every one-off route.

> **Note** The score is *edge coverage*, not equality. A journey that walks the whole happy path and then adds three rework loops still scores 1.00 for that path, because every intended transition is present. Use the Conformance Check norms (below) or the Statistics route table to detect the extra traffic.

Happy paths also feed the AI supported Documentation report: if none are defined, that view warns that conformance data will not be included.

Conformance Check

> Requires: **Power user** or **Administrator**

Conformance Check compares the *measured* value of each transition against a target value you store on that transition — its **norm**. Norms are held per project **and per metric** in backend storage, so the Count

norms and the Avg Time norms of the same edge are independent, and norms follow your account to any browser.

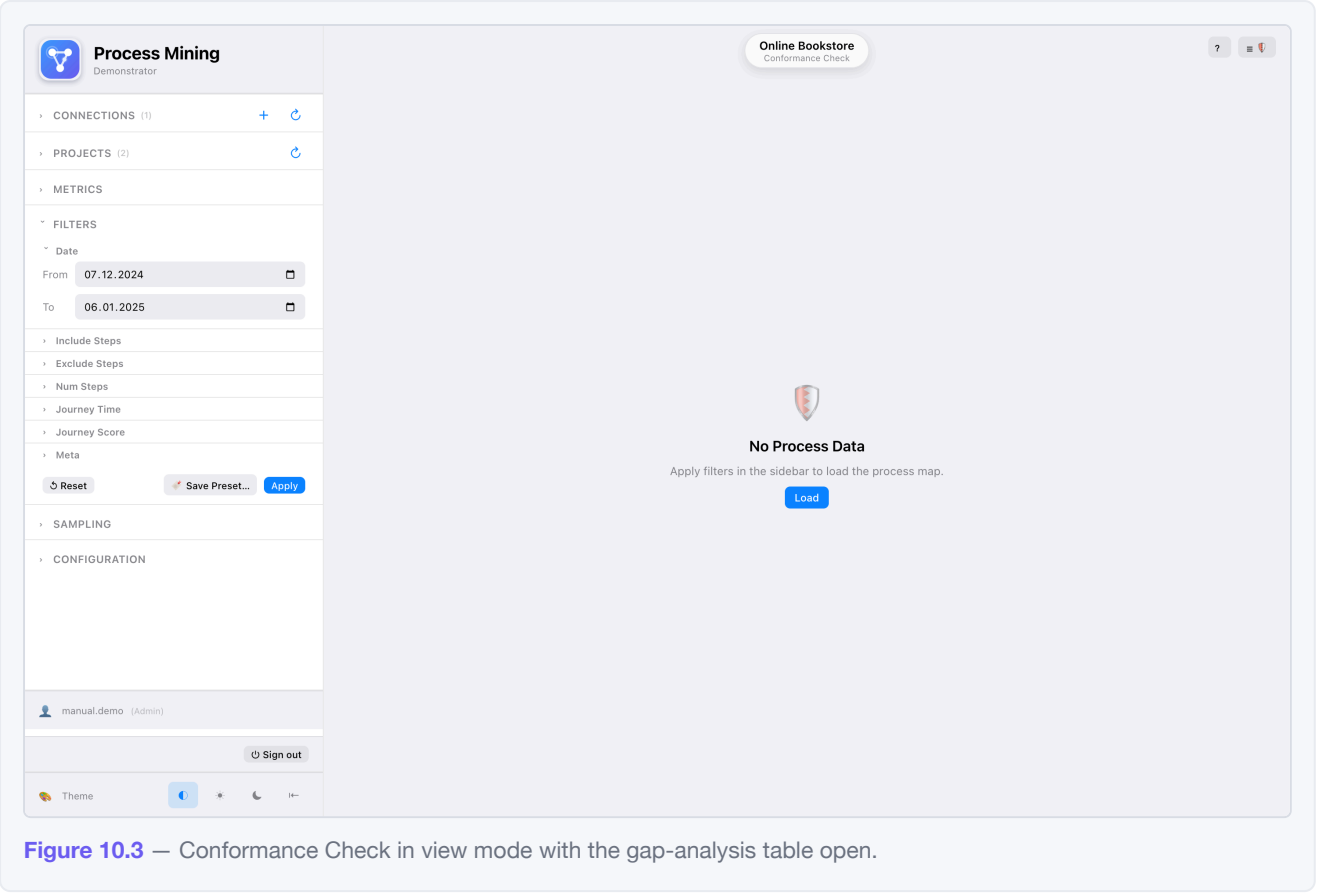


Figure 10.3 — Conformance Check in view mode with the gap-analysis table open.

The toolbar

Control	Behaviour
Metric chips (# Count, ⏱ Avg Time, ⌄ Min Time, ⌃ Max Time, 〰 Std Dev)	Select which metric's norms you are viewing and editing. The choice persists per project.
Norm is a minimum (checkbox)	Setting `compliance.normIsMinimum`, default **off**. Off: the actual value must be ≤ the norm to comply (the norm is a cap — the usual reading for times). On: the actual value must be ≥ the norm (a floor — the usual reading for throughput shares).
Show gaps / **Hide gaps**	Opens and closes the 460 px gap-analysis table on the right. Tooltip: "Toggle the gap-analysis table".
Edit norms / **Done**	Switches between view and edit mode; the button is highlighted while editing.
KPI handle (chevron)	Collapses the standard KPI strip (`compliance.kpiExpanded`, default on). Collapsed it summarises "N violations · M compliant".

Under the toolbar a status line reads "❌ N violation(s) ✅ M compliant — K without norm". In edit mode it is joined by the hint "Click an edge to set its norm".

View mode and edit mode

In **view mode** the flowchart is a compliance overlay: edges are green (compliant), red (violation) or grey (no norm defined), their width still encodes the actual value, and each label is a solid badge showing the **actual** value. In **edit mode** every edge is drawn uniform grey at width 6 and the badge shows the **stored norm** instead ("—" when none is set; the badge is accent-coloured where a norm exists and faint grey where it does not). The deliberate flattening in edit mode stops the visual weight of the data from distracting you while you set targets.

Setting a norm

1. Select the metric whose norm you want to set.
2. Press **Edit norms**.
3. Click the edge (the line has a generous invisible hit area, and the label is clickable too). The norm editor opens, headed "<from> → <to>" with the line "Actual: <value>" — for Count the wording is "Actual: <value> of outgoing".
4. Type the target. The placeholder is **Target %** for Count and **Target value (seconds)** for the time metrics. Press **Enter** or **Set** to save, **Clear** to delete the norm.
5. Press **Done** to return to the compliance overlay.

> **Note** Pressing **Set** with an empty or non-numeric input clears the norm rather than storing a zero. Clearing is therefore always safe and never leaves a hidden "0" target behind.

Count norms are percentages

For the Count metric a norm is not an absolute number of traversals — it is the transition's **share of the source node's outgoing flow**, so the norms leaving one step should total 100 %. The editor helps you keep that budget: it shows "X% on N other edge(s) from "<step>"" and "**Y% remaining** to reach 100%", and if your entry would push the node over, it displays a red border and the message "Total would be Z% — over 100%".

> **Warning** The budget helper warns but does not block. You can store a set of Count norms that adds up to more than 100 % of a node's outgoing flow; the compliance colours will then be arithmetically unreachable for at least one edge.

Reading the gap table

Press **Show gaps** for a tabular reading of the same data. Rows are sorted by severity — violations first, then by descending delta — so the worst offender is always the first row. Self-loop edges (a step following itself) are excluded from the table.

Column	Content
From / **To**	The transition's source and target step.
Actual	The measured value under the current filters, formatted per metric (Count as a percentage, time metrics as a duration, otherwise an integer).
Norm	The stored target for the selected metric.
Delta	Distance between actual and norm — the size of the gap.
Status	❌ **Violation** (red), ✅ **Compliant** (green) or — **No norm** (grey).

A **violation** means only this: for the selected metric, this transition's measured value falls on the wrong side of the target you stored, under the direction set by **Norm is a minimum**. It is not a statement about the journey as a whole, and it changes as soon as you change the filters, the metric or the direction checkbox. Transitions with no norm are never violations — they are simply unmeasured, which is what the "— K without norm" figure in the status line is for.

With no project selected the view shows 🗺️ **No Project Selected**; with no data loaded it shows 🛡️ **No Process Data** — "Apply filters in the sidebar to load the process map." with a **Load** button. The node layout of this view is stored under the chart mode "Conformance".

Simulation

> Requires: **Power user** or **Administrator**

Simulation fits a Markov model to the process graph currently loaded in the **A-Chart** and generates synthetic journeys from it: transition probabilities come from the observed occurrence counts, edge durations are drawn from a lognormal distribution fitted to each edge's average and standard deviation, and journeys arrive as a Poisson process. Nothing is written to the database — the result lives in the selected simulation slot for the session and can be exported or used as an A/B data source.

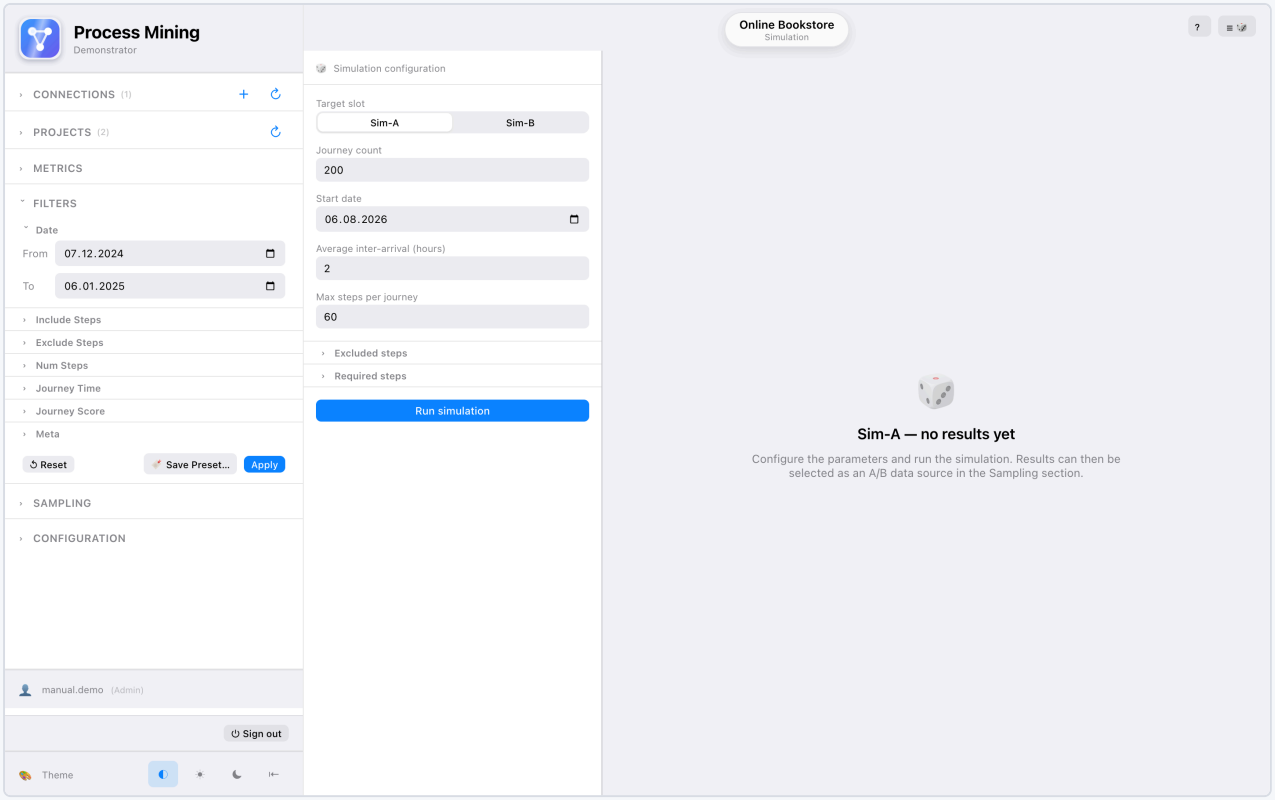


Figure 10.4 — Simulation: configuration column on the left, results with the Flow, Variants, Charts and Event log tabs on the right.

Configuration parameters

The left column, headed "🎲 Simulation configuration", is 340 px wide and holds every parameter.

Parameter	Default	Range / notes
Target slot (segmented Sim-A | Sim-B)	Sim-A	Two independent result slots, so one simulation can be compared with another in A/B Comparison.
Journey count	200	Minimum 1. The backend refuses runs above 100 000 (see below).
Start date	today	Timestamp of the first simulated arrival.
Average inter-arrival (hours)	2	Minimum 0.1, step 0.1. Mean of the exponential inter-arrival distribution; must be a finite number.
Max steps per journey	60	Minimum 2. Hard stop for a random walk that never reaches an end-of-process step. The backend limit is 10 000.
Excluded steps	none	Multi-select picker (◉ selected / ○ not) with a count pill and an ⊗ clear button. Hint: "Removed before the Markov model is built." — the step and all its transitions are cut out before probabilities are computed.
Required steps	none	Same picker. Hint: "Journeys not visiting all of these are discarded." — generated journeys missing any required step are thrown away and regenerated.
Resources per step	none	Collapsible list of every step with a factor field (default 1). The factor multiplies the time of every transition *leaving* that step: <1 = more resources (faster), >1 = fewer (slower). The field accepts any positive number — whole or fractional, above or below 1 (e.g. `0.5`, `1.5`, `12`) — and only factors that differ from 1 are sent.
Transition time overrides	none	Collapsible editor of per-transition what-ifs. Each row picks a `from → to` edge and either scales its observed mean (`×`) or sets an absolute mean in minutes (`=`); the row shows the observed time and the resulting target. A step's resource factor still applies on top.

A walk ends when it reaches a step flagged as end-of-process, when the model has no outgoing transition left, or when **Max steps per journey** is reached — whichever comes first.

> **Note** The whole configuration is **retained while you move to another view and back** — target slot, counts, dates, excluded/required steps, the resource levers and even which sections are expanded. It is not reset on each visit, so you can flip between Simulation, the charts and A/B Comparison without re-entering anything and only reset a value when you actually want to change it.

> **Note** With **Required steps** set, a run can finish with fewer journeys than you asked for. The generator makes a bounded number of attempts (twenty times the journey count, and never fewer than a thousand); if the required combination is rare in the observed graph, the attempts run out first. Compare the **Journeys** tile in the results header with the journey count you configured.

Resource levers: transition-time what-ifs

Excluding a step models removing work; the **resource levers** model the other kind of change — keeping the process shape but altering how long parts of it take, to see what dedicating more (or fewer) resources to a step does to the whole graph. Routing probabilities are untouched; only the durations move, and cycle time, bottlenecks and variant timing all respond. Two levers combine:

- **Resources per step** — a single factor scaling the time of every transition leaving a step. A factor of `0.5` on a step says "double its capacity", halving the time cases spend leaving it; `2.0` halves the capacity. It is the quick "throw staff at this step" knob.
- **Transition time overrides** — finer control over one `from → to` hand-off: either scale its observed mean by a multiplier (`×0.5` = twice the resources on that specific transition) or set an absolute mean time in minutes. A step's resource factor still applies on top of an edge override, so the two compose multiplicatively.

Internally, each transition's duration is drawn from the lognormal fitted to its observed average and standard deviation; a lever multiplies that draw. Because scaling a lognormal by a constant scales its mean by the same constant while leaving its shape unchanged, the spread and right-skew stay realistic — the whole distribution shifts faster or slower rather than collapsing to a single value. An absolute override re-centres the mean on the target you type and keeps the same relative spread. Levers that resolve to no change (a factor of 1, or a non-positive or non-finite value) are ignored.

> **Note** The resource levers are a *direct* time what-if, not an emergent queue. Cycle times are still sampled independently per event, so raising the journey count or the arrival rate alone will not create waiting time — to study contention, set the levers to the slower times you expect under load. The natural workflow is to run the unchanged process into **Sim-A** and the lever-adjusted version into **Sim-B**, then compare them in A/B Comparison (including a Process Similarity score).

Running a simulation

1. Load the A-Chart with the filters that describe the population you want to model. Simulation has no filters of its own — it consumes the A-Chart's graph.
2. Switch to **Simulation** and choose the target slot, **Sim-A** or **Sim-B**.
3. Set journey count, start date, average inter-arrival and max steps per journey; add excluded and required steps if needed.
4. Press **Run simulation**. The button shows a spinner and is disabled while the run is in flight, and whenever no A-Chart graph is available.
5. Read the results in the tabs on the right. Re-run as often as you like — each run replaces the content of its slot.

Before any run the results area shows 🎲 **Sim-A — no results yet** (or Sim-B) with the text "Configure the parameters and run the simulation. Results can then be selected as an A/B data source in the Sampling section." With no project selected it shows 🗺️ **No Project Selected** — "Select a project from the sidebar."

Reading the results

The results header carries six tiles: **Journeys** 👥, **Avg cycle time** ⏱, **Shortest** 🐇, **Longest** 🦕, **Std dev** 〰 and **Variants** ⑂. Below them a segmented control switches between four tabs, with ⤓ **Export CSV** alongside.

Flow

The simulated process graph drawn in the standard flowchart, with the currently selected transition metric, the usual zoom and fit controls, and a node layout that persists per slot (chart mode `sim_<slot>`). This is the tab to compare against the A-Chart: the shapes should match, and where the simulated flow

differs from the observed one, the Markov assumption — that the next step depends only on the current step — is not holding for that part of the process.

Variants

Table with **Path**, **Count**, **Share (%)** and **Avg cycle time**, ordered by count.

Charts

"Cycle-time distribution" — a 12-bucket histogram of journey durations — and "Top variants" — the first 12 variants as bars labelled V1…V12, with the full path in the tooltip.

Event log

Journey | Step | Timestamp table, prefaced by the note "Showing the first 1 000 of N events. Use "Export CSV" for the full log."

⤓ **Export CSV** downloads the *complete* event log — not the 1 000-row preview — as `Sim-A-event-log.csv` or `Sim-B-event-log.csv`, with the columns `journeyId`, `step`, `timestamp`.

Using a simulation as an A/B data source

Each simulation slot can stand in for real data on either side of the A/B Comparison.

1. Run a simulation into **Sim-A** (and, for a two-scenario comparison, a second one into **Sim-B**).
2. Switch to **A/B Comparison** and make the side you want to change active by clicking its panel header.
3. In the sidebar **Sampling** section choose **Sim-A** or **Sim-B** as that side's data source instead of Original or Sample 1–3.
4. The panel reloads from the simulation. Its **Active Sample** KPI tile now reads "Sim-A" or "Sim-B" with that run's journey count.

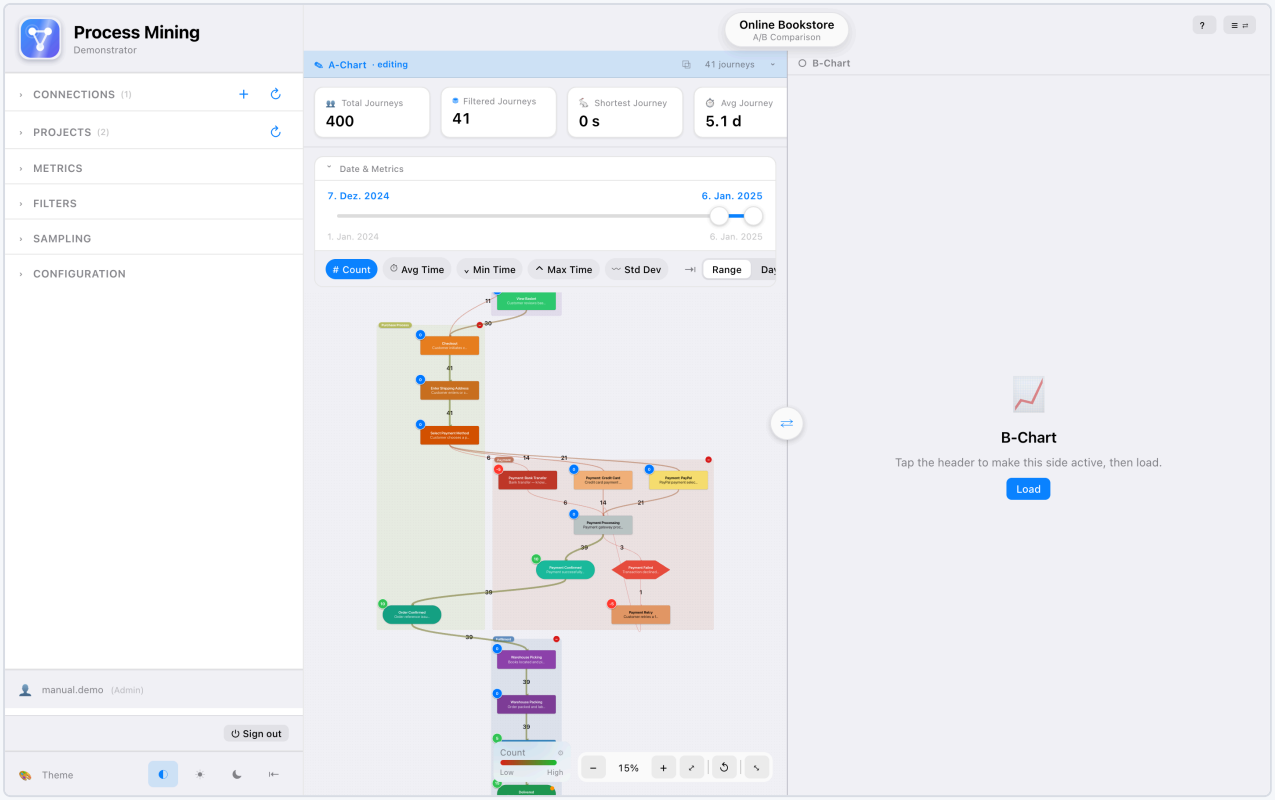


Figure 10.5 — A simulated scenario on one side of the A/B Comparison, with the similarity badge below.

Re-running a slot that is currently displayed as an A/B data source refreshes that side automatically — you do not need to reselect it.

> **Warning** Simulation output is **read-only** wherever it appears. The canvas shows a red notice — "Sim-A: simulation data is read-only — interactive filtering, drill-down and step editing are not available." — and the ✓ **Require in journeys** and ⊖ **Exclude from journeys** items disappear from the node popover. To change what a simulated scenario contains, change the **Excluded steps** / **Required steps** configuration and run it again.

Guard rails and error messages

A simulation runs inside the shared backend, so an oversized request would slow down or crash the service for every user. The size is therefore checked *before* the run starts and rejected outright; the message appears in red above the **Run simulation** button.

Condition	Message
Journey count above 100 000	"Too many journeys to simulate (<count>). The maximum is 100,000."
Max steps per journey above 10 000	"maxStepsPerJourney (<steps>) exceeds the limit of 10,000."
Journeys × steps above 20 000 000	"The requested run is too large (<count> journeys × <steps> steps). Keep journeys × steps under 20,000,000."
Non-finite inter-arrival value	"avgInterArrivalHours must be a finite number."
Run still going after 120 seconds	"Simulation timed out (<count> journeys). Try fewer journeys."
Out of memory during the run	"Simulation ran out of memory (<count> journeys). Try fewer journeys."
Any other server-side failure	"Simulation failed. Please try again." (the detail is written to the server log, not returned to the browser)

Both the product limit and the per-dimension limits exist because a cyclic process graph generates one event per step: a modest journey count combined with a very large **Max steps per journey** produces just as many events as a huge journey count. Every rejection is also recorded in the backend log under the operation "simulation", so an administrator can see who hit which ceiling.

A second guard applies before the request is even sent. If the A-Chart holds no process graph, the configuration column shows an orange warning — "Load the A-Chart first — simulation needs an observed process graph." — and the **Run simulation** button stays disabled; attempting the run from elsewhere reports "Load the A-Chart first — simulation needs a process graph."

> **Tip** Start small. A run of the default 200 journeys completes in a moment and tells you immediately whether the model produces plausible variants; only then scale the journey count up for a distribution you intend to quote.

11 AI Documentation and Notes

> This chapter covers the two views that turn a process map into written record: **AI supported Documentation**, which sends the current A-Chart transition table to a language model and renders the answer as a structured report, and **Notes**, the annotation layer that lets you attach threaded, prioritised comments to individual steps and transitions. Both are available to every signed-in user; neither requires the power or administrator role. Read the AI section if you produce process documentation for others, and the Notes section if you work with colleagues on the same database — notes are stored in the database itself, not in your browser, so they travel with the data.

AI supported Documentation

The **AI supported Documentation** view (🧠 in the ≡ **Switch view** menu) produces a written analysis of your process, in **two stages with a clear division of labour**. First a language model does the **analysis**: it is sent **only the current A-Chart transition table** and an instruction, and returns **structured findings** (a title, an executive summary and numbered sections). Then the server does the **assembly**: it lays those findings out — together with the process-flow **Sankey** and the locally-computed **Happy Path** and **Conformance** sections — into one polished, self-contained HTML report, styled with the accent colour and letterhead set by your administrator. The report is shown in its own frame; 🖨 **Save as PDF** exports it via the browser's print-to-PDF (the same way the reference report was produced).

The report's language model is configured once by an administrator in the admin **Reporting** tab, while the **style & sections** (accent, letterhead, logo and position, and which optional sections to include) and the **analysis prompt** are both configured **per connection and project** — so a single installation can brand and tailor reports differently for each customer. A project with no style of its own uses a plain default theme. The report resolves its language model live on every run: the **active connection's own LLM overrides** the global default set in the admin Reporting tab, and if the connection defines none the global default is used. Because it is read live, editing a model in the admin takes effect on your next report with no reconnect.

> **Note** The view **always analyses the A-Chart dataset and the A-Chart filters**, no matter which view you were in when you switched to it. If you want the report to describe a particular segment, set that segment up in the A-Chart first, tap **Apply**, and then switch to AI supported Documentation. The B-Chart, samples and simulation slots are never used as the source.

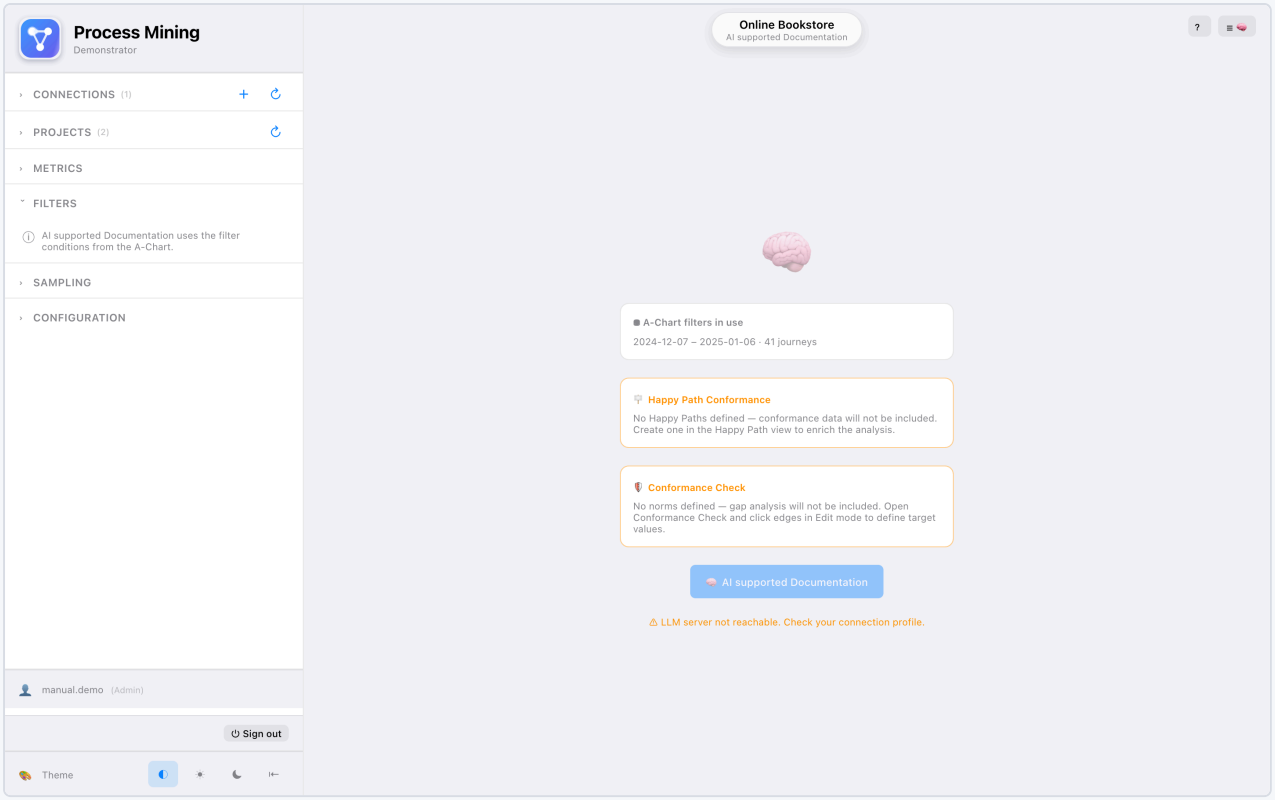


Figure 11.1 — The AI supported Documentation view: the three pre-run notice cards summarise the A-Chart filter context and the locally computed material that will be appended to the report.

What is sent to the language model

Only two things leave the deployment: the **transition table of the currently loaded A-Chart** (the aggregated directly-follows pairs behind the map you can see) and the **prompt template** stored for this project. They are posted to the base URL of the **effective LLM** — the active connection's own LLM if it defines one, otherwise the global default configured in the admin Reporting tab.

Everything else in the report is computed on your own servers and is **never sent to the model**:

Report material	Origin	Sent to the LLM?
Transition table of the A-Chart	Exasol query behind the current map	Yes
Prompt template	Stored per project (sidebar Configuration)	Yes
Journey Paths chapter	Computed locally	No
Happy Path Conformance chapter	Computed locally	No
Conformance – Gap Analysis chapter	Computed locally	No
Process-flow Sankey diagram	Rendered in your browser, embedded as vector art	No
Notes chapter (project notes)	NOTES table in your Exasol database	No

Warning If the connection points at a **cloud** model endpoint, the transition table — step names and their aggregated counts and durations — leaves your network and is processed by that provider. Step names frequently carry business meaning. Where that matters, ask your administrator to attach a **locally hosted** OpenAI-compatible server (Ollama, llama.cpp, vLLM or LM Studio all work) so that nothing crosses the network boundary. Individual journeys, event IDs and meta values are never part of the payload — only the aggregated transition table is.

Warning AI models can produce results that are incorrect, incomplete or misleading. **Verify every finding independently** before acting on it or circulating the report.

Requirements

The feature is only usable when the connection you are currently connected to has a model server attached and that server answers.

A report language model

The report is analysed by the **effective LLM**, resolved per run: the **active connection's own LLM takes precedence**, and when it defines none the **global default** from the admin **Reporting** tab (its own base URL, model and optional key) is used. Both are read live, so an edited model applies to the next report without a reconnect. Because a global default can always stand in, the run button is **not** gated on the connection's LLM reachability dot — you can generate a report even when the connection carries no interactive LLM.

The connection's own LLM (optional)

Independently, a connection may bundle its own OpenAI-compatible LLM server for the interactive features; when attached, the connection card shows an **LLM** line and a second status dot — **blue** when reachable, **orange** when not. That dot reflects the interactive LLM, not the report LLM.

A selected project

Without one the view shows  **No Project Selected**.

Requires: **any signed-in user.** The LLM base URL, model and API key are configured by an administrator (or, for their own connections, by a power user) — you cannot set them from this view. API keys are encrypted at rest and are never sent to the browser.

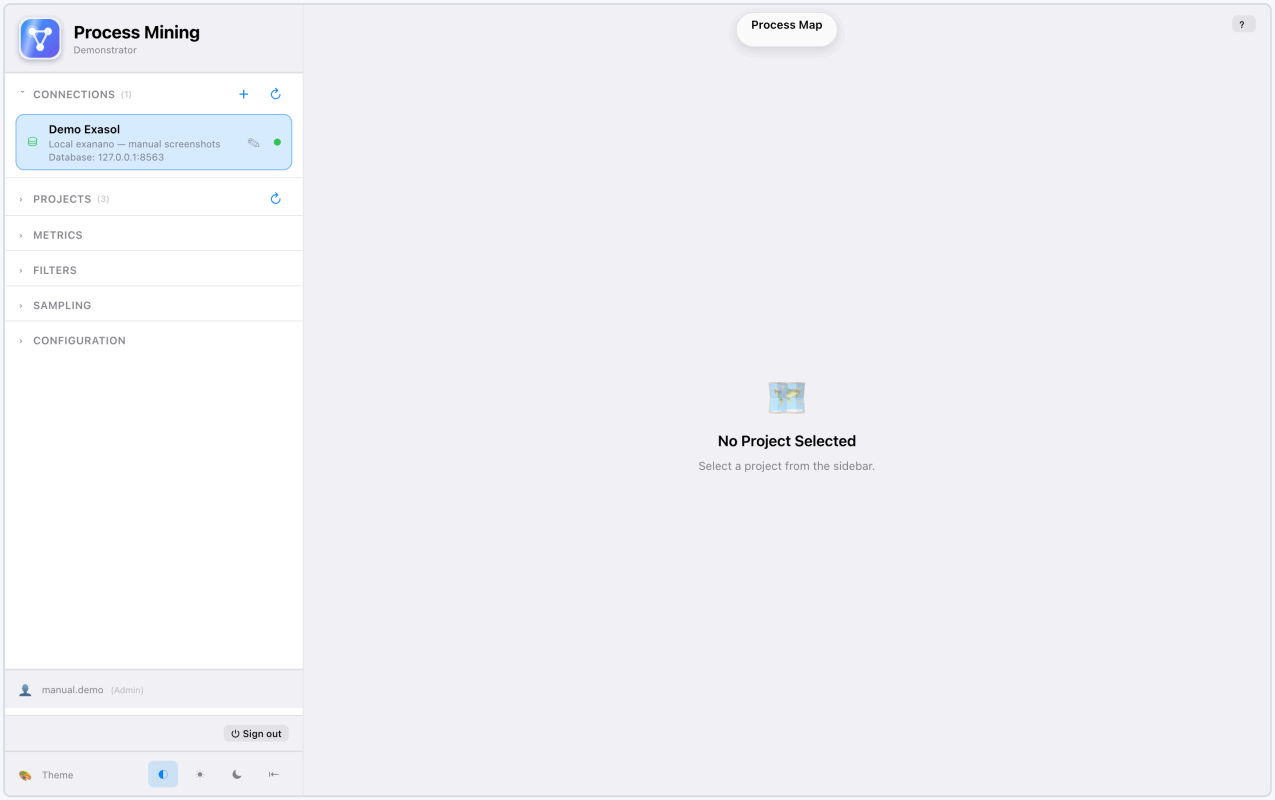


Figure 11.2 — Connection cards in the sidebar. The second status dot is blue when the attached model server is reachable — the precondition for running an AI analysis.

The pre-run screen

Before the first run the view shows a large 🧠 glyph and three notice cards that tell you exactly what the report will be based on. Read them: two of the three warn you when material is missing.

Card	Content
● **A-Chart filters in use**	A one-line summary of the dataset: `<from> – <to> · N journeys · include: … · exclude: … · <meta values>`. This same summary is repeated in the report header and on the report toolbar.
Happy Path Conformance	Warning-orange when nothing is defined: "*No Happy Paths defined — conformance data will not be included. Create one in the Happy Path view to enrich the analysis.*" Otherwise "*N path(s) will be evaluated: <names>*".
Conformance Check	Warning when no norms exist: "*No norms defined — gap analysis will not be included. Open Conformance Check and click edges in Edit mode to define target values.*" Otherwise "*N norm(s) across M metric(s) — gap analysis will be appended to the report.*"

Note The two warning cards are not errors — the report runs perfectly well without happy paths or norms, it is simply thinner. Note also that **Happy Path and Conformance Check are power/admin-only views**. As a regular user you can read a report enriched with them, but you cannot create the underlying paths or norms yourself; ask a power user or administrator to define them once, after which every user's report includes them.

Running an analysis

1. Connect to the connection whose project you want to document. The report uses the **report LLM** set in the admin Reporting tab, so the connection does not need its own interactive LLM attached.
2. Open the **A-Chart**, set the date window, included/excluded steps and meta filters you want the report to describe, and tap **Apply**.
3. Open the ≡ **Switch view** menu in the top-right corner and choose **AI supported Documentation** (🧠).
4. Check the three notice cards. If the Happy Path or Conformance Check card warns that material is missing and you want it included, define it first and come back.
5. Tap the 🧠 **AI supported Documentation** run button. While the model works, the view shows a spinner, the text **"AI is analyzing"** and the filter summary.
6. When the answer arrives the report replaces the pre-run screen.

If the run fails, the view shows ⚠ **Analysis Failed** with the error message returned by the server and a ↻ **Try Again** button. A common cause is the report LLM being unconfigured or unreachable — check the admin **Reporting** tab's **Test** button.

Reading the generated report

The report opens with a **cover** — the model's report title and subtitle, your letterhead, the line *Prepared for <name>* with the generation date, and the model's **executive summary** — followed by a **clickable table of contents**. Each entry in that contents list jumps to its chapter, both on screen and in the exported PDF. Every chapter then **starts on a fresh page under a large section title**, so the printed report reads one topic per page. The chapters, present only when their material exists, are:

1. **Analysis** — the model's findings, laid out as numbered sections with any tables it returned.
2. **Process flow** — the process-flow **Sankey** embedded as a vector image, with its caption.
3. **Conformance – Gap Analysis**, **Happy Path Conformance** and **Journey Paths** — the locally-computed material, each its own chapter, present only when the corresponding paths or norms exist.
4. **Notes** — the project's annotations, split into **Open** and **Resolved** sub-tables, each **sorted by severity (most severe first)** and then by date. The columns are **Severity**, **Element**, **Title**, **Note**, **Author** and **Date**. This is where the annotation layer described in the second half of this chapter enters the printed record.

The report toolbar carries the filter summary and three controls:

Show prompt

Opens the sheet **"Prompt sent to the LLM"** containing the exact prompt text that was transmitted. Use it when you need to reproduce or audit a result.

🖶 Print / PDF

Prints **just the report** — not the surrounding application — through the browser's own print dialogue. Choose "Save as PDF" there to file the report.

↻ Re-run

Runs the analysis again with the current filters and prompt, using the report language model configured in the admin **Reporting** tab.

> **Note** Markdown support in the report is deliberately restricted. Headings, bold, italic, inline code, fenced code blocks, bullet and ordered lists, block quotes, horizontal rules and pipe tables all render. Links are limited to `http`, `https`, `mailto` and relative targets — any other scheme is neutralised — and raw HTML is sanitised, so scripts and event handlers in a model's answer are stripped before display. Images and nested lists are not rendered.

Configuring the prompt

The prompt is the instruction the model receives alongside the transition table. It is edited from the sidebar under **Configuration** → **AI related** → **LLM Prompt** and is stored **per connection and project** — the very same mapping an administrator manages in the admin **Reporting** tab, so a change here is reflected there and vice-versa. Editing it is a **power-user, developer or administrator** capability: regular users do not see the LLM Prompt entry at all, and the report simply uses whatever prompt those roles have configured (or the built-in default when none is set). You must be connected and have a project selected to edit it; leaving the text empty clears the stored prompt so the default applies.

1. Sign in as a power user, developer or administrator, connect, and select the project whose prompt you want to change.
2. Open the sidebar, expand **Configuration**, then the **AI related** group, and tap **Edit** next to **LLM Prompt**.
3. Adjust the instruction (or tap **Use Default** to start from the built-in template) and **Save** — it is stored for the current connection and project.
4. Switch to **AI supported Documentation** and tap **↻ Re-run** — or the run button, on a first run — to apply it.
5. Confirm with **Show prompt** that the text sent matches what you intended.

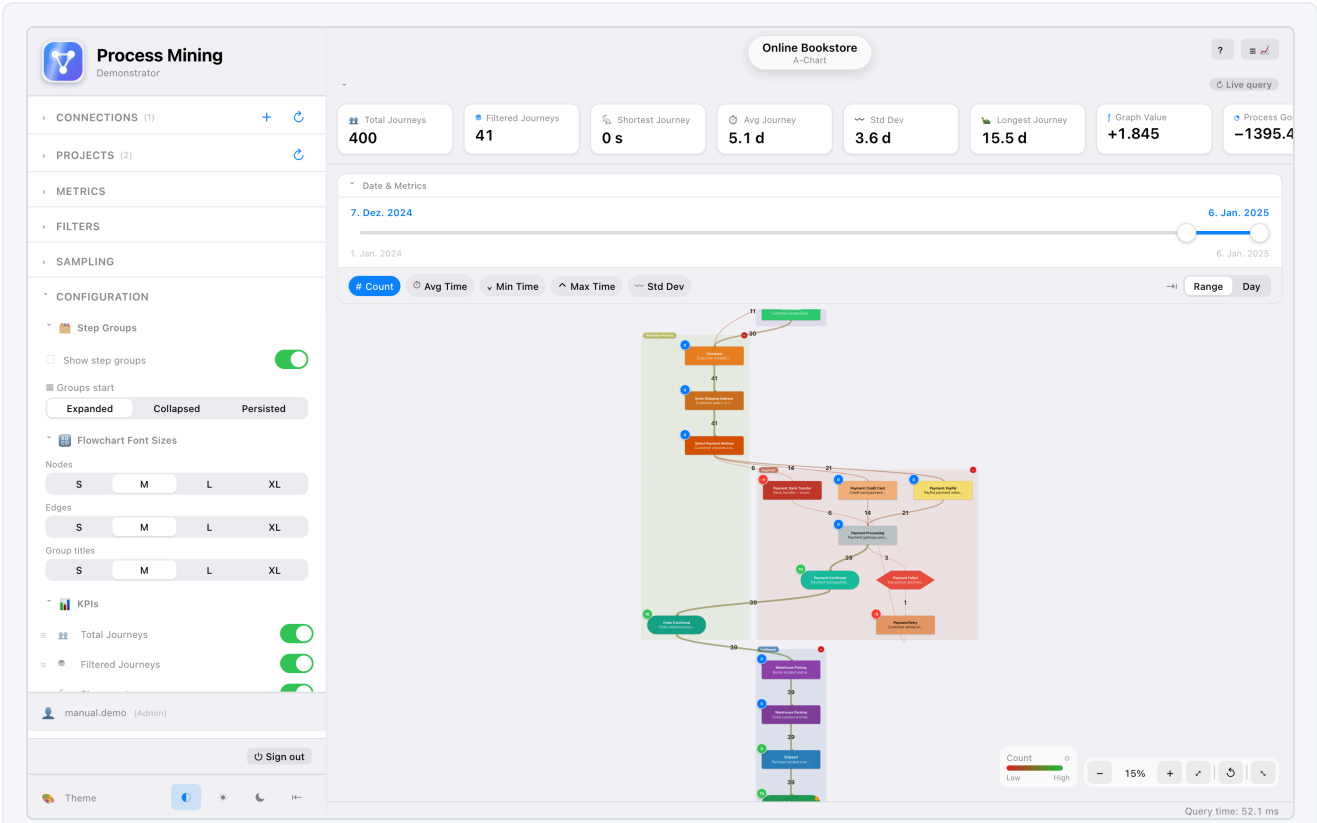


Figure 11.3 — The sidebar Configuration section, with display options, KPIs and the step editor. The **AI related → LLM Prompt** editor appears only for power users, developers and administrators.

Troubleshooting a blank or failing report

Symptom	Cause and fix
The run button is greyed out and an orange note reads "⚠ *LLM server not reachable. Check your connection profile.*"	The connection has an LLM server attached but it does not answer. Confirm the second status dot is orange rather than blue, then ask your administrator to check the base URL, model name and API key — those live only in the administrator's configuration and cannot be corrected from the app.
The report stays blank and no run button appears at all.	The active connection has **no** LLM server attached — the card shows only a **Database** line and a single status dot. Ask an administrator (or the power user who owns the connection) to attach an OpenAI-compatible server.
⚠️ **Analysis Failed** with an error message.	The model server was reached but rejected or aborted the request. The message comes from the server itself; read it, then use ↻ **Try Again**.
🗺️ **No Project Selected**.	Choose a project in the sidebar first.
The report describes the wrong data.	The view uses the A-Chart's dataset and filters, never the view you came from. Re-filter the A-Chart, tap **Apply**, then ↻ **Re-run**.
Happy-path or gap-analysis chapters are missing.	No happy paths or no norms are defined for this project — the pre-run cards say so explicitly. Both are defined from the power/admin-only Happy Path and Conformance Check views.

Notes

Notes are annotations attached to the process map. Click a step or a transition, add an observation, and it stays with that element — visible to you on every later visit, and to your colleagues when you share it. Each note is a **thread**: other users add comments underneath, and the note is closed by marking it resolved rather than deleted.

Notes are stored in a **NOTES table in your Exasol database**, created automatically on first use if the connecting database user holds CREATE TABLE rights. Because they live in the database rather than the browser, notes travel with the data and are visible to every user of that database — subject to the shared flag on each note — regardless of which machine wrote them.

> **Note** A note always attaches to a **node** (a process step) or an **edge** (a transition between two steps). There is no free-standing note; the **Notes** view is the project-wide collection of every note recorded against that project's nodes and edges, and it is that collection which is appended to the AI report as the **User Comments** chapter.

Who a note belongs to

The author of a note is the **signed-in application user** — not the shared database login used by the connection. For directory (LDAP) accounts the author is shown by real name, taken from the directory's `cn` attribute. Every note also records who last contributed to the thread and, importantly, the **complete filter context at the moment of creation** — the date window, the included and excluded steps and the meta values then in force — so an observation such as "this loop is unusually heavy" remains interpretable months later when the map is filtered differently.

Creating a note on a node or a transition

1. On any process map — A-Chart, B-Chart, either A/B panel, Individual Journey, Happy Path's left pane, Conformance Check or a simulation flow — click the **node** or the **edge** you want to annotate. Edge labels are clickable, and the edge line itself carries a wide invisible hit area, so you do not have to hit the stroke exactly.
2. In the context popover choose ✎ **Show Notes (N)**, where N is how many notes the element already carries.
3. If N is zero, the **note editor** opens directly on a new note. If N is one or more, a list sheet titled **Notes — <target>** opens showing each existing note as a card, with **+ New note** and **Close**. One node or edge can carry several independent notes.
4. Fill in **Title (optional)…** and **Your note…**. The title is optional; the body is not — **Save** stays disabled for a new note until the text is non-empty.
5. Optionally set an **Importance** level and tick **Share with other users of this database**.
6. Tap **Save**.

> **Note** Simulation output is read-only, and its context popovers hide the ✓ **Require in journeys** and ⊖ **Exclude from journeys** actions — but **Show Notes** and **Show description** remain available, so you can still annotate a simulated map.

The note editor

The editor opens as a sheet titled **New Note** or **Note** (🗒). Its header names the target: ⬭ followed by the step name for a node, or ↝ followed by `from → to` for a transition, together with the importance badge and a ✓ **Resolved** marker when applicable.

Control	Purpose	Who may use it
Title (optional)… / **Your note…**	Title and body of a new note.	The creator
Notes & comments so far	Read-only history box on an existing note, newest contribution on top.	Everyone who can see the note
Comment title (optional)… / **Add a note or comment…**	Appends a comment to the thread. The comment's title becomes the note's displayed heading.	Everyone who can see the note
Mark this note / issue as resolved	Closes the note; a green ✓ **Resolved** pill then appears in the list.	Everyone who can see the note
Importance — • Normal, 𝐢 Info, ★ Important, ⚠ Urgent	Priority badge. The label reads "*Importance (set by the author)*"; the buttons are disabled for everyone else.	Author only
Share with other users of this database	Makes the note visible to other users of the same database.	Author only
Delete	Removes the note and its whole thread.	Author only, existing notes only
Cancel / **Close** · **Save**	Discard or commit. **Save** is disabled until something has actually changed.	Everyone

Below the controls a meta block records **Filter context: <snapshot summary>**, **Created <date> by <name>** and **Last activity <date> by <name>**.

Threaded comments and resolving

Opening an existing note shows the full thread read-only at the top and an **Add a comment** box beneath it. Anyone who can see the note — the author, and every other user when the note is shared — may add a comment; it is placed at the top of the thread and recorded with that person's name and the time. Giving each comment a short title is worthwhile, because the **most recent title becomes the note's heading** wherever the note is displayed, separately from the body.

Resolving is deliberately open to everyone who can see a note rather than restricted to its author: the person who fixes an issue is often not the person who spotted it. Changing importance, changing sharing and deleting are restricted to the author, so a colleague cannot silently downgrade or remove someone else's observation.

Importance levels

Level	Badge	Colour
Normal (default)	•	Muted
Info	ℹ	Blue
Important	★	Orange
Urgent	⚠	Red

Deleting a note

> Requires: **the note's author**. No role grants deletion of another user's note — not even Administrator, from within this view.

1. Open the note in the editor.
2. Tap **Delete** in the footer.
3. Confirm on the sheet **"Delete this note?"**, which warns: "*This removes the note and its whole comment thread from the database for everyone who can see it.*"

> **Warning** Deletion removes the entire thread — every comment other people contributed — from the database, for everyone. If the point is that the issue is closed rather than that the record was a mistake, tick **Mark this note / issue as resolved** instead and leave the history in place.

The Notes overview

Choose **Notes** (🗒) from the ≡ **Switch view** menu to see every note on the project in one list, **independent of the current chart filters**. Switching into this view — as into the A-Chart, B-Chart, A/B Comparison or AI supported Documentation — also reloads the notes list from the database.

With no notes yet, the view shows 🗒 **No Notes** — "*Click a node or an edge on the process map and choose "Show Notes" to add one.*"

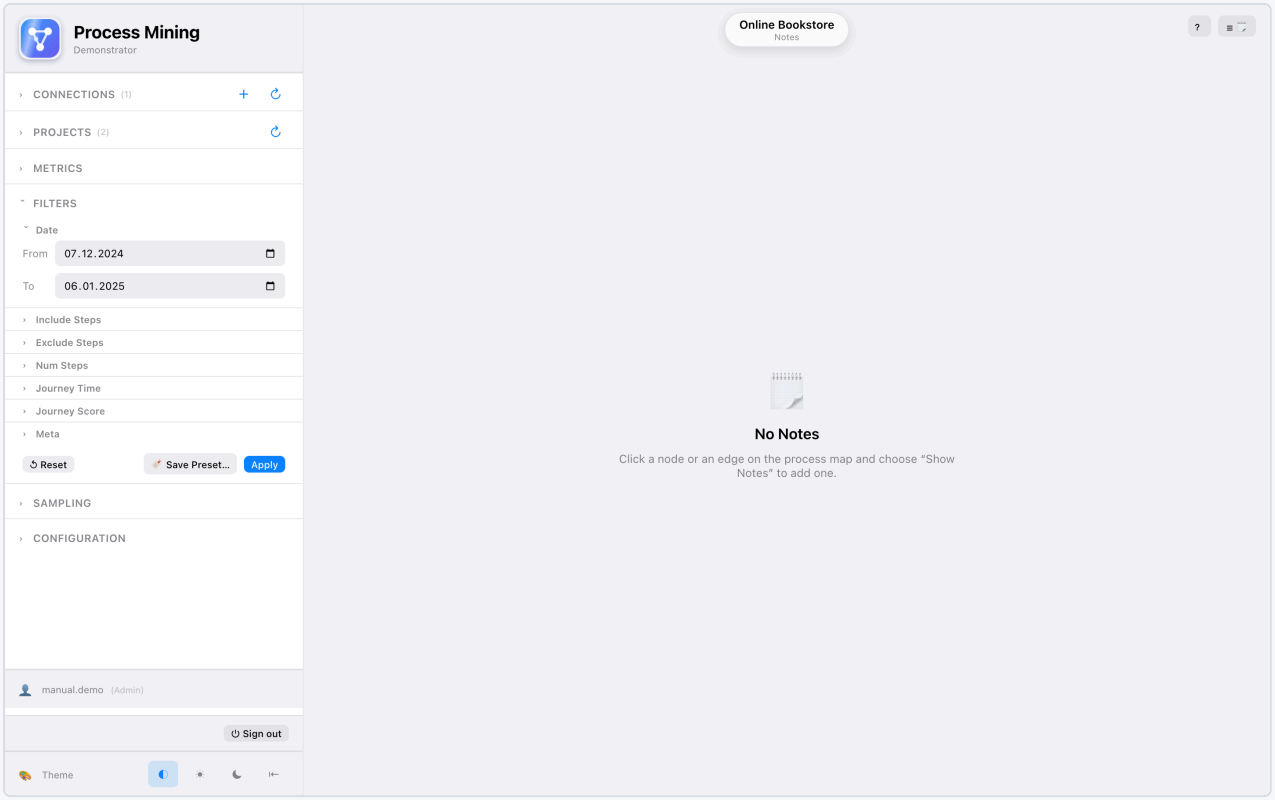


Figure 11.4 — The Notes view: importance KPI tiles across the top, search and filter rows, and grouped note cards with pagination.

A KPI strip runs across the top: **Total Notes** 🗒️, then one coloured tile per importance level (Normal •, Info ℹ️, Important ★, Urgent ⚠️), then **Resolved** ✓ in green. The per-importance tiles respect every active filter **except the importance filter itself**, so selecting one level still lets you see how the remaining notes are distributed.

Search, filters and paging

The first row holds a 🔍 search box (placeholder **Filter notes**) which matches against note text, title and target name, an ⊗ to clear it, the count **"N of M"**, and **↻ Reload**. The second row holds the filters, all defaulting to **All**:

Control	Options	Default
Type	All / Node / Edge	All
Importance	All / Normal / Info / Important / Urgent	All
Status	All / Unresolved / Resolved	All
User	All, plus one entry per distinct author (by real name)	All
Time	Any time / Last 7 days / Last 30 days / Last 90 days	Any time
Sort	Newest first / Oldest first	Newest first
Group by importance	On / off — groups the list from Urgent down to Normal under coloured headers such as "⚠ Urgent (N)"	On
Per page	5 / 10 / 20	5

Each card shows the target icon and name, the importance badge, a ✓ **Resolved** pill where applicable, a 👥 icon for shared notes (tooltip "*Shared with other users*"), the author, the creation date, the title (or "—" when there is none), the note text and the filter-context line. Clicking a card opens the editor. When the filters exclude everything the list reads **"No notes match the current filters."**

The footer pages the result: **‹ Prev** · **Page X of Y** · **Next ›** · **first–last of N**. Changing any filter or the sort order resets you to page 1 — worth knowing if you are working through a long list and adjust a filter halfway.

> **Tip** Set **Status** to **Unresolved** and **Sort** to **Oldest first** to work a backlog from the bottom up, or filter **Importance** to **Urgent** before a review meeting. Combined with the **User** filter this gives you "everything <colleague> flagged and nobody has closed".

Note badges on the map

Annotated elements are marked directly on the process map so you can see at a glance where the commentary is:

Yellow ✎ badge on a node
: The step carries at least one note. Tooltip: "*This node has notes*".

Yellow ✎ bubble beside an edge label
: The transition carries at least one note. Tooltip: "*This transition has notes*".

Node badges are drawn as part of the node-annotation layer controlled by **Show node notes** in **Configuration → Steps**, which also governs the shortened step description under the node name. The badges scale with the flowchart font settings — `graph.node.scale` for nodes and `graph.edge.scale` for the bubbles on edges — so raising the node or edge font size in Configuration makes them easier to spot on a dense map.

Troubleshooting notes

Symptom	Cause and fix
Alert **Note Not Synced** — "*Note could not be written to the database: <error>*" (or "*…could not be updated: …*")	The write to the NOTES table failed. The connecting database user needs **CREATE TABLE** the first time a note is written on a schema, and **INSERT / DELETE on NOTES** thereafter. Only your database administrator can grant those — the application cannot.
A colleague cannot see your note.	The note is not shared. Open it and tick **Share with other users of this database** — only you, as the author, can do this.
The importance buttons are greyed out.	You are not the author. Importance and sharing are author-only; you can still comment and resolve.
A note you expected is missing from the list.	Check the **Status**, **Importance**, **User** and **Time** filters and the search box, then tap ↻ **Reload**. The list is independent of the chart filters, so a chart filter is never the reason.
The yellow badges do not appear on the map.	**Show node notes** is turned off in **Configuration** → **Steps**.

> **Tip** The two halves of this chapter are designed to work together. Notes recorded against nodes and edges are collected automatically into the **User Comments** chapter of every AI report, in chronological order and identified by author. Annotating as you analyse therefore builds the human commentary for the next report at no extra cost — and because notes carry the filter context in which they were written, that commentary stays readable even when the report is generated from a different window.

PART III

Administration

12 Administration: Overview, App Control and TLS

This chapter introduces the administration interface — the separate web surface from which the whole installation is configured — and then covers its first two tabs in full. You learn how to reach the panel and sign in, how its top bar and tab strip are laid out, what the profile menu does and why the default-password banner must be dealt with on day one. The **App Control** tab is then described in detail: installing and removing the licence, reading the licence status banner, understanding Demo Mode and the one-time grace countdown, and restarting the servers. The **TLS / SSL** tab follows: the three connection modes, generating a self-signed certificate, uploading a certificate of your own, activating one, and the fact that a single restart re-binds all three browser surfaces at once. The chapter is written for administrators; nothing described here is reachable by other users.

Requires: **Administrator** — every task in this chapter

12.1 Reaching the administration interface

The administration interface is not a page inside the main application. It is a separate server, on its own port, with its own sign-in and its own session cookie. This separation is deliberate: it lets you keep the admin panel off the network path that ordinary users take, and it means an ordinary application session can never be escalated into an administrative one.

Surface	HTTP port	HTTPS port	Environment variables
Main application (the GUI)	8080	8443	`PMW_FRONTEND_PORT` / `PMW_FRONTEND_HTTPS_PORT`
Administration interface	8090	8453	`PMW_ADMIN_PORT` / `PMW_ADMIN_HTTPS_PORT`
Integration console	8100	8463	`PMW_INTEGRATION_PORT` / `PMW_INTEGRATION_HTTPS_PORT`
Actions	8110	8473	`PMW_ACTIONS_PORT` / `PMW_ACTIONS_HTTPS_PORT`

The integration console defaults to the admin ports plus ten. Which of the two port columns is live depends on the TLS mode described in section 12.6.

12.1.1 Signing in

1. Open `http://<server>:8090/` (or `https://<server>:8453/` when TLS is required) in a browser. Visiting `/` without a session redirects you to `/login`.
2. Type your **Username**. The **Sign in** button stays disabled until the username field is non-empty.

3. Type your **Password** and press **Sign in**. The button shows a spinner while the request is in flight.
4. If your account has two-factor authentication enrolled, the panel switches to the code step: enter the six-digit code (or one of your recovery codes) in **Authentication code** and press **Verify**.
5. The dashboard opens on the **App Control** tab.

The sign-in page mirrors the main application's panel exactly; the only difference is the second title line, which reads **Administration**. Where a directory server is configured, a green or red LED with *Directory server available* / *Directory server unavailable* appears below the form; it is a cached probe and never blocks the page. A 🔑 **Sign with Passkey** button is shown only when the browser supports WebAuthn *and* you reached the panel by a real host name — passkeys cannot be bound to a bare IP address.

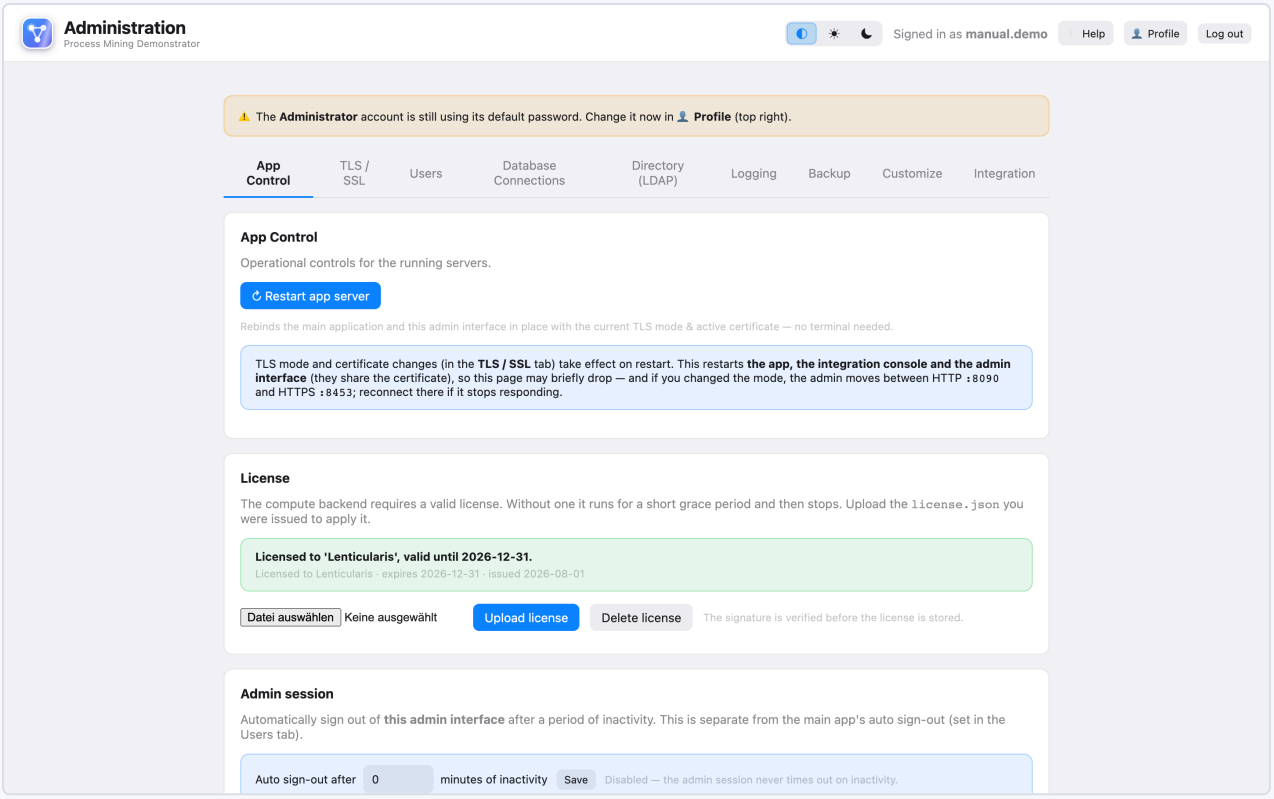


Figure 12.1 — The administration dashboard: top bar, tab strip and the App Control tab.

> **Note** Only administrators may sign in here. A correct password belonging to a non-administrator produces exactly the same message as a wrong password — "*Invalid credentials, or the account is not an administrator.*" — so the page never reveals whether an account exists. Directory (LDAP) accounts can sign in only if **Also allow directory sign-in to this admin interface** is enabled in the Directory tab *and* the account has been promoted to administrator locally. Local administrator accounts always work; they are the break-glass path.

> **Note** Admin and application sessions are not interchangeable. The admin cookie is `pmw_admin`; it is bound to the admin surface and is rejected by the application and by the integration console, and vice versa. Signing in on one surface does not sign you in on the others. Conversely, **Log out** invalidates *every* outstanding session of that user on all three surfaces, not just the cookie in the current browser.

Sign-in attempts are throttled per source IP address. After repeated failures the endpoint answers HTTP 429 with "*Too many failed attempts. Try again in about N seconds.*" An unauthenticated `GET /health` is always

open and returns `{"status": "ok"}`; use it for load-balancer probes.

12.1.2 The default-password warning

The first start of a fresh installation seeds a built-in local administrator with the username **Administrator** and the password **Administrator** (both overridable at first start with `PMW_DEFAULT_ADMIN_USER` and `PMW_DEFAULT_ADMIN_PASSWORD`). While that default password is still in place, a persistent banner sits at the top of the dashboard:

> ⚠ The Administrator account is still using its default password.
> Change it now in 👤 Profile (top right).

Warning Change this password before the server is reachable by anyone else. The banner disappears permanently as soon as the built-in Administrator's password is changed. Note also that the built-in Administrator is never automatically locked by the failed-sign-in threshold — it is the break-glass account — so a weak password on it is not protected by lockout, only by the per-IP throttle.

12.2 Dashboard layout

12.2.1 The top bar

The top bar is present on every tab and holds four items:

Theme control
A segmented control with ◐ **System**, ☀ **Light** and ☾ **Dark**. The choice is stored per browser and applied before the first paint, so there is no flash of the wrong theme. It affects only your own browser, not other administrators.

Signed in as <username>
The identity the current session belongs to.

❔ **Help**
Opens an overlay with the built-in topics: Admin Interface, TLS / SSL, Users & Sign-in, Database Connections, Rebuild from a script (API), Directory (LDAP), Logging, Backup & Restore, Customize, and License & Demo Mode.

👤 **Profile**
Your own account's security settings — see section 12.3.

Log out
Ends the session and returns to the sign-in page. It bumps your session epoch server-side, which also ends your application and integration sessions.

12.2.2 The tab strip

The tabs always appear in this order, with **App Control** selected when the dashboard opens:

#	Tab	What it configures	Covered in
1	**App Control**	Restart, licence, admin idle sign-out, display timezone	This chapter
2	**TLS / SSL**	Connection mode and certificates	This chapter
3	**Users**	Accounts, roles, sign-in policy, passkeys, two-factor	Chapter 13
4	**Database Connections**	Database and LLM servers, user assignment, projects	Chapter 14
5	**Directory (LDAP)**	Directory sign-in, service account, search filters	Chapter 15
6	**Logging**	Severity, rotation, the log viewer and exports	Chapter 16
7	**Backup**	Manual and scheduled backups, restore	Chapter 17
8	**Customize**	Sign-in page background and branding	Chapter 18
9	**Reporting**	AI report language model, style and per-connection/project analysis prompts	This chapter (§12.8)
10	**Integration**	Enabling the integration console	Chapter 19

Several **opt-in feature tabs** sit alongside these, each a simple on/off toggle (off by default) that takes effect immediately: **Event Receiver** (the AI-agent ingestion sinks), **MCP Server**, **Actions**, and **Notifications** — the last enables the Dashboard's notification rules and their background check engine (see §6 "Notifications"). Each is described with the feature it governs.

> **Tip** The Help overlay's own "Admin Interface" topic lists the tabs but omits **Integration**. The tab is nevertheless present and functional; use the table above as the authoritative list.

Every action you take in this panel is written to the shared structured log, together with your username and client IP. Destructive actions — deletions, restores, clearing the log, generating or revoking tokens — are deliberately recorded at **WARN** severity so that they stand out when the log is filtered.

12.3 The Profile overlay

The 👤 **Profile** button in the top bar opens an overlay covering your *own* administrator account. Its introduction reads: "*Your own admin account — password, passkeys and two-factor. These protect both this admin interface and the main app.*" It has three sections.

12.3.1 Changing your password

1. Open 👤 **Profile** in the top bar.
2. Type the new password in **New password** and repeat it in **Confirm**.
3. Press **Change password**.

An empty field is rejected with "*Enter a new password.*" and a mismatch with "*The passwords don't match.*" On success every *other* session of your account — application, integration console and any other browser on the admin panel — is invalidated immediately; your current session is re-issued in the same response, so

you are not signed out of the page you are working in. The log records "*changed their password — other sessions invalidated*".

12.3.2 Passkeys and two-factor

The remaining two sections let you register passkeys (Touch ID, Windows Hello, a security key) and set up TOTP two-factor authentication for your own account. Both are permissions that must first be granted to your account in the **Users** tab; until then the overlay states "*Passkeys are not enabled for your account — enable them in the Users tab first.*" or "*Two-factor is not enabled for your account — enable it in the Users tab first.*" The same credential works on all three surfaces of the host. These features are described in full in Chapter 13; the mechanics are identical whether you manage them here or in the application.

> **Note** Passkeys require a secure context (HTTPS, or localhost) and a real host name. If you reach the panel by IP address, the passkey controls hide themselves and the overlay explains: "*Passkeys need a hostname, not an IP address. Reach this panel by its name over HTTPS — e.g. its '.local' name — instead of its IP to add or use one.*"

12.4 The App Control tab

App Control is the operational tab. It contains four cards, in this order:

Card	Purpose	Takes effect
App Control	Restart the running servers (section 12.5)	Immediately, on demand
License	Install or remove the licence (section 12.4.1)	Within a few seconds; no restart
Admin session	Idle auto sign-out for *this* panel, 0–1440 minutes	Immediately (see Chapter 13)
Timezone	Display timezone for all admin timestamps and the backup schedule	Immediately (see Chapter 16)

The **Admin session** and **Timezone** cards are covered where their effects are visible — idle sign-out alongside the application's own idle policy in Chapter 13, and the display timezone alongside the log viewer and backup scheduler in Chapters 16 and 17. The rest of this section deals with licensing; the restart button follows in section 12.5.

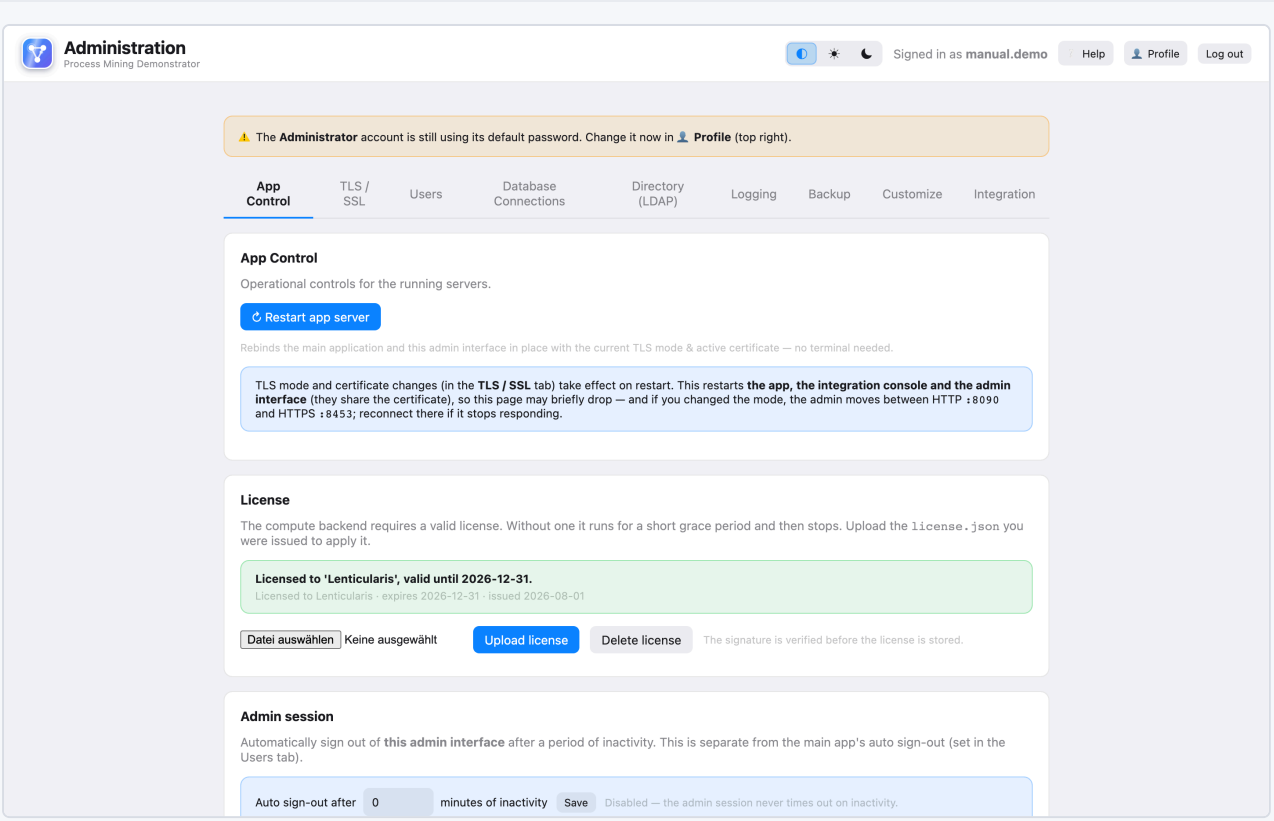


Figure 12.2 — App Control: the restart button, the licence status banner, the admin idle sign-out and the display timezone.

12.4.1 Licensing

The compute backend — the component that runs the mining queries — requires a valid licence. The card explains it plainly: "*The compute backend requires a valid license. Without one it runs for a short grace period and then stops. Upload the* `license.json` *you were issued to apply it.*"

A licence is a small JSON document signed with an Ed25519 key by the vendor. The signature is verified before anything is stored, so a hand-edited expiry date or a file from another installation is refused outright. The installed file lives at `data/license.json`.

Licence states

The status banner loads with "*Checking license…*" and then settles into one of three colours:

Colour	State	Banner text
Green	Valid	"*Licensed to '<name>', valid until <date>.*" with the sub-line "*Licensed to <name> · expires <date> · issued <date>*"
Orange	Expired	"*License for '<name>' expired on <date>.*"
Red	Missing or invalid	"*No license installed.*", "*License signature is not valid.*", "*License file is not valid JSON.*", "*License file is missing 'license' or 'signature'.*", "*License signature is malformed.*", "*License has no valid expiry date.*", or "*License file could not be read: …*"

Installing a licence

1. Open the **App Control** tab and find the **License** card.

2. In **Upload license**, choose the `license.json` file you were issued. The file input accepts `.json` only.
3. Press **Upload license**.
4. Wait for the confirmation "*License installed. The backend applies it within a few seconds.*" The status banner turns green and shows the licensee and expiry date.

No restart is needed: the backend polls for the licence file and picks up a new one within seconds. If the compute backend has already stopped because the grace period expired, uploading a valid licence here is still possible — see section 12.4.2.

Upload error	Meaning
HTTP 400 with the specific reason from the table above	The file is not a valid, correctly signed licence. Nothing is stored.
HTTP 413 "*License file is too large.*"	The encoded file exceeds 100,000 base64 characters. A genuine licence is far smaller.
"*Invalid file content.*"	The upload could not be decoded at all.

> **Note** A rejected upload never replaces the licence you already have. The signature is checked first, and only a verified file is written to disk.

Removing a licence

1. In the **License** card, press **Delete license**.
2. Confirm the dialog: "*Delete the installed license? — The compute backend drops to Demo Mode and stops after the grace period unless a new license is applied.*"
3. The result is either "*License removed — the backend is now in Demo Mode.*" or, if there was nothing to remove, "*No license was installed.*"

> **Warning** Deleting the licence does *not* hand you a fresh grace period. The demo window is granted once per installation and is not restored by removing and re-adding a licence. If the window has already been spent, the compute backend stops as soon as the licence is gone.

12.4.2 Demo Mode and the grace countdown

An installation with no valid licence runs in **Demo Mode**. The first unlicensed run stamps a deadline into the marker file `data/demo_grace.json`; the window is **30 minutes** by default and is configured at deployment time with `PMW_LICENSE_GRACE_SECS`. The stamp is written once per installation and is never rewritten.

While the window is open, users of the main application see an orange banner on the sign-in panel — "***Demo Mode** — remaining time: N min*", or "***No License installed***" when the window has been spent. The banner refreshes every 30 seconds, so the countdown visibly ticks down.

Uploading a valid licence at any point during the window cancels the shutdown; the banner disappears at the users' next refresh.

What happens when the grace period expires

When the deadline passes, the **compute backend stops**. The main application can no longer run queries; API calls that reach for the backend fail, and users see "*The compute backend is not reachable. Start it with ./run.sh or check PMW_BACKEND_URL.*" (HTTP 502).

> **Note** The administration interface is deliberately unaffected. It keeps running after the backend has stopped so that a licence can always be uploaded — you never have to reach a terminal to recover an installation whose demo window has expired. Upload the licence in the **License** card as usual, then restart the app server (section 12.5) if the backend does not come back on its own.

Situation	Compute backend	Admin interface	Action
Valid licence installed	Runs	Runs	None
No licence, grace window open	Runs; countdown shown to users	Runs	Upload the licence before the window closes
No licence, grace window spent	Stopped	Runs	Upload a valid licence here, then restart
Licence expired	Treated as unlicensed	Runs	Obtain and upload a renewed licence

12.5 Restarting the app server

The **App Control** card at the top of the tab carries the caption "*Operational controls for the running servers.*" and a single primary button, **↻ Restart app server** (tooltip: "*Rebind the app + admin listeners with the current TLS settings*"). Its side note explains the purpose: "*Rebinds the main application and this admin interface in place with the current TLS mode & active certificate — no terminal needed.*"

A permanent information banner sits beside it:

```
TLS mode and certificate changes (in the TLS / SSL tab) take effect on
restart. This restarts the app, the integration console and the admin
interface (they share the certificate), so this page may briefly drop —
and if you changed the mode, the admin moves between HTTP :8090 and
HTTPS :8453; reconnect there if it stops responding.
```

1. Make the changes you want to apply — typically a TLS mode or certificate change, or enabling the integration console.
2. Return to the **App Control** tab and press **↻ Restart app server**.
3. Confirm the dialog: "*Restart now? — The app AND this admin interface rebind their HTTP/HTTPS listeners with the current TLS settings. Anyone using the app will briefly disconnect, and if you changed the TLS mode this admin page may move between :8090 (HTTP) and :8453 (HTTPS).*"
4. The page reports "*Restart signalled. Listeners rebind with the current TLS plan; reconnect on the new address if this page stops responding.*" and a toast shows "*Restarting (app pid N, admin pid M)…*"

5. If the panel stops responding, reload it on the address the new TLS mode implies — `http://<host>:8090/` for Off, `https://<host>:8453/` when HTTPS is in play.

Internally the restart signals the launchers in sequence: first the application, then the integration console (best effort — it may not be running), then the admin panel's own launcher. Because the admin server tears itself down last, the HTTP response can occasionally race the teardown; a missing response after a confirmed restart is not an error.

> **Warning** One button restarts three servers. Everyone using the main application is briefly disconnected, and any long-running query in flight is lost. Restart during a maintenance window where possible.

Status	Message	What to do
409	"*The app server is not managed by the launcher (no PID file). Start it with `./run.sh`, then try again.*"	The application was started outside the launcher; restart it the way it was started.
409	"*The app server process is not running.*"	Start the application with `./run.sh`.
500	"*Invalid PID file.*"	The launcher state is corrupt; restart the installation from the shell.
500	"*Could not signal the app server: <os error>*"	The operating system refused the signal — usually a permissions problem.

12.6 The TLS / SSL tab

This tab decides how the browser surfaces accept connections and which certificate they present. It holds two cards: the mode selector at the top and the certificate store below it.

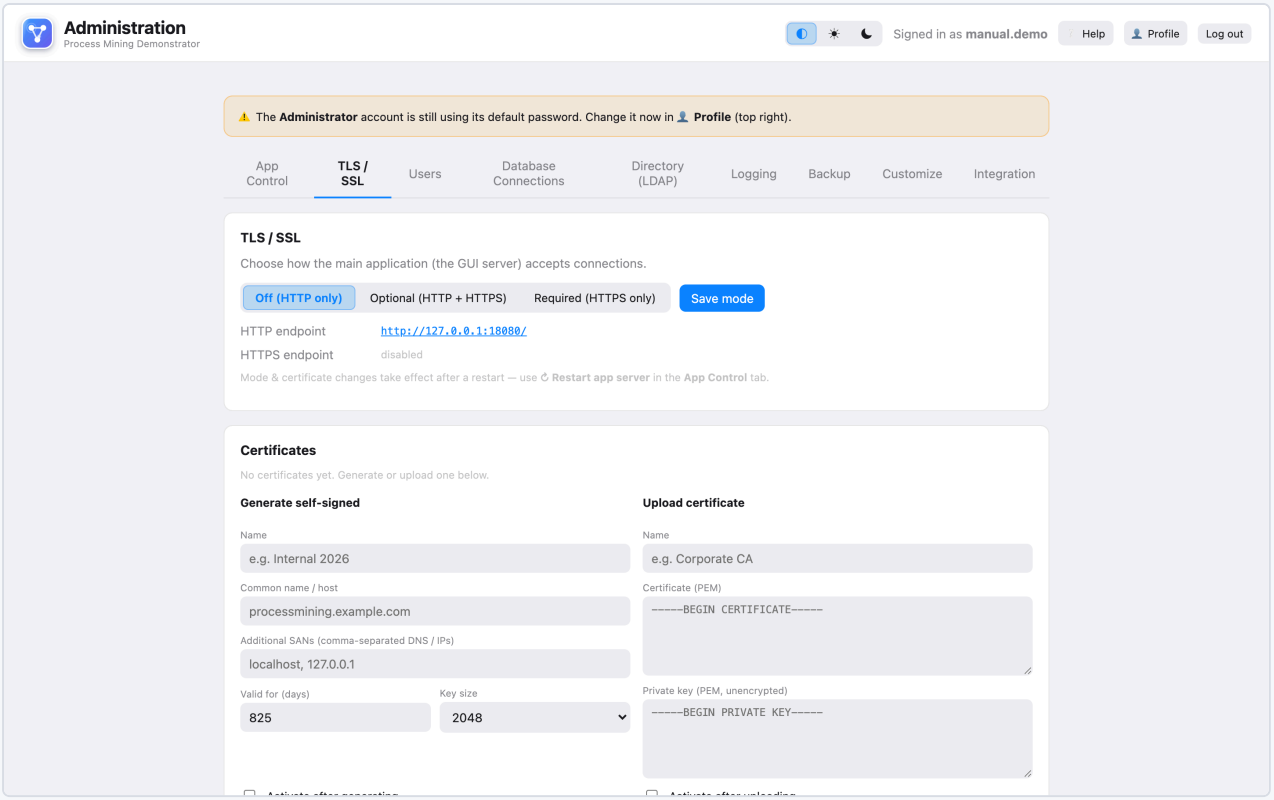


Figure 12.3 — TLS / SSL: the three connection modes, the resulting endpoint plan and the certificate store.

12.6.1 The three modes

The card is captioned "*Choose how the main application (the GUI server) accepts connections.*" and offers a segmented control with three positions. The currently stored mode is highlighted.

Mode	HTTP endpoint	HTTPS endpoint	Use it when
Off (HTTP only)	`http://<host>:8080/`	disabled	The server sits behind a reverse proxy that terminates TLS, or on an isolated evaluation network.
Optional (HTTP + HTTPS)	`http://<host>:8080/`	`https://<host>:8443/`	You are migrating to HTTPS and want both addresses live during the transition.
Required (HTTPS only)	disabled	`https://<host>:8443/`	Normal production. Also the prerequisite for passkeys away from localhost.

After you load or save the mode, the card prints the resulting **endpoint plan**: an *HTTP endpoint* row and an *HTTPS endpoint* row, each showing either a clickable URL or the word *disabled*.

1. Open the **TLS / SSL** tab.
2. Select **Off (HTTP only)**, **Optional (HTTP + HTTPS)** or **Required (HTTPS only)**.
3. Press **Save mode**. The toast confirms "*TLS mode saved — restart in App Control to apply*".
4. Check the endpoint plan and, if the mode needs HTTPS, make sure a certificate is marked **active** (section 12.6.2).
5. Go to **App Control** and press **↻ Restart app server**.

6 Reconnect on the address shown in the endpoint plan.

If the selected mode needs a certificate and none is active, the card warns: "*No active certificate — HTTPS cannot start. Generate or upload one and activate it below.*"

> **Warning** If a mode that requires HTTPS is saved while no certificate is active, every server **falls back to HTTP** on the next restart, so that nothing becomes unreachable. This is a safety net, not a configuration: the installation is then serving in clear text despite being set to *Required*. Always confirm that a certificate is active before you restart, and confirm the address afterwards.

> **Note** The mode and the active certificate are shared. The main application, this administration interface and the integration console all follow the same setting and present the same certificate. There is no way to run the panel over HTTPS while leaving the application on HTTP. The card's footer says so: "*Mode & certificate changes take effect after a restart — use* ↻ ***Restart app server*** *in the* ***App Control*** *tab.*"

12.6.2 The certificate store

The **Certificates** card lists every certificate known to the installation. When it is empty it reads "*No certificates yet. Generate or upload one below.*"

Column	Contents
Name	The name you gave the certificate. The one currently in use carries a green **active** pill.
Subject	The subject read from the certificate itself.
Type	*Self-signed* or *CA-signed*.
Expires	The not-after date.
Actions	**Activate** (hidden on the row that is already active), **Download**, **Delete**.

Download saves the certificate in PEM form as `<Name>.crt`, with spaces in the name replaced by underscores — useful for distributing a self-signed certificate to client trust stores. **Delete** asks `Delete certificate "<name>"?` and is recorded in the log at WARN severity.

> **Tip** Keep the active certificate valid. An expired certificate does not degrade gracefully — HTTPS clients simply refuse to connect. The **Expires** column is the one to check during routine maintenance.

12.6.3 Generating a self-signed certificate

The left half of the card creates a certificate on the server. This is the fastest route to working HTTPS on an internal network, at the cost of a trust warning in browsers that have not been given the certificate.

Field	Default	Notes
Name	—	How the certificate is listed. Placeholder: *e.g. Internal 2026*.
Common name / host	—	The host name clients will use. Placeholder: *processmining.example.com*.
Additional SANs	—	Comma-separated extra DNS names and IP addresses. Placeholder: *localhost, 127.0.0.1*.
Valid for (days)	**825**	Minimum 1, maximum 3650.
Key size	**2048**	2048, 3072 or 4096 bits.
Activate after generating	off	Tick to make the new certificate active at once.

1. Fill in **Name** and **Common name / host**.
2. Add any further names or addresses under **Additional SANs**, separated by commas. Include every name and IP by which the server is actually reached, otherwise browsers report a name mismatch.
3. Adjust **Valid for (days)** and **Key size** if your policy demands it.
4. Tick **Activate after generating** if this certificate should be used immediately.
5. Press **Generate**. The toast confirms "*Certificate generated*" and the certificate appears in the table.
6. Restart from **App Control** to put it into service.

> **Tip** For passkeys to work, clients must reach the server by a host name over HTTPS — never by IP. Put that host name in **Common name / host** and add its aliases (for example the `.local` name) to **Additional SANs**.

12.6.4 Uploading your own certificate

The right half of the card takes a certificate issued elsewhere — by a corporate CA or a public one.

1. Enter a **Name** for the entry (placeholder: *e.g. Corporate CA*).
2. Paste the full certificate chain into **Certificate (PEM)**.
3. Paste the matching key into **Private key (PEM, unencrypted)**.
4. Tick **Activate after uploading** if it should become the active certificate.
5. Press **Upload**. The toast confirms "*Certificate uploaded*".
6. Restart from **App Control** to put it into service.

> **Warning** The private key must be **unencrypted** PEM — the server has no way to prompt for a passphrase at start-up. Decrypt the key before pasting it, and treat the clipboard and any intermediate file accordingly.

Each of the two PEM fields accepts at most 200,000 characters; anything larger is rejected. Validation errors raised while parsing the certificate or key are surfaced verbatim as the toast text.

12.6.5 Activating a certificate

1. In the certificate table, press **Activate** on the row you want to use. The green **active** pill moves to that row.
2. Go to the **App Control** tab and press ↻ **Restart app server**.
3. Reconnect and confirm in the browser that the expected certificate is being presented.

> **Note** Activation only marks the certificate as the one to use. Nothing changes on the wire until the restart — the running listeners keep the certificate they were bound with. The same is true of the TLS mode.

Generation, upload and activation are logged at INFO severity under the operation `tls`; deletion is logged at WARN. You can therefore reconstruct the certificate history of an installation from the Logging tab by filtering on that operation.

12.7 Putting it together: enabling HTTPS from scratch

The following sequence takes a fresh installation from plain HTTP to HTTPS on all three surfaces.

1. Sign in to the admin panel at `http://<host>:8090/`.
2. Open 👤 **Profile** and change the built-in Administrator's password. The default-password banner disappears.
3. Open **App Control** and install the licence you were issued; check that the status banner turns green.
4. Open **TLS / SSL** and generate a self-signed certificate — or upload your CA-issued one — with **Activate after generating** / **Activate after uploading** ticked.
5. Verify that the certificate row shows the green **active** pill.
6. Set the mode to **Required (HTTPS only)** and press **Save mode**. Confirm that the endpoint plan shows the HTTPS URL and that the HTTP row reads *disabled*, and that no "No active certificate" warning is displayed.
7. Open **App Control** and press ↻ **Restart app server**, confirming the dialog.
8. Reconnect on `https://<host>:8453/`. Tell users the application has moved to `https://<host>:8443/`, and — if the integration console is enabled — its users to `https://<host>:8463/`.
9. If a self-signed certificate was used, download it from the certificate table and distribute it to the client trust stores.

> **Warning** If the admin panel does not answer on `https://<host>:8453/` after the restart, try `http://<host>:8090/`. Answering there means the HTTPS fall-back was taken because no certificate was active; return to the TLS / SSL tab, activate one and restart again.

12.8 The Reporting tab

The **Reporting** tab governs the high-gloss **AI supported Documentation** report (see §11). The report is produced in two stages — a language model analyses the transition table and returns structured findings, then the server assembles those findings plus the process Sankey, the Happy Path / Conformance sections and the project notes into a styled, printable document — a cover with executive summary, a clickable table of contents, and one topic per page. This tab configures all three parts, and is entirely separate from the LLM attached to a connection (which the app uses for the interactive analysis).

12.8.1 Report language model

An OpenAI-compatible endpoint dedicated to the report: a `/v1`-style **Server URL**, a **Model** and an optional **API key**. **Test** probes it and lists the available models. Leave the API-key field blank to keep the stored key; leave the whole **Server URL** blank to fall back to the LLM configured on the connection. Press **Save language model** to store it. The model is shared by every project; the key is encrypted at rest and never returned to the browser.

> **Tip** For confidentiality, prefer a **locally hosted** model server (Ollama, llama.cpp, vLLM, LM Studio). Only the aggregated transition table is ever sent to it; the Sankey, Happy Path and Conformance sections are assembled locally and never leave your network.

12.8.2 Style & sections (per project)

The report look is configured **per (connection, project)**, so one environment can brand reports differently for each customer — a service provider running many customers from the same installation gives each project its own theme. Pick the **connection**, then one of its **projects** (loaded live from its database, shown by title, stored by id); the fields below load that project's saved style, or defaults when it has none.

Each project's style holds an **accent colour** (a `#rrggbb` hex, applied to headings, the executive-summary callout and table headers), an optional **organisation / letterhead** line, an optional **logo** with a **Left**/**Right** position and a **size** slider (0.5×–2×, aspect ratio kept), and three switches to include or omit the **process-flow Sankey**, the **Happy Path** section and the **Conformance gap analysis**. Press **Save style** to store it for the selected project, or **Remove** to revert that project to the plain default theme. Configured projects are listed below with Edit / Remove actions.

The **logo** accepts a PNG, JPEG, WebP or SVG. Raster images are scaled down and re-encoded in the browser before upload, so even a large original ends up small; the stored logo is placed on the **first page** of that project's reports, on the chosen side. When both a logo and an organisation name are set, the logo sits on its chosen side and the organisation name appears as a letterhead on the opposite side; with no logo, the organisation name leads the cover.

> **Note** A project with no style of its own uses a plain built-in theme (accent, no letterhead or logo, all sections included). There is no shared company-wide default — configure each customer's projects explicitly.

12.8.3 Analysis prompts

The instruction the report model follows can be tailored per process by **linking a prompt to a connection and a project**. Choose the **connection** from its dropdown; the **project** dropdown then lists that connection's projects (shown by title, stored by id), loaded live from its database. Pick the project, type the **prompt**,

and **Save prompt**. The saved mappings are listed below with Edit / Remove actions. When a project has no prompt of its own, the app's per-project prompt template is used instead.

13 Administration: Users and Permissions

This chapter describes the **Users** tab of the administration interface: how you create accounts, what each of the three roles (Administrator, Power user, Developer) actually grants, how you enable, disable, unlock and delete accounts, how you reset passwords, and how you allow passkeys and two-factor authentication for individual users or for everybody at once. It also sets out the three global access controls that live on the same tab — whether the main application requires a sign-in at all, when idle sessions are signed out, and after how many failed sign-ins an account is disabled. It is written for administrators. Every action described here requires an administrator account, because the administration interface admits nobody else.

Requires: **Administrator**

13.1 Opening the Users tab

The administration interface runs on its own port, separate from the main application: `http://<host>:8090` or, when TLS is switched on, `https://<host>:8453`. Only administrators may sign in. Local administrator accounts always work — this is deliberate, so that a directory outage can never lock you out of your own server. Directory (LDAP) accounts reach this interface only when **Also allow directory sign-in to this admin interface** is ticked in the **Directory (LDAP)** tab *and* the account has been promoted to administrator here.

1. Open the administration interface and sign in.
2. Select the **Users** tab in the tab strip (the order is App Control · TLS / SSL · **Users** · Database Connections · Directory (LDAP) · Logging · Backup · Customize · Integration).
3. The card header reads “Only enabled users will be allowed to sign in to the main application.”

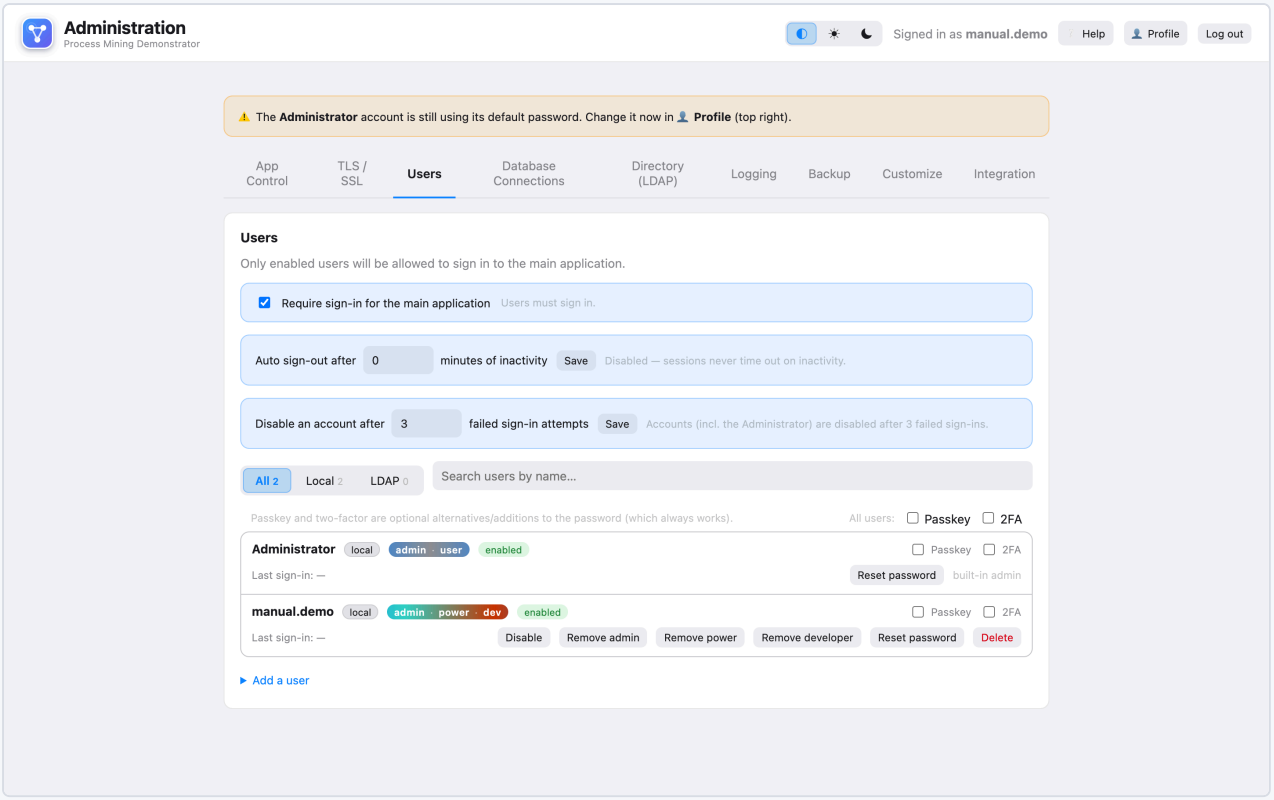


Figure 13.1 — The Users tab: the three global access controls, the filter and search row with the all-users Passkey and 2FA master checkboxes, and the per-user cards below.

> **Note** On a fresh installation the built-in account **Administrator** exists with the password **Administrator** (the username can be changed with the environment variable `PMW_DEFAULT_ADMIN_USER`). Until that password is changed, every page of the administration dashboard carries the banner "⚠ The **Administrator** account is still using its default password. Change it now in 👤 **Profile** (top right)." Changing the password clears the banner permanently.

13.2 The three global access controls

Three banners sit above the user list. They apply to the installation as a whole, not to a single account. Each takes effect as soon as it is applied — no restart is needed.

13.2.1 Require sign-in for the main application

A checkbox that applies instantly (toast "Access updated"). The hint under it reads "Users must sign in." when on, and "The app is open — no sign-in required." when off. The default is **on**.

With the box ticked, every visitor to the main application meets the sign-in panel and every `/api/*` call requires a session; without one the server answers 401 "Authentication required." Only the health probe is open.

With the box cleared the application runs in **open mode**: it loads with no sign-in panel and, importantly, with **no user identity at all**. Per-user settings and filter presets then share a single profile, notes have no author, the sidebar footer (name, Two-factor, Passkeys, Sign out) disappears, the idle auto sign-out stops running,

and every power-user and administrator feature is hidden in the browser and refused by the server, because there is no identity to check a role against.

> **Warning** Open mode is a single-user convenience, not a permission level. Turning sign-in off does not make everyone a power user — it makes everyone anonymous, and anonymous callers get 403 from the power-user endpoints. The integration console and the administration interface ignore this toggle entirely; both always require a sign-in.

13.2.2 Auto sign-out after N minutes of inactivity

A number field (0–1440) with a **Save** button. This is the **main application's** idle timeout. The hint reads “Idle sessions are signed out after N min.” when N is greater than zero, and “Disabled — sessions never time out on inactivity.” at 0.

In the main application an invisible watcher observes mouse movement, mouse buttons, key presses, touches, scrolling and the wheel. After N idle minutes it signs the user out, and the sign-in panel shows the orange notice “You were signed out due to inactivity.” While the user is active the browser pings the session endpoint (at most every 15–60 seconds) so that the server-side sliding window stays open even when no other request is made.

> **Note** The same value also governs the **integration console's** server-side session window, but that surface has no client-side watcher. An idle developer is not bounced to the login screen; their session simply expires, and they discover it when their next action fails. The **administration interface** has its own, separate setting in **App Control → Admin session**; the value here does not affect it.

13.2.3 Disable an account after N failed sign-in attempts

A number field (0–100) with **Save** (toast “Failed-sign-in lockout updated”). The hint reads “Accounts (incl. the Administrator) are disabled after N failed sign-ins.” or, at 0, “Disabled — accounts are never locked on failed sign-ins.” Fresh installations default to **3**.

The counter records **consecutive wrong-password attempts** against a local account, pooled across all three surfaces — application, integration console and administration interface share one store. A successful sign-in resets it. The following never count towards it:

- wrong two-factor codes (a mistyped or expired code can never disable an account),
- failed passkey ceremonies,
- directory (LDAP) accounts, which have no local password,
- attempts against an account that is already disabled.

A locked account shows a red **locked** pill (tooltip “Disabled after too many failed sign-ins — Unlock to restore.”) and its action button reads **Unlock**. A lockout also lifts itself after **15 minutes**; the expiry is applied lazily at the next sign-in attempt, so simply trying again after the window works. That 15-minute period is fixed — only the threshold is configurable in the interface.

Warning Despite the hint's wording, the **built-in Administrator is counted but never automatically locked**. It is the break-glass account; its protection is the per-IP throttle and slow password hashing, not a lockout. Other administrator accounts do lock normally. If your only non-built-in administrator locks itself out and you cannot wait out the window, restart the server with `PMW_RESET_LOCKOUTS=1` to clear every lockout in the installation.

Control	Range	Default	Scope
Require sign-in for the main application	on / off	on	Main application only
Auto sign-out after N minutes of inactivity	0–1440 (0 = off)	—	Main application (client watcher) and integration console (server window)
Disable an account after N failed sign-in attempts	0–100 (0 = off)	3	All local accounts, all three surfaces

13.3 Finding and reading a user card

Above the list sit the filter chips **All** / **Local** / **LDAP**, each showing a live count, and a search box “Search users by name…” that matches the username or the display name. When nothing matches, the list shows “No matching users.”

Each user occupies a two-line card:

Line 1

The username (with the smaller display name beside it for directory users), a source pill reading **local** or **LDAP**, a **removed from directory** pill when a directory sync no longer finds the account, the role badge, the access pill (**enabled** in green, **disabled** or **locked** in red), and the two per-user checkboxes **Passkey** and **2FA**.

Line 2

“Last sign-in: <date>” — or **“Never signed in”** when the account has not yet signed in (common for directory accounts brought in by the user-list sync before their first login) — followed by the action buttons described in the rest of this chapter.

The role badge condenses the additive role flags into one label. The base segment is *admin* if the user is an administrator, otherwise *power*, otherwise *user*; administrators always show a second segment (*power* or *user*) so that you can see whether they also hold power privileges; the *dev* segment is appended as an add-on. A user with more than one role gets a single animated gradient pill such as `admin·power` or `admin·user·dev`; a single role is a plain pill. The tooltip spells the roles out, for example “Roles: administrator, power user (manages own connections), developer (integration console)” or simply “Regular user”.

13.4 Creating a user

Requires: **Administrator**

1. On the **Users** tab, expand the collapsed **Add a user** section at the foot of the list.
2. Type the **Username**. It must be unique among local accounts.

3. Type an initial **Password**. There is no confirmation field here, so type carefully; you can always reset it afterwards.
4. Tick **Administrator** only if this account is to administer the server. Leave it clear for everybody else.
5. Select **Create user**. The new account appears in the list as enabled, with no roles beyond the one you ticked.
6. If the account also needs the power or developer privileges, grant them from the buttons on the new card (see 13.5). There is no checkbox for them at creation time.

Message	Cause
Username is required.	The username field was left empty.
Password is required.	The password field was left empty. Accounts are never created without a password.
A user named '<x>' already exists.	A local account of that name is already present. Search the list for it rather than creating a variant.

Note You do not create directory users here. An LDAP account appears in the list automatically after its first successful sign-in, provisioned as a plain, enabled user. From that moment it behaves like any other row: you can grant it roles, allow passkeys and two-factor, assign database connections, disable it or delete it. Administrator rights are *never* granted by the directory — a local administrator must promote the account here.

Keeping the list in step with the directory

Waiting for each person to sign in before their account appears can be confusing. Turn on **Automatically sync the user list** in the **Directory (LDAP)** tab (default **every 5 minutes**; the interval is configurable) and a background job mirrors the directory into this list: **new** accounts appear before anyone signs in, a **changed** email or display name is refreshed, and an account **removed** from the directory is **disabled and flagged “removed from directory”** — it is never deleted, so its role and database-connection assignments survive, and if the account returns to the directory it is automatically re-enabled. A directory name that collides with an existing *local* account is skipped, and local accounts are never changed by a sync.

When a directory is enabled, the Users tab shows a **Directory sync** bar with a **↻ Refresh from directory** button to run the sync on demand, alongside the time and outcome of the last run (how many were added, updated and removed, or the error if the directory could not be reached). This is the same sync the background job runs; either way, no account can sign in unless it is enabled and, for the admin interface, promoted to administrator.

13.5 Roles and what each one grants

Roles are **independent, additive flags**, not a ladder. A user can hold any combination of Administrator, Power and Developer; the implicit base is the plain user. In the main application's sidebar footer the identity is labelled with the *highest* role held (Admin > Power user > Developer > User), which is a display convenience only — the underlying flags are checked individually.

13.5.1 Administrator

Grants access to this administration interface and everything in it: users and permissions, TLS certificates, the directory configuration, logging, backups and restores, the licence, customisation, and switching the integration console on or off. An administrator also manages **all** database connections, not merely their own, and has the power-user analysis features in the main application.

Grant or revoke it with **Make admin** / **Remove admin** on the user's card.

13.5.2 Power user

A main-application privilege. Power users create and manage **their own** database connections from the application sidebar, and they get the advanced-analysis views — **Conformance Check**, **Happy Path** and **Simulation** — which are hidden from the view menu for plain users. They may also use journey **sampling** (creating and deleting representative subsets and A/B sources); the server refuses it for anyone else with "Journey sampling is available to power users and administrators only."

Grant or revoke it with **Make power** / **Remove power**.

13.5.3 Developer

Purely an additional access grant to the **integration console** (the separate surface on `:8100` / `:8463`). It does not include any power-user feature, although developers, like power users, may manage their own database connections. A user who holds only the Developer role sees the ordinary main application, plus the integration console.

Grant or revoke it with **Make developer** / **Remove developer**. Both actions are recorded in the shared log as "granted/revoked the Developer role for '<user>'".

13.5.4 Capability matrix

Use this table as the reference when deciding which flags an account needs.

Capability	Plain user	Power	Developer	Admin
Sign in to the main application (enabled account)	Yes	Yes	Yes	Yes
Sign in to the integration console	No (403)	No (403)	Yes	Yes
Sign in to the administration interface	No	No	No	Yes
Use database connections assigned to them	Yes	Yes	Yes	Yes
Create and manage their own database connections	No	Yes	Yes	Yes (manages all)
Conformance Check, Happy Path, Simulation	No (hidden)	Yes	No (unless also Power)	Yes
Journey sampling (subsets, A/B sources)	No	Yes	No (server refuses)	Yes
Notes (create and comment)	Yes	Yes	Yes	Yes
Enrol a passkey; set up two-factor	Only if you allow it per user — these flags are role-independent			
Manage users, TLS, LDAP, backups, licence, customisation, integration on/off	No	No	No	Yes

13.5.5 Role changes take effect immediately

Every request re-validates the session against the live user record, so a role change does not wait for the user to sign out. Revoking the Developer role from someone who is working in the integration console drops them, on their very next request, to a full-page **Access denied** screen reading “The integration console is available to developers and administrators only. Ask an administrator to grant you the Developer role.” with a ⏻ **Sign out** button. Anything they were driving from that console is no longer theirs to start, stop or monitor there. Revoking the administrator role ends their administration session the same way; revoking the power role removes the advanced views and refuses their sampling calls from that moment on.

> **Warning** Revoke a role only when you are prepared for work in flight to be interrupted. There is no grace period and no warning shown to the user before the change lands.

> **Note** The server refuses any change that would leave the installation without an administrator: “This would leave no enabled administrator. Promote or enable another admin first.” Promote or enable the replacement before demoting or disabling the incumbent.

13.6 Enabling, disabling and unlocking accounts

> Requires: **Administrator**

Only enabled users may sign in to the main application. The card's first action button reads **Disable** for an enabled account, **Enable** for one you disabled, and **Unlock** when the account was disabled by the failed-

sign-in lockout rather than by you.

1. Find the account with the search box or the **All** / **Local** / **LDAP** chips.
2. Select **Disable** to bar the account, or **Enable** / **Unlock** to restore it.
3. Check the access pill afterwards: **enabled** (green), **disabled** or **locked** (red).

Disabling is immediate and retroactive: the account's live sessions are re-validated on their next request and die, on all three surfaces at once. A user whose password is correct but whose account you disabled is told exactly that — “This account has been disabled. Contact an administrator.” — while someone typing a wrong password only ever sees the generic “Invalid username or password.” The specific reason is revealed only after the password has been proved, so that the messages cannot be used to discover which accounts exist.

> **Tip** Disabling and re-enabling an account clears its failed-sign-in counter, so it doubles as a manual reset when someone has been fumbling their password but is not yet locked out.

13.7 Resetting a password

> Requires: **Administrator**

1. Find the user's card. The **Reset password** button is shown for **local users only** — it is hidden on LDAP rows, because those accounts have no local password.
2. Select **Reset password**. The browser prompts “New password for <user>:”.
3. Type the new password and confirm the prompt. An empty value is refused with “Password is required.”
4. Communicate the password to the user by a channel other than the one you are administering.

> **Warning** Any password change bumps the user's session epoch, which invalidates **every** outstanding session token for that user on all three surfaces. Someone whose password you reset while they are working is silently signed out at their next request and must sign in again with the new password. This is the intended behaviour for a compromised account; it is a surprise if you meant it as a courtesy.

> **Note** Administrators change their *own* password in the 👤 **Profile** overlay (top right), not here. That path is the one exception to the rule above: the current session is re-issued in the same response, so the administrator making the change stays signed in while all of their other sessions are invalidated. The log records “changed their password — other sessions invalidated”.

13.8 Allowing passkeys

A passkey (Touch ID, Windows Hello, a security key) is an **alternative** to the password, never a replacement: the password always remains a fallback. Users cannot enrol one until you allow it. The master row above the list carries the caption “Passkey and two-factor are optional alternatives/additions to the password (which always works).”

13.8.1 Allowing a passkey for one user

1. Find the user's card.
2. Tick the **Passkey** checkbox on line 1 (tooltip “Allow this user to enrol and sign in with a passkey”). The change applies at once.
3. Tell the user to open 🔑 **Passkeys** in the application sidebar footer — or, for an administrator, 👤 **Profile** → **Passkeys** — name the device and select **Add a passkey**.

13.8.2 The all-users master toggle

The **All users:** row offers a **Passkey** checkbox (tooltip “Allow every user to enrol and sign in with a passkey”). It is shown as ticked only when *every* user currently holds the permission, and toggling it writes the flag for all existing users at once (toast “Passkeys allowed/disabled for all users”).

> **Note** The master toggle is a bulk edit, not a default. It changes the users who exist at the moment you click it; accounts created afterwards start without the permission and need their own tick.

13.8.3 Consequences and prerequisites

- **Revoking the permission blocks passkey sign-in immediately.** It is checked at every step of the ceremony, and the management controls disappear from the user's interface. The registered credentials are not what is checked at sign-in time — the permission is — so revocation is an effective, instant kill switch.
- A passkey registered on a host works on **all surfaces of that host**: application, integration console and administration interface share the relying-party identifier (the hostname without the port).
- Passkeys require a WebAuthn-capable browser, a **secure context** (HTTPS, or localhost) and a **real hostname**. WebAuthn rejects bare IP addresses and single-label hosts. Reached by IP, the sign-in button and the management controls hide themselves and the user sees “Passkeys need a hostname, not an IP address. Reach the server by its name over HTTPS — e.g. its ‘.local’ name — instead of its IP, then try again.”
- Sign-in is **username-first**: the user types their username, then selects 🔑 **Sign with Passkey**. There is no one-click, usernameless flow.
- If the application and the administration interface live on different sub-domains, set `PMW_PASSKEY_RP_ID` to the shared parent domain and list the expected origins in `PMW_PASSKEY_ORIGINS`.

> **Warning** A passkey sign-in **satisfies two-factor by itself** and skips the code step, even for users who are enrolled in TOTP. If your policy is “always a one-time code”, do not allow passkeys.

13.9 Allowing two-factor authentication

Two-factor authentication is a time-based one-time code (TOTP) layered *on top of* the password. Two independent facts govern it per user: whether you have **allowed** it, and whether the user has **enrolled** (confirmed a code from their authenticator app). One secret protects all three surfaces.

13.9.1 Allowing 2FA for one user or for everybody

1. Find the user's card and tick the **2FA** checkbox on line 1 (tooltip “Allow this user to set up two-factor (TOTP)”), or
2. tick the **2FA** checkbox in the **All users:** master row to write the flag for every existing user at once (toast “Two-factor allowed/disabled for all users”).
3. Tell the user what will happen at their next sign-in (see below). No further administrator action is needed.

> **Warning** Allowing 2FA is the same thing as **requiring** it. A user who is allowed but not yet enrolled cannot decline: their next sign-in — on the application, the integration console or the administration interface — halts after the password and forces enrolment. The caption changes to “Set up two-factor to continue”, a QR code appears with the text “Two-factor is required for your account. Scan this with an authenticator app (Google Authenticator, 1Password…), then enter the 6-digit code.”, and no session is issued until a correct code confirms the authenticator. Nothing is stored server-side until then, so there are no half-enrolled accounts.

Immediately after a successful enrolment the user is shown **10 recovery codes**, once, under the heading “Save your recovery codes” with the text “Each works once if you lose your authenticator. They won't be shown again.” Each code works exactly once, anywhere a TOTP code is accepted. They are stored only as salted hashes: **you cannot view or recover them for a user**, and regenerating a set invalidates the previous one entirely.

13.9.2 Resetting 2FA for a user who lost their authenticator

There is no administrator command that resets an enrolment while preserving it. The supported recovery path is to withdraw the permission, let the user back in with their password alone, then restore the permission so that they enrol afresh.

1. Ask the user first whether they still have an unused **recovery code** — entering one at the code step signs them in without any administrator involvement.
2. If not, find their card and **untick** the **2FA** checkbox. Password-only sign-in works again from that moment.
3. Have the user sign in and, from the sidebar, open 🔒 **Two-factor** and select **Turn off**, entering a current code if they can — this deletes the stale secret and its recovery codes. If they cannot produce a code, leave the enrolment in place and continue.
4. Re-tick the **2FA** checkbox on their card when they are ready to enrol the new device.

> **Warning** Un-ticking **2FA** does **not** delete the enrolment — it only relaxes the login gate. Re-ticking the box rearms the *same* authenticator instantly, with no new enrolment step. If the old authenticator is in the wrong hands, have the user turn two-factor off from their own sheet (which does delete the secret and the recovery codes) before you re-allow it, or reset their password as well.

> **Note** A user cannot silently strip their own two-factor: turning it off requires a current code or a recovery code, so a hijacked session cannot remove the second factor. Wrong codes are throttled per source IP but **never** count towards the account lockout — a mistyped code can never disable an account.

13.10 Deleting a user

> Requires: **Administrator**

1. Find the user's card.
2. Select the danger button **Delete**.
3. Confirm the dialogue `Delete user "<name>"?`.

Deleting an account also deletes its passkeys and its recovery codes, and any live session it holds fails on its next request. Deleting a directory user removes the local row only; the account still exists in the directory and will be re-created as a plain, enabled user the next time it signs in successfully — with every role, permission and connection assignment gone. Prefer **Disable** when you want the account kept but barred.

> **Warning** Deletion is not offered as reversible anywhere in the interface, and the deletion is recorded in the shared log at WARN severity so that it stands out during an audit.

13.11 The built-in Administrator

The bootstrap account is protected in the interface and again on the server, so the protection holds even for a hand-crafted request:

Attempted action	Server response
Disable it	The built-in Administrator account cannot be disabled.
Remove its administrator role	The built-in Administrator account must remain an administrator.
Delete it	The built-in Administrator account cannot be deleted.

Its card therefore shows only the **Passkey** and **2FA** checkboxes, **Reset password**, an **Unlock** button if it is ever flagged as locked, and the note **built-in admin** (tooltip “The built-in administrator cannot be disabled, demoted or deleted.”).

> **Tip** Treat the built-in Administrator as a break-glass credential: give it a long, unique password, store that password where your organisation stores emergency credentials, and do the day-to-day administration from a named personal administrator account so that the log attributes every change to a person.

13.12 Working to least privilege

The role flags are independent precisely so that you can grant the narrowest useful set. The following practices follow from the behaviour described above.

Start every account as a plain user

Creating a user without the **Administrator** tick and adding **Make power** or **Make developer** only when a concrete task requires it costs one extra click and avoids the far more common mistake of a whole team holding administrator rights.

Do not use Administrator as a synonym for “experienced”

The advanced-analysis views and journey sampling come from the **power** flag, not from the administrator flag. An analyst who needs Conformance Check, Happy Path and Simulation needs **Make power** — nothing more.

Grant Developer only for the integration console

It confers no power features. If a data engineer also needs sampling or the advanced views, grant both flags explicitly rather than reaching for the administrator role.

Keep at least two administrators

The server refuses to leave you with none (“This would leave no enabled administrator. Promote or enable another admin first.”), but a single administrator who forgets their password and is not the built-in account is an avoidable outage: other administrator accounts *do* lock after the configured number of failed attempts.

Prefer Disable to Delete

Disabling bars the account and kills its sessions at once while preserving its roles, connection assignments and history. Delete when the person has left for good.

Decide the second factor as a policy, not per person

Because a passkey sign-in skips the code step, allowing both passkeys and 2FA for the same population means the effective second factor is whichever the user chooses. If a code is mandatory in your organisation, allow 2FA and leave passkeys off.

Review the list periodically

Use the **LDAP** chip to see which directory accounts have signed in at least once and now hold local roles, and the “Last sign-in” line to spot dormant accounts. Directory users keep their locally granted roles until you remove them, even if the directory entry changes.

13.13 Quick reference: what each control does the moment you use it

Action	Immediate effect
Create user	Account exists and is enabled; it may sign in to the main application straight away.
Make admin / Remove admin	Access to the administration interface begins or ends on the next request; a removed administrator's admin session dies at once.
Make power / Remove power	Advanced-analysis views and journey sampling appear or disappear; the server refuses sampling for a demoted user from that moment.
Make developer / Remove developer	Integration-console access begins or ends on the next request; a revoked user is dropped to the “Access denied” screen. Logged as granting/revoking the Developer role.
Disable	Sign-in barred; all live sessions on all three surfaces die on their next request.
Enable / Unlock	Sign-in restored and the failed-sign-in counter cleared.
Reset password	Every session of that user on every surface is invalidated; the user must sign in again with the new password.
Passkey checkbox on	The user may enrol and sign in with a passkey (given HTTPS and a hostname).
Passkey checkbox off	Passkey sign-in is blocked immediately and the management controls disappear.
2FA checkbox on	Two-factor becomes mandatory: the next sign-in forces enrolment, or asks for a code if already enrolled.
2FA checkbox off	Password-only sign-in works again; the enrolled secret is retained and re-arms if you tick the box again.
All-users Passkey / 2FA checkbox	Writes the flag for every user that exists now; later accounts are unaffected.
Delete	The account, its passkeys and its recovery codes are removed; the action is logged at WARN.

Note Every action on this tab is written to the shared structured log, visible in the **Logging** tab, with the client IP and the acting administrator's username. Destructive actions are logged at WARN severity by design so that they stand out among the routine entries.

14 Administration: Database Connections and Directory

> This chapter covers the two administration tabs that decide *where* the application reads its data from and *who* is allowed to sign in from outside: **Database Connections** and **Directory (LDAP)**. It explains every field of the connection editor, the TLS options for the database link, how to attach an OpenAI-compatible LLM server and what the built-in SSRF guard will and will not allow, how to list and delete projects, how the optional pre-materialized transitions table works and how to keep it fresh from a scheduler, and how to configure directory sign-in so that directory accounts merge cleanly with local ones. It is written for the person who administers the installation; ordinary users never see these screens.

> Requires: **Administrator** — both tabs live in the admin interface (HTTP :8090, HTTPS :8453) and are reachable only by accounts with the administrator role.

14.1 The Database Connections tab

A *connection* is a named definition of one Exasol database — host, port, credentials, schema, TLS settings — together with an optional LLM server and a list of the users who may use it. The tab header states the model in one sentence: “Define a database (and optional LLM) server, then assign it to the users who may use it. Each user sees only the connections assigned to them in the main application.”

Two facts follow from that model and are worth internalising before you edit anything:

- Connections are **defined centrally and assigned individually**. A user who is not assigned a connection cannot select it, and a direct attempt is refused with “This connection is not available to you.”
- Every signed-in user gets their **own** database session against the connection. Users never share a live session, so one user's query can never see another user's transaction. Anonymous mode (sign-in disabled) shares a single session bucket.

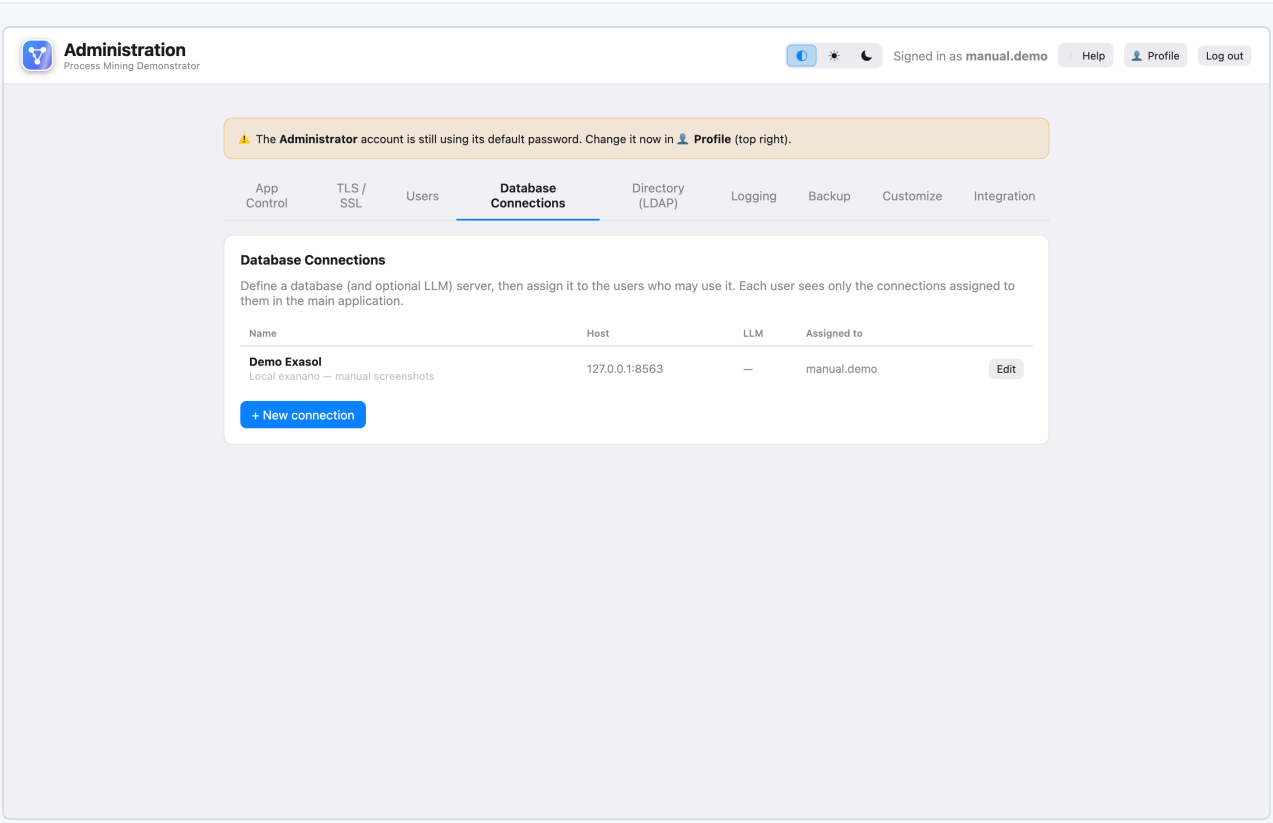


Figure 14.1 — The Database Connections tab: the connection list with host, LLM status and assignments.

14.1.1 The connection list

Column	Shows
Name	The connection name in bold, with its optional comment underneath.
Host	`host:port`.
LLM	A green **yes** pill when an LLM server URL or API key is stored, otherwise "—".
Assigned to	The assigned usernames, comma-separated, or **nobody**.
Edit	Opens the connection editor.

When nothing is defined yet the table shows a single row: "No connections defined yet."

1. Open the admin interface and select the **Database Connections** tab.
2. Click **+ New connection**. The editor opens with Port **8563**, Certificate mode **Verify** and Minimum RSA key size **2048** pre-filled.
3. Fill in the Database sub-tab (see 14.2), tick the users who may use the connection, and click **Save**.

14.1.2 The connection editor

The editor has three inner tabs — **Database**, **LLM** and **Projects** — and a common footer with **Save** (primary), **Test connection**, **Cancel**, and, for an already-saved connection, **Delete** (danger).

The **Projects** tab button is disabled while the connection is unsaved; its tooltip reads “Save the connection first to list its projects.” After a successful save the editor reloads with fresh data and the Projects tab unlocks. Saving without a name is refused with the toast “Name is required.” Deleting asks `Delete connection "<name>"?` and is recorded in the log at WARN severity, together with the connection's rebuild token and materialization status, which are deleted with it.

> **Warning** Saving *any* change to a connection, changing its assignments, or deleting it immediately drops every live session currently using it. Affected users must reconnect. This is deliberate: a revoked assignment has to take effect at once rather than at the end of someone's working day. If a user's connect happens to be in flight while you save, they receive “This connection changed during sign-in. Please reconnect.”

14.2 The Database sub-tab

14.2.1 Fields

Field	Default / placeholder	Notes
Name	“e.g. Production Exasol”	Required. Shown to users in the main application.
Comment	“optional”	Free text, displayed under the name in the list.
Host	“db.example.com”	Exasol host name or address.
Port	8563	The Exasol default; used everywhere a port field appears.
Username	—	The database account the application uses.
Schema	“optional”	Optional for connecting, but *required* for provisioning, for listing or deleting projects and for rebuilding transitions.
Password	bullets	Write-only. When a password is stored the label reads “(set — leave blank to keep)”. Leave blank to keep it; type to replace it.
Use TLS	off	Reveals the three certificate fields below. Without it, the database link is not encrypted at all.
Certificate mode	Verify (system trust store)	See 14.2.2.
Fingerprint (SHA-256)	“optional”	Only meaningful in *Pin fingerprint* mode.
Minimum RSA key size	2048	Stored and carried through every operation for compatibility with the macOS application's connection records, but **not enforced** by the web backend.

> **Note** Do not present **Minimum RSA key size** to your security reviewers as an active control. The value round-trips faithfully through connect, test, provision, demo generation and rebuild, but the web backend's TLS options do not currently apply it.

14.2.2 Certificate modes

Verify (system trust store)

The default. The server certificate must chain to a trust anchor the operating system already trusts, and the host name must match.

Pin fingerprint

The driver pins the server certificate to the SHA-256 fingerprint you enter. Colons are stripped, and the remaining value must be pure hexadecimal, 16 to 128 characters long.

Accept any (insecure)

Disables certificate verification *and* host name checking. Use it only for laboratory or self-signed setups; anyone on the network path can impersonate the database.

Mode parsing is whitespace-tolerant and fails *closed*: any stored value the application does not recognise — a typo, a legacy record, restored data from an older backup — is treated as **Verify**, never as "no verification".

> **Warning** In *Pin fingerprint* mode a blank or non-hexadecimal fingerprint refuses the connection outright with: "This connection is set to pin a certificate fingerprint, but the fingerprint is missing or not hexadecimal. Enter the server's SHA-256 fingerprint, or change the certificate mode." This closes an earlier fail-open behaviour in which an unusable fingerprint silently disabled verification while the interface still claimed the certificate was pinned.

14.2.3 Create schema and tables

The banner on the Database sub-tab reads: "Creates the schema named above and the process-mining tables (PROJECTS, JOURNEYS, STEPS, METAS, NOTES) if they don't exist, using the credentials entered here. This requires a database account permitted to **CREATE SCHEMA** and **CREATE TABLE** — only your database administrator can grant those rights; this application cannot."

1. Enter Host, Port, Username, Password and, crucially, a **Schema** name. Without one the button reports "Enter a schema name first."
2. Optionally tick **Also build pre-materialized transitions after provisioning** (default off). This is available only in the admin interface.
3. Click **Create schema & tables**. The button shows "Creating…" while it works.
4. Read the result. Success is green: "Created: schema <name>, PROJECTS, JOURNEYS, STEPS, METAS, NOTES", optionally followed by "· transitions built (N pairs)" or "· transitions build failed: <error>". Failure is red and shows the database error, or "Could not create the schema."

Provisioning connects *without* opening a schema (the target may not exist yet), then issues `CREATE SCHEMA IF NOT EXISTS`, `OPEN SCHEMA` and the table DDL in the order PROJECTS, JOURNEYS, STEPS, METAS, NOTES. All DDL uses `IF NOT EXISTS`, so re-running it never disturbs existing data. If a step fails, the result still lists everything that was created *before* the failure, so partial progress is visible. The schema identifier is safely double-quoted (embedded quotes are doubled), which means mixed-case and special names work — but remember that a quoted name is case-sensitive in Exasol.

Warning Provisioning uses the password **as typed in the form**, not the stored one. On a saved connection with a blank password field it runs with an empty password and usually reports an authentication failure even though the saved connection works perfectly. Retype the password before provisioning, testing or generating demo content.

Note Because `IF NOT EXISTS` leaves pre-existing tables untouched, a JOURNEYS table that already exists does not receive the physical layout the application's DDL declares for new tables. Ask your DBA to apply it manually:

```
ALTER TABLE JOURNEYS DISTRIBUTE BY EVENT_ID;
ALTER TABLE JOURNEYS PARTITION BY EVENT_TIME;
```

Distributing by EVENT_ID co-locates all events of one journey on one cluster node, so the transition query's window function and every `GROUP BY EVENT_ID` run node-local; partitioning by EVENT_TIME lets Exasol prune by date range.

14.2.4 Assign to users

The right-hand column of the Database sub-tab is headed **Assign to users** and contains a checkbox for every user account known to the installation. Its empty state is “No users to assign.” Only ticked users see the connection in the main application.

1. Open the connection and tick the users who need it.
2. Click **Save**.
3. Tell the affected users to reconnect — the save has already dropped any live sessions on this connection.

Note Directory (LDAP) users appear in this grid only after their first successful sign-in, because that is when their local record is created. Assign connections to them afterwards, exactly as for local accounts.

Tip Power users may create and manage their own connections in the main application, and a connection they create is auto-assigned to them. As an administrator you can see and edit those connections here too — including options such as pre-materialized transitions that the in-app editor does not expose.

14.3 The LLM sub-tab

The sub-tab is labelled “LLM (optional)”. It attaches one OpenAI-compatible server to this connection; the AI-supported documentation features in the main application use it.

Field	Placeholder	Notes
Server URL	https://api.openai.com/v1	The base URL of any OpenAI-compatible endpoint. Subject to the SSRF guard below.
Model	gpt-4o	The model name sent with requests.
API key	bullets	Write-only, like the database password: the hint “(set — leave blank to keep)” appears once a key is stored.

14.3.1 The reachability probe

Whenever a user connects, and whenever you press **Test connection**, the application issues `GET <Server URL>/models` with `Authorization: Bearer <API key>` if a key is set, using a **5-second** timeout. The server counts as reachable unless the request errors or answers with HTTP 500 or above — a 4xx still means a server is there. With no URL configured the LLM is simply reported as not reachable, which is not an error. Failures are recorded in the log under operation `llm-test`; the message users see is “LLM server not reachable.”

14.3.2 The SSRF guard

The LLM URL is user-supplied, so the backend validates it before making any outbound request:

- Only `http://` and `https://` are accepted — anything else is refused with “Only http:// and https:// URLs are allowed.” A URL without a host gives “The URL has no host.”
- The host name is resolved and **every** resolved address is checked. If any one of them is disallowed the whole URL is refused: “The URL resolves to <reason> (<address>); refused.” A host that cannot be resolved gives “The host could not be resolved: <host>”.
- Always blocked: link-local and cloud-metadata addresses (including 169.254.169.254), multicast, reserved and unspecified addresses. IPv4-mapped IPv6 addresses are unwrapped before the check.
- **Loopback and private (RFC 1918) targets are allowed by default**, because a local LLM server — Ollama, LM Studio, an internal vLLM — is a first-class use case. Hardened deployments set the environment variable `PMW_BLOCK_PRIVATE_LLM_HOSTS=1` to refuse those too, with the reasons “a loopback address” and “a private address”.
- The outbound connection is **pinned to the vetted IP address**, so DNS rebinding cannot swap the target between the check and the connection, while SNI and certificate verification stay bound to the real host name (internationalised host names are canonicalised first so the pin always applies). Redirects are disabled, so a 3xx response cannot bounce the request to an unvetted host.

> **Note** A guard refusal is never shown to end users as an error — the LLM simply appears unreachable. The reason is written to the log as “LLM server URL refused (SSRF guard): …” under operation `llm-test`. If an LLM you believe to be correct stays stubbornly unreachable, look there first.

14.4 Testing a connection

1. Open the connection and retype the database password (and the LLM API key if you want that half tested too).

2. Click **Test connection** in the editor footer. The button shows “Testing…”.
3. Read the result banners: “Database: connection OK” or “Database: <error>”; and, when an LLM URL is filled in, additionally “LLM: reachable — N models”, “LLM: <error>” or “LLM server not reachable.”

The test opens and immediately closes a throwaway session, so it never disturbs anyone's active connection. Both halves are logged, under operations `connection` / `db-test` and `llm-test`. The test uses the credentials *as typed*: a blank field tests without a secret rather than falling back to the stored one.

Timeout	Value
Exasol connect (socket open and login)	15 s, fixed
Exasol socket, per statement	300 s, fixed
LLM reachability probe	5 s
Rebuild, provisioning, project operations	No application-level timeout; bounded only by the 300 s socket timeout per statement

14.5 The Projects sub-tab

This sub-tab lists what is actually stored in the connection's schema and is the only place in the admin interface where you can remove a project's data.

The header reads “Projects in this schema” with a **↻ Refresh** button, and the caption names the schema: “Projects stored in `<schema>`, with their journey and event counts.” The listing uses the **stored** connection credentials, so no retyping is required here.

Column	Shows
Project	The project title in bold, with the raw project id on a second line when it differs. An untitled project shows its PROJECT_ID as the title.
Journeys	Distinct EVENT_IDs, right-aligned and thousands-separated.
Events	Row count, right-aligned and thousands-separated.
Delete	Danger button; see the procedure below.

Possible states: “Save the connection first to list its projects.” (unsaved connection), “Open the Projects tab to load them.”, “Loading projects…”, “No projects found in this schema.”, or a red error — either the database error or the fallback “Could not read projects.”

> **Note** The counts consider **ORIGINAL data only**; sample sets are excluded, so the numbers here are smaller than the raw row count of a schema that contains samples. A project is listed if it exists in *either* PROJECTS or JOURNEYS, which makes orphans on either side visible. A missing table is tolerated rather than treated as an error, so an unprovisioned schema yields an empty list.

14.5.1 Deleting a project

> Requires: **Administrator** (in the main application: ownership of the connection plus the admin, power or developer role).

1. Open the connection, switch to **Projects** and press **↻ Refresh** so you are looking at current counts.
2. Click **Delete** on the project's row.
3. Read the confirmation carefully: `Delete project "<id>"?` — "This clears its rows from PROJECTS, JOURNEYS, STEPS, METAS, NOTES and TRANSITIONS_RAW in this schema. This cannot be undone."
4. Confirm. The toast reports the scale of what was removed: "Project deleted (N events)". Failures surface the database error as a toast.

The deletion issues `DELETE … WHERE PROJECT_ID = '<id>'` against each table in this order — children before the parent:

Order	Table	What is lost
1	JOURNEYS	The event log itself, *including every sample set* (no SAMPLE_SET filter is applied).
2	STEPS	Step descriptions, colours, shapes, scores, end-of-process flags and step groups.
3	METAS	The three meta-column titles.
4	NOTES	Every note and comment thread attached to the project.
5	TRANSITIONS_RAW	The pre-materialized transition pairs, if the table exists.
6	PROJECTS	The project row itself.

Tables that are absent are **skipped, not treated as errors** — NOTES is created lazily and TRANSITIONS_RAW is optional, so neither is guaranteed to exist. The response lists the tables that were actually cleared. Every deletion is logged at WARN severity under operation `connection`, naming the user, the project id, the connection and the outcome.

> **Warning** There is no undo and no recycle bin. The application's own backups deliberately exclude event data — they capture settings, presets, layouts and connection definitions, not JOURNEYS — so a deleted project can be restored only from your database's own backups. The project's notes are destroyed with it.

14.6 Pre-materialized transitions

14.6.1 What TRANSITIONS_RAW is and why it exists

The process map is a directly-follows graph. Normally the application computes it **live** on every reload: a `LEAD()` window over JOURNEYS pairs each event with its successor (ordered by EVENT_TIME, then STEP_ID), and the result is aggregated into count, average, minimum, maximum and standard deviation of the gap per (from-step, to-step) pair. On large logs that window function dominates the query time, and it is paid again for every filter change.

`TRANSITIONS_RAW` stores the pairing **once**: one row per consecutive step pair, with the columns PROJECT_ID, EVENT_ID, FROM_STEP, TO_STEP, FROM_TIME, TO_TIME, DUR_SECS (the gap, precomputed in seconds) and SAMPLE_SET. When the map reads from it, the request-time query is a plain `GROUP BY` aggregate with no window function at all — in the words of the admin interface, “much faster for interactive filtering on large logs”.

The table is a sixth, optional table alongside the five canonical ones. It is built with `CREATE OR REPLACE TABLE … AS SELECT` into a staging table so that every column type — critically EVENT_ID, whose type varies between deployments — is inherited from JOURNEYS exactly; a hard-coded type would make the semi-join fail with Exasol's “Incomparable Types” error. The pairing covers every project and every sample set in one pass, partitioning by (PROJECT_ID, SAMPLE_SET, EVENT_ID) because an EVENT_ID is only unique within one project and sample set. After the build the application applies `DISTRIBUTE BY EVENT_ID` on a best-effort basis (a failure there never aborts the rebuild — distribution is only an optimisation), counts the rows, then drops the old table and renames the staging table into place. Readers are never blocked; the swap leaves a sub-millisecond window with no live table, during which reads simply fall back to the live query.

14.6.2 The per-connection toggle

The banner on the Database sub-tab carries the bold checkbox **Use pre-materialized transitions** and this explanation: “Reads the process map from a prebuilt `TRANSITIONS_RAW` table instead of running the windowed query live — much faster for interactive filtering on large logs. It falls back to the live query until the table is built, so **rebuild it after each load of `JOURNEYS`**.”

The flag is stored per connection and defaults to **off**. It is captured when a user connects *and re-read on every request*, so toggling it takes effect on the user's next reload — they do not have to reconnect. The checkbox exists only in the admin interface; the in-app connection editor used by power users does not expose it, even for connections they own.

With the flag on, the map query reads TRANSITIONS_RAW using the same filter clauses as the live query (the table carries PROJECT_ID, SAMPLE_SET and EVENT_ID). **Any** read failure — table missing, never built, dropped by a DBA — is non-fatal: the application falls back to the live query so the map never breaks, and logs a warning under operation `materialize`: “Pre-materialized transitions are enabled but reading TRANSITIONS_RAW failed for project … — falling back to the live query. Cause: …”.

14.6.3 How the application indicates live versus pre-materialized

Users can always see which path served their map. A pill in the detail pane header (not shown in the Individual Journey view) reports the mode:

Pill	Meaning	Tooltip
⚡ Pre-materialized	The map was read from TRANSITIONS_RAW.	“Transitions are read from the pre-materialized TRANSITIONS_RAW table.”
⚠ Live (not built)	The flag is on but the table is unusable, so the live query ran instead.	“Pre-materialized transitions are enabled for this connection, but TRANSITIONS_RAW is not built yet — running the live query meanwhile. Rebuild it in the admin interface (Connections → Rebuild now).”
↻ Live query	The flag is off; this is the normal live computation.	“Transitions are computed live from the event log on each load.”

The graph API reports the same information as `transitionsMode` (`materialized`, `live` or `fallback`) together with `queryMs`, the server-side wall-clock time of the database queries, which the chart footer displays as, for example, “123 ms” or “1.23 s”. Comparing that figure before and after a rebuild is the simplest way to judge whether materialisation is worth it on your data.

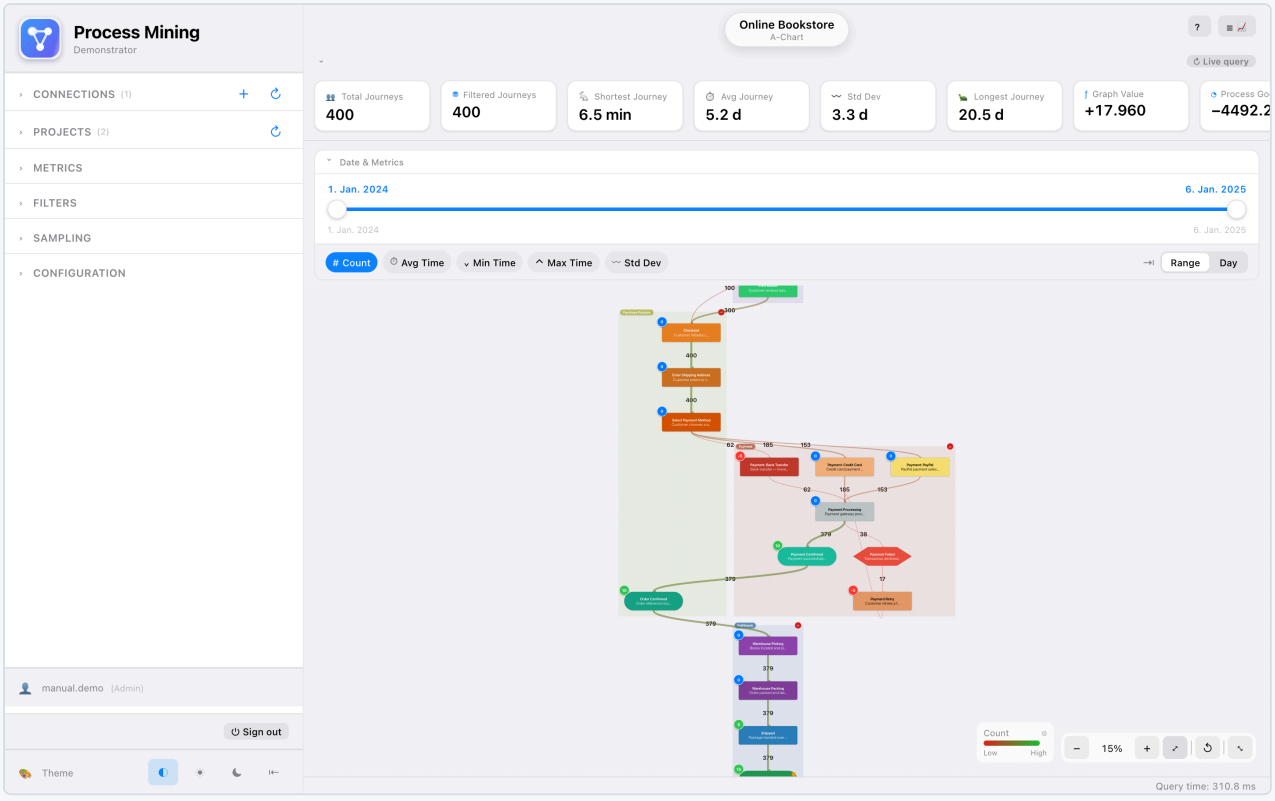


Figure 14.2 — The transitions-mode pill and query time appear with the process map in the main application.

Warning TRANSITIONS_RAW is a *snapshot*. It goes stale the moment JOURNEYS changes, and nothing rebuilds it automatically — not an ETL load, not demo-content generation, not creating or deleting a sample set. If the table still exists it will serve *stale* pairs without complaint; if it was never built the map shows “⚠ Live (not built)”. Make a rebuild the last step of every load.

14.6.4 Rebuild now

1. Open the connection in the admin interface and make sure a **Schema** is set — a rebuild without one fails with “A schema name is required.”
2. Save the connection if you have not already; **Rebuild now** is disabled until then, with the tooltip “Save the connection first”.
3. Click **Rebuild now**. The status shows “Rebuilding… (this runs the pairing once and may take a while)”.
4. Wait for the result: “Built N pairs.” in green, or the error text in red.

The last outcome is stored per connection and shown again whenever the editor is reopened, as one of:

- “Not built yet — the live query is used until you rebuild.”
- “Last built: <date> · N pairs.”
- “Last rebuild failed: <error>” (red).

A rebuild is idempotent and safe to repeat. Two rebuilds of the *same* connection never overlap — they would race on the staging swap — so a second concurrent call is refused with HTTP 409 “A rebuild is already running for this connection.” Different connections rebuild independently.

> **Tip** When you provision a new schema, tick **Also build pre-materialized transitions after provisioning**. The table is empty until JOURNEYS is loaded, but its structure exists immediately, so the flag can be switched on straight away and the first real rebuild has nothing to create.

14.6.5 Rebuild from a script (API) and the bearer token

Inside the same banner is a collapsible section headed **Rebuild from a script (API)**: “Let a scheduler (cron / ETL) rebuild *this connection* right after loading its JOURNEYS, without an admin login. The token below is scoped to this connection only, and calls are rate-limited.”

The endpoint lives on the **admin service**, not the application:

```
POST /api/connections/<connection-id>/rebuild-transitions
```

It accepts either a signed-in admin session or the header `Authorization: Bearer <token>`, and it uses the connection's **stored** credentials — so the calling script never handles a database password.

1. Save the connection. Until then the status line reads “save the connection first”.
2. Expand **Rebuild from a script (API)** and click **Generate / rotate token**. Confirm the dialog: “Generate a new token for this connection? Any existing token stops working.”
3. Copy the token from the panel headed “New token (copy it now — it is not shown again):” and store it in your scheduler's secret store.

4. Use the 📋 button to copy the ready-made curl example, which contains the real token while it is still visible on screen:

```
curl -k -X POST \
  -H "Authorization: Bearer <token>" \
  "https://<admin-host>:8453/api/connections/<connection-id>/rebuild-transitions"
```

5. Add that call as the final step of the ETL job that loads JOURNEYS.

To rotate, repeat step 2: the new token replaces the old one instantly, so update the scheduler in the same maintenance window. To withdraw access entirely, use **Revoke** (shown only while a token is set), which confirms with “Revoke this connection's token? A scheduler using it will stop working.” The status line tracks the state as “a token is set for this connection” or “no token set”. Deleting the connection deletes its token as well.

Property	Behaviour
Scope	A token only ever authorises the rebuild endpoint of *its own* connection.
Storage	A 32-byte URL-safe random value; only its SHA-256 hash is stored, and it is verified in constant time. The plaintext is unrecoverable — if it is lost, rotate.
Rate limit	60-second cooldown per connection for token callers: HTTP 429 “Rebuild throttled — try again in Ns.” Admin-session calls are never throttled.
Single flight	HTTP 409 “A rebuild is already running for this connection.”
Other errors	401 “Not authenticated” without a valid session or token; 404 “No such connection.”
Auditing	Every rebuild logs “…rebuilt materialised transitions for connection '<name>' (<id>) — ok, N pairs” or “… failed: <error>” under operation `materialize`, with the actor recorded as the admin username or `api-token`. Token generation and revocation are logged at WARN.

Note The `-k` in the example curl skips TLS verification and exists only for installations still using a self-signed certificate. Remove it once you have installed a certificate your scheduler's host trusts. Also remember that the port in the example is the *admin* port (HTTPS 8453 / HTTP 8090 by default, overridable with `PMW_ADMIN_HTTPS_PORT` / `PMW_ADMIN_PORT`), not the application's.

14.7 In-database sampling

Alongside pre-materialized transitions, the Database sub-tab carries a second per-connection performance option for very large logs: the checkbox **Build sample sets inside the database**. It changes *how* a sample set (Sample 1–3) is created — not what the sampling methods do — and, like the materialized-transitions flag, it is off by default and only exposed in the admin connection editor.

With the flag **off** (the default), creating a sample uses the ordinary path: the application reads the candidate journey ids for the chosen method out of the database, picks the sample in Python, and re-inserts the chosen journeys in batches. This is fine for small and mid-sized logs. On a very large log — on the order of hun-

dreds of millions of events — extracting every journey id to the application and re-inserting in batches is slow enough to time out.

With the flag **on**, the whole sample is built **inside Exasol**: a single set-based `INSERT … SELECT` picks the journeys with pure SQL and writes the sample rows in one statement, with nothing extracted to the application. All three methods are supported and produce statistically equivalent results:

- **Random** — a uniform sample over journeys (each journey drawn once, independent of its length).
- **Temporal** — proportional across calendar-month buckets of the journey start time.
- **Path-diverse** — proportional across distinct journey variants (the ordered step sequence).

The temporal and path-diverse methods use a **clean proportional allocation**: each bucket gets its share, and when the shares sum to more than the requested count the surplus is trimmed at random across all buckets, so no month or variant is favoured. The build runs on its own database connection with the statement timeout lifted, so a large one-shot write is not cut off mid-run.

> **Note** This flag only affects *sample creation*. Existing samples, the sampling methods offered in the app, and everything downstream are unchanged. Turn it on for the connection that holds the very large log; leave it off elsewhere — small projects gain nothing and keep the simpler app-side path. Which path a given *Create sample* took is recorded in the admin Logging tab under operation `sampling` (`in-database` vs `app-side` , with the journey count and seconds), which is the quickest way to confirm the flag is taking effect.

14.8 Directory (LDAP)

The Directory tab lets the *main application* accept sign-ins from an LDAP directory using a search-and-bind flow. Its header states the merge model: “Directory users are created here automatically on first login as plain, enabled users — grant them a role or database connections like any other user. Local accounts always keep working.”

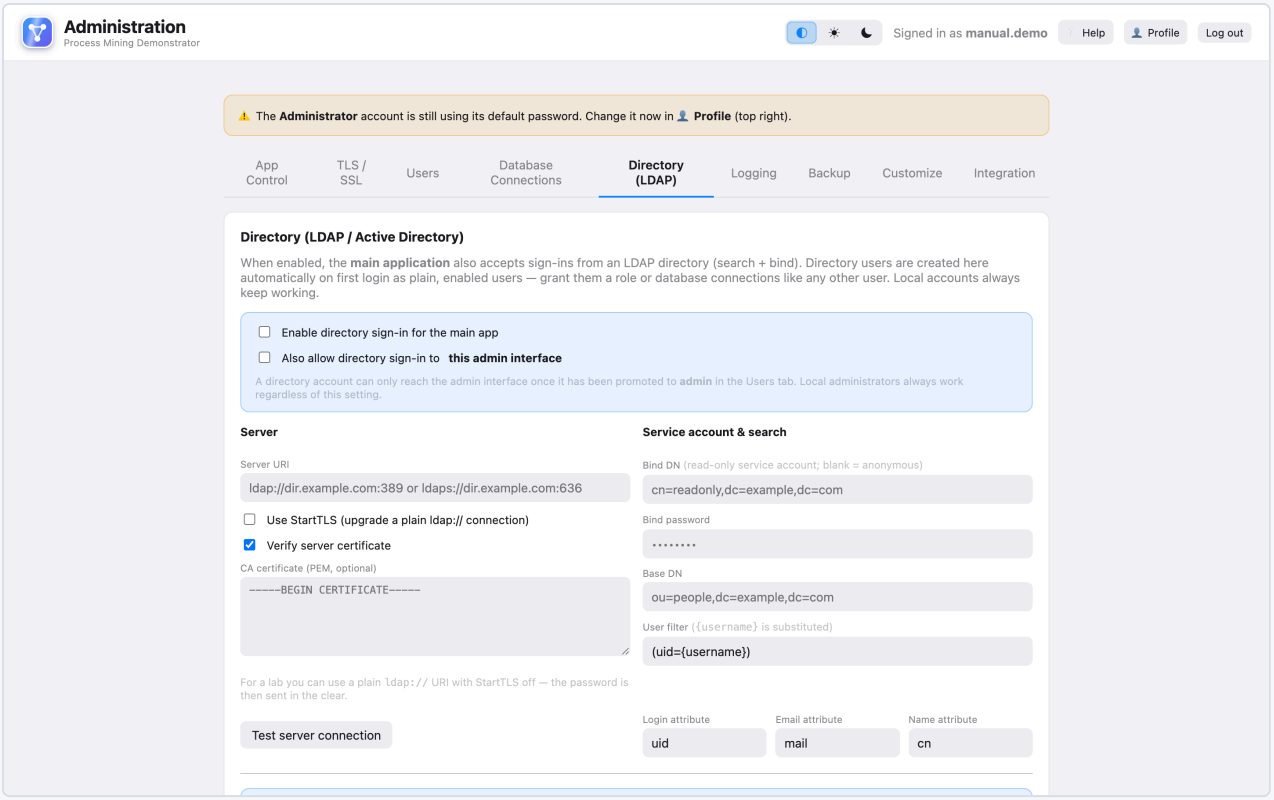


Figure 14.3 — The Directory (LDAP) tab: server settings, the service account and the two test tools.

14.8.1 Enabling directory sign-in

Three checkboxes sit in the top banner:

Enable directory sign-in for the main app

Turns on LDAP authentication for the application itself.

Also allow directory sign-in to this admin interface

Extends it to the admin interface. Note the accompanying text: “A directory account can only reach the admin interface once it has been promoted to **admin** in the Users tab. Local administrators always work regardless of this setting.”

Show the Authentication Server availability indicator on the login panels

Controls the small availability LED shown below the sign-in form (see Signing in). When on (the default), both the app and admin login panels display a live green/red indicator reporting whether the directory server is reachable. Turn it off to hide that indicator from users while keeping directory sign-in working. This setting only has an effect once a directory is configured.

> **Warning** The administrator role is never granted by the directory. Group membership, directory attributes and organisational units confer nothing; every role is granted explicitly in the Users tab. Keep at least one *local* administrator account — it is your break-glass access if the directory becomes unreachable.

14.8.2 Server settings

Field	Default / placeholder	Notes
Server URI	“ldap://dir.example.com:389 or ldaps://dir.example.com:636”	Plain LDAP or LDAPS.
Use StartTLS (upgrade a plain ldap:// connection)	off	Upgrades a plain connection to TLS after connecting.
Verify server certificate	**on**	Leave it on in production.
CA certificate (PEM, optional)	—	Supply the issuing CA when the directory uses a private PKI.

The tab notes that for a laboratory you may use a plain `ldap://` URI with StartTLS off — in which case the password travels in the clear. Do not do this on a production network.

14.8.3 Service account and search

Field	Default / placeholder	Notes
Bind DN	“cn=readonly,dc=example,dc=com”	A read-only service account; leave blank for an anonymous bind.
Bind password	bullets	Write-only: the hint “(set — leave blank to keep)” appears once stored. Unlike the database tests, a blank field here *reuses the stored password* when testing.
Base DN	“ou=people,dc=example,dc=com”	The subtree searched for user entries.
User filter	`(uid={username})`	`{username}` is substituted with what the user typed.
Login attribute	`uid`	Becomes the username inside the application.
Email attribute	`mail`	
Name attribute	`cn`	Becomes the display name shown beside the username.

14.8.4 Configuring and testing directory sign-in

> Requires: **Administrator**

1. Enter the **Server URI** and choose the transport: LDAPS, or plain LDAP with **Use StartTLS** ticked. Leave **Verify server certificate** on and paste the issuing CA into **CA certificate (PEM, optional)** if it is not publicly trusted.
2. Click **Test server connection**. Success reads “Server reachable — service bind OK”; a failure appears as a warning with the error text.
3. Fill in **Bind DN**, **Bind password**, **Base DN** and the **User filter**, then adjust the three attribute names if your directory does not use uid/mail/cn.

4. In **Test a user login**, enter a real **Test username** and **Test password** and click **Test**. Three results are stacked:
   - “Service bind: OK” or “Service bind failed: <error>”
   - “Search matched N entry/entries (<DN>)” — informational when exactly one entry matched, a warning otherwise
   - “User login: OK — <username> (<dn>)” or a warning such as “User login failed”
5. Tick **Enable directory sign-in for the main app**, and **Also allow directory sign-in to this admin interface** only if you intend directory-based administrators.
6. Click **Save**. The toast confirms “Directory settings saved”.

> **Tip** If the user test reports more than one matching entry, tighten the **User filter**. A filter that can match several entries makes sign-in ambiguous, and the multi-match result is deliberately shown as a warning rather than as success.

> **Note** **Test server connection** checks the server and the service bind alone; **Test a user login** performs the full search-and-bind for one account. Use the first to diagnose network, TLS and service-account problems, and the second to diagnose Base DN, filter and attribute problems. All saves and tests are recorded in the log under operation `ldap`.

14.8.5 How directory users merge with local ones

Directory accounts are not pre-provisioned. A directory user becomes visible to you only after they have signed in successfully for the first time, at which point a local record is created automatically: a plain, enabled user with no roles and no connections. From then on the account behaves like any other in the Users tab, with these differences:

Aspect	Local account	Directory account
Source pill in the Users list	**local**	**LDAP**, with the display name shown beside the username
Password	Stored by the application; **Reset password** is available	Verified by the directory; **Reset password** is hidden — change it in the directory
Roles (admin, power, developer)	Granted in the Users tab	Granted in the Users tab — never inherited from the directory
Passkey and two-factor permissions	Per user or via the all-users master row	Identical — both can be allowed for directory users too
Database connections	Assigned in the Database Connections tab	Identical, once the account exists
Enable / Disable / Unlock	Available	Available — disabling here blocks sign-in even if the directory still accepts the password

The **All / Local / LDAP** filter chips at the top of the Users tab, each with a live count, let you see the two populations separately.

> **Note** Once a directory is configured, both sign-in pages show a small **directory-server LED** — a green or red dot with “Directory server available” or “Directory server unavailable”. It comes from a cached probe with a 20-second lifetime and never delays the page. A red dot before anyone has even typed a password usually points at the network or the service account rather than at the user.

14.9 Operational checklist

Situation	What to do
A test or provisioning run fails with an authentication error on a connection you know works	Retype the password. Test, provision and demo generation use the typed value, never the stored one.
Users report the map is slow after an ETL load	Check the pill in the detail pane. “⚠ Live (not built)” means TRANSITIONS_RAW is unusable — press **Rebuild now**, then automate it with the API token.
The map shows figures that do not match the source system	Suspect a stale TRANSITIONS_RAW. Rebuild it, or untick **Use pre-materialized transitions** to compare against the live query.
An LLM server is configured but always shows as unreachable	Look in the Logging tab for operation `llm-test`. An SSRF-guard refusal is logged there but never shown to users.
A scheduler's rebuild call returns 429	It is inside the 60-second per-connection cooldown. Call it once per load, not in a retry loop.
A rebuild token was lost	It cannot be recovered — only its hash is stored. Generate a new one and update the scheduler.
A directory user cannot be assigned a connection	They have never signed in. The local record — and therefore the assignment checkbox — is created on first successful sign-in.
Everyone was disconnected unexpectedly	Someone saved the connection, changed its assignments or deleted it. This always drops live sessions immediately.

15 Administration: Logging, Backup and Customisation

> This chapter covers the last four tabs of the admin interface: **Logging**, **Backup**, **Customize** and **Integration**. Between them they answer four separate operational questions — what the product recorded and how you read it back, what survives a rebuild of the machine and how you restore it, what the sign-in pages look like, and whether the integration console exists at all. Everything described here lives on the admin interface (`http://<host>:8090` or `https://<host>:8453`) and everything here requires an administrator account.

Requires: **Administrator** — every tab, control and procedure in this chapter.

The log: one file, three writers

There is exactly one structured log. The main application, the admin interface and the integration console all write into it, so a single view shows a user's sign-in, the SQL the backend then ran on their behalf, and the certificate an administrator activated an hour earlier. The **Logging** tab is the reader for that file: it configures what gets recorded, filters what you see, and exports or clears what has accumulated.

All timestamps in the viewer, in the row popup and in downloaded log files are rendered in the **display time-zone** set under **App Control** → **Timezone**. Changing that setting re-renders every timestamp immediately; it does not rewrite the stored entries and it needs no restart.

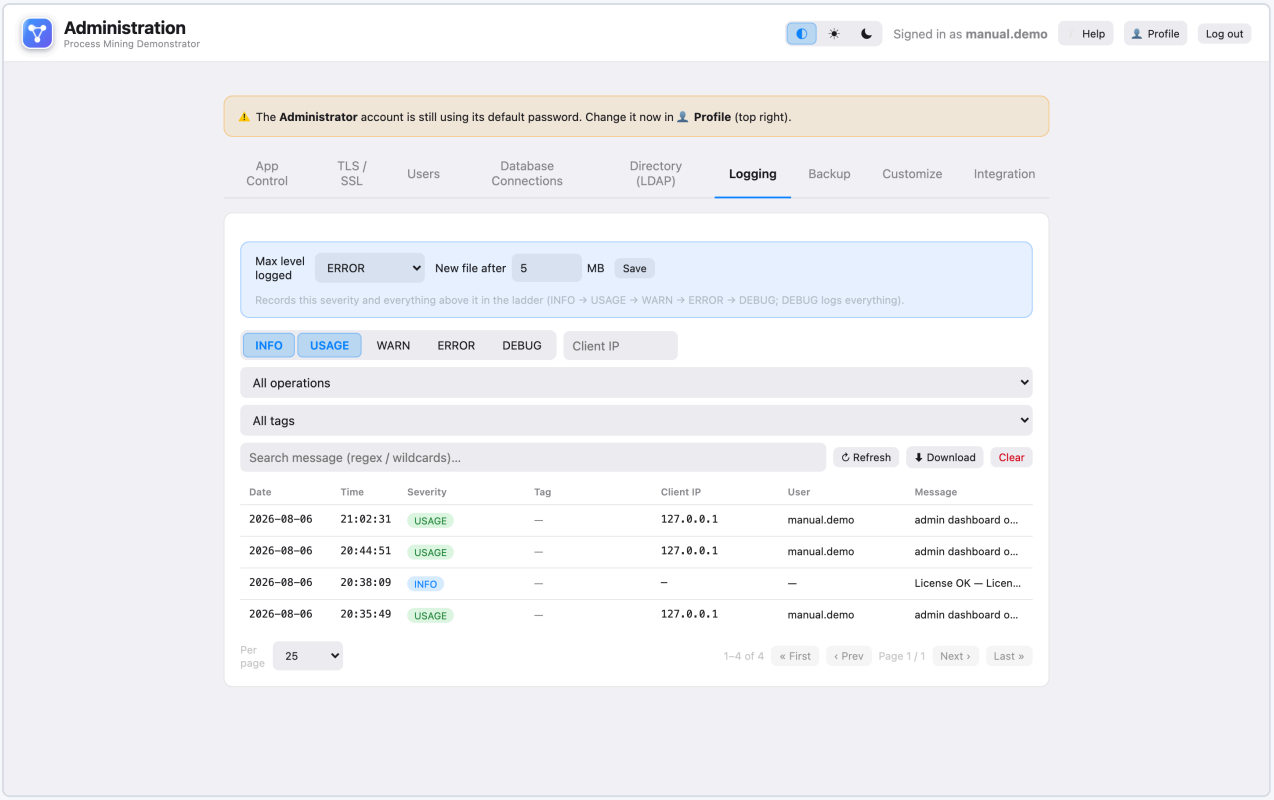


Figure 15.1 — The Logging tab: the configuration banner at the top, the filter row beneath it, and the paged entry table.

The severity ladder

Severities are not ranked by urgency in this product. They form a **ladder** — **INFO** → **USAGE** → **WARN** → **ERROR** → **DEBUG** — and the **Max level logged** setting is cumulative from the top: choosing a level records that severity *and every level before it*. This is why DEBUG, which would be the quietest level in many products, is the loudest one here.

Setting	Records	Use it when
INFO	INFO only	You want nothing but the routine trail and can accept losing warnings and errors.
USAGE	INFO + USAGE	You are interested in who used what, and not in faults.
WARN	INFO + USAGE + WARN	You want destructive administrative actions on record.
ERROR *(default)*	INFO + USAGE + WARN + ERROR	Normal operation. Everything except SQL traces.
DEBUG	Everything, including full SQL traces	Diagnosing a query or a performance problem, for a bounded period.

> **Warning** **DEBUG** records every SQL statement the backend issues. On a busy installation this fills the live log quickly, and because only the newest ten archive files are kept, a long DEBUG session can push older history out of the archive altogether. Switch to DEBUG for a specific investigation and switch back afterwards.

Setting the level and the rotation size

1. Open the **Logging** tab.
2. In the configuration banner, choose a value in **Max level logged**. The caption restates the rule: records this severity and everything above it in the ladder.
3. Set **New file after [N] MB** — the size at which the live log rotates. Accepted range is **1** to **1000** MB; the default is **5** MB. The server additionally enforces a 50 KB floor internally, so a rotation can never be triggered by a nearly empty file.
4. Click **Save**. The toast *"Logging config saved"* confirms it. The change applies to what is written from that moment on; it does not retroactively add or remove entries.

Tags, and how they differ from operations

Every entry carries an **operation** (what part of the product wrote it) and, optionally, a **tag**. Tags are cross-cutting labels: they are independent of both severity and operation, and they exist so that you can pull one kind of activity out of a mixed log with a single click.

USER

Every deliberate *user action* — a successful state-changing request (create, update, delete, connect, run an import, change a setting…) made by a signed-in user, recorded at **USAGE**. Filter to **USER** for an audit trail of what users did. Reads and background polls are not actions and stay at **DEBUG**. These come from the compute backend, the single point every application and integration action passes through, so an action is logged once, not once per proxy.

DATA

Data-load events. Because a data import is also a user action, its entries carry both **DATA** and **USER**.

SQL

SQL traces. These belong to the DEBUG rung of the ladder, so they only appear when **Max level logged** is DEBUG.

BACKUP/RESTORE

Every backup and restore action — exports, server-side runs, inspections, restores and their failures.

The **All tags** dropdown is populated from the tags actually present in the current log, so a tag that has never been written does not appear as an option. Untagged entries show **—** in the Tag column.

The table below lists every operation that appears in the **All operations** dropdown and the tag it carries.

Operation	What it records	Tag
`login`	Sign-ins on any of the three surfaces	—
`logout`	Sign-outs	—
`page`	Page requests served	—
`config`	Configuration changes made in the admin interface	—
`license`	Licence upload and removal	—
`tls`	Certificate generation, upload, activation and deletion; TLS mode changes	—
`connection`	Database connection saves, deletions, project deletions and connection tests	—
`llm-test`	LLM reachability tests from the connection editor	—
`ldap`	Directory settings saves and directory tests	—
`backup`	Backup exports, server-side runs, inspections and restores	**BACKUP/RESTORE**
`customize`	Login-page background changes	—
`restart`	Restart requests issued from App Control	—
`material-ize`	Rebuilds of a connection's pre-materialised `TRANSITIONS_RAW` table	—
`sql`	SQL statement traces with their execution time	**SQL**

Data-load activity is tagged **DATA** *regardless of the operation that produced it.*

> **Note** Destructive actions are deliberately recorded at **WARN** so they stand out in a log that is otherwise dominated by routine traffic. This covers user and connection deletions, project deletions, certificate deletions, restores, rebuild-token generation and revocation, and clearing the log itself. Certificate generation, upload and activation are recorded at INFO; backup exports and inspections at USAGE.

Reading and filtering the log

The filter row sits between the configuration banner and the table. All filters apply to the *whole* log, not just the page on screen, and any change to a filter returns you to page 1.

Control	Behaviour
Severity chips	One toggle per level, multi-select. The display default is **INFO and USAGE selected**; WARN, ERROR and DEBUG entries are hidden until you toggle them on. Selecting every chip is the same as “show all”.
Client IP	Free-text match on the originating address.
All operations	Dropdown built from the operations present in the log.
All tags	Dropdown built from the tags present in the log.
Search message	Free-text search over the message, accepting regular expressions and wildcards. Debounced by roughly 300 ms as you type.

> **Warning** The severity chips are a *view* filter only. They never change what is recorded — that is solely the **Max level logged** setting. An entry hidden by a chip is still in the file; an entry never recorded cannot be brought back by any chip.

The table shows Date, Time, Severity (colour pill: INFO blue, USAGE green, WARN orange, ERROR red, DEBUG grey), Tag, Client IP, User and Message. Every row is exactly one line and long messages are ellipsised, so the table stays readable at a glance.

1. Click any row to open the **Log entry** popup with the complete entry: Date, Time, Severity, Client IP, User, Operation, Tag and the full message.
2. For an `sql` entry the statement is **pretty-printed** in the popup, and its execution time — parsed from the `SQL (12.3 ms):` prefix of the message — is shown as a separate **Execution time** field.
3. Press `Esc` or click **Close** to dismiss the popup.

Paging controls sit below the table: **Per page** offers 10, **25** (default), 50 and 100, with **« First**, **‹ Prev**, **Next ›** and **Last »**, a “start–end of total” readout and “Page X / Y”. When nothing matches, the table shows *“No matching log entries.”* and the pager reads “No entries”. **↻ Refresh** re-reads the log without clearing your filters.

Exporting a filtered view

1. Set the severity chips, operation, tag, client IP and search text so the table shows exactly the entries you want.
2. Click **↓ Download**. The browser saves the *currently filtered* view as `pmw-log.log`, up to a maximum of **5000** entries.

> **Tip** Because the export follows the filter, the fastest way to hand a specific incident to support is to search for the message text or filter by client IP first, then download. Timestamps in the exported file use the configured display timezone, so state that timezone when you send the file on.

Rotation, archives and clearing

When the live log passes the configured size it is rotated out to `data/logs/pmw-<timestamp>.log`. Only the **newest ten** archive files are kept; older ones are deleted. Archives are plain files on the server — copy them

off the machine if you need history beyond that window, and include `data/logs/` in your filesystem-level backups, because the product's own backup file does not contain the log.

1. Click **Clear** (danger) in the filter row.
2. Confirm the dialogue: *“Clear the live log? — This empties the current log (rotated archive files are kept).”*
3. The live log is emptied. The clearing action is itself written into the now-empty log at **WARN**, so the record of who cleared it survives.

What the log never contains

Log entries name the action, the actor and the affected object — never the secret involved. This is deliberate, and it is worth knowing precisely, because it defines what a leaked log can and cannot expose:

- Passwords are never written. A password change is recorded as *“changed their password — other sessions invalidated”*.
- Database passwords, LLM API keys and directory bind passwords are not written by connection tests, LDAP tests or saves; those entries record the operation and its outcome only.
- Rebuild tokens are shown in the browser once at generation and stored only as a hash; the log records that a token was generated or revoked, at WARN, and not its value.
- The backup encryption password is never written. A backup export records the include-set and whether the resulting file was **encrypted** or **PLAINTEXT** — the fact, not the password.
- Two-factor secrets are stored encrypted and recovery codes only as hashes, so neither can appear in a log message.

Backup

The **Backup** tab does two independent jobs: it produces backup files (on demand or on a schedule) and it restores from one. Both halves share the same include options and the same encryption password.

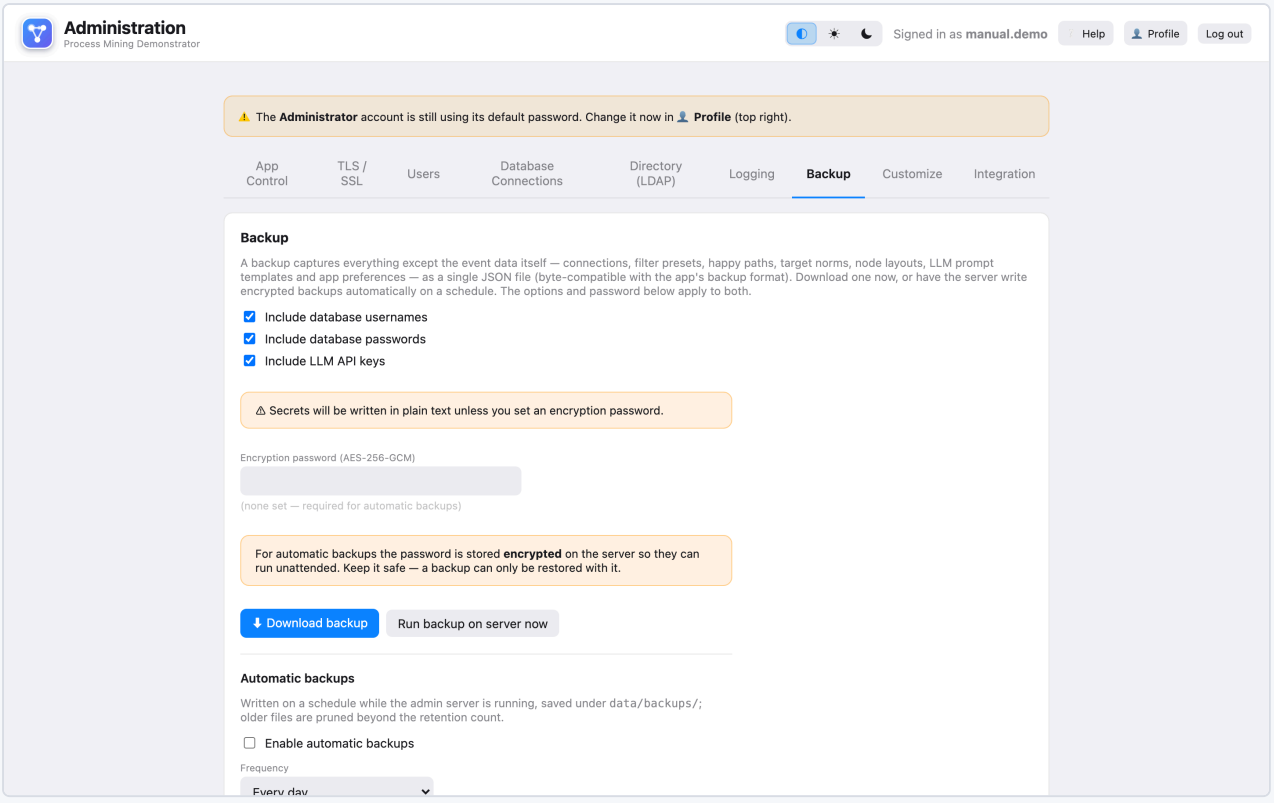


Figure 15.2 — The Backup tab: the backup card with its include options, encryption password and schedule, and the Restore card below it.

What a backup contains

A backup captures **everything except the event data itself**, as a single JSON file that is byte-compatible with the main application's own backup format. Concretely it holds:

- database connections and their server settings;
- filter presets;
- happy paths;
- target norms;
- saved node layouts;
- LLM prompt templates;
- application preferences and settings.

> **Warning** The mined data — projects, journeys, steps, metas and notes — lives in the customer database, not in the product, and is therefore **not** in the backup. A backup restores your configuration onto a working installation; it does not reconstruct an event log. Back the database up separately, by whatever means your database administrator uses.

Include options and the encryption password

Control	Default	Effect
Include database usernames	checked	Writes the connection usernames into the file.
Include database passwords	checked	Writes the stored connection passwords into the file.
Include LLM API keys	checked	Writes the stored LLM API keys into the file.
Encryption password (AES-256-GCM)	none set	Encrypts the whole file. Shows the hint *"(set — leave blank to keep it; also reused for downloads)"* once a password is stored, and *"(none set — required for automatic backups)"* otherwise.

Whenever secrets are included and no password is typed or stored, the card shows the inline warning *"⚠ Secrets will be written in plain text unless you set an encryption password."* A standing note explains the trade-off for unattended operation: for automatic backups the password is stored **encrypted** on the server so the schedule can run without you.

> **Warning** A backup can only be restored with the password it was written with. There is no recovery path. Store the password somewhere independent of the machine that holds the backups — if you lose it, every encrypted backup you hold is unreadable.

Downloading a backup

1. Open the **Backup** tab and set the three include checkboxes as required.
2. Type an **Encryption password**, or leave the field blank to reuse the password already stored on the server.
3. Click ↓ **Download backup**. The file is saved as `ProcessMining-Backup-YYYY-MM-DD.json`.

> **Note** A blank password field is not the same as "no encryption". If an automatic-backup password is stored, downloads silently reuse it and are encrypted too. To produce a genuinely unencrypted file you must have no stored password at all.

Running a backup on the server

1. Set the include options and, if needed, the password.
2. Click **Run backup on server now**.
3. The file is written into `data/backups/` immediately, using the typed password or the stored one, and the toast reports *"Backup written on server: <file>"*.

If neither a typed nor a stored password exists, the run fails with *"Set a backup password first."* Manual server-side runs appear in the status line marked "· manual".

Automatic backups

Automatic backups are written on a schedule while the admin server is running, saved under `data/backups/`, with older files pruned beyond the retention count.

1. Set an **Encryption password** and save it — enabling the schedule without one is refused with HTTP 400 *"Set a backup password before enabling automatic backups."*
2. Tick **Enable automatic backups**.
3. Choose a **Frequency** and fill in the fields it reveals (see the table below).
4. Check the live line — *"Runs: <cron> — <human summary>"*, for example "Every day at 02:00" — to confirm the schedule reads as you intended.
5. Set **Keep newest backups**: the retention count, minimum 1, default **30**.
6. Click **Save schedule**. The toast *"Backup schedule saved"* confirms it; an unparsable cron expression is rejected with 400 *"Invalid schedule: <reason>"*.

Frequency	Fields shown	Notes
Every hour	Minute (0–59)	Fires once an hour at that minute.
Every day	Hour (0–23), Minute (0–59)	Summarised as "Every day at 02:00".
Every week	Weekday (Sunday–Saturday), Hour, Minute	Summarised as "Every Monday at 02:00".
Every month	Day of month (**1–28 only**), Hour, Minute	The range stops at 28 so the schedule is valid in every month, February included.
Custom (cron)	Cron expression	Five fields: minute hour day-of-month month weekday. Placeholder `0 2 * * *`.

The status line under the schedule reports one of: *"Last backup: <when> — <file> (N KB)"*, a red *"Last attempt failed: <error> — <when>"*, or *"No backup has run yet."*

> **Note — how the scheduler actually runs** The scheduler lives **inside the admin server process**. If the admin interface is stopped, no automatic backups are written, and none are caught up afterwards. It wakes every 15 seconds and fires at most once per minute when the cron expression matches the wall clock of the **display timezone** configured under App Control — so "daily at 02:00" means 02:00 in that timezone, and changing the timezone moves the schedule. Files are named `pmw-backup-YYYYMMDD-HHMMSS-<microseconds>.json`; retention pruning deletes the oldest files beyond the keep-count. A scheduled tick that finds no stored password records the failure *"No backup password is set."*

Restoring from a backup

Restore is a three-stage flow: load the file, review what it holds, then choose what to apply. The intro states the consequence plainly — this overwrites the corresponding settings.

1. In the **Restore** card, choose a file with **Backup file** (`.json`). It is inspected automatically as soon as it is selected.

2. If the file is encrypted, type its password into **Password (if the backup is encrypted)** and click **Inspect** again.
3. Read the **Backup contents** summary: *Created*, *Connections: N*, *Projects with settings: N*, an *Includes* line (usernames or —, passwords or "no passwords", API keys or "no API keys") and — most importantly — **Overwrites:** the names of the connections that will be replaced.
4. Untick anything you do not want applied. The options, all checked by default, are: **App settings & preferences**, **Connections & servers**, **Usernames**, **LLM API keys**, **Database passwords**, **Saved node layouts**, **Target norms**, **Happy paths**, **Filter presets**.
5. Click **Restore** (danger) and confirm: *"Restore the selected items from this backup? — This overwrites the current settings."* The toast *"Backup restored"* confirms completion.

> **Warning** Read the **Overwrites:** line before you restore. Those checkboxes are the *only* granularity available — you cannot restore a single connection, or merge one backup's presets into another's. A connection named in that line is replaced wholesale by the version in the file.

Message	Meaning
"This backup is encrypted — enter its password and Inspect again."	The file is AES-256-GCM encrypted; supply the password.
A decryption failure	The password is wrong for this file.
"The backup file format is invalid or corrupted."	Not a readable backup file.
"Backup file is too large."	Above the decoded cap of roughly 18 MB.
"Invalid file content."	The upload could not be decoded.
"Choose a backup file first"	Restore or Inspect was pressed with no file selected.

What backup and restore write to the log

Every action in this tab is tagged **BACKUP/RESTORE**, so filtering the Logging tab by that tag gives you the complete backup history in one view:

Action	Severity	Recorded detail
Export (download or server run)	USAGE	The include-set, and whether the file was **encrypted** or **PLAINTEXT**
Inspect	USAGE	That a backup file was inspected
Restore	**WARN**	The restore options that were selected
Any failure	WARN / ERROR	The reason for the failure

Customize: the login-page background

The **Customize** tab holds branding and appearance settings for the application and the admin interface; its header notes that more options are to come. Today it contains one section, **Login Page**, which sets the background for *both* sign-in pages — the main application's and the admin interface's — from a single choice.

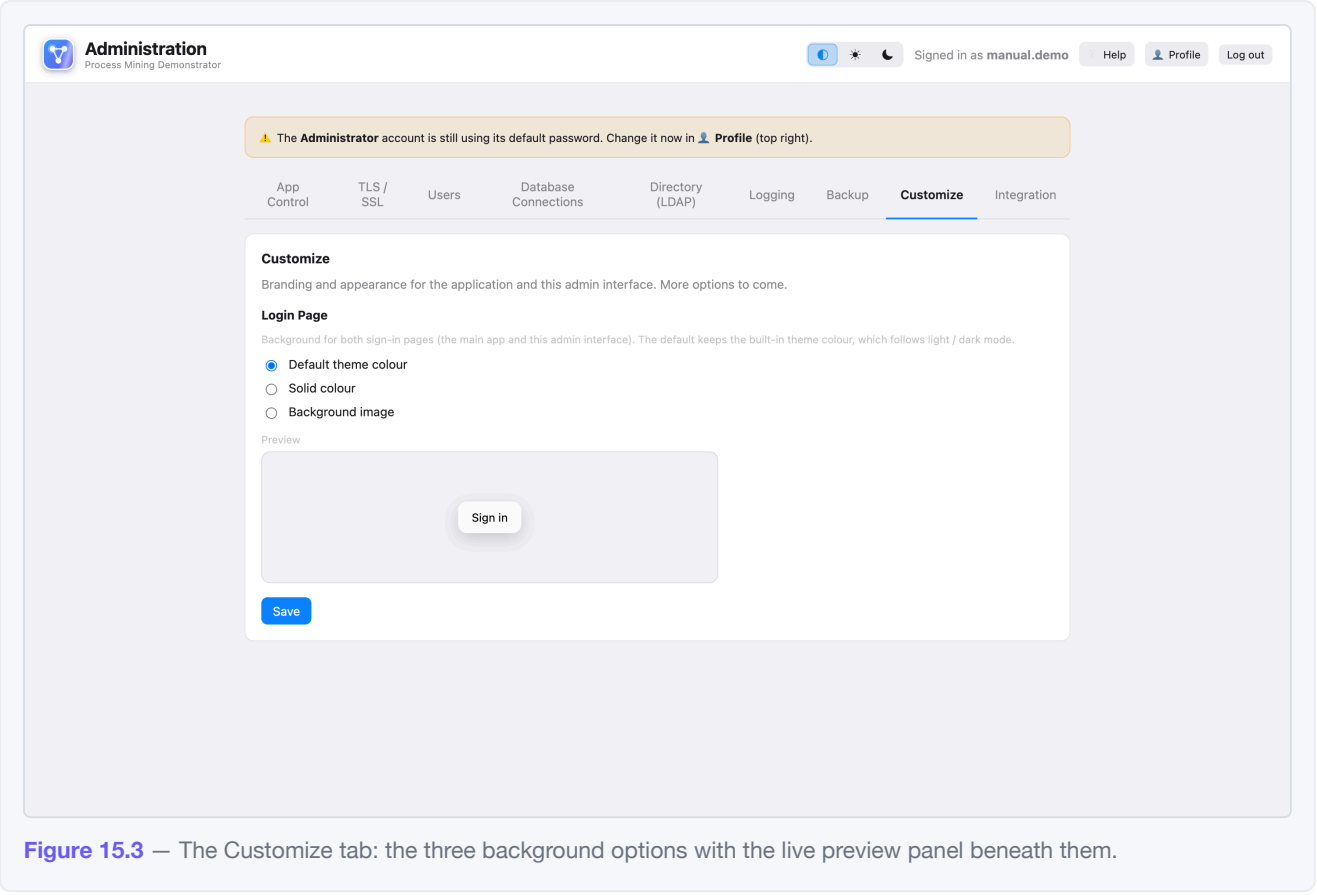


Figure 15.3 — The Customize tab: the three background options with the live preview panel beneath them.

Option	What it does
Default theme colour *(default)*	Keeps the built-in theme colour, which follows the viewer's light / dark mode.
Solid colour	Reveals a colour picker and a hex field that stay in sync. The server validates the value: *"Color must be a #rrggbb hex value"*.
Background image	Reveals a file input accepting **PNG, JPEG, GIF, WebP or SVG** — or you can drag an image straight onto the preview.

1. Open the **Customize** tab and select one of the three radio options.
2. For **Solid colour**, pick a colour or type a `#rrggbb` value.
3. For **Background image**, choose a file or drag one onto the preview. The toast *"Image ready — click Save to store it"* confirms the image has been prepared.
4. Check the **Preview** box, which renders a mock **Sign in** panel over your chosen background.
5. Click **Save**. The result line reads *"Saved — new sign-ins use it immediately."* and a toast confirms "Login background saved". No restart is needed; the next render of either sign-in page picks it up.

Images are processed in the browser before upload. A file over **30 MB** is rejected outright (*“Image file is too large (max 30 MB)”*). Raster images are scaled to a **2560 px** long edge and re-encoded as WebP, with JPEG as a fallback, so an ordinary photograph fits comfortably. **SVG is kept exactly as supplied** and may therefore be animated. The stored image is capped server-side at roughly **3 MB** (*“Image is too large (max ~3 MB)”*). Other validation messages are *“Choose a background image first”* and *“Choose a valid background image first”*.

> **Note** Over a background **image** the sign-in panel becomes 50 % translucent so the picture shows through. A solid colour or the default theme colour keeps the panel opaque. Check legibility in the preview before saving — a busy photograph can make the username and password fields hard to read.

Background changes are logged under operation `customize`, naming the new background as “a solid colour (#…)”, “an uploaded background image” or “the default theme colour”.

Integration: turning the console on and off

The **Integration** tab is the master switch for the integration console — the fourth surface, used to configure data sources. This tab decides whether that surface exists; what you do inside it is covered in its own chapter.

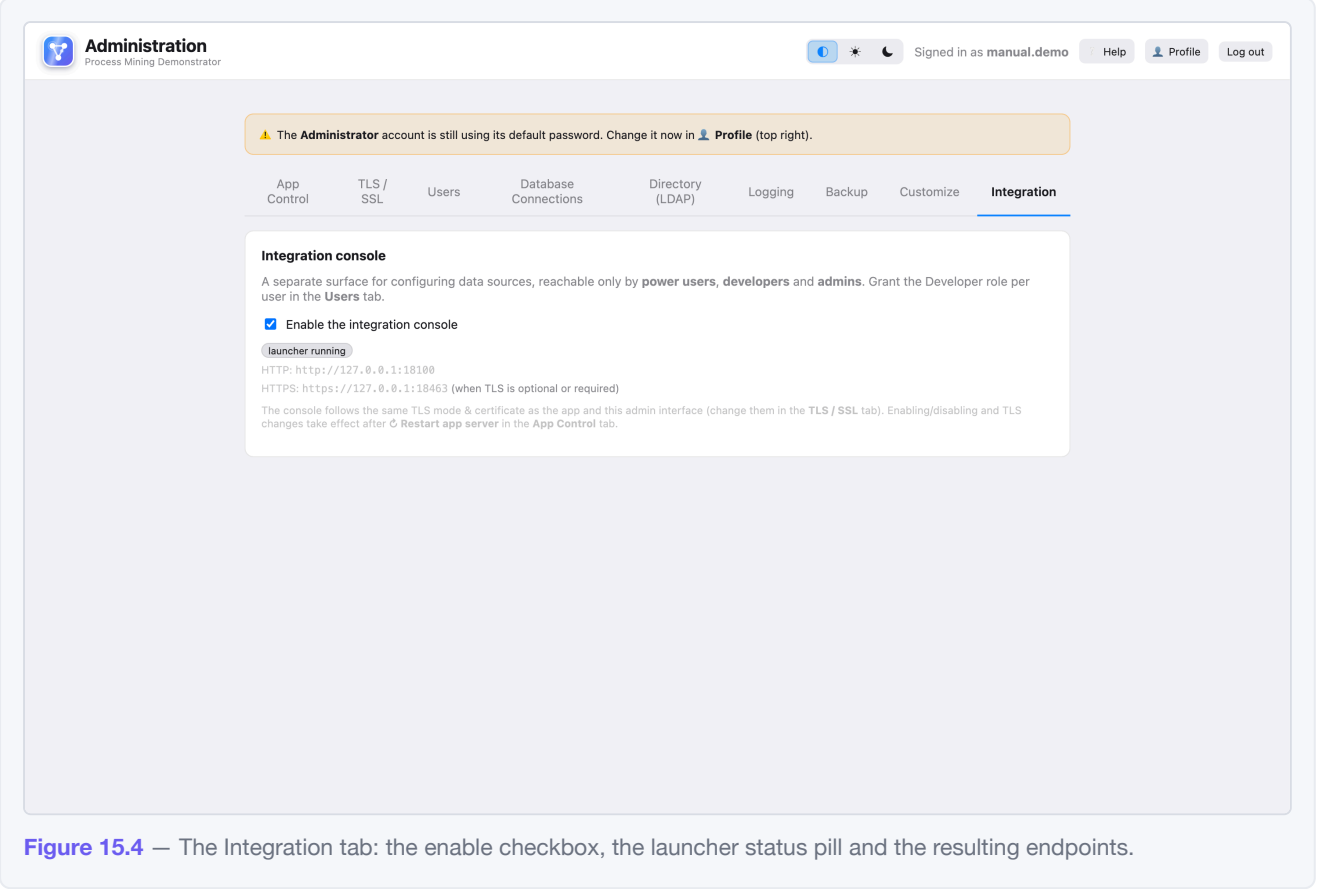


Figure 15.4 — The Integration tab: the enable checkbox, the launcher status pill and the resulting endpoints.

The card describes the console as a separate surface for configuring data sources, reachable only by **power users**, **developers** and **administrators**, and points you to the **Users** tab to grant the Developer role per user.

1. Open the **Integration** tab.

2. Tick or untick **Enable the integration console**. The toast reads *“Integration console enabled — restart to apply”* or *“Integration console disabled — restart to apply”*.
3. Go to the **App Control** tab and click **↻ Restart app server**. Until you do, the setting is stored but has no effect.
4. Return to the **Integration** tab and check the status pill — **launcher running** or **launcher not detected** — and the endpoints listed beneath it.

The status area lists the console's addresses: **HTTP** `http://<host>:8100` and **HTTPS** `https://<host>:8463`, the latter only when the TLS mode is Optional or Required. Both default to the admin ports plus ten and can be overridden by environment variables. The console follows the same TLS mode and shares the same active certificate as the application and the admin interface; both are changed in the **TLS / SSL** tab.

> **Warning — the restart fan-out** There is only one restart button, and it restarts **three** things: the application, then the integration console, then the admin interface itself. Anyone using the application is briefly disconnected. Because the admin interface restarts too, this page may stop responding mid-request — and if you also changed the TLS mode, the admin interface moves between `http://<host>:8090` and `https://<host>:8453`, so you must reconnect on the new address. The launcher status pill is best-effort: “launcher not detected” immediately after a restart may simply mean the process has not written its PID file yet.

> **Tip** Group your changes. TLS mode, certificate activation and the integration on/off switch all wait for the same restart, so make all of them first and restart once, rather than restarting after each.

PART IV

The Integration Console

16 The Integration Console

> The integration console is the fourth surface of the product, alongside the application, the administration interface and the login screen. It is where log files and other raw data are turned into the JOURNEYS events the application analyses. This chapter is your orientation: what the console is for, who is allowed in and how that permission is enforced, how the window is laid out, what each of the three sidebar sections holds, how connections are managed from here, and how to read the **Abstraction layer** status card, the six ingestion KPI tiles and the live pipeline flowchart. Defining source types and running imports have chapters of their own; read this one first so the surface makes sense before you build anything on it.

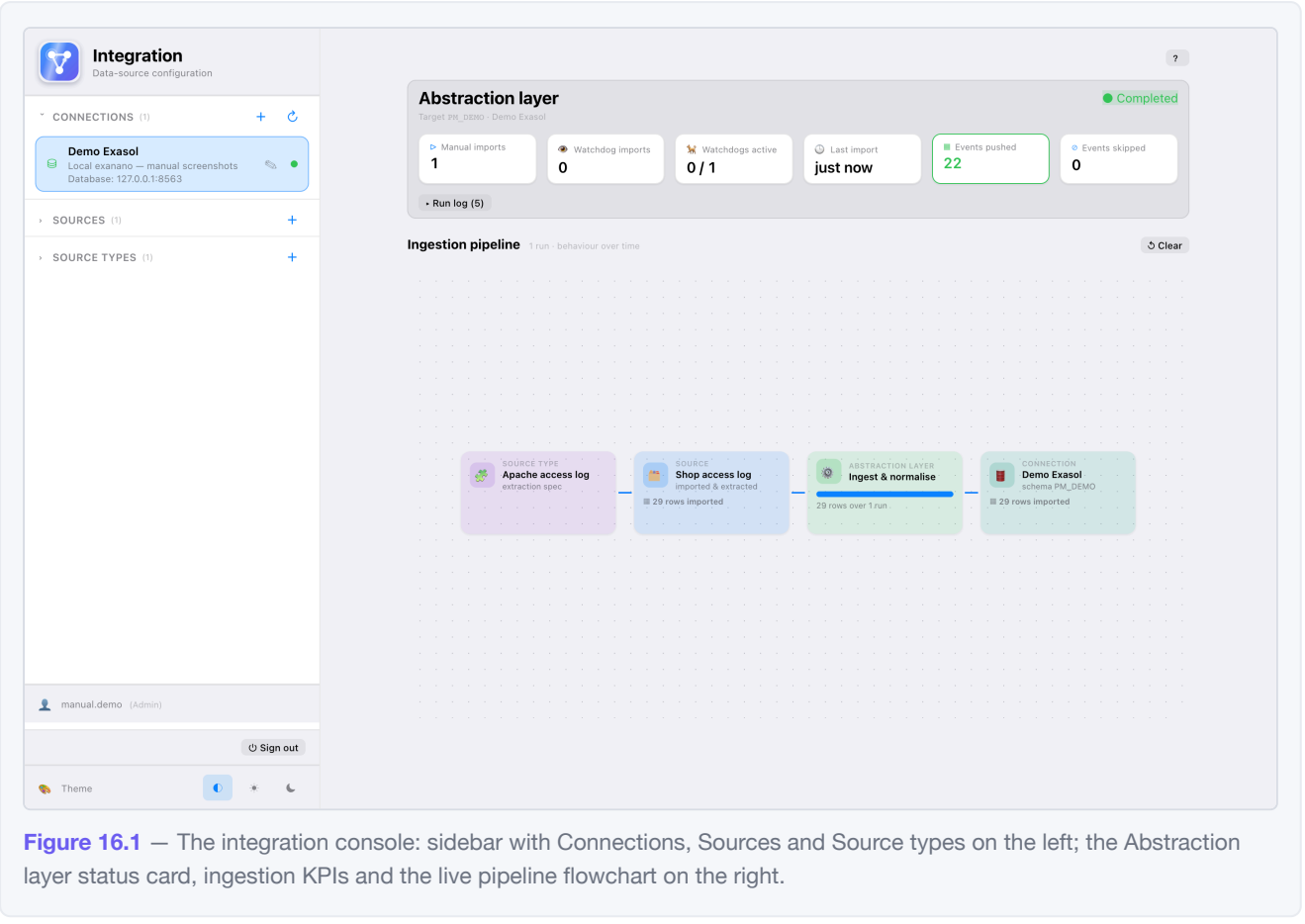


Figure 16.1 — The integration console: sidebar with Connections, Sources and Source types on the left; the Abstraction layer status card, ingestion KPIs and the live pipeline flowchart on the right.

What the console is for

The application reads process data that is already in the database. The integration console is the tool that puts it there. In the console you:

Define a source type

An extraction specification: a pasted sample line from your log plus a regular expression for each field you want out of it — timestamp, case id, step, up to three business attributes and any number of helper values used only for matching. A source type is the parser.

Define a source

Where the data actually comes from — today a file on the server, addressed by path — together with its encoding, the source type applied to each line, the transaction bracket and, optionally, a watchdog that imports newly appended lines by itself.

Run an import

Point a source at one of your assigned connections and a project id, and the abstraction layer reads the file, applies the source type and writes JOURNEYS rows — creating the PROJECTS, STEPS and METAS companion rows for you.

Watch what happened

The status card, the KPI tiles and the pipeline flowchart show the run in flight and, over time, the accumulated behaviour of every source, source type and destination you have used.

Everything you create in the console is **owned by you**. Your source types and sources are yours alone; another developer signing in to the same deployment sees their own, not yours.

Access, sign-in and roles

> Requires: **Developer** or **Administrator**

The console is served on its own port, by convention the administration port plus ten: **8100** for HTTP and **8463** for HTTPS with the default port assignment. It follows the same TLS plan as the application and the administration interface. In Docker the published ports are offset by 2000.

Setting	Default	Purpose
`PMW_INTEGRATION_PORT`	administration port + 10 (8100)	HTTP port of the console
`PMW_INTEGRATION_HTTPS_PORT`	8463	HTTPS port of the console
`PMW_INTEGRATION_HOST`	the administration host (127.0.0.1)	Interface the console binds to

Who may sign in

Only users holding the **Developer** role or the **Admin** role may use the console. Power users and plain users are refused: the server declines the sign-in and drops the session. This is deliberate — the data-source surface writes directly into the analysis database, so it is restricted more tightly than the application itself, where power users have wide privileges.

Sign-in is always required on this surface, even when the deployment's global “require login” setting is switched off for the application. The only exception is a deployment that runs entirely without sign-in (single-user or development mode, where no user identity exists at all); there the surface is open, in step with every other part of the product.

1. Open the console's address in your browser — for a default local installation `http://localhost:8100`, or the HTTPS address your administrator gave you.
2. While the session check runs the console shows a large spinner with the text **Starting…**.

3. The shared login screen appears, with the subtitle **Integration** under the product logo so you can tell at a glance which surface you are signing in to. Enter your credentials.
4. If your deployment mandates two-factor authentication and you have not yet enrolled, the login screen stays up until enrolment is complete — exactly as in the application.
5. The console loads, applies your personal theme setting and reads your assigned data-source connections.

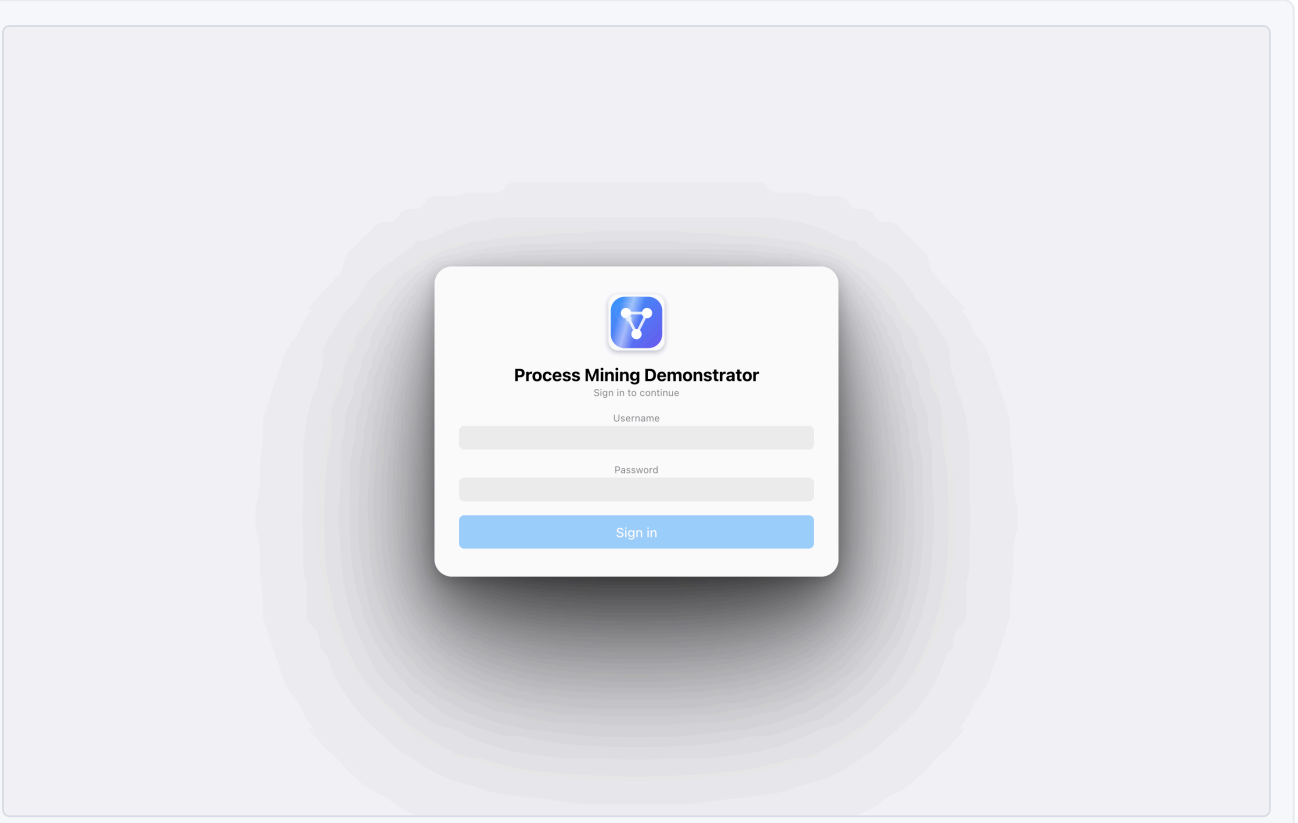


Figure 16.2 — The shared login screen. On the integration console it carries the subtitle “Integration” under the logo.

> **Note — idle sign-out** If your administrator has set an idle timeout (admin **Users** tab), the console signs you out after that many minutes of inactivity and returns you to this login screen with the notice “You were signed out due to inactivity”, exactly as the main application does. Because the console is always role-gated, idle sign-out applies here even when the global “require sign-in” setting is off.

Why the API re-checks your role

The console's own port is only the first line of defence. All integration endpoints live under `/api/integration` on the shared compute backend, and the application proxies `/api/*` for any enabled user. If the role were only checked at the console's front door, a power user could drive the whole data-source surface through the application's proxy instead. To close that, the role check is repeated **server-side on the API router**, so it covers every integration endpoint — including any added in a later release.

Situation	Result
The deployment runs without sign-in (no user identity)	Open, no restriction
The administrator has switched the integration surface off	HTTP 403 — *“The integration console is turned off.”*
The user is unknown, disabled, or lacks the Developer/Admin role	HTTP 403 — *“You need the Developer role to use the integration features.”*

Note Because the switch is enforced on the API and not only on the port, turning the integration surface off in the administration interface disables the entire integration API as well. The switch is on by default.

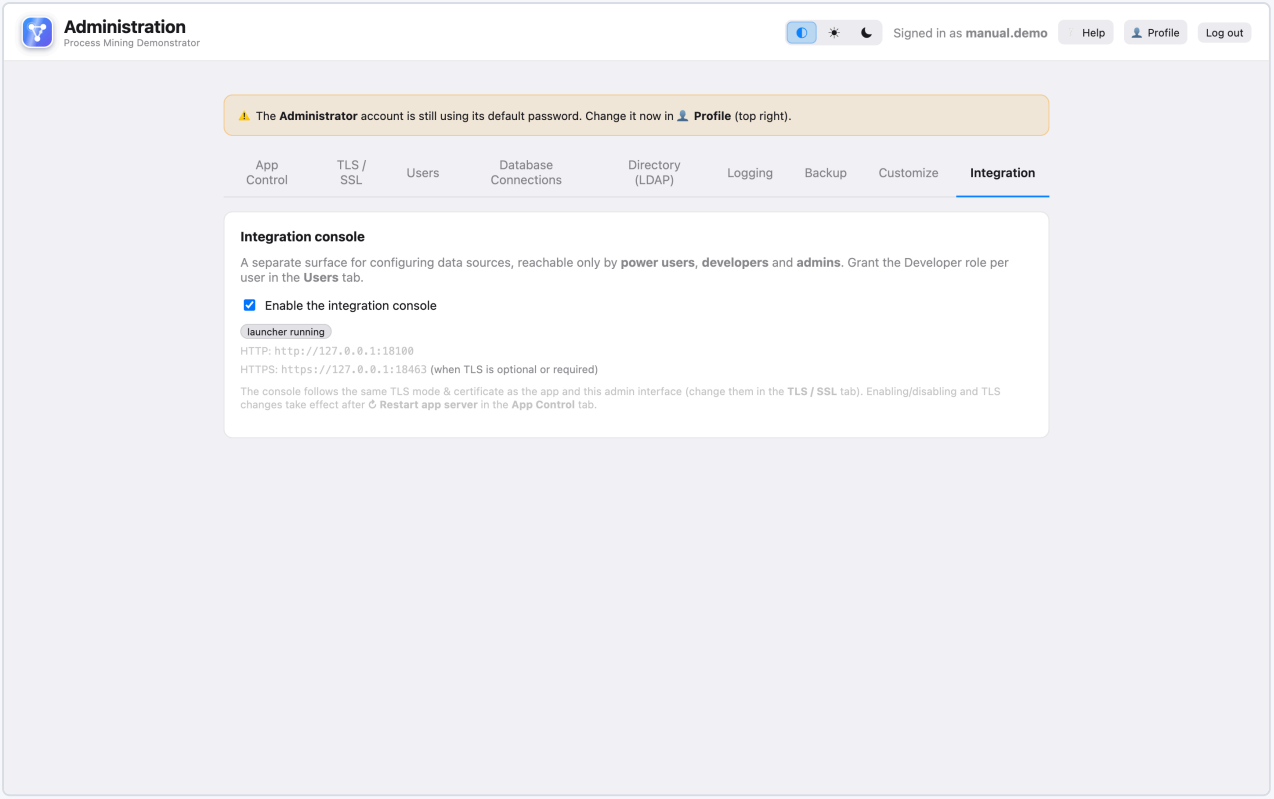


Figure 16.3 — The administration interface holds the deployment-wide switch that enables or disables the integration surface.

If your role is revoked while you are signed in

The console does not silently degrade. If an administrator removes your Developer role during your session, the console replaces its whole window with an **Access denied** page: the product logo, the heading “Access denied”, the text *“The integration console is available to developers and administrators only. Ask an administrator to grant you the Developer role.”* and a ⏻ **Sign out** button.

The console layout

The window is divided the same way as the application: a sidebar on the left, a main pane on the right. The main column is centred and no wider than 900 px, so the pipeline and status card stay readable on a large

screen.

The sidebar

From top to bottom the sidebar contains:

1. The brand header — the logo, the title **Integration** and the caption **Data-source configuration**.
2. Three collapsible accordion sections: **Connections**, **Sources** and **Source types**. At most one is open at a time, and clicking the header of the open section closes it. Each header shows a chevron, the title, the item count in parentheses when it is greater than zero, and one or two trailing icon buttons.
3. Pinned at the bottom, the shared identity footer: a first row reading 👤 **<your display name> (<role>)** — Admin, Power user, Developer or User — and a second row of buttons, 🔒 **Two-factor** and 🔑 **Passkeys** where your deployment allows them, plus ⏻ **Sign out**.
4. Below the footer, the theme bar with the light, dark and system switcher.

The console follows your personal `app.theme` setting, shared with the application: light, dark or system. “System” tracks your operating system's colour scheme live. Until your settings have loaded after sign-in, the operating system's scheme is used.

The main pane

The main pane holds four things, in this order:

The toolbar

A single **?** Help button in the top right, opening the same Help panel as the application.

The Abstraction layer card

Status pill, target line, the six KPI tiles and — after a run that produced messages — the run log. See the status panel below.

The Ingestion pipeline header row

The caption showing how many runs have been recorded, and the ↺ **Clear** button.

The pipeline canvas

The live flowchart of your ingestion behaviour.

The three sidebar sections

Connections

The **Connections** section lists the database connections *assigned to you*, as clickable badges sorted alphabetically by name. Each badge shows a ⛁ icon — green when that connection is connected, accent blue otherwise — the connection name (or “(unnamed)”), the optional comment, a line reading **Database: <host>:<port>** (“(no host)” when the host is empty) and, when an LLM is configured, **LLM: <url>** or “(configured)”. About three badges are visible at once; the list scrolls beyond that.

The header carries two buttons: **+** (“New connection”), shown only to users who may manage connections, and ↻ (“Refresh connections”), which reloads the assigned list.

If you have no connections, the empty state depends on your permissions: managers see *“No connections yet. Use + above to create one and assign users.”*, everyone else sees *“No connections assigned to you. Ask an administrator to grant access.”* Any connection error is shown in red under the list.

Sources

The **Sources** section lists your data sources, with **+** (“Add a source”) in the header. Each badge shows the kind icon (for a File source), the source name, and a sub-line with the kind label and a summary of its configuration — for a File source, its path, or “(no path)” when none is set. A eye after the name means the watchdog is switched on for that source (tooltip “Watchdog on — auto-imports new lines”). The ▷ button opens the Run dialog and the ✕ button deletes the source after a confirmation; clicking anywhere else on the badge opens the edit wizard. Sources, the wizard and the Run dialog are covered in their own chapter.

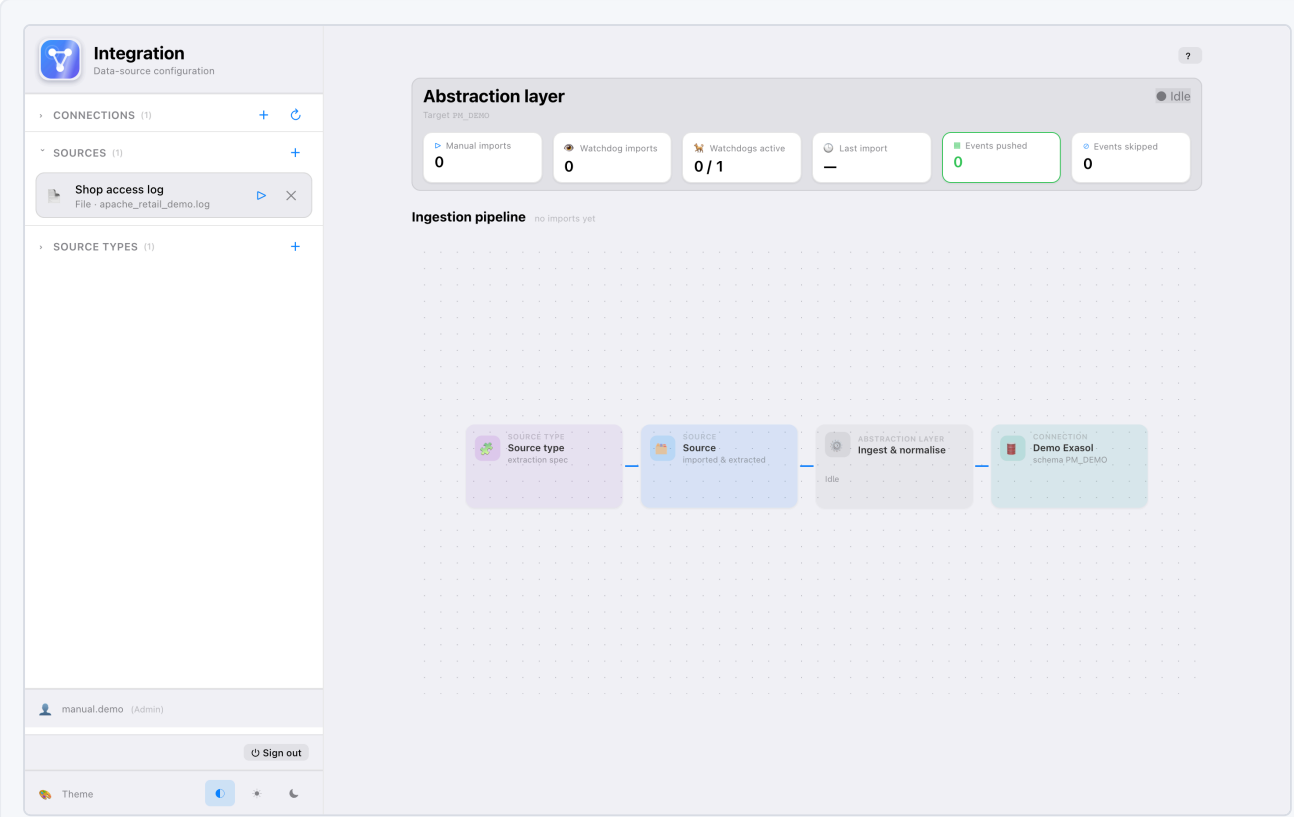


Figure 16.4 — The Sources section, with a watchdog-enabled source and its Run and Delete buttons.

Source types

The **Source types** section lists your extraction specifications, with **+** (“Add a source type”) in the header. Each badge shows a icon, the name and a sub-line reading “<n> field(s)” or “no fields yet”. Clicking a badge opens the source-type wizard; ✕ deletes it after a confirmation. The wizard — sample line, field mapping, compound steps and the example JOURNEYS record — has its own chapter.

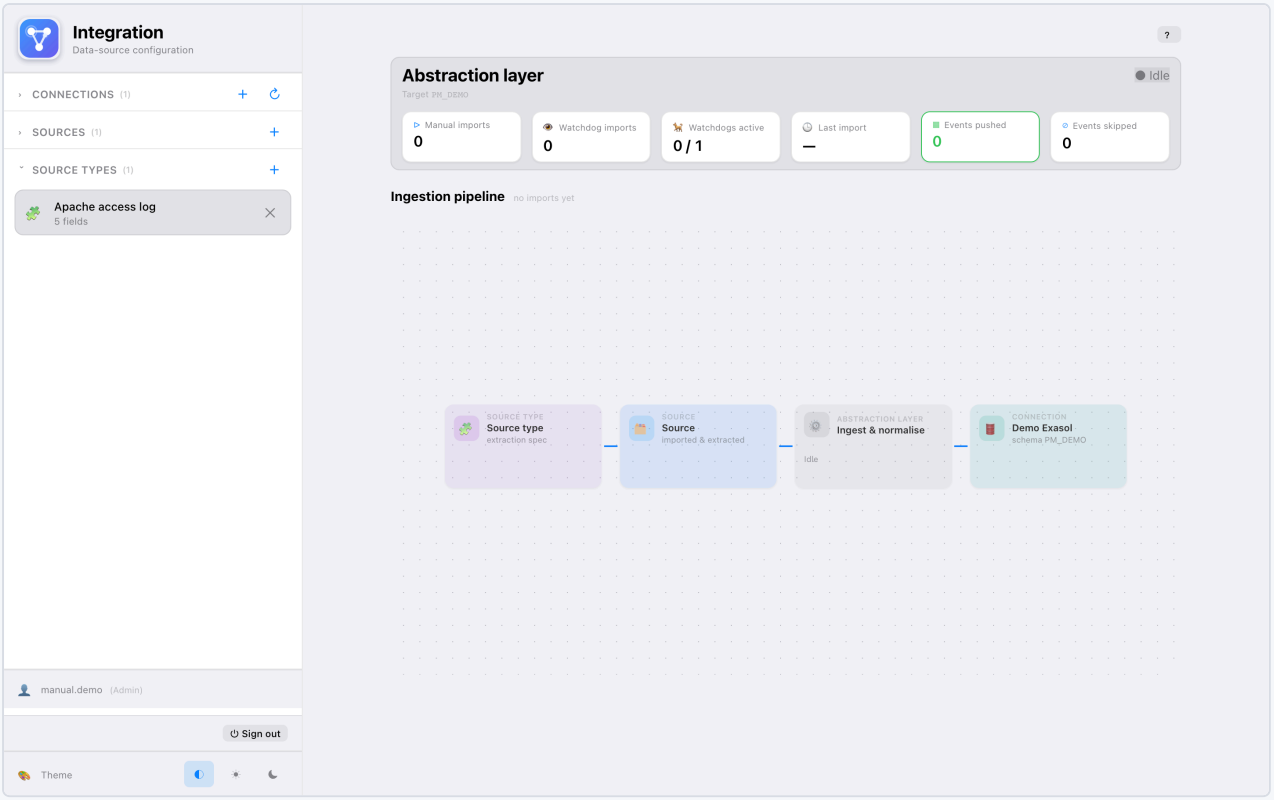


Figure 16.5 — The Source types section. Each badge reports how many extraction fields the type defines.

> **Warning** Deleting a source type does not cascade. Sources that reference it keep a dangling link and their runs fail with the warning “This source has no source type linked” until you edit the source and pick a new one.

Managing connections from the console

> **Requires: ownership of the connection (Power user, Administrator or Developer role to create one)**

You do not have to switch to the application to set up a destination database. The console embeds the same connection editor. What differs from the application is not the editor but the permission model, and the distinction is easy to trip over:

Assignment

An *assigned* connection is one you may **use**: connect to it from the sidebar, run an import into it, and name it as a watchdog destination. Assignment is granted by whoever owns the connection.

Ownership

An *owned* connection is one you may **edit**. Only owned connections show the ✎ pencil on their badge.

A connection that an administrator created and assigned to you is therefore read-only for you: you can import into it all day, but you cannot change its host, credentials or user assignments. This is why the pencil is missing from some badges and present on others.

Connecting and disconnecting

1. Open the **Connections** section in the sidebar.
2. Click a connection badge to connect to it. The badge shows a status dot: green for **Connected**, orange for **Not connected**.
3. Click the active badge again to disconnect.
4. Clicking a different badge while one is active disconnects the current connection first, then connects the new one.

> **Note** You do not need to be connected in order to import. The Run dialog and the watchdog open the destination themselves using the connection's stored credentials. Connecting from the sidebar is what sets the *default* target shown in the status card and preselects a destination in the Run dialog.

Creating a connection

1. Click **+** in the **Connections** header. The **New Connection** editor opens as a wide sheet with the ⛁ icon.
2. On the **Database / LLM Details** tab, fill in at least a **Name** — leaving it blank fails with “Name is required.” — and the **Host**, **Port** (default 8563), **Username**, **Schema** and **Password**.
3. Optionally switch on **Use TLS** and choose a **Certificate mode**: “Verify (system trust store)” (the default), “Pin fingerprint” or “Accept any (insecure)”. A pinned fingerprint goes in **Fingerprint (SHA-256)**; **Minimum RSA key size** defaults to 2048.
4. Optionally configure an LLM: **Server URL**, **Model** and **API key**.
5. Tick the usernames that should be able to use this connection under **Assigned users**.
6. Click **Test** to verify. The result appears inline — green when everything answered, orange when one component failed, red when all did, with text such as “Database OK · LLM OK (n models)”.
7. Click **Save**. The new connection appears in the sidebar and, being owned by you, carries the ✎ pencil.

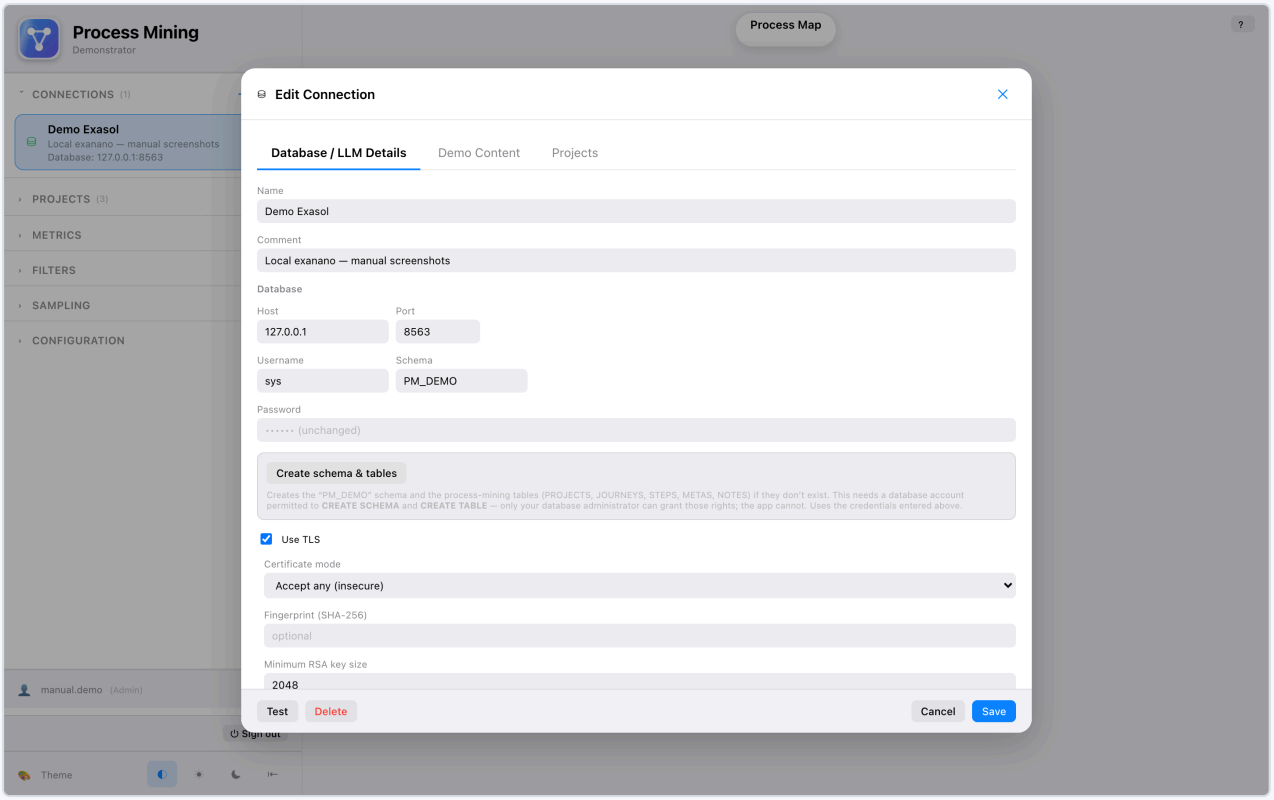


Figure 16.6 — The connection editor, shared between the application and the integration console.

Editing an owned connection

1. Hover the badge of a connection you own and click the ✎ pencil. The click does not connect or disconnect — it only opens the **Edit Connection** editor.
2. Change what you need. A stored password shows the placeholder **•••••• (unchanged)**; leave it blank to keep the stored value.
3. Use **Create schema & tables** if the target schema does not exist yet. The button stays disabled until a schema name is entered, and it needs a database account with CREATE SCHEMA and CREATE TABLE rights — only your DBA can grant those.
4. Click **Save**, or **Delete** to remove the connection. Deletion asks to confirm: *"<name> will be removed for all assigned users. This cannot be undone."*

The editor's other two tabs are equally available here: **Demo Content**, which generates a Retail, Finance/Insurance or Transportation demonstration data set into a schema (journey count default 500, range 1–20 000), and **Projects**, which lists the projects in the connection's schema with their journey and event counts and lets you delete one. Both are described in full in the chapter on connections and projects.

The Abstraction layer status panel

The status card is the console's live readout of the extraction engine. It is driven by a single poll of the status endpoint every **1.5 seconds**, and re-polls immediately whenever you change the active connection.

The card is titled **Abstraction layer** and carries a state pill on the right:

Pill	Colour	Meaning
• Idle	Grey	Nothing has run yet in this session
Running	Accent blue	An extraction is in progress
Completed	Green	The most recent run finished successfully
Failed	Red	The most recent run failed; the message is shown below

Under the title, the **target line** tells you where an import would land: **Target <schema> · <connection name>** when you are connected and the connection has a schema; *“The active connection has no target schema”* when it does not; and *“Pick a connection on the left to choose the target”* when you are not connected at all.

Two warning banners can appear: the poll error, if the status endpoint cannot be reached, and **Last run failed: <error>** after a failure.

When the most recent run produced log messages, a toggle button ▸ **Run log (<n>)** appears below the KPI strip. Expanding it shows a scrollable monospace list of that run's messages, for example:

```
reading /data/logs/access.log → SALES.JOURNEYS (project LIVE-LOG)
pushed 500 events, skipped 2 unparseable line(s)
created project 'LIVE-LOG'
watchdog: 14 new line(s) from /data/logs/access.log
```

Note The status is per user and always reflects the *most recent* run only — a new run resets its counters. Only one extraction runs per user at a time; a second one started while the first is in flight fails with “An extraction is already running for this user.” Different users can run concurrently.

The ingestion KPI tiles

Six tiles sit inside the status card, using the same visual language as the application's KPI strip. They wrap onto further rows in a narrow window.

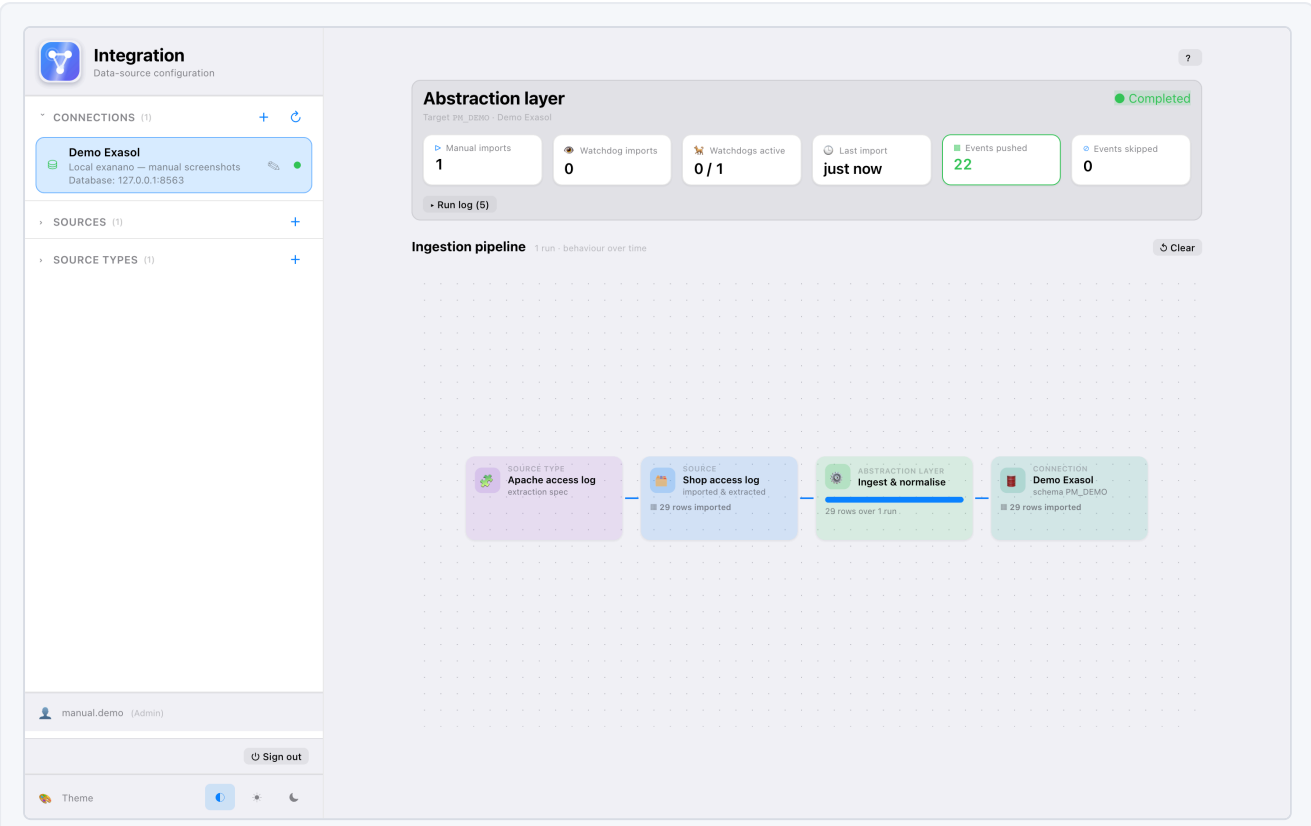


Figure 16.7 — The ingestion KPI tiles — Manual imports, Watchdog imports, Watchdogs active, Last import, Events pushed and Events skipped — above the live pipeline flowchart, shown here after an import has run.

Tile	Icon	What it shows
Manual imports	▷	How many recorded runs you triggered yourself, from the Run dialog.
Watchdog imports	👁	How many recorded runs the watchdog started on its own.
Watchdogs active	🐕	A live “<active> / <total>”: your file sources with the watchdog switched on, out of all your file sources. Green when at least one is active, “—” until a status has loaded.
Last import	🕒	The age of the newest run: “just now” under 45 seconds, then “Nm ago”, “Nh ago”, “Nd ago”; “—” when there are none. While a run is in flight it reads **running…** in the accent colour.
Events pushed	▦	Total journey events written across the recorded history. Always green.
Events skipped	⊘	Total source lines that could not be parsed across the history. **Orange whenever it is greater than zero.**

The split between **Manual imports** and **Watchdog imports** exists because both write into the same tables through the same engine; without the split you could not tell an unattended import from one you started.

> **Tip** Treat **Events skipped** as your correctness signal. A run that reports thousands of events written and thousands skipped almost always means the linked source type's regular expressions only fit part of the log. Open the source type and test its fields against a sample line from the file that is actually being read.

Warning If your deployment has switched the watchdog loop off entirely, the third tile's label changes to **Watchdogs (off)** and the tile turns orange. A source whose watchdog is "on" then never polls. The tile is deliberately loud about this, because the alternative is a count that quietly does nothing.

Note Every tile except **Watchdogs active** is computed from the console's own recorded run history, not from the backend. They therefore survive a backend restart, and they are reset by **↺ Clear** — see Clearing the history.

The ingestion pipeline flowchart

The canvas below the status card draws your whole ingestion behaviour as one accumulating graph, in four columns:

```
[Source type] ⟶ [Source] ⟶ [Abstraction layer] ⟶ [Connection]
```

Nodes accumulate across runs

This is not a picture of the last run. Nodes are **reused**: there is one node per distinct source-type name, one per source name and one per destination connection name, with the **Abstraction layer** as the single hub in the middle. Edges are drawn for whatever combinations have actually run, so several sources, types and destinations fan in and out of the hub. Over a few days the chart becomes a portrait of how data really reaches your database — which is exactly what it is for.

Every run that has actually started is recorded, whichever way it was triggered: a manual run from the Run dialog, a watchdog poll, or a push through the API. The entry for a run in flight updates in place as the poll progresses.

Node	Icon	Kicker	Subtitle
Source type	🧩	SOURCE TYPE	extraction spec
Source	📦	SOURCE	imported & extracted
Abstraction layer	⚙	ABSTRACTION LAYER	progress bar and state label
Connection	🛢	CONNECTION	schema <schema>, or "destination database"

Row counts

The **Source** and **Connection** nodes each carry a line reading ▦ **<n> rows imported**. This is the cumulative total across every recorded run that touched that node — not the last run's figure. Reading across the chart therefore tells you how much each file has contributed and how much has landed in each destination database. Each **Source** node also shows 🕒 **<date, time>** — the moment of its *most recent* import, in your local time — so you can see at a glance when each file was last loaded.

The hub itself shows either a live state — "Running", with real progress once the totals are known and an indeterminate animation before that — or, between runs, **<total> rows over <n> run(s)**. With an empty history it simply reads "Idle".

Colour and state

Each node is tinted with a semi-transparent wash of its own accent colour — purple for a source type, accent blue for a source, the current phase colour for the layer, teal for a connection. The wash is 14 % opaque in light mode and 20 % in dark, so the canvas dot-grid shows through and the chart stays pastel rather than solid.

State	Appearance
Running	The wash deepens to 24 % (30 % in dark mode) and the node gains an accent ring
Completed	Green-tinted border
Failed	Red border
Idle	Reduced opacity

Edges take their colour from their own most recent run: green while it is running, red after a failure, blue otherwise. While a run is live, a single animated dot travels the run's path node by node in order — type, source, layer, connection — taking roughly 0.9 seconds per edge and looping until the run ends. Because edges remember their last outcome individually, a red edge tells you precisely which combination failed even when a later run of a different source succeeded.

Before anything has run, the canvas shows a dim idle skeleton — a generic “Source type” feeding a generic “Source”, the layer, and either the connection you are currently connected to or “(no connection)” — so the pipeline's shape stays visible from the first visit.

Moving, resizing and resetting the canvas

The automatic arrangement is a starting point, not a constraint. You can lay the graph out to match how you think about your data flows.

1. **Drag a node** to move it. The position is saved for you personally and survives polls, page reloads and backend restarts.
2. **Drag the background** to pan the canvas. Zoom is limited to between 0.3× and 1.5×, and scroll-wheel zoom is switched off so that scrolling the page over the canvas behaves normally. Nodes cannot be selected or wired together — the graph is a readout, not an editor.
3. **Drag the grip strip along the bottom edge** of the canvas (tooltip “Drag to resize the canvas”) to change its height, anywhere between **520 px and 1400 px**. The height is saved for you as well.
4. Click ⤢ **Reset layout** in the top right to discard both the saved positions and the saved height and return to the automatic arrangement. The button appears only once you have moved a node or resized the canvas.

> **Note** The canvas also grows by itself as the number of node rows increases, and a height you dragged is never allowed to shrink below what the current rows need — so adding a fifth source cannot push nodes out of sight.

The run history and clearing it

The live status only ever describes the most recent run, so the console keeps its own list, appending each run as it starts. The header above the canvas reads **Ingestion pipeline** with the caption *“no imports yet”* or

“<n> run(s) · behaviour over time”. The list is capped at **50 runs**; older entries drop off and the caption then reads “50+ runs”.

1. Click ↺ **Clear** next to the caption (tooltip “Clear the ingestion history”). The button appears only when runs have been recorded.
2. The confirmation dialog **Clear ingestion history** asks: *“Reset the pipeline flowchart, removing all <n> recorded run(s)? This only clears the console view — nothing already imported is affected.”*
3. Click **Clear history**. The flowchart returns to its idle skeleton and all KPI tiles except **Watchdogs active** return to zero.

> **Warning** The run history, the node layout and the canvas height are stored in your browser's local storage, per user and therefore per browser profile. Signing in from a different machine shows that machine's history, which may be empty; a browser with storage disabled keeps nothing at all. None of this affects the database — clearing the history deletes no imported data, and losing the history loses no rows.

How the pieces work together

A run started in the Run dialog is the same run the status card reports, the same run that lights up the pipeline, and the same run the KPI tiles count — they are three views of one live status poll, updating together in real time. That is worth knowing when something looks wrong: if the Run dialog shows progress but the pipeline does not, the problem is the browser, not the import; if the status pill says **Failed** and an edge on the canvas has turned red, both are telling you about the same failure, and the message is on the status card and in the Run dialog's checkpoint panel.

With the surface understood, the remaining integration chapters build on it in the order you will use them: define a source type, define a source that uses it, then run the import and — where the data keeps arriving — hand the job to the watchdog.

17 Source Types: Teaching the System to Read a Log

> A *source type* is the parser you hand to the integration console: an example record plus a set of named selectors that pull the case id, the process step, the event time and up to three business attributes out of every record of a file. It carries a *data format* that decides *how* fields are located — a **regular expression** per line for unstructured **text** logs, or a **path selector** per field for semi-structured **JSON** and **XML**. This chapter explains the concept, walks through all three steps of the source-type wizard, documents every field role and the database column it ends up in, covers regex authoring with the live highlighter *and* the JSON/XML path pickers, timestamp format analysis, META business titles, the Step tab and the automatic STEPS rows, field locking and cascade rename, and helper fields that are deliberately never written to the database. It closes with a complete worked example on an Apache access log. It is written for developers and administrators who prepare data sources; ordinary users never see these screens.

> Requires: **Developer** or **Administrator** — source types live in the integration console (HTTP :8100, HTTPS :8463). Sign-in is always required on this surface, and power users and plain users are refused, both at the console login and at every `/api/integration/*` endpoint ("You need the Developer role to use the integration features.").

17.1 What a source type is

The integration console separates *where* data comes from (a **source** — chapter 18) from *how it is read* (a **source type**). One source type can be linked to many sources, and a source without a linked source type cannot be run at all.

A source type consists of exactly five things:

A name
: 1–120 characters, shown in the sidebar and in the source wizard's **Source type** dropdown.

A data format
: `text` (the default, and what every pre-existing source type is treated as), `json` or `xml`. It decides whether fields are located by a regex or by a path, and — for XML — carries the *record element* (the repeating element that is one record).

A sample record
: One representative record — a log line, one JSON object, or one XML element — up to 20,000 characters. It is stored with the source type and is the canvas on which you map every field; it is never imported.

An extraction spec
: An ordered list of fields, each with a name, a role, a selector (a **regex** for text, a **path** for JSON/XML) and — for timestamps — an inferred parse format.

Optional compound step rules

Rules that build the step name from more than one field. Up to 200 rules. They match on the extracted field *values*, so they work for **every format** — text, JSON and XML.

Source types are owned per user: you see and edit only your own. Everything you build here is applied line by line at import time — the extractor compiles each regex once per run and runs it against every line of the file.

> **Note** Every field is defined by you — either by highlighting a segment of a text sample and letting the console generalise it into a regex (or typing the regex), or, for JSON/XML, by clicking a field in the record to assign its path (or typing the path). For semi-structured files the wizard *suggests* a starting set of fields with their paths (guessing id/step/timestamp from the key or element names), which you then confirm or adjust — nothing is imported until you save the spec you can see.

17.2 The Source types section

The sidebar's third accordion section is **Source types**, with a count in parentheses once at least one exists. Each entry is a badge showing a 🧩 icon, the name, and a sub-line reading "*n* field(s)" or "no fields yet". About three badges are visible at a time; the list scrolls beyond that. When you have none, the section reads "No source types yet. Use + to define one."

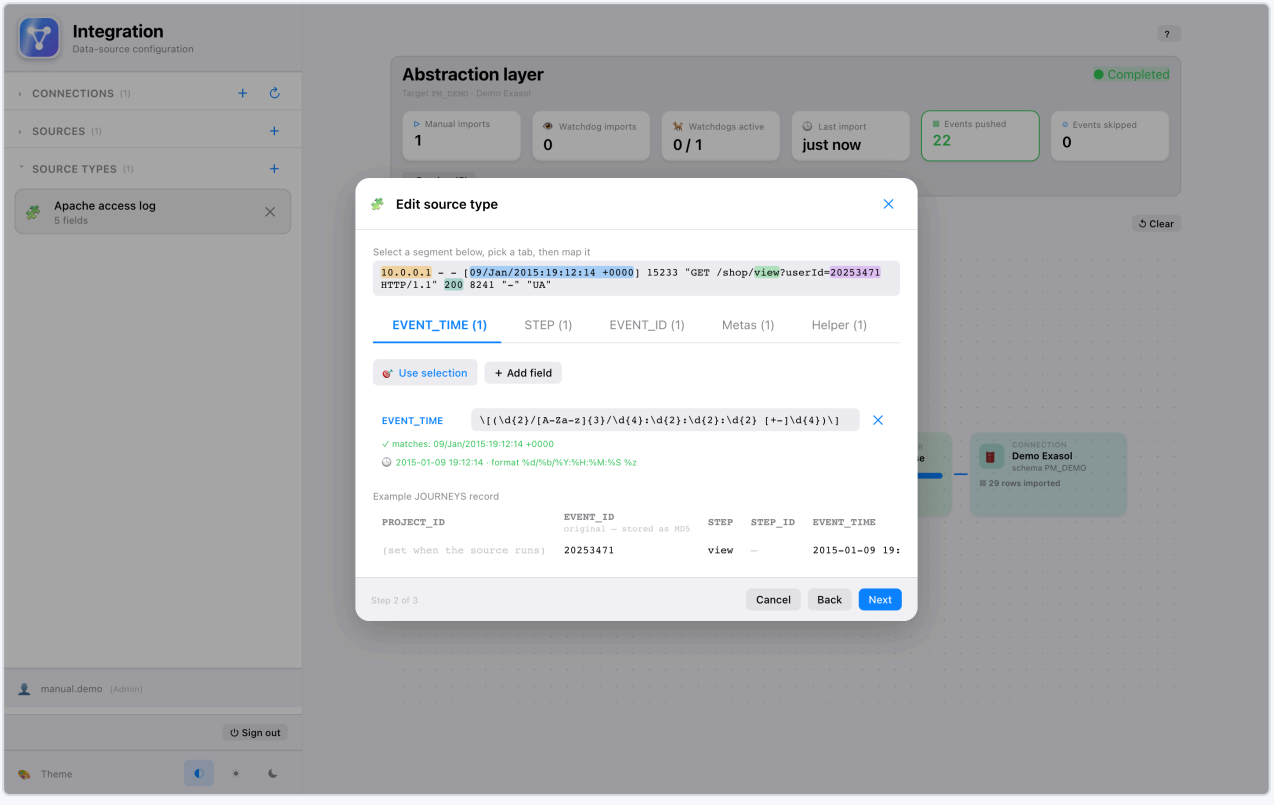


Figure 17.1 — The source-type wizard, step 2: each field is mapped to a role (id, step, timestamp, meta or helper) with its own regular expression. The highlighter shows what each expression captures against the sample line.

1. Sign in to the integration console and open the **Source types** section in the sidebar. Only one accordion section is open at a time; clicking an open header closes it.

2. Click **+** (“Add a source type”) to create one, or click anywhere on an existing badge (“Edit source type”) to reopen it.
3. To delete, click the ✕ on the badge. The confirmation dialog is titled **Delete source type** and reads “Delete the source type “<name>”? Sources that reference it will need a new one.” Confirm with **Delete source type**.

> **Warning** Deleting a source type does **not** cascade. Sources that referenced it keep a dangling link: their Run dialog shows “This source has no source type linked — edit it and pick one so the lines can be parsed.” and the ▷ **Run extraction** button is disabled until you pick a replacement. Watchdog polls record the checkpoint error “The source has no source type linked.”

17.3 The wizard at a glance

The wizard opens as a sheet titled **New source type** or **Edit source type** with the 🧩 icon. It has three steps and a footer that always shows **Step N of 3**, **Cancel**, **Back** (from step 2 onwards), **Next** (disabled on step 1 until a name is entered) and, on step 3, **Create source type** or **Save changes** (“Saving…” while the save is in flight).

Step	What you do
1 — Basics	Name the source type and paste one example log entry.
2 — Field mapping	Map every field on the sample, review the live matches, add compound step rules.
3 — Review	Check the highlighted sample, the field list, the compound summary and the example JOURNEYS record, then save.

Saving re-validates the name. If it is blank, the wizard returns you to step 1 with the error “A name is required.”

17.4 Step 1 — Basics

1. Type a **Name** (the field is autofocused; placeholder “e.g. Apache access log, App JSON log”). Use the log format's own name rather than the system it came from, because one source type is normally reused across several files.
2. Choose the **Data format** — *Text — unstructured (regex)*, *JSON — array or JSONL (paths)*, or *XML (paths)*. Loading a file (below) sets this for you from what the file actually is; the selector lets you override or set it when you paste a record by hand.
3. Give the type an example record. Either load it from a file (recommended — see below) or paste one into the monospace text area — labelled **…or paste an example log entry** for text and **…or paste one example record** for JSON/XML, with a matching placeholder.
4. Click **Next**.

Loading an example from a file

Under **Load an example from a file**, press **Choose a file…**. The wizard lists the files in the sandboxed sources directory (the same directory a File source reads from — chapter 19); nested files show their path relative to the root, and hidden dot-files are omitted. Pick one and the wizard reads the head of the file, works out its **data format** — shown as a **Text**, **JSON** or **XML** badge next to the file name — and previews the first records.

The format is sniffed from the first non-blank byte: `{` or `[` is JSON, `<` is XML, anything else is text. JSON is further recognised as a top-level *array* or as *JSONL/NDJSON* (one object per line); XML is parsed with a hardened, XXE-safe parser and its repeating child element is offered as the record element. Whichever it is, the first five records are previewed as cards you click (or drag) to choose the example. For **JSON/XML** there is no delimiter to choose — the records are structural; skip to §17.5a for how their fields are mapped.

For a **text** file the detected **record delimiter** appears in a dropdown, with the number of times each candidate was found. You can override the guess if it is wrong. The supported delimiters are:

Delimiter	Bytes	Typical source
CRLF	`\r\n`	Windows line endings
LF	`\n`	Unix newline (most logs)
CR	`\r`	Classic Mac line ending
FF	`\f` (0x0C)	Form feed / page break
RS	0x1E	ASCII record separator
NUL	0x00	Null-delimited records
Blank line	`\n\n`	Multi-line records (a stanza per record)

> **Note** Detection is heuristic: it counts each candidate in the first 256 KB of the file and picks the most frequent, preferring CRLF over a bare LF (every CRLF also contains an LF). Changing the dropdown re-splits the same file on the delimiter you choose.

The first five records are previewed as cards. **Click a record** — or drag it into the example box below — to choose it as the example the mapping step works from. The chosen card is highlighted and the record fills the example box, so the selection is always visible; there is no separate confirmation panel. Clicking is the same action as dragging, for keyboard and pointer users.

> **Tip** Pick a record that carries *every* field you want to map — an example with an empty referrer or a missing user, for instance, will leave you unable to highlight those segments.

17.5 Step 2 — Field mapping

Step 2 is where the work happens. What it looks like depends on the data format. For **text** the instruction line reads “Select a segment below, pick a tab, then map it” and §17.5.1–17.5.9 apply. For **JSON** and **XML** the sample is shown read-only and you map fields by *path* — see §17.5a. The role tabs, the timestamp

analysis (§17.5.6), the META business titles (§17.5.7) and the Example JOURNEYS record (§17.7) are shared by all three formats.

17.5.1 The highlighted sample

The sample line is rendered in a monospace, selectable box. Every field's regex is run against it continuously — entirely in the browser — and each match is painted as a tinted mark in that field's *role* colour at 32 % alpha. Overlaps resolve first-come: a field earlier in the list claims its span first, so if two regexes fight over the same characters only the earlier one is highlighted. If you left the sample empty, the box reads “Paste a sample on the previous step.”

The highlighter is the fastest way to see whether a regex is doing what you think. A pattern that is too greedy is immediately obvious as a wash of colour across half the line.

17.5.2 Role tabs and what each role becomes

Below the sample are five role tabs, in order **Timestamp · Step · Id · Metas · Helper**. Each shows a coloured dot and, when fields of that role exist, a count in parentheses.

Tab	Stored role	Colour	Label / name	Where it lands in the database
Timestamp	`timestamp`	Accent blue	Fixed label `EVENT_TIME`	`JOURNEYS.EVENT_TIME`, a real TIMESTAMP normalised to `YYYY-MM-DD HH:MM:SS`.
Step	`step`	Green	Fixed label `STEP`	`JOURNEYS.STEP`, possibly replaced by a compound rule. Every new step name also gets a row in `STEPS`.
Id	`id`	Purple	Fixed label `EVENT_ID`	`JOURNEYS.EVENT_ID`, stored as the hex **MD5 digest** of the captured value.
Metas	`meta`	Orange	Editable name (default `meta_1`, `meta_2`, …)	The first three meta fields, in field order, become `META_1`, `META_2`, `META_3`. A fourth is silently unused.
Helper	`aux`	Teal	Editable name (default `helper_1`, …)	**No column at all.** Extracted per line and offered to compound rules only.

Two columns of `JOURNEYS` are never yours to set: `STEP_ID` is always NULL (it is filled at load time in the main application) and `SAMPLE_SET` is always the literal string `ORIGINAL`. `PROJECT_ID` comes from the run, not from the source type.

> **Warning** Only the **first** field of each single-instance role is used at import time. If you define two timestamp, id or step fields, the second one is ignored — it still paints the sample, which makes the mistake easy to miss. Likewise only the first three meta fields are written.

> **Note** `EVENT_ID` is pseudonymised unconditionally and is not configurable. The captured id — which is often a login or a user name — is hashed with MD5 before it reaches the database. The same raw id always produces the same digest, so journeys still correlate correctly, but the original value cannot be recovered from the database. The wizard's record preview shows the original value with the column note “original — stored as MD5” so you can check that you captured the right thing.

17.5.3 Mapping a segment with the highlighter

1. Select the tab for the role you want to create — the buttons below the tabs are tinted in the active role's colour so you can see which role you are about to assign.
2. Select the segment in the sample box with the mouse. Select exactly the value, without surrounding punctuation.
3. Click 🎯 **Use selection**. The console sends the selection's offsets to the backend, which generalises the text into a regex and returns it together with the value it captures. A new field of the active role appears with that regex already filled in.
4. If nothing is selected the console refuses with “Select a piece of the sample above first.”

The generalisation is deliberately simple and predictable:

In the selection	Becomes
A run of *n* digits	`\d{n}` , or `\d+` when the run is longer than 8 digits
A run of letters	`[A-Za-z]+`
A run of whitespace	`\s+`
Punctuation	Kept as an escaped literal
The character(s) immediately before the selection (when not a space)	Up to 8 characters of escaped literal context, prepended *outside* the capture group to anchor the match

Every generated regex has exactly one capturing group and uses a syntax subset valid in both Python and JavaScript — no named groups, no back-references — so the pattern the browser previews is byte-for-byte the pattern the extractor runs.

17.5.4 Writing a regex by hand

＋ Add field adds a blank field of the active role so you can type the pattern yourself. Each field row sits on a grey chip and contains, from left to right: the role label (for timestamp, step and id) or an editable name box (for meta and helper), a monospace regex input with the placeholder “regex (one capture group = the value)”, and a remove button.

Three conventions matter:

- Matching uses *search*, not a full-line match — the pattern may match anywhere on the line.
- If the pattern has capture groups, **group 1** is the value. If it has none, the whole match is the value.
- A regex that fails to compile is treated as “never matches”. At import time it does not raise an error; it just quietly contributes nothing, which is why the live feedback in the wizard matters.

The empty state of a tab reads “No <role> yet — select a segment above and map it, or add a field.”

17.5.5 Live match feedback

Under every regex input, the console tests the pattern against the sample as you type. All regex testing on this screen runs client-side.

Message	Meaning
Green ✓ **matches: <value>**	The pattern matched; this is the value that would be extracted.
Red ⚠ **invalid regex**	The pattern does not compile. Fix it — an invalid pattern never matches anything.
Grey • **no match in the sample**	The pattern compiles but finds nothing on this line.
Grey • **enter a regex**	The field has no pattern yet.

17.5.6 Timestamp format analysis

Whenever a timestamp field matches the sample, the captured value is sent to the backend for analysis. The row then shows either:

- Green 🕒 **<YYYY-MM-DD HH:MM:SS> · format <fmt>** — the value was recognised, and the inferred parse format is stored on the field automatically so that every line is later parsed the same way; or
- Orange ⚠ **couldn't recognise the date — stored as extracted** — the value could not be parsed.

The normalisation target is always `YYYY-MM-DD HH:MM:SS`. Runs of whitespace inside the value are collapsed before parsing. The analyser recognises a wide range of shapes:

Family	Examples
ISO-8601	With fractional seconds, `Z`, or `±HH:MM` / `±HHMM` offsets
Numeric dates	`YYYY/MM/DD`, `YYYY.MM.DD`, `DD.MM.YYYY`, `DD/MM/YYYY`, `DD-MM-YYYY`, `MM/DD/YYYY`, `MM-DD-YYYY`
Spelled-out months	“03 Aug 2026”, “Aug 3, 2026”
Clock variants	12-hour clocks with AM/PM
Log conventions	Apache/NCSA common log (`03/Aug/2026:14:05:09 +0000`), RFC-2822, C `asctime`, syslog (`Aug 3 14:05:09`)
Unix epochs	Exactly 10 digits (seconds), 13 (milliseconds) or 16 (microseconds), interpreted as UTC

Warning Two ambiguities are resolved by policy, and you must verify the result in the wizard. First, purely numeric dates such as `08/03/2026` are read **day-first** (European) by default; month-first (US) is preferred only when the value also carries a 12-hour AM/PM clock. If your log is American and uses a 24-hour clock, check the green preview line before saving. Second, **syslog timestamps carry no year** and are filled with the current UTC year — importing an old yearless log, particularly around New Year, mis-years the events.

Note The timestamp is not optional. A source cannot be run at all if its source type has no timestamp field: the run is refused with “The source type has no timestamp field.” And at import time a line only becomes an event if *all three* of id, step and timestamp were captured *and* the timestamp normalised successfully. Everything else counts as a skipped line.

17.5.7 META business titles

Meta fields carry a second input beside the regex, with the placeholder “Business name (e.g. Book ID) — shown in the app”. This is the human-readable title for the attribute. It is written to `METAS.META_1_TITLE`, `META_2_TITLE` and `META_3_TITLE` for the project, and it is what analysts see as the attribute name in the main application. It also appears as the column note in the wizard's record preview.

The field name itself (`meta_1`, `status_code`, whatever you type) is internal: it identifies the field in compound rule conditions. The title is external. Give a meta field a short technical name and a proper business title.

Warning The `METAS` row is **create-once**. It is written only when at least one meta field has a title configured, and only if the project has no row yet. Editing titles later does *not* rewrite an existing `METAS` row — the analyst keeps seeing the old names. Settle the titles before the first import into a project, or clear the project and re-import.

17.5.8 Helper fields, and why they exist

Helper fields (role `aux`) are extracted from every line exactly like any other field, and then thrown away. They are written to no column whatsoever. Their sole purpose is to give compound step rules something to match on.

The reasoning is the scarcity of META columns: a journey row has exactly three. An HTTP status code, a result code or a log level is usually needed only to decide *what the step is called*, not as a business attribute worth reporting on. Putting such a value in a helper field keeps all three META columns free for genuine business attributes. The wizard states this in the compound-steps explainer: “Values needed only for matching can be extracted as helper fields, which are never written to the database, so your META columns stay free for business attributes.”

Helper names are shown in teal, and their tooltip reads “Compound rules reference a helper by this name” — a reminder that the name, not a hidden identifier, is the link between the field and the rules.

17.5.9 Field locking and cascade rename

Compound rules reference fields **by name**. Two consequences follow, and the wizard implements both.

Locking. When a field is referenced by at least one compound rule, its ✕ remove button is replaced by an orange 🔒 lock. The lock is not clickable (the cursor shows “not allowed”) and its tooltip reads: “Used by N compound step rule(s) (<rule steps>). Remove the field from those rules first — deleting it now would stop them matching. Renaming is safe: the rules follow the new name.” An orange caption under the row repeats the message: “🔒 Used by N compound rule(s): <steps> — remove it there before deleting this field. Renaming is safe.” Deletion is additionally blocked in code, so no path through the interface can orphan a rule.

Cascade rename. Renaming a referenced field automatically rewrites every rule condition to the new name. This is why renaming is safe while deleting is not: after a rename the rules still point at a field that exists, whereas after a deletion they would point at nothing and would silently stop matching — the worst kind of failure, because the import would still succeed and simply produce the wrong step names.

1. To rename a meta or helper field, type the new name in its name box. Any compound rule conditions naming it are updated as you type.
2. To delete a locked field, open each rule listed in the lock's tooltip, remove or re-point the condition that names the field, close the rule editor with **Done**, and return to the field row — the ✕ is back.

17.5a Semi-structured formats: JSON and XML

When the data format is JSON or XML, a field's value has a natural *location* in the document — a key path, or an element/attribute — rather than a text pattern. Forcing a regex onto structured data is brittle, so these formats map fields by a **path selector** instead. The regex highlighter and 🎯 **Use selection** do not apply; everything else about roles, timestamps, META titles, compound steps and the record preview is identical (a helper field for a JSON/XML source simply carries a *path* instead of a regex).

The sample record is shown in a read-only box (pretty-printed JSON, or the XML element). Below the role tabs the wizard lists the record's **pickable fields** as clickable rows, each showing its path selector and the value it resolves to in the sample. Pick a role tab, then **click a field** to map it — a single-value role (Time-stamp, Step, Id) takes that path; the Metas tab adds another meta. You can also press **+ Add field** and type the path by hand. Each field row shows, live, the value the path extracts (**✓ value: …**), so a wrong path is obvious immediately. A filter box appears above the list once a record has more than six fields.

17.5a.1 JSON paths

A JSON source may physically be a **top-level array** `[{…}, {…}]` or **JSONL/NDJSON** (one JSON object per line). Either way each object is one record, and a field is located by a dotted key path with optional array indexing:

Path	Selects
`user.id`	The `id` under the `user` object.
`items[0].sku`	The `sku` of the first array element. Negative indices count from the end (`items[-1]`).
`["weird.key"]`	Bracket-quote a key that contains a dot, bracket or space.
`$.user.id`	A leading `$` is optional and ignored.

A path that resolves to a single scalar (string, number, boolean) yields that value; booleans render as `true` / `false` to match JSON. A path that lands on an object or array, or on nothing, yields no value — so that record is skipped if it was the id, step or timestamp. This is a deliberately small subset: dot and bracket navigation only, no wildcards, filters or functions.

17.5a.2 XML paths and the record element

For XML the source type also stores a **record element** — the repeating element that is one record, relative to the document root (e.g. `event` for `<events><event/>…</events>`). It is guessed from the most common

direct child of the root and shown in an editable box at the top of the mapping; each field's path is then resolved *relative to one record element*:

Path	Selects
`@id`	An attribute of the record element itself.
`step`	The text of a child element.
`payload/user`	The text of a nested descendant.
`user@id`	An attribute of a child element.
`.`	The record element's own text.

Warning (XXE safety) User XML is parsed exclusively through a hardened parser (`defusedxml`) that refuses external entities, DTD retrieval and billion-laughs expansion. A malicious document is rejected with a parse error rather than fetching a server file or exhausting memory — both in the wizard's detection and at import time. Never is a raw `xml.etree` parser run on a source file.

Import strategy differs by format. Line-oriented sources (text and JSONL) import only what was appended since the last run — a byte-offset *delta upload* (chapter 19). A whole-document source (a JSON array or an XML file) cannot be byte-delta'd, so it is *import-once by content signature*: re-running an unchanged file imports nothing, and a changed file re-imports the whole document. The pipeline never de-duplicates, so clear the project first if a re-import would double the rows. A whole-file parse is size-capped (16 MB by default) to bound memory; JSONL and text stream line by line and are not capped.

17.6 The Step tab and the STEPS table

The **Step** tab holds the plain step field and, below it, the **Compound steps** section — marked “optional”, with a count when rules exist and a **+ Add rule** button. Compound rules always produce a STEP value, which is why they live on this tab and nowhere else. They match on the extracted field *values*, so the section works for **every format** — for a JSON/XML source a helper field carries a path instead of a regex, but rules are authored and evaluated exactly the same way.

The behaviour in one paragraph: rules are evaluated in order and the **first fully-matching rule wins**; all conditions of a rule must hold; if no rule matches, the plain step field's captured value is used unchanged. Rules are therefore strictly additive — they can only relabel an event that the plain step field already produced. Under the rule list the wizard evaluates them live against the sample and reports either “✓ For the sample line the step becomes “<step>”.” or “• No rule matches the sample line — the plain STEP value would be used.” The full rule editor, its operators and the helper-field panel are covered in chapter 18.

Whatever step name comes out — plain or compound — is also used to populate the `STEPS` table in the target schema. After the events of a run have been pushed, the extractor creates a `STEPS` row for every step name it saw that has no `(PROJECT_ID, STEP)` row yet:

Column	Value assigned automatically
DESCRIPTION	Empty.
Background colour	Picked deterministically from a fixed ten-colour palette by hashing the step name, so the same step name always gets the same colour — across projects and across runs.
Foreground colour	`#ffffff`.
SCORE	`0`, so automatically created steps appear but do not skew scoring.
Shape	Picked deterministically from `stadium`, `round`, `hex`, `circle`, using an independent seed so that shape and colour do not correlate.
END_OF_PROCESS	`false`.
BELONGS_TO	NULL.

The run log records this as “created <n> new step(s): <names>”, and the count appears in the run's result detail. Existing rows are never touched, so any styling an analyst applied in the main application survives every later import.

> **Tip** Because step names drive both the process map and the automatic STEPS rows, keep them stable. A regex that captures a request path including its query string, for instance, produces a new step — and a new STEPS row — for every distinct query, which turns the process map into confetti. Strip variable parts in the regex.

17.7 The Example JOURNEYS record

As soon as at least one field exists, steps 2 and 3 show a one-row table labelled **Example JOURNEYS record**. It is the single most useful control in the wizard: it shows exactly what the current spec would write for the pasted sample line. Empty values render as “—”.

Column	Shows
PROJECT_ID	Muted “(set when the source runs)” — it comes from the Run dialog or the watchdog configuration.
EVENT_ID	The extracted id, with the column note “original — stored as MD5”.
STEP	The step value. A matching compound rule wins over the plain step field; the column note then reads “compound”.
STEP_ID	Empty and muted — always NULL at import.
EVENT_TIME	The normalised timestamp, or the raw extracted value if the format was not recognised.
META_1 … META_3	The first three meta fields in order; the column note is each meta's business title, or its name if no title is set.
SAMPLE_SET	Muted “ORIGINAL” — always this literal.

17.8 Step 3 — Review and save

The review step repeats the sample under the heading **Preview** (highlighted for text, the raw record for JSON/XML), the name with a **format badge** (and the XML record element), and a list of every field showing its role dot, role label, name, its **regex or path**, and — for timestamps — "→ <format>". Below that, a **Compound steps** summary (text only) renders each rule as `<step> ← field is "value" and …`, followed by the Example JOURNEYS record. With nothing defined it reads "No extraction fields defined."

1. Read the field list top to bottom and confirm that each role appears exactly as often as you intend — one timestamp, one id, one step, at most three metas.
2. Check the timestamp's inferred format.
3. Check the Example JOURNEYS record against what you expect the row to look like.
4. Click **Create source type** (or **Save changes**).

Only **complete** compound rules are stored: a rule needs a non-blank resulting step and at least one condition naming a field. Incomplete rules are dropped by the wizard and, independently, by the backend — so a half-built rule left over from an interrupted editing session can never relabel events at import time. Conditions with a blank field name are dropped, and an unrecognised operator falls back to `is`. In the same spirit, a field saved with an unrecognised role silently becomes a meta field.

17.9 Linking a source type to a source

A source type does nothing on its own. To use it, open a source (or create one) and pick it from the **Source type** dropdown on step 2 of the source wizard, whose help text reads "The parser applied to each line when this source runs." The default is "— none —". If you have not defined any yet, the dropdown is replaced by "No source types yet — define one in the Source types section first."

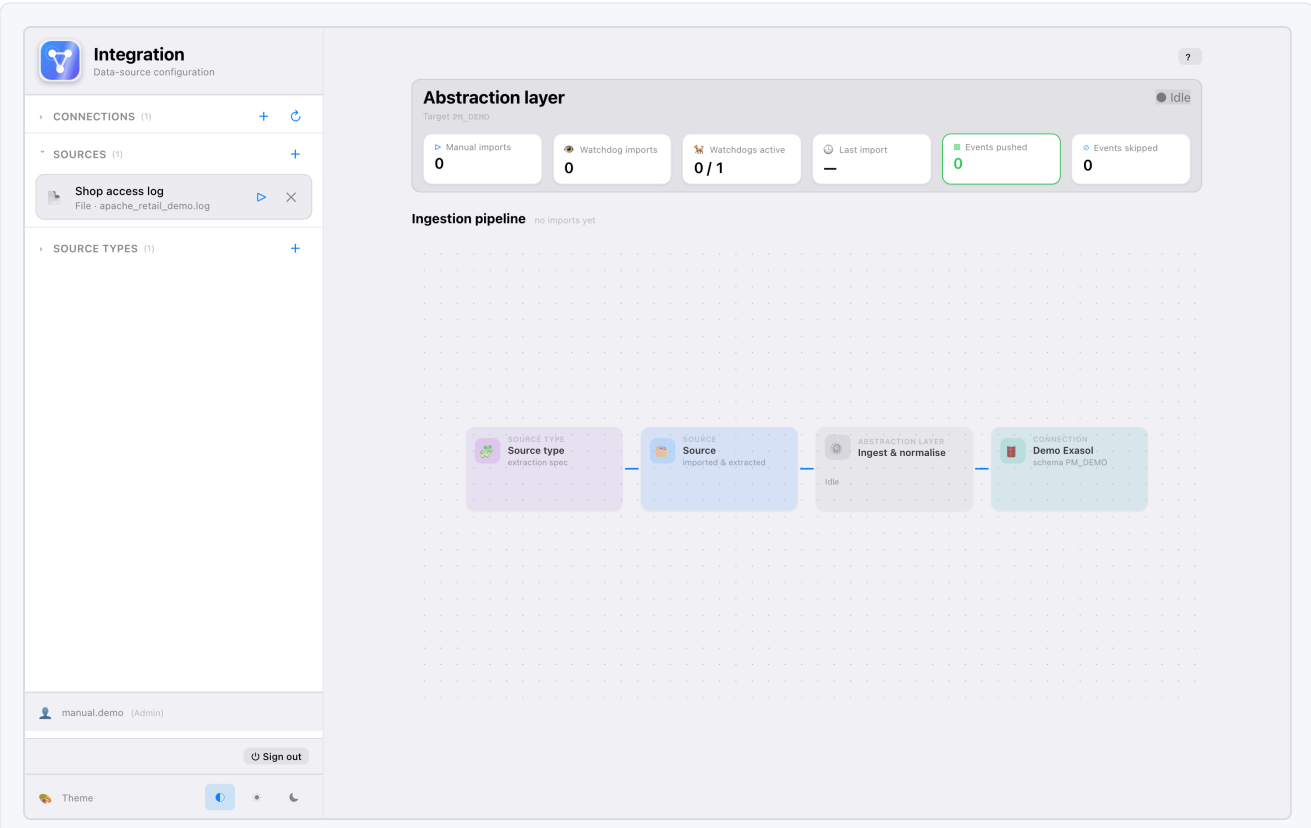


Figure 17.2 — The Sources section: each source links to exactly one source type, which supplies its parser.

> **Tip** Editing a source type takes effect on the *next* run of every source that links to it; nothing already imported changes. If a first import produced wrong step names, fix the regexes here, delete the project in the destination schema (Connection editor → Projects tab), reset the source's checkpoint, and import again.

17.10 Worked example: an Apache access log

The bundled example logs are seeded into the sandbox directory on first start, so this example can be followed end to end on a fresh installation. Assume the file contains combined-format lines like this one:

```
10.0.4.12 - jbrown [03/Aug/2026:14:05:09 +0000] "POST /shop/login HTTP/1.1" 200 1043
"https://shop.example.com/" "Mozilla/5.0"
```

Six fields make this a usable process log. Paste the line on step 1 as the sample, then build the following on step 2 — highlighting the value and clicking 🎯 **Use selection** gets you most of the way, after which you tidy the generated pattern by hand so it survives lines that differ from the sample.

Tab	Name	Regex	Captures
Timestamp	`EVENT_TIME`	`\[([^\]]+)\]`	`03/Aug/2026:14:05:09 +0000`
Id	`EVENT_ID`	`^\S+ \S+ (\S+)`	`jbrown`
Step	`STEP`	`" (?:GET|POST|PUT|DELETE|HEAD|PATCH) ([^ ?"]+)`	`/shop/login`
Metas	`method` — title “HTTP method”	`"([A-Z]+) /`	`POST`
Metas	`bytes` — title “Response size (bytes)”	`" \d{3} (\d+)`	`1043`
Metas	`client_ip` — title “Client IP”	`^(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})`	`10.0.4.12`
Helper	`status`	`" (\d{3})`	`200`

Four things in that table are worth dwelling on.

The timestamp. `\[([^\]]+)\]` takes everything between the square brackets. The analyser recognises the Apache/NCSA common-log shape, the row turns green — 🕒 **2026-08-03 14:05:09 · format …** — and the parse format is stored on the field. Note that the bracketed value is the only place on the line where the date lives, so the anchor is safe.

The case id. The third whitespace-separated token of a combined-format line is the authenticated user. Using it as the case id means one journey per user. That value is a login, which is precisely why `EVENT_ID` is MD5-hashed before it is written — the analysts see stable, correlated journeys without the database ever holding the user names. If the log has no authenticated user (a `-` in that position) the id capture still succeeds and produces a single journey for all anonymous traffic; in that case use a session cookie or a request-id header instead.

The step. The regex captures the request path but deliberately stops at a space, a `?` or a quote, so the query string never becomes part of the step name. Without the `?` in the character class, `/shop/search?q=hammer` and `/shop/search?q=nails` would be two different steps and two different `STEPS` rows.

The helper. The HTTP status is a matching value, not a business attribute — the analyst wants to see “login failed” on the map, not a column of numbers. Extracting it as a **helper** keeps all three META columns available for method, response size and client IP, and writes nothing extra to the database. On the Step tab, two compound rules then turn the pair (path, status) into a meaningful activity:

```
Rule 1   Step becomes "login successful"
         when  STEP    is           /shop/login
         and   status  is           200

Rule 2   Step becomes "login failed"
         when  STEP    is           /shop/login
         and   status  starts with  5
```

Rule order matters — the first fully matching rule wins — and a line whose path is not `/shop/login` matches neither rule, so its plain step value (`/shop/checkout` , say) is used unchanged. Comparisons are case-insensitive and trimmed on both sides. The green ✓ on a rule badge tells you it matches the pasted sample; because the sample carries status 200, the wizard reports “✓ For the sample line the step becomes “login successful”.” and the record preview's STEP column gains the note “compound”.

The resulting JOURNEYS row for the sample line is:

```
PROJECT_ID  (set when the source runs)
EVENT_ID    jbrown            → stored as MD5
STEP        login successful  → compound
STEP_ID     (NULL)
EVENT_TIME  2026-08-03 14:05:09
META_1      POST              → "HTTP method"
META_2      1043              → "Response size (bytes)"
META_3      10.0.4.12         → "Client IP"
SAMPLE_SET  ORIGINAL
```

17.11 Limits, and what actually happens at import time

Item	Limit
Source type name	1–120 characters, required
Sample record	≤ 20,000 characters
Field name	1–60 characters, required
Field regex (text)	≤ 2,000 characters
Field path (JSON/XML)	≤ 1,000 characters
XML record element	≤ 500 characters
Whole-file parse (JSON array / XML)	≤ 16 MB (JSONL and text stream line by line, uncapped)
Timestamp format token	≤ 120 characters (stored only when non-empty)
META business title	≤ 200 characters (stored only when non-empty after trimming)
Compound rules	≤ 200 per source type; resulting step ≤ 500 characters; condition field ≤ 60, value ≤ 500 characters
Line length at import	Every line is truncated to its first **2,000 characters** before any regex runs
Per-match time	0.25 seconds wall-clock (when the optional `regex` module is installed)

Warning The 2,000-character line cap applies before matching, so a regex aimed at content beyond that point never matches — a long JSON line with the interesting attribute at the end is a classic trap. Blank lines are always skipped. And a regex that backtracks catastrophically does not fail the run: the match times out after 0.25 seconds and the line is counted as *skipped*. A source type with a pathological pattern therefore shows up as a large “Events skipped” count and a slow import rather than as an error.

17.12 Troubleshooting

Symptom	Cause and remedy
Run refused with “The source type has no timestamp field.”	Add a field on the Timestamp tab. It is the one mandatory role.
Run refused with “Link a source type to this source first.”	The source has no `sourceTypeId`, or it points at a deleted source type. Edit the source and pick one.
Run finishes with “⚠ … No line matched — check the linked source type's regexes fit this file's format”	Lines were read but nothing was extracted. Copy a real line from the source's file preview into the sample and re-check every field's match feedback.
A high “Events skipped” KPI (the tile turns orange above zero)	Lines were read but failed the id/step/timestamp gate, or a regex timed out. Usually the timestamp: check the inferred format, and check that the values on other lines look like the sample's.
Dates come out a month off	Day-first versus month-first ambiguity. Verify the green timestamp preview in the wizard; a US log with a 24-hour clock is read day-first.
A meta column is empty even though the field matches	It is the fourth or later meta field. Only the first three are written; reorder or remove one.
The analyst sees the wrong attribute names	The `METAS` row was created on the first import and is never updated. Clear the project and import again.
A field cannot be deleted (🔒)	A compound rule references it. Remove the condition in the rule first; renaming, by contrast, is always safe.
A JSON/XML run reports “Nothing new to import — the file is unchanged”	Whole-document sources are import-once by content signature. To force a re-import, change the file, or run with delta off; clear the project first if you don't want duplicated rows.
A JSON/XML run fails with a parse error	The document is malformed, or an XML file tripped the XXE-safe parser (external entity / DTD). Fix the document; entity expansion is refused by design.
An XML import produces no events	The record element is wrong — check the box at the top of the XML mapping matches the repeating element, and that each field path is relative to *one* record.

18 Compound Steps

> A log line rarely names the activity you want to see in the process map. It names half of it — the endpoint, the command, the message code — and leaves the other half in a second field: a status code, a result flag, a return value. *Compound steps* are the mechanism that joins those halves back together at import time, so that the event stored in `JOURNEYS.STEP` reads "login successful" rather than "login". This chapter explains why compound steps exist, how to write and test rules in the source type wizard, exactly how the rules are evaluated when a source runs, how helper fields keep your three META columns free, and which mistakes cost people the most time. It is written for the person who defines source types in the integration console.

Requires: **Developer** or **Administrator**. Compound steps are edited inside the source type wizard on the integration console (HTTP :8100, HTTPS :8463). Power users and plain users cannot sign in to that surface, and every `/api/integration/*` endpoint refuses them with "You need the Developer role to use the integration features."

Note (all formats) The examples in this chapter use a text (regex) source type, but compound steps work for **every** data format. Rules match on the extracted field *values*, not on how they were extracted — so a JSON or XML source type carries rules in exactly the same way. The only difference is the selector: a helper field on a JSON/XML source type is a *path* (e.g. `status`, `@status`) rather than a regex, and the inline "Add a helper field" panel accepts a path there. Everywhere below, read "the value the field captures" as "the value the field extracts".

18.1 Why compound steps exist

Consider a single line from an Apache access log:

```
10.0.0.7 - - [03/Aug/2026:14:05:09 +0000] "POST /shop/login HTTP/1.1" 200 512 sess=abc-123
```

A regex over the request path extracts the step `login`. So does the next line — the one whose status code is `500`. Both events land in the process map as the same activity, and the map can no longer tell a successful sign-in from a failed one. The information is present in the line; it is simply in a different field.

Compound steps close that gap. A source type may carry an ordered list of **rules**. Each rule names a resulting step and one or more **conditions** over the fields the source type extracts. When every condition of a rule holds for a line, that rule's step text replaces the step the plain `step` field captured. When no rule matches, the plain step value is used unchanged — compound rules are strictly *additive*, never destructive.

The second half of the mechanism is the **helper field**. The status code in the example above is needed only to decide the step name; it has no business meaning and does not deserve one of the three META columns that carry business attributes such as an order number or a customer segment. A helper field (internally the role `aux`) is extracted from every line like any other field and offered to the rules by name, but it is written to **no database column at all**. The wizard states this in the explainer shown when a Step tab has no rules yet:

Note “Build the step from more than one field when the log splits it — e.g. step ‘login’ plus status ‘200’ becomes ‘login successful’. Values needed only for matching can be extracted as helper fields, which are never written to the database, so your META columns stay free for business attributes.”

18.2 Where the rules live

Compound rules belong to a **source type**, not to a source. They are stored with the source type's name, its pasted sample line and its field list, and they are owned per user. Every source that links to that source type is parsed with those rules, so editing a rule changes the behaviour of every future import through that source type — manual runs and watchdog imports alike.

The editor is the **Step** tab of step 2 (“Field mapping”) of the source type wizard. Compound rules appear there and nowhere else, because a rule always produces a STEP value; there is no equivalent for timestamps, ids or meta fields.

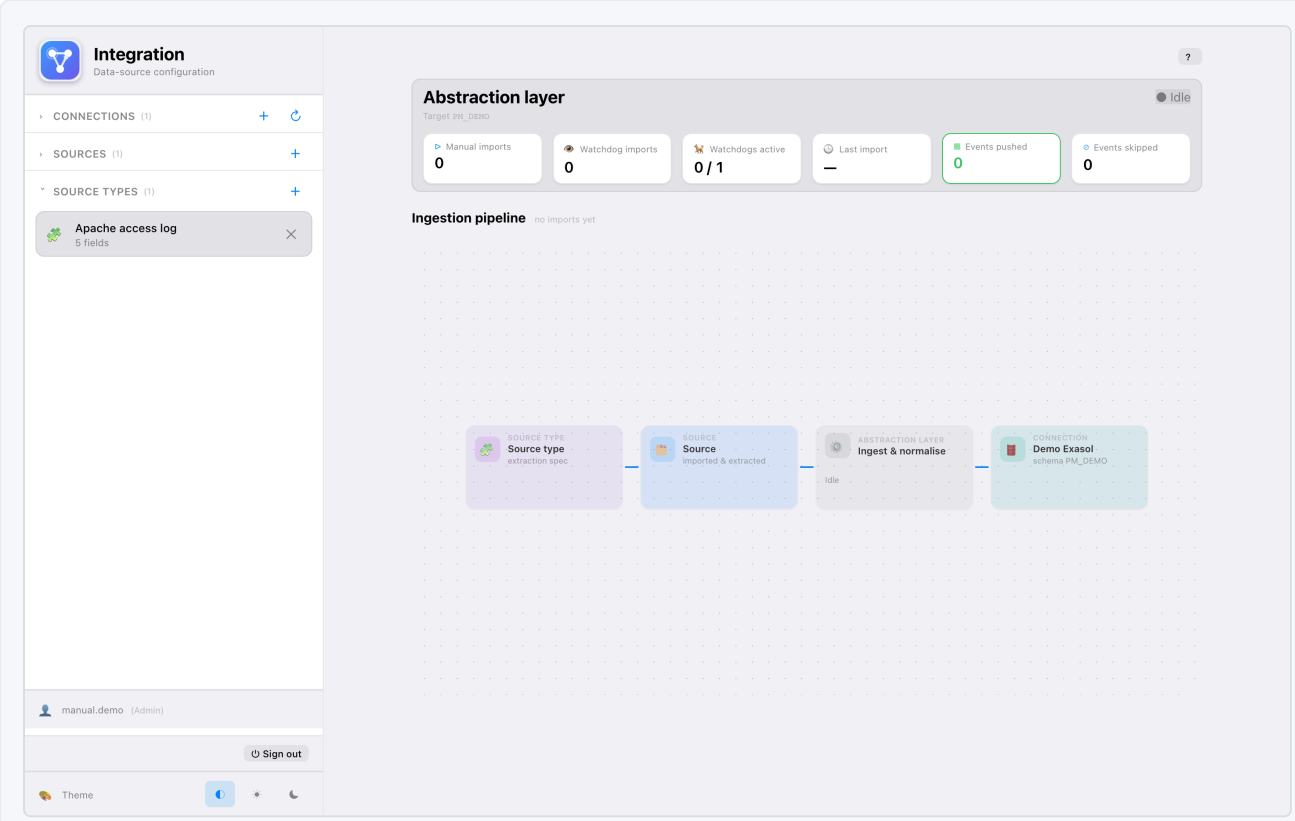


Figure 18.1 — The Source types section of the console sidebar. Each source type carries the extraction fields and the compound-step rules applied when a linked source runs.

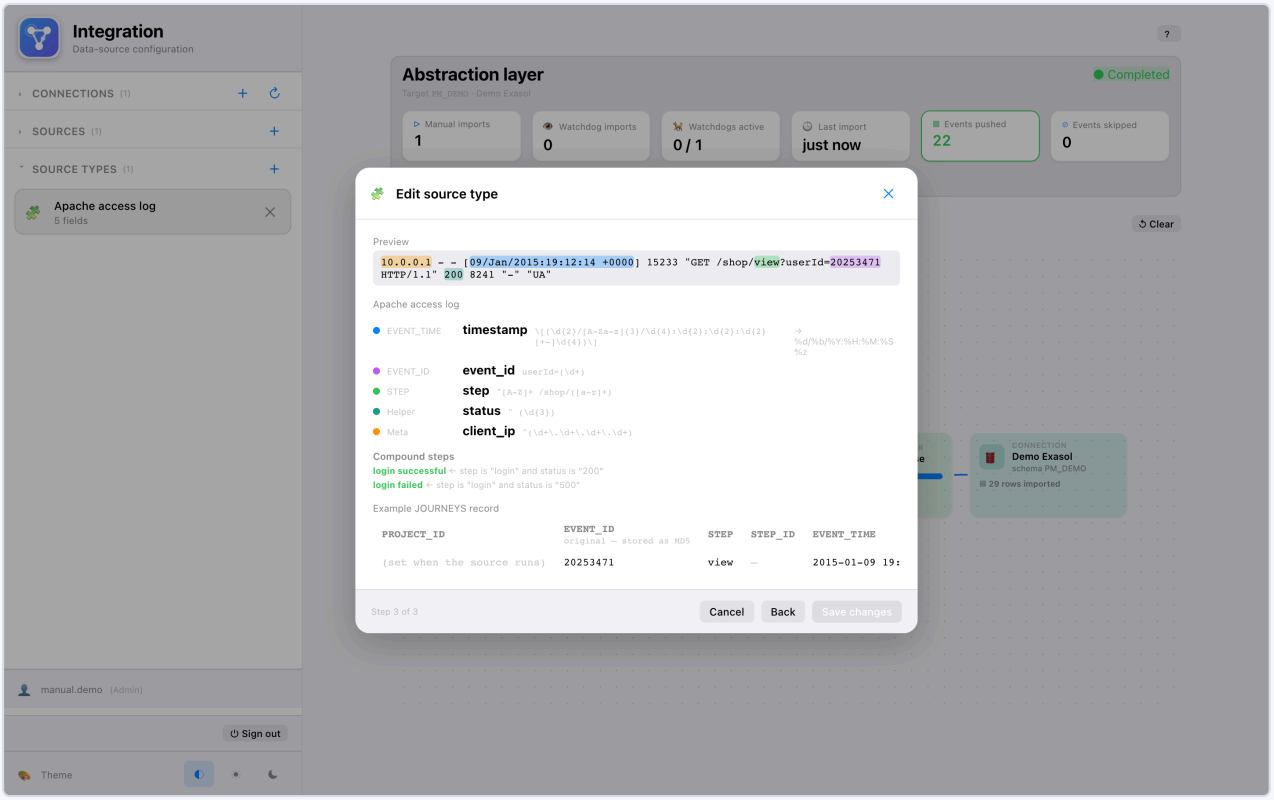


Figure 18.2 — The final wizard step shows the defined compound rules (“login successful ← step is `login` and status is `200` ”) above a preview of the resulting `JOURNEYS` record, so you can confirm the mapping before saving.

18.3 The Compound steps panel

Inside the Step tab, below the plain step field, sits a section headed **Compound steps** with the caption “optional”, a rule count in parentheses once at least one rule exists, and a **+ Add rule** button.

Rules render as **badges** in a scrolling list. Roughly three badges are visible at a time (each is at least 52 px high, the list is capped at 190 px); beyond that the list scrolls. Each badge shows:

Element	Meaning
Order number	The rule's position in the evaluation sequence. Tooltip: “Rule N — checked in this order”. Order is significant: the first fully-matching rule wins.
Resulting step	The text the rule writes into `STEP`, shown in green, or “(no step yet)” while the rule is unfinished.
Conditions summary	Each condition rendered as `field <operator label> “value”`, joined with “ · ”, or “no conditions yet”.
Green ✓	Shown when this rule matches the sample line pasted on step 1. Tooltip: “Matches the sample line”.
×	Removes the rule (tooltip “Remove rule”). There is no confirmation prompt.

Clicking anywhere else on a badge opens the rule editor.

Under the list the wizard evaluates the whole rule set against the sample line and reports the outcome in one line: green “✓ For the sample line the step becomes ‘<step>’.” when some rule matches, or grey “• No rule

matches the sample line — the plain STEP value would be used." when none does. The second message is not an error; it is the normal reading for a sample line that happens to be an ordinary event.

> **Tip** The preview in the wizard runs the *same* comparison logic as the backend evaluator, including case handling and trimming. If the badge shows a green tick against your sample line, the same line in the real file will be relabelled the same way.

18.4 The rule editor

Clicking a badge — or **+ Add rule** — opens a nested sheet titled **Compound step rule** with the icon ⚗. Edits take effect immediately; the **Done** button in the footer simply closes the sheet. Beside it the footer shows a live verdict against the sample line: "✓ matches the sample line" or "• does not match the sample line".

18.4.1 Controls

Control	Purpose
Step becomes	The resulting step text, shown in bold green. Placeholder "e.g. login successful". Written verbatim into `JOURNEYS.STEP`.
Conditions — all must hold	The condition list. The first row is labelled **when**, every further row **and**.
Field dropdown	Lists every *named* field of the source type. Helper fields are suffixed " (helper)". The empty option reads "(field)".
Operator dropdown	One of the six operators in 18.5.
Value input	The comparison value (placeholder "value"). Beside it a live hint **= <captured value>** shows what that field captures from the sample line; tooltip "What this field captures from the sample line".
–	Removes a condition. Shown only while the rule has more than one condition.
+ Add condition	Appends another empty condition row.
Add a helper field	The inline helper-field panel described in 18.6.

+ Add rule creates a rule pre-seeded with **two** conditions, because a compound step by definition combines at least two fields, and pre-selects the plain step field in the first condition — almost every rule starts with "when step is …".

18.4.2 Adding a rule

1. Open the source type wizard (click a badge in the **Source types** section, or **+** to create one) and go to step 2, **Field mapping**.
2. Select the **Step** tab. Make sure a plain step field exists and its match line reads "✓ matches: <value>" against your sample.

3. In the **Compound steps** section click **+ Add rule**. The rule editor opens with two conditions, the first already pointing at the step field.
4. Type the resulting activity name into **Step becomes**, for example `login successful`.
5. Complete the first condition: leave the field as the step field, choose the operator **is** and type the plain step value, for example `login`.
6. For the second condition pick the field that carries the other half of the activity — a helper field such as `status` — choose an operator, and type the value. Use the **= <captured value>** hint beside the input to see exactly what the field captures from your sample.
7. Check the footer verdict. If your sample line is one this rule should relabel, it must read "✓ matches the sample line".
8. Click **Done**, then repeat for the remaining variants. Verify the order numbers on the badges.
9. Go to step 3 and click **Create source type** / **Save changes**.

> **Note** Only **complete** rules survive the save. A rule needs non-blank text in **Step becomes** *and* at least one condition that names a field; conditions with no field selected are dropped, and a rule left with none is discarded entirely. This is deliberate: a half-built rule abandoned in the wizard can never silently relabel events at import time.

18.5 Conditions and operators

A condition is a triple: a field name, an operator, and a value. The field is referenced **by name**, which is why renaming a field is safe (see 18.7) and deleting one is not.

On screen	Stored id	Holds when…
is	`eq`	The captured value equals the condition value.
is not	`ne`	The captured value differs from the condition value.
contains	`contains`	The condition value occurs anywhere in the captured value.
starts with	`startswith`	The captured value begins with the condition value.
ends with	`endswith`	The captured value ends with the condition value.
matches regex	`regex`	The condition value, read as a regular expression, is found anywhere in the captured value.

Four properties of the evaluation matter in practice:

All conditions must hold

The conditions of one rule are combined with AND. There is no OR: express alternatives as two rules with the same resulting step. A rule with no valid condition never matches at all.

Case-insensitive and trimmed

Every operator except `regex` compares case-insensitively, and both sides are trimmed of surrounding whitespace first. Log casing is rarely dependable, so `is` "OK" matches a captured "ok ". When case re-

ally matters, use **matches regex**, which is the exact-comparison escape hatch.

A missing value makes the condition false

If the field's regex captured nothing on that line, every condition on it is false — including **is not**. "Missing" is not treated as "different". To catch lines where a field is absent you must find something else to test.

An unusable regex simply never matches

A **matches regex** value that is not a valid pattern makes the condition false rather than failing the import. The same is true of a field whose extraction regex does not compile.

18.6 Helper fields

Helper fields carry the role `aux`. They are extracted from every line exactly like a meta field — same regex convention, capture group 1 if the pattern has groups, otherwise the whole match, searched anywhere in the line — but they are written to no column. Only the first three meta fields map to `META_1` … `META_3`; anything you extract purely for matching should therefore be a helper, not a meta, or you will spend a business column on an HTTP status code.

Helpers are rendered in teal. New ones are named `helper_1`, `helper_2`, … (skipping names already taken); the name input carries the tooltip "Compound rules reference a helper by this name".

You can create a helper in two places: on the **Helper** role tab of step 2 like any other field, or — more conveniently — from inside the rule editor.

1. In the rule editor, find the **Add a helper field** panel.
2. Type a name into the first input (placeholder "name, e.g. status").
3. Type the extraction pattern into the second input (placeholder "regex with one capture group"). Watch the live feedback: green "✓ captures '<value>' from the sample", orange "⚠ invalid regex", or "• no match in the sample line".
4. Click **Add** (disabled until both inputs are filled). The helper is created as an `aux` field of the source type and is immediately pointed at by the first empty condition — or, if every condition already has a field, by a newly appended one.
5. Choose the operator and type the value for that condition.

> **Note** The panel's own explainer states the trade: "Extract a value only needed for matching (an HTTP status, a result code). Helper fields are never written to the database, so your three META columns stay free for business attributes."

18.7 Renaming and deleting referenced fields

Because conditions reference fields by name, the wizard protects the link in both directions.

Renaming cascades. If you rename a field that any rule references, every condition naming it is rewritten to the new name automatically. Nothing silently stops matching.

Deleting is blocked. On a field row, the ✕ remove button is replaced by an orange 🔒 lock as soon as any rule references that field. The lock is not clickable (the cursor shows “not allowed”) and its tooltip reads: “Used by N compound step rule(s) (<rule steps>). Remove the field from those rules first — deleting it now would stop them matching. Renaming is safe: the rules follow the new name.” An orange caption under the row repeats the message: “🔒 Used by N compound rule(s): <steps> — remove it there before deleting this field. Renaming is safe.” The same guard exists in the code, so no route through the interface can orphan a rule.

1. Open the rule editor of each rule listed in the lock's tooltip.
2. Remove the condition on the field with **–**, or point the condition at a different field.
3. Close the rule editor with **Done**. The lock on the field row turns back into ✕.
4. Delete the field and save the source type.

18.8 What happens at import time

Compound rules are applied while a source runs — from the Run dialog or from a watchdog poll; the two behave identically. For each non-blank line of the file, truncated to its first 2000 characters, the extractor does the following:

1. Runs the regex of **every named field** of the linked source type, including helper fields. Regexes are compiled once per run; one that fails to compile is treated as never matching.
2. Applies the mandatory gate: the line yields an event only if a case id, a step and a timestamp were all captured *and* the timestamp normalises successfully. Otherwise the line is **skipped** and counted in the **Events skipped** KPI.
3. Evaluates the compound rules in order. The first rule all of whose conditions hold supplies the step text. If none matches, the plain step field's captured value is kept.
4. Writes the row into `JOURNEYS`, with the derived value in `STEP`.
5. After the run, creates a `STEPS` row for every step name seen that did not have one, with an empty description, a colour and shape chosen deterministically from the step name, `SCORE` 0, `END_OF_PROCESS` false and `BELONGS_TO` NULL. The run log records “created <n> new step(s): <names>”.

> **Warning** Compound evaluation happens *after* the id/step/timestamp gate, not instead of it. A line whose plain step regex captures nothing produces no event at all, no matter how well a compound rule's other conditions fit. Compound rules can only *relabel* an event that would have been written anyway.

Because the derived step is an ordinary step name from the database's point of view, it takes part in everything downstream: it becomes a node in the process map, it gets its own colour and shape in `STEPS`, and it can be scored, described and grouped like any other activity.

18.8.1 The regex timeout guard

Every regex the extractor runs — field extractions and **matches regex** conditions alike — is executed under a **0.25-second wall-clock timeout** per match, provided the third-party `regex` module is installed in the backend environment. The reason is blunt: CPython's standard `re` engine does not release the interpreter

lock while matching, so a catastrophically backtracking pattern on a hostile line would freeze the entire compute backend — including the watchdog loop and every other user's request — until it finished.

The consequences are asymmetric and worth remembering:

Where the time-out hits	Result
A field extraction regex	The line counts as unparseable and is **skipped**. It shows up in the run detail ("… n skipped") and in the **Events skipped** KPI, which turns orange above zero.
A **matches regex** condition	The condition evaluates to **false**. The rule does not match, evaluation continues with the next rule, and the event is still written — with the plain step, or a later rule's step.

Warning Without the `regex` module the backend falls back to the standard library and *no* timeout applies. A pathological pattern then runs to completion. Keep expressions simple, avoid nested quantifiers such as `(\w+)+`, and prefer anchored literal context over open-ended wildcards.

18.9 A worked example

The source is an Apache-style access log with a session id appended. The sample line pasted on step 1 of the wizard is the first of these:

```
10.0.0.7 - - [03/Aug/2026:14:05:09 +0000] "POST /shop/login HTTP/1.1" 200 512 sess=abc-123
10.0.0.9 - - [03/Aug/2026:14:05:11 +0000] "POST /shop/login HTTP/1.1" 500 216 sess=def-456
10.0.0.7 - - [03/Aug/2026:14:07:44 +0000] "GET /shop/basket HTTP/1.1" 200 980 sess=abc-123
10.0.0.7 - - [03/Aug/2026:14:09:02 +0000] "POST /shop/checkout HTTP/1.1" 402 120 sess=abc-123
10.0.0.3 - - [03/Aug/2026:14:11:20 +0000] "GET /shop/search HTTP/1.1" 200 44 sess=ghi-789
```

18.9.1 The fields

Tab	Name	Regex	Written to
Timestamp	`timestamp`	`\[([^\]]+)\]`	`EVENT_TIME`, normalised to `2026-08-03 14:05:09`
Id	`id`	`sess=([A-Za-z0-9-]+)`	`EVENT_ID`, stored as an MD5 digest
Step	`step`	`"[A-Z]+ /shop/([a-z]+)`	`STEP` — unless a rule overrides it
Metas	`client` (title "Client IP")	`^(\S+)`	`META_1`
Helper	`status`	`" (\d{3})`	nothing — matching only
Helper	`method`	`"([A-Z]+) /`	nothing — matching only

Two META columns remain free for genuine business attributes, because `status` and `method` are helpers.

18.9.2 The rules

#	Step becomes	Conditions (all must hold)
1	login successful	`step` **is** “login” · `status` **is** “200”
2	login failed	`step` **is** “login” · `status` **starts with** “5”
3	checkout declined	`step` **is** “checkout” · `status` **is not** “200”
4	basket viewed	`step` **is** “basket” · `method` **is** “GET”

18.9.3 The result

Line	`step`	`status`	`method`	First matching rule	STEP written
1	login	200	POST	1	login successful
2	login	500	POST	2	login failed
3	basket	200	GET	4	basket viewed
4	checkout	402	POST	3	checkout declined
5	search	200	GET	— none	search (plain step)

Line 5 shows the fallback: no rule covers `search`, so the plain step value is written unchanged. Line 2 shows why **starts with** is often the right operator for status families — it catches 500, 502 and 503 with one rule. Line 4 shows **is not** doing useful work: any non-200 checkout is a decline.

After the run the `STEPS` table contains rows for `login successful`, `login failed`, `basket viewed`, `checkout declined` and `search` — five activities where the plain extraction would have produced four indistinguishable ones.

18.9.4 Checking the outcome before you save

Steps 2 and 3 of the wizard show a one-row table labelled **Example JOURNEYS record**, built from the sample line with the current specification. Watch the `STEP` column: when a compound rule wins, the cell shows the derived text and the column note reads **compound**. The `EVENT_ID` cell shows the original captured id with the note “original — stored as MD5”, and `EVENT_TIME` shows the normalised timestamp. Step 3 also lists a **Compound steps** summary in the compact form `<step> ← field is "value" and …`, which is the fastest way to read the whole rule set at a glance before saving.

18.10 Limits

Item	Limit
Rules per source type	200
Resulting step text	500 characters
Condition field name	60 characters
Condition value	500 characters
Field regex	2000 characters
Sample line stored with the source type	20 000 characters
Line length read from the source file	2000 characters (the rest of the line is discarded)
Regex match time	0.25 seconds per match, when the `regex` module is present

An operator the backend does not recognise — only possible through the API, not the wizard — is silently treated as `eq`.

18.11 Common mistakes

Writing the general rule before the specific one

The *first* fully-matching rule wins, so a broad rule placed early shadows every narrower rule after it. Put the most specific rules first and read the badge order numbers before saving.

Omitting the step condition

A rule that says only "`status` starts with 5" relabels *every* 5xx line as, say, "login failed" — searches, checkouts and all. Nearly every rule should pin the plain step field as well.

Expecting a rule to rescue a line that produced no event

If the id, step or timestamp regex fails, the line is skipped before rules are considered. A large **Events skipped** count with no compound effect points at the field regexes, not at the rules.

Expecting is not to fire on a missing value

A condition on a field that captured nothing is false for every operator. If a status code is absent from some lines, "`status` is not 200" will not match them.

Relying on case

All operators except **matches regex** compare case-insensitively after trimming, so "GET", "get" and " Get " are the same value. If you genuinely need case-sensitive matching, use **matches regex**.

Using a meta field for a matching-only value

Only the first three meta fields are written, to `META_1` … `META_3`; a fourth is silently ignored. Extract status codes, result flags and the like as *helper* fields so the META columns stay available for business attributes.

Leaving a rule half-finished

A rule with no step text, or with no condition that names a field, is dropped on save without warning. If a rule disappears after saving, it was incomplete.

Targeting content past character 2000

Every line is truncated to its first 2000 characters before extraction. A regex aimed at a field beyond that point never matches, and the condition that uses it is always false.

Typos in the resulting step name

Step names are written verbatim and auto-create `STEPS` rows. “login successful” and “Login Successful” become two separate activities in the process map. Keep a consistent naming convention across all rules of a source type.

Expecting rules to change events already imported

Rules are applied at import time only. Editing them has no effect on rows already in `JOURNEYS`. To relabel a historical import you must delete the project in the destination schema, reset the source's checkpoint, and import the file again from the top — resetting a checkpoint on its own deletes nothing from the database and would simply append everything a second time.

Deleting a source type that sources still use

Deletion does not cascade. A source whose source type has gone keeps a dangling link, its Run button warns “This source has no source type linked”, and the watchdog records the checkpoint error “The source has no source type linked.” Pick a replacement source type on the source before running it again.

> **Tip** After changing rules, run the source once against a scratch project id with delta upload switched off, and inspect the process map. New or missing activity names show up immediately, and the run log line “created <n> new step(s): <names>” tells you exactly which step names your rules produced.

19 Sources and Running an Import

A *source* tells the integration console *where* the raw data is; a *source type* (Chapter 18) tells it *how* to read each line. This chapter covers the remaining half of the job: creating a File source, pointing it at a file inside the sandboxed sources directory, previewing the lines, linking a source type, choosing how often the import commits, and then running the import into a destination connection and project. It closes with the two things that surprise people most — how the read checkpoint decides what a delta import picks up, and the fact that events are always appended and are never de-duplicated. Read it before you import into a schema anyone else relies on.

Requires: **Developer** or **Administrator**. Sources, source types and checkpoints are owned per user — you only ever see and run your own.

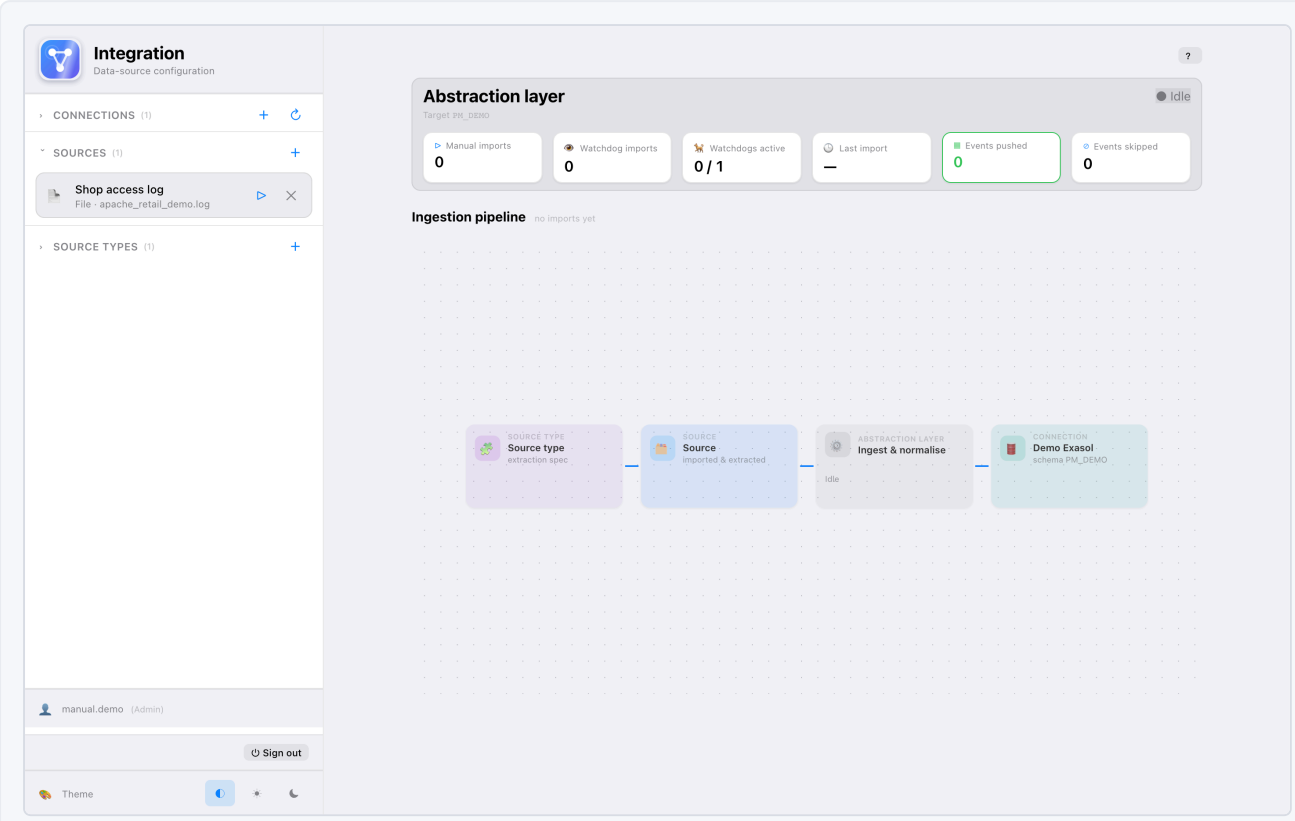


Figure 19.1 — The **Sources** section of the integration sidebar. Each badge shows the kind icon, the source name, the configured path, a ▷ Run button and a ✕ Delete button.

19.1 Before you start

An import needs four things to be in place. If any of them is missing the run is refused before a single line is read, with the message shown below.

Prerequisite	Where it comes from	Message if missing
A readable file	The source's **File path or glob** field	`This source has no file path.` / `File not found.`
A linked source type	The source's **Source type** dropdown	`Link a source type to this source first.`
A timestamp field in that source type	The source type wizard, **Timestamp** tab	`The source type has no timestamp field.`
A destination connection assigned to you, with a target schema	Assigned by an administrator; schema set in the connection editor	`Connection not found.` / `"<name>" has no target schema configured.`

> **Note** Only the **File** kind can be run today. The wizard also lists **Database**, **REST API** and **Object storage**, but they are greyed out and marked "· coming soon". A source of any other kind is refused with `Only file sources can be run yet.`

19.2 Where source files must live

Every file read by the console — the preview, a manual import, a watchdog poll — goes through one path resolver. By default that resolver confines you to a single directory, the **sources directory**, so that a developer account cannot use the console to read arbitrary files from the server.

Setting	Default	Effect
`PMW_INTEGRATION_FILES_DIR`	`<data dir>/integration_files`, where the data directory is `PMW_DATA_DIR` or `<project root>/data`	The sandbox root. Created automatically at start-up, parents included. In Docker it is a mounted volume, so you can drop log files into it from the host.
`PMW_INTEGRATION_ALLOW_ANY_PATH`	off	When set to `1`, `true`, `yes` or `on` (case-insensitive), the sandbox is switched off entirely and *any* absolute path the server process can read is accepted; `~` is expanded.

In the default, sandboxed mode the rules are:

- A **relative** path is taken relative to the sources directory — so `access.log` means `<files dir>/access.log`.
- An **absolute** path is accepted only if it already lies inside the sources directory.
- The real path is resolved with symlinks followed, and the result must *still* be inside the (likewise resolved) root. Traversal with `../` and symlinks pointing outside are rejected.

Error text	Cause
`No file path given.`	The path is empty or only whitespace.
`That path is outside the allowed sources directory.`	Traversal, a symlink escape, an absolute path elsewhere on the machine (on Windows, also a different drive).
`File not found.`	The resolved path does not exist.
`Not a regular file.`	The path resolves to a directory, a device or a socket.

Warning `PMW_INTEGRATION_ALLOW_ANY_PATH` lets anyone with the Developer role read any file the backend process can read, and preview it in the browser. Enable it only on a developer-trusted deployment, never on a shared or production installation.

Tip On start-up the backend copies every `*.log` file from the installation's `examples/` directory into the sources directory, unless a file of that name is already there. Existing files are never overwritten, so you can experiment with the bundled demo log immediately, and your own edits to it survive a restart.

19.3 Creating a File source

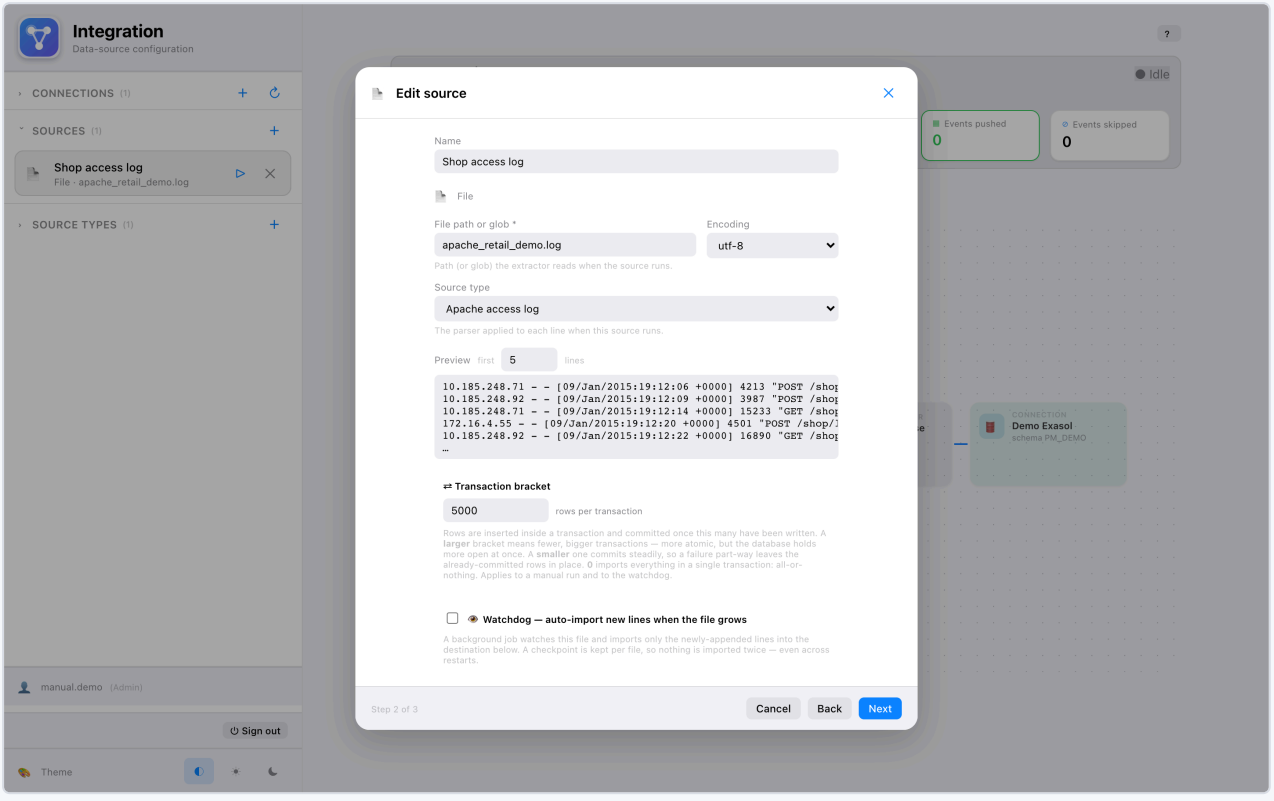


Figure 19.2 — The File source editor: the file path, the encoding, a live preview of the first lines, the linked source type and the transaction-bracket setting.

Sources are created and edited in the same three-step wizard. The footer always shows **Step N of 3** together with **Cancel**, **Back** and **Next**, and on the last step **Create source** or **Save changes**. Opening an existing source from the sidebar takes you straight to step 2, since its kind is already fixed.

1. In the sidebar, open the **Sources** section and click **+** (*Add a source*). The **New source** sheet opens at step 1.
2. On **Choose a source kind**, select the **File** card — "A log or data file the extractor reads at run time." A green dot marks the selected kind. Click **Next**.
3. Enter a **Name** (required; for example `File — access log`). **Next** stays disabled until the name is filled in.
4. Fill in **File path or glob**, the required field marked with `*`. Type a path relative to the sources directory, or an absolute path inside it.
5. Choose the **Encoding** that matches the file: `utf-8` (default), `latin-1` or `utf-16`.
6. Check the **Preview** box underneath (see §19.3.2) to confirm the console is reading the file you meant.
7. Pick the **Source type** that parses these lines. The first entry is **— none —**, which is also the default.
8. Set the ⇄ **Transaction bracket** if the default of 5000 rows per commit does not suit the import (see §19.4).
9. Optionally switch on 👁 **Watchdog — auto-import new lines when the file grows** and complete its destination fields (Chapter 20).
10. Click **Next**, check the review page — kind icon, name, and every configured field as label → value — and click **Create source**.

19.3.1 Field reference

Field	Default	Notes
Name *	—	1–120 characters. Saving without one gives `A name is required.` and returns you to step 2.
File path or glob *	—	Help text: "Path (or glob) the extractor reads when the source runs." Resolved through the sandbox described in §19.2. Saving without it gives `<Label> is required.`
Encoding	`utf-8`	Used for the import. Bytes that cannot be decoded are replaced rather than causing an error, so a wrong encoding shows up as mangled values, not as a failure.
Source type	**— none —**	"The parser applied to each line when this source runs." If you have not defined any, the wizard says *"No source types yet — define one in the Source types section first."*
rows per transaction	5000	See §19.4. Minimum 0, stepped in thousands.
👁 **Watchdog**	off	Background auto-import; covered in Chapter 20. It shares this source's checkpoint and transaction bracket.

> **Warning** Despite its label, **File path or glob** is *not* expanded as a glob today. The path is resolved to exactly one regular file; a pattern such as `/data/logs/*.log` produces `File not found.` Point the source at a single file.

> **Note** The whole configuration of a source is stored as one block and must serialise to 8000 characters or fewer, otherwise the save is refused with `Source settings are too large.`

19.3.2 The line preview

Below the path field, the row **Preview first [N] lines** shows the beginning of the file exactly as the extractor will see it. `N` defaults to **5** and is clamped to **1–50**; an invalid entry falls back to 5. The preview refreshes roughly 350 ms after you stop typing in the path or changing `N`.

What you see	Meaning
Monospace lines	The first `N` *non-blank* lines. Blank lines are skipped everywhere — preview, import and progress counting.
A trailing `…`	Further non-blank content follows the lines shown.
“Enter a file path to preview it.”	The path field is still empty.
“No lines.”	The file exists but has no non-blank content.
Red text	A sandbox or file error, verbatim — for example `File not found.`

> **Note** Two limits apply to the preview and to the import alike. Every line is truncated to its first **2000 characters**, so a regex aimed at content beyond that never matches. And the preview always decodes as UTF-8, whatever **Encoding** you selected — a `latin-1` file may therefore look slightly wrong in the preview yet import correctly.

19.3.3 Linking a source type

The linked source type is what turns a line into an event: its timestamp, id, step, meta and helper regexes plus any compound step rules are applied to each line at run time. Two consequences are worth remembering:

- A source with **— none —** selected cannot be run. The Run dialog shows the banner *“This source has no source type linked — edit it and pick one so the lines can be parsed.”* and ▷ **Run extraction** stays disabled.
- Deleting a source type does *not* cascade. Sources that referenced it keep a dangling link and behave exactly as if none were selected until you pick a new one.

> **Warning** Changing the **File path** of an existing source deletes that source's read checkpoint. The next import — manual or watchdog — therefore reads the new file from the very beginning. That is usually what you want, but if the new path points at the same content under a different name, everything is imported again.

19.4 The transaction bracket

The ⇄ **Transaction bracket** panel holds a single number input labelled **rows per transaction**. Rows are written inside a transaction and committed as soon as that many rows have been written; when you set it to `0` the label gains the suffix **(one transaction for the whole import)**.

Value entered	Value used	Behaviour
(blank or non-numeric)	5000	The default.
`0` or negative	0	One single transaction for the whole import — all rows become durable together, or none do.
`1` – `999`	1000	Raised to the minimum bracket.
`1000` – `1000000`	as entered	Commits every `n` rows.
above `1000000`	1000000	Clamped to the maximum.

A larger bracket means fewer, bigger transactions — more atomic, but the database holds more work open at once. A smaller one commits steadily, so a failure part-way through leaves the already-committed rows in place and makes progress durable as the run proceeds. The number of brackets committed so far is reported in the status panel as *commits*. Autocommit on the destination driver is switched off for the duration of every run, so the bracket — not the driver — decides when work becomes durable. The setting applies to manual runs and to watchdog imports alike.

> **Warning** With a bracket greater than 0, a run that fails half-way leaves the brackets it already committed in the database, while the checkpoint stays where it was (§19.6). Retrying therefore re-reads those lines and writes them a *second* time. If a mid-run failure must never duplicate rows, set **rows per transaction** to `0` and accept one long-open transaction instead.

19.5 Running an import

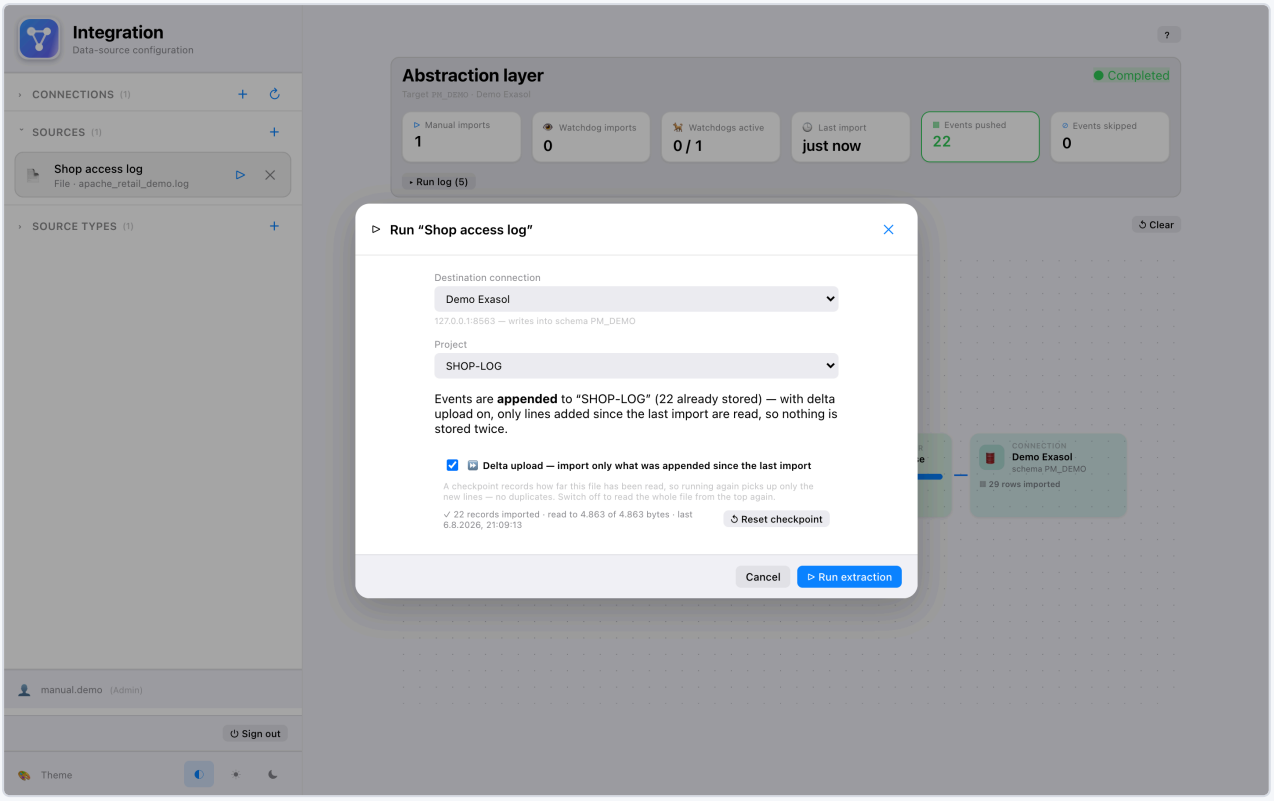


Figure 19.3 — The Run dialog. You choose the **Destination connection** and the **Project** (an existing one or a new id), leave **Delta upload** on to import only new lines, and press **Run extraction**. The checkpoint panel below records how far the file has been read.

Click the ▷ button on a source badge (tooltip *Run extraction*) to open the **Run “<source name>”** sheet.

1. Choose the **Destination connection**. The dropdown offers **— pick a connection —** plus every connection assigned to you. Under it, the target summary reads **<host>:<port> — writes into schema <schema>**, or “(none set)” when the connection has no schema.
2. Choose the **Project**. The first entry is **+ New project…**; below it are the projects already present in that schema, shown as `<projectId> — <title>`.
3. If you picked **+ New project…**, type the identifier into **New project id** (for example `RETAIL-DEMO`). It is written into `JOURNEYS.PROJECT_ID` for every event, and the `PROJECTS` row is created for you.
4. Decide whether ⏩ **Delta upload — import only what was appended since the last import** stays ticked. It is on by default.
5. Check the checkpoint panel underneath so you know what the run will read (§19.6).
6. Click ▷ **Run extraction**. The button reads “Running…” while the import is in flight; **Cancel** becomes **Close** once a result is shown.

19.5.1 The destination connection

You do *not* need to connect to the database in the main app first. The backend opens the chosen destination itself, using the connection's stored credentials, and closes it again when the run ends — including

when it fails. The exception is a connection you happen to be actively connected to in the main app: that live session is reused instead, with autocommit turned off for the run.

The dropdown pre-selects, in this order: the connection this source last imported into (remembered on the server per source, and dropped if the assignment has since been revoked), then the main app's currently active connection, then the first connection assigned to you.

> **Note** The dropdown is a convenience, not the authorisation. The server re-checks that the connection is assigned to you on every run. An unassigned — or simply non-existent — connection id both return the same message, `Connection not found.`, so that knowing an id reveals nothing about whether it exists.

If nothing is assigned to you the sheet shows the banner *"No connections are assigned to you — ask an administrator, or create one in the main app."* If the connection cannot be opened, the run fails with `Could not connect to "<name>": <error>`; if it has no target schema, with `"<name>" has no target schema configured.`

19.5.2 Choosing the project

Filling the project list opens the destination database on demand; while that happens the sheet shows *"Reading the projects in that schema…"*. The outcomes are:

Projects found

They are listed with their identifier and, when different, their title. The project you last imported into is pre-selected — but only when the same connection is selected; on a different connection nothing is pre-selected.

None found

"That schema has no projects yet — this import creates the first one." A reachable schema that has no `PROJECTS` table at all counts as an empty list, not an error.

Listing failed

An orange line, *"Could not list existing projects (<error>) — you can still type a new project id below."* The import itself is unaffected.

If the remembered project no longer exists in that schema, its identifier is kept in the **New project id** field rather than being silently discarded, so a re-created project keeps its name.

> **Warning — events are appended and never de-duplicated** Selecting an existing project shows the banner *"Events are **appended** to "<id>" (<n> already stored)"*. There is no de-duplication anywhere in the pipeline: nothing compares a new row against what is already in `JOURNEYS`. With **Delta upload** on, only newly-appended lines are read, so nothing is stored twice. With it off, the whole file is read again and every event already imported is written a second time. To reload a project from scratch, delete the project first (the connection editor's **Projects** tab does this) and then import with delta upload switched off.

19.5.3 Delta upload

⏩ **Delta upload** is ticked by default. With it on, a checkpoint records how far the file has been read and the run picks up only the lines appended since — the help text reads *"no duplicates"*. With it off, the file is read from the top, ignoring the stored offset, and the run restarts the checkpoint's imported-records counter rather than adding to it.

> **Note** One read consumes at most **8 MB** of new bytes. The first delta run on a large existing file therefore imports only the first ~8 MB; run it again (or let the watchdog poll again) to continue draining the backlog, cap by cap.

19.5.4 Progress and results

While the import runs, a progress bar shows **Importing <done> / <total> log records…**, or **Reading the file…** and an indeterminate animation before the totals are known. The same progress appears on the Abstraction layer node of the pipeline canvas, since both read the same live status, polled every 350 ms.

Result	Appearance	Meaning
Success	Green ✓ with a detail line such as `12,340 events written, 3 skipped, 5 new step(s), project created`	Events were written. "Skipped" counts source lines that could not be parsed.
Nothing new	Neutral grey — *"Nothing new to import — the file has not grown since the last import."* or *"Nothing to import — the file is empty."*	No lines were read at all. This is the normal, expected outcome of a delta run with no new data, and is deliberately not styled as a problem.
No line matched	Orange ⚠ *"… No line matched — check the linked source type's regexes fit this file's format (edit the source type and test against a sample line)."*	Lines *were* read but produced no events — a real misconfiguration of the source type.
Failure	Red text, e.g. `Could not connect to "<name>": …` or `Extraction failed: …`	The run was aborted. See §19.4 for what may already be committed.

Validation before the run produces `Pick a destination connection.` or `A project id is required.`

> **Note** Only **one extraction per user** may run at a time. Starting a second — including a manual run while your own watchdog is importing — fails with `An extraction is already running for this user.` Different users are unaffected and may import concurrently.

Unparseable lines are skipped and counted rather than aborting the run; the count appears both in the result detail and in the **Events skipped** KPI tile, which turns orange as soon as it is greater than zero. Regexes are executed with a 0.25-second timeout per match where the optional `regex` module is installed, so a catastrophically backtracking pattern costs you skipped lines rather than a frozen backend.

19.6 Checkpoints

Each source has exactly one checkpoint, stored on the server. It is what makes "import only what is new" possible, and it is **shared between the manual delta run and the watchdog** — both advance the same offset, which is precisely what stops the two triggers importing a line twice between them.

> **Note — a manual run imports the whole file at once** A manual **Run extraction** reads everything from the checkpoint to the end of the file in a single run, however large the file is (it is streamed in bounded chunks internally, so it is never loaded into memory whole). The watchdog is different: to keep each unattended poll cheap it reads at most one ~8 MB chunk per poll and lets a large backlog drain over several polls. So if you point the watchdog at a huge pre-existing file it will catch up over a few polls, whereas a manual run finishes it in one.

19.6.1 What is stored

Field	Meaning
`byteOffset`	Where the next read resumes. Only complete lines — up to the last newline — are ever consumed, so the offset always stops on a line boundary and a partial trailing line is left for next time.
`size`	The file's size at the last read, used to detect truncation.
`signature`	The MD5 of the file's first 64 bytes — a fingerprint that does not change as the file grows by appends, used to detect replacement.
`records`	The cumulative number of events imported from this file.
`updatedAt`	When the checkpoint was last written, shown in your local time.
`lastError`	The last failure message, or empty. Cleared by the next successful read.

The checkpoint panel in the Run dialog renders this as one of three lines: red ⚠ **<last error>**; ✓ **<n> records imported · read to <bytes> of <bytes> bytes · last <date and time>**; or *"No checkpoint yet — this is the first import of this file."*

19.6.2 Delta versus full re-read

Every manual run is checkpointed, whether or not delta upload is ticked.

- **Delta run** — reads from `byteOffset` to the end of the file and *adds* to the records counter.
- **Full run** (delta unticked) — reads from byte 0, ignoring the stored offset and signature, and *restarts* the records counter.
- **Empty file, or nothing appended** — a normal outcome, not an error. The run still executes, so it appears in the pipeline and the KPIs, and simply writes no rows.

19.6.3 Rotation, truncation and replacement

Log files are rotated, and the console detects the two common shapes of that without you doing anything:

Truncation or rotation in place

The file is now smaller than the stored offset. Reading restarts at byte 0.

Replacement under the same name

The size did not shrink, the offset is greater than zero, but the stored signature no longer matches the file's first 64 bytes. The offset must therefore point into unrelated content, so the file is re-read from byte 0.

> **Warning** A file shorter than 64 bytes gets an empty signature and cannot be fingerprinted. Replacement of such a file without a size decrease goes undetected, and the import silently resumes at an offset that now means something else.

> **Note** A single “line” longer than the whole 8 MB read cap is treated as unparseable log content: the offset jumps past it rather than retrying it forever.

19.6.4 Failed runs leave the checkpoint alone

The checkpoint advances only after the run's rows are safely written — offset, size, signature and record count are updated, and `lastError` is cleared, only on success. A failed run leaves all of it exactly where it was, so the next attempt re-reads the same lines instead of skipping them. Combine this with §19.4: a failure part-way through a bracketed import means some rows are already committed *and* will be read again on the retry.

19.6.5 Resetting the checkpoint

↺ **Reset checkpoint** appears in the Run dialog's checkpoint panel (and, for watchdog sources, in the source wizard). Its tooltip reads *Forget how far the file has been read, so the next import starts from the top*. It is disabled while a run is in progress and when there is no checkpoint to forget.

1. Open the Run dialog for the source (▷ on its badge), or edit the source and scroll to the watchdog panel.
2. Read the current checkpoint line so that you know how many records have been counted so far.
3. Click ↺ **Reset checkpoint**. The next import — manual *or* watchdog — starts at byte 0.

> **Warning** Resetting a checkpoint is not a database cleanup. Nothing is removed from `JOURNEYS`. Unless you delete the destination project first, the next import appends the file's events all over again.

The checkpoint is also discarded automatically when you change the source's file path (§19.3.3).

19.7 What the import writes

The File extractor reads the file line by line, applies the linked source type's regexes, and writes one `JOURNEYS` row per line that yields a complete event. A line produces an event only if a case id, a step *and* a timestamp were all captured and the timestamp normalises successfully; anything else counts as one skipped line. Events are pushed in batches of 500 rows, and progress is reported every 200 processed lines.

JOURNEYS column	Written value
`PROJECT_ID`	The project id chosen in the Run dialog (or configured on the watchdog).
`EVENT_ID`	The **hex MD5 digest** of the raw captured id — never the raw value.
`STEP`	The captured step value, replaced by a compound rule's step when one matches.
`STEP_ID`	Always `NULL`; it is filled in at load time by the main app.
`EVENT_TIME`	A real `TIMESTAMP`, normalised to `YYYY-MM-DD HH:MM:SS`.
`META_1`, `META_2`, `META_3`	The first three meta fields of the source type, in field order; `NULL` where the regex did not match. A fourth meta field is silently unused, and helper fields are never written at all.
`SAMPLE_SET`	Always the literal string `ORIGINAL`.

> **Warning** The `EVENT_ID` hashing is unconditional and cannot be switched off. Because the same raw id always produces the same digest, journeys still correlate correctly across events — but the original identifier (which is often a login or customer number) cannot be recovered from the database afterwards. Treat this as the pseudonymisation it is: if you need the raw id for business reporting, capture it additionally as a meta field, with all the data-protection consequences that carries.

After the events are written, the extractor creates the companion rows that the main app needs, always create-if-missing and never update-if-present:

`PROJECTS`
If the project id has no row, one is created with `TITLE` set to the project id and an empty description. The run log line is `created project '<id>'`.

`METAS`
Only when at least one meta field carries a business title. A row with `META_1_TITLE` … `META_3_TITLE` is created if the project has none. Existing rows are never rewritten, so editing meta titles later does not change a `METAS` row that already exists.

`STEPS`
Every step name seen in the run that has no row for this project gets one: empty description, a background colour chosen deterministically from a fixed ten-colour palette (the same step name always gets the same colour), white foreground, `SCORE` 0 so auto-created steps do not skew scoring, a deterministically chosen shape, `END_OF_PROCESS` false and no parent. The run log line is `created <n> new step(s): <names>`.

19.8 Where a run is recorded

Every import writes to three places you can consult afterwards:

- The **Abstraction layer** status card — state pill, target line, KPI tiles and the expandable **Run log** with lines such as `reading <path> → <schema>.JOURNEYS (project <id>)`, `pushed 500 events, skipped 2 unparseable line(s)` and `created project 'X'`. At most the 50 most recent messages are kept.
- The **ingestion pipeline** canvas and the KPI tiles, which accumulate every run that has actually started — manual or watchdog.

- The **admin log** under tag `DATA`: `import started: …`, `import finished: … — <detail>`, and on failure `import failed: … — <error>` at error level.

> **Tip** On success the destination connection and project are remembered on the source, so the Run dialog reopens on the same target next time. Renaming or otherwise editing the source in the wizard carries that memory forward — it is only lost when the connection assignment itself is revoked.

20 The Watchdog: Automatic Imports

> A manual run imports a file when you press a button. The *watchdog* imports it whenever it grows — unattended, without a browser, without anybody being signed in. This chapter explains what the watchdog does and how it differs from a manual run, how to switch it on for a file source and which destination it writes into, why that destination has to be stored with the source, how the authorisation is re-checked on every single poll, how to read and reset the checkpoint, where failures are recorded, and which deployment-wide switches an operator has over the whole loop. It is written for developers who configure sources in the integration console, and for the administrator or operator who runs the server they sit on.

Requires: **Developer** or **Administrator** — the watchdog is configured in the integration console (HTTP :8100, HTTPS :8463), which only these two roles may sign in to. The deployment-wide switches in section 20.10 are environment variables and require access to the server, not to any screen.

20.1 What the watchdog does

The watchdog is a single background loop inside the compute backend. It starts with the backend process and is cancelled when the backend shuts down. On every tick it lists **all file sources of all owners**, decides which of them are due, and polls each due source in turn: it opens the source's file, reads the complete lines that have been appended since that source's checkpoint, runs them through the linked source type exactly as a manual import does, writes the resulting events into the configured destination, and advances the checkpoint.

Everything the extractor does is identical to a manual run — the same regexes, the same compound step rules, the same timestamp normalisation, the same MD5 pseudonymisation of the case id, the same automatic creation of missing `PROJECTS`, `STEPS` and `METAS` rows, and the same transaction bracket (**rows per transaction**, default 5000). What differs is who triggers it, what it is allowed to read, and where the result is reported.

	Manual run (Run dialog)	Watchdog poll
Trigger	You press ▷ **Run extraction**.	The background loop, when the source's interval has elapsed.
Destination	Chosen in the dialog each time (remembered as a preselection).	Fixed: the connection and project id stored in the source's watchdog settings.
Session	Reuses your live database session if you happen to be connected to exactly that connection; otherwise the backend opens the connection itself.	Always opens the connection itself with its stored credentials, and closes it again after the poll.
Scope of the read	**Delta upload** is on by default but can be switched off to re-read the whole file from the top.	Always delta. There is no “read everything again” option; use **Reset checkpoint** (20.7) instead.
Result	Shown in the Run dialog, the status card, the KPIs and the pipeline.	Shown in the status card, the KPIs and the pipeline — and, when it fails, on the checkpoint and in the admin log.
Nothing new	A run still happens and reports *“Nothing new to import — the file has not grown since the last import.”*	No run is started at all. The checkpoint is merely refreshed and any old error cleared, so the pipeline stays quiet.

Both triggers share **one checkpoint per source**. That is deliberate: it is exactly what stops a manual top-up and the watchdog from importing the same line twice between them.

20.2 Switching the watchdog on for a source

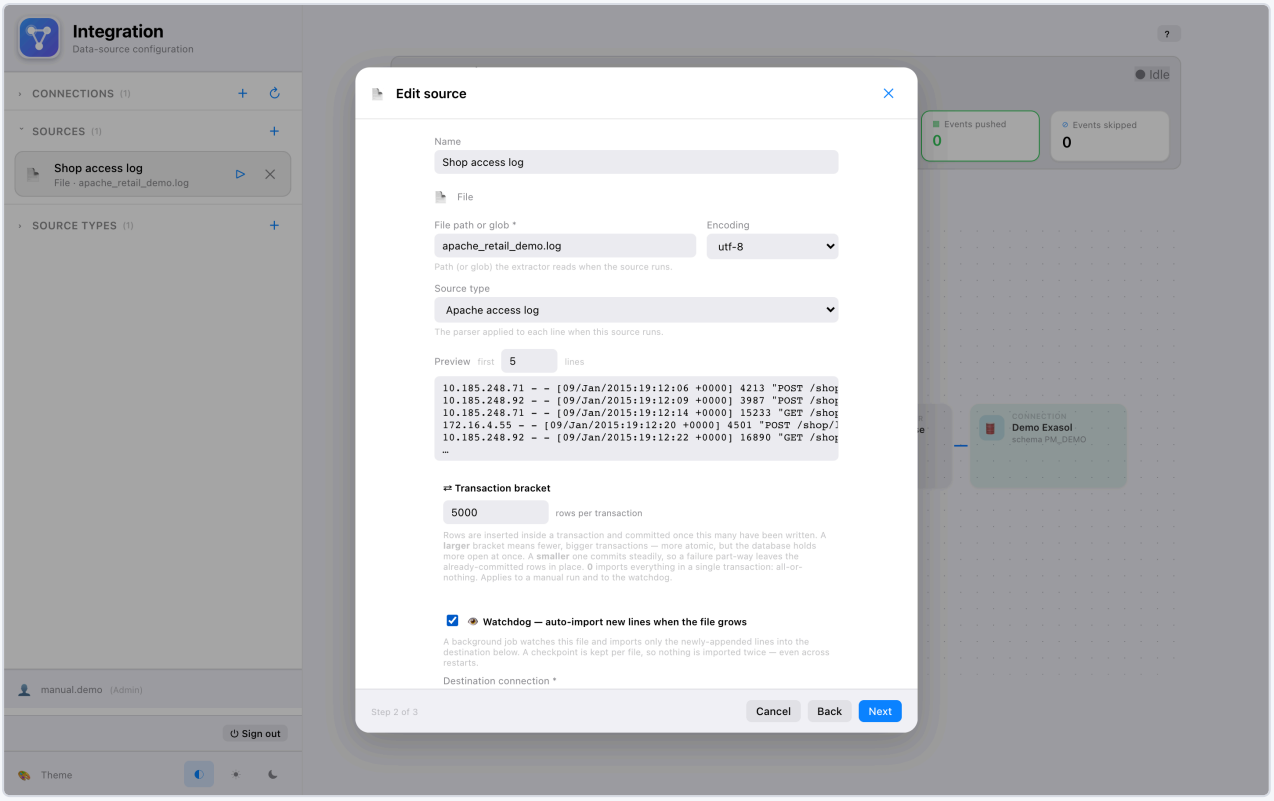


Figure 20.1 — The watchdog section of the source editor. Once enabled, you set the destination connection, the project id and the poll interval; the source then imports newly appended lines on its own.

The watchdog is configured inside the source itself, on step 2 of the source wizard, in the panel headed 👁 **Watchdog — auto-import new lines when the file grows**. The panel explains itself on screen: *“A background job watches this file and imports only the newly-appended lines into the destination below. A checkpoint is kept per file, so nothing is imported twice — even across restarts.”*

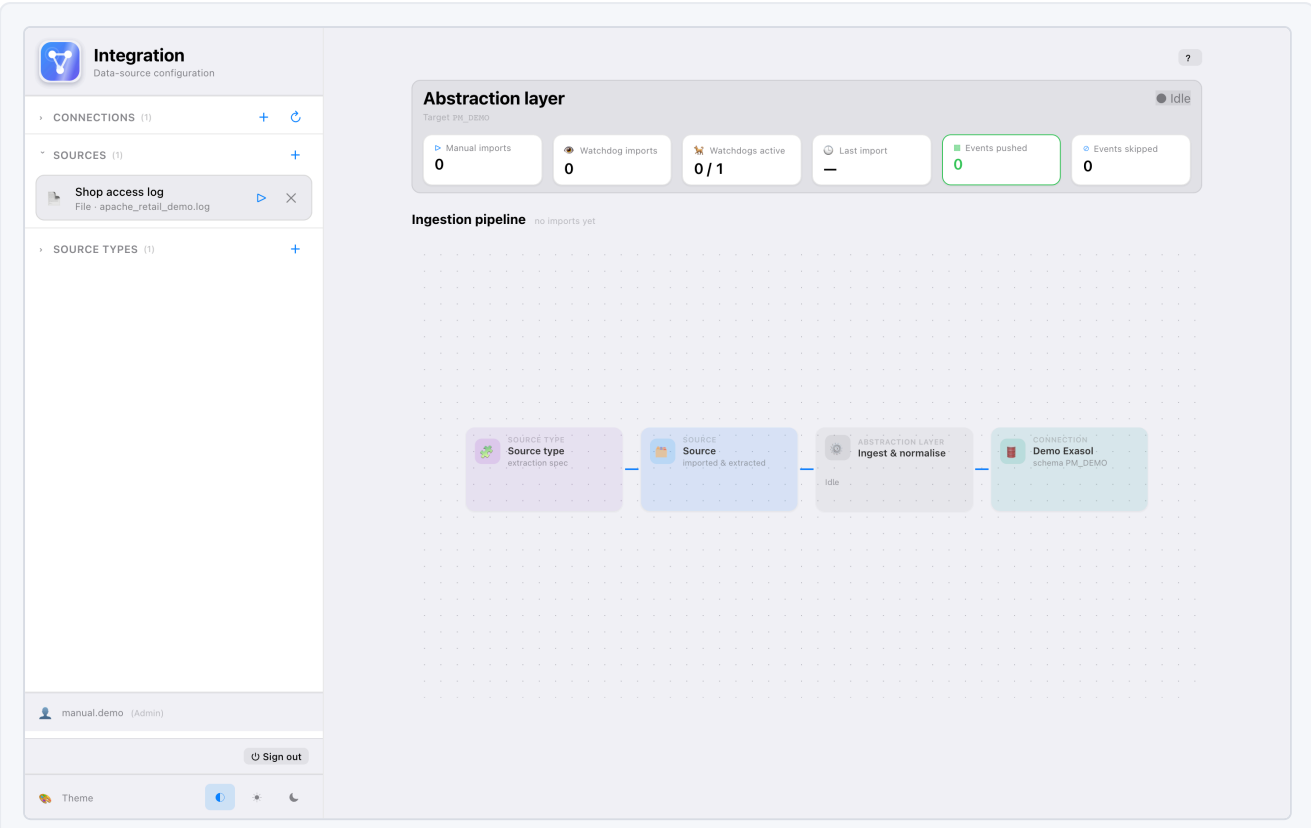


Figure 20.2 — The Sources section of the integration console sidebar. A source with the watchdog switched on carries an eye after its name.

1. In the sidebar, open the **Sources** section and click the source you want to automate. (To create one first, use the **+** button; see the previous chapter.) The wizard opens at step 2, **Configuration**.
2. Check that **File path or glob** points at the file that grows, and that a **Source type** is selected. A source with no source type linked can never be imported, by hand or by watchdog.
3. Tick the checkbox in the **Watchdog** panel. Three further fields appear.
4. Choose the **Destination connection**. The dropdown lists the connections assigned to you and starts on "— pick a connection —". If it is empty you will see *"No connections are assigned to you — ask an administrator, or create one in the main app."*
5. Type the **Project id** the events are written into, for example `LIVE-LOG`. The `PROJECTS` row is created for you on the first import if it does not exist.
6. Set **Every (seconds)** — the poll interval. The default is 30 and the minimum is 5.
7. Click **Next**, review step 3, then **Save changes** (or **Create source** for a new source).

Field	Required	Default	Notes
Watchdog checkbox	—	off	The per-source on/off switch. Only file sources have it.
Destination connection	yes	none	Must be a connection assigned to you, both now and at every later poll (20.5).
Project id	yes	none	Written into `JOURNEYS.PROJECT_ID` for every event of every poll.
Every (seconds)	—	30	Hard minimum 5. Values below 5, or missing, are raised to the bound by both the form and the backend.
Rows per transaction	—	5000	Configured in the ⇄ **Transaction bracket** panel of the same source and applies to watchdog polls exactly as to manual runs.

If you enable the watchdog but leave the destination or the project id blank, saving is refused and the wizard returns you to step 2 with *“The watchdog needs a destination connection and a project id.”* If the connection you picked is not assigned to you, the backend refuses the save with *“The watchdog's destination connection is not assigned to you.”*

> **Tip** Once the watchdog is on, the source badge in the sidebar gains a 👁 eye after its name (tooltip “Watchdog on — auto-imports new lines”), and its ▷ Run button turns green with the tooltip “Watchdog active — run an import now as well”. You can still run a watched source by hand at any time; because both share the checkpoint, the manual run simply takes whatever the watchdog has not yet picked up.

20.3 Why the destination is stored with the source

A manual run can ask you where to write, because you are standing in front of it. The watchdog cannot: it runs headless, with no signed-in user, no browser and no active database session. There is nothing to ask and nobody to ask. The destination therefore has to be part of the source's own configuration — the connection id and the project id are stored in the source's watchdog block, and the loop uses the connection's **stored credentials** (the username and password an administrator saved on the connection) to open the database for the duration of the poll and close it again afterwards.

This is the same mechanism the manual run uses when you name a connection you are not currently connected to, and the same one the administrative rebuild job uses. It is what makes an import possible with no session at all.

> **Note** Because the watchdog acts as the source's *owner*, its run updates that owner's status, KPIs and pipeline in the console — not the administrator's. If a developer's watchdog is importing, that developer sees it live in their own console; nobody else does.

20.4 The authorisation model

Storing credentials with a source raises an obvious question: what stops a developer from pointing a watchdog at a connection they were never given? The answer is that the assignment is the gate, and it is checked twice.

- **At save time** — enabling the watchdog on a connection that is not assigned to you is rejected outright with *"The watchdog's destination connection is not assigned to you."* This is a fail-fast convenience, not the real protection.
- **On every poll** — before it opens anything, the loop re-checks that the source's owner is still assigned to the destination connection. This check is repeated for the lifetime of the watchdog, on every single poll.

The consequence is the one administrators care about: **revoking a connection assignment stops that user's watchdogs immediately**. The very next poll fails with `The destination connection is not assigned to you.`, the checkpoint does not advance, and nothing further is written. There is no window in which a stored connection id keeps writing with credentials the user no longer has rights to.

> **Warning** Revoking access stops future imports; it does not remove what has already been imported. Rows written by earlier polls stay in the destination schema and must be removed there (for example from the **Projects** tab of the connection editor) if they should not remain.

20.5 When a poll happens

Two intervals combine.

The loop tick

The background loop wakes every `PMW_INTEGRATION_WATCHDOG_TICK` seconds — default **10**, hard minimum **5**. On each tick it lists every file source of every owner and decides which are due. This is deployment-wide.

The per-source interval

Every (seconds) in the source's watchdog panel — default **30**, minimum **5**. A source is due when it has never run, when its checkpoint has no usable timestamp, or when at least this many seconds have elapsed since the checkpoint was last touched.

Because the tick is the loop's own heartbeat, a source is never polled more often than the tick: setting an interval of 5 seconds on a deployment ticking every 10 gives you a poll roughly every 10 seconds, not every 5.

> **Note** "Since the checkpoint was last touched" means exactly that: *every* checkpoint write updates the timestamp, including a poll that found nothing new and including a manual run. The interval therefore measures the time since the last import activity of any kind on that source, not since the last import that actually wrote rows.

20.6 What one poll reads

A poll reads only **complete** lines beyond the stored byte offset; a partial trailing line that the writing application has not finished is left for the next poll. Blank lines are skipped and each line is truncated to its first 2000 characters, exactly as in a manual run.

A single poll reads at most **8 MB** of new bytes. A larger backlog — typically the first poll on a file that already existed — is drained across successive polls, 8 MB at a time, until the file is caught up. This is normal and needs no intervention; it simply means the first few polls after enabling a watchdog on a large file import a slice each.

Rotation and replacement are detected automatically. If the file has shrunk below the stored offset (truncation, or a fresh file at the same name), reading restarts from byte 0. If the file's head has changed — the watchdog fingerprints the first 64 bytes — while the size did not shrink, the stored offset is assumed to point into unrelated content and the file is likewise re-read from byte 0.

> **Warning** Events are **appended** and never de-duplicated. Anything that makes the watchdog re-read from byte 0 — a detected rotation, a checkpoint reset, a change of file path — imports that file's events into the project a second time. The database has no idea it has seen them before.

20.7 The checkpoint status display and Reset

The checkpoint is the watchdog's memory: how far into the file it has read, how large the file was, the fingerprint of its head, how many events it has imported in total, when it was last touched, and the last error if there was one. It is shown in two places.

In the **source wizard**, directly under the watchdog fields (for a source that already exists), you see one of:

Display	Meaning
⚠ *<last error>* (red)	The most recent poll failed. See 20.8 for the wording and the causes.
✓ *<n> records imported · last checked <date and time>*	Healthy. The count is cumulative across every poll and every manual delta run of this source; the time is your local time.
Not run yet — the watchdog will pick it up shortly.	No checkpoint exists. The next due poll reads the file from the start.

In the **Run dialog** the same checkpoint appears in more detail: *"✓ <n> records imported · read to <bytes> of <bytes> bytes · last <date and time>"*, or the red error, or *"No checkpoint yet — this is the first import of this file."* The two byte figures tell you whether a backlog is still draining: while "read to" is smaller than the file size, more is pending.

20.7.1 Resetting the checkpoint

Both places carry a ↺ **Reset checkpoint** button (in the Run dialog it is disabled while a run is in flight or when no checkpoint exists; its tooltip reads "Forget how far the file has been read, so the next import starts from the top").

1. Open the source in the wizard, or open the Run dialog from the ▷ button on the source badge.
2. Click ↺ **Reset checkpoint**. The stored offset, size, fingerprint, record count and last error are discarded.
3. The next import — whether the next watchdog poll or a manual run — starts at byte 0 of the file.

> **Warning** A reset forgets a position; it deletes nothing from the database. Re-importing after a reset writes the file's events into the project a **second time**. If you want a clean reload, delete the project in the destination schema first (connection editor, **Projects** tab), then reset.

> **Note** Changing a source's **File path or glob** deletes its checkpoint automatically — the old offset means nothing in a different file — so the next poll reads the new file from the beginning. Renaming the source, or editing any other field, keeps the checkpoint.

20.8 When a poll fails

A watchdog runs unattended, so a silent failure would go unnoticed. Every failure is therefore recorded in two places at once: on the source's **checkpoint**, as the last error you see in the wizard and the Run dialog, and in the **admin log** as a warning under the tag `DATA`, operation `import`, attributed to the source's owner. The log line reads:

```
watchdog import failed: '<source name>' — <message>
```

The offset does **not** advance on failure, so the next poll retries exactly the same lines. One misconfigured source never stops the other sources and never stops the loop.

Recorded message	Cause and remedy
`Watchdog is missing a file path, project id or connection.`	The source's watchdog block is incomplete. Reopen the wizard and fill in the starred fields.
`The source has no source type linked.`	No source type is selected, or the one it referenced was deleted — deleting a source type does not cascade. Pick a source type on step 2.
`The destination connection is not assigned to you.`	The owner's assignment was revoked. An administrator must reassign the connection, or you must choose a different destination.
`The watchdog's destination connection no longer exists.`	The connection was deleted. Choose another destination.
`The destination connection has no target schema.`	The connection has no schema configured. An administrator (or the connection's owner) must set one.
`No file path given.` / `File not found.` / `Not a regular file.`	The configured path is blank, missing, or is a directory rather than a file.
`That path is outside the allowed sources directory.`	The path escapes the sandbox root (including via `..` or a symlink). Move the file into the sources directory, or use a relative path.
`Could not connect: <error>`	The destination database could not be opened — host down, credentials rejected, TLS refused.
`Import failed: <error>`	The extraction or the write failed. One common case is `An extraction is already running for this user.` — see 20.9.

Successful polls are logged too. When new lines were imported, the admin log records, at usage level under tag `DATA`:

```
watchdog imported <n> new event(s) from '<source>' into <connection>/<schema> (project <id>)
```

Problems with the loop itself, rather than with one source, appear as `watchdog: could not list sources: <error>` and `watchdog: source '<name>' poll failed: <error>`.

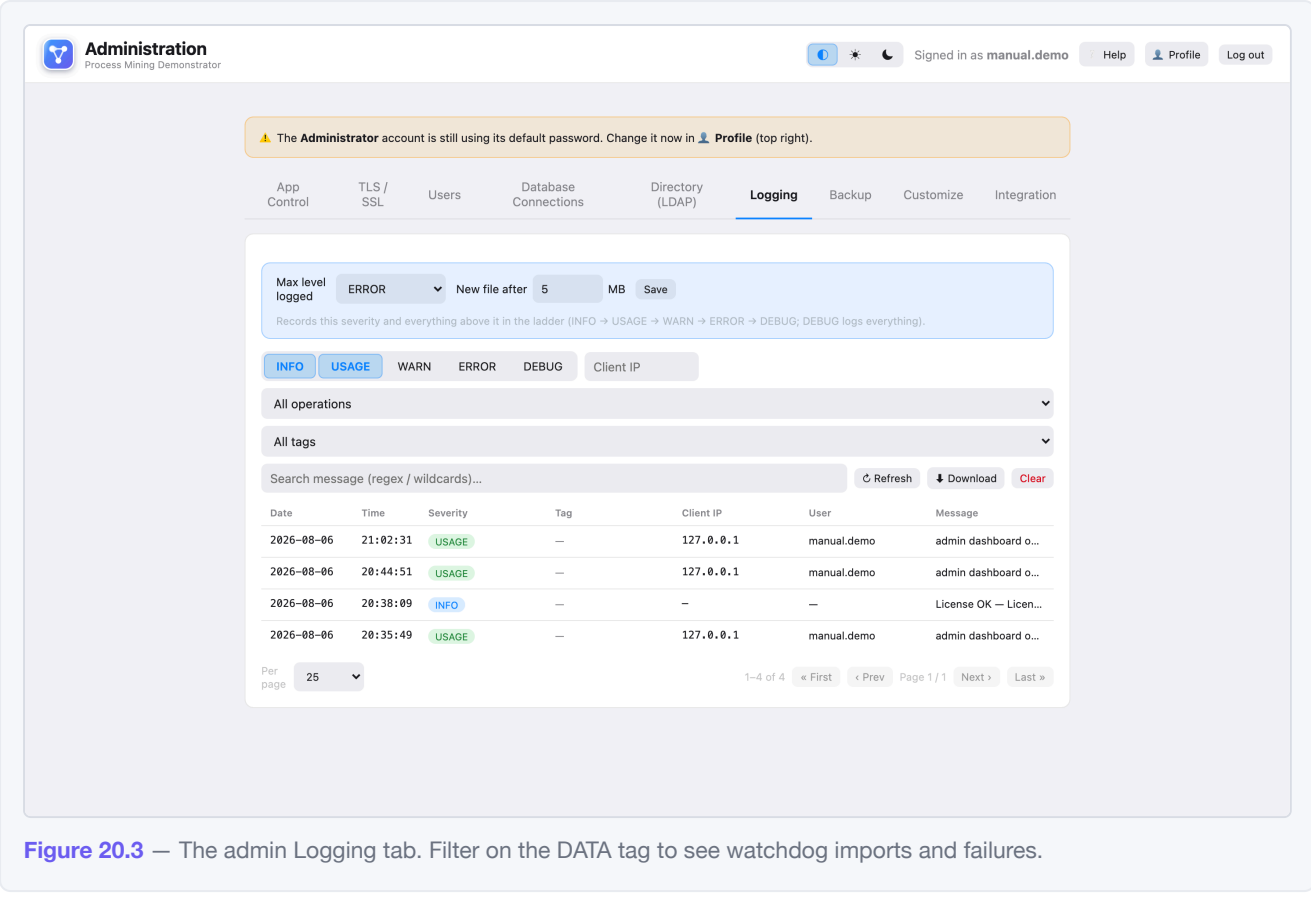


Figure 20.3 — The admin Logging tab. Filter on the DATA tag to see watchdog imports and failures.

> **Warning** If **rows per transaction** is greater than 0 and a poll fails part-way through, the brackets already committed stay committed, but the checkpoint does not advance. The next poll therefore re-reads those lines and can write them a second time. Set the transaction bracket to **0** (“one transaction for the whole import”) if a failed import must be all-or-nothing; the cost is that the database holds the entire import open.

20.9 Watching the watchdog from the console

A watchdog import is a normal run of the abstraction layer, tagged with the trigger `watchdog`, so it shows up everywhere a manual run does.

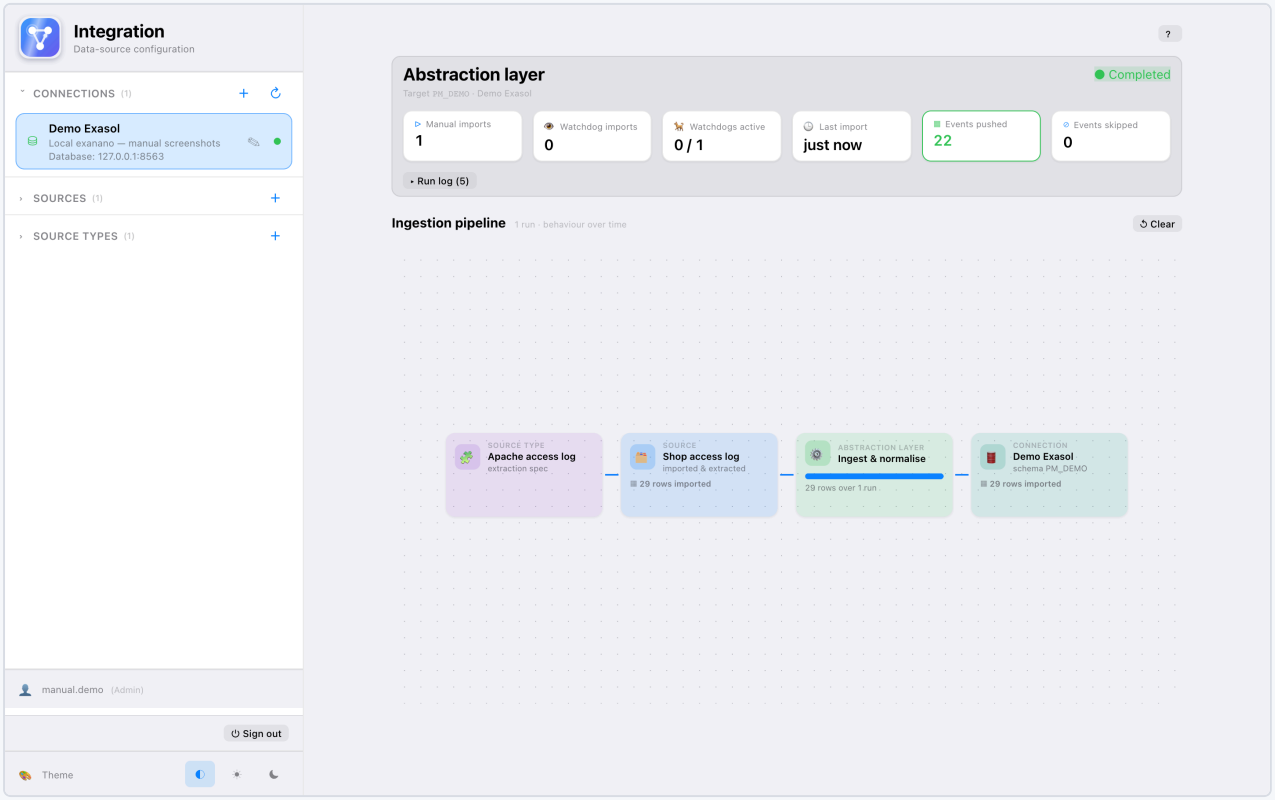


Figure 20.4 — The integration console: the Abstraction layer status card, the ingestion KPIs and the live pipeline.

- The **Abstraction layer** card switches to **Running** while a poll imports, then **Completed** or **Failed**, and its **Run log** carries lines such as *“watchdog: 14 new line(s) from /var/log/app/access.log → …”* and *“pushed 500 events, skipped 2 unparseable line(s)”*. A failed run is repeated as the banner **“Last run failed: <error>”**.
- The KPI strip separates the two triggers: **Manual imports** (▷) and **Watchdog imports** (👁) count the recorded runs of each kind, while **Watchdogs active** (🐕) shows a live “<active> / <total>” — your file sources with the watchdog switched on out of all of them — and turns green when at least one is active.
- The pipeline flowchart adds an edge for each combination that has actually run, so a watchdog's source type, source and destination appear on the canvas alongside your manual imports, with the cumulative row counts on the source and connection nodes.

Note Only **one extraction runs per user at a time**. If you press ▷ **Run extraction** while your own watchdog is mid-import (or the other way round), the second one is refused with `An extraction is already running for this user.` For a manual run this appears as an error in the Run dialog; for the watchdog it is recorded on the checkpoint as `Import failed: An extraction is already running for this user.` and re-tried automatically on the next poll. Different users are unaffected by each other.

20.10 The deployment-wide switch

Requires: **Administrator** / server operator — these are environment variables read at backend start, not settings in any interface.

Variable	Default	Effect
`PMW_INTEGRATION_WATCHDOG`	**on** (`1`)	The global on/off switch for the whole loop. Truthy values are `1` , `true` , `yes` , `on` . When it is off, the loop exits immediately at start-up and *no* source is ever polled, whatever its own watchdog setting says.
`PMW_INTEGRATION_WATCHDOG_TICK`	**10** seconds	The base loop tick (see 20.5). Hard minimum 5 seconds. Lower it if you need per-source intervals near the 5-second floor to be honoured; raise it to reduce idle polling on a busy server.

```
# disable all automatic imports on this backend
PMW_INTEGRATION_WATCHDOG=0

# or keep them, but only look every 30 seconds
PMW_INTEGRATION_WATCHDOG_TICK=30
```

The console does not hide the fact that the loop is off. The status endpoint reports it, and the **Watchdogs active** KPI tile relabels itself to **Watchdogs (off)** and turns orange — so a source whose watchdog is "on" but that never actually polls is visible at a glance instead of quietly doing nothing.

> **Tip** Switching the loop off is the cleanest way to freeze all automatic ingestion during a database migration or a maintenance window. Per-source settings and checkpoints are untouched; when you switch it back on, each due source resumes from exactly where it stopped.

20.11 Operational guidance: log rotation

The watchdog is normally pointed at a file that a live application keeps appending to, which in practice means a file that some rotation scheme keeps renaming, compressing or truncating. The detection described in 20.6 handles the common cases, but a few operational rules keep it reliable.

- **Poll considerably faster than you rotate.** The watchdog only ever looks at the configured path. Lines that are written to the active file and then rotated away before the next poll are never read, because the file at that path is a different file by the time the watchdog gets there. An interval of 30 seconds against an hourly rotation is safe; an interval of ten minutes against a rotation every five is not.
- **Point the source at the stable name.** Configure the path of the live file (`access.log`), not of a rotated generation (`access.log.1`). The label on the field says "File path or glob", but the path is resolved to one regular file — wildcards are not expanded, so a pattern intended to match rotated generations simply fails with `File not found.`
- **Copy-truncate and create-new are both detected.** Truncation is caught by the size dropping below the stored offset; a brand-new file at the same name is caught by the changed fingerprint of its first 64 bytes. Either way the watchdog restarts at byte 0 of the new file.
- **Beware very small files.** A file shorter than 64 bytes cannot be fingerprinted. If such a file is replaced in place by another one of the same or greater size, the replacement goes undetected and the watchdog carries on reading from the old offset. Files of that size are rarely worth watching in the first place.
- **Expect a burst after every rotation.** Because a detected rotation re-reads from byte 0, the poll after a rotation imports the whole of the new file so far. That is correct for a genuinely fresh file — but if your ro-

tation scheme seeds the new file with content that was already imported, those events are appended again.

- **Keep the sandbox in mind.** Source paths are confined to the integration files directory unless the deployment has explicitly opted out. If your log lives elsewhere, arrange for the rotation scheme to write or link into that directory rather than loosening the sandbox.
- **Watch the skipped counter.** A rotation that changes the log format (a new application version, a different formatter) produces lines that the linked source type's regexes no longer match. The import will not fail; it will succeed with everything skipped. The orange **Events skipped** KPI is your early warning.

> **Note** Nothing about the watchdog depends on the console being open. It lives in the compute backend, polls on its own timetable and writes with stored credentials. Closing the browser, signing out, or never signing in at all changes nothing — only the two environment switches in 20.10, the per-source checkbox, and the connection assignment do.

PART V

Reference

21 Reference: The Exasol Data Model

> Everything the application shows you comes out of one Exasol schema. This chapter is the complete reference for that schema: the six tables, every column with its type and default, what the `SAMPLE_SET` column means and how samples and simulations live alongside the original data, how `EVENT_ID` is formed in the demo datasets and what that means when you want to trace a journey back to a source record, how the process map is derived from the event log, and what changes when the optional `TRANSITIONS_RAW` table is in play. It closes with a summary of the provisioning DDL, guidance on sizing, and the list of database privileges the application needs. It is written for database administrators, ETL developers and anyone who has to load, size or govern the data — not for everyday chart users.

The schema at a glance

The application reads and writes six tables inside a single Exasol schema. The schema name is part of each connection definition, so different connections may point at different schemas on the same or different servers. Five tables are the canonical set; the sixth, `TRANSITIONS_RAW`, is an optional performance table that the application builds for itself.

Table	Grain	Written by	Created by
`PROJECTS`	One row per project	Demo generator; project deletion	Provisioning
`JOURNEYS`	One row per (event, step) — the event log	Your ETL; sampling; demo generator	Provisioning
`STEPS`	One row per (project, step)	The Step editor; demo generator	Provisioning
`METAS`	One row per project	Demo generator	Provisioning
`NOTES`	One row per note thread	The Notes feature	Lazily, on first use
`TRANSITIONS_RAW`	One row per consecutive step pair	The rebuild job	The rebuild job

The source tables normally arrive with your own data — you load them from your ETL and the application only reads them. The application can also create them itself; see Schema provisioning. All provisioning DDL uses `IF NOT EXISTS`, so re-running it never disturbs existing data and never alters an existing table.

PROJECTS

One row per process-mining project. A project is the top-level unit of separation: every other table carries `PROJECT_ID` and every query the application issues is scoped to one project.

Column	Type	Constraint / default
`PROJECT_ID`	VARCHAR(100)	NOT NULL, PRIMARY KEY
`TITLE`	VARCHAR(500)	default `''`
`DESCRIPTION`	VARCHAR(2000)	default `''`

The project picker lists projects ordered by `TITLE`. A project with an empty title is displayed under its `PROJECT_ID` instead.

> **Note** A `PROJECTS` row is not strictly required for data to be visible. The project listing in the connection editor reports a project if it exists in *either* `PROJECTS` or `JOURNEYS`, so an orphan on either side is visible rather than silently invisible. This is deliberate: it lets you spot a load that wrote events without registering the project, and a project registration whose events never arrived.

JOURNEYS — the event log

One row per (event, step). This is the only large table in the schema, and the only one your ETL normally writes.

Column	Type	Constraint / default	Meaning
`PROJECT_ID`	VARCHAR(100)	NOT NULL	Owning project
`EVENT_ID`	VARCHAR(200)	NOT NULL	The journey (case) identifier
`STEP`	VARCHAR(500)	NOT NULL	The activity name
`STEP_ID`	DECIMAL(18,0)	—	Orders steps that share the same `EVENT_TIME`
`EVENT_TIME`	TIMESTAMP	NOT NULL	When the step happened
`META_1`	VARCHAR(1000)	—	Free-form attribute 1
`META_2`	VARCHAR(1000)	—	Free-form attribute 2
`META_3`	VARCHAR(1000)	—	Free-form attribute 3
`SAMPLE_SET`	VARCHAR(20)	default `'ORIGINAL'`	Which data set the row belongs to

A journey is the set of all rows sharing one `EVENT_ID` within one project and one sample set. Steps are ordered by `EVENT_TIME`, and where two steps carry the same timestamp — common when a source system stamps a whole batch at once — by `STEP_ID`. If your source has no natural sub-second ordering, populate `STEP_ID` with an increasing counter per journey; otherwise the order of same-second steps is arbitrary and the derived process map will show pairs that never occurred in that order.

Physical layout

When the application creates `JOURNEYS` itself, the DDL declares a physical layout tuned for the queries the map issues:

`DISTRIBUTE BY EVENT_ID`

Co-locates all events of one journey on one cluster node, so the transition query's `LEAD()` window and every `GROUP BY EVENT_ID` filter run node-local instead of shuffling rows across the cluster. `EVENT_ID` is high-cardinality, so the rows still spread evenly.

`PARTITION BY EVENT_TIME`

Lets Exasol prune by date range. It helps the selection of event IDs that are active in the filter window more than it helps the final scan. Exasol requires the partition column to differ from the distribution column, which is why the two are split this way.

Warning Provisioning never alters an existing table. Because all DDL is written with `IF NOT EXISTS`, a `JOURNEYS` table that already exists keeps whatever layout it has — you get no error and no warning, only slower queries. A DBA must apply the layout by hand:

```
ALTER TABLE JOURNEYS DISTRIBUTE BY EVENT_ID;
ALTER TABLE JOURNEYS PARTITION BY EVENT_TIME;
```

EVENT_ID: type tolerance and hashing

The declared type of `EVENT_ID` is `VARCHAR(200)`, but in real deployments it varies — sites use `VARCHAR`, `HASHTYPE` or `DECIMAL` depending on what their source systems produce. The application tolerates all of them. This is also why `TRANSITIONS_RAW` is deliberately built with `CREATE TABLE AS SELECT` rather than an explicit column list: its `EVENT_ID` then inherits the exact type of the `JOURNEYS` column. Hard-coding a type there would make the semi-join against `JOURNEYS` raise Exasol's *Incomparable Types* error on any site whose column is not a `VARCHAR`.

The three built-in demo datasets do **not** use readable order numbers as journey identifiers. Each demo `EVENT_ID` is the **MD5 hex hash** of a synthetic key:

Dataset	Project	Synthetic key hashed into EVENT_ID
Retail — Online Bookstore	`BOOKSTORE`	`ORD-000001`, `ORD-000002`, …
Finance — Online Credit Application	`CREDIT`	`CRA-000001`, `CRA-000002`, …
Transportation — Flight Booking & Management	`FLIGHTS`	`FLT-000001`, `FLT-000002`, …

In each case the key is the three-letter prefix plus a six-digit sequence number, and the value stored in `EVENT_ID` is the MD5 hash of that string in hexadecimal.

Note The consequence for tracing back to a source record: an `EVENT_ID` you read off the Individual Journey view or copy out of a variant listing is a one-way hash, not an order number. You cannot invert it. To find the underlying case you must hash forward — compute the MD5 of the candidate key and compare — or keep your own mapping table. The same reasoning applies to production loads: if your ETL hashes or pseudonymises the case key on the way in (a common requirement when the event log leaves the source system's access controls behind), plan for the lookup direction you will actually need before you load, because the application only ever shows you the stored identifier.

> **Tip** If you need operators to be able to identify a case from the chart, carry a human-readable reference in one of the three `META` columns. Meta values are displayed and filterable; `EVENT_ID` is only ever matched exactly.

SAMPLE_SET: originals, samples and simulations

`SAMPLE_SET` is what lets one `JOURNEYS` table hold your real data and several derived working sets side by side. It takes four values.

Value	Meaning
`ORIGINAL`	The source data as loaded. This is the default for new rows and cannot be deleted from the application.
`SAMPLE_1`	First derived working set.
`SAMPLE_2`	Second derived working set.
`SAMPLE_3`	Third derived working set.

Everywhere the application filters by sample set, `NULL` is treated as `ORIGINAL`. The generated predicate is `SAMPLE_SET = 'ORIGINAL' OR SAMPLE_SET IS NULL`, so a legacy table whose rows were loaded before the column existed continues to work unchanged and its rows count as original data.

> **Warning** Whenever a count differs from what you expect, check for `NULL` values in `SAMPLE_SET`. They are invisible in the sample picker but they are counted as `ORIGINAL` everywhere else.

The quiet migration

If the column is missing, the application adds it on demand rather than failing. It issues, ignoring failures (for instance because the column already exists):

```
ALTER TABLE JOURNEYS ADD COLUMN SAMPLE_SET VARCHAR(20) DEFAULT 'ORIGINAL';
UPDATE JOURNEYS SET SAMPLE_SET = 'ORIGINAL' WHERE SAMPLE_SET IS NULL;
```

Because the failures are ignored, an account without `ALTER` rights simply leaves the table as it is — the application then keeps working through the `IS NULL` branch of the filter, but no samples can be created.

Samples are physical copies

Creating a sample does not create a view, a flag or a reference. It `INSERT`s the chosen journeys' rows back into `JOURNEYS` with the new `SAMPLE_SET` value, in batches of 200 event IDs per `INSERT`. Every row of every selected journey is duplicated. Three sample sets covering a quarter of the log each therefore add roughly 75 % to the size of the table.

Three sampling strategies are offered: `random`, `temporal` and `pathDiverse`. Whichever you choose, the storage behaviour is the same.

Deleting a sample deletes only the rows carrying that `SAMPLE_SET` value. Deleting `ORIGINAL` is not possible through the application — the call is a no-op — so a mis-click can never destroy your source data through the sampling controls.

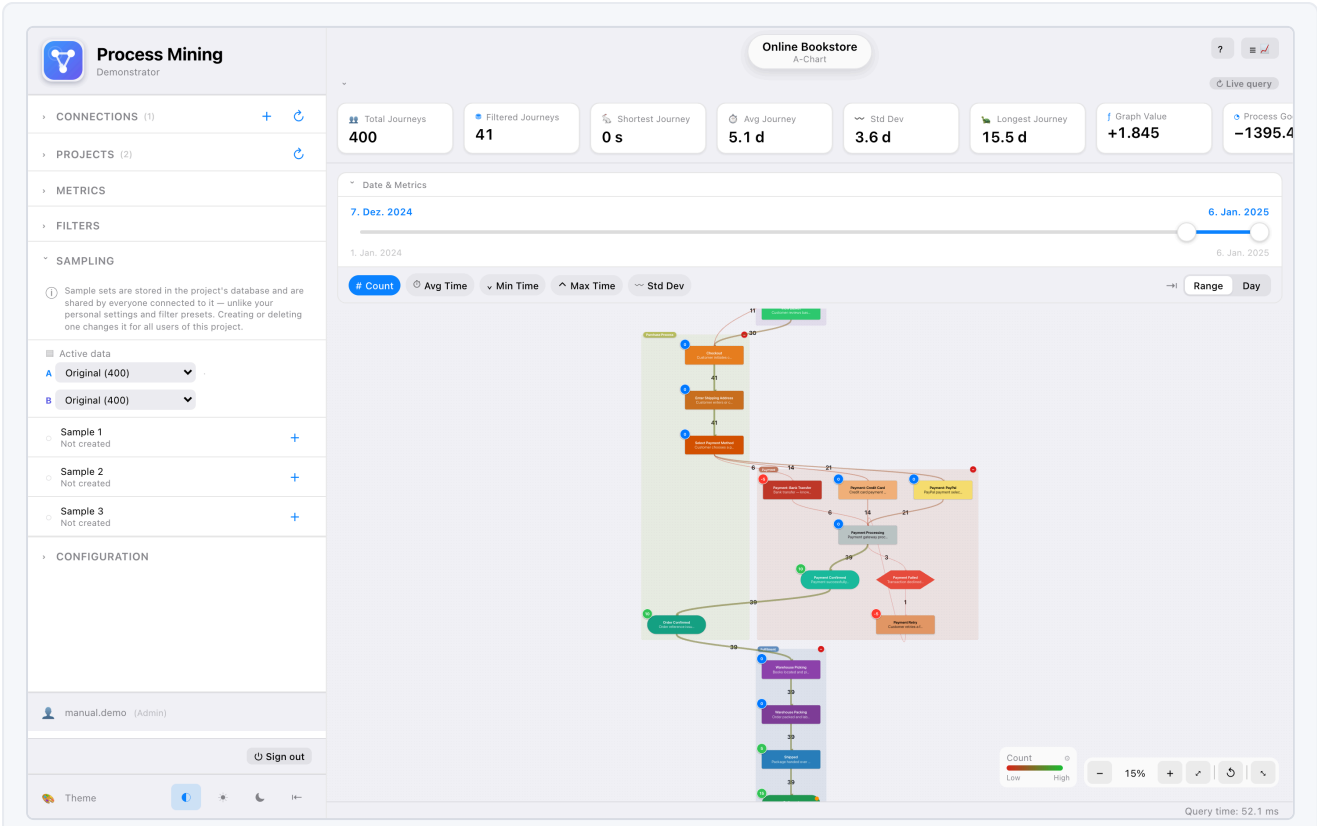


Figure 21.1 — The sampling controls. Each sample you create is a physical copy of the selected journeys' rows inside `JOURNEYS`.

> **Warning** Sample deletion is irreversible. There is no recycle bin and no undo. The rows are removed with a `DELETE` and can only be recovered by re-running the sampling — which, for the `random` strategy, will not produce the same selection.

STEPS

Presentation and scoring for each step, edited through the application's Step editor. This table is what makes the map look like your process rather than a set of grey boxes.

Column	Type	Constraint / default
`PROJECT_ID`	VARCHAR(100)	NOT NULL, part of the primary key
`STEP`	VARCHAR(500)	NOT NULL, part of the primary key
`DESCRIPTION`	VARCHAR(2000)	default `''`
`BG_COLOR`	VARCHAR(30)	default `''`
`FG_COLOR`	VARCHAR(30)	default `''`
`SCORE`	DECIMAL(18,2)	nullable
`SHAPE`	VARCHAR(50)	default `''`
`END_OF_PROCESS`	BOOLEAN	default FALSE
`BELONGS_TO`	VARCHAR(500)	nullable — the step group

Where a value is missing or is not a string, the reader falls back to: the step name for the description, `blue` for the background colour, `white` for the foreground colour and `stadium` for the shape. A step that appears in the transitions but has no `STEPS` row at all is synthesised on the fly in grey (`gray` background, `white` foreground, `stadium` shape). Grey nodes on the map are therefore a reliable signal that your event log contains an activity nobody has described yet.

> **Warning** `STEPS` must hold exactly one row per (`PROJECT_ID` , `STEP`). The application caches the table per user and per project and joins it to the event log to attach scores; duplicate rows would multiply journey rows in that join and inflate every score-derived figure. If you load `STEPS` from an ETL rather than through the Step editor, enforce uniqueness there.

The cache is refreshed when a step is edited in the Step editor, when the user disconnects or connects to another connection (it is never carried across connections), when a manager deletes a project from the connection editor, and when demo content is generated. A `STEPS` row changed behind the application's back — by a direct SQL update, for example — is picked up only after one of those events.

METAS

The display labels for the three free-form attribute columns of `JOURNEYS` . One row per project.

Column	Type	Constraint / default
`PROJECT_ID`	VARCHAR(100)	NOT NULL, PRIMARY KEY
`META_1_TITLE`	VARCHAR(500)	default `''`
`META_2_TITLE`	VARCHAR(500)	default `''`
`META_3_TITLE`	VARCHAR(500)	default `''`

If a title is empty or the row is absent, the interface falls back to generic meta labels. Filling this table is the cheapest single improvement you can make to a new project: it turns "Meta 1" into "Payment Method"

everywhere in the filter bar, the statistics and the exports.

NOTES

`NOTES` is not part of the provisioning set. It is created on demand with `CREATE TABLE IF NOT EXISTS` the first time anyone uses notes, and then quietly migrated — each migration statement's failure is ignored — to add the `IS_SHARED`, `EDITED_BY`, `IMPORTANCE`, `RESOLVED` and `TITLE` columns and to widen `NOTE` from the legacy `VARCHAR(8000)` to `VARCHAR(100000)`, so that append-only comment threads have room to grow.

Column	Type	Constraint / default
`ID`	VARCHAR(36)	NOT NULL, PRIMARY KEY
`PROJECT_ID`	VARCHAR(100)	NOT NULL
`NOTES_DATE`	TIMESTAMP	NOT NULL — created at
`EDITED_DATE`	TIMESTAMP	nullable
`NOTE_USER`	VARCHAR(200)	default `''` — the author's login name
`NOTE`	VARCHAR(100000)	default `''` — the thread text, newest comment prepended on top
`IS_SHARED`	BOOLEAN	default FALSE
`EDITED_BY`	VARCHAR(200)	default `''`
`IMPORTANCE`	VARCHAR(20)	default `'NORMAL'` — one of NORMAL, INFO, IMPORTANT, URGENT
`RESOLVED`	BOOLEAN	default FALSE
`TITLE`	VARCHAR(500)	default `''`
`TARGET_TYPE`	VARCHAR(10)	default `'node'` — `node` or `edge`
`TARGET_FROM`	VARCHAR(500)	default `''`
`TARGET_TO`	VARCHAR(500)	nullable — set for edge notes
`FILTER_SNAPSHOT`	VARCHAR(4000)	JSON snapshot of the filters in force when the note was written

Any `IMPORTANCE` value other than the four listed normalises to `NORMAL` on read. Visibility fails closed: a user sees their own notes plus unowned notes (`NOTE_USER = ''`) and shared ones. Only the author may delete a note; when sign-in is disabled the author is the empty string, so in anonymous mode every note is unowned and visible to everyone.

How the process map is derived from the event log

Nothing in the schema stores a process model. The directly-follows graph you see on the canvas is computed from `JOURNEYS` every time you load or filter a map.

The live query

The derivation is a self-join expressed as a window function. Conceptually the application does this:

1. Select the rows of the chosen project and sample set that survive the current filters.
2. Within each journey, order the rows by `EVENT_TIME` and then by `STEP_ID`, and use `LEAD()` to pair every row with its immediate successor. Each pair is one observed transition, and the difference between the two timestamps is its duration.
3. Aggregate the pairs by (from-step, to-step), producing `COUNT` and the `AVG`, `MIN`, `MAX` and `STDDEV` of the gap. Those five figures are what the edge labels, the metric colouring and the duration statistics display.

```
-- conceptually equivalent to
SELECT FROM_STEP, TO_STEP,
       COUNT(*), AVG(DUR), MIN(DUR), MAX(DUR), STDDEV(DUR)
FROM (
  SELECT STEP AS FROM_STEP,
         LEAD(STEP)       OVER (PARTITION BY EVENT_ID ORDER BY EVENT_TIME, STEP_ID) AS
TO_STEP,
         LEAD(EVENT_TIME) OVER (PARTITION BY EVENT_ID ORDER BY EVENT_TIME, STEP_ID) -
EVENT_TIME AS DUR
  FROM JOURNEYS
  WHERE PROJECT_ID = ? AND (SAMPLE_SET = ? OR ...)
)
WHERE TO_STEP IS NOT NULL
GROUP BY FROM_STEP, TO_STEP;
```

The last row of every journey pairs with nothing and drops out; that is why a journey of *n* steps contributes *n* – 1 transitions. On large logs the window function dominates the cost of the query, and it is paid again on every filter change.

TRANSITIONS_RAW: what the materialised table changes

`TRANSITIONS_RAW` performs the pairing once and stores the result, so that the map query at request time is a pure `GROUP BY` aggregate with no window function at all. The admin interface describes it as "much faster for interactive filtering on large logs".

Column	Meaning
`PROJECT_ID`	Owning project
`EVENT_ID`	The journey — type inherited from `JOURNEYS`
`FROM_STEP`	The predecessor activity
`TO_STEP`	The successor activity
`FROM_TIME`	Timestamp of the predecessor
`TO_TIME`	Timestamp of the successor
`DUR_SECS`	Precomputed gap in seconds
`SAMPLE_SET`	Which data set the pair belongs to

The table carries `PROJECT_ID`, `SAMPLE_SET` and `EVENT_ID`, which is exactly what the filter clauses need, so the same filters apply unchanged. Build mechanics worth knowing:

- It is built with `CREATE OR REPLACE TABLE … AS SELECT` into a staging table `TRANSITIONS_RAW_STAGE`, so every column type — critically `EVENT_ID` — inherits from `JOURNEYS` exactly.
- The pairing covers **every project and every sample set in one pass**. The window partitions by (`PROJECT_ID`, `SAMPLE_SET`, `EVENT_ID`), because an `EVENT_ID` is only unique within one project and one sample set.
- After the build the job attempts `ALTER TABLE … DISTRIBUTE BY EVENT_ID` on a best-effort basis — a failure there never aborts the rebuild, since distribution is only an optimisation — then counts the rows, then swaps the table in with `DROP TABLE IF EXISTS TRANSITIONS_RAW` followed by a `RENAME TABLE`.
- Readers are never blocked by the build. The swap leaves a sub-millisecond window in which no live table exists, during which reads simply fall back to the live query.
- The operation is idempotent and safe to re-run. It requires a schema name and returns `{ok, error, rows, built_at}`, where `rows` is the pair count and `built_at` an ISO-8601 UTC timestamp.

Reading is equally forgiving. With the per-connection flag **Use pre-materialized transitions** switched on, *any* read failure — table missing, never built, mid-swap — is non-fatal: the application falls back to the live query so that the map never breaks, and logs a warning under the operation `materialize`. The detail-pane pill tells you which path served the map: "⚡ Pre-materialized", "⚠ Live (not built)" or "↻ Live query".

> **Warning** `TRANSITIONS_RAW` is a snapshot and goes stale the moment `JOURNEYS` changes. Loading data, generating demo content and creating or deleting a sample do **not** rebuild it automatically. If the table still exists you will see pairs computed from the old log; if it does not, you get the live fallback. Make the token-authenticated rebuild call the last step of every ETL run that touches `JOURNEYS`.

Schema provisioning DDL summary

> Requires: **Administrator**, **Power user** or **Developer** in the application — *and* a database account permitted to CREATE SCHEMA and CREATE TABLE

Create schema & tables is available in the admin panel (Connections tab → connection editor → **Database** inner tab) and in the main application's connection editor sheet (**Database / LLM Details** tab). Both surfaces run the same backend routine:

1. Connect *without* opening a schema, because the target schema may not exist yet.
2. `CREATE SCHEMA IF NOT EXISTS` for the schema named in the form.
3. `OPEN SCHEMA`.
4. Run each table's `CREATE TABLE IF NOT EXISTS` in the order `PROJECTS`, `JOURNEYS`, `STEPS`, `METAS`, `NOTES`.

The routine returns `{ok, error, created}`. On failure, `created` lists what was made *before* the failure, so partial progress is reported honestly — schema created but a table refused, for example. On success the banner reads "Created: schema <name>, PROJECTS, JOURNEYS, STEPS, METAS, NOTES". A blank schema name is refused with "A schema name is required.", and the admin panel pre-checks with the toast "Enter a schema name first."

The schema identifier is safely double-quoted, with embedded quotes doubled, so mixed-case and special-character names work. Remember that a quoted identifier is case-sensitive in Exasol: a schema created as `"Demo"` is not the same object as `DEMO`.

The effective shape of the DDL is:

```
CREATE SCHEMA IF NOT EXISTS "DEMO";
OPEN SCHEMA "DEMO";

CREATE TABLE IF NOT EXISTS PROJECTS (
  PROJECT_ID   VARCHAR(100)  NOT NULL PRIMARY KEY,
  TITLE        VARCHAR(500)  DEFAULT '',
  DESCRIPTION  VARCHAR(2000) DEFAULT ''
);

CREATE TABLE IF NOT EXISTS JOURNEYS (
  PROJECT_ID  VARCHAR(100)  NOT NULL,
  EVENT_ID    VARCHAR(200)  NOT NULL,
  STEP        VARCHAR(500)  NOT NULL,
  STEP_ID     DECIMAL(18,0),
  EVENT_TIME  TIMESTAMP     NOT NULL,
  META_1      VARCHAR(1000),
  META_2      VARCHAR(1000),
  META_3      VARCHAR(1000),
  SAMPLE_SET  VARCHAR(20)   DEFAULT 'ORIGINAL'
)
DISTRIBUTE BY EVENT_ID
PARTITION BY EVENT_TIME;

CREATE TABLE IF NOT EXISTS STEPS (
  PROJECT_ID      VARCHAR(100)  NOT NULL,
  STEP            VARCHAR(500)  NOT NULL,
  DESCRIPTION     VARCHAR(2000) DEFAULT '',
  BG_COLOR        VARCHAR(30)   DEFAULT '',
  FG_COLOR        VARCHAR(30)   DEFAULT '',
  SCORE           DECIMAL(18,2),
  SHAPE           VARCHAR(50)   DEFAULT '',
  END_OF_PROCESS  BOOLEAN       DEFAULT FALSE,
  BELONGS_TO      VARCHAR(500),
  PRIMARY KEY (PROJECT_ID, STEP)
);

CREATE TABLE IF NOT EXISTS METAS (
  PROJECT_ID    VARCHAR(100) NOT NULL PRIMARY KEY,
  META_1_TITLE  VARCHAR(500) DEFAULT '',
  META_2_TITLE  VARCHAR(500) DEFAULT '',
  META_3_TITLE  VARCHAR(500) DEFAULT ''
);
```

`NOTES` follows with the columns listed in the NOTES section. `TRANSITIONS_RAW` has no provisioning DDL at all — it only ever comes into existence through a rebuild.

Tip In the admin panel only, the checkbox **Also build pre-materialized transitions after provisioning** (default unchecked) runs a rebuild immediately after a successful provision. The resulting table is empty until `JOURNEYS` is loaded, but the structure exists and the per-connection flag can be switched on straight away. The result line gains the suffix " · transitions built (N pairs)" or " · transitions build failed: <error>".

Warning Provisioning, testing and demo generation all use the password **typed into the form**, not the stored one. On a saved connection whose password field shows the "unchanged" placeholder, the operation runs with an empty password and normally reports an authentication failure even though the connection itself works. Retype the password before using any of these three buttons.

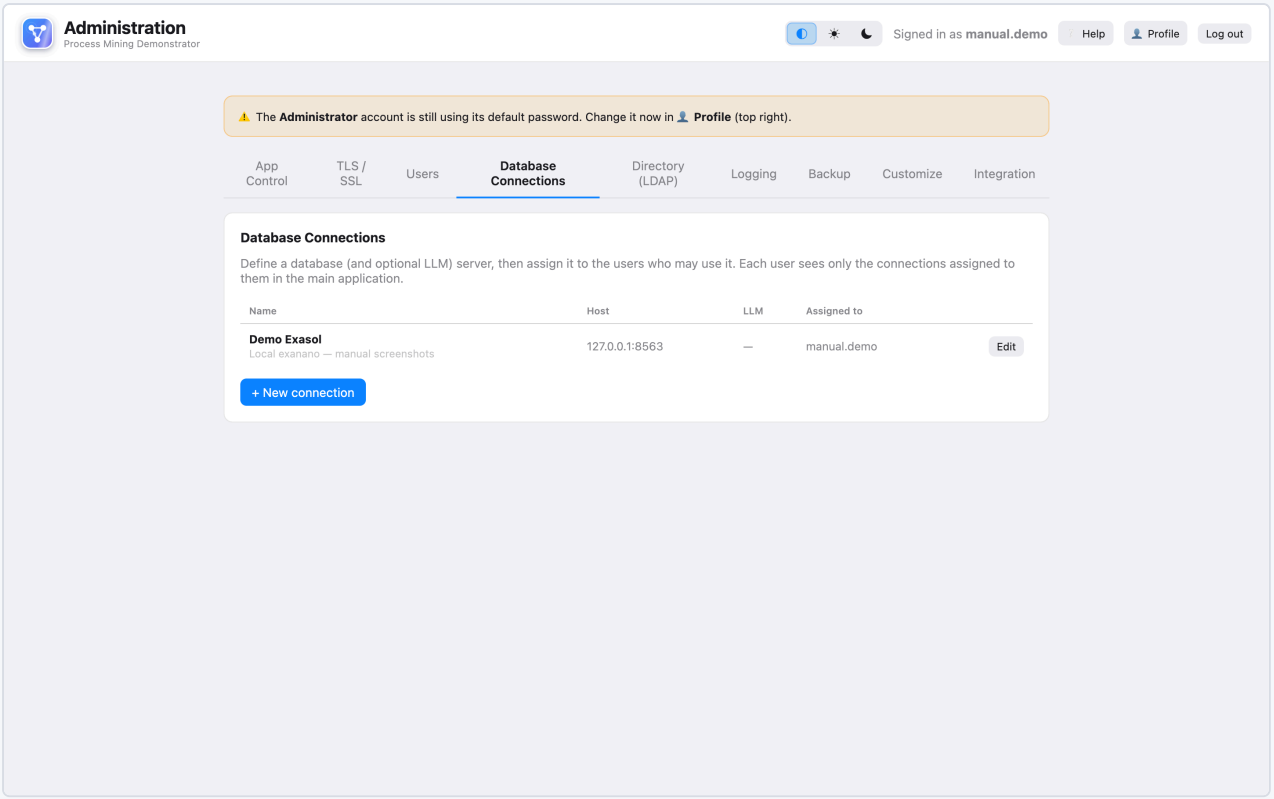


Figure 21.2 — The admin panel's Connections tab, from which the schema is provisioned and `TRANSITIONS_RAW` is rebuilt.

Privileges the application needs

The provisioning banner states the boundary plainly: "This requires a database account permitted to CREATE SCHEMA and CREATE TABLE — only your database administrator can grant those rights; this application cannot." The application never attempts to grant anything to itself. Decide which of the following capabilities the connection's database account should have, according to what you want that connection to be able to do.

Capability	Needed for	Statements issued
SELECT on the schema	Everything. A read-only connection can browse maps, statistics and journeys.	`SELECT` on all six tables
INSERT, UPDATE, DELETE	The Step editor, project metadata, notes, sampling	`INSERT` / `UPDATE` / `DELETE` on `STEPS`, `METAS`, `NOTES`, `JOURNEYS`
CREATE TABLE in the schema	Creating `NOTES` lazily on first use; building `TRANSITIONS_RAW_STAGE`	`CREATE TABLE IF NOT EXISTS`, `CREATE OR REPLACE TABLE … AS SELECT`
ALTER TABLE	The `SAMPLE_SET` migration, the `NOTES` column migrations, the distribution hint on `TRANSITIONS_RAW`	`ALTER TABLE … ADD COLUMN`, `ALTER TABLE … DISTRIBUTE BY`
DROP TABLE and RENAME TABLE	The `TRANSITIONS_RAW` staging swap	`DROP TABLE IF EXISTS`, `RENAME TABLE`
CREATE SCHEMA	**Create schema & tables** and demo generation only	`CREATE SCHEMA IF NOT EXISTS`

Demo generation additionally requires `INSERT`, and its replacement step requires `DELETE` on `PROJECTS`, `METAS` and `JOURNEYS` for the dataset's own project. Project deletion from the connection editor issues `DELETE … WHERE PROJECT_ID = '<id>'` against `JOURNEYS`, `STEPS`, `METAS`, `NOTES`, `TRANSITIONS_RAW` and `PROJECTS`, in that order — children before `PROJECTS`. Absent tables are skipped rather than treated as errors, and the response's `tables` array lists what was actually cleared.

> **Tip** A practical split for production: give analysts' connections `SELECT` plus `INSERT` / `UPDATE` / `DELETE` so notes, steps and samples work, and keep `CREATE SCHEMA`, `DROP` and `RENAME` for a separate maintenance connection used by the administrator and by the scheduled rebuild. Because the rebuild endpoint uses the *stored* credentials of the connection it names, a scheduler can drive the maintenance connection without ever handling a database password.

> **Warning** Project deletion is permanent. There is no undo and no application-level backup, and it erases every note attached to the project along with the events. The confirmation dialog spells out the six tables it clears; read it before confirming.

Sizing the schema

Only `JOURNEYS` and `TRANSITIONS_RAW` grow with your data. `PROJECTS`, `METAS` and `STEPS` are configuration-sized — tens to hundreds of rows — and `NOTES` grows with human activity, not with the log, although a single note row can hold up to 100,000 characters of thread text.

Work with row counts rather than bytes:

`JOURNEYS`, original data
: *journeys* × *average steps per journey* rows. This is the base figure for every other estimate.

`JOURNEYS`, **with samples**

Add the full row count of every sampled journey again, per sample set. Samples are physical copies, so a project with three 25 % samples occupies about 1.75 times the base. Budget for the worst case — three sample sets at 100 % would quadruple the table.

`TRANSITIONS_RAW`

Roughly *base rows – journeys* pairs per project and sample set, because the final step of each journey has no successor. Its rows are narrower than `JOURNEYS` rows (no `META` columns) but it covers *all* projects and *all* sample sets in one table, so size it against the whole schema, not one project.

Points that affect load and query time rather than storage:

- Inserts from the sampling and demo-generation paths are batched at 200 journeys' rows per `INSERT`. Your own ETL should batch at least as coarsely.
- Demo generation is hard-capped at **20,000 journeys** server-side. A larger request is silently clamped to exactly 20,000; the field's own maximum is the same figure and its default is 500. Use a real load, not the demo generator, to build a volume-test data set beyond that size.
- Demo event times are spread randomly over 2024-01-01 08:00 to 2024-12-31 22:00, so date filters on demo data always operate within that single year.
- Heavy statistics queries — variants, routes and the goodness metric — are subject to a query timeout of 30 s by default. On a log large enough to hit it you will see "Statistics query timed out (30 s). Narrow your date range or filters and try again." Variant and route listings are additionally capped at 10,000 rows.
- Every statement runs under a 300 s socket timeout. Rebuilds, provisioning and demo generation have no application-level timeout of their own and are bounded only by that per-statement limit.

> **Note** One user's queries are serialised. The database driver is synchronous and a single connection cannot safely run concurrent statements, so each user's queries queue behind one another on that user's own session. Different users run in parallel on their own sessions. When you size the cluster, size it for concurrent *users*, and remember that a long rebuild launched from a scheduler runs on its own stored-credential session and does not block anybody's map.

Loading checklist

A summary of the operational rules that follow from the model above.

1. Create or verify the schema and the five canonical tables, either from your own DDL or with **Create schema & tables**.
2. If `JOURNEYS` pre-existed, apply `DISTRIBUTE BY EVENT_ID` and `PARTITION BY EVENT_TIME` by hand — provisioning will not do it for you.
3. Register the project in `PROJECTS` and fill its `METAS` row so the meta filters are labelled.
4. Load `JOURNEYS`. Populate `STEP_ID` wherever timestamps can collide, and set `SAMPLE_SET` to `ORIGINAL` (or leave it null — it is read as original).
5. Populate `STEPS` with one row per activity, unique on (`PROJECT_ID`, `STEP`), or let operators build it up in the Step editor and watch for grey nodes as the to-do list.

6. If the connection has **Use pre-materialized transitions** enabled, call the rebuild endpoint as the final step of the load. Confirm afterwards that the map's pill reads "⚡ Pre-materialized" and not "⚠ Live (not built)".

22 Reference: Writing an Extractor

> The File extractor that the integration console ships with is not a special case: it is an ordinary plug-in written against a small, stable contract, and any other source — a CSV export, a message queue, a REST endpoint, a second log format — can be added the same way. This chapter is the reference for that contract. It documents the four types an extractor programs against (`ExtractorInfo`, `Extractor`, `IngestSession`, `ExtractResult`), the registry, the lifecycle of a run and how status and progress reach the console, the two ingest backends and the transaction bracket, how a destination is opened with stored credentials, the sandboxed file helpers, a complete worked skeleton in Python with its file layout and registration, and how to test the result without a database. It is written for developers with access to the product's source tree; nothing in this chapter is done from the browser.

Requires: write access to the deployment's `backend/` source tree. Running the finished extractor through the integration console additionally requires the **Developer** or **Administrator** role.

Where the code lives

The whole abstraction layer is one Python package, `backend/app/integration/`. An extractor only ever imports from `contract` (the stable surface) and, if it reads files, from `files`. Everything else is machinery it is deliberately kept away from.

Module	What it holds	An extractor imports it?
`contract.py`	`ColumnType`, `ExtractorInfo`, `ExtractResult`, `IngestError`, and the `IngestSession` / `Extractor` protocols	**Yes** — this is the API
`files.py`	The sandboxed path resolver and the line/delta readers	Yes, for file-shaped sources
`layer.py`	`AbstractionLayer`, the concrete session, and the process-wide `layer` singleton	Only to register
`backends.py`	`InMemoryIngestBackend`, `SqlIngestBackend`, identifier validation, bracket clamping	No — the caller wires these
`destinations.py`	`open_stored_connection()` — opens a saved connection for a headless run	No
`status.py`	`LayerStatus` / `LayerState`, the per-user snapshot the console polls	No
`extractors.py`	The built-in `FileExtractor` and `register_builtin_extractors()`	Reference implementation
`watchdog.py`, `compound.py`, `parsing.py`	The background poller, compound-step rules, timestamp/field detection	No

> **Note** An extractor never opens a database connection, never builds SQL, never commits, and never decides where rows land. It is handed an `IngestSession` already bound to the target schema of the connection the user (or the watchdog) chose, and it pushes rows through that. This is what keeps extractors database-agnostic and what lets the same code be tested against an in-memory target.

The contract

ExtractorInfo — the identity

`ExtractorInfo` is a frozen dataclass carrying the plug-in's identity. It is what `GET /api/integration/extractors` returns and what the console displays.

Field	Type	Meaning
`id`	str	The registry key. Must be globally unique, stable across versions, and a slug: lower-case letters, digits and single hyphens (`^[a-z0-9]+(-[a-z0-9]+)*$`). Examples: `file` , `csv-folder` .
`name`	str	The display name, e.g. `File` .
`version`	str	A free-form version string, e.g. `1.0` .
`description`	str	One sentence shown next to the name. Defaults to the empty string.

Registration enforces both rules and reports them as `ValueError` :

```
Extractor id 'Not A Slug' must be a slug (a-z, 0-9, '-').
An extractor with id 'file' is already registered.
```

Extractor — the protocol

An extractor is any object with an `info` attribute and a `run(session)` method. There is no base class to inherit from; the protocol is structural.

```
class Extractor(Protocol):
    info: ExtractorInfo

    def run(self, session: IngestSession) -> ExtractResult:
        ...
```

`run` is synchronous

The layer executes it off the event loop in a worker thread (`asyncio.to_thread`). Do not make it a coroutine, and do not block the event loop by calling it directly.

Push incrementally

Read and push in batches rather than buffering the whole source. `push` accepts a generator and streams it, so a source of any size can be handed over without materialising it.

Raising means failure

Any exception marks the run `failed`, records the message as `lastError` in the user's status, rolls back the open transaction bracket, and propagates to the caller. Returning an `ExtractResult` marks the run `completed`.

Do not keep the session

The session is created per run and is invalid once `run` returns. Never store it on the instance for later use.

IngestSession — the only handle to the database

Member	Signature	Behaviour
`schema`	property → str	The target schema of the selected connection. Every table this session writes lives there. Validated as a SQL identifier when the session is created.
`define_table`	`define_table(table, columns, *, keys=())`	Declares the table and creates it if absent (`CREATE TABLE IF NOT EXISTS`). Idempotent — call it on every run. `columns` maps column name → `ColumnType` and also fixes the column **order** used for inserts. `keys` names the business-key columns and is informational for now. Must be called for a table before any `push` to it. Also records the table in `tablesTouched`, in order of first touch.
`push`	`push(table, rows) -> int`	Appends rows (each a mapping of column name → value) and returns how many were written. Missing columns become NULL. Rows are flushed in internal batches of **1000**. Every flush adds to the live `recordsPushed` counter and to the transaction bracket.
`log`	`log(message)`	Appends a progress line to the status panel. Only the most recent **50** lines are kept; older ones are dropped.
`progress`	`progress(done, total=None)`	Sets the progress-bar counters `recordsDone` / `recordsTotal`. Cheap — call it every few hundred items. A `total` of 0 means "unknown".
`existing_keys`	`existing_keys(table, key_columns) -> set[tuple[str, ...]]`	The key tuples already present in the table, every value stringified and NULL rendered as the empty string. Use it for create-if-missing of small metadata tables (a project row, step definitions). It reads **all** keys of the table — never call it on a fact table.

> **Warning** `push` rejects columns that were not declared. A row containing a key that is not in the `define_table` mapping raises `IngestError` with `Unknown column(s) for JOURNEYS: GHOST`, which fails the whole run. Pushing to a table you never declared raises `Call define_table('JOURNEYS', ...) before push().`, and declaring a table with an empty column mapping raises `Table 'JOURNEYS' must declare at least one column.`

ExtractResult — what the run reports

The result is purely informational: the layer has already counted every row that went through `push`. The result is where the extractor says what those rows *meant*.

Field	Default	Meaning
`records`	0	Successful items — for the File extractor, journey events. Becomes `eventsWritten` in the status and the `records` field of the run response.
`skipped`	0	Source items that could not become a record. Becomes `eventsSkipped`. Report this honestly: it is the KPI that makes a silently misconfigured source visible as "lots skipped" rather than merely as a small record count.
`tables`	`()`	Tuple of table names the extractor touched.
`detail`	`""`	The human-readable summary shown to the user, e.g. `1 240 events written, 17 skipped, 3 new step(s)`.

> **Note** `recordsPushed` in the status and `records` in the result are different numbers on purpose. `recordsPushed` counts *every* row that crossed the session, including the project, step and meta rows an extractor creates on the side; `records` counts only the domain objects the extractor set out to produce.

ColumnType and the portable type mapping

Extractors declare portable types; the backend maps them to the concrete database. This is the only reason an extractor written for the demonstrator does not contain a single line of Exasol-specific SQL.

`ColumnType`	Value	Exasol type	Accepted Python values
`STRING`	`string`	`VARCHAR(2000000)`	Any value; rendered as text with NUL bytes stripped and single quotes doubled
`INT`	`int`	`DECIMAL(18,0)`	`int`
`DECIMAL`	`decimal`	`DECIMAL(36,6)`	`int`, `float` (non-finite values are rejected with `Non-finite numeric value: …`)
`TIMESTAMP`	`timestamp`	`TIMESTAMP`	`datetime` (rendered to millisecond precision), `date`
`BOOL`	`bool`	`BOOLEAN`	`True` / `False`

Identifier rules

Schema, table and column names must match a conservative grammar: a letter or underscore, then letters, digits or underscores, up to **128** characters. Anything else is **rejected, never escaped**, with `Invalid SQL identifier: 'EVENTS; DROP TABLE X'`. Names are therefore safe to interpolate into DDL and DML, and no source data can ever reach a statement as an identifier.

IngestError

`IngestError` is what the layer raises for an invalid ingest operation: unknown table, undeclared column, bad identifier, unknown extractor id, or a concurrent run. Extractors normally let it propagate — it is recorded as the run's failure exactly like any other exception. Extractors may also raise it themselves when they detect a condition the layer would consider fatal.

The registry

`AbstractionLayer` holds the registry and the per-user status. One process-wide instance exists, exported as `layer` from `app.integration`; that is the object the API and the watchdog share.

Method	Effect
`layer.register(extractor)`	Adds the extractor under `extractor.info.id`. Raises `ValueError` on a non-slug id or a duplicate.
`layer.unregister(id)`	Removes it if present; silent when it is not.
`layer.get(id)`	The registered extractor, or `None`. Use it to make registration idempotent.
`layer.extractors()`	The list of `ExtractorInfo`s behind `GET /api/integration/extractors` and the `registeredExtractors` count in the status.
`layer.status_for(user)`	That user's `LayerStatus`, created on first use. A `None` user maps to the empty key, which is how deployments without sign-in are handled.
`layer.run(...)`	The coroutine that drives one run. See below.

Registration happens **at import time**, plug-in style. The package's `__init__.py` imports the built-ins and calls `register_builtin_extractors(layer)`, so merely importing `app.integration` anywhere in the process is enough for the console to list them.

> **Tip** Guard your registration with `if layer.get(MY_ID) is None:` as the built-in does. Module reloads during tests would otherwise trip the duplicate check and fail the import.

Descriptor entries versus configured instances

Some extractors cannot be run straight from the registry because a run needs configuration that only exists per source — a path, an encoding, a project id, a field list. The built-in File extractor solves this with a two-object pattern that is worth copying:

1. A **descriptor** object is registered. It carries the real `ExtractorInfo` so the extractor appears in `GET /api/integration/extractors`, but its `run` raises immediately: `The File extractor is run per source, not from the registry.`
2. The **configured instance** is built per run by the API or the watchdog, with everything it needs passed to its constructor, and handed to `layer.run(extractor=instance, …)` as an object rather than an id.

`layer.run` accepts either form: a string is looked up in the registry (`No extractor registered with id 'nope'.` if absent), and any object satisfying the protocol is used as-is.

The lifecycle of a run

Every run — a console **Run**, a watchdog poll, or a call from your own code — goes through the same coroutine. Its parameters are what the console's pipeline canvas is drawn from.

```
result = await layer.run(
    user=user,                          # status is per user; None/"" for no-sign-in
    extractor=instance_or_id,
    backend=backend,                    # SqlIngestBackend or InMemoryIngestBackend
    schema=schema,                      # target schema of the chosen connection
    connection_id=conn_id,
    connection_name=conn.name,
    source_name=source.name,
    source_type_name=st.name,
    trigger="manual",                   # or "watchdog"
    transaction_rows=transaction_rows,
)
```

What the layer does, in order:

1. **Resolves the extractor.** A string id is looked up; a missing id raises `IngestError`.
2. **Takes the per-user run slot.** If that user already has a run in flight it raises `An extraction is already running for this user.` Different users run concurrently; the same user cannot, which is precisely what stops a manual import and a watchdog poll from importing the same lines twice.
3. **Resets the user's status** and sets `state: running`, the trigger, the extractor id and name, the source, source-type, connection and schema names, `startedAt`, and zeroes every counter (`recordsPushed`, `eventsWritten`, `eventsSkipped`, `commits`, `recordsDone`, `recordsTotal`), the table list, the error and the message log. The status therefore always describes the most recent or in-flight run only.
4. **Creates the session** bound to the backend, the schema and the transaction bracket.
5. **Runs `extractor.run(session)` in a worker thread.** While it runs, every `push`, `log` and `progress` call updates the same status object under a lock, so `GET /api/integration/status` shows live progress.
6. **Closes the final transaction bracket** with a commit, also off the event loop.
7. **On success:** sets `state: completed` and `finishedAt`, copies `result.records` into `eventsWritten` and `result.skipped` into `eventsSkipped`, writes the admin-log line `extractor <id> pushed <n> rows to <schema>` (usage level, operation `integration`), and returns the result.
8. **On failure:** rolls the backend back (a failing rollback is swallowed so it cannot mask the real error), sets `state: failed`, `finishedAt` and `lastError`, logs `extractor <id> failed: <error>` as a warning, and re-raises.
9. **Always:** releases the user's run slot.

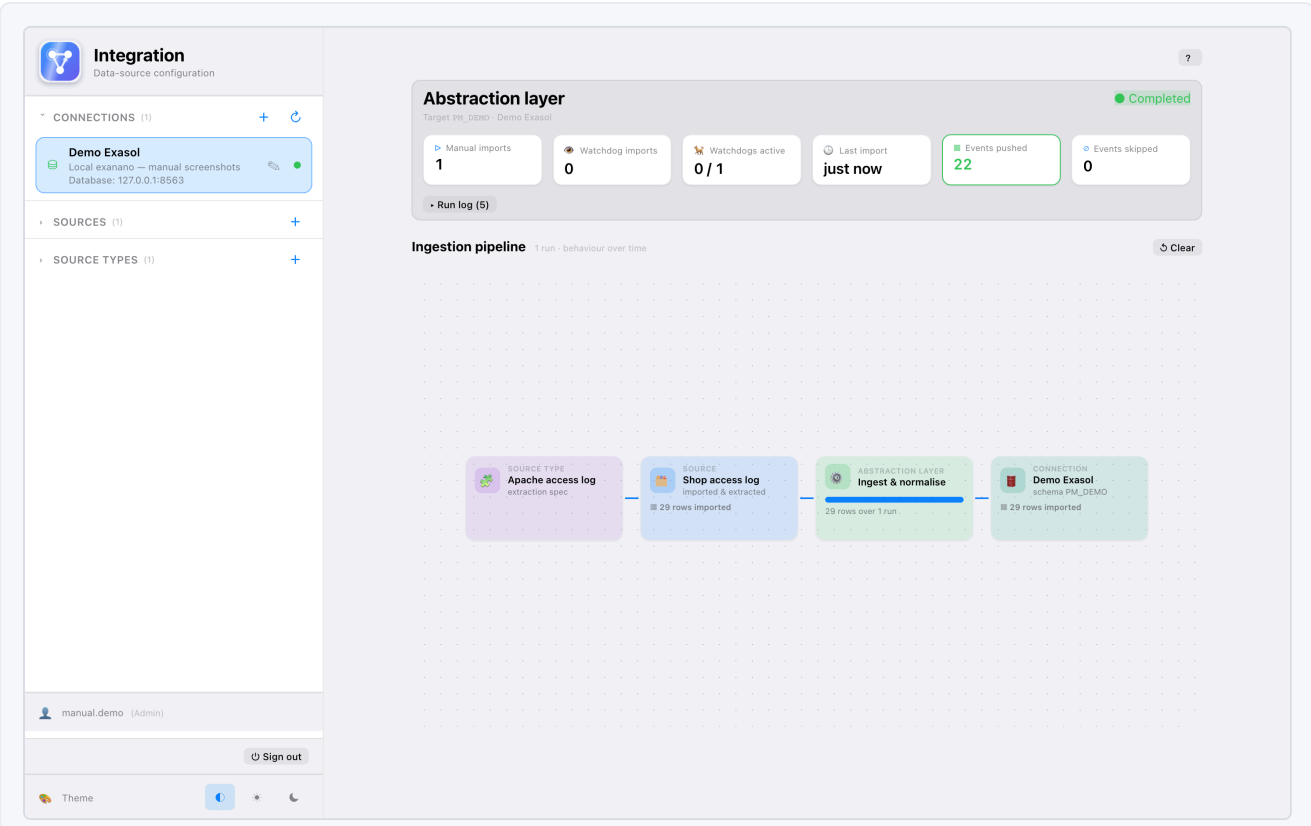


Figure 22.1 — The console's pipeline canvas and KPI strip are drawn entirely from the per-user layer status — the same fields any extractor populates through `log`, `progress` and `push`.

What your calls become on screen

You call	Status field	Where it shows
`session.push(...)`	`recordsPushed`	The live "rows pushed" figure, all tables included
`session.progress(done, total)`	`recordsDone` / `recordsTotal`	The progress bar (`recordsTotal` 0 = indeterminate)
`session.log(msg)`	`messages`	The message list, last 50 lines
`session.define_table(...)`	`tablesTouched`	The table chips, in order of first touch
return `ExtractResult`	`eventsWritten`, `eventsSkipped`	The two event KPIs, set at completion
(the layer)	`commits`	Durable progress — brackets committed so far
raise	`lastError`, `state`	The failure banner

The two backends and the transaction bracket

An `IngestBackend` is the seam between the portable ingest API and a concrete target. Extractors never see one; the code that starts the run picks it.

`InMemoryIngestBackend`

Keeps schemas, tables and rows in dictionaries. No database, no dependencies. It is the local development target and the reference implementation of the ingest semantics for tests; it additionally counts `commits` and `rollbacks` so a test can assert the bracketing. Inserting into a table that was never created raises `Table <schema>.<table> was not defined.`

`SqlIngestBackend`

The production target. It generates schema-qualified DDL and DML and runs it through a caller-supplied `run_sql` callable, plus optional `commit` and `rollback` callables that belong to whoever owns the connection. Identifiers are validated, values are rendered as defensively escaped literals, and each `INSERT` statement carries at most **1000** rows. Without the commit/rollback callables it is a no-op on both — the driver's autocommit then ends every statement, which is what the SQL-recording tests expect.

Transaction rows

The `transaction_rows` argument is the bracket size: the session counts rows written since the last commit and commits as soon as the bracket is full. This keeps the database's transaction small on a large import and makes rows durable as the run progresses.

Configured value	Effective value	Meaning
not set, or not a number	**5000**	The default bracket
`0` or any value ≤ 0	**0**	One transaction for the entire import — fully atomic, but the database holds all of it open
1 … 999	**1000**	Floored to one insert batch
1000 … 1 000 000	as given	Used unchanged
above 1 000 000	**1 000 000**	Clamped

Normalise user-supplied values with `clamp_transaction_rows()` from `app.integration.backends` rather than trusting the number. After `run` returns, the layer issues a final commit for the last, possibly partial bracket. That final commit is skipped only when nothing was written since the previous commit *and* at least one commit already happened; the very first commit always runs, even with zero rows, to close out the transaction any DDL or SELECT opened.

> **Warning** On failure only the **open** bracket is rolled back. Brackets already committed stay committed — that is the point of bracketing. A failed run can therefore leave earlier rows in the database while the caller's read checkpoint does not advance, so a retry re-reads the same input and **duplicates those rows**. Nothing in the layer de-duplicates. If your extractor's source cannot tolerate that, run it with `transaction_rows=0` for a single atomic transaction, and document the trade-off.

Opening a destination with stored credentials

A run's destination is always a *stored* connection, opened with the credentials an administrator saved — never with whatever the caller happens to have connected in their browser. That is what makes the trigger

irrelevant: a console run, a watchdog poll and a headless push all name a connection by id and land in the same place.

```
from app.integration.destinations import open_stored_connection
from app.integration import SqlIngestBackend

opened, run_sql = open_stored_connection(conn)      # blocking — call via asyncio.to_thread
backend = SqlIngestBackend(
    run_sql=run_sql, commit=opened.commit, rollback=opened.rollback,
)
try:
    result = await layer.run(..., backend=backend, schema=conn.schema)
finally:
    opened.close()                                  # the caller owns the handle
```

`open_stored_connection` returns the raw handle and a synchronous SQL runner bound to it (a list of rows for a SELECT, an empty list otherwise), and it **switches autocommit off** so that the layer's bracket, not the driver, decides when work becomes durable. The caller owns the handle and must close it, on the failure path as well as the success path.

When the caller already has a live session pointing at exactly the same connection, the API reuses it instead of opening a second one — again turning autocommit off first, and routing `commit` and `rollback` through that live session. A connection that cannot be opened is reported to the user as `Could not connect to "<name>": <error>`, and a connection whose stored target schema is blank as `"<name>" has no target schema configured.`

> **Note** Whoever triggers a run must re-check that the connection is assigned to the acting user, on *every* run — the assignment, not the presence of a connection id, is the authorisation. The watchdog repeats this check on every poll for exactly this reason, so revoking an assignment stops it immediately.

Sandboxed file access

If your extractor reads files, use the helpers in `app.integration.files`. They are the only sanctioned path from a user-supplied string to an open file, and every one of them funnels through `resolve_source_file`. Do not call `open()` on a configured path yourself — that bypasses the sandbox.

Helper	Returns	Notes
`resolve_source_file(path)`	`Path`	Resolves and validates. Raises `FileAccessError`.
`read_preview(path, limit=5)`	`{lines, truncated}`	First `limit` non-blank lines. `limit` is clamped to **1…50**. Always decodes as UTF-8 with replacement, regardless of any configured encoding.
`iter_lines(path, encoding)`	generator of str	Streams non-blank lines for extraction. Safe to hand straight to a loop that pushes in batches.
`count_lines(path, encoding)`	int	A cheap pre-pass counting non-blank lines — the denominator for `progress(done, total)`.
`read_new_lines(path, encoding, offset)`	`{lines, new_offset, size, signature, rotated}`	Complete lines appended since `offset`. A partial trailing line is left for the next read.
`read_delta(path, encoding, offset, signature)`	same	`read_new_lines` plus replacement detection: a changed head with an unchanged or grown size means the file was replaced, so it is re-read from byte 0.
`seed_demo_files()`	—	Copies the bundled example logs into the sandbox on start-up. Never overwrites; best-effort.

The limits below apply to every one of these helpers, so they apply to any extractor built on them:

Limit	Value	Consequence
Line length	**2000** characters	Every line is truncated first. A pattern targeting content beyond character 2000 never matches.
Blank lines	always skipped	Also excluded from the progress denominator.
Bytes per delta read	**8 MB**	A larger backlog is drained across successive reads; `new_offset < size` signals more is pending.
Single line longer than 8 MB	skipped past	Treated as unparseable content rather than re-read forever.
Fingerprint	first **64** bytes	Files shorter than that get an empty signature, so in-place replacement without shrinking goes undetected.
Decoding	configured encoding, errors replaced	Undecodable bytes never raise.

Path resolution raises `FileAccessError` with exactly one of these messages, which callers surface as HTTP 400: `No file path given.`, `That path is outside the allowed sources directory.`, `File not found.`, `Not a regular file.` In the default sandboxed mode a relative path is taken relative to `PMW_INTEGRATION_FILES_DIR`, an absolute path is accepted only if it already lies inside that root, and the real path with symlinks resolved must still be inside it.

> **Warning** Setting `PMW_INTEGRATION_ALLOW_ANY_PATH` to `1`, `true`, `yes` or `on` disables the sandbox entirely: any absolute path the server process can read is then accepted. Use it only in a developer-trusted deployment.

A worked skeleton

The example below is a complete, working extractor for a comma-separated event export. It is not shipped with the product; it exists to show every part of the contract in one place — declaration, streaming push, progress, create-if-missing metadata, and the result.

File layout

```
backend/
  app/
    integration/
      __init__.py          ← register the new extractor here
      contract.py
      files.py
      layer.py
      csv_extractor.py     ← the new module
  tests/
    test_csv_extractor.py  ← the new test
```

The extractor

```
"""Example extractor: reads a comma-separated export and writes JOURNEYS events."""

from __future__ import annotations

import csv
import hashlib
from datetime import datetime

from .contract import (
    ColumnType, ExtractResult, ExtractorInfo, IngestError, IngestSession,
)
from .files import resolve_source_file

CSV_EXTRACTOR_ID = "csv-file"
_BATCH = 500

_JOURNEYS_COLUMNS = {
    "PROJECT_ID": ColumnType.STRING,
    "EVENT_ID": ColumnType.STRING,
    "STEP": ColumnType.STRING,
    "STEP_ID": ColumnType.INT,
    "EVENT_TIME": ColumnType.TIMESTAMP,
    "META_1": ColumnType.STRING,
    "META_2": ColumnType.STRING,
    "META_3": ColumnType.STRING,
    "SAMPLE_SET": ColumnType.STRING,
}
_PROJECTS_COLUMNS = {
    "PROJECT_ID": ColumnType.STRING,
    "TITLE": ColumnType.STRING,
    "DESCRIPTION": ColumnType.STRING,
}


def csv_extractor_info() -> ExtractorInfo:
    return ExtractorInfo(
        id=CSV_EXTRACTOR_ID,
        name="CSV file",
        version="1.0",
        description="Reads a comma-separated export and writes JOURNEYS events.",
    )


class CsvExtractor:
    """One configured run over one CSV file."""
```

```
info = csv_extractor_info()

def __init__(self, *, path: str, encoding: str, project_id: str) -> None:
    self._path = path
    self._encoding = encoding or "utf-8"
    self._project_id = project_id

# ── the one method the layer calls ──────────────────────────────
def run(self, session: IngestSession) -> ExtractResult:
    session.define_table(
        "JOURNEYS", _JOURNEYS_COLUMNS, keys=["PROJECT_ID", "EVENT_ID", "STEP"],
    )
    real = resolve_source_file(self._path)   # sandboxed; may raise FileAccessError
    session.log(
        f"reading {self._path} → {session.schema}.JOURNEYS "
        f"(project {self._project_id})"
    )

    written = skipped = processed = 0
    batch: list[dict] = []
    with real.open("r", encoding=self._encoding,
                   errors="replace", newline="") as fh:
        for record in csv.DictReader(fh):
            processed += 1
            row = self._row(record)
            if row is None:
                skipped += 1
            else:
                batch.append(row)
                if len(batch) >= _BATCH:
                    written += session.push("JOURNEYS", batch)
                    batch = []
            if processed % 200 == 0:
                session.progress(processed)
    if batch:
        written += session.push("JOURNEYS", batch)
    session.progress(processed)

    session.log(f"pushed {written} events, skipped {skipped} unusable row(s)")
    created = self._ensure_project(session)

    detail = f"{written} events written, {skipped} skipped"
    if created:
        detail += ", project created"
    return ExtractResult(
        records=written, skipped=skipped,
```

```
            tables=("JOURNEYS", "PROJECTS"), detail=detail,
        )

    # ── one source item → one row, or None ──────────────────────────────
    def _row(self, record: dict) -> dict | None:
        case_id = (record.get("case") or "").strip()
        step = (record.get("activity") or "").strip()
        raw_time = (record.get("time") or "").strip()
        if not (case_id and step and raw_time):
            return None                       # counts as skipped, never as an error
        try:
            event_time = datetime.strptime(raw_time, "%Y-%m-%d %H:%M:%S")
        except ValueError:
            return None
        return {
            "PROJECT_ID": self._project_id,
            # Pseudonymise the case id — the raw value never reaches the database.
            "EVENT_ID": hashlib.md5(case_id.encode("utf-8")).hexdigest(),
            "STEP": step,
            "STEP_ID": None,
            "EVENT_TIME": event_time,
            "META_1": record.get("channel"),
            "META_2": None,
            "META_3": None,
            "SAMPLE_SET": "ORIGINAL",
        }

    # ── create-if-missing metadata ──────────────────────────────────────
    def _ensure_project(self, session: IngestSession) -> bool:
        session.define_table("PROJECTS", _PROJECTS_COLUMNS, keys=["PROJECT_ID"])
        if (self._project_id,) in session.existing_keys("PROJECTS", ["PROJECT_ID"]):
            return False
        session.push("PROJECTS", [{
            "PROJECT_ID": self._project_id,
            "TITLE": self._project_id,
            "DESCRIPTION": "",
        }])
        session.log(f"created project {self._project_id!r}")
        return True


class _CsvExtractorDescriptor:
    """Registry-only entry so the extractor is listed; runs use a configured instance."""

    info = csv_extractor_info()

    def run(self, session: IngestSession) -> ExtractResult:
```

```
        raise IngestError("The CSV extractor is run per source, not from the registry.")


def register_csv_extractor(layer) -> None:
    """Idempotent registration — safe under module reloads in tests."""
    if layer.get(CSV_EXTRACTOR_ID) is None:
        layer.register(_CsvExtractorDescriptor())
```

Registering it

1. Add the module to `backend/app/integration/` as shown above.
2. Open `backend/app/integration/__init__.py` and import the registration function next to the existing built-ins:

   ```
   from .csv_extractor import CsvExtractor, register_csv_extractor
   ```

3. Call it beside the existing registration, after `layer` is imported:

   ```
   register_builtin_extractors(layer)
   register_csv_extractor(layer)
   ```

4. Add the public names to `__all__` so callers can import them from the package rather than the module.
5. Restart the compute backend. Confirm the entry appears in `GET /api/integration/extractors` and that `registeredExtractors` in `GET /api/integration/status` has gone up by one.

Driving it

To run the configured instance from your own endpoint or job, build the backend, run the layer, and close anything you opened:

```
from app.integration import SqlIngestBackend, layer
from app.integration.backends import clamp_transaction_rows
from app.integration.destinations import open_stored_connection

opened, run_sql = await asyncio.to_thread(open_stored_connection, conn)
backend = SqlIngestBackend(
    run_sql=run_sql, commit=opened.commit, rollback=opened.rollback,
)
try:
    result = await layer.run(
        user=user,
        extractor=CsvExtractor(path=path, encoding="utf-8", project_id="RETAIL"),
        backend=backend,
        schema=conn.schema,
        connection_id=conn.id,
        connection_name=conn.name,
        source_name=source.name,
        trigger="manual",
        transaction_rows=clamp_transaction_rows(cfg.get("transactionRows")),
    )
finally:
    await asyncio.to_thread(opened.close)
```

Tip If your source can be read incrementally, keep the read cursor outside the extractor and advance it **only after the run succeeds**, exactly as the file checkpoint does. A cursor advanced before or during a run silently loses data when the run fails.

Testing an extractor

Every extractor can be tested end to end with no database at all: register it on a private `AbstractionLayer()`, run it against an `InMemoryIngestBackend`, and assert on the rows that landed and on the status the console would have shown. Use a private layer instance rather than the shared `layer` singleton so tests cannot interfere with each other.

1. Create `backend/tests/test_csv_extractor.py` alongside the existing `test_integration_*.py` files.

2. Point the sandbox at a temporary directory before importing anything that reads it, then reload `app.config` and `app.integration.files` so the new value takes effect:

```
@pytest.fixture
def env(tmp_path, monkeypatch):
    monkeypatch.setenv("PMW_INTEGRATION_FILES_DIR", str(tmp_path / "files"))
    monkeypatch.delenv("PMW_INTEGRATION_ALLOW_ANY_PATH", raising=False)
    import app.config as config
    importlib.reload(config)
    import app.integration.files as files_mod
    importlib.reload(files_mod)
    return config
```

3. Write a fixture file into `config.INTEGRATION_FILES_DIR` and run the extractor:

```
def test_csv_extractor_writes_rows(env):
    (env.INTEGRATION_FILES_DIR / "events.csv").write_text(
        "case,activity,time\nA1,login,2026-08-03 14:05:09\n"
    )
    mem = InMemoryIngestBackend()
    result = asyncio.run(AbstractionLayer().run(
        user="dev",
        extractor=CsvExtractor(path="events.csv", encoding="utf-8",
                               project_id="RETAIL"),
        backend=mem, schema="MINING", transaction_rows=1000,
    ))
    assert result.records == 1
    rows = mem.tables["MINING"]["JOURNEYS"]["rows"]
    assert rows[0]["STEP"] == "login"
    assert rows[0]["SAMPLE_SET"] == "ORIGINAL"
```

4. Run the suite with `./test.sh backend`, or a single file with `cd backend && ../.venv/bin/python -m pytest tests/test_csv_extractor.py`.

Things worth asserting, all of which the existing integration tests already demonstrate:

What to check	How
Rows and their values	`mem.tables[schema][table]["rows"]`
Missing columns become NULL	Push a partial row and assert the absent column is `None`
The status the console would show	`layer.status_for("dev").public()` — check `state`, `recordsPushed`, `tablesTouched`, `messages`, `lastError`
Transaction bracketing	`mem.commits` and `mem.rollbacks`; 2500 rows at a 1000-row bracket give 3 commits and 0 rollbacks
Failure handling	Raise inside `run` and assert `state == "failed"`, the message in `lastError`, one rollback, and that already-committed brackets remain
Generated SQL	`SqlIngestBackend(run_sql=sql.append)` and assert on the recorded statements — DDL, escaping and the 1000-row batching
Sandbox enforcement	Assert `FileAccessError` for `../secret`, a symlink out of the root, a directory and a missing file
Registry rules	Non-slug ids and duplicate registration both raise `ValueError`

Rules to design against

A short checklist of the behaviours that most often surprise a first-time extractor author:

Nothing is de-duplicated

The layer appends. It never updates, never merges, never checks whether a row exists. Idempotence, if you need it, has to come from your read cursor — and from the caveat about committed brackets above.

One run per user

A second run for the same user fails immediately. Design any scheduler you build around that, and treat the error as "retry later", not as a data problem.

Declare before you push

`define_table` fixes the column set and the column order; unknown columns are a hard error, not a warning.

Count skipped items honestly

An item you cannot turn into a record should increment `skipped`, not raise. Failing the whole run over one malformed source item throws away everything already read.

`existing_keys` is for metadata only

It reads every key in the table. On a fact table with millions of rows that is a full scan on every run.

Report progress, then report it again

Without `progress` calls the console shows an indeterminate bar for the whole run. Call it every few hundred items and give a total as soon as you know one.

The id is permanent

The slug is the plug-in key stored and displayed everywhere. Changing it later orphans anything that referenced the old value.

23 Troubleshooting and FAQ

> This chapter is a problem-and-remedy reference. Each entry names a **symptom** as you actually see it on screen, explains the **cause** in terms of how the application works, and gives the **fix** — as a numbered procedure wherever the fix is something you carry out. It covers sign-in failures, database connection failures, empty process maps, blank AI reports, imports that write nothing or write everything twice, the file watchdog, permission refusals on the integration console, licence expiry and the demo grace period, and an empty project list in the administration interface. Exact on-screen messages are quoted throughout so that you can match what you see against what is written here. Some remedies require an administrator; those carry a role callout. The chapter closes with a short list of frequently asked questions.

23.1 Before you start: where the evidence is

Three places hold nearly all the diagnostic information this application produces.

The message on screen

User-facing messages are deliberately generic where they touch security (so that an error cannot be used to discover whether an account or a connection exists) and specific everywhere else. Quote the message exactly when you report a problem.

The administration Logging tab

All three servers — the main application, the administration interface and the integration console — write into one shared structured log. Each entry carries a timestamp, severity, client IP, user, operation, an optional tag and the message. Filter by **operation** to isolate a subsystem (`login`, `db-connect`, `db-test`, `db-sql`, `db-timeout`, `llm-test`, `materialize`, `import`, `connection`, `backup`, `tls`, `ldap`), or by **tag** to follow a whole activity (`SQL`, `DATA`, `BACKUP/RESTORE`).

The status information in the interface itself

The connection card's status dots, the transitions pill on the process map, the checkpoint panel on a file source, and the integration console's KPI tiles each report state that no log can give you as quickly.

> **Tip** The log records only what the configured severity allows. The default **Max level logged** is **ERROR**, which records INFO, USAGE, WARN and ERROR but not DEBUG. Raise it to **DEBUG** while you reproduce a database problem — that level adds the per-statement SQL trace (tag `SQL`, with the execution time of each statement) — and lower it again afterwards, because DEBUG is voluminous.

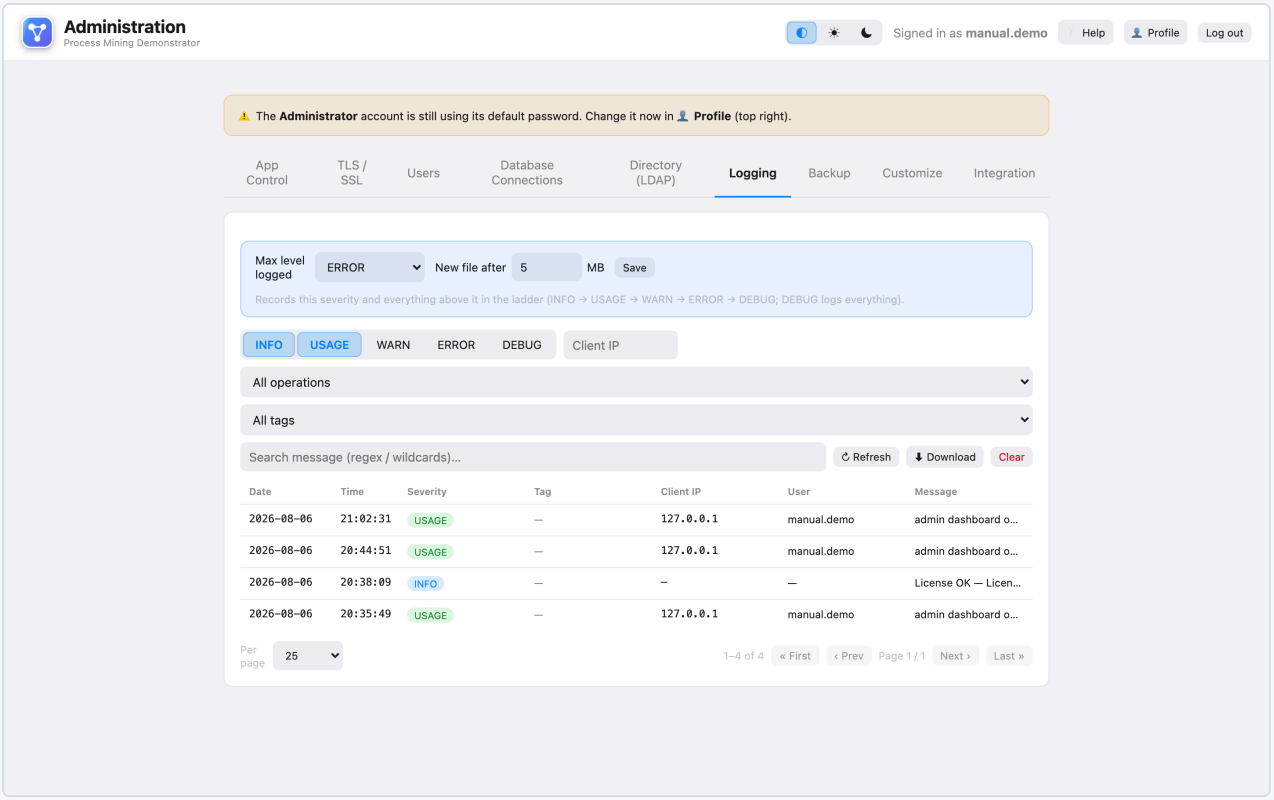


Figure 23.1 — The Logging tab. Filter by severity, client IP, operation or tag, narrow further with a regular expression, then click a row to open the full entry.

23.2 You cannot sign in

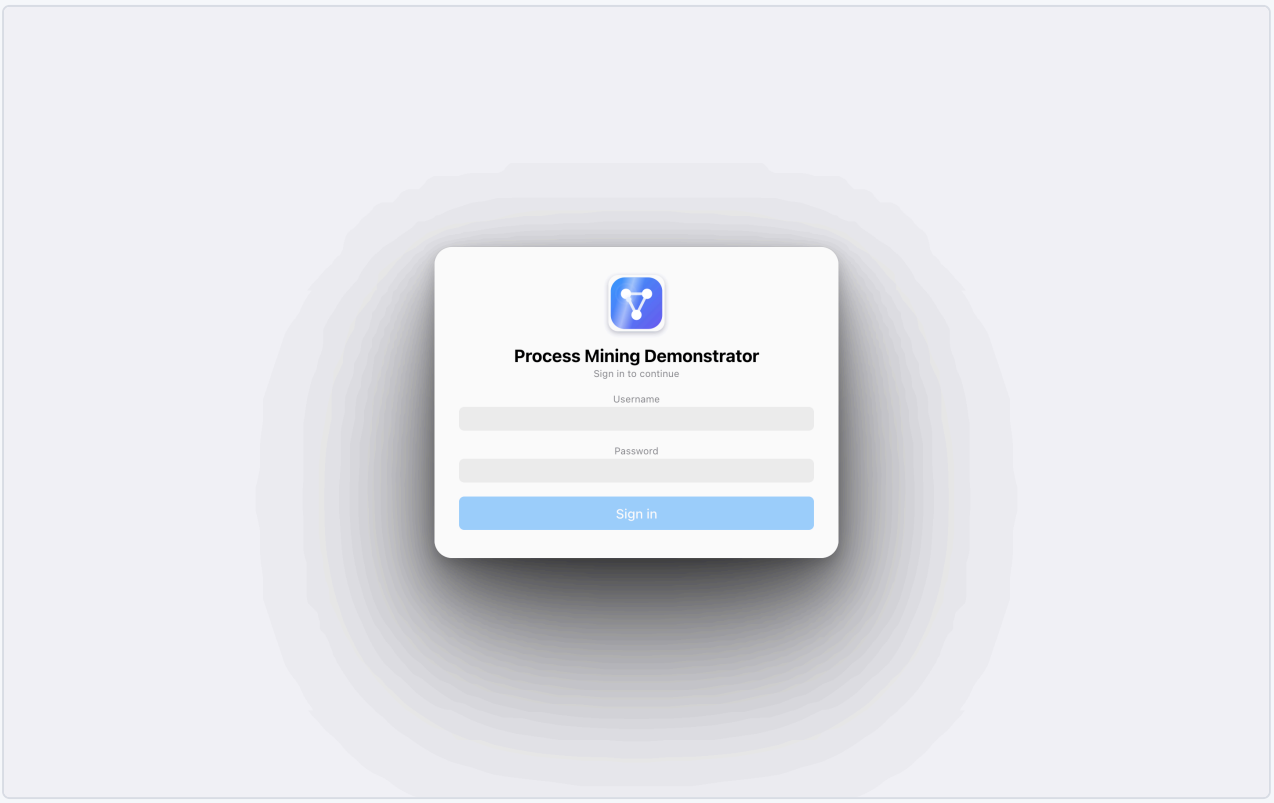


Figure 23.2 — The sign-in panel. Errors appear in a red box above the fields; the orange notices above it report an idle sign-out or an unlicensed installation.

23.2.1 “Invalid username or password.”

Symptom. The main application or the integration console shows “Invalid username or password.” in the red error box; the password field is cleared and the username is kept. On the administration interface the equivalent message reads “Invalid credentials, or the account is not an administrator.”

Cause. The credentials are wrong — or the account is unknown, or it is disabled and the password you typed was also wrong. The message is deliberately identical in all of these cases so that it cannot be used to discover which usernames exist. On the administration interface the same message is also shown for a *correct* password belonging to a non-administrator, for exactly the same reason.

Fix.

1. Retype the password. Note that a failed attempt counts towards the lockout threshold — see 23.2.2 before you try repeatedly.
2. If you are signing in to the administration interface, confirm with an administrator that your account carries the administrator role. A perfectly valid application account gets this same message here.
3. Directory (LDAP) accounts must exist in the directory. If the sign-in panel shows a red LED with “Directory server unavailable”, the directory itself is not answering; local accounts still work.

23.2.2 “This account has been locked after too many failed sign-in attempts.”

Symptom. The full message is “This account has been locked after too many failed sign-in attempts. Contact an administrator.” In the administration interface the account's card carries a red **locked** pill (tooltip “Disabled after too many failed sign-ins — Unlock to restore.”) and its action button reads **Unlock**.

Cause. Consecutive wrong-password attempts against a local account reached the threshold set in **Users → Disable an account after N failed sign-in attempts**, which is **3** on a fresh installation (0 turns the feature off). The counter is shared by all three surfaces, so wrong passwords typed at the application, the console and the administration interface all count towards the same total. A successful sign-in resets it.

Note what does *not* count: wrong two-factor codes, failed passkey ceremonies, directory (LDAP) accounts, and attempts against an already-disabled account. A mistyped authenticator code can therefore never disable an account.

Fix. Any one of the following.

1. **Wait.** A lockout lifts itself after **15 minutes**. The expiry is applied lazily, at the next sign-in attempt, so simply trying again after the window works. This interval is fixed and has no control in the interface — only the threshold is configurable.
2. **Have an administrator unlock the account.** In the administration interface, **Users** tab, find the account and press **Unlock** (or **Enable**). This also clears the failure counter.
3. **Break glass.** If the only remaining administrator is locked out, restart the servers with the environment variable `PMW_RESET_LOCKOUTS=1`, which clears every lockout in the installation.

> **Note** The built-in **Administrator** account is counted but never auto-locked. It is the recovery account, and it must not be possible to disable it merely by knowing its name. It is protected instead by the per-IP throttle and by slow password hashing. The hint under the setting says “incl. the Administrator”; in practice the built-in account is exempt, while every other administrator account is not.

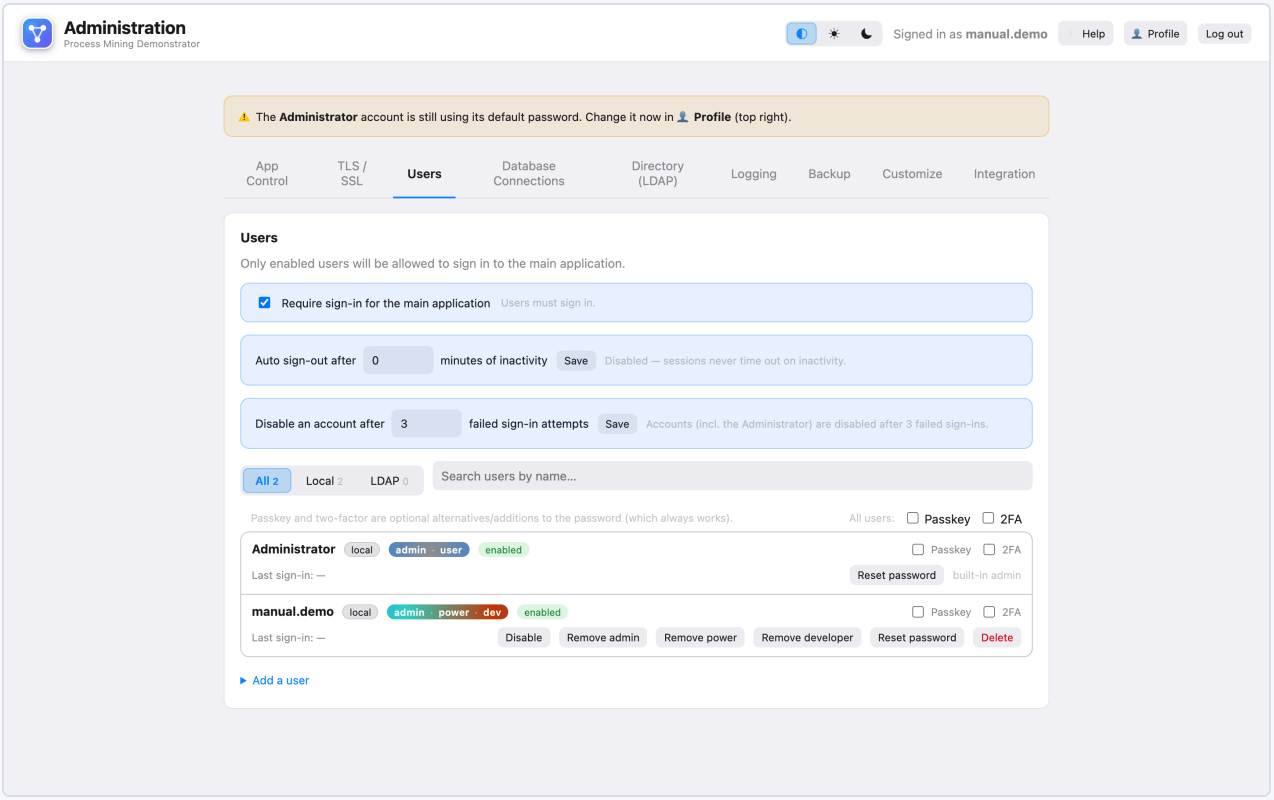


Figure 23.3 — The Users tab. A locked account shows the red locked pill; the Unlock button restores it and clears the counter.

23.2.3 “This account has been disabled.”

Symptom. “This account has been disabled. Contact an administrator.” This message appears only when the password you typed was *correct*; with a wrong password you get the generic message instead.

Cause. An administrator has cleared the account's access, or the account was never enabled. Only enabled users may sign in.

> Requires: **Administrator**

Fix. In the administration interface, **Users** tab, locate the account and press **Enable**. Disabling and re-enabling an account also clears its lockout counter.

23.2.4 “Too many attempts. Try again shortly.”

Symptom. HTTP 429 with “Too many attempts. Try again shortly.” on the application and the integration console, or “Too many failed attempts. Try again in about N seconds.” on the administration interface.

Cause. The per-IP throttle, which is independent of the account lockout: after **3 failures within 15 minutes from one source IP**, that address waits a **5-minute cooldown** measured from its last failure. It applies to password sign-ins, two-factor verification, forced enrolment, the two-factor turn-off confirmation and (in the administration interface) passkey completion. The throttle is held in memory per process and uses the socket peer address; a forwarded-for header is deliberately ignored.

Fix. Wait out the cooldown, then sign in correctly — only a fully successful sign-in clears the counter. Restarting the servers also clears it, since it is not persisted.

23.2.5 The sign-in stops and demands a QR code

Symptom. After the password is accepted the panel does not sign you in. Instead the caption changes to "Set up two-factor to continue" and a QR code appears with the text "Two-factor is required for your account. Scan this with an authenticator app (Google Authenticator, 1Password…), then enter the 6-digit code." There is no way past it except **Back**, which abandons the sign-in.

Cause. This is not a fault. Allowing two-factor for an account makes it **mandatory**. When an administrator ticks the **2FA** box for a user who has not yet enrolled, the user's next sign-in on any surface halts after the password and forces enrolment. No session is issued until a code confirms the authenticator, which is also why there are never half-enrolled accounts.

Fix.

1. Scan the QR code with an authenticator application, or use the "Can't scan? Key" secret shown beneath it.
2. Type the current 6-digit code into **6-digit code** and press **Confirm & sign in**. A wrong code gives "That code didn't match. Try again." and keeps the same QR — you do not need to rescan.
3. The next screen shows ten one-time **recovery codes**. Save them now. They are displayed once, they cannot be recovered later by anybody including an administrator, and each signs you in once if you lose your authenticator. Press **Copy codes**, then **Continue**.

> **Warning** The enrolment QR is valid for **10 minutes**. If you leave it open longer you get "Setup expired. Start again." and must restart from the username and password.

23.2.6 The authentication code is rejected

Symptom. "Incorrect code. Try again." at the code step, or "Your sign-in session expired. Start again." ("… Please start again." in the administration interface).

Cause. Either the code is wrong, expired or a recovery code that has already been used; or more than **5 minutes** passed between the password step and the code step, which is the lifetime of the short-lived "password ok" state. Codes are 6 digits on 30-second steps with one step of clock-skew tolerance either way, so a device whose clock has drifted by more than about a minute will fail every time.

Fix.

1. Wait for a fresh code and retype it. Spaces are ignored.
2. Check the clock on the device running the authenticator; enable automatic time synchronisation.
3. If you no longer have the authenticator, type one of your **recovery codes** in the same field. Dashes, spaces and case are ignored. Each code works once.
4. If both are lost, an administrator must untick **2FA** for your account in the **Users** tab. Your password alone then signs you in and you can enrol again. There is no administrator function that resets two-factor while preserving the enrolment, because the secret cannot be read back.

> **Note** Unticking **2FA** relaxes the requirement but does *not* delete the stored secret. Re-ticking it re-arms the same authenticator with no new enrolment. Only turning two-factor off from the 🔒 **Two-factor** sheet (which requires a current code) deletes the secret and the recovery codes.

23.2.7 Right credentials, wrong surface

Symptom. Your username and password work in one place and are refused in another.

Cause. The three surfaces admit different people, and their session cookies are audience-bound: signing in to one does not sign you in to the others.

Surface	Default ports (HTTP / HTTPS)	Who may sign in	Refusal you see
Main application	8080 / 8443	Any enabled user, in any role	“Invalid username or password.”
Integration console	8100 / 8463	Developers and administrators only — power users are refused	403 “You need the Developer role to use the integration console. Ask an administrator to grant it.”
Administration interface	8090 / 8453	Administrators only; directory accounts only when explicitly permitted and locally promoted	“Invalid credentials, or the account is not an administrator.”

Fix. Ask an administrator for the grant the surface requires: the **Developer** role (Users tab, **Make developer**) for the integration console, or the administrator role for the administration interface. Note that the console's role check runs *before* any second factor, so a refused power user never even reaches the code step.

> **Note** The integration console always requires a sign-in, even when the main application is running in open mode with “Require sign-in for the main application” switched off. The administration interface does too.

23.2.8 “Console unavailable”

Symptom. The integration console answers a browser navigation with a 503 page titled **Console unavailable**: “The integration console has been turned off by the administrator.” Programmatic calls get the same text as JSON.

Cause. The checkbox **Enable the integration console** in the administration interface's **Integration** tab is cleared.

> Requires: **Administrator**

Fix. Tick **Enable the integration console** (toast “Integration console enabled — restart to apply”), then press ↻ **Restart app server** in the **App Control** tab. The change is stored immediately but only takes effect on restart.

23.2.9 The passkey button is missing, or a passkey sign-in fails

Symptom. The 🔑 **Sign with Passkey** button does not appear; or it appears and produces “Passkey sign-in failed.” (“…Check the passkey is registered and still enabled.” in the administration interface); or enrolment reports “Passkeys need a hostname, not an IP address.”

Cause. One of four:

- The browser does not support WebAuthn (“This browser does not support passkeys.”).

- The page was reached by a bare IP address or a single-label host. WebAuthn binds a passkey to a domain name and rejects both, so the controls hide themselves. Off localhost, a secure context (HTTPS) is required as well.
- An administrator has not allowed passkeys for the account, or has revoked the permission — which blocks passkey sign-in immediately, existing passkeys included.
- You did not type a username. The flow is username-first: type the name, then press the button.

Fix.

1. Reach the server by a resolvable name over HTTPS — its `.local` Bonjour name or a DNS entry — and issue the TLS certificate for that name. Password sign-in works over an IP either way.
2. Ask an administrator to tick **Passkey** for your account in the **Users** tab, then enrol from 🔑 **Passkeys** next to **Sign out** while signed in with your password.
3. For a deployment that splits the application and the administration interface across sub-domains, set `PMW_PASSKEY_RP_ID` to the shared parent domain and list the expected origins in `PMW_PASSKEY_ORIGINS`.
4. If enrolment reports “A passkey for this account already exists on this device.”, remove it from the device's own passkey settings (on iOS: Settings → Passwords) and try again.

23.2.10 You are signed out unexpectedly

Symptom. The application returns to the sign-in panel, sometimes with the orange notice “You were signed out due to inactivity.”; or requests start failing with 401 “Authentication required.”

Cause. One of the session rules below.

Rule	Value	Effect
Idle auto sign-out (main application)	Set per installation in **Users** (0–1440 min; 0 disables)	Signs you out after N idle minutes and shows the inactivity notice
Idle auto sign-out (administration)	A separate setting in **App Control → Admin session**	Redirects to the sign-in page with the same notice
Session lifetime with no idle timeout	12 h application and console, 8 h administration	The cookie simply expires
Absolute session lifetime	7 days from the original sign-in	Ends even a continuously active session
Sign-out anywhere	Immediate	Invalidates every one of your sessions on all three surfaces
Password change or administrator reset	Immediate	Invalidates all your sessions (an administrator changing their own password keeps that one session)
Account disabled, deleted, or role revoked	Immediate	The session dies at the next request

Fix. Sign in again. If it happens constantly, ask an administrator to review the idle timeout. If you lose the integration console specifically and see the **Access denied** screen (“The integration console is available to developers and administrators only…”), your Developer role was revoked mid-session.

Note The integration console has no client-side idle watcher. Its session expires server-side after the same idle window as the application, but nothing bounces you to the sign-in screen — you discover it when your next action fails.

23.3 Database connection failures

When you tap a connection card, the first status dot turns orange and an error is reported. The messages a normal user sees are deliberately generic: no Exasol codes, internal hostnames or SQL-state fragments reach the browser. The full driver detail is written to the log under operation `db-connect`. Power users and administrators running **Test** see the same categories with the raw driver detail appended.

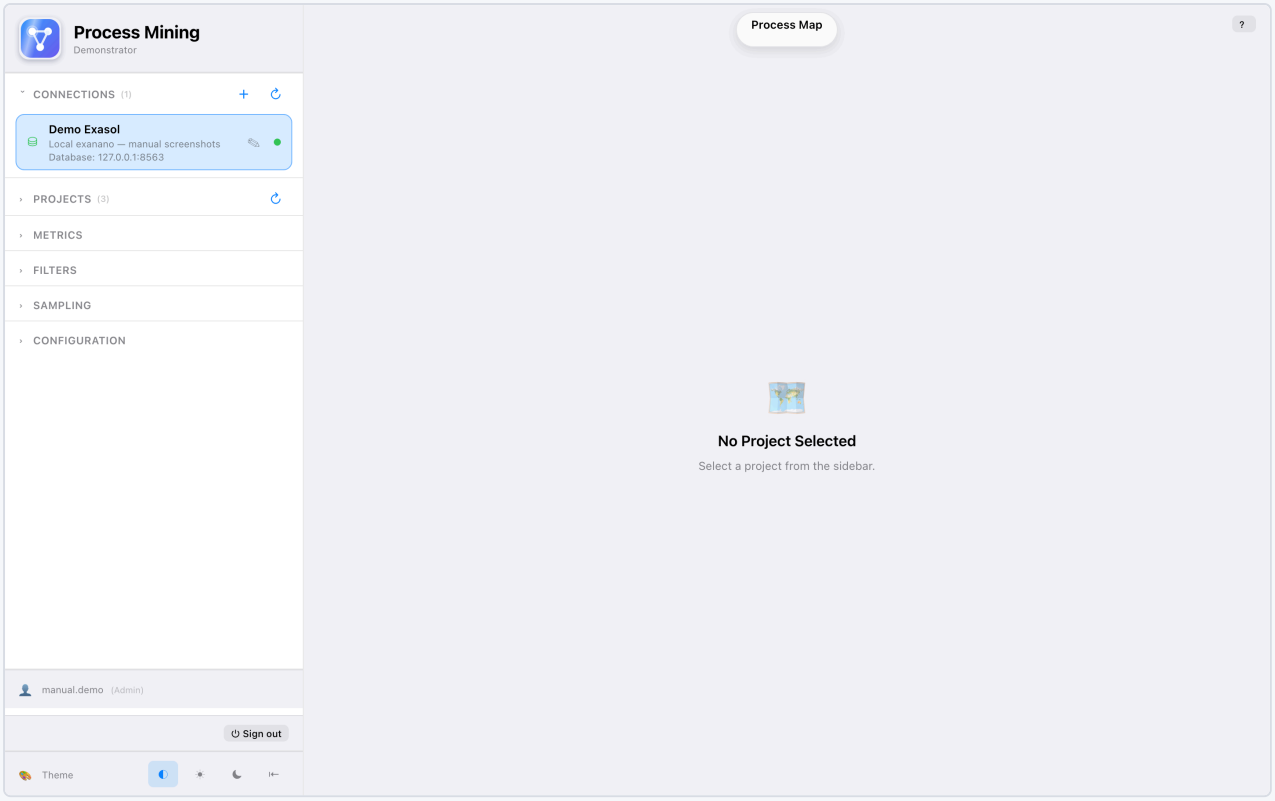


Figure 23.4 — The Connections section. The first dot is the database state (green connected, orange connecting or failed); the second, when present, is the LLM server.

23.3.1 The message catalogue

Message	Cause	Fix
“Authentication failed — check your username and password.”	The database rejected the credentials stored on the connection.	The connection's database user and password are managed by an administrator (or by the power user who created it). Ask them to correct and re-test it. You never type a database password into the application.
“Connection timed out. Verify the host address, port, and network connectivity.”	The socket did not open within the fixed **15-second** connect timeout — usually a firewall silently dropping packets, or an unreachable network.	Check routing and firewall rules between the server and the database host and port (8563 by default).
“Connection refused. No server is accepting connections at this host and port.”	The host answered but nothing is listening on that port.	Verify the port number and that the database is running.
“Host not found. Check the hostname spelling and your DNS / network connectivity.”	Name resolution failed.	Correct the hostname, or use an address that resolves from the application server.
“TLS certificate error. Try 'Skip verification' or configure a fingerprint in the server's security settings.”	The connection uses TLS in **Verify (system trust store)** mode and the server's certificate is not trusted, has expired, or does not match the hostname.	See 23.3.2.
“This connection is set to pin a certificate fingerprint, but the fingerprint is missing or not hexadecimal. Enter the server's SHA-256 fingerprint, or change the certificate mode.”	Certificate mode is **Pin fingerprint** but the fingerprint field is empty or contains something that is not hexadecimal.	See 23.3.3.
“The database rejected the request.”	An Exasol query error — most often a missing schema, a missing table, or no SELECT privilege.	Check the schema name on the connection and the account's privileges (see 23.3.4). The detail is in the log.
“Connection failed. Check the host, port, and credentials.”	Fallback for anything not classified above.	Read the log entry under operation `db-connect`.
“This connection changed during sign-in. Please reconnect.”	An administrator edited or deleted the connection while your (slow) connect was still in flight, so the freshly opened session was dropped.	Refresh the Connections list with ↻ and connect again.
“This connection is not available to you.” (403) / “Connection not found.” (404)	The connection is no longer assigned to you, or no longer exists.	Refresh the list; ask an administrator to reassign it.

Message	Cause	Fix
“Not connected to a database.” (409)	A data request arrived with no live session — typically after a forced disconnect.	Tap the connection card again.

23.3.2 TLS certificate errors

Cause. Certificate mode is **Verify (system trust store)**, the default, and the database's certificate cannot be validated against the trust store — a self-signed certificate, a private CA the server does not know, an expired certificate, or a hostname mismatch.

> Requires: **Power user** (own connections) or **Administrator**

Fix. Open the connection editor and choose the mode that matches your situation.

1. **Verify (system trust store)** — correct for a certificate issued by a CA the server trusts. If that is what you intend, install the CA certificate in the server's trust store rather than weakening the connection.
2. **Pin fingerprint** — correct for a self-signed or private-CA certificate whose identity you can confirm out of band. Paste the server's SHA-256 fingerprint into **Fingerprint (SHA-256)**. Colons are stripped; the value must be pure hexadecimal, 16 to 128 characters.
3. **Accept any (insecure)** — disables certificate *and* hostname verification. Acceptable only in a laboratory.
4. Press **Test connection**, then **Save**.

> **Warning** Mode parsing fails closed: any stored value that is not recognised — a typo, or legacy data from a restored backup — is treated as **Verify**, never as no verification. A connection that used to work after a restore and now reports a certificate error has most likely landed in this state.

> **Note** The field **Minimum RSA key size** (default 2048) is stored and carried through every operation for compatibility with the macOS application's connection records, but the web backend does not currently enforce it. Do not rely on it to solve a handshake problem.

23.3.3 Fingerprint missing or mismatched

Symptom. The connection refuses to open with the “set to pin a certificate fingerprint” message, or — where the fingerprint is well-formed but wrong — with the generic TLS certificate error.

Cause. In **Pin fingerprint** mode the driver pins the server's certificate to the value you supplied. A blank or non-hexadecimal value is refused outright rather than silently falling back to no verification, which is why the message is so specific. A well-formed but incorrect fingerprint means the certificate on the server is not the one you pinned — either it was legitimately replaced, or you are not talking to the server you think you are.

Fix. Obtain the current SHA-256 fingerprint from the database administrator, paste it into the field and test. If the certificate was rotated, update the pin. If it was not rotated, do not update the pin — investigate.

23.3.4 Connected, but features fail

Symptom. The connection opens, but a specific feature reports an error while the rest of the application works.

Cause. The database account lacks a privilege that only that feature needs.

Feature	Required database privilege
Every read feature (maps, charts, KPIs)	SELECT on PROJECTS, JOURNEYS, STEPS, METAS
Notes (first use creates the table)	CREATE TABLE, then INSERT and DELETE on NOTES
Journey sampling (adds the SAMPLE_SET column)	ALTER TABLE and INSERT on JOURNEYS
The in-application Step Editor	UPDATE on STEPS
Create schema & tables, demo content	CREATE SCHEMA, CREATE TABLE and (for demo data) INSERT

Fix. Ask the database administrator to grant the missing right. The application cannot grant database privileges to itself.

23.3.5 “Test connection” fails although the saved connection works

Symptom. A connection that users connect to happily reports an authentication failure the moment you press **Test connection**, **Create schema & tables** or **Generate** in the editor.

Cause. These three operations use the password *as typed in the form*, never the stored one. On a saved connection the password field is blank (it shows “•••••• (unchanged)” or the hint “(set — leave blank to keep)”), so the test runs with an empty password.

Fix. Retype the password into the field before testing or provisioning. Leave it blank again — or simply press **Cancel** — if you do not want to change the stored secret; a blank field on save keeps the stored value unchanged.

23.3.6 The Connections list is empty

Symptom. The sidebar **Connections** section lists nothing.

Cause. No connection has been assigned to your account. Each user sees only their assigned connections.

Fix. Press ↻ in the Connections header first — the list does not refresh itself after an administrator grants access. If it is still empty, ask an administrator to assign one in **Database Connections**. If you hold the power role, create your own with the **+** button; every connection you create is assigned to you automatically.

> **Note** Saving any change to a connection, changing its assignments, or deleting it immediately disconnects every user currently using it. That is intentional — a revoked assignment must take effect at once — but it means a connection can drop out from under you while you work.

23.3.7 “Unable to load data” after the page has been idle

Symptom. After the browser has been left open and idle for a while — or after an automatic idle sign-out and a fresh sign-in — the first action (a refresh, opening a project, loading a chart) fails with *“Unable to load data”*, and in older builds you had to reconnect the connection and reselect the project before anything worked again.

Cause. The backend keeps one long-lived database session per connection. A network device between the server and the database — very commonly the `host.docker.internal` NAT when the database is reached from inside a container — silently drops a socket that has sat idle, while the backend still believes the session is open. The next statement then meets a dead socket and the driver raises *“Exasol connection was closed”*.

Fix. None is needed on current builds. The backend now detects a dropped-socket error, transparently reopens the connection and retries the statement once, so the action simply succeeds. A genuine query error (bad SQL, a missing object) is *not* retried — it surfaces immediately as before. Only if the reopen itself fails (the database is genuinely down or the credentials no longer work) will you see a connection error, and then the remedies in 23.3.1 apply. If you are on an older build and still see this, reconnect the connection once; upgrading removes the need.

23.4 The process map is empty or wrong

23.4.1 No nodes appear

Symptom. A project is selected but the map area stays empty, or **Filtered Journeys** reads 0 while **Total Journeys** is greater than 0.

Cause and fix, in the order worth checking:

1. **The date window excludes your data.** When a project is first selected, A-Chart loads the last **30 days** before the latest event (**Configuration** → **Default date window**; **0** loads the full range). Widen the slider, or set the default to 0 and reload the project — the setting takes effect on the next project load and does not move the slider on the current map.
2. **The filters exclude everything.** Every filter works at journey level: a journey qualifies in full or not at all. An **Exclude Steps** entry, a narrow **Num Steps**, **Journey Time** or **Journey Score** range, or a meta value with a typo will empty the map. Press **Reset**, then **Apply**.
3. **You did not press Apply.** Filters are never applied automatically. In A/B Comparison, **Apply** reloads only the active side — the one marked with the pencil icon and the “editing” label.
4. **You are looking at B-Chart.** B-Chart starts empty by design. Apply filters, or press **Load** on its empty state.
5. **The selected sample set is empty.** If the A or B slot in the **Sampling** section points at a sample that was deleted, or at a stored simulation result that was never produced, there is nothing to draw. Switch the slot back to **Original data**.

23.4.2 The map shows “⚠ Live (not built)”

Symptom. The pill in the chart header reads ⚠ **Live (not built)** instead of ⚡ **Pre-materialized**, and interactive filtering is slower than expected.

Cause. The connection has **Use pre-materialized transitions** switched on, but the `TRANSITIONS_RAW` table is missing, has never been built, or could not be read. This is a fail-safe, not a fault: the map falls back to the live windowed query so it never breaks. The exact cause is logged under operation `materialize`.

> Requires: **Administrator**

Fix.

1. Open **Database Connections**, edit the connection, and press **Rebuild now** in the **Use pre-materialized transitions** banner. The status line then reads "Last built: <date> · N pairs."
2. If the rebuild fails, read the red status text and the `materialize` entries in the log.
3. Schedule a rebuild as the last step of every load of JOURNEYS. The table is a snapshot and goes stale the moment the event log changes; generating demo data and creating or deleting samples do not rebuild it either. Use the per-connection bearer token and `POST /api/connections/<id>/rebuild-transitions` on the administration server for that.

> **Note** Two rebuilds of one connection never overlap — the second gets 409 "A rebuild is already running for this connection." Token-triggered rebuilds also carry a 60-second per-connection cooldown (429 "Rebuild throttled — try again in Ns."); an administrator pressing **Rebuild now** is never throttled.

23.4.3 A KPI tile shows "—", or Statistics reports a timeout

Symptom. The **Process Goodness** tile shows a dash, or the Statistics view reports "Statistics query timed out (30 s). Narrow your date range or filters and try again."

Cause. Heavy statistics queries (variants and routes, and the goodness metric) carry a **30-second** timeout, configurable with `PMW_QUERY_TIMEOUT`. The goodness query groups journeys by path pattern, which is expensive on a large, heavily filtered dataset. Timeouts are logged under operation `db-timeout`. A dash can also simply mean that *no step has a SCORE* — with no scores configured in STEPS the tile is hidden altogether rather than dashed.

Fix. Narrow the date range or add filters and reload. For a lasting improvement, index `JOURNEYS (PROJECT_ID, EVENT_TIME)` and `JOURNEYS (PROJECT_ID, EVENT_ID)`, and consider enabling pre-materialized transitions on the connection.

23.5 AI supported Documentation stays blank

Symptom. The AI supported Documentation view produces no report, or produces only the locally computed sections.

Cause. The view sends the current A-Chart transition table and your prompt template to an OpenAI-compatible endpoint configured *on the active connection*. Without a reachable model server there is nothing to render. The journey-paths table, happy-path conformance, the conformance gap analysis and your notes are computed locally and never sent to the model — which is why those sections can appear while the model's answer does not.

Fix.

1. Look at the connection card. A second status dot appears only when an LLM server is attached, and it must be **blue**. Orange means the model server did not answer; no second dot at all means no LLM is configured on this connection.
2. Ask an administrator (or the power user who owns the connection) to fill in **Server URL**, **Model** and, if required, **API key** on the connection's **LLM** tab, and to press **Test connection**. A successful probe reports “LLM: reachable — N models”; a failure reports “LLM server not reachable.”
3. The probe is a `GET <serverURL>/models` with a 5-second timeout. A server that is merely slow to start will read as unreachable; retry once it is up.
4. If the URL looks correct and the probe still fails, check the log under operation `llm-test`. The outbound SSRF guard refuses some URLs outright and records why — for example “Only http:// and https:// URLs are allowed.” or a refusal because the host resolves to a link-local or cloud-metadata address. Such a refusal is never shown to end users; the model simply reads as unreachable. Loopback and private addresses are allowed by default so that local model servers work, unless the deployment sets `PMW_BLOCK_PRIVATE_LLM_HOSTS=1`.

> **Warning** AI models can produce results that are incorrect, incomplete or misleading. Verify every finding independently before acting on it.

23.6 Import problems (integration console)

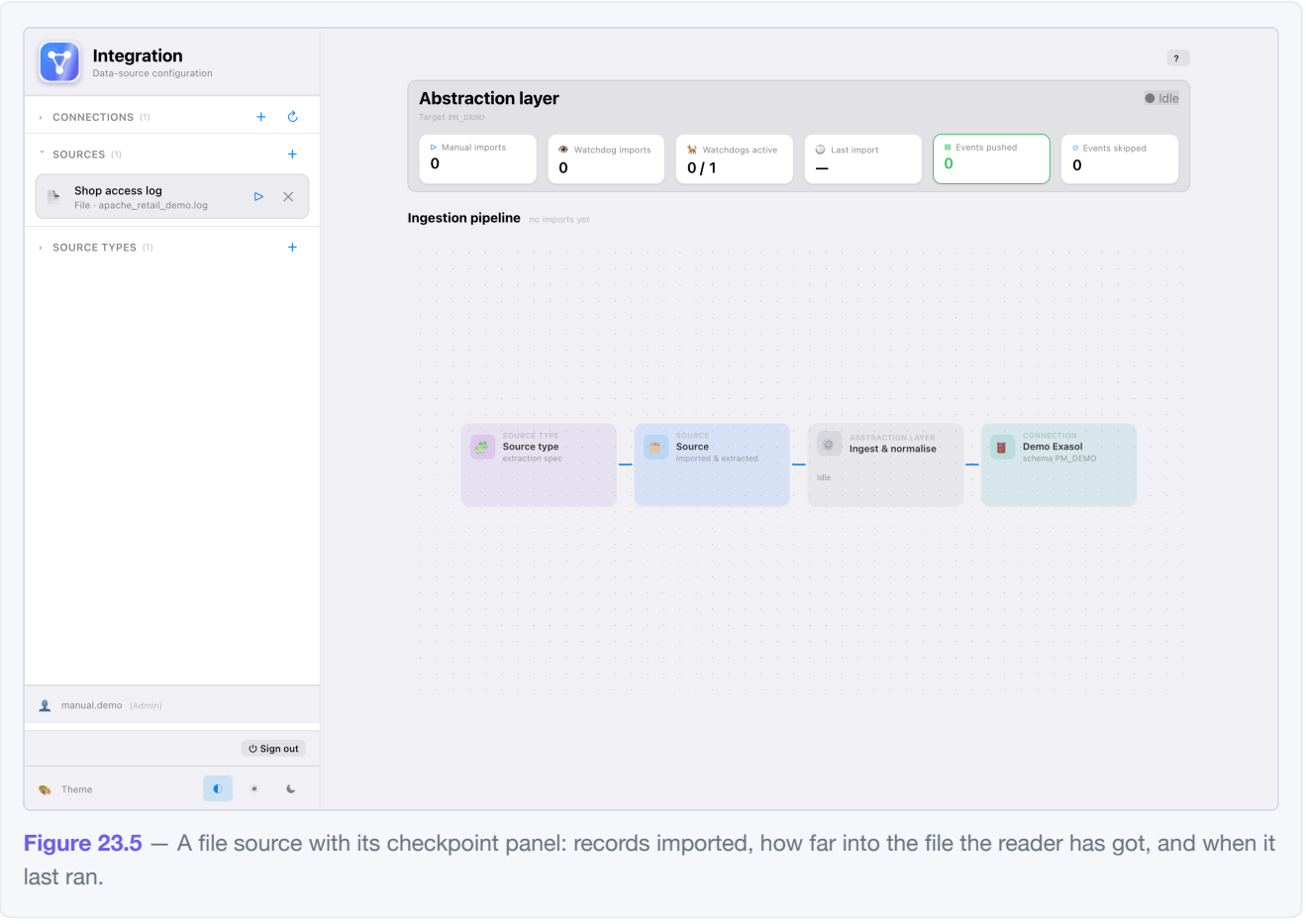


Figure 23.5 — A file source with its checkpoint panel: records imported, how far into the file the reader has got, and when it last ran.

23.6.1 The import wrote zero rows

Symptom. A run finishes without error but nothing new appears in the project.

Cause. There are two entirely different reasons, and the run result distinguishes them. This is why the result reports `linesRead` separately from `records`.

What the run reports	What happened	Fix
"Nothing new to import — the file has not grown since the last import."	A delta run found no bytes appended past the checkpoint. Normal, not an error.	Nothing to do. Append to the file, or switch delta upload off to read from the top again.
"Nothing to import — the file is empty."	The file has no content at all.	Check the path. Remember relative paths resolve inside the server's integration files directory.
Lines were read but **0 events written** and a high **Events skipped** count	A real misconfiguration: the source type's regular expressions do not match this log. A line becomes an event only if the case id, the step *and* the timestamp are all captured and the timestamp normalises.	See below.

Fix for the third case.

1. Open the linked source type and paste a genuine line from the log as the sample. Each field row shows live what it captured; a wrong pattern is immediately visible.
2. Check that a field with the role **EVENT_TIME** exists — without one the run is refused with "The source type has no timestamp field."
3. Check the **Example JOURNEYS record** panel. It shows exactly the row the specification would produce, marks the id as "stored as MD5" and marks a compound-derived step with "compound".
4. Give each field's regular expression exactly one capturing group — that group is the value. Without a group the whole match is used.
5. Remember that every line is truncated to its first **2000 characters** before matching. A pattern aimed at content beyond that will never match.
6. Verify the timestamp interpretation. Numeric dates default to **day-first** (European); a 12-hour AM/PM clock flips the interpretation to month-first. Yearless syslog timestamps are given the current UTC year, which mis-years an old log imported around the turn of the year.

> **Tip** A non-zero **Events skipped** KPI is highlighted in orange for exactly this reason. It exists so that a silently misconfigured source type shows up as "lots skipped" rather than merely as a small event count.

23.6.2 Only part of a large file was imported

Symptom. A first import of a large existing file stops well short of the end; running it again imports more.

Cause. One read — a manual run or a watchdog poll — consumes at most **8 MB** of new bytes. A larger backlog is drained across successive reads, cap by cap. A single "line" longer than the whole 8 MB cap is treated as unparseable content and skipped past rather than re-read for ever.

Fix. Press ▷ again (or let the watchdog poll again) until the checkpoint reaches the end of the file. The checkpoint panel shows how far into the file the reader has got.

23.6.3 Duplicate events after re-importing

Symptom. After importing the same file twice the project holds every event twice; journey counts and transition counts are inflated.

Cause. Events are never de-duplicated. The importer appends one JOURNEYS row per parsed event; it does not check whether an identical row already exists. Three routes lead here:

- Switching **delta upload** off, which re-reads the file from the top.
- Pressing ↺ **Reset checkpoint**, which forgets the read position but deletes nothing from the database.
- Retrying after a run that failed part-way. On failure the currently open transaction bracket is rolled back, but brackets already committed stay committed — while the checkpoint does *not* advance for a failed run. The retry therefore re-reads lines whose rows are already in the database.

Fix.

1. To reload a project genuinely: delete the project first, in the connection editor's **Projects** tab, then import again. Deletion clears the project's rows from PROJECTS, JOURNEYS, STEPS, METAS, NOTES and TRANSITIONS_RAW and cannot be undone — notes attached to the project go with it.
2. To make a failed run leave nothing behind, set the source's **Transaction bracket** to **0**, which commits the whole import as a single transaction: all of it lands, or none of it. The default is **5000** rows; any other value is clamped to the range 1000 to 1,000,000, and a non-numeric value falls back to 5000.
3. Leave delta upload on for routine top-ups. Running the same source twice then adds only what was appended since the last run.

> **Warning** There is one checkpoint per source, shared by manual runs and the watchdog. Resetting it also makes the watchdog re-read the file from the start. Changing a source's **file path** deletes the checkpoint automatically, so the next import — manual or watchdog — reads the new file from the beginning.

> **Note** Changed compound-step rules apply only to *future* imports. Rows already written keep the step names they were given. To re-apply changed rules to existing data, delete the project and import again.

23.6.4 The watchdog is not importing

Symptom. A file source is marked with 👁 but its checkpoint never advances, or the console reports no watchdog imports.

Cause and fix. Work down this list; the failure reason is recorded on the source's checkpoint as **lastError** and also written to the log (tag `DATA`, operation `import`) as “watchdog import failed: '<source>' — <message>”.

Recorded message	Cause	Fix
“Watchdog is missing a file path, project id or connection.”	The watchdog block is incomplete.	Open the source wizard and fill in all three.
“The source has no source type linked.”	No parsing recipe is attached.	Link a source type.
“The destination connection is not assigned to you.”	The assignment is re-checked on *every* poll. Revoking it stops the watchdog at once.	Ask an administrator to reassign the connection to the source's owner.
“The watchdog's destination connection no longer exists.”	The connection was deleted.	Point the watchdog at an existing connection.
“The destination connection has no target schema.”	The connection's schema field is blank.	Set the schema on the connection.
“File not found.” / “Not a regular file.” / “That path is outside the allowed sources directory.”	The sandboxed path resolver rejected the path. By default only files under the server's integration files directory may be read; traversal and symlink escapes are refused.	Move the file into the sandbox directory, or use a path inside it. An operator may opt out with `PMW_INTEGRATION_ALLOW_ANY_PATH=1` in trusted deployments.
“Could not connect: …”	The destination database is unreachable with its stored credentials.	Test the connection in its editor.
“Import failed: An extraction is already running for this user.”	Only one extraction runs per user at a time; a manual run and a watchdog poll collided.	Nothing — the watchdog retries on its next poll.

If no error at all is recorded, check the two switches that silence the watchdog wholesale:

1. The per-source switch in the wizard, under the file path, must be on.
2. The deployment-wide switch `PMW_INTEGRATION_WATCHDOG` must not be off. When it is off the console's status reports `watchdogEnabled: false`, and a source marked “enabled” never polls. The base loop tick is `PMW_INTEGRATION_WATCHDOG_TICK` (default 10 seconds, minimum 5); a source's own **poll interval** defaults to 30 seconds with a minimum of 5.

> **Note** The watchdog runs headless, with no signed-in session. It acts as the source's owner and opens the destination with the connection's stored credentials — which is precisely why the destination connection and project are stored *with the source* rather than picked at run time, and why the assignment is re-checked on every poll.

23.6.5 Permission denied on the integration endpoints

Symptom. An integration request fails with HTTP 403 and one of two messages: “You need the Developer role to use the integration features.” or “The integration console is turned off.”

Cause. Every integration endpoint sits behind one router-level gate on the shared compute backend, so it protects the API no matter which surface proxies the call. In order, the gate checks: whether the deployment runs without sign-in at all (in which case the surface is open); whether the administrator has switched the in-

tegration surface off; and finally whether the signed-in user exists, is enabled, and holds the **Developer** or administrator role. Power users are deliberately refused — the check exists precisely so that the main application's proxy cannot be used to bypass the console's own role gate.

> Requires: **Administrator**

Fix.

1. For “You need the Developer role…”: in the administration interface, **Users** tab, press **Make developer** on the account (logged as “granted the Developer role for '<user>'”). The role is re-checked on every request, so a grant takes effect immediately — and a revocation drops the user to the **Access denied** screen mid-session.
2. For “The integration console is turned off.”: tick **Enable the integration console** in the **Integration** tab and restart from **App Control**.

23.6.6 Other refusals when running a source

Message	Meaning
“Only file sources can be run yet.”	The source kind is not `file`; file is the only kind implemented.
“This source has no file path.”	Fill in the path in the source wizard.
“Link a source type to this source first.”	No source type is linked, or the linked one is not yours.
“Connection not found.” (404)	The chosen destination is not assigned to you, or does not exist. The same message covers both, so a connection id alone reveals nothing.
“‘<name>’ has no target schema configured.”	Set the schema on that connection.
“Pick a destination connection first.”	No connection was named and no session connection is active.
“The watchdog's destination connection is not assigned to you.”	Raised when *saving* a source with the watchdog enabled — a fail-fast version of the per-poll check.
“Source settings are too large.”	The serialised source configuration exceeds 8000 characters.

23.7 Licence expiry and demo mode

23.7.1 The application stops responding after a while

Symptom. The sign-in panel carries the orange banner “**Demo Mode** — remaining time: N min”, then “**No License installed**”. Eventually the application's API calls fail with 502 “The compute backend is not reachable. Start it with ./run.sh or check PMW_BACKEND_URL.” The administration interface keeps working normally.

Cause. The application requires a valid licence. Without one it runs for a **one-time grace period** — 30 minutes by default, set with `PMW_LICENSE_GRACE_SECS` — after which the **compute backend stops**. The grace window is granted once per installation and stamped in a marker file; restarting does not renew it, and delet-

ing a licence cannot reset it. The administration interface is deliberately left running so that a licence can always be applied.

> Requires: **Administrator**

Fix.

1. Open the administration interface (it is still reachable) and go to **App Control** → **License**.
2. Choose the issued `license.json` file and press **Upload license**. The signature is verified before anything is stored; a forged or corrupt file is rejected with the specific reason and nothing is written.
3. On success the panel reports “License installed. The backend applies it within a few seconds.” No restart is needed — the backend polls for it. Uploading a valid licence *during* the grace period cancels the pending shutdown.
4. Confirm the green status banner: “Licensed to '<name>', valid until <date>.”

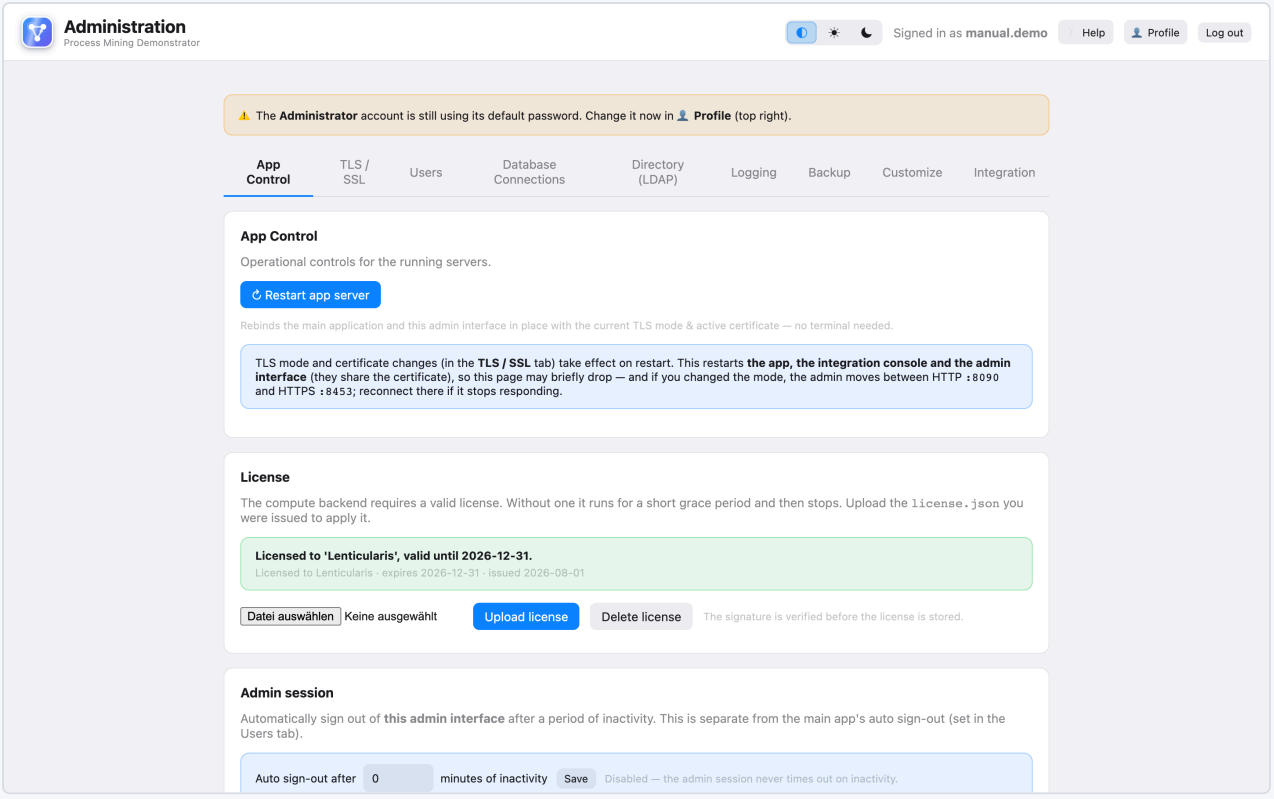


Figure 23.6 — App Control. The License card shows who the installation is licensed to and when the licence expires, and accepts a replacement without a restart.

23.7.2 The licence status banner is red or orange

Banner	Meaning and remedy
Orange: “License for '<name>' expired on <date>.”	The licence is genuine but out of date. Obtain a renewal and upload it.
“No license installed.”	Nothing has been uploaded, or the licence was deleted. Upload one.
“License signature is not valid.” / “License signature is malformed.”	The file was altered or is not the one you were issued. Request a fresh file; do not edit a licence by hand.
“License file is not valid JSON.” / “License file is missing 'license' or 'signature'.”	The wrong file was uploaded, or it was truncated in transit.
“License has no valid expiry date.” / “License file could not be read: …”	Contact your supplier with the exact text.
413 “License file is too large.” / “Invalid file content.”	The upload exceeded the size cap or was not decodable. Check that you selected the right file.

> **Note** 502 “The compute backend is not reachable…” and 504 “The compute backend timed out.” are proxy errors, not licensing errors as such. They mean the compute backend is down or overloaded — which an expired grace period causes, but so does a backend that was never started.

23.8 Administration interface problems

23.8.1 The projects list is empty

Symptom. In **Database Connections** → **Projects** the table shows “No projects found in this schema.”, “Save the connection first to list its projects.”, or a red “Could not read projects.”

Cause and fix, by message:

State	Cause	Fix
“Save the connection first to list its projects.” (the tab button is disabled)	The listing uses the connection's *stored* credentials, which do not exist until the connection is saved.	Press **Save**. The Projects tab unlocks when the editor reloads.
“No projects found in this schema.”	The schema really is empty; or the schema field points somewhere else; or the tables have not been provisioned. A missing table is tolerated rather than reported, so a schema without PROJECTS or JOURNEYS yields an empty list rather than an error.	Check the **Schema** field on the **Database** tab, press **↻ Refresh**, and use **Create schema & tables** if the schema has not been provisioned.
“A schema name is required.”	The connection has no schema configured. A schema is optional for connecting but required for listing, provisioning and demo generation.	Fill in the schema and save.
Red “Could not read projects.” or a database error	The stored credentials fail, or the account lacks SELECT on PROJECTS and JOURNEYS. Note that this listing does *not* use the password typed in the form.	Press **Test connection** after retyping the password, correct the stored credentials, and check privileges.
Journey or event counts look too low	The counts deliberately consider **ORIGINAL data only** — sample sets are excluded. Journeys are distinct EVENT_IDs; events are rows.	None. This is by design.

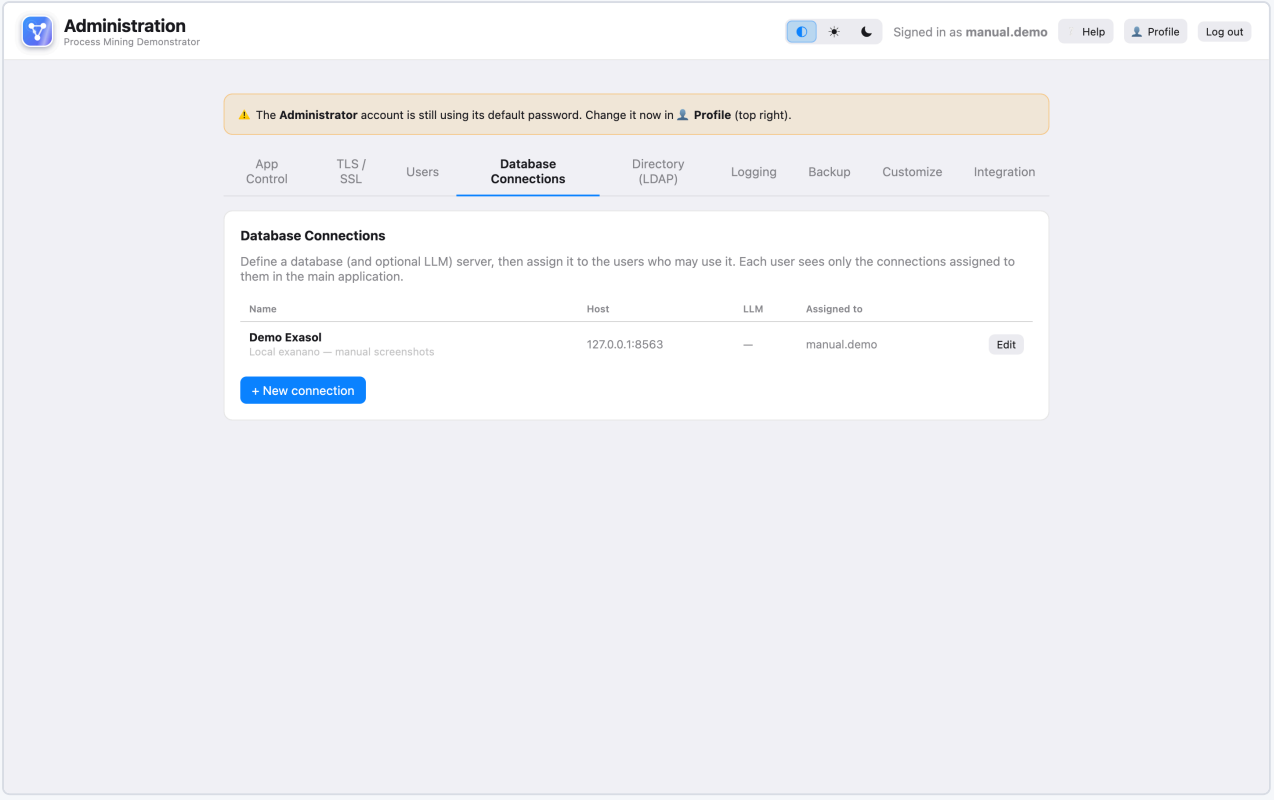


Figure 23.7 — The connection editor's Projects tab lists what is actually stored in the schema, with journey and event counts for the original data.

Tip A project appears in this list if it exists in *either* PROJECTS or JOURNEYS, so an orphan on one side is still visible. An untitled project displays its raw project id as the title.

23.8.2 The administration page stops responding after a restart

Symptom. You pressed ↻ **Restart app server** and the administration page no longer loads.

Cause. One restart rebinds all three servers — the main application, the integration console and the administration interface — because they share the TLS mode and the active certificate. If you changed the TLS mode, the administration interface itself moves between `http://…:8090` and `https://…:8453`.

Fix. Reconnect on the other address. If the restart fails outright, the exact messages are: 409 “The app server is not managed by the launcher (no PID file). Start it with `./run.sh`, then try again.”, 409 “The app server process is not running.”, 500 “Invalid PID file.” or 500 “Could not signal the app server: <os error>”.

Note If a mode requires HTTPS but no certificate is active, every server falls back to HTTP on restart so that nothing — including the administration interface — becomes unreachable. The TLS tab warns about this in advance: “No active certificate — HTTPS cannot start. Generate or upload one and activate it below.”

23.9 Frequently asked questions

Why can I not see the Conformance Check, Happy Path and Simulation views?

They are shown only to power users and administrators. They are also hidden — for everybody — when the installation runs in open mode with sign-in switched off, because there is then no identity to hold a role. Their APIs answer 403 in that state. There is no way to have both no-login access and the power features at once.

Why can I not see the Sampling section?

Same reason. Sampling rewrites the shared sample sets for everyone on the connection, so it is a power-user capability rather than a personal one.

Why does the map look different for a colleague using the same project?

Filters, presets, node layouts, KPI arrangement and personal settings are per user. Notes, sample sets and everything in the STEPS table are shared, because they live in the database.

Why can I not edit a connection that appears in my sidebar?

The ✎ button appears only on connections you created. A connection an administrator created and assigned to you can be used but not edited.

I typed a friendly reference into the Event ID field and got nothing.

Anything that is not already a 32-character hash is MD5-hashed before the query. In the demo datasets the stored EVENT_ID is the MD5 hash of a synthetic reference such as `ORD-000001`, so typing that reference works. Imported data behaves the same way: the integration console always stores EVENT_ID as the MD5 of the captured id, so the raw identifier cannot be recovered from the database.

Why did my filter preset not update after I changed the filters?

A preset is a snapshot taken at the moment you saved it. Later filter changes do not update it. Save a new preset, or rename and re-save.

Why is the Process Similarity badge missing in A/B Comparison?

The variant set is capped at 500 per side (1000 combined). Beyond the cap the badge is hidden rather than showing a misleading score from a biased sample. Narrow the filters on either side.

Where do rotated log files and scheduled backups go?

Rotated logs to `data/logs/` (only the newest ten archives are kept; **Clear** empties the live log only). Scheduled backups to `data/backups/`, named by timestamp and pruned to the retention count.

Why did no scheduled backup run last night?

The scheduler runs inside the administration server process — nothing is written while that server is down. A tick with no stored password records the failure "No backup password is set." The cron expression is matched against the wall clock of the display timezone set in **App Control** → **Timezone**.

Can an administrator recover my recovery codes or my two-factor secret?

No. The secret is stored encrypted and the recovery codes only as hashes. The recovery path is for an administrator to untick **2FA** for your account so that your password alone signs you in and you can enrol again.

24 Glossary

This chapter defines every term used elsewhere in the manual, in the application's own vocabulary — the words that appear on screen, in the database schema and in error messages. Entries are listed alphabetically and grouped by initial letter. Where a term belongs to a feature described in detail elsewhere, the entry names the chapter that covers it. Use this chapter as a reference while reading any other chapter; it is written for all audiences, and entries that apply only to a particular role say so.

Note Where a term is a literal on-screen control, a database object or an environment variable, it is printed exactly as the application spells it — including the American spelling of labels such as **Use pre-materialized transitions** and the table name `TRANSITIONS_RAW`. Everything else follows British spelling.

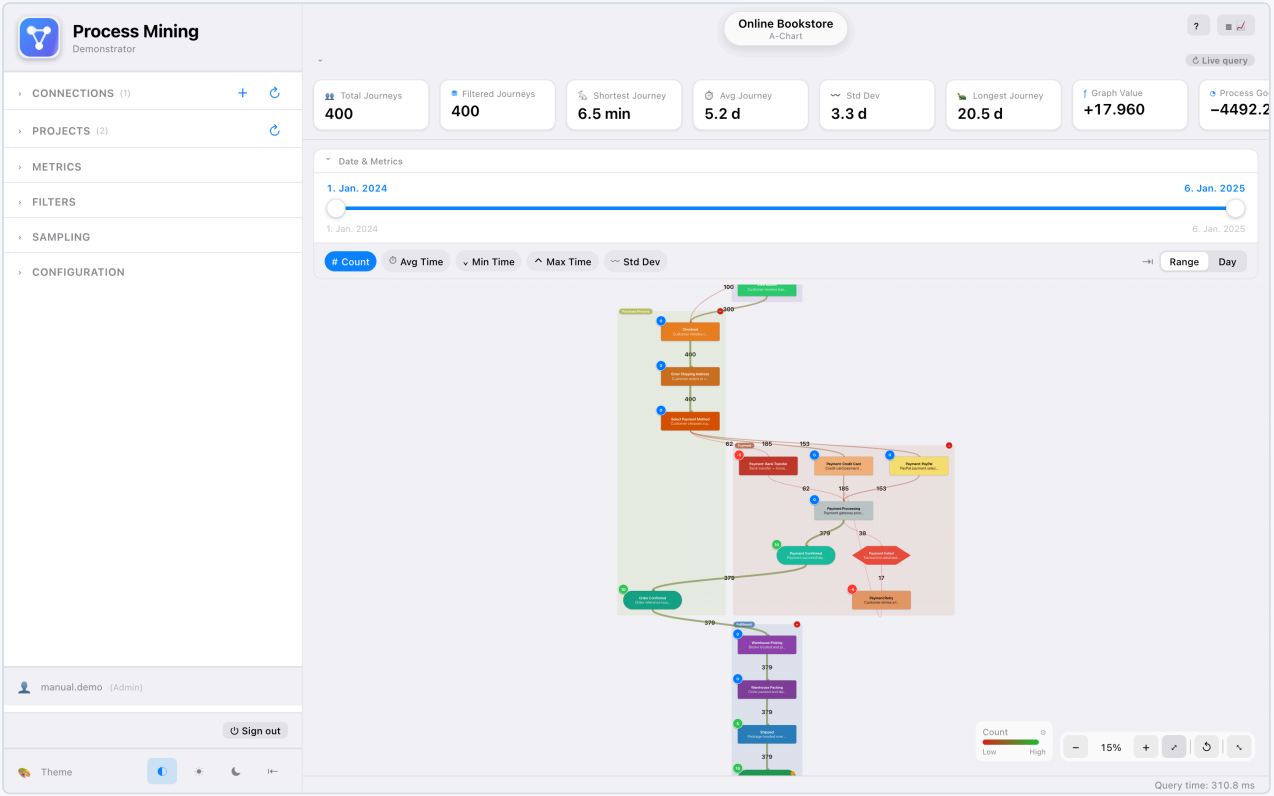


Figure 24.1 — The vocabulary of the process map: nodes are steps, arrows are transitions, the dashed box is a step group, and the tiles above are KPIs.

A

Abstraction layer

The part of the Integration console that sits between an extractor and the target database. Extractors never write SQL themselves; they push records through the layer, which creates tables, batches inserts, counts progress and manages transaction brackets. Its summary card heads the console's pipeline monitoring page.

A-Chart

The primary process map. It shows every journey matching the current filter set as one aggregated directly-follows graph, and loads automatically when you select a project, using the default date window (30 days unless changed). See the Chart Views chapter.

A/B Comparison

The split view that shows A-Chart on the left and B-Chart on the right at the same time, each with its own filters, presets and KPI strip. The sidebar edits whichever side is active. See also Valve, Process Similarity.

Administrator

The highest role. An administrator can do everything a Power user can, over every connection, and additionally has the admin interface for users, TLS, the directory, licensing, logging, backups and customisation. See the Users & Permissions chapter.

Admin interface

The separate administration surface on its own port — 8090 for HTTP, 8453 for HTTPS by default — with its own sign-in that admits administrators only. It keeps working even when the compute backend has stopped, so a licence can always be applied.

AI supported Documentation

The view that sends the current A-Chart transition table and your prompt template to the connection's LLM server and renders the answer as a Markdown report. Journey paths, happy-path conformance, the gap analysis and your notes are computed locally and appended without being sent to the model.

Alignment cost

The building block of Process Similarity: `missing_edges / total_edges_in_variant`. A cost of 0 means the graph contains every consecutive pair the variant needs; 0.5 means half are missing. Variants with a single step carry no edges and are skipped.

Anonymous mode

How the main application behaves when an administrator turns off **Require sign-in for the main application**. There is then no user identity, so all settings and presets fall back to one shared profile and every Power/Admin-only capability is unavailable — the relevant APIs answer 403.

Assignment

The grant that makes a connection visible and usable to a particular user. Assignments are checked on the server for every connect, import and watchdog poll — never trusted from a dropdown — and revoking one immediately disconnects that user's live session and stops any watchdog using the connection.

Avg Time

The transition metric that measures the average elapsed time between two steps, displayed as a readable duration such as 4m, 1.2h or 3.5d. Its default colour scale is orange.

Aux field

The internal role name for a helper field in a source type. See Helper field.

B

Backup

A single JSON file containing everything except the event data itself: connections and servers, filter presets, happy paths, target norms, saved node layouts, LLM prompt templates and preferences. The format is interchangeable with the macOS version of the application. See the Backup & Restore chapter.

B-Chart

An independent second process map with its own filter set, for exploring a different segment. Unlike A-Chart it starts empty: set filters and press **Apply**, or use **Load** on its empty state.

Break-glass account

A local administrator account, which always works regardless of directory configuration. The seeded `Administrator` account is the recovery route into the admin interface and is exempt from automatic lockout.

C

Case

A synonym for journey — one end-to-end instance of the process. The manual uses "journey" throughout because that is the application's own wording.

Certificate mode

The per-connection setting that governs how the database server's TLS certificate is checked: **Verify (system trust store)**, **Pin fingerprint** or **Accept any (insecure)**. Parsing fails closed — an unrecognised stored value is treated as Verify, never as no verification.

Checkpoint

The per-source record of how far a file has been read: byte offset, file size, signature, cumulative imported records, last update time and last error. There is exactly one per source and it is shared by manual delta runs and the watchdog, which is what stops the two importing the same line twice. It advances only after a successful run.

Colorise edges by weight

The Configuration option that tints each transition arrow using the colour scale of the active metric. Thickness always encodes the value independently of colour.

Compound step

A rule in a source type that derives the STEP value from several extracted fields — for example splitting one "login" activity into "login successful" and "login failed" using an HTTP status helper. Conditions are combined with AND, rules are evaluated in order and the first fully matching rule wins; if none match, the plain step value is used unchanged.

Compute backend

The third server process, alongside the main application and the admin interface. It runs the database queries and the integration API, and it is the component that stops when no valid licence is present.

Conformance Check

The advanced view that overlays target norms on a transition metric and colours each edge green or red according to whether it meets its norm. Available to Power users and administrators only.

Conformance score

The Happy Path measure: a journey-count-weighted average of edge coverage, from 1.00 (every journey follows the ideal path) to 0.00 (no journey shares a single transition with it). Each journey is scored against the route through the splits it matches best.

Connection

One Exasol database server plus an optional OpenAI-compatible LLM server, stored together with credentials, target schema and TLS options. Connections are created by administrators or Power users, assigned to individual users, and only one is active at a time.

Count

The default transition metric: how many times a transition occurred. It is always available and its default colour scale is green.

Coverage factor

The term `(filtered_journeys / total_journeys)^0.5` that scales raw Process Goodness, so a score reflects representativeness as well as quality. Filters capturing half the journeys reduce the score by roughly 29 per cent.

D

DATA

The log tag that groups every data-import entry — manual or watchdog, success or failure — in the admin Logging tab. Successful imports are recorded at USAGE level, failures at WARN or ERROR with the reason.

Default date window

The Configuration setting controlling how many days back the map shows when a project first loads. It defaults to 30; 0 loads the project's full range. It takes effect on the next project load and does not move the slider on the current map.

Delta upload

The import mode, on by default, that reads only the lines appended to a file since the source's checkpoint. Running the same source again therefore tops the project up rather than storing everything a second time. See also Checkpoint, Watchdog.

Demo content

A ready-made dataset generated into a connection's schema from the connection editor's Demo Content tab — Online Bookstore (retail), Online Credit Application (finance) and Flight Booking & Management (transportation). The default is 500 journeys and the server caps any request at 20,000.

Demo Mode

How the application runs with no valid licence: a one-time grace period, shown on the sign-in panel as "Demo Mode — remaining time", after which the compute backend stops. The period is granted once per installation and restarting does not renew it.

Developer

An additional grant, independent of the three roles, given by the **Developer** checkbox in the admin Users tab. It admits an account to the Integration console. Power users are deliberately excluded; only developers and administrators may enter.

Directly-follows graph

The graph formed by aggregating every chronologically consecutive pair of steps within the same journey. It is what the process map draws: one node per step, one arrow per observed pair.

Directory (LDAP)

Sign-in against an LDAP or Active Directory server using search + bind: a read-only service account finds the login name under the base DN, then the application re-binds as that user with the supplied password. A directory user is created locally as a plain, enabled account on first successful sign-in.

E

Encoding

The character encoding a file source is decoded with, `utf-8` by default; undecodable bytes are replaced rather than raising an error. The preview endpoint always decodes as UTF-8, whatever the source is configured with.

End of process

The `END_OF_PROCESS` flag on a step, shown on the map as an orange dot on the node. See also Exit marker.

Event

A synonym for step: one activity within a journey, stored as one row of JOURNEYS.

Event log

The JOURNEYS table — one row per event, and the only large table in the schema. Everything the application draws is derived from it.

EVENT_ID

The journey (case) identifier. Rows sharing an EVENT_ID form one journey. Demo datasets and integration imports store it MD5-hashed, so the raw identifier never lands in the database in the clear.

EVENT_TIME

The timestamp of an event. It orders the steps of a journey (with STEP_ID as tie-breaker) and drives every date filter.

Events skipped

The Integration console KPI counting source lines that could not be turned into an event — no case id, no step, or a missing or unparseable timestamp. A non-zero value is highlighted in orange and usually means the source type's regular expressions do not fit the log.

Exasol

The analytical database the application reads from and writes to. The free Community Edition and the official Docker image both work; for the Docker image and Exasol 7.x set the connection's minimum RSA key size to 1024 bits.

Excluded steps

In Simulation, steps removed from the Markov model entirely before the run, together with everything that becomes unreachable as a result. Not to be confused with the **Exclude Steps** filter, which removes whole journeys from the map.

Exit marker

The orange arrow drawn below every end node — a step with no outgoing transitions. Its counterpart is the green entry arrow above every start node. Both are drawn outside a step group's dashed border.

Extractor

A pluggable component that reads one kind of source and pushes parsed records through the abstraction layer. One ships today: the File extractor, which reads a File source line by line and writes JOURNEYS events using the linked source type's regular expressions.

F

File source

The only source kind available today: a file path plus an encoding, linked to a source type. File reads are sandboxed to the server's integration files directory unless an operator sets `PMW_INTEGRATION_ALLOW_ANY_PATH=1`.

Filtered Journeys

The KPI counting distinct journeys that match all current filters — the population the map visualises. Contrast Total Journeys.

Fingerprint

The SHA-256 hash of a database server's certificate, entered in the connection's **Fingerprint (SHA-256)** field when the certificate mode is **Pin fingerprint**. Colons are stripped and the value must be pure hexadecimal, 16 to 128 characters; a blank or non-hexadecimal value refuses the connection rather than silently skipping verification.

G

Gap analysis

The Conformance Check table opened with **Show gaps**, listing from, to, actual, norm, delta and status for every edge, violations first. It is also appended to the AI Documentation report.

Graph Value

The KPI summing step score × visit count over every scored node on the map, where visit count is the larger of a node's incoming and outgoing transition occurrences so that start and end nodes are counted correctly.

Grace period

See Demo Mode.

H

Happy Path

An ideal step sequence you define for a project, including splits for legitimate alternatives and their rejoin points, against which real journeys are scored. Available to Power users and administrators only.

Helper field

A field extracted from every source line for the sole purpose of being matched by compound rules — an HTTP status, a result code, a queue name. It is written to no database column, so all three META columns stay free for business attributes.

I

Importance

The level attached to a note: NORMAL (the default), INFO, IMPORTANT or URGENT, shown as a coloured badge. Only the note's author can change it.

Include Steps / Exclude Steps

Journey-level filters. Include keeps only journeys passing through any selected step; Exclude removes every journey passing through any selected step. A step chosen for one list is automatically disabled in the other.

Individual Journey

The view showing the complete step sequence of a single journey identified by EVENT_ID. Anything typed into the Event ID field that is not already a 32-character hash is MD5-hashed for you.

IngestSession

The only handle an extractor receives to the target database, bound to the connection's schema for the duration of one run. It offers `define_table`, `push`, `log`, `progress` and `existing_keys` — extractors never issue SQL themselves.

Integration console

The fourth surface, on the admin port plus 10 — 8100 for HTTP, 8463 for HTTPS by default — where data sources are defined and imports are run and monitored. It reuses the main application's sign-in and admits developers and administrators only.

J

Journey

One end-to-end instance of a process — all JOURNEYS rows sharing an EVENT_ID. Every filter in the application works at journey level: a journey either qualifies in full or not at all.

Journey sampling

Creating up to three named sample sets from a project's original data so that exploration of a very large event log stays interactive. Restricted to Power users and administrators because samples are shared by everyone on the connection. See Sample set.

Journey Score

The filter keeping journeys whose summed step scores fall within a range. Negative scores are supported, so the right handle can be dragged below zero to isolate problem paths.

Journey Time

The filter keeping journeys whose first-to-last span falls within a range.

JOURNEYS

The event-log table: PROJECT_ID, EVENT_ID, STEP, STEP_ID, EVENT_TIME, META_1 to META_3 and SAMPLE_SET. See the Data Model chapter.

K

KPI

One tile in the strip above the process map — Total Journeys, Filtered Journeys, Shortest/Avg/Longest Journey, Std Dev, Graph Value, Process Goodness and, in A/B mode, Process Similarity. Tiles are reordered by dragging — either on the strip itself or in the Configuration → KPIs list — and switched on or off from that list or by hovering a strip tile and clicking its × ; the two views share one order and visibility, stored per user. The strip appears only once a chart has loaded data.

L

Licence

The file uploaded in **App Control** → **License** that unlocks the application. The panel shows who it is licensed to and when it expires. Without one the application runs in Demo Mode.

Live query

The default way the process map is built: a window function pairs each event with its successor on every reload. Shown on the map as the pill **↻ Live query**. Contrast Pre-materialized transitions.

LLM

The optional OpenAI-compatible model server attached to a connection — base URL, model and optional API key — that enables the AI supported Documentation view. Its reachability is probed with a five-second request and shown as the second, blue status dot on the connection card.

Lockout

The automatic disabling of an account after N failed sign-in attempts, set in the admin Users tab. The default is 3 and 0 turns it off. The built-in Administrator is exempt and protected by a per-IP throttle instead.

M

MD5 hash

The one-way digest used for case identifiers. Demo datasets store the MD5 of a synthetic reference such as `ORD-000001` ; integration imports store the MD5 of the captured raw id unconditionally. The same raw id always yields the same digest, so journeys still correlate, but the original value cannot be recovered.

Meta field

One of the three free-text, case-level attribute columns META_1 to META_3 — department, customer segment, payment method and the like. Their business names come from the METAS table and appear as the Meta 1–3 filters, which perform a case-insensitive contains search.

METAS

The table holding one row per project with META_1_TITLE, META_2_TITLE and META_3_TITLE. Where a title is missing the interface shows generic meta labels.

Metric

See Transition metric.

Min Time / Max Time

The transition metrics reporting the shortest and longest observed elapsed time for a transition. Default colour scales blue and red respectively.

N

Node

One distinct process step drawn on the map. Its colour, shape, score badge and description come from the STEPS table; steps appearing in transitions with no STEPS row are drawn grey.

Norm

A target value set on a transition in Conformance Check, stored per project and per metric. For Count a norm is a percentage of all traffic leaving the same source node; for time metrics it is an absolute number of seconds. The **Norm is a minimum** toggle flips the comparison.

Note

An annotation anchored to a node or an edge, stored in the NOTES table in the project database. A note is a thread: anyone who can see it may add a comment, but only the author can change its importance or sharing or delete it. Each note records its author, its last contributor and the complete filter context at creation time.

Num Steps

The filter keeping journeys whose total step count falls within a range.

O

Operation

The field in every log entry naming the code path that wrote it — for example `login`, `connection`, `db-connect`, `db-timeout`, `materialize`, `import` or `llm-test`. Contrast Tag, which groups an entire activity across operations.

Optimise layout

The Configuration toggle enabling barycenter crossing-minimisation, which produces a cleaner automatic arrangement on complex graphs.

ORIGINAL

The SAMPLE_SET value marking the source data, as opposed to a sample. Wherever the application filters by sample set, a NULL SAMPLE_SET counts as ORIGINAL, so tables predating the column behave unchanged. ORIGINAL rows can never be deleted by deleting a sample.

P

Passkey

A WebAuthn credential — Touch ID, Windows Hello or a hardware security key — that signs a user in instead of a password. It is always an alternative, never a replacement: the password remains as fallback.

Passkeys require a secure context and a real hostname; they do not work over a bare IP address.

Path pattern

See Variant.

Pipeline

The Integration console's flowchart of imports — source type → source → abstraction layer → connection — with nodes reused across runs and a dot travelling the active path while an import runs. Its history is stored per user in the browser, so it is per device.

Power user

The middle role. A Power user does everything a Regular user does and additionally creates, manages and assigns their own database connections, provisions schemas, generates demo data, runs journey sampling, and uses the advanced views — Conformance Check, Happy Path and Simulation. They manage only the connections they created.

Pre-materialized transitions

The optional per-connection performance mode that reads the process map from the prebuilt `TRANSITIONS_RAW` table instead of pairing events live on every request. It is off by default and fails safe: any read failure falls back to the live query. Shown on the map as ⚡ **Pre-materialized**, or ⚠ **Live (not built)** when enabled but not yet usable.

Preset

A named snapshot of every filter dimension — date range, included and excluded steps, meta values, step-count, journey-time and score bounds. Presets are stored per project, persist between sessions, are included in backups, and are captured at the moment of saving: later filter changes do not update them.

Process Goodness

The composite quality KPI combining path quality, efficiency and coverage into one number that can be positive or negative. It is shown only when at least one step has a SCORE, and shows "—" if its query exceeds the 30-second timeout.

Process map

The interactive directly-follows graph at the centre of the application: nodes for steps, arrows for transitions whose thickness, opacity, colour and label reflect the selected transition metric relative to the highest value on the map.

Process Similarity

The A/B-only score $Q(T_1, T_2)$, between 0 and 1, measuring how behaviourally alike the two filtered processes are: 0.4 × variant alignment + 0.4 × scored-node agreement + 0.2 × joint coverage. Green from 0.70, blue from 0.30, red below. The badge is hidden when either side exceeds 500 variants.

Project

The top-level container for one process, identified by PROJECT_ID and listed in the PROJECTS table. Every journey, step definition, meta title, note, preset, norm, happy path and saved layout belongs to exactly one project.

PROJECTS

The table holding one row per project: PROJECT_ID, TITLE and DESCRIPTION. The project picker lists projects ordered by title.

Provisioning

Creating the schema and the process-mining tables (PROJECTS, JOURNEYS, STEPS, METAS, NOTES) with the **Create schema & tables** button. All provisioning DDL uses IF NOT EXISTS, so re-running never disturbs existing data. It requires a database account with CREATE SCHEMA and CREATE TABLE rights.

Q

Q score

See Process Similarity.

queryMs

The server-side wall-clock time of the database queries behind a chart load, shown in the chart footer as, for example, "123 ms" or "1.23 s".

R

Rebuild token

A per-connection bearer token letting a scheduler or ETL step trigger that connection's `TRANSITIONS_RAW` rebuild without an admin login. It is shown once, stored only as a hash, scoped to its own connection, and rate-limited to one rebuild per 60 seconds.

Recovery codes

The one-time codes issued when two-factor authentication is set up, each of which signs the user in once if the authenticator is unavailable. They are shown once, stored only as hashes, and can be regenerated — which invalidates the previous set.

Regular user

The baseline role, and the default for a new account. A Regular user views maps, charts and KPIs, applies filters and presets, reads and writes notes, and manages their own settings and layout on the connections assigned to them.

Required steps

In Simulation, a post-run filter keeping only journeys that visit every required step at least once. Because non-qualifying journeys are generated and then discarded, the reported total can be lower than the configured Journey Count.

Role

The permission level carried by a signed-in account: Regular, Power or Administrator, each building on the previous one. The Developer grant is separate and additional. Roles require a signed-in identity — see Anonymous mode.

Rotation

Replacement of a log file that keeps the same name. The watchdog detects it by the file shrinking below the stored offset, or by the MD5 signature of the first 64 bytes changing, and re-reads from the start. Files shorter than 64 bytes cannot be fingerprinted.

S

Sample set

A named subset of a project's journeys, stored as physical copies of their rows in JOURNEYS under a SAMPLE_SET value of SAMPLE_1, SAMPLE_2 or SAMPLE_3. Up to three exist per project; original rows are never modified, and deleting a sample removes only its own rows. Samples live in the database and are therefore shared by every user of that project.

Sampling strategy

How journeys are chosen for a sample: **Random** (uniform, fast and unbiased), **Temporal Stratified** (proportional across calendar-month buckets) or **Path Diversity** (coverage-maximising across distinct variants, so rare paths appear).

Sandbox

The directory under which file sources may read, `PMW_INTEGRATION_FILES_DIR`, defaulting to `<data dir>/integration_files`. Relative paths resolve inside it and any path escaping it — via `..` or a symlink — is rejected with "That path is outside the allowed sources directory."

Schema

The Exasol schema named on a connection, into which every table is read and written. It is optional for merely connecting but required for provisioning, demo generation, project listing and every import.

Score

The optional numeric value on a step in STEPS, shown as a badge on the node — green for positive, red for negative, blue for zero. Scores drive Journey Score, Graph Value, Process Goodness and the node-agreement term of Process Similarity. Steps without a score contribute zero.

Severity

The cumulative ladder of log levels — INFO, USAGE, WARN, ERROR, DEBUG. Choosing a maximum level always keeps the cheaper levels; DEBUG is recorded only when explicitly selected.

Shape

The outline of a node, taken from STEPS: stadium, round, hex or circle. Steps created automatically by an import get a shape and colour chosen deterministically from the step name, so the same name always looks the same.

Signature

The MD5 of a file's first 64 bytes, stored in the checkpoint so that an appended file is recognised as the same file while a replaced one is not. See Rotation.

Sim-A / Sim-B

The two slots in which a simulation run can be stored so that it can be selected as an A or B data source in the Sampling section, and thus compared in A/B Comparison against real data or against the other simulation.

Simulation

The advanced view that builds a first-order Markov chain from the graph currently loaded in Chart A and walks it to generate synthetic journeys, with lognormal edge durations and Poisson arrivals. Available to Power users and administrators only.

Source

In the Integration console, a concrete thing to import — today a File: a path, an encoding, a linked source type, a transaction bracket and an optional watchdog block.

Source type

A reusable recipe for parsing one log format: an example line plus the regular expressions that extract the timestamp, case id, step, up to three meta fields and any helper fields, together with any compound-step rules. Every field is defined by hand, by design.

SSRF guard

The check applied to a connection's LLM URL before it is contacted: only http and https are allowed, every resolved address is vetted, the outbound connection is pinned to the vetted address and redirects are disabled. Loopback and private addresses are permitted by default so that local model servers work.

Statistics

The view analysing all journey routes matching Chart A's filters, with a **Routes** table of distinct variants and an **Analytics** set of visualisations — duration histogram, step traffic and transition heat map.

Std Dev

Both a transition metric (the standard deviation of a transition's elapsed times, indicating inconsistent durations) and a KPI (the standard deviation of whole-journey durations).

Step

A single activity within a journey — one row of JOURNEYS — and, in aggregate, one node on the map. Also called an event.

Step editor

The Configuration → Steps panel where a step's background and foreground colour, shape, score, group and description are set. Changes are written straight to the STEPS table and the map redraws.

Step group

The dashed, coloured box with a name pill wrapping every step sharing a BELONGS_TO value. A group can be collapsed, in which case connection counts are summed and times weight-averaged, and moved as a whole by dragging its box.

STEPS

The table holding per-step presentation and scoring: DESCRIPTION, BG_COLOR, FG_COLOR, SCORE, SHAPE, END_OF_PROCESS and BELONGS_TO, keyed by PROJECT_ID and STEP.

T

Tag

The log field that groups an entire activity across operations and severities. Three exist: **SQL** (every entry quoting an executed statement), **BACKUP/RESTORE** (the whole custody trail of a backup file) and **DATA** (data imports).

TLS mode

The application-wide setting **Off**, **Optional** or **Required**, governing the main application (8080/8443), the admin interface (8090/8453) and the Integration console (8100/8463) alike, all sharing one active certifi-

cate. If a mode needs a certificate and none is active, each server falls back to HTTP so nothing becomes unreachable. Not to be confused with a connection's certificate mode, which concerns the database link.

Total Journeys

The KPI counting distinct journeys in the project with no filters applied. It does not change when filters do. Contrast Filtered Journeys.

Transaction bracket

The number of rows an import commits together, set per source. The default is 5000; other values are clamped to between 1000 and 1,000,000; 0 means one single transaction for the whole import. On failure the open bracket is rolled back while already-committed brackets remain.

Transition

A chronologically consecutive pair of steps within the same journey, drawn as one arrow on the map. Its metric values — count and the four time statistics — are aggregated over every occurrence in the filtered population.

Transition metric

The value driving each arrow's thickness, opacity, colour and label: Count, Avg Time, Min Time, Max Time or Std Dev. Each chart view remembers its own selection and each metric has its own colour scale.

TRANSITIONS_RAW

The optional table storing one row per consecutive step pair — PROJECT_ID, EVENT_ID, FROM_STEP, TO_STEP, FROM_TIME, TO_TIME, DUR_SECS, SAMPLE_SET — so the map query becomes a plain aggregate. It goes stale as soon as JOURNEYS changes and must be rebuilt after each load. See Pre-materialized transitions.

Trigger

What started an import: `manual` (a ▷ Run in the console) or `watchdog` (a background poll). The console splits its import KPIs along this distinction.

Two-factor authentication

A time-based one-time code from an authenticator application, required after the password once an administrator enables it for an account. Enabling it makes enrolment mandatory at the next sign-in. A passkey sign-in skips the code step.

V

Valve

The ⇄ control on the divider in A/B Comparison. Open, both panels share one viewport and every pan, zoom and node drag mirrors instantly; closed, they move independently — and closing first snapshots A's view into B. A is always the master.

Variant

A distinct journey path — one ordered sequence of steps, however many journeys followed it. Variants are what the Statistics Routes table lists, what Process Similarity aligns, and what Process Goodness weights by relative frequency.

View

One of the ten screens reached from the ≡ menu at the top right: A-Chart, B-Chart, A/B Comparison, Individual Journey, AI supported Documentation, Statistics, Conformance Check, Happy Path, Notes and Simulation. Each remembers its own filters and metric, and the last three named are hidden from Regular users.

W

Watchdog

The background job that polls a file source at its configured interval and imports only newly appended lines, so a continuously written log streams into the database on its own. It runs headless, acts as the source's owner, uses the connection's stored credentials, and re-checks the assignment on every poll — revoking it stops the watchdog.

WebAuthn

The standard behind passkeys. See Passkey.

X

X-PMW-User

The trusted header carrying the signed-in identity from the interface proxy to the compute backend after session validation. The backend refuses direct calls that do not carry the proxy-authentication secret, which is what makes the role checks impossible to bypass.

Tip The in-application help panel, opened with ⌘/ or **Ctrl+/**, carries a shorter version of many of these definitions in context. Use it while working and this chapter when you need the precise wording, the units or the limits.

Warning Several pairs of terms look alike and mean different things: **Exclude Steps** (a journey filter) versus **Excluded steps** (a Simulation model change); **certificate mode** (the database link) versus **TLS mode** (the application's own ports); **sample set** (a stored subset of journeys) versus **delta upload** (an import that adds to what is already stored); and the Conformance Check **norm** versus the Happy Path **conformance score**. Check the entry before assuming which one a colleague means.

PART VI

Actions

25 Actions

> An *action* is a small, business-readable query you attach to process-map nodes. From a node's menu you can show the last log entries from that step, from the steps around it, or a transition table for everything that follows — and the result always honours whatever filters you currently have set on the chart. Actions are written in a tiny script language that the system translates into database SQL for you. This chapter explains how the feature is turned on, how to author actions on the Actions surface, the script language and its clauses, how availability decides where an action appears, and how an action runs from the app.

Requires: **Administrator** to enable the feature; **Developer** (or Administrator) to author actions; and any signed-in user *except a plain standard user* (that is, Power, Developer or Administrator) to run them from the app. The feature is **off** until an administrator enables it.

25.1 Enabling the feature

Actions are opt-in. In the admin interface open the **Actions** tab and tick **Enable Actions**, then restart the app server from **App Control** so the surface binds its listeners. While the feature is off, the Actions surface serves a "disabled" page and the app shows no **Actions** entry in node menus.

The Actions authoring surface runs on its own port — by default HTTP `:8110` and HTTPS `:8473` — and follows the same TLS mode and certificate as the app and admin interface. Only developers and administrators may sign in to it.

25.2 Authoring actions

Open the Actions surface, sign in, then pick a **connection** and a **project** at the top. Actions are saved per connection *and* project, so a service provider hosting several customers keeps each customer's actions separate. The builder then shows the saved actions for that project on the left and an editor on the right.

As you type, the editor validates the script live, restates it in plain English, and shows the exact SQL it will run. The **Test** panel runs the script against a node you choose (standing in for `THIS`) so you can see real rows before saving. **Save** stores the action; **Delete** removes it.

25.3 The script language

An action is a handful of clauses. Keywords are case-insensitive, and `NODE` / `NODES` and `ENTRY` / `ENTRIES` are interchangeable. A clause keyword may sit on its own line with its value on the following line(s), or share a line with its value.

Clause	Meaning
`AVAILABILITY`	Where the action appears in node menus: `ALL NODES`, or a comma-separated list of step names (e.g. `LOGIN, LOGOUT, PAYMENT`).
`SHOW`	What to display: `LAST [N] LOG ENTRIES` (N defaults to 1); `TRANSITION TABLE WITH …` listing one or more metrics (see below); or `FLOWCHART IN SEPARATE PANEL` to open another project's process map — optionally `FOR` one or more metrics to make the panel's edge metric switchable.
`FROM`	For log entries and transition tables: `NODE(…)` relative to the clicked node — `THIS`, `PREVIOUS`, `FOLLOWING`, `ALL FOLLOWING`, `ALL PREVIOUS` (combine them, e.g. `NODE(THIS, ALL FOLLOWING)`). For a flowchart: a target `<connection>::<project>`.
`SORT`	Optional. `ASCENDING` or `DESCENDING` (log entries by time; the default is most recent first).
`WHERE`	Optional filter. This version supports `EVENT_ID :: [id, id, …]` (the `::` reads as "is one of").

A minimal example — show the single most recent event of the clicked node:

```
AVAILABILITY
    PAYMENT
SHOW
    LAST 1 LOG ENTRY
FROM
    NODE(THIS)
SORT
    DESCENDING
```

A transition table can list any of the chart's metrics, in any combination, as quoted names after `WITH`: `"COUNT"`, `"%JOURNEY%"` (share of all filtered journeys traversing the edge), `"%OUTGOING%"` (branching share of the source node's outgoing journeys), and the transition-time metrics `"AVG TIME"`, `"MIN TIME"`, `"MAX TIME"` and `"STD DEV"` (in seconds). Friendly short forms are accepted too — e.g. `AVG`, `MIN`, `MAX`, `STDDEV`, `PERCENTAGE`, `JOURNEY %`. Each metric becomes a column, in the order you list them. As a shorthand, `TRANSITION TABLE WITH ALL METRICS` includes every metric at once.

A transition table for the clicked node and everything downstream, over the most recent activity:

```
AVAILABILITY
    ALL NODES
SHOW
    TRANSITION TABLE WITH "COUNT", "%JOURNEY%"
    FOR LAST 50 LOG ENTRIES
FROM
    NODE(THIS, ALL FOLLOWING)
```

A **flowchart** action opens another project's full process map in a panel over the current one, scoped to the chart's current date range (all other filters are dropped, since the target project has its own steps). The `FROM` names the target as `<connection>::<project>` — the connection and project names exactly as they appear to you:

```
AVAILABILITY
    PAYMENT
SHOW
    FLOWCHART IN SEPARATE PANEL
FROM
    01 - Exasol Nano @ Macbook Pro::Airport Passenger Flow Analysis
```

The target must be a connection **assigned to you**; if it is the one you are already on the map loads from your live session, otherwise the server opens it briefly with its stored credentials. The panel is a fully interactive process map (zoom, pan, hover) — a read-only cross-reference into a related process.

Add `FOR` plus one or more comma-separated metrics to let the panel switch the edge metric — a drop-down appears above the map. Use any of `Count`, `%JOURNEY%`, `%OUTGOING%`, `AVG TIME`, `MIN TIME`, `MAX TIME`, `STD DEV` (short forms work too), or `ALL METRICS` for every metric. The `FOR` clause may come before or after `IN SEPARATE PANEL`:

```
AVAILABILITY
    PAYMENT
SHOW
    FLOWCHART IN SEPARATE PANEL FOR Count, Avg Time
FROM
    01 - Exasol Nano @ Macbook Pro::Airport Passenger Flow Analysis
```

25.4 Availability — where an action appears

The `AVAILABILITY` clause decides which nodes list the action. `ALL NODES` makes it appear on every node; a list of step names limits it to those steps. When you right-click a node in the process map, only the actions whose availability matches that node show up in the **Actions** section of its menu.

25.5 Running an action

In the process map, click a node and choose the action from the **Actions** section of its menu. The system resolves the `FROM` selectors against the live graph (so `PREVIOUS`, `FOLLOWING` and their transitive `ALL` forms become concrete steps), runs the query, and shows the result — a log-entry table or a transition table — over the map.

> **Actions always respect the chart's current filters** — the date range, included and excluded steps, the meta filters, the step-count / journey-time / score ranges and the sample set. The same action therefore answers a different question as you narrow the view, which is the point: it reads the situation you are looking at rather than the whole database.

25.6 Limits of this version

The `WHERE` clause supports an `EVENT_ID` list only; richer conditions are not yet available. On a transition table, a `FOR LAST N LOG ENTRIES` qualifier is applied as an approximate recency cap rather than an exact windowing. Log-entry results are capped to a safe maximum number of rows.

PART VII

The API Server - Event Receiver

26 The API Server - Event Receiver

> Every other way of getting data into the demonstrator *pulls* it: you import a file, or the watchdog reads one as it grows. The *API Server - Event Receiver* is the opposite — it is a small HTTP/HTTPS endpoint the demonstrator opens so that an external program, typically an AI agent, can *push* journey events into a project as they happen. Each sink is bound to one connection and one project, is protected by its own bearer token, auto-creates any step it has never seen, and can be watched live on a node-based monitor. This chapter explains when to reach for a sink, how an administrator turns the module on, how a developer defines a sink and reads its token, the exact JSON the endpoint accepts, the limits and error codes it enforces, the downloadable agent instructions, and the live monitor.

Requires: **Administrator** to enable the module; **Developer** (or Administrator) to define a sink and see its token, in the integration console (HTTP :8100, HTTPS :8463). The running endpoints need no human role — they authenticate the calling program with a per-sink bearer token. The module is **off** until an administrator enables it.

26.1 What a sink is, and when to use one

A sink is a long-running API server that accepts journey rows over HTTP and writes them straight into a connection's `JOURNEYS` table, exactly as an import would — the same MD5 pseudonymisation of the case id, the same automatic creation of missing `PROJECTS` and `STEPS` rows, the same `ORIGINAL` sample set. What differs is the trigger and the shape of the data: instead of a file parsed in one run, a sink takes many small JSON POSTs over time, each carrying one event or a short batch of events, from a program you do not control from inside the demonstrator.

Reach for a sink when the source of truth is a *live process* rather than a log file — most obviously an AI agent that emits a step every time it takes an action, but equally any application that can make an authenticated HTTPS request. Use a file source and the watchdog (chapters 19 and 20) instead when the data already lands in a file, or when you want the parsing, compound-step rules and delta-checkpointing that the extractor gives you. A sink does no parsing: the caller sends already-structured rows.

	File import / watchdog	API Server - Event Receiver
Direction	The demonstrator **pulls** from a file.	The caller **pushes** over HTTP.
Shape of data	Raw log lines, parsed by a source type.	Structured JSON rows; no parsing.
Destination	Any project id, chosen per run.	One fixed project per sink (its project code).
Who may write	A signed-in developer, or the headless watchdog with stored credentials.	Any program holding the sink's bearer token.
Unknown steps	Created automatically.	Created automatically.

26.2 Enabling the module

The sink module is opt-in, like Actions. In the admin interface open the **Event Receiver** tab and tick **Enable the API Server - Event Receiver**. Toggling the switch takes effect immediately and needs no restart: the sink ports stay bound either way, but while the module is off every request is answered with `503` and nothing is written. The tab also shows how many sinks are configured and whether the sink supervisor process is running.

Each sink listens on a fixed, pre-exposed port from a pool, so the container can publish them once. The pool is HTTP `8120–8129` and, under TLS, the paired HTTPS ports `8483–8492`. In the shipped Docker setup these are published on the host with a `+10000` offset — HTTP `18120–18129` and HTTPS `18483–18492` — so a caller outside the container uses the host port. The scheme and port that are actually live follow the deployment's TLS mode and certificate, exactly like the other surfaces (see chapter 2): with a certificate in **Optional** or **Required** mode a sink answers on its HTTPS port; otherwise on plain HTTP.

> **Note** The module runs as its own supervisor process, one listener per sink. Adding, editing or removing a sink, or a change to the TLS mode or certificate, makes the supervisor rebind its listeners automatically — no terminal, no restart. Enabling or disabling the module does not even need a rebind.

26.3 Defining a sink

A sink is a data source of a new kind, created in the integration console's **Sources** section like any other.

1. In the console sidebar open **Sources** and press **+** to add a source. For **Source type** choose **API Server - Event Receiver**.
2. Give the sink a **name** (how it appears in the console and the monitor).
3. Choose the **Connection** it writes into. The dropdown lists only the connections assigned to you; the connection must have a schema. This is where the events land, using the connection's stored credentials.
4. Enter a **project code** of 1–10 characters (the project's `TITLE_SHORT`). All of this sink's events go into the one project with that code; it is created on the first write if it does not exist, and reused if it does.
5. Pick a **port** from the free ports in the pool. Each option shows *both* its ports — e.g. `HTTP 8120 · HTTPS 8483` — so you can see what an agent will address. Ports already taken by other sinks are not offered; a sink cannot share a port.
6. Set the **TLS** preference: ticked (the default) means agents are told to use the sink's **HTTPS** port, unticked means plain **HTTP**. This is a per-sink preference for the endpoint shown in the request and the SKILL.md; the listeners still bind whatever the deployment TLS mode allows, and the shown scheme falls back to what the deployment can actually serve, so it never advertises a port that isn't bound.
7. Save. The confirmation screen shows the sink's **bearer token once** (see 26.4) and a link to the exact request an agent should send (see 26.6).

One connection can back several sinks (each with a different project code and port), and different developers can each own sinks; you see and manage only your own. A sink whose connection is later unassigned

from you, whose port falls outside the pool, or that has no project code, is quietly skipped by the supervisor rather than started.

26.4 The bearer token

Each sink has one secret bearer token that the caller must send on every request. Only a **SHA-256 hash** of the token is ever stored; the plaintext is shown **once**, on the confirmation screen right after you create the sink, and cannot be retrieved afterwards. Copy it then and store it wherever the agent keeps its secrets.

If a token is lost or leaked, open the sink in the **Sources** list and use the 🔑 **regenerate** action. Because regenerating invalidates the live token the instant it runs — locking out every agent still using the old one — it asks for confirmation first. A fresh token is then minted and shown once; the previous token stops working as soon as the supervisor rebinds (which the console triggers for you). There is deliberately no way to read an existing token back — regeneration is the only recovery.

> **Handle the token like a password** Anyone who holds it can write events into the sink's project for as long as the module is enabled. Send it only over the HTTPS endpoint in any real deployment, never commit it to a repository, and regenerate it if in doubt.

26.5 The ingest API

A sink exposes two endpoints on its port:

`POST /ingest`
: Bearer-authenticated. The body is a single JSON object, or an array of them, each describing one journey event. The events are written to the sink's connection and project; any step seen for the first time is added to `STEPS` with a freshly allocated activity id, a deterministic colour and shape. Returns `{ingested, newSteps, projectId}`.

`GET /health`
: Unauthenticated liveness. Returns `{"ok": true, "enabled": <bool>}` and nothing else — deliberately no sink name or configuration, since anyone who can reach the port sees it.

Authenticate by sending the token as a bearer credential: `Authorization: Bearer <token>`. Each entry recognises the following fields; only `eventId` and `step` are required.

Field	Meaning
`eventId` **(required)**	The journey / request id — reuse it across every event of one run. It is MD5-pseudonymised on write, so the raw value never reaches the database. Also accepted as `event_id`.
`step` **(required)**	A `KIND:qualifier` naming the request or action (see below) — the whole string becomes one node on the map. An unknown step is created on the fly.
`description`	The step's detail (the query run, URL fetched, recipient …), up to **256 characters**; longer text is truncated. Stored as the event's **Action** attribute (`META_1`). Also accepted as `meta1`.
`client`	The calling client — `Claude`, `ChatGPT`, …. Stored as the **Client** attribute (`META_2`). Also accepted as `meta2`.
`user`	The end user, if known. Stored as the **User** attribute (`META_3`). Also accepted as `meta3`.
`eventTime`	ISO-8601 timestamp. Without an offset it is stored as given (local wall-clock time); with an offset or a trailing `Z` it is converted to the display timezone set in the admin interface. Missing or blank means "now" in that timezone. Also accepted as `event_time`.

The sink writes a per-project `METAS` row titling the three attribute columns **Action**, **Client** and **User**, so the app labels them in its Metas table and case-level filters — you can filter a project's journeys by client or user.

26.5.1 Naming steps: `KIND:qualifier`

Keep the `step` KIND from a small, stable vocabulary and add the specific target after a colon, so each node is self-describing. The first event of a journey names the originating request; each later event names one action taken:

- `SKILL:<skill name>` — e.g. `SKILL:summarize-sales`
- `TOOL:<type>:<tool name>` — e.g. `TOOL:mcp:airfield_get`
- `DATABASE:<database name>` — e.g. `DATABASE:sales_db`
- `WEB:<external|internal>`, `EMAIL:<external|internal>`, `FILE:<local|remote>`
- `APP:<app name>` — e.g. `APP:Slack`
- `REQUEST` — the first event, when the request is not itself a skill or tool call

All the events in one sink land in the one project the sink is coded for; you do not send a project id, and you cannot write into another project through the same sink.

26.6 Example request, and the SKILL.md for agents

A minimal call, against a TLS deployment where the sink's HTTPS port is published on the host as `18483`:

```
curl -sk https://YOUR-HOST:18483/ingest \
  -H "Authorization: Bearer YOUR-SINK-TOKEN" \
  -H "Content-Type: application/json" \
  -d '[
    {"eventId":"req-42","step":"SKILL:summarize-sales","description":"Summarize the Q3 sales
report","client":"Claude","user":"alice","eventTime":"2026-09-14T10:00:00Z"},
    {"eventId":"req-42","step":"DATABASE:sales_db","description":"SELECT region, SUM(amount)
FROM sales WHERE quarter='Q3'","client":"Claude","user":"alice","eventTime":"2026-09-
14T10:00:03Z"},
    {"eventId":"req-42","step":"EMAIL:external","description":"Send the compiled report to
alice@example.com","client":"Claude","user":"alice","eventTime":"2026-09-14T10:00:09Z"}
  ]'
```

You do not have to work the URL out by hand. The confirmation screen (and the 🔌 **details** action on a sink in the **Sources** list) opens the ingest-details popup: the exact endpoint URL and a tabbed **“Send data to the endpoint”** panel of ready-to-run examples, one per language, with the sink's URL and token already filled in. Every tab carries a shared **Copy** and ⬇ **Download** button that acts on the tab you are viewing and names the file for that language.

Two fields in that popup make the download ready to use as-is:

- The **endpoint URL is editable and saved per sink**. The console auto-detects the host URL, but if agents reach the sink through a reverse proxy or a public domain, type that URL and press **Save** — it is then shown here and baked into every example on every tab.
- A **token field** is pre-filled the moment you create or regenerate the sink; otherwise paste the token you saved. Whatever is in the field is embedded into every example. The token itself is never stored (only its hash is kept), so re-opening the popup in a later session needs a paste — the app says so.

26.6.1 Ready-to-run code in your language

The panel offers the same call in nine forms, sorted alphabetically along the tabs: **AI Agents (SKILL.md)**, **C#**, **CLI (curl)**, **Go**, **JavaScript**, **Mojo**, **Python**, **Rust** and **TypeScript**. Pick a tab, then **Copy** or **Download** it — the downloaded file is named for the language (`send_events.py` , `send_events.sh` , `SendEvents.cs` , `send_events.rs` , `send_events.go` , `send_events.mjs` , `send_events.ts` , `send_events.mojo`) or `SKILL.md` for the agent tab. Each example already contains the endpoint URL and the bearer token from the fields above, the two headers, and the same journey body; when the endpoint is HTTPS (the certificate is self-signed by default) each also disables certificate verification the way that language expects — `curl -k` , Python `verify=False` , C#’s `ServerCertificateCustomValidationCallback` , Rust’s `danger_accept_invalid_certs` , Go’s `InsecureSkipVerify` — so it runs against a fresh deployment without edits.

The **Python** tab, for example, uses `requests` :

```
import requests

URL = "https://YOUR-HOST:18483/ingest"
TOKEN = "YOUR-SINK-TOKEN"

events = [
    {"eventId": "req-42", "step": "SKILL:summarize-sales",
     "description": "Summarize the Q3 sales report and email it to the requester",
     "client": "Claude", "user": "alice", "eventTime": "2026-09-13T10:00:00"},
    # … one object per action, all sharing the eventId …
]

resp = requests.post(
    URL,
    headers={"Authorization": f"Bearer {TOKEN}",
             "Content-Type": "application/json"},
    json=events,
    verify=False,  # self-signed cert — point at your CA bundle in production
)
resp.raise_for_status()
print(resp.json())
```

The other tabs are equally complete: **Go** uses `net/http`, **Rust** uses `reqwest` + `tokio`, **C#** is a .NET6+ top-level program using `HttpClient`, **JavaScript** and **TypeScript** use Node18+'s built-in `fetch` (the TypeScript tab adds an `interface` for the event shape), and **Mojo** — which has no native HTTP client yet — drives Python's `requests` through Mojo↔Python interop.

26.6.2 An example SKILL.md

The downloaded file is plain Markdown with a YAML frontmatter header (so an agent runtime can register it as a skill) and the sink's real URL and token filled in. Abbreviated, it reads:

```
---
name: AI-Agent-Logger
description: A Skill for logging AI actions into the Process Mining Demonstrator
---

# Skill: Emit process-mining journey events to "Agent Sink"

## How to model a journey
Give every event of one run the same `eventId`. Each event's `step` is a
KIND, qualified after a colon so the map is specific:
- `SKILL:<name>`, `TOOL:<type>:<name>`, `DATABASE:<db>`,
  `WEB:<external|internal>`, `EMAIL:…`, `FILE:…`, `APP:<name>`, `REQUEST`
Put the detail in `description` (≤256 chars → the "Action" attribute);
`client` (Claude/ChatGPT/…) and `user` record who ran it.

## Endpoint
- POST https://YOUR-HOST:18483/ingest
- Authorization: Bearer <YOUR-SINK-TOKEN>
- Content-Type: application/json
- Target project: AGENTLOG

## Request body  (one object, or an array of them)
| field | required | notes |
|-------|----------|-------|
| eventId | yes | the journey id; reuse across a run |
| step | yes | KIND:qualifier — becomes a node |
| description | rec. | the step's detail, ≤256 chars → META_1 (Action) |
| client | rec. | the calling client → META_2 (Client) |
| user | opt. | the end user → META_3 (User) |
| eventTime | no | ISO-8601; defaults to now |

## Example
curl -sk https://YOUR-HOST:18483/ingest \
  -H "Authorization: Bearer <YOUR-SINK-TOKEN>" \
  -H "Content-Type: application/json" \
  -d '[{"eventId":"req-42","step":"SKILL:summarize-sales",
        "description":"Summarize the Q3 sales report",
        "client":"Claude","user":"alice","eventTime":"2026-09-14T10:00:00Z"},
       {"eventId":"req-42","step":"DATABASE:sales_db",
        "description":"SELECT region, SUM(amount) FROM sales WHERE quarter=Q3",
        "client":"Claude","user":"alice","eventTime":"2026-09-14T10:00:03Z"},
       {"eventId":"req-42","step":"EMAIL:external",
        "description":"Send the compiled report to alice@example.com",
        "client":"Claude","user":"alice","eventTime":"2026-09-14T10:00:09Z"}]'

## Responses
```

```
200 {ingested, newSteps, projectId} · 400 bad payload · 401 bad token
· 413 too large · 503 module off · 502 destination DB unavailable
```

Hand that file to the agent and it can log a whole run — a request followed by each action it takes — with no further setup on your side.

> **Tip** Send a short batch (an array) rather than one request per event when your agent produces events in bursts. Each event still becomes one journey row; batching just cuts the number of round trips and transactions.

26.7 Limits and error codes

A sink is the one surface that takes untrusted input from the network, so it is deliberately strict.

Status	When
`200`	Written. The body reports how many events were ingested and which step names were newly created.
`400`	The body is not JSON, or an entry is missing a non-empty `eventId` or `step`, or an `eventTime` is not valid ISO-8601. Deterministic — do not retry unchanged.
`401`	Missing or wrong bearer token.
`413`	The request body exceeds the size ceiling (**2 MB** by default), or carries more than the maximum entries per request (**5000** by default). Both are refused before any parsing or database work. Split the payload and resend.
`503`	The module is disabled in the admin interface.
`502`	The destination database was temporarily unavailable. The message is intentionally generic — the underlying driver detail (which names internal hosts, the database user and the schema) is written to the server log, never returned to the caller. Safe to retry after a short pause.

A sink reuses one database connection across posts and refreshes it if it has sat idle, reconnecting and retrying once if a write meets a dropped socket — so a transient network drop surfaces as a brief `502` at worst, not a permanent failure. The two limits are environment-tunable (`PMW_SINK_MAX_BODY_BYTES`, `PMW_SINK_MAX_ENTRIES`) for operators who need larger batches.

26.8 The live monitor

The integration console's main pane carries a node-based monitor for sinks, beneath the file-ingestion pipeline and shown once you have defined at least one sink. It is the sink-side sibling of that pipeline: one horizontal lane per sink, reading left to right

[🤖 AI agents] → [🔌 sink : port] → [📊 project] → [🛡 connection]

and refreshed every few seconds. Each lane shows, at a glance:

- a **liveness dot** on the sink node — green when the listener answered its health check, red when it did not respond, grey when the module is switched off;
- the sink's live **scheme and port**;

- the **event and journey counts** for its project, read from the destination database, and **when the last event landed**;
- the destination **connection and schema**.

When a sink's event count grows between two refreshes, its lane lights up with a travelling dot — the “data is arriving” signal, since sinks keep no throughput counter of their own. A sink whose connection cannot be read shows a short problem note on its lane rather than failing the whole view. Because the counts come from the database rather than from the sink process, the monitor reflects everything in the project, however it was written, and the per-connection reads are cached for a few seconds so several open consoles cannot pressure the database.

> **Note** The monitor reads only what the backend can see without contacting the sink processes: stored configuration, whether the module is on and the supervisor alive, a localhost health probe per listener, and the destination-database counts. It does not show per-request throughput or an error history — watch the `502` rate in the server log for that.

PART VIII

The MCP Server

27 The MCP Server

The previous chapter described how an external program pushes data *into* the demonstrator. The *MCP Server* is the mirror image: an endpoint through which an AI client — Claude, ChatGPT or any other client that speaks the Model Context Protocol — asks the demonstrator questions and gets analysis back. It exposes twenty-one tools covering connections, projects, metadata and attribute values, the process map, transition metrics, variants, statistics, journey search and lookup, notes, dashboard notifications, and — for power users — segment comparison, bottlenecks, trends, outcome drivers and conformance rules; most share the application's filter vocabulary. Its only writes are notes and toggling a notification rule: there is no ingest, no event editing, no sampling. This chapter explains what the server is for, how an administrator turns it on, how the identity chain from an OAuth token to a Process Mining user works, how to configure Authentik and the server itself, what each tool returns, what to watch for behind a reverse proxy, and how to read its error codes.

Requires: **Administrator** to configure the OAuth settings and enable the module, in the admin interface (HTTP :8090, HTTPS :8453). Callers need no human role at the endpoint — they present an OAuth access token issued by your Authentik server, which is mapped to an existing Process Mining user. The module is **off** until an administrator enables it.

OAuth only, directory-backed The MCP server authenticates callers **exclusively via OAuth** — it has no login of its own. You must deploy an OAuth provider such as **Authentik** or **Keycloak** and configure it in the Administration page (**MCP Server**), with user federation from **OpenLDAP** or **Active Directory**. Only Process Mining users defined in a configured directory service can use the MCP server.

27.1 What the MCP server is, and when to use one

The Model Context Protocol is a convention by which an AI client discovers and calls tools on a remote server. The demonstrator implements it as a seventh surface so that a question asked in a chat — *“which variants dominate the credit application process?”*, *“how much slower are international passengers?”* — can be answered from live data rather than from a screenshot pasted into the conversation.

It answers the same questions the work bench answers, through the same analytical code, with the same filters. What it does not do is replace the work bench: there is no map to look at, no drill-down, no A/B view. Reach for the MCP server when the analysis needs to reach a person or a system *outside* the application — a written summary, a comparison across projects, a figure quoted in a report — and use the application itself when you want to see the process.

	API Server - Event Receiver	MCP Server
Direction	The caller **writes** journey events.	The caller **reads** analysis (and may write notes).
Authentication	A per-sink bearer token the demonstrator issues.	An OAuth access token your Authentik issues.
Scope of access	One connection and one project, fixed per sink.	Every connection assigned to the mapped user.
Ports	A pool: `8120–8129` / `8483–8492`.	A single pair: `8130` / `8493`.
Can it change data?	Yes — that is its purpose.	Only notes (`create_note`, `update_note`) and enabling/disabling a notification rule (`set_notification_enabled`). Event data is never changed.

27.2 Enabling the module

Like Actions and the Event Receiver, the MCP server is opt-in. In the admin interface open the **MCP Server** tab, fill in the OAuth settings described in section 27.5, press **Test Authentik**, and only then tick **Enable the MCP server**. Enabling and disabling take effect immediately and need no restart; while the module is off the port stays bound but every request is answered with `503`.

The server listens on the admin port **+40**: container `8130` for HTTP and `8493` for HTTPS, published on the host with the usual `+10000` offset as `18130` and `18493`. Which of the two is live follows the deployment's TLS mode and certificate, exactly as for the other surfaces (chapter 2).

> **Use HTTPS** OAuth clients require it for anything that is not a loopback address, so in practice the MCP server is only reachable by a hosted client when TLS is on. Turn it on in the **TLS / SSL** tab; a TLS change takes effect after ↻ **Restart app server** in App Control.

27.3 The identity chain

Every call carries an `Authorization: Bearer` header holding an access token. The server resolves that token to a Process Mining user in five steps, and a failure at any one of them is reported distinctly:

1. **Signature.** The token must be an RS256 JWT signed by a key published in your Authentik's JWKS document. Verification is offline — the keys are fetched once and cached — so no call is made to Authentik per request.
2. **Issuer.** The token's `iss` must match the configured issuer. Authentik emits it with a trailing slash; both spellings are accepted, so you may configure either.

3. **Audience.** Checked only if you have filled in the **Audience** field, and **recommended**: Authentik does not set `aud` to the client id unless you add a property mapping, but once you do, filling in the field ties each token to *this* application so a token minted for a different app on the same Authentik cannot be replayed here. Left blank, the check is skipped and any validly-signed token from the issuer is accepted (still gated by the issuer, signature, user-mapping and group checks) — prefer setting it when the same Authentik serves more than one application.
4. **Group.** If a **Required group** is configured, the token's `groups` claim must contain it. This is a second gate in addition to whatever binding you placed on the Authentik application.
5. **User.** The **username claim** (default `preferred_username`) is matched, case-insensitively, against the Process Mining user list. The user must already exist and be enabled.

Once mapped, that user's **assigned database connections** decide what the caller may see — the same boundary the application enforces for the same person signing in through the browser. A connection not assigned to them is not listed by `list_connections` and is refused if named directly.

> **Note** The MCP user is a Process Mining user like any other, created in the admin interface's **Users** tab and given connections in **Database Connections**. Authentik decides *who may connect*; the demonstrator decides *what they may see*. Neither substitutes for the other.

27.4 Configuring Authentik

On the Authentik side you need one provider and one application.

1. Create an **OAuth2/OpenID Provider**. Give it your standard authorization flow, and choose client type **Public** for interactive clients that use PKCE, or **Confidential** if the client stores a secret.
2. Give the provider an **RSA signing key**. This is what makes the access token a verifiable RS256 JWT; without it the MCP server cannot check the signature offline.
3. Add the client's **redirect URIs**. A hosted client has a fixed callback — for Claude it is `https://claude.ai/api/mcp/auth_callback`. A client that runs on the user's own machine binds a random loopback port instead, so it needs a port-agnostic pattern in regular-expression matching mode rather than a fixed URI.
4. Include the scopes `openid`, `profile` and `email`, plus `offline_access` if you want the client to be able to refresh its token.
5. Create an **Application** bound to that provider and give it a slug. The slug determines the issuer URL, so note it: `https://<authentik>/application/o/<slug>/`.
6. Restrict who may use the application with a policy, group or user binding.

> **Dynamic Client Registration** Some clients register themselves rather than being handed a client id. If yours does, enable DCR on the provider — the client then supplies its own redirect URI at registration time and the fixed list above is not consulted for it. If DCR is off and the client expects it, sign-in fails before any redirect URI is ever compared, which makes it look like a redirect problem when it is not.

27.5 Configuring the MCP server

In the admin interface's **MCP Server** tab:

Issuer URL

The application's issuer, `https://<authentik>/application/o/<slug>/`. It must be the URL the *client* can reach, not an internal one — discovery is performed by the client, from wherever it runs.

JWKS URL

Leave blank to auto-discover from the issuer, or paste the `…/jwks/` endpoint explicitly.

Audience / Client ID

Recommended. Set it to the client id once you have mapped an `aud` claim onto the access token, so tokens issued for other apps on the same Authentik cannot be replayed here. Blank disables the check.

Required group

Optional. Matched against the token's `groups` claim.

Username claim

Which claim carries the username. Default `preferred_username`; use `email` if your Process Mining usernames are email addresses.

Test Authentik fetches the discovery document and the JWKS and reports how many signing keys it found. Use it before enabling the module, and again after any change — it catches an unreachable host or a wrong slug immediately.

> **The test does not exercise token validation** It confirms that Authentik is reachable and publishing keys, which is necessary but not sufficient. A configuration that passes the test can still reject every token, so verify with a real client before declaring success.

27.6 The tools

The tools fall into six groups — discover, analyse, cases, notes, notifications and power analysis — and three access levels:

Everyone

Every user who can sign in to the MCP server may call the discovery, analysis, case and note-reading tools, always within their assigned connections.

Notes — the only writes, and only your own

`create_note` and `update_note` apply the application's own note rules. The author is always the signed-in user; the note id and its times are set by the server; a thread is append-only (a comment is prepended, nothing is rewritten). Unlike the application — where a person may collaborate on a shared note by commenting, resolving or retitling it — `update_note` edits **only notes the caller authored**: a shared note someone else wrote is readable through `get_notes` but this surface will not change it on the caller's behalf. Another user's private note is answered exactly like a missing one, so ids cannot be probed. Every field is validated — step names must exist in the project, text is limited to 4000 characters and a title to 200, control and bidi-override characters are stripped, and no text can imitate a thread entry's header to pose as another author — a full thread (100,000 characters) takes no more comments, the permission rule is checked again inside the database write itself, each user may make at most

`PMW_MCP_NOTE_WRITES_PER_MIN` (default 20) note changes a minute and own at most `PMW_MCP_MAX_NOTES_PER_PROJECT` (default 500) notes in one project, and every write leaves an entry in the admin log naming the user and the note, never its text.

Power users and administrators

`compare_segments`, `get_bottlenecks`, `get_trend`, `get_outcome_drivers` and `check_conformance` are reserved for users with the power-user role (granted with **Make power** in the Users tab) and for administrators. Anyone else receives a polite refusal that names the role and points to the everyday tools; no query is run.

> **Times** Note timestamps — created, edited and each comment's stamp — are local time in the display timezone chosen in the admin interface (App Control → **Timezone**), and the MCP server returns them with that zone's UTC offset, for example `2026-09-23T12:34:45+02:00`. Journey event times are the event log's own recorded times.

Each tool is listed below with one worked example: the question a user might ask, the call an AI client makes, and an excerpt of the answer (values are illustrative).

Tool	Who	Returns
`list_connections`	Everyone	The database connections you may query.
`list_projects`	Everyone	Projects on one connection — or, with connectionId omitted, on every connection you may query; includeCounts adds a journey count per project.
`get_metadata`	Everyone	Meta-attribute titles, step names, each step's definition (stepDetails: description, score, belongsTo group, endOfProcess, shape) and the event date range.
`get_attribute_values`	Everyone	The values each meta attribute takes, with journey counts, most frequent first — so you know what to pass as meta1/meta2/meta3.
`get_process_map`	Everyone	The directly-follows map: steps (nodes) and transitions (edges) with counts and timing.
`get_transition_metrics`	Everyone	Per step pair: count and average/median/min/max/stddev transition time (seconds).
`get_variants`	Everyone	Distinct journey paths and how often each occurs, most frequent first.
`get_statistics`	Everyone	Journey count, journey-duration statistics and the process-goodness score.
`get_journey`	Everyone	One case's ordered events, by business case id or stored hash, with its meta values, start/end and total duration.
`find_journey`	Everyone	Which project(s) hold a case id — searches every project on every connection you may query (or one connectionId), ready for get_journey.
`find_journeys`	Everyone	The individual cases behind an aggregate — slowest, longest or by step — with optional full path. Returned eventIds feed get_journey.
`get_notes`	Everyone	The notes on a project's steps and transitions — filter by severity, status and scope, optionally grouped; with a summary of counts. Times are local (Admin Console display zone) with the UTC offset.
`create_note`	Everyone — write	Create a note on a step (step) or a transition (fromStep + toStep). You become the author; id and time are set by the server. text required (≤ 4000), title ≤ 200, severity default NORMAL, scope default personal. Rate-limited per user, and capped at a maximum number of notes you may own per project.
`update_note`	Everyone — write	On a note YOU authored: add a comment (prepended to the thread, with an optional title), set status open/resolved, or change severity/scope. This tool edits only your own notes — you can read others' shared notes with get_notes and reply in the app, but it won't edit another user's note on your behalf. Existing text is never rewritten; a full thread (100,000 characters) takes no more comments. Rate-limited per user.
`list_notifications`	Everyone	The dashboard notification rules on a connection: each rule's target (fromStep → toStep), metric, comparator and threshold, severity, interval and enabled flag, plus its current state (triggered, last value, when).

		Ordered most-urgent, most-recently triggered first; optional projectId and triggeredOnly filters. Requires the feature to be enabled.
`set_notification_enabled`	Power / dev / admin — write	Enable or disable one rule by id (from list_notifications). A disabled rule is not evaluated and shows no alert; rules are shared per connection, so this affects everyone assigned. The id shape is validated, a missing rule and one on an unassigned connection return the same “not found”, and writes are rate-limited per user.
`compare_segments`	Power users & admins	Two slices of one project side by side: each segment's count, durations, goodness and end steps, then the biggest differences (B minus A) in end-step shares, transition times, transition frequency, and transitions found in only one segment.
`get_bottlenecks`	Power users & admins	Where time is lost: transitions by total waiting time (occurrences × average), the slowest typical transitions (median), rework (steps repeated in a journey) and self-loops.
`get_trend`	Power users & admins	The process over time: per day, week or month (by journey start) the journey count, average and median duration, and — with outcomeSteps — the reach rate of those steps.
`get_outcome_drivers`	Power users & admins	Why an outcome happens: the overall rate of reaching outcomeSteps, and the meta values and visited steps that raise or lower it (rate with / without, delta in percentage points, lift). Association, not cause.
`check_conformance`	Power users & admins	Test journeys against up to 10 rules — requires {step, ifStep?}, forbidden {step}, precedes {before, after}, max_duration {maxSecs}, max_gap {fromStep, toStep, maxSecs} — and count violations with example case ids.

The common filter: sampleSet (ORIGINAL | SAMPLE_1..3), fromDate / toDate (ISO dates), includedSteps (journeys visiting all of them), excludedSteps (journeys visiting none of them) — up to 200 step names each — and meta1 / meta2 / meta3 (up to 256 characters; case-insensitive substring match — see get_attribute_values). The per-journey power tools (bottleneck rework, trend, outcome drivers, conformance, end steps) keep whole journeys that are active in the date window rather than cutting them at its edges.

Discover

What exists: connections, projects, steps and attribute values.

`list_connections` · **Everyone**

The database connections you may query. *Parameters:* `none`

“Which Process Mining connections can I use?”

```
// call
{
  "name": "list_connections",
  "arguments": {}
}
// result (excerpt; illustrative values)
[{"id": "7e31de50-…", "name": "02 – Air Travel", "schema": "PM_AIR", "comment": ""},
 {"id": "38891b38-…", "name": "03 – Finance", "schema": "PM_FIN", "comment": ""}]
```

`list_projects` · **Everyone**

Projects on one connection — or, with connectionId omitted, on every connection you may query; includeCounts adds a journey count per project. *Parameters:* `connectionId?, includeCounts?, sampleSet?`

“Which process has the most journeys?”

```
// call
{
  "name": "list_projects",
  "arguments": {
    "includeCounts": true
  }
}
// result (excerpt; illustrative values)
[{"connectionName": "02 – Air Travel", "projectId": 2,
  "title": "Airport Passenger Flow", "journeyCount": 148187},
 {"connectionName": "03 – Finance", "projectId": 1,
  "title": "Online Credit Application", "journeyCount": 20000}, …]
```

`get_metadata` · **Everyone**

Meta-attribute titles, step names, each step's definition (stepDetails: description, score, belongsTo group, endOfProcess, shape) and the event date range. *Parameters:* `connectionId, projectId`

“Show me all steps of the flight booking process.”

```
// call
{
  "name": "get_metadata",
  "arguments": {
    "connectionId": "7e31de50-…",
    "projectId": 5
  }
}
// result (excerpt; illustrative values)
{"metaTitles": {"meta1": "Journey Type", "meta2": "Airline", "meta3": "Payment Method"},
 "steps": ["Add Services", "Confirm Booking", …],
 "stepDetails": [{"step": "Confirm Booking", "description": "Booking confirmed and ticket is-
sued",
                  "score": 15, "belongsTo": "Payment", "endOfProcess": true, "shape": "stadi-
um"}, …],
 "dateRange": {"from": "2024-01-01T00:02:11", "to": "2024-12-31T23:51:40"}}
```

`get_attribute_values` · Everyone

The values each meta attribute takes, with journey counts, most frequent first — so you know what to pass as meta1/meta2/meta3. *Parameters:* `connectionId, projectId, meta? (meta1|meta2|meta3|all), limit? (50), filter`

“Which payment methods occur in the bookstore?”

```
// call
{
  "name": "get_attribute_values",
  "arguments": {
    "connectionId": "64254f7e-…",
    "projectId": 1,
    "meta": "meta1"
  }
}
// result (excerpt; illustrative values)
{"attributes": [{"meta": "meta1", "title": "Payment Method", "distinctValues": 3,
   "values": [{"value": "Credit Card", "journeys": 9120},
              {"value": "PayPal", "journeys": 6874},
              {"value": "Bank Transfer", "journeys": 4006}], "truncated": false}],
 "filterHint": "… case-insensitive and also matches substrings."}
```

Analyse

Aggregates over many journeys; every one takes the common filter.

`get_process_map` · **Everyone**

The directly-follows map: steps (nodes) and transitions (edges) with counts and timing. *Parameters:* `connectionId, projectId, filter`

“Draw the process map of the airport passenger flow.”

```
// call
{
  "name": "get_process_map",
  "arguments": {
    "connectionId": "7e31de50-…",
    "projectId": 2
  }
}
// result (excerpt; illustrative values)
{"steps": {"ENTER Check-In": {"description": "Passenger checks in at the desk", "score": 0,
…}, …},
 "transitions": [{"fromStep": "ENTER Check-In", "toStep": "LEAVE Check-In",
                  "occurrences": 44295, "avgSecs": 507.8, "medianSecs": 480.0, …}, …]}
```

`get_transition_metrics` · **Everyone**

Per step pair: count and average/median/min/max/stddev transition time (seconds). *Parameters:* `connectionId, projectId, filter`

“How long does security take for passengers in 2024?”

```
// call
{
  "name": "get_transition_metrics",
  "arguments": {
    "connectionId": "7e31de50-…",
    "projectId": 2,
    "fromDate": "2024-01-01",
    "toDate": "2024-12-31",
    "includedSteps": [
      "ENTER Security Check"
    ]
  }
}
// result (excerpt; illustrative values)
[{"fromStep": "ENTER Security Check", "toStep": "LEAVE Security Check",
  "occurrences": 145198, "avgSecs": 1020.0, "medianSecs": 1020.0,
  "minSecs": 240.0, "maxSecs": 1800.0, "stdDevSecs": 466.7}, …]
```

`get_variants` · **Everyone**

Distinct journey paths and how often each occurs, most frequent first. *Parameters:* `connectionId, projectId, filter, limit?`

“What are the three most common credit-application paths?”

```
// call
{
  "name": "get_variants",
  "arguments": {
    "connectionId": "38891b38-…",
    "projectId": 1,
    "limit": 3
  }
}
// result (excerpt; illustrative values)
[{"path": "Bank -> Application Received -> Application Checked -> Credit Check -> Accepted -> …",
  "journeyCount": 4210, "stepCount": 7, "totalScore": 25}, …]
```

`get_statistics` · **Everyone**

Journey count, journey-duration statistics and the process-goodness score. *Parameters:* `connectionId, projectId, filter`

“How long do affiliate credit applications take?”

```
// call
{
  "name": "get_statistics",
  "arguments": {
    "connectionId": "38891b38-…",
    "projectId": 1,
    "meta3": "Affiliate"
  }
}
// result (excerpt; illustrative values)
{"journeyCount": 9870,
 "durations": {"minSecs": 1804.0, "avgSecs": 201544.0, "medianSecs": 172800.0, …},
 "processGoodness": 3.41}
```

Cases

Individual journeys — find them, then open one.

`get_journey` · **Everyone**

One case's ordered events, by business case id or stored hash, with its meta values, start/end and total duration. *Parameters:* `connectionId, projectId, eventId, sampleSet?`

“Show me credit application CRA-000123.”

```
// call
{
  "name": "get_journey",
  "arguments": {
    "connectionId": "38891b38-…",
    "projectId": 1,
    "eventId": "CRA-000123"
  }
}
// result (excerpt; illustrative values)
{"eventId": "CRA-000123", "storedEventId": "5e05bf5d94a6fb2c78e7722c3ec4b07b",
 "startDate": "2024-06-12T09:59:28", "endDate": "2024-06-12T11:01:11", "durationSecs":
3703.0,
 "meta": {"Applied Credit Sum": "> 25.000 EUR", "Income Class": "Low", "Channel":
"Affiliate"},
 "events": [{"step": "Affiliate", "eventTime": "2024-06-12T09:59:28"}, …,
            {"step": "Rejected", "eventTime": "2024-06-12T11:01:11"}]}
```

`find_journey` · **Everyone**

Which project(s) hold a case id — searches every project on every connection you may query (or one connectionId), ready for get_journey. *Parameters:* `eventId, connectionId?, sampleSet?`

“Where is case FLT-000124?”

```
// call
{
  "name": "find_journey",
  "arguments": {
    "eventId": "FLT-000124"
  }
}
// result (excerpt; illustrative values)
{"eventId": "FLT-000124", "storedEventId": "69ab1ef5…",
 "matches": [{"connectionName": "02 - Air Travel", "projectId": 5,
              "title": "Flight Booking & Management", "startDate": "2024-10-30T23:22:09",
              "durationSecs": 1380.0, "stepCount": 14}],
 "searched": {"connections": 5, "projects": 8}}
```

`find_journeys` · **Everyone**

The individual cases behind an aggregate — slowest, longest or by step — with optional full path. Returned eventIds feed get_journey. *Parameters:* `connectionId, projectId, filter, orderBy?, limit? (20), min/maxDurationSecs?, min/maxSteps?, includePath?`

“The five slowest orders that hit Payment Failed, with their paths.”

```
// call
{
  "name": "find_journeys",
  "arguments": {
    "connectionId": "64254f7e-…",
    "projectId": 1,
    "limit": 5,
    "includedSteps": [
      "Payment Failed"
    ],
    "includePath": true
  }
}
// result (excerpt; illustrative values)
[{"eventId": "8c46773b…", "startDate": "2024-04-21T04:36:09", "durationSecs": 1134.0,
  "stepCount": 17, "meta": {"Payment Method": "Bank Transfer", …},
  "path": "Login -> Browse Catalog -> … -> Payment Failed -> Payment Retry -> …"}, …]
```

Notes

The human layer on the process. The two note-writing tools, together with `set_notification_enabled`, are the only writes on the MCP surface.

`get_notes` · **Everyone**

The notes on a project's steps and transitions — filter by severity, status and scope, optionally grouped; with a summary of counts. Times are local (Admin Console display zone) with the UTC offset. *Parameters:* `connectionId, projectId, severity?, status?, scope?, groupBy?`

“Show all closed notes, then all open notes, as separate groups.”

```
// call
{
  "name": "get_notes",
  "arguments": {
    "connectionId": "7e31de50-…",
    "projectId": 2,
    "groupBy": "status"
  }
}
// result (excerpt; illustrative values)
{"total": 1, "summary": {"byStatus": {"open": 0, "resolved": 1}, …},
 "groupBy": "status",
 "groups": [{"key": "resolved", "count": 1, "notes": [{"id": "E422E185-…",
   "title": "A Note for testing", "severity": "URGENT", "status": "resolved",
   "scope": "shared", "author": "dirk.beerbohm", "target": "ENTER Check-In",
   "createdAt": "2026-09-23T12:34:45+02:00", "editedAt": "2026-09-23T12:36:42+02:00"}]}]}
```

`create_note` · Everyone — write

Create a note on a step (step) or a transition (fromStep + toStep). You become the author; id and time are set by the server. text required (≤ 4000), title ≤ 200, severity default NORMAL, scope default personal. Rate-limited per user, and capped at a maximum number of notes you may own per project. *Parameters:* `connectionId, projectId, step | fromStep + toStep, text, title?, severity?, scope?`

“Add a shared, important note on the security transition.”

```
// call
{
  "name": "create_note",
  "arguments": {
    "connectionId": "7e31de50-…",
    "projectId": 2,
    "fromStep": "ENTER Security Check",
    "toStep": "LEAVE Security Check",
    "title": "Security wait",
    "text": "Median 17 min; peaks before 07:00.",
    "severity": "IMPORTANT",
    "scope": "shared"
  }
}
// result (excerpt; illustrative values)
{"created": true,
 "note": {"id": "9B1C…", "title": "Security wait", "severity": "IMPORTANT",
          "status": "open", "scope": "shared", "author": "dirk.beerbohm",
          "target": "ENTER Security Check → LEAVE Security Check", "targetType": "edge",
          "createdAt": "2026-09-24T08:15:02+02:00", "editedAt": null}}
```

`update_note` · **Everyone — write**

On a note YOU authored: add a comment (prepended to the thread, with an optional title), set status open/resolved, or change severity/scope. This tool edits only your own notes — you can read others' shared notes with get_notes and reply in the app, but it won't edit another user's note on your behalf. Existing text is never rewritten; a full thread (100,000 characters) takes no more comments. Rate-limited per user. *Parameters:* `connectionId, projectId, noteId, comment?, title?, status?, severity?, scope?`

“Resolve the test note and say why.”

```
// call
{
  "name": "update_note",
  "arguments": {
    "connectionId": "7e31de50-…",
    "projectId": 2,
    "noteId": "E422E185-0557-4E22-8DB2-BC62B30DA426",
    "comment": "Test complete.",
    "status": "resolved"
  }
}
// result (excerpt; illustrative values)
{"updated": true, "changes": ["comment", "status"],
 "note": {"id": "E422E185-…", "status": "resolved", "lastEditedBy": "dirk.beerbohm",
          "editedAt": "2026-09-24T08:16:40+02:00", …}}
```

Power analysis

Deeper analysis, reserved for users with the power-user role (and administrators). Anyone else gets a polite refusal naming the role.

`compare_segments` · **Power users & admins**

Two slices of one project side by side: each segment's count, durations, goodness and end steps, then the biggest differences (B minus A) in end-step shares, transition times, transition frequency, and transitions found in only one segment. *Parameters:* `connectionId, projectId, segmentA, segmentB (filter objects + label), limit? (10), minOccurrences? (30), sampleSet?`

“How do Bank Transfer orders differ from PayPal orders?”

```
// call
{
  "name": "compare_segments",
  "arguments": {
    "connectionId": "64254f7e-…",
    "projectId": 1,
    "segmentA": {
      "meta1": "Bank Transfer",
      "label": "Bank Transfer"
    },
    "segmentB": {
      "meta1": "PayPal",
      "label": "PayPal"
    }
  }
}
// result (excerpt; illustrative values)
{"segments": [{"label": "Bank Transfer", "journeyCount": 4006, …},
              {"label": "PayPal", "journeyCount": 6874, …}],
 "differences": {
   "endSteps": [{"step": "Payment Failed", "shareA": 0.12, "shareB": 0.03, "deltaPoints":
-9.0}, …],
   "transitionTime": [{"fromStep": "Payment Processing", "toStep": "Payment Confirmed",
                       "avgSecsA": 5400.0, "avgSecsB": 45.0, "deltaSecs": -5355.0}, …],
   "transitionFrequency": […], "onlyInA": […], "onlyInB": […]}}
```

`get_bottlenecks` · **Power users & admins**

Where time is lost: transitions by total waiting time (occurrences × average), the slowest typical transitions (median), rework (steps repeated in a journey) and self-loops. *Parameters:* `connectionId, projectId, filter, limit? (10), minOccurrences? (30)`

“Where do credit applications lose the most time?”

```
// call
{
  "name": "get_bottlenecks",
  "arguments": {
    "connectionId": "38891b38-…",
    "projectId": 1
  }
}
// result (excerpt; illustrative values)
{"totalWaitSecs": 3.1e9,
 "byTotalWaitTime": [{"fromStep": "Credit Check", "toStep": "Senior Agent Approval",
   "occurrences": 6120, "avgSecs": 87942.0, "totalWaitSecs": 538205040.0,
   "shareOfWaitTime": 0.17}, …],
 "slowestTypicalTransitions": […],
 "rework": [{"step": "Application Checked", "journeys": 5230, "extraVisits": 5890}, …],
 "selfLoops": […]}
```

`get_trend` · **Power users & admins**

The process over time: per day, week or month (by journey start) the journey count, average and median duration, and — with outcomeSteps — the reach rate of those steps. *Parameters:* `connectionId, projectId, filter, granularity? (day|week|month), outcomeSteps?, limit?`

“Is the denied-boarding rate changing month by month?”

```
// call
{
  "name": "get_trend",
  "arguments": {
    "connectionId": "7e31de50-…",
    "projectId": 2,
    "granularity": "month",
    "outcomeSteps": [
      "DENIED Boarding Dom",
      "DENIED Boarding Int"
    ]
  }
}
// result (excerpt; illustrative values)
{"granularity": "month",
 "periods": [{"period": "2024-01-01", "journeys": 12530, "avgDurationSecs": 4210.0,
              "medianDurationSecs": 4080.0, "outcomeJourneys": 598, "outcomeRate": 0.0477},
…]}
```

`get_outcome_drivers` · **Power users & admins**

Why an outcome happens: the overall rate of reaching outcomeSteps, and the meta values and visited steps that raise or lower it (rate with / without, delta in percentage points, lift). Association, not cause. *Parameters:* `connectionId, projectId, filter, outcomeSteps, minSupport? (30), limit? (10)`

“What makes a credit application end in Rejected?”

```
// call
{
  "name": "get_outcome_drivers",
  "arguments": {
    "connectionId": "38891b38-…",
    "projectId": 1,
    "outcomeSteps": [
      "Rejected"
    ]
  }
}
// result (excerpt; illustrative values)
{"journeys": 20000, "outcomeJourneys": 5400, "outcomeRate": 0.27,
 "raisesOutcome": {"attributes": [{"title": "Income Class", "value": "Low",
     "outcomeRate": 0.61, "rateWithout": 0.16, "deltaPoints": 34.0, "lift": 2.26}, …],
   "steps": […]},
 "lowersOutcome": {"attributes": […], "steps": […]}}
```

`check_conformance` · **Power users & admins**

Test journeys against up to 10 rules — requires {step, ifStep?}, forbidden {step}, precedes {before, after}, max_duration {maxSecs}, max_gap {fromStep, toStep, maxSecs} — and count violations with example case ids. *Parameters:* `connectionId, projectId, filter, rules, examples? (5)`

“Is payment always processed before a booking is confirmed, within 5 minutes?”

```
// call
{
  "name": "check_conformance",
  "arguments": {
    "connectionId": "7e31de50-…",
    "projectId": 5,
    "rules": [
      {
        "type": "precedes",
        "before": "Process Payment",
        "after": "Confirm Booking"
      },
      {
        "type": "max_gap",
        "fromStep": "Process Payment",
        "toStep": "Confirm Booking",
        "maxSecs": 300
      }
    ]
  }
}
// result (excerpt; illustrative values)
{"journeysChecked": 20000,
 "rules": [{"type": "precedes", "rule": "'Process Payment' must happen before 'Confirm
Booking'",
            "violations": 0, "violationRate": 0.0, "conformanceRate": 1.0, "exampleEventIds":
[]},
           {"type": "max_gap", "violations": 0, …}]}
```

`connectionId` is the connection's string identifier as `list_connections` returns it; `projectId` is an integer as `list_projects` returns it. Mixing the two up is the most common first mistake.

`get_journey` is the odd one out: it takes no filter, and it returns individual events rather than an aggregate. It exists because every other tool summarises many journeys, so a question about *one* case — what actually happened to booking `FLT-000123` — has no answer in a summary.

`find_journeys` is what gets you there when you do *not* already know the case id. It applies the ordinary filter, then returns the matching cases themselves rather than a summary of them, ordered by `orderBy`: `DURATION_DESC` (the default, slowest first), `DURATION_ASC`, `START_DESC`, `START_ASC`, `STEPS_DESC` or `STEPS_ASC`. `limit` defaults to 20; `minDurationSecs`, `maxDurationSecs`, `minSteps` and `maxSteps` bound the whole journey rather than an individual event; and `includePath` adds each journey's full step path, which costs an extra aggregation and is therefore off by default. The usual shape of an investigation is three calls: `get_statistics` for the average, `find_journeys` for the outliers behind it, then `get_journey` for the one case that explains them.

`list_projects` spans connections for the same reason. Asked which process is the largest, a client that must call it once per connection and then `get_statistics` once per project spends a dozen round trips on

a one-line answer; omitting `connectionId` and setting `includeCounts` answers it in one. Each entry names the connection it came from, and a connection that cannot be opened is reported as a single entry carrying `error` — one unreachable database degrades that row instead of failing the listing.

`get_notes` reaches the human layer on top of the data: the notes people attach to a project's steps and transitions. Each note carries a **severity** (`NORMAL`, `INFO`, `IMPORTANT` or `URGENT`), a **status** (open or resolved) and a **scope** — a private *personal* note or a *shared* one visible to the whole team. Any of these narrows the result (`severity` takes a list; `status` and `scope` take one value), and `groupBy` buckets the output by `severity`, `status`, `scope` or `target` (the annotated step or transition). Every response also carries a `summary` with per-severity, per-status and per-scope counts, so "how many open urgent notes are on this process?" is answered without reading each one. Visibility is the application's own rule enforced server-side: the caller sees shared notes and their own, and never another user's private notes.

> **Note** `JOURNEYS` never stores a case id in the clear: ingest writes its **MD5 digest**, whether the events arrived from a file import or from a sink. `get_journey` therefore applies the application's own normalisation — a business id such as `FLT-000123` is hashed before the lookup, while a 32-character hex string is taken to be an already-stored id. Both reach the same journey the individual-journey view shows, and neither requires the caller to know that any hashing happens. `find_journeys` reports the stored digest as its `eventId`, since the plain-text id is nowhere to report; feeding that value straight back to `get_journey` is the intended drill-down.

27.7 The filter vocabulary

Most tools accept the same optional filter, mirroring the application's own (`compare_segments` takes one filter per segment):

`fromDate`, `toDate`
ISO dates bounding the events. Day granularity — there is no time-of-day filtering.

`includedSteps`
Keep only journeys that visit **all** of the named steps.

`excludedSteps`
Drop journeys that visit **any** of the named steps.

`meta1`, `meta2`, `meta3`
Match on the three meta attributes, whatever `get_metadata` reports them to be titled for that project. The match is case-insensitive and also matches part of a value; `get_attribute_values` lists the values that occur.

`sampleSet`
`ORIGINAL` (the default), or `SAMPLE_1`, `SAMPLE_2`, `SAMPLE_3`.

> **Splitting a population** Because `includedSteps` is an AND and `excludedSteps` is a NOT-ANY, a two-way split is expressed as one call each way on the step that distinguishes them. In the shipped airport dataset, excluding `ENTER Passport Control` gives the domestic journeys and including it gives the international ones; the two counts add up to the unfiltered total. An either/or question — "denied boarding, domestic *or* international" — needs one call per alternative, because no filter expresses OR.

27.8 Behind a reverse proxy

Exposing the MCP server under a public domain is the normal deployment, because a hosted AI client has to reach it from the internet. Three things must be right, and two of them are easy to get wrong.

1. **Preserve the path.** Forward `/mcp` to the backend as `/mcp`. In nginx a `proxy_pass` with a trailing slash strips the prefix and the backend answers `404`.
2. **Route the discovery paths too.** When an unauthenticated call arrives, the server answers `401` with a `WWW-Authenticate` header pointing at `/.well-known/oauth-protected-resource` — at the **root of the host**, not under `/mcp`. A proxy that routes only `/mcp` sends that request somewhere else, the client cannot discover the authorization server, and it reports a sign-in failure that looks nothing like a routing problem.
3. **Advertise reachable URLs.** Send `X-Forwarded-Proto` and `X-Forwarded-Host` so the metadata carries the public address. The issuer it names must resolve **from the client's network**; an internal hostname or a Docker-internal name is unreachable for a hosted client and the connection will never complete.

27.9 Limits, error codes and troubleshooting

Response	Meaning and remedy
`503`	The module is disabled. Enable it in the MCP Server tab.
`401` + `WWW-Authenticate`	No token, or one that failed signature, issuer or audience checks. The client should run the OAuth flow. If it loops, check the issuer with **Test Authentik**.
`403` no matching user	The username claim does not match an enabled Process Mining user. Create the user with that exact name, or change the **Username claim**.
`403` not in the required group	The token's `groups` claim lacks the configured group. Note that a group must exist in Authentik *and* have the user as a member for it to appear in the token.
Connection is not assigned to you	Assign it to the user in the admin interface's **Database Connections** tab.
“Sorry — … reserved for power users”	A deeper-analysis tool was called by a user without the power-user role. Grant it with **Make power** in the Users tab if that user should have it.
Limit of *N* note changes per minute	A client is writing notes in a loop. Wait a minute; raise `PMW_MCP_NOTE_WRITES_PER_MIN` only if the volume is intended.
You already own *N* notes in project … (the limit is …)	The per-project note cap was reached. Delete some in the app, or raise `PMW_MCP_MAX_NOTES_PER_PROJECT` (0 disables it) if the volume is intended.
Only the note's author can change it from here	The note is shared and authored by someone else. The MCP server edits only your own notes; collaborate on the shared note (comment, resolve, retitle) in the Process Mining app instead.
Step ‘…’ does not exist (create_note)	Step names must match the project exactly; list them with `get_metadata`.
This note's thread is full	The thread reached 100,000 characters. Start a new note for the follow-up.
A batch may hold at most 20 messages	The client sent more than 20 JSON-RPC calls in one request; it must split them.

Four environment variables bound the server: `PMW_MCP_MAX_ROWS` (default 1000) caps the rows any one call returns, `PMW_MCP_JWKS_CACHE_SECS` (default 3600) sets how long Authentik's signing keys are cached, `PMW_MCP_NOTE_WRITES_PER_MIN` (default 20) limits each user's note writes per minute, and `PMW_MCP_MAX_NOTES_PER_PROJECT` (default 500, 0 disables) caps how many notes one user may own in a project.

> **Note** Only the *transport* used to fetch the JWKS skips certificate verification, on the assumption that Authentik is a trusted internal host. Token integrity does not depend on that transport: the signature is checked cryptographically against the keys it returns, so a tampered token fails regardless.

> **When a client connects but shows no tools** The transport is up and the token exchange succeeded, but every tool call is being refused — almost always at step 4 or 5 of the identity chain. Check the group and the username claim before touching anything else; the connection state in the client reflects only the OAuth handshake, not whether a single call has ever been answered.